Part VII

Identifiability and symmetries

Chapter 32

Symmetries in Bayesian neural networks

Symmetries are transformations that leave a system unchanged. These transformations may preserve the whole system or only certain properties of it. In many areas of science, especially mathematics and physics, symmetries are essential because they allow us to simplify complex problems and understand systems more easily.

One of the most famous results on symmetries in physics is **Noether's theorem** [1247]: every continuous symmetry of a physical system corresponds to a conserved quantity, as when time-invariance of the laws of physics yields conservation of energy. Symmetries also make hard equations tractable; for example, by assuming spherical symmetry around a massive object, Schwarzschild obtained the first exact solution to Einstein's field equations and, with it, the prediction of black holes [1478]. We invoke these only as motivation: the structure that matters in this chapter is the symmetry of the parametrisation, to which we turn now.

In recent years, machine learning—and especially deep learning—has also been shown to involve important symmetries. These appear in two main ways:

1. **Symmetries in the data space:** These are transformations of the input data (and possibly the labels) that should not change the model's output. For example, in image classification, moving a digit within an image (like in the MNIST dataset) should not change the predicted label. This kind of symmetry is the focus of *geometric deep learning* [210].

2. **Symmetries in the parameter space:** These occur in the way neural networks are built. Deep models use layers of weights (matrices) and activation functions like ReLU or tanh. In many cases, different arrangements or values of weights can produce exactly the same output function. These transformations in parameter space do not affect the final result of the model but can cause redundancy during training or evaluation [493].

In the context of *Bayesian deep learning* [174, 1105], handling symmetries is crucial. It is important to be precise about what is at stake: in *exact* Bayesian model averaging the predictive distribution lives in function space, and it is not harmed by a function having several parameter representatives, since these representatives all contribute the same predictions. The practical difficulties are instead inherited by the tools we use to *approximate* the posterior. Ignoring symmetries can make weight-space posterior approximations and convergence diagnostics misleading, and can make sampling or variational inference inefficient, because the same solution is revisited many times in different but equivalent forms. Accounting for symmetries therefore makes weight-space inference more efficient and easier to interpret.

The Bayesian message. A single thread runs through this chapter. A group G of function-preserving transformations acts on the parameter space; its orbits are the equivalence classes of parameters that implement the same function, and along each orbit both the network map and the likelihood are constant. Whether the *posterior* is also constant along an orbit is a separate question: it requires that the prior *measure* (prior density together with the base measure) be invariant under the transformation as well. Permutation symmetries are typically posterior symmetries under exchangeable priors, whereas continuous scaling symmetries usually preserve the likelihood but not a standard Gaussian prior (L2 regularisation). Keeping likelihood symmetries and posterior symmetries distinct organizes the discussion of sampling, variational inference, ensembles, and mode connectivity that follows.

Structure of the Chapter. This chapter is organized as follows: First, we introduce the general concepts of **groups, symmetries, and invariance**, and explain how they relate to machine learning. Next, we focus on **weight-space symmetries** in deep neural networks, showing how different parameter configurations can represent the same function. Then, we explore various approaches to **handle symmetries by choosing a representative** of each equivalence class, especially in Bayesian settings, including symmetry-aware optimisation and sampling. Finally, we discuss **mode connectivity** and its implications for understanding the structure of the loss landscape.

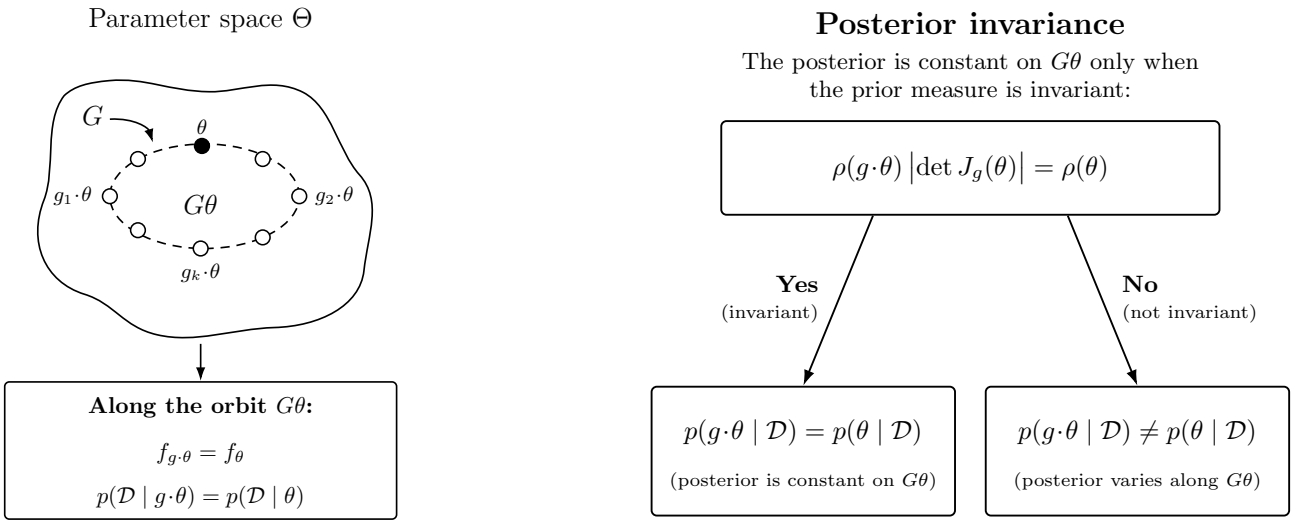


Figure 32.1: Likelihood and posterior invariance along a parameter-space orbit. The network function and likelihood are constant on $G \cdot \theta$, while posterior invariance additionally requires invariance of the prior measure.

We study the consequences of these symmetries for posterior sampling in chapter 33 and their identification in chapter 34.

32.1 Background: symmetries and invariance

32.1.1 Basic definitions

We start by defining the basic concepts that will be used throughout this chapter and beyond. We first recall the definitions of groups and subgroups.

Definition 32.1 (Group)**.** *A* group *is a non-empty set G together with a binary operation $\cdot : G \times G \to G$ (often written as $a \cdot b$) that satisfies the following axioms for all $a, b, c \in G$:*

- ***Associativity:*** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
- ***Identity element:*** *There exists an element $e \in G$ such that $e \cdot a = a \cdot e = a$ for all $a \in G$. This element e is called the* identity element*.*
- ***Inverse element:*** *For every $a \in G$, there exists an element $b \in G$ such that $a \cdot b = b \cdot a = e$, where e is the identity element. The element b is called the* inverse *of a and is often denoted a^{-1}.*

Definition 32.2 (Subgroup)**.** *Let G be a group. A subset $H \subseteq G$ is called a* subgroup *of G if H is itself a group under the group operation inherited from G.*

The subgroups G and $\{e\}$, where e is the identity element of G, are called the *improper subgroups* of G. All other subgroups are called *proper subgroups*.

Theorem 32.1. *A nonempty subset H of a group G is a subgroup if and only if the following two conditions hold:*

1. *If $h, k \in H$, then $h \cdot k \in H$.*
2. *If $h \in H$, then $h^{-1} \in H$.*

Proof. If H is a subgroup, then it must satisfy the group axioms, so conditions (1) and (2) clearly hold.

Conversely, suppose H satisfies conditions (1) and (2). We show that H is a group under the operation inherited from G.

Since G is a group, the operation is associative, and thus associativity also holds in H.

Because H is nonempty, there exists some $h \in H$. By condition (2), $h^{-1} \in H$. Then, by condition (1), $h \cdot h^{-1} = e \in H$, so the identity element e belongs to H.

Thus, H contains the identity, is closed under the group operation, and contains inverses of its elements. Therefore, H is a subgroup of G. □

32.1.2 Symmetric groups

More specifically, we now define the symmetric group, which will be needed in the next parts.

Definition 32.3 (Symmetry)**.** *A* symmetry *of a set X is a bijective transformation $f : X \to X$ such that f preserves the structure of X.*

The notion of "structure" depends on the context:

- In **geometry**, the structure refers to *shape and size.*
- In **algebra**, it refers to *operations* (e.g., a group operation).
- In **physics**, it refers to *invariance under a transformation* (e.g., time or space translation).

Definition 32.4 (Permutation in $\mathbb{R}^n$)**.** *Let $x = \big[x_1, x_2, \dots, x_n\big] \in \mathbb{R}^n$. A* permutation *of x is a vector $\sigma(x) = (x_{\sigma(1)}, x_{\sigma(2)}, \dots, x_{\sigma(n)})$, where σ is a bijection from the set $\{1, 2, \dots, n\}$, denoted as $[n]$, to itself.*

Each permutation in $\mathbb{R}^n$ σ can be represented by an $n \times n$ *permutation matrix* $\boldsymbol{\pi}_\sigma$, which is obtained by permuting the rows (or columns) of the identity matrix according to σ.

Example: Let σ be a permutation in $\mathbb{R}^3$ defined by

$$\sigma(1) = 2, \quad \sigma(2) = 3, \quad \sigma(3) = 1.$$

Then the corresponding permutation matrix $\boldsymbol{\pi}_\sigma$ is:

$$\boldsymbol{\pi}_\sigma = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}.$$

The action of σ on a column vector $x \in \mathbb{R}^3$ is given by matrix multiplication:

$$\sigma(x) = \boldsymbol{\pi}_\sigma x.$$

Definition 32.5 (Symmetric Group)**.** *Let $n \in \mathbb{N}$. The* symmetric group *on n elements, denoted by $\mathfrak{S}_n$, is the set of all bijections (or permutations) from the set $\{1, 2, \dots, n\}$ to itself. That is,*

$$\mathfrak{S}_n = \{\sigma \colon [n] \to [n] \mid \sigma \text{ is a bijection}\}.$$

The group operation on $\mathfrak{S}_n$ is the composition of functions: for $\sigma, \tau \in \mathfrak{S}_n$, the product $\sigma \circ \tau$ is the bijection defined by

$$(\sigma \circ \tau)(i) = \sigma\big(\tau(i)\big), \quad \text{for all } i \in [n].$$

Proposition 32.1. *The symmetric group $\mathfrak{S}_n$, under composition of functions, forms a group: the identity element is the identity function, and each permutation has an inverse given by its inverse function.*

Proof. To show that $\mathfrak{S}_n$ forms a group, we verify the group axioms:

- **Closure:** Let $\sigma, \tau \in \mathfrak{S}_n$. Then σ and τ are bijections from $[n]$ to itself. The composition $\sigma \circ \tau$ is also a bijection, and hence $\sigma \circ \tau \in \mathfrak{S}_n$.
- **Associativity:** Let $\sigma, \tau, \rho \in \mathfrak{S}_n$ be permutations. We must show that function composition is associative; that is,

  $$(\sigma \circ \tau) \circ \rho = \sigma \circ (\tau \circ \rho).$$

  Let $i \in \{1, 2, \dots, n\}$ be arbitrary. We compute both sides of the equation evaluated at i:

  $$\begin{aligned} \big((\sigma \circ \tau) \circ \rho\big)(i) &= (\sigma \circ \tau)(\rho(i)) \\ &= \sigma(\tau(\rho(i))), \end{aligned}$$

and

$$\begin{aligned}\big(\sigma \circ (\tau \circ \rho)\big)(i) &= \sigma\big((\tau \circ \rho)(i)\big) \\ &= \sigma(\tau(\rho(i))).\end{aligned}$$

Since both expressions evaluate to $\sigma(\tau(\rho(i)))$ for all i, we conclude that

$$(\sigma \circ \tau) \circ \rho = \sigma \circ (\tau \circ \rho),$$

as functions.

Therefore, function composition is associative in $\mathfrak{S}_n$, and the associativity axiom holds.

- **Identity element:** The identity function $\mathrm{id}\colon [n] \to [n]$, defined by $\mathrm{id}(i) = i$ for all i, is a bijection and thus belongs to $\mathfrak{S}_n$. For any $\sigma \in \mathfrak{S}_n$,
$$\sigma \circ \mathrm{id} = \mathrm{id} \circ \sigma = \sigma.$$
- **Inverses:** Since $\sigma \in \mathfrak{S}_n$ is a bijection, it has an inverse function $\sigma^{-1}\colon [n] \to [n]$ such that
$$\sigma \circ \sigma^{-1} = \sigma^{-1} \circ \sigma = \mathrm{id}.$$
Because σ^{-1} is also a bijection, we have $\sigma^{-1} \in \mathfrak{S}_n$.

Thus, $\mathfrak{S}_n$ satisfies all the group axioms and is therefore a group under composition of functions. □

32.1.3 Group actions, invariant functions, and equivariant functions

In this chapter, we will often refer to equivariant functions, we formalise and introduce them below.

Definition 32.6 (Group Action)**.** *Let G be a group and X a set. A* (left) group action *of G on X is a function*

$$\cdot\colon G \times X \to X, \quad (g, x) \mapsto g \cdot x,$$

satisfying the following two properties for all $g, h \in G$ and all $x \in X$:

1. ***Identity:*** *$e \cdot x = x$, where $e \in G$ is the identity element.*
2. ***Compatibility:*** *$(gh) \cdot x = g \cdot (h \cdot x)$.*

We then say that G acts on *X, and X is called a G-set.*

Definition 32.7 (Orbit)**.** *The orbit of an element $x \in X$ under the action of a group G, denoted $G \cdot x$, is the set of all elements in X to which x can be moved by the action of elements in G: $G \cdot x = \{g \cdot x \mid g \in G\}$.*

Definition 32.8 (Stabilizer)**.** *The* stabilizer *of an element $x \in X$ is the subgroup $G_x = \{g \in G \mid g \cdot x = x\}$ of transformations that leave x fixed.*

Definition 32.9 (Quotient set)**.** *The set of orbits $X/G = \{G \cdot x \mid x \in X\}$ is called the* quotient *of X by the action of G. Choosing one element from each orbit defines a set of* representatives*, and a map sending each x to its representative is a* canonicalisation*.*

Definition 32.10 (G-invariant Function)**.** *Let G be a group acting on a set X, and let $f\colon X \to Y$ be a function into some set Y. We say that f is* G-invariant *(or invariant under the action of G) if for all $g \in G$ and all $x \in X$, we have*

$$f(g \cdot x) = f(x).$$

In other words, f is constant on the orbits of the group action.

Definition 32.11 (G-Equivariant Function)**.** *Let G be a group acting on two sets X and Y via actions*

$$\cdot_X\colon G \times X \to X, \quad (g, x) \mapsto g \cdot_X x,$$

and

$$\cdot_Y\colon G \times Y \to Y, \quad (g, y) \mapsto g \cdot_Y y.$$

A function $f : X \to Y$ is said to be G-equivariant *if for every $g \in G$ and every $x \in X$,*

$$f(g \cdot_X x) = g \cdot_Y f(x).$$

That is, applying a group element before or after f yields the same result.

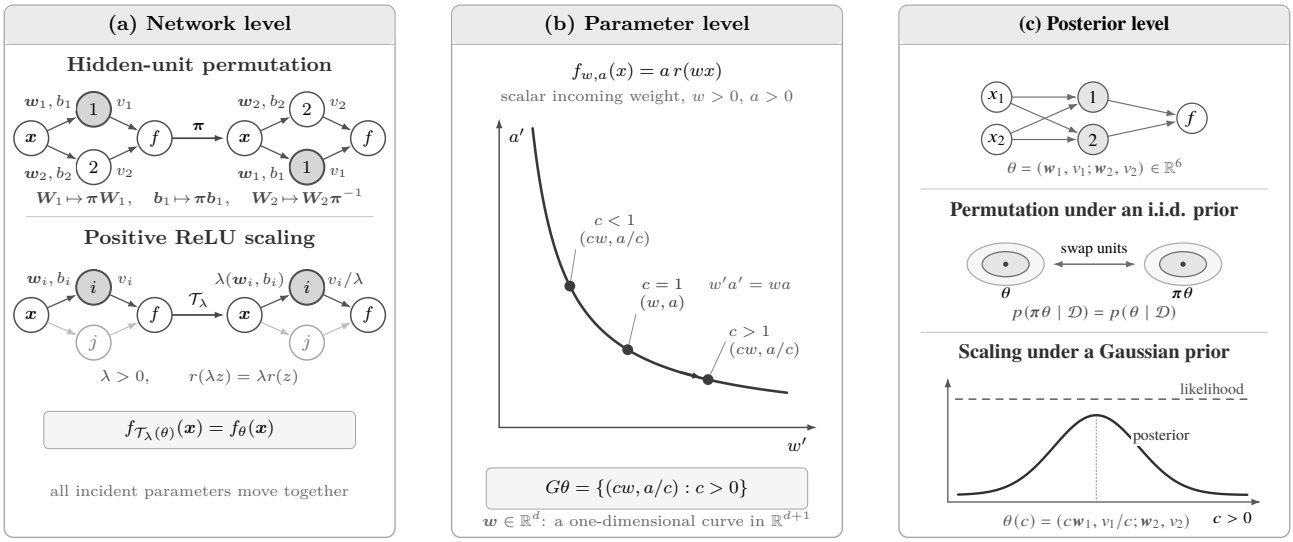


Figure 32.2: Three views of hidden-unit symmetries in a one-hidden-layer ReLU MLP. (a) A permutation reorders each hidden unit together with all its incident parameters. Positive scaling sends $(\boldsymbol{w}_i, b_i, v_i)$ to $(\lambda\boldsymbol{w}_i, \lambda b_i, v_i/\lambda)$ for $\lambda > 0$, leaving the implemented function unchanged. (b) For $f_{w,a}(x) = a\, r(wx)$ with $w, a > 0$, the equivalent parameters $(cw, a/c)$ form a branch of the hyperbola $w'a' = wa$. For vector-valued incoming weights, this remains a one-dimensional curve in a higher-dimensional parameter space. (c) In a 2–2–1 BNN without biases, $\theta = (\boldsymbol{w}_1, v_1; \boldsymbol{w}_2, v_2) \in \mathbb{R}^6$. An exchangeable prior yields identical posterior copies under hidden-unit permutation. Along the ReLU scaling orbit, however, the likelihood is constant while the posterior density generally varies under an independent Gaussian prior.

32.2 Going deep into weight-space symmetries in neural networks

There are two main mathematical frameworks for working on parameter-space symmetries. Most papers use the simpler but less general matrix formalism, describing symmetries as transformations of matrices, although some authors prefer the directed acyclic graph formalism. This chapter summarizes the existing literature, presenting the results and intuitions in the matrix formalism.

With the group-theoretic vocabulary of the previous section in place, we can state the unifying picture. Once we restrict attention to invertible transformations, the weight-space symmetries of a fixed architecture form a group G acting on the parameter space $\mathbb{R}^d$, and the map $\theta \mapsto f_\theta$ is G-invariant: it is constant on each orbit $G \cdot \theta$. The orbits are exactly the sets of parameters that implement the same function. Removing redundancy then amounts to choosing one representative per orbit, i.e. working on the quotient $\mathbb{R}^d/G$.

Definition 32.12 (Weight-space symmetries)**.** *Fix a network architecture, and let f_θ denote the function it implements with parameter vector $\theta \in \mathbb{R}^d$. A* weight-space symmetry *of this architecture is an invertible map $\mathcal{T} : \mathbb{R}^d \to \mathbb{R}^d$ that preserves the implemented function for* every *parameter vector, that is,*

$$\forall \theta \in \mathbb{R}^d,\ \forall x \in \mathcal{X} : \quad f_\theta(x) = f_{\mathcal{T}(\theta)}(x). \tag{32.1}$$

Equivalently, $f_{\mathcal{T}(\theta)} = f_\theta$ as functions. The set of all such maps forms a group under composition, the weight-space symmetry group *of the architecture.*

Notation. Throughout, $P_m \subset \mathbb{R}^{m\times m}$ denotes the set of $m\times m$ permutation matrices, i.e. the matrix representation of the symmetric group $\mathfrak{S}_m$; a permutation matrix $\boldsymbol{\pi}_\sigma$ acts on a vector by $\boldsymbol{\pi}_\sigma x = (x_{\sigma^{-1}(1)}, \ldots, x_{\sigma^{-1}(m)})^\top$ and is orthogonal, so $\boldsymbol{\pi}_\sigma^{-1} = \boldsymbol{\pi}_\sigma^\top$. We use the same notation P_h for $h \times h$ permutation matrices when permuting attention heads. We reserve σ for permutations and write $\hat{\sigma}$ (always hatted) for an estimated standard deviation. We write $\circ$ for the Hadamard (element-wise) product, following the book convention.

32.2.1 Weight-space symmetries in neural networks

The most important weight-space symmetries in deep neural networks arise either as a permutation of hidden units, which reorders the neurons of a layer, or as a rescaling of the outputs of a layer, which we refer to as

scaling in the following. These symmetries and some others – also classically encountered in neural networks – are summarised in table 32.1.

Permuting weights

The foundational work on weight-space symmetries is due to Hecht-Nielsen [724], who formally introduced the notion of *permutation symmetries* in feedforward neural networks. They also established an upper bound on the number of such symmetries based on the number of permissible permutations per layer.

Proposition 32.2 (Permutation symmetries)**.** *Consider a one-hidden-layer network*

$$f_\theta(x) = \boldsymbol{W}_2\, r(\boldsymbol{W}_1 x + b_1) + b_2,$$

with $\boldsymbol{W}_1 \in \mathbb{R}^{m\times n}$, $b_1 \in \mathbb{R}^m$, $\boldsymbol{W}_2 \in \mathbb{R}^{k\times m}$, $b_2 \in \mathbb{R}^k$, and an activation r applied element-wise. Let $\boldsymbol{\pi} \in P_m$ be a permutation matrix acting on the m hidden units. Then the transformation

$$\boldsymbol{W}_1 \mapsto \boldsymbol{\pi}\boldsymbol{W}_1, \qquad b_1 \mapsto \boldsymbol{\pi} b_1, \qquad \boldsymbol{W}_2 \mapsto \boldsymbol{W}_2\boldsymbol{\pi}^{-1}, \qquad b_2 \mapsto b_2,$$

is a weight-space symmetry: for all $x \in \mathbb{R}^n$,

$$\boldsymbol{W}_2\boldsymbol{\pi}^{-1} r(\boldsymbol{\pi}\boldsymbol{W}_1 x + \boldsymbol{\pi} b_1) + b_2 = \boldsymbol{W}_2 r(\boldsymbol{W}_1 x + b_1) + b_2.$$

The construction holds at every hidden layer of a deeper network, so the permutation symmetries of an L-hidden-layer network of widths $m_1, \dots, m_L$ form the finite group $P_{m_1} \times \cdots \times P_{m_L}$ of order $\prod_\ell m_\ell!$.

Proof. Write

$$\boldsymbol{h} = \boldsymbol{W}_1 x + b_1,$$

so that $f_\theta(x) = \boldsymbol{W}_2 r(\boldsymbol{h}) + b_2$. Under the transformation, the pre-activation becomes

$$\boldsymbol{h}' = \boldsymbol{\pi}\boldsymbol{W}_1 x + \boldsymbol{\pi} b_1 = \boldsymbol{\pi}(\boldsymbol{W}_1 x + b_1) = \boldsymbol{\pi}\boldsymbol{h}.$$

Since $\boldsymbol{\pi}$ only permutes the entries of $\boldsymbol{h}$ and r is applied element-wise, the permutation commutes with the activation:

$$r(\boldsymbol{\pi}\boldsymbol{h}) = \boldsymbol{\pi} r(\boldsymbol{h}).$$

Hence the transformed output is

$$\boldsymbol{W}_2\boldsymbol{\pi}^{-1} r(\boldsymbol{\pi}\boldsymbol{h}) + b_2 = \boldsymbol{W}_2\boldsymbol{\pi}^{-1}\boldsymbol{\pi} r(\boldsymbol{h}) + b_2 = \boldsymbol{W}_2 r(\boldsymbol{h}) + b_2,$$

using $\boldsymbol{\pi}^{-1}\boldsymbol{\pi} = \boldsymbol{I}_m$. The output is therefore unchanged, completing the proof. □

Scaling weights

The "layer" aspect of deep neural networks not only entails permutation symmetries but also symmetries related to changes in value. Again, the mechanism is simple: the first layer is modified so that the output changes in scale, sign, etc., and the second layer reverses this change. However, contrary to changing the order of the neurons, changes in value may prove impossible to reverse, depending on the chosen activation function.

One early example of symmetries relative to the intensity of the output are *sign-flip symmetries* introduced by Albertini, Sontag, and Maillot [19]. They appear in the case of odd activation functions. Further generalisations to *asymptotically constant activation functions* (excluding ReLU) were proposed by Kůrková [953], who described a broader class of functional symmetries.

At this time, as one can read in Kůrková [953], non-asymptotically constant functions were not considered relevant for training deep neural networks. However, ReLU, which is unbounded for large positive inputs, later emerged as critical, notably due to its efficiency in large-scale computations. The characterisation of scaling symmetries, therefore, needed to be extended. This was done, for instance, by Petzka [1321]:

Proposition 32.3 (Scaling symmetries)**.** *Consider a one-hidden-layer network*

$$f_\theta(x) = \boldsymbol{W}_2\, r(\boldsymbol{W}_1 x + b_1) + b_2,$$

with $\boldsymbol{W}_1 \in \mathbb{R}^{m\times n}$, $b_1 \in \mathbb{R}^m$, $\boldsymbol{W}_2 \in \mathbb{R}^{k\times m}$, $b_2 \in \mathbb{R}^k$, and assume the activation r is positively homogeneous of degree one*, i.e., $r(\lambda t) = \lambda r(t)$ for all $\lambda > 0$ and $t \in \mathbb{R}$ (this holds for ReLU and leaky ReLU).*

Let $\boldsymbol{\lambda} \in (\mathbb{R}_{>0})^m$ be a vector of positive scaling coefficients and $\boldsymbol{D} = \mathrm{diag}(\boldsymbol{\lambda})$. Then the transformation

$$\boldsymbol{W}_1 \mapsto \boldsymbol{D}\boldsymbol{W}_1, \qquad b_1 \mapsto \boldsymbol{D}b_1, \qquad \boldsymbol{W}_2 \mapsto \boldsymbol{W}_2\boldsymbol{D}^{-1}, \qquad b_2 \mapsto b_2,$$

is a weight-space symmetry: for all $x \in \mathbb{R}^n$,

$$\boldsymbol{W}_2\boldsymbol{D}^{-1}r(\boldsymbol{D}\boldsymbol{W}_1x + \boldsymbol{D}b_1) + b_2 = \boldsymbol{W}_2r(\boldsymbol{W}_1x + b_1) + b_2.$$

Remark. Degree-one positive homogeneity is essential. A degree-two positively homogeneous activation such as $t \mapsto (t_+)^2$ does *not* yield this symmetry, and negative rescalings ($\lambda < 0$) are not ReLU symmetries because $r(\lambda t) \neq \lambda r(t)$ in general.

Proof. Write

$$\boldsymbol{h} = \boldsymbol{W}_1x + b_1,$$

so that $f_\theta(x) = \boldsymbol{W}_2r(\boldsymbol{h}) + b_2$. Under the transformation, the pre-activation becomes

$$\boldsymbol{h}' = \boldsymbol{D}\boldsymbol{W}_1x + \boldsymbol{D}b_1 = \boldsymbol{D}(\boldsymbol{W}_1x + b_1) = \boldsymbol{D}\boldsymbol{h}.$$

Because $\boldsymbol{D} = \mathrm{diag}(\boldsymbol{\lambda})$ with $\boldsymbol{\lambda} > 0$ and r is positively homogeneous of degree one applied element-wise,

$$r(\boldsymbol{D}\boldsymbol{h}) = \boldsymbol{D}r(\boldsymbol{h}).$$

Therefore the transformed output is

$$\boldsymbol{W}_2\boldsymbol{D}^{-1}r(\boldsymbol{D}\boldsymbol{h}) + b_2 = \boldsymbol{W}_2\boldsymbol{D}^{-1}\boldsymbol{D}r(\boldsymbol{h}) + b_2 = \boldsymbol{W}_2r(\boldsymbol{h}) + b_2 = f_\theta(x),$$

since $\boldsymbol{D}^{-1}\boldsymbol{D} = \boldsymbol{I}_m$. Thus the network output is unchanged. □

Petzka, Trimmel, and Sminchisescu [1321] give a complete characterisation for fully connected, scalar-output ReLU networks with one hidden layer. After excluding reducible and degenerate representations (including zero units, positively proportional hidden units, coincident activation hyperplanes, and certain exact cancellations), any two parameterisations realizing the same function differ only by permutations of hidden units and positive neuron-wise rescalings. This characterisation does not directly extend to deeper networks [591]: because later ReLU layers operate on the nonnegative image of preceding layers, even nondegenerate parameter configurations can admit additional, parameter-dependent symmetries arising from interactions between layers.

Gegenfurtner, Grillo, and Montúfar [591] precise these effects for generic three-layer ReLU networks in the bottleneck regime. They give a description of the corresponding parameter fibers and identify new continuous and discrete symmetries. In particular, a first-layer activation hyperplane hidden by the second layer may be translated while compensating through the second-layer biases. Relatedly, but without attempting a symmetry classification, Kobialka et al. [908] study how overparameterisation and the prior jointly reshape Bayesian neural-network posteriors. They identify balancedness, weight reallocation along equal-probability manifolds, and prior conformity as three consequences of redundancy, showing that overparameterisation can produce structured, prior-aligned weight-space posteriors.

32.2.2 Weight-space symmetries on special layers

We extend the previous results to different layers to demonstrate that symmetries can also appear independently from the "layer" aspect of deep neural networks, due to the use of specific functions.

Softmax layer

Proposition 32.4 (Softmax symmetries)**.** *In models ending with a softmax layer, additive shifts to the input logits by a constant vector (shared across all samples) do not affect the output probabilities:*

$$\mathit{softmax}(\mathbf{z} + c\mathbf{1}) = \mathit{softmax}(\mathbf{z}), \quad \forall c \in \mathbb{R},\ \mathbf{z} \in \mathbb{R}^n.$$

Proof. The softmax function is defined for $\mathbf{z} \in \mathbb{R}^n$ by

$$\mathrm{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \quad \text{for } i = 1, \dots, n.$$

Now consider a constant shift $c \in \mathbb{R}$, and let $\mathbf{z}' = \mathbf{z} + c\mathbf{1}$, so that $z'_i = z_i + c$ for all i. Then:

$$\text{softmax}(\mathbf{z}')_i = \frac{e^{z_i+c}}{\sum_{j=1}^n e^{z_j+c}} = \frac{e^c \cdot e^{z_i}}{e^c \cdot \sum_{j=1}^n e^{z_j}} = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} = \text{softmax}(\mathbf{z})_i.$$

Since this holds for each component i, we conclude that: $\text{softmax}(\mathbf{z} + c\mathbf{1}) = \text{softmax}(\mathbf{z})$. □

This logit-shift invariance translates into an unidentifiable direction in the final linear layer. Suppose the pre-softmax logits are produced by $\boldsymbol{z}(x) = \boldsymbol{W}\boldsymbol{h}(x) + \boldsymbol{b}$ with K classes, where $\boldsymbol{h}(x)$ is the penultimate feature vector. For any feature-space vector $\boldsymbol{u}$ and scalar c, the transformation

$$\boldsymbol{W} \mapsto \boldsymbol{W} + \mathbf{1}_K \boldsymbol{u}^\top, \qquad \boldsymbol{b} \mapsto \boldsymbol{b} + c\,\mathbf{1}_K$$

adds $(\boldsymbol{u}^\top \boldsymbol{h}(x) + c)\,\mathbf{1}_K$ to every logit vector and therefore leaves the softmax output unchanged. The softmax thus has a $(\dim \boldsymbol{h} + 1)$-dimensional family of equivalent linear heads. Standard identifiability constraints fix one class row and its bias to zero, or impose zero-sum constraints across the class rows and biases.

Normalisation layers

Batch normalisation, layer normalisation, instance normalisation, and group normalisation introduce additional symmetries in the network through their affine reparametrisation of activations. These layers were introduced and analysed in Ba et al. [69], Ioffe and Szegedy [817], Ulyanov et al. [1648], van Laarhoven [1665], with further work revealing implications for optimisation and generalisation. The precise symmetry depends both on *which* normalisation is used and on *which* statistics it employs (batch statistics during training versus stored running statistics at evaluation), so we state the assumptions explicitly.

Proposition 32.5 (BatchNorm Channel-scaling symmetry)**.** *Consider a linear layer with weight matrix $\boldsymbol{W} \in \mathbb{R}^{m\times n}$ followed by a batch-normalisation layer with learnable scale and shift $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^m$, using* batch *statistics and the idealized form without the numerical constant ϵ,*

$$\text{norm}(\boldsymbol{z}, \boldsymbol{\beta}, \boldsymbol{\gamma}) = \frac{\boldsymbol{z} - \hat{\boldsymbol{\mu}}}{\hat{\boldsymbol{\sigma}}} \circ \boldsymbol{\gamma} + \boldsymbol{\beta},$$

where $\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\sigma}}$ are the per-channel mean and standard deviation of $\boldsymbol{z} = \boldsymbol{W}x$. Let $\boldsymbol{\lambda} \in (\mathbb{R}_{>0})^m$ and $\boldsymbol{D} = \text{diag}(\boldsymbol{\lambda})$. Then, for all $x \in \mathbb{R}^n$,

$$\text{norm}(\boldsymbol{D}\boldsymbol{W}x, \boldsymbol{\beta}, \boldsymbol{\gamma}) = \text{norm}(\boldsymbol{W}x, \boldsymbol{\beta}, \boldsymbol{\gamma}).$$

Proof. Let $\boldsymbol{z} = \boldsymbol{W}x \in \mathbb{R}^m$, and let $\tilde{\boldsymbol{z}} = \text{diag}(\boldsymbol{\lambda})\boldsymbol{z} = \text{diag}(\boldsymbol{\lambda})\boldsymbol{W}x$.

The normalisation of $\boldsymbol{z}$ is given by:

$$\text{norm}(\boldsymbol{z}, \boldsymbol{\beta}, \boldsymbol{\gamma}) = \frac{\boldsymbol{z} - \hat{\boldsymbol{\mu}}}{\hat{\boldsymbol{\sigma}}} \circ \boldsymbol{\gamma} + \boldsymbol{\beta}.$$

Now consider the transformed input $\tilde{\boldsymbol{z}} = \text{diag}(\boldsymbol{\lambda})\boldsymbol{z}$. Then:

$$\text{norm}(\tilde{\boldsymbol{z}}, \boldsymbol{\beta}, \boldsymbol{\gamma}) = \frac{\tilde{\boldsymbol{z}} - \tilde{\boldsymbol{\mu}}}{\tilde{\boldsymbol{\sigma}}} \circ \boldsymbol{\gamma} + \boldsymbol{\beta},$$

where $\tilde{\boldsymbol{\mu}} = \text{diag}(\boldsymbol{\lambda})\hat{\boldsymbol{\mu}}$ and $\tilde{\boldsymbol{\sigma}} = \text{diag}(\boldsymbol{\lambda})\hat{\boldsymbol{\sigma}}$, since both the mean and standard deviation scale linearly under element-wise positive scaling.

Therefore,

$$\begin{aligned}
\text{norm}(\tilde{\boldsymbol{z}}, \boldsymbol{\beta}, \boldsymbol{\gamma}) &= \frac{\text{diag}(\boldsymbol{\lambda})\boldsymbol{z} - \text{diag}(\boldsymbol{\lambda})\hat{\boldsymbol{\mu}}}{\text{diag}(\boldsymbol{\lambda})\hat{\boldsymbol{\sigma}}} \circ \boldsymbol{\gamma} + \boldsymbol{\beta} \\
&= \frac{\text{diag}(\boldsymbol{\lambda})(\boldsymbol{z} - \hat{\boldsymbol{\mu}})}{\text{diag}(\boldsymbol{\lambda})\hat{\boldsymbol{\sigma}}} \circ \boldsymbol{\gamma} + \boldsymbol{\beta} \\
&= \left(\frac{\boldsymbol{z} - \hat{\boldsymbol{\mu}}}{\hat{\boldsymbol{\sigma}}}\right) \circ \boldsymbol{\gamma} + \boldsymbol{\beta} \\
&= \text{norm}(\boldsymbol{z}, \boldsymbol{\beta}, \boldsymbol{\gamma}),
\end{aligned}$$

which completes the proof. □

Remark. With the ϵ added inside the square root, or with *stored* running statistics that do not co-scale with the activations, the cancellation is only approximate. The result also does not transfer unchanged to other normalisations: under LayerNorm the statistics are computed across the feature dimension, so an *arbitrary* coordinate-wise positive scaling does not cancel; under GroupNorm the invariance requires the scale $\boldsymbol{\lambda}$ to be constant within each normalised group.

Proposition 32.6 (Bias-shift symmetry of normalisation layers)**.** *Consider a linear layer with weight matrix $\boldsymbol{W} \in \mathbb{R}^{m\times n}$ followed by a normalisation layer with learnable scale and shift $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^m$, and let $\boldsymbol{c} \in \mathbb{R}^m$ be a constant shift of the pre-normalisation activations $\boldsymbol{z} = \boldsymbol{W}x$. Two cases arise.*

1. ***Centering with current activations.*** *If the normalisation subtracts the mean of the current activations, then adding $\boldsymbol{c}$ shifts the mean by the same amount and is removed by the centering step. The output is invariant with $\boldsymbol{\beta}$* unchanged*:*
$$\mathrm{norm}(\boldsymbol{W}x + \boldsymbol{c}, \boldsymbol{\beta}, \boldsymbol{\gamma}) = \mathrm{norm}(\boldsymbol{W}x, \boldsymbol{\beta}, \boldsymbol{\gamma}).$$
2. ***Centering with stored statistics.*** *If the normalisation uses fixed stored statistics $\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\sigma}}$ that do not move with $\boldsymbol{c}$, then the shift propagates through and must be absorbed by the shift parameter:*
$$\mathrm{norm}(\boldsymbol{W}x + \boldsymbol{c},\ \boldsymbol{\beta} - \boldsymbol{\gamma} \circ \boldsymbol{c} \oslash \hat{\boldsymbol{\sigma}},\ \boldsymbol{\gamma}) = \mathrm{norm}(\boldsymbol{W}x, \boldsymbol{\beta}, \boldsymbol{\gamma}),$$
where $\oslash$ denotes element-wise division.

In both cases the result follows from the fact that normalisation centres and rescales its input: a constant shift of the input is either annihilated by centering against the current mean (case 1) or exactly compensated by the shift parameter $\boldsymbol{\beta}$ (case 2). No scaling vector $\boldsymbol{\lambda}$ is involved in this symmetry.

Multi-head attention layer

Convention. Throughout this section, multi-head attention (MHA) uses h heads with $d = h\,d_h$ and projects an input sequence $x \in \mathbb{R}^{T\times d_{\text{model}}}$ by $\boldsymbol{Q} = x(\boldsymbol{W}^Q)^\top$, $\boldsymbol{K} = x(\boldsymbol{W}^K)^\top$, $\boldsymbol{V} = x(\boldsymbol{W}^V)^\top$, computes per-head attention, concatenates the heads, and applies the output projection $\boldsymbol{W}^O$. With this right-multiplication convention, block permutation and scaling matrices act on the *right* of the activations and on the *left* of the stored weight matrices.

Proposition 32.7 (Head Permutation Symmetries in MHA)**.** *Let a MHA module use h attention heads, with query, key, and value weight matrices $\boldsymbol{W}^Q, \boldsymbol{W}^K, \boldsymbol{W}^V \in \mathbb{R}^{d\times d_{model}}$, partitioned into h blocks along the output dimension. Let $\boldsymbol{W}^O \in \mathbb{R}^{d_{model}\times d}$ be the output projection matrix.*

Let $\boldsymbol{\pi} \in P_h$ (note that $\boldsymbol{\pi} \in \mathbb{R}^{h\times h}$) be a permutation matrix acting on the head dimension by permuting the order of the heads, and let $\boldsymbol{\Pi} \in \mathbb{R}^{d\times d}$ be its block-wise extension acting on the concatenated output of all heads. Then, for all input sequences $x \in \mathbb{R}^{T\times d_{model}}$, the following transformation leaves the output of the MHA layer invariant:

$$\boldsymbol{W}^Q \mapsto \boldsymbol{\Pi}\boldsymbol{W}^Q, \quad \boldsymbol{W}^K \mapsto \boldsymbol{\Pi}\boldsymbol{W}^K, \quad \boldsymbol{W}^V \mapsto \boldsymbol{\Pi}\boldsymbol{W}^V, \quad \boldsymbol{W}^O \mapsto \boldsymbol{W}^O\boldsymbol{\Pi}^{-1}.$$

That is,

$$MHA(x; \boldsymbol{W}^Q, \boldsymbol{W}^K, \boldsymbol{W}^V, \boldsymbol{W}^O) = MHA(x; \boldsymbol{\Pi}\boldsymbol{W}^Q, \boldsymbol{\Pi}\boldsymbol{W}^K, \boldsymbol{\Pi}\boldsymbol{W}^V, \boldsymbol{W}^O\boldsymbol{\Pi}^{-1}).$$

Here $\boldsymbol{\Pi} = \boldsymbol{\pi}\otimes\boldsymbol{I}_{d_h}$ and $\boldsymbol{\Pi}^{-1} = \boldsymbol{\pi}^{-1}\otimes\boldsymbol{I}_{d_h}$ are the block-permutation matrix and its inverse acting on the concatenated heads, $\otimes$ is the Kronecker product, and $d_h = d/h$.

The proof of proposition 32.7 is left as an exercise for the reader.

Proposition 32.8 (Head-wise Query-Key Scaling Symmetry)**.** *Let h be the number of attention heads, and $d = h \cdot d_h$ the total dimension of the concatenated heads. For any positive scaling vector $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_h) \in (\mathbb{R}_{>0})^h$, define the block-diagonal matrix $\mathbf{A} \in \mathbb{R}^{d\times d}$ as:*

$$\mathbf{A} = \mathrm{diag}(\alpha_1\boldsymbol{I}_{d_h}, \ldots, \alpha_h\boldsymbol{I}_{d_h}),$$

and let $\mathbf{A}^{-1}$ be its inverse:

$$\mathbf{A}^{-1} = \mathrm{diag}(\alpha_1^{-1}\boldsymbol{I}_{d_h}, \ldots, \alpha_h^{-1}\boldsymbol{I}_{d_h}).$$

Then for all input sequences $x \in \mathbb{R}^{T\times d_{model}}$, the following transformation leaves the MHA output invariant:

$$\boldsymbol{W}^Q \mapsto \mathbf{A}\boldsymbol{W}^Q, \quad \boldsymbol{W}^K \mapsto \mathbf{A}^{-1}\boldsymbol{W}^K.$$

That is,

$$\mathrm{MHA}(x; \boldsymbol{W}^Q, \boldsymbol{W}^K, \boldsymbol{W}^V, \boldsymbol{W}^O) = \mathrm{MHA}(x; \mathbf{A}\boldsymbol{W}^Q, \mathbf{A}^{-1}\boldsymbol{W}^K, \boldsymbol{W}^V, \boldsymbol{W}^O).$$

Table 32.1: Weight-space symmetries of common neural-network building blocks. Gaussian-prior invariance (PI) refers to an independent isotropic prior $\mathcal{N}(0, \sigma^2 I)$ on all weights and biases. Y: the prior measure is invariant under the transformation ($|\det J_{\mathcal{T}}| = 1$). N: the likelihood is invariant but the prior is not ($|\det J_{\mathcal{T}}| \neq 1$). m = hidden-layer width; L = number of hidden layers; K = number of output classes; $d_{\boldsymbol{h}}$ = penultimate feature dimension.

Symmetry	**Assumptions**	**Group / transform.**	**PI**	**Inference consequence**
MLP hidden layers				
Hidden-unit permutation	Any element-wise activation; nondegenerate units	$(\mathfrak{S}_m)^L$; finite, order $(m!)^L$; permutes rows of (W_ℓ, b_ℓ) and columns of $W_{\ell+1}$	Y	$(m!)^L$ identical posterior modes
ReLU positive scaling	Degree-1 pos. homogeneous activation; nondegenerate units	$(\mathbb{R}_{>0})^{mL}$; continuous Lie group, dim. mL; scales rows of (W_ℓ, b_ℓ) by $\lambda_i > 0$ and columns of $W_{\ell+1}$ by λ_i^{-1}	N	Connected equal-loss ridges in the likelihood; posterior density varies along ridge under Gaussian prior
Odd-activation sign flip	Odd activation; nondegenerate units	$(\mathbb{Z}/2\mathbb{Z})^{mL}$; finite, order 2^{mL}; negates row i of (W_ℓ, b_ℓ) and column i of $W_{\ell+1}$	Y	2^{mL} additional identical posterior copies
Classification output layer				
Softmax logit shift	K-class softmax output layer	$\mathbb{R}^{d_{\boldsymbol{h}}+1}$; continuous; $W \mapsto W + \mathbf{1}_K \boldsymbol{u}^\top$, $\boldsymbol{b} \mapsto \boldsymbol{b} + c\,\mathbf{1}_K$	N	$(d_{\boldsymbol{h}}+1)$-dimensional unidentifiable subspace in the final linear map
Normalisation layers				
BatchNorm channel scaling & bias shift	BatchNorm with learnable $\boldsymbol{\gamma}, \boldsymbol{\beta}$; batch statistics; idealized ($\varepsilon = 0$)	$(\mathbb{R}_{>0})^m \ltimes \mathbb{R}^m$; continuous; scales and shifts pre-norm activations, absorbed by $\boldsymbol{\gamma}, \boldsymbol{\beta}$	N	Likelihood ridge; posterior density varies under Gaussian prior; cancellation approximate for $\varepsilon > 0$ or stored running statistics
LayerNorm & GroupNorm	GroupNorm with G groups; constant symmetry within each normalised group	$(\mathbb{R}_{>0})^G \ltimes \mathbb{R}^G$; continuous; group-restricted scaling and shift only	N	Narrower ridge than BatchNorm; arbitrary coordinate-wise scaling breaks LayerNorm invariance

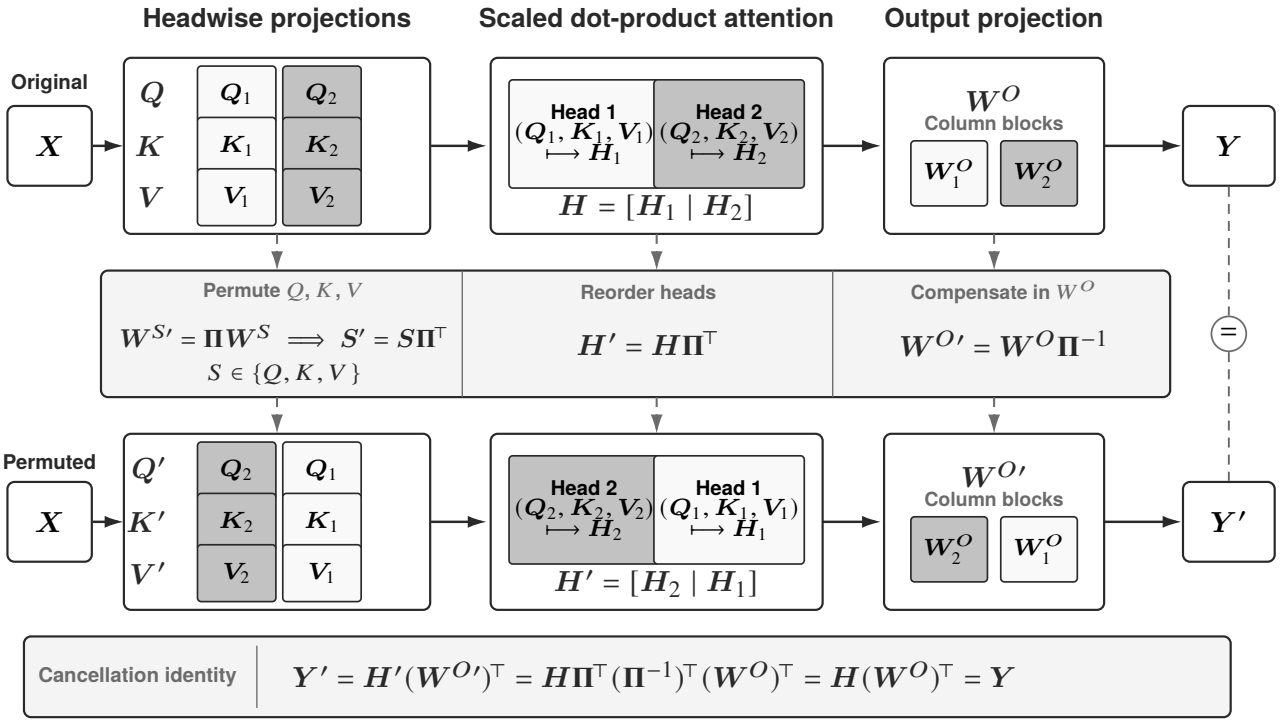


Figure 32.3: Two-head illustration of attention-head permutation symmetry. The block permutation $\mathbf{\Pi} = \boldsymbol{\pi} \otimes \boldsymbol{I}_{d_h}$ permutes the row blocks of $\boldsymbol{W}^Q$, $\boldsymbol{W}^K$, and $\boldsymbol{W}^V$, thereby reordering the corresponding query, key, value, and attention-head blocks. Applying the inverse permutation to the column blocks of the output projection, $\boldsymbol{W}^{O\prime} = \boldsymbol{W}^O\mathbf{\Pi}^{-1}$, exactly cancels this reordering, so that the MHA output is unchanged: $\boldsymbol{Y}' = \boldsymbol{Y}$. For the two-head transposition illustrated here, $\mathbf{\Pi}^{-1} = \mathbf{\Pi}$.

Proof. We analyse the effect of the proposed transformation on the MHA computation. The MHA mechanism proceeds as follows:

1. Compute the query, key, and value projections:

$$\boldsymbol{Q} = x\boldsymbol{W}^{Q\top}, \quad \boldsymbol{K} = x\boldsymbol{W}^{K\top}, \quad \boldsymbol{V} = x\boldsymbol{W}^{V\top}.$$

   Each of these has shape $T \times d$, with $d = h \cdot d_h$, and can be split into h blocks of size d_h per head.

2. Apply attention separately for each head $i = 1, \ldots, h$:

$$\text{head}_i = \text{Softmax}\left(\frac{\boldsymbol{Q}_i\boldsymbol{K}_i^\top}{\sqrt{d_h}}\right)\boldsymbol{V}_i,$$

   where $\boldsymbol{Q}_i, \boldsymbol{K}_i, \boldsymbol{V}_i \in \mathbb{R}^{T\times d_h}$ are the corresponding blocks for head i.

3. Concatenate the heads and project with $\boldsymbol{W}^O$:

$$\text{MHA}(x) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \cdot \boldsymbol{W}^{O\top}.$$

Now consider modifying $\boldsymbol{W}^Q$ and $\boldsymbol{W}^K$ using the block-diagonal scaling matrix $\mathbf{A}$ and its inverse:

$$\boldsymbol{W}^Q_{\text{new}} = \mathbf{A}\boldsymbol{W}^Q, \quad \boldsymbol{W}^K_{\text{new}} = \mathbf{A}^{-1}\boldsymbol{W}^K.$$

Then:

$$\boldsymbol{Q}_{\text{new}} = x\boldsymbol{W}^{Q\top}\mathbf{A}^\top, \quad \boldsymbol{K}_{\text{new}} = x\boldsymbol{W}^{K\top}(\mathbf{A}^{-1})^\top.$$

Hence the attention score matrix becomes:

$$\frac{\boldsymbol{Q}_{\text{new}}\boldsymbol{K}_{\text{new}}^\top}{\sqrt{d_h}} = \frac{(x\boldsymbol{W}^{Q\top}\mathbf{A}^\top)(x\boldsymbol{W}^{K\top}(\mathbf{A}^{-1})^\top)^\top}{\sqrt{d_h}}.$$

Since $\mathbf{A}$ is diagonal, it is symmetric ($\mathbf{A} = \mathbf{A}^\top$), and the central factor satisfies $\mathbf{A}^\top\big((\mathbf{A}^{-1})^\top\big)^\top = \mathbf{A}\mathbf{A}^{-1} = \boldsymbol{I}$. Thus,

$$\frac{\boldsymbol{Q}_{\text{new}}\boldsymbol{K}_{\text{new}}^\top}{\sqrt{d_h}} = \frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_h}},$$

which implies:

$$\text{Softmax}\left(\frac{\boldsymbol{Q}_{\text{new}}\boldsymbol{K}_{\text{new}}^\top}{\sqrt{d_h}}\right) = \text{Softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_h}}\right).$$

As a result, the attention weights for each head are unchanged, and the resulting attention outputs are the same for each head. Since the values $\boldsymbol{V}$ and output projection $\boldsymbol{W}^O$ are unchanged, the final MHA output remains invariant. □

Proposition 32.9 (Head-wise Value Scaling Symmetry)**.** *Let h be the number of attention heads and $d = h \cdot d_h$ the total concatenated head dimension. For any invertible scaling vector $\boldsymbol{\lambda} = (\lambda_1, \ldots, \lambda_h) \in (\mathbb{R} \setminus \{0\})^h$, define the block-diagonal matrix $\boldsymbol{\Lambda} = \text{diag}(\lambda_1 \boldsymbol{I}_{d_h}, \ldots, \lambda_h \boldsymbol{I}_{d_h}) \in \mathbb{R}^{d\times d}$ and its inverse $\boldsymbol{\Lambda}^{-1} = \text{diag}(\lambda_1^{-1}\boldsymbol{I}_{d_h}, \ldots, \lambda_h^{-1}\boldsymbol{I}_{d_h})$.*

Then for all input sequences $x \in \mathbb{R}^{T\times d_{model}}$, the following transformation leaves the MHA output invariant:

$$\boldsymbol{W}^V \mapsto \boldsymbol{\Lambda}\boldsymbol{W}^V, \quad \boldsymbol{W}^O \mapsto \boldsymbol{W}^O\boldsymbol{\Lambda}^{-1}.$$

That is,

$$\text{MHA}(x; \boldsymbol{W}^Q, \boldsymbol{W}^K, \boldsymbol{W}^V, \boldsymbol{W}^O) = \text{MHA}(x; \boldsymbol{W}^Q, \boldsymbol{W}^K, \boldsymbol{\Lambda}\boldsymbol{W}^V, \boldsymbol{W}^O\boldsymbol{\Lambda}^{-1}).$$

Proof. Let us examine the MHA computation steps. For an input $x \in \mathbb{R}^{T\times d_{\text{model}}}$:

- Compute projections:
$$\boldsymbol{Q} = x\boldsymbol{W}^{Q\top}, \quad \boldsymbol{K} = x\boldsymbol{W}^{K\top}, \quad \boldsymbol{V} = x\boldsymbol{W}^{V\top} \in \mathbb{R}^{T\times d}.$$
- Split $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ into heads:
$$\boldsymbol{Q} = [\boldsymbol{Q}_1 \| \cdots \| \boldsymbol{Q}_h], \quad \boldsymbol{K} = [\boldsymbol{K}_1 \| \cdots \| \boldsymbol{K}_h], \quad \boldsymbol{V} = [\boldsymbol{V}_1 \| \cdots \| \boldsymbol{V}_h],$$
where $\boldsymbol{Q}_i, \boldsymbol{K}_i, \boldsymbol{V}_i \in \mathbb{R}^{T\times d_h}$.
- Each attention head is:
$$\text{head}_i = \text{Softmax}\left(\frac{\boldsymbol{Q}_i\boldsymbol{K}_i^\top}{\sqrt{d_h}}\right)\boldsymbol{V}_i.$$
- Final output:
$$\text{MHA}(x) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \cdot \boldsymbol{W}^{O\top}.$$

Now apply the transformation $\boldsymbol{W}^V \mapsto \boldsymbol{\Lambda}\boldsymbol{W}^V$, so that:

$$\boldsymbol{V}' = x\boldsymbol{W}^{V\top}\boldsymbol{\Lambda}^\top = \boldsymbol{V}\boldsymbol{\Lambda}^\top.$$

Because $\boldsymbol{\Lambda}$ is block-diagonal, this scales each head's values:

$$\boldsymbol{V}_i' = \lambda_i \boldsymbol{V}_i \quad \text{for each } i.$$

Therefore, the new attention heads are:

$$\text{head}_i' = \text{Softmax}\left(\frac{\boldsymbol{Q}_i\boldsymbol{K}_i^\top}{\sqrt{d_h}}\right) \cdot \lambda_i \boldsymbol{V}_i = \lambda_i \cdot \text{head}_i.$$

Now concatenate:

$$\text{Concat}(\text{head}_1', \ldots, \text{head}_h') = \text{Concat}(\lambda_1\text{head}_1, \ldots, \lambda_h\text{head}_h) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \cdot \boldsymbol{\Lambda}^\top.$$

Finally, apply the transformed output projection:

$$\text{MHA}'(x) = \text{Concat}(\text{head}_1', \ldots, \text{head}_h') \cdot (\boldsymbol{W}^O\boldsymbol{\Lambda}^{-1})^\top.$$

Substitute the concatenated form:

$$= \big(\text{Concat}(\text{head}_1, \ldots, \text{head}_h) \cdot \boldsymbol{\Lambda}^\top\big) \cdot (\boldsymbol{\Lambda}^{-1})^\top \boldsymbol{W}^{O\top}.$$

But since $\boldsymbol{\Lambda}^\top(\boldsymbol{\Lambda}^{-1})^\top = \boldsymbol{I}$, we have:

$$\text{MHA}'(x) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) \cdot \boldsymbol{W}^{O\top} = \text{MHA}(x).$$

□

We summarise the MHA-related symmetries in table 32.2.

Table 32.2: Weight-space symmetries of common transformers building blocks. Gaussian-prior invariance (PI) refers to an independent isotropic prior $\mathcal{N}(0, \sigma^2 I)$ on all weights and biases. Y: the prior measure is invariant under the transformation. N: the likelihood is invariant but the prior is not. m = hidden-layer width; h = number of attention heads; $d_h = d/h$ = per-head dimension; K = number of output classes.

Symmetry	**Assumptions**	**Group / transform.**	**PI**	**Inference consequence**
Multi-head attention				
Attention-head permutation	h equal-size d_h-dim heads; MHA with W^Q, W^K, W^V, W^O	$\mathfrak{S}_h$; finite, order $h!$; block-permutes rows of (W^Q, W^K, W^V) and columns of W^O via $\mathbf{\Pi} = \boldsymbol{\pi} \otimes I_{d_h}$	Y	$h!$ equivalent configurations per attention layer; compounds with hidden-unit symmetry in transformer blocks
QK inverse scaling	h-head MHA; per-head query and key projections	$(\mathbb{R}_{>0})^h$; continuous; $W_i^Q \mapsto \alpha_i W_i^Q$, $W_i^K \mapsto \alpha_i^{-1} W_i^K$ per head i	N	Equal attention-score manifold per head; $\|W_i^Q\|_F$ and $\|W_i^K\|_F$ change asymmetrically; posterior varies
Value-output inverse scaling	h-head MHA; invertible per-head scaling ($\lambda_i \neq 0$)	$(\mathbb{R} \setminus \{0\})^h$; continuous; $W_i^V \mapsto \lambda_i W_i^V$, $W_i^O \mapsto \lambda_i^{-1} W_i^O$ per head i	N	Connected equal-output manifold; more general than ReLU scaling (allows sign flips)

32.3 From likelihood symmetries to posterior symmetries

The previous section catalogued transformations $\mathcal{T}$ that leave the implemented function unchanged, $f_{\mathcal{T}(\theta)} = f_\theta$. We now make explicit what they do, and do *not*, imply for Bayesian inference. This is the central point for Bayesian deep learning: a function symmetry is automatically a *likelihood* symmetry, but it is a *posterior* symmetry only under an additional condition on the prior. Tables 32.1 and 32.2 include a column indicating whether the usual Gaussian prior is also invariant under the transformation.

Likelihood invariance. Because the likelihood depends on θ only through the function f_θ, any weight-space symmetry preserves it:

$$p(\mathcal{D} \mid \mathcal{T}(\theta)) = p(\mathcal{D} \mid \theta), \qquad \text{since } f_{\mathcal{T}(\theta)} = f_\theta.$$

Equivalently, the likelihood is constant on every orbit $G \cdot \theta$.

When is the posterior invariant? Write the posterior density (with respect to a chosen base measure on $\mathbb{R}^d$, typically Lebesgue) as $p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)\, \rho(\theta)$, where ρ is the prior density. For an invertible, differentiable transformation $\mathcal{T}$ with Jacobian $J_\mathcal{T}$, the pushforward of the posterior under $\mathcal{T}$ has density proportional to $p(\mathcal{D} \mid \mathcal{T}^{-1}(\theta))\, \rho(\mathcal{T}^{-1}(\theta))\, |\det J_{\mathcal{T}^{-1}}(\theta)|$. Using likelihood invariance, the posterior is invariant under $\mathcal{T}$ if and only if the prior *measure* is invariant, i.e.

$$\rho(\mathcal{T}(\theta))\, \big|\det J_\mathcal{T}(\theta)\big| = \rho(\theta) \qquad \text{for all } \theta. \tag{32.2}$$

The factor $|\det J_\mathcal{T}|$ is exactly the change in the base measure; ignoring it is a common source of confusion. A function symmetry that violates (32.2) still leaves the likelihood flat along the orbit, but the posterior density *varies* along it.

Permutations are posterior symmetries (under exchangeable priors). A hidden-unit permutation acts by an orthogonal coordinate permutation, so $|\det J_\mathcal{T}| = 1$. If the prior is exchangeable, i.e. the per-unit incoming and outgoing weights and biases are independent and identically distributed across the m hidden units, then ρ is permutation-invariant and (32.2) holds. Permutations are therefore genuine posterior symmetries: the posterior has, up to stabilizers and degenerate units, $(m!)^L$ identical copies, one per ordering. Canonical ordering of units can remove these copies.

ReLU scaling is usually *not* a posterior symmetry. The scaling transformation $\boldsymbol{W}_1 \mapsto \mathrm{diag}(\boldsymbol{\lambda})\boldsymbol{W}_1$, $\boldsymbol{b}_1 \mapsto \mathrm{diag}(\boldsymbol{\lambda})\boldsymbol{b}_1$, $\boldsymbol{W}_2 \mapsto \boldsymbol{W}_2\, \mathrm{diag}(\boldsymbol{\lambda})^{-1}$ is linear in θ with

$$\big|\det J_\mathcal{T}\big| = \prod_{i=1}^{m} \lambda_i^{(n+1)-k},$$

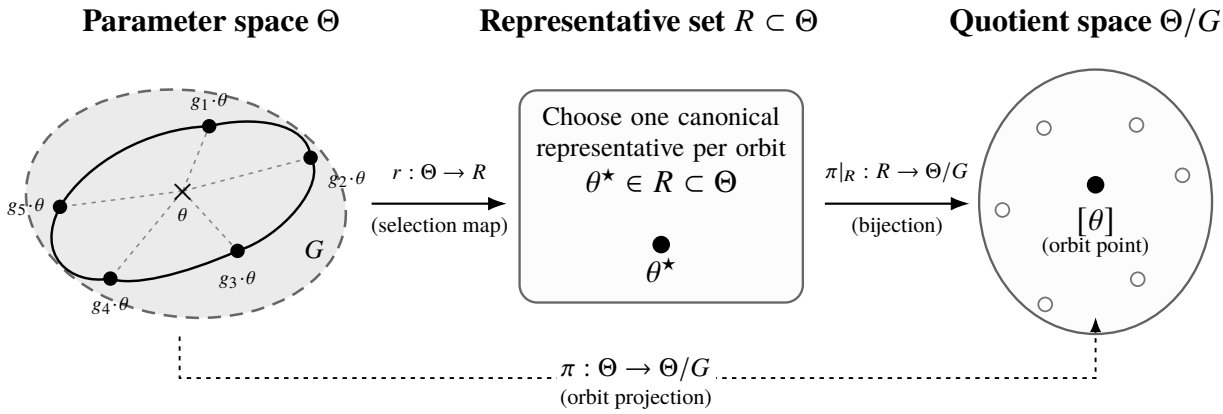


Figure 32.4: Quotienting parameter space by a symmetry group. The orbit $\mathcal{O}_\theta = \{g{\cdot}\theta \mid g \in G\}$ contains parameter vectors related by the action of G. A representative-selection map $r\colon \Theta \to R$ chooses a canonical parameter $\theta^\star$ from each orbit, where $R \subset \Theta$ contains one representative per orbit. The quotient projection $\pi\colon \Theta \to \Theta/G$ maps every parameter vector in the same orbit to the common equivalence class $[\theta]$. Consequently, the restriction $\pi|_R\colon R \to \Theta/G$ is a bijection, so the representative set R provides a concrete realisation of the quotient space.

since each of the $n+1$ entries of row i of $(\boldsymbol{W}_1, \boldsymbol{b}_1)$ is multiplied by λ_i and each of the k entries of column i of $\boldsymbol{W}_2$ by λ_i^{-1}. Under an independent Gaussian prior (equivalently, ℓ_2 weight decay), $\rho(\mathcal{T}(\theta)) \neq \rho(\theta)$ in general, and the factor above does not restore (32.2). Hence ReLU scaling preserves the likelihood, producing a flat *ridge*, but tilts the posterior density along that ridge: it is a likelihood symmetry, not a posterior symmetry. It becomes a posterior symmetry only for a scale-invariant prior chosen so that (32.2) holds.

Inference on the quotient. When a transformation *is* a genuine posterior symmetry, the redundant copies carry no extra information and can be removed by restricting inference to a symmetry-free reference set, one representative per orbit, equipped with the correct quotient measure (the pushforward of the prior under the canonicalisation map). Sampling or variational inference on this quotient avoids revisiting equivalent modes and yields better-calibrated diagnostics, while leaving the predictive distribution in function space unchanged. The practical procedures of the next section construct such representatives.

32.4 Choosing a representative modulo symmetries

Weight-space symmetries are transformations of the parameters that leave the function implemented by the network unchanged. Although they do not affect the output, they introduce redundancy into the parameter space, which could be problematic for several tasks:

- **Visualisation and interpretation of parameter space:** symmetries can distort geometry, making it hard to interpret loss landscapes [826, 980].
- **Bayesian inference:** redundant modes can slow down or confuse posterior sampling in Bayesian neural networks [980, 1029].
- **Weight averaging and ensembles:** permutation symmetries explain why *naive* parameter averaging can fail, because matched units may appear in different orders in two independently trained networks. Aligning the networks modulo their symmetries before averaging is what makes the operation meaningful, as in *Git Re-Basin* [15].

We select one representative from each orbit, i.e. a canonicalisation of the quotient $\mathbb{R}^d/G$ of section 32.1. In this section we describe practical procedures for two common symmetries in two-layer ReLU MLPs, *scaling* and *permutation* of the hidden units. We assume throughout that no hidden unit is degenerate (no zero row of $\boldsymbol{W}_1$ and no zero column of $\boldsymbol{W}_2$); degenerate units have to be handled separately because they break the one-to-one correspondence between units and orbit coordinates.

Discrete versus continuous symmetries. The two symmetries are of fundamentally different types, and they require different tools. For an MLP with L hidden layers of width m, the permutations form a *finite* group of size $(m!)^L$, whereas the ReLU positive scalings form a *continuous* Lie group $(\mathbb{R}_{>0})^{mL}$ of dimension mL. Under exchangeable priors, the discrete symmetries produce a finite set of equivalent posterior modes, for which a canonical ordering of the units is a natural representative. The continuous likelihood symmetries instead produce positive-dimensional loss orbits, and yield posterior *ridges* only when the prior and base measure also preserve the symmetry; here a representative constraint or an equi-normalisation is the appropriate tool. We treat the two cases in turn and then combine them. In the Bayesian setting, canonical ordering of hidden units has been used to improve BNN inference [1342], and symmetry-free reference sets together with symmetry-aware MCMC have been proposed to sample the quotient directly [1747].

32.4.1 Scaling symmetry and its representative

As described in proposition 32.3, if r is positively homogeneous of degree one, then for any positive vector $\boldsymbol{\lambda} \in (\mathbb{R}_{>0})^m$ and $\boldsymbol{D} = \mathrm{diag}(\boldsymbol{\lambda})$,

$$\boldsymbol{W}_2\, r(\boldsymbol{W}_1 x + \boldsymbol{b}_1) + \boldsymbol{b}_2 = \boldsymbol{W}_2 \boldsymbol{D}^{-1}\, r(\boldsymbol{D}\boldsymbol{W}_1 x + \boldsymbol{D}\boldsymbol{b}_1) + \boldsymbol{b}_2.$$

Scaling row i of $(\boldsymbol{W}_1, \boldsymbol{b}_1)$ by λ_i and column i of $\boldsymbol{W}_2$ by λ_i^{-1} leaves the output unchanged. To select a representative, we fix a canonical scale by normalising each row of $\boldsymbol{W}_1$ (including the corresponding entry of $\boldsymbol{b}_1$) to unit ℓ_2 norm and compensating in $\boldsymbol{W}_2$.

Pseudocode for removing scaling symmetry:

```
def canonical_scaling(W1, W2):
    # W1: weight matrix from input to hidden (m x n)
    # W2: weight matrix from hidden to output (k x m)

    m = W1.shape[0]  # number of hidden units
    eps = 1e-8       # small constant to avoid division by zero

    # Step 1: Compute L2 norm of each row of W1
    lambdas = np.linalg.norm(W1, axis=1) + eps  # shape (m,)

    # Step 2: Normalise W1 rows
    W1_canon = W1 / lambdas[:, None]            # shape (m x n)

    # Step 3: Rescale W2 columns
    W2_canon = W2 * lambdas[None, :]            # shape (k x m)

    return W1_canon, W2_canon
```

Listing 32.1: Fix the scaling representative for a 2-layer MLP

This procedure maps all parameter configurations in the same scaling orbit to a single representative in which every row of $\boldsymbol{W}_1$ has unit norm. The degree-one positive homogeneity of the activation guarantees that the network output is unchanged, since the compensating rescaling of $\boldsymbol{W}_2$ exactly cancels the rescaling of $\boldsymbol{W}_1$. The ℓ_2 norm is the standard choice, but other norms such as ℓ_1 or the max-norm can be used.

32.4.2 Permutation symmetry and its representative

As described in proposition 32.2, permuting the order of the hidden units does not affect the function computed by the network:

$$\boldsymbol{W}_2\, r(\boldsymbol{W}_1 x + \boldsymbol{b}_1) + \boldsymbol{b}_2 = \boldsymbol{W}_2 \boldsymbol{\pi}^{-1}\, r(\boldsymbol{\pi}\boldsymbol{W}_1 x + \boldsymbol{\pi}\boldsymbol{b}_1) + \boldsymbol{b}_2,$$

where $\boldsymbol{\pi}$ is a permutation matrix. To select a representative, we sort the hidden units by a fixed scalar score and reorder the rows of $(\boldsymbol{W}_1, \boldsymbol{b}_1)$ and the columns of $\boldsymbol{W}_2$ accordingly.

Choice of an invariant score. A subtle point is that the score must be a genuine invariant of the orbit we still wish to quotient. Sorting by the raw row norm $\|\boldsymbol{W}_1[i]\|$ is *not* valid in the presence of scaling symmetry: two parameters in the same scaling orbit can have different row norms and hence different sort orders, so the

representative would not be well defined. We therefore fix the scale first (previous subsection) and then sort by a quantity that is invariant under the residual symmetry. After normalising the rows of $\boldsymbol{W}_1$ to unit norm, all of them have score 1, so the row norm is uninformative and any tie-break would come from the arbitrary `eps` rather than from the data. A meaningful choice is the *outgoing* weight magnitude, the norm of column i of $\boldsymbol{W}_2$ after normalisation, which equals the product of incoming and outgoing norms in the original parametrisation and is therefore invariant along the scaling orbit. The pseudocode below assumes the rows of $\boldsymbol{W}_1$ have already been normalised.

Pseudocode for removing permutation symmetry:

```
def canonical_permutation(W1, W2):
    # Assumes rows of W1 are already L2-normalised (scale fixed).
    # W1: input-to-hidden weights (m x n)
    # W2: hidden-to-output weights (k x m)

    # Step 1: Score each hidden unit by a scaling-invariant quantity:
    #         the outgoing-weight norm (column norm of W2).
    scores = np.linalg.norm(W2, axis=0)  # shape (m,)

    # Step 2: Sort hidden units by descending score
    perm = np.argsort(-scores)           # permutation indices

    # Step 3: Apply the permutation consistently
    W1_canon = W1[perm, :]               # permute rows
    W2_canon = W2[:, perm]               # permute columns

    return W1_canon, W2_canon
```

Listing 32.2: Choose a permutation representative for a scale-normalised 2-layer MLP

Sorting by a scaling-invariant outgoing-weight norm fixes a canonical order of the hidden units, so that all parameter configurations related by a permutation (after the scale has been fixed) map to a single representative. This is particularly useful when comparing or averaging models, visualizing learning dynamics, or interpreting individual neurons. When units are nearly tied, an alternative to a brittle scalar sort is to *align* the units to those of a fixed reference model by solving a matching problem (a linear assignment), as done in model-merging methods.

32.4.3 Combining scaling and permutation representatives

To select a joint representative for both symmetries, we first fix the scale by normalising the hidden-unit rows, and then sort the units by the scaling-invariant criterion above. Because the sort key is chosen to be invariant under the scaling that we have already fixed, the order of the two operations is well defined and the composition yields a unique, well-defined representative of the parameter orbit for a given network function. This is the canonicalisation corresponding to the joint group of permutations and positive scalings. For deeper networks, the scale-fixing step generalises to multi-layer *equi-normalisation*, which balances the scales across all layers [1564].

32.5 Mode connectivity

The representatives of the previous section are useful precisely because the loss landscape is shaped by symmetries. Two networks that are functionally equivalent, or merely similar, may sit far apart in parameter space simply because their hidden units are permuted or rescaled. *Mode connectivity* studies whether two trained solutions can be joined by a path of low loss, and the symmetry viewpoint explains much of what is observed: aligning one network to another modulo permutation symmetry often turns an apparent high-loss interpolation into a near-flat one. Beyond the linear case considered here, Draxler et al. [451] showed that independently trained minima are typically connected by *nonlinear* low-loss curves, and Brea et al. [204] analysed how permutation symmetries organize the minima into connected structures. We make these notions precise and then revisit alignment.

Let $\Theta \subset \mathbb{R}^d$ be the parameter space of a DNN. A DNN is a function $f_\theta : \mathcal{X} \to \mathcal{Y}$, parameterised by weights $\theta \in \Theta$. Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n \subset \mathcal{X} \times \mathcal{Y}$, the network is trained to minimise the empirical loss

$$\mathcal{L}(\theta) := \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_\theta(x_i), y_i\big),$$

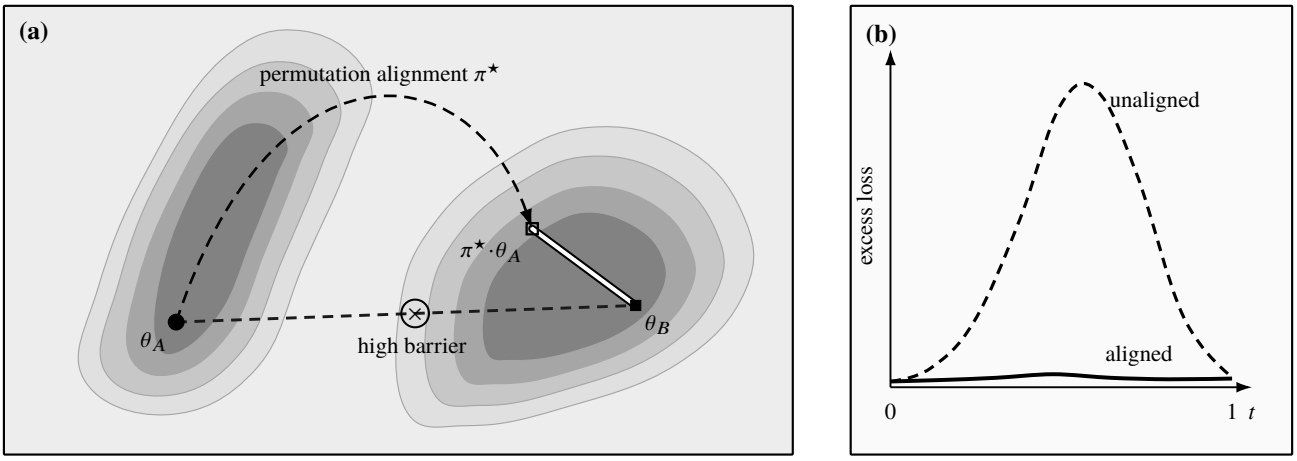


Figure 32.5: **Permutation alignment reveals linear mode connectivity. (a)** Two independently trained solutions θ_A and θ_B lie in different low-loss basins, so their naive linear interpolation (dashed) crosses a high-loss region. A hidden-unit permutation $\pi^\star$ maps θ_A to the functionally equivalent representative $\pi^\star \cdot \theta_A$ in the basin of θ_B; the aligned interpolation (solid) then remains at low loss. **(b)** Schematic excess loss along the two segments: the unaligned interpolation has a pronounced barrier, whereas the aligned interpolation is nearly flat.

where ℓ is a task-dependent loss function (e.g., cross-entropy). We do not assume $\mathcal{L} \geq 0$: for instance a Gaussian negative log-likelihood with small variance can be negative.

Definition 32.13 (Low-loss set)**.** *Let $L_{\min} =\in f_{\theta\in\Theta}\mathcal{L}(\theta)$ and fix a tolerance $\epsilon > 0$. The* low-loss set *is*

$$S_\epsilon := \{\theta \in \Theta \mid \mathcal{L}(\theta) \leq L_{\min} + \epsilon\},$$

the set of parameters whose loss is within ϵ of the best attainable value.

Definition 32.14 (Loss Barrier [493, 554])**.** *Let $\theta_A, \theta_B \in \Theta$. Define the* loss barrier *between θ_A and θ_B as*

$$\mathcal{B}(\theta_A, \theta_B) := \max_{t\in[0,1]} \widetilde{\mathcal{L}}_t(\theta_A, \theta_B),$$

where

$$\widetilde{\mathcal{L}}_t(\theta_A, \theta_B) := \mathcal{L}\big((1-t)\theta_A + t\theta_B\big) - \big((1-t)\mathcal{L}(\theta_A) + t\mathcal{L}(\theta_B)\big).$$

The loss barrier measures the excess loss along the linear interpolation between θ_A and θ_B, relative to the linear interpolation of endpoint losses.

Definition 32.15 (Linearly Mode-Connected Solutions)**.** *Two solutions $\theta_A, \theta_B \in S_\epsilon$ with approximately equal losses are* linearly mode-connected *(LMC) if the linear interpolation between them stays at near-constant loss, i.e., $\mathcal{B}(\theta_A, \theta_B) \approx 0$.*

Symmetry and the loss barrier. It is tempting to state a proposition asserting that two functionally equivalent networks have a zero loss barrier. As written, such a statement is a tautology: if $\mathcal{T}_\star(\theta_A) = \theta_B$ for a function-preserving $\mathcal{T}_\star$, then the linear interpolation between $\mathcal{T}_\star(\theta_A)$ and θ_B is the constant path at θ_B, so the barrier vanishes by construction and nothing has been learned about the landscape. The substantive content is empirical and concerns *distinct* parameter vectors.

Empirical observation (permutation alignment reduces the barrier) [15, 493]. Let θ_A and θ_B be two networks of the same architecture trained *independently* (different random seeds), so that in general $\theta_A \neq \theta_B$ and the naive linear interpolation between them crosses a high-loss region. Entezari et al. conjectured, and Ainsworth et al. gave a practical method (Git Re-Basin) showing, that there is often a permutation $\boldsymbol{\pi}^\star$ of the hidden units such that the *aligned* network $\boldsymbol{\pi}^\star \cdot \theta_A$ and the reference θ_B have a near-zero linear loss barrier:

$$\mathcal{B}\big(\boldsymbol{\pi}^\star \cdot \theta_A,\ \theta_B\big) \approx 0, \qquad \text{while } \boldsymbol{\pi}^\star \cdot \theta_A \neq \theta_B.$$

This is an empirical observation about independently trained solutions: the two aligned parameter vectors remain distinct, yet the path between them stays at low loss in their experiments.

The practical reading is that an *apparent* positive barrier between two good solutions is often, at least in part, an artifact of permutation misalignment in parameter space. Removing that misalignment by aligning the networks modulo permutation symmetry frequently reveals an underlying low-loss connection.

Definition 32.16 (Local Convexity)**.** *Let $f : \mathbb{R}^n \to \mathbb{R}$ be twice continuously differentiable in a neighbourhood of a point $\overline{x} \in \mathbb{R}^n$. We say that f is* locally convex at $\overline{x}$ *if its Hessian matrix*

$$\nabla^2 f(\overline{x}) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j}(\overline{x})\right]_{i,j=1}^{n}$$

is ***positive semidefinite****, that is, $d^\top \nabla^2 f(\overline{x})\, d \;\geq\; 0 \quad$ for all $d \in \mathbb{R}^n$.*

We use positive semi-definiteness *rather than strict positive definiteness. In any model with continuous likelihood symmetries (ReLU positive scaling, BatchNorm channel scalings, and similar), the Hessian has zero eigenvalues in the directions tangent to the symmetry orbit, so strict positive definiteness is unavailable. Positive semidefiniteness keeps the convexity intuition while remaining compatible with these flat symmetry directions.*

Definition 32.17 (Locally convex region)**.** *Let $\mathcal{L} : \Theta \to \mathbb{R}$ be twice continuously differentiable. A set $C \subset \Theta$ is called a* locally convex region *if*

$$d^\top \nabla^2 \mathcal{L}(\theta)\, d \;\geq\; 0 \quad \text{for all } \theta \in C \text{ and all } d \in \mathbb{R}^d,$$

i.e. $\nabla^2 \mathcal{L}(\theta) \succeq 0$ throughout C.

Proposition 32.10 (Convexity inequality along segments inside a locally convex region)**.** *Let C be a locally convex region as defined in Definition 32.17. If $\theta_A, \theta_B \in C$ and the whole line segment*

$$[\theta_A, \theta_B] := \{(1-t)\theta_A + t\theta_B : \; t \in [0,1]\}$$

is contained in C, then for all $t \in [0,1]$,

$$\mathcal{L}\big((1-t)\theta_A + t\theta_B\big) \;\leq\; (1-t)\,\mathcal{L}(\theta_A) + t\,\mathcal{L}(\theta_B) \;\leq\; \max\{\mathcal{L}(\theta_A), \mathcal{L}(\theta_B)\}.$$

In particular, the excess loss $\widetilde{\mathcal{L}}_t(\theta_A, \theta_B)$ is nonpositive for every t, so the linear loss barrier along the segment satisfies $\mathcal{B}(\theta_A, \theta_B) \leq 0$: there is no barrier between θ_A and θ_B along this segment.

Proof. Define the one–dimensional restriction $g(t) := \mathcal{L}\big((1-t)\theta_A + t\theta_B\big)$ for $t \in [0,1]$. Then g is twice differentiable and

$$g''(t) \;=\; (\theta_B - \theta_A)^\top \nabla^2 \mathcal{L}\big((1-t)\theta_A + t\theta_B\big)\,(\theta_B - \theta_A) \;\geq\; 0$$

because the segment lies in C and $\nabla^2 \mathcal{L} \succeq 0$ on C. Hence g is convex on $[0,1]$, so $g(t) \leq (1-t)g(0) + tg(1)$. This yields the first inequality; the second is a basic bound for convex combinations. □

Proposition 32.11 (Weight averaging vs. output averaging, to first order [824])**.** *Let $f_\theta : \mathcal{X} \to \mathbb{R}$ denote a (scalar) twice continuously differentiable network output with respect to parameters $\theta \in \mathbb{R}^d$. Let $\{\theta_i\}_{i=1}^n \subset C$ lie in a region C where a local neural tangent kernel/linearisation holds, i.e., $f_\theta(x)$ is locally affine in θ. Define the Stochastic Weight average (SWA) $\theta_{\mathrm{SWA}} := \dfrac{1}{n}\displaystyle\sum_{i=1}^{n} \theta_i$ and deviations $\Delta_i := \theta_i - \theta_{\mathrm{SWA}}$ (so $\sum_i \Delta_i = 0$). Then, for every $x \in \mathcal{X}$,*

$$f_{\theta_{\mathrm{SWA}}}(x) \;=\; \frac{1}{n}\sum_{i=1}^{n} f_{\theta_i}(x) \;+\; O(\Delta^2), \qquad \text{where } \Delta := \max_i \|\Delta_i\|.$$

Proof. By first-order Taylor expansion at θ_{SWA},

$$f_{\theta_i}(x) = f_{\theta_{\mathrm{SWA}}}(x) + \langle \nabla_\theta f_{\theta_{\mathrm{SWA}}}(x),\, \Delta_i \rangle + O(\|\Delta_i\|^2).$$

Averaging over i gives

$$\frac{1}{n}\sum_{i=1}^{n} f_{\theta_i}(x) = f_{\theta_{\mathrm{SWA}}}(x) + \Big\langle \nabla_\theta f_{\theta_{\mathrm{SWA}}}(x),\, \frac{1}{n}\sum_{i=1}^{n} \Delta_i \Big\rangle + O(\Delta^2) = f_{\theta_{\mathrm{SWA}}}(x) + O(\Delta^2),$$

since $\sum_i \Delta_i = 0$. If f_θ is globally affine in θ, all second-order terms vanish and the equality is exact. □

Remark (scope of the result). Proposition 32.11 is a *local, first-order* statement about a scalar network output $f_\theta(x)$. It says that, near a common point and to first order, averaging the weights approximates averaging the outputs. It should not be read as an unconditional equivalence between SWA and a deep ensemble. In classification, for example, the predictive distribution applies a softmax to the logits, and averaging logits (which is what a first-order argument on the pre-softmax output gives) is *not* the same as averaging the predicted probabilities of the ensemble members. The two coincide only when the second-order terms are negligible and the read-out is linear; in general the ensemble of probabilities and the single averaged-weight model differ.

Intuitively, the set of weights $\{\theta_i\}_i$ used for weight averaging are sampled along a single training trajectory within a connected basin, where the loss surface is approximately flat or convex. Along such paths, the loss barrier between endpoints is essentially zero. By proposition 32.10, this implies that the loss of the averaged model θ_{SWA} is no worse (and typically better) than the average loss of the individual models with weights $\{\theta_i\}_i$.

This idea was first exploited in *Fast Geometric Ensembling* (FGE) [581], then formalised in *Stochastic Weight Averaging* (SWA) [824], and extended in *SWAG* [1116] and *Tracking of Deep neural network weight Distributions* (TRADI) [553], which leverage weight averaging as a cheap alternative to variational inference for posterior approximation. More recently, *Model Soup* [1776] and *Model Ratatouille* [1374] average weights from models sharing the same initialisation but trained under different fine-tuning strategies. Their approach ensures that the weights $\{\theta_i\}_i$ lie in regions connected by low-loss paths, thus maintaining small loss barriers across models.

Beyond averaging along a single trajectory, methods such as *Git Re-Basin* [15], *ZipIt* [1566], and *Starlight* [1544] merge *independently* trained models. The key step is to align the models modulo permutation symmetries before averaging or merging: by matching the hidden units of one network to those of the other, the two solutions are brought into a common, comparable parametrisation in which their interpolation has a small loss barrier, which is what makes the combination effective.

32.6 Conclusion

Symmetries are a fundamental aspect of neural network parameterisations, arising whenever distinct parameter configurations represent the same predictive function. Throughout this chapter, we have shown that these symmetries naturally emerge from common architectural components, including hidden-unit permutations, positive scaling transformations in homogeneous networks, normalisation layers, and multi-head attention. Viewing these transformations through the lens of group actions provides a unified framework for understanding the geometry of neural network parameter spaces and the resulting equivalence classes of models.

From a Bayesian perspective, these symmetries have important implications for posterior inference. Function-preserving transformations always induce invariances of the likelihood, whereas posterior invariance additionally depends on the prior respecting the same symmetry. Consequently, parameter symmetries can lead to multiple equivalent posterior modes or continuous low-probability valleys in weight space, complicating posterior exploration, variational approximations, and model comparison. Recognizing these structures is therefore essential for interpreting Bayesian uncertainty in parameter space and for designing inference algorithms that efficiently explore the posterior distribution.

We also discussed practical strategies for handling symmetries, including canonical representatives, symmetry-aware alignment, and parameter transformations prior to averaging or merging models. Rather than eliminating symmetries, these approaches operate on equivalence classes of parameters, allowing optimisation and inference procedures to avoid redundant exploration of identical solutions. This perspective also explains recent advances in mode connectivity, Git Re-Basin, and weight-space model merging, where aligning networks modulo their symmetries reveals unexpectedly simple geometric relationships between independently trained solutions.

More broadly, understanding neural network symmetries bridges abstract mathematical concepts such as group actions and quotient spaces with practical challenges in Bayesian deep learning. As models continue to grow in size and architectural complexity, symmetry-aware optimisation and inference are likely to become increasingly important, both for improving computational efficiency and for developing a more principled understanding of posterior geometry in overparameterised neural networks.

Chapter 33

Symmetries and sampling

The implications of weight-space symmetries in Bayesian neural networks on sampling-based inference are not straightforward to assess. This chapter discusses the influencing factors for how benign or adverse the posterior density presents itself to sampling, identifying a trade-off between symmetric structures and the degree of regularisation. Besides affecting the design of the sampling procedure, symmetries can result in misleading convergence diagnostics.

33.1 Posterior Landscape in the Presence of Symmetries

33.1.1 Posterior Landscape

Assessing the difficulty to be expected for a sampler traversing the posterior state space requires the consideration of both *topological* and *geometrical* aspects. The former pertains to the number and relative location of *modes*, while the latter concerns questions of *curvature*. Together, these properties characterise the posterior *landscape*. Behind this lies a consideration of posterior *mass* (volume times density): Bayesian model averaging integrates over parameter values that could have generated the data. Both the number of such parameters with non-zero posterior density (volume) and the degree of support by the data under the given prior (density) govern the Bayesian model average. The *typical set* is the set of parameter states with high posterior mass whose joint density comes close to the total expected density of the parameter distribution. More formally, assume $z_1, \ldots, z_N$ are i.i.d. samples from a distribution p on a measurable space. The ϵ-typical set $\mathcal{A}_\epsilon^N$ of the distribution p is given by the set of all length-N sequences whose joint (negative log) density is ϵ-close to the (Shannon) entropy of the distribution:

$$\mathcal{A}_\epsilon^N = \left\{ (z_1, \ldots, z_N) : \left| -\frac{1}{N} \log p(z_1, \ldots, z_N) - \mathrm{H}(p) \right| \leq \epsilon \right\} \tag{33.1}$$

together with

$$\mathrm{H} = \mathbb{E}_{Z \sim p} \left[-\log p(Z) \right].$$

By the asymptotic equipartition property, the typical set $\mathcal{A}_\epsilon^N$ has probability approaching one as $N \to \infty$ [337, Ch. 3]. Therefore, sampling it thoroughly is crucial for a faithful assessment of predictive uncertainty: failing to capture the full expanse of models that could have plausibly generated the observed data will leave us overconfident in our predictions. Parameter states outside the typical set, on the other hand, are negligible for inference.

The ease of covering the typical set depends on the structure of the posterior landscape. More complexity in the hypersurface implies a larger number of samples required for accurate inference. Intuitively, the need to visit a large number of modes and collect enough samples from their surrounding area strains the available computational budget. This is especially true when the modes are located far away from each other with areas of low density in between. Such dispersion, as will be discussed shortly, can occur as a consequence of weight-space symmetries, and becomes increasingly likely in higher-dimensional spaces. Samplers will have difficulties crossing the unattractive areas separating modes.

The intractability and high-dimensional nature of neural network posteriors make their analysis, be it visual or mathematical, difficult. We can still approach the problem systematically by recognizing that Bayesian inference bears a connection to empirical risk minimisation when framing the negative log-likelihood (NLL) as a loss function. The widely used *cross-entropy* loss in classification and $L2$ loss in regression are such NLL-type objectives. Then, following from Bayes' theorem, the (negative log) posterior landscape is a prior-adjusted version of the *loss*

landscape, up to a normalising constant corresponding to the marginal log-likelihood:

$$-\log \underbrace{p(\theta|\mathcal{D})}_{\text{posterior}} = \underbrace{\hat{\mathcal{L}}(\theta, \mathcal{D})}_{\text{loss}} - \log \underbrace{p_0(\theta)}_{\text{prior}} + \text{const.} \tag{33.2}$$

We will subsequently analyse the respective impact of the loss (or likelihood) and prior component on the posterior landscape.

33.1.2 Impact of Symmetries

Assume for a moment a uniform parameter prior (we will add prior influence to our considerations in an instant). The shape of the resulting posterior density is solely determined by the likelihood in accordance with Eq. (33.2). There are two reasons why the loss landscape can turn out multi-modal, i.e., have more than one local optimum.

1. The observed data support multiple, *functionally diverse* hypotheses. Functional diversity in a given pair of hypotheses $f(\cdot;\theta), f(\cdot;\tilde{\theta})$, $\theta, \tilde{\theta} \in \Theta$, means that we have $f(x;\theta) \neq f(x;\tilde{\theta})$ for a general input $x \in \mathcal{X}$. Although there is only one true model, the posterior lends support to multiple candidates. This kind of ambiguity is to be expected in finite-data settings where the observed data do not suffice to identify the model that has generated them.

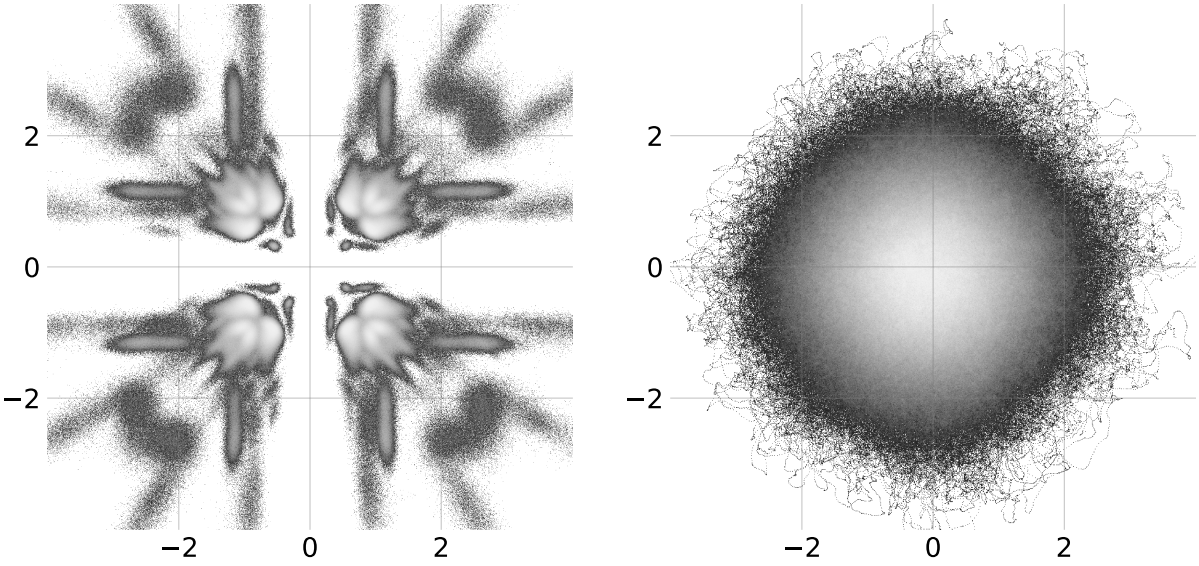
2. Various, *functionally equivalent* parameter states $\theta, \tilde{\theta}$ give rise to the *same* functional mapping, assigning identical network outputs to a given input x, i.e., $f(x;\theta) = f(x;\tilde{\theta})$. This happens precisely in the case of weight-space symmetries as discussed in the previous chapter.

Permutation symmetries are the major driver of symmetry-induced multi-modality. Combinatorially, many equal-loss replicates exist for every functionally diverse mode simply by suitable permutation of weights. Since any two given permutations of a parameter vector can be separated by a large distance in the parameter space, with valleys of low density in between, (naive) MCMC samplers will accrue exponentially long mixing times. Note that such posterior multi-modality occurs not only for our hypothetical uniform prior, but also for the more realistic case of a Gaussian prior with diagonal, isotropic covariance structure. Those priors are widely popular for Bayesian neural networks, usually with a zero mean to encourage parameter sparsity. Multivariate Gaussians with diagonal covariance fully factorise. Together with isotropy, this implies invariance to parameter permutations, as all marginal prior densities are identical. To see this, let $\boldsymbol{\pi} \in \mathbb{R}^{d\times d}$ be a permutation matrix with $\boldsymbol{\pi}^\top \boldsymbol{\pi} = I_d$. Consider a normal random variable $Z \sim \mathcal{N}(\mathbf{0}, \Sigma)$, $\Sigma = \sigma^2 I_d$. Then, the random variable $Z' = \boldsymbol{\pi} Z$ of the permuted inputs retains the original covariance due to the orthogonality of $\boldsymbol{\pi}$: $\boldsymbol{\pi}\Sigma\boldsymbol{\pi}^\top = \sigma^2 \boldsymbol{\pi} I_d \boldsymbol{\pi}^\top = \sigma^2 I_d$. The immediate consequence of this phenomenon is the *non-identifiability* of the Bayesian neural network models. From a theoretical perspective, this induces parameter non-identifiability despite a well-defined posterior over functions [904]. Practically speaking, permutation symmetries threaten to make inference vastly more complex and expensive. Since permutation transformations can create symmetric copies in far-flung areas of the parameter space, the resulting modes can be widely dispersed and disconnected.

The prevailing *ReLU* activation $\text{ReLU}(x) = \max(0, x)$ in today's architecture creates a different, continuous type of symmetry (see Chapter 32). It results in level sets of constant-loss parameters that are *connected*. This way, parameter symmetry contributes to the phenomenon of *mode connectivity* [15, 493], describing locally optimal solutions in the weight space linked by paths of near-constant, low loss. Most parameters along such a path produce well-performing models.

ReLU-induced scaling symmetries result in connected areas of constant loss by design. Positive homogeneity of degree one implies the familiar rescaling symmetry, that is, for any hidden layer with ReLU activations, scaling the incoming weights and biases by $c > 0$ and the outgoing weights by c^{-1} leaves the network function invariant. If a hidden unit has pre-activation $a = W^\top x + b$ and outgoing weight vector v, then $(W, b, v) \mapsto (cW, cb, c^{-1}v)$ preserves $f(\cdot;\theta)$ for all inputs. This creates connected level sets of equal loss. A simple two-parameter illustration is the bilinear map $g(\theta_1, \theta_2) = \theta_1\theta_2$, for which $\{(\theta_1, \theta_2) : \theta_1\theta_2 = \kappa\}$ is a one-dimensional manifold of equal outputs for any fixed $\kappa \neq 0$, while $\{(\theta_1, \theta_2) : \theta_1 = 0\} \cup \{(\theta_1, \theta_2) : \theta_2 = 0\}$ forms a connected locus of degenerate solutions. Such symmetries can turn isolated optima into extended ridges in parameter space. Beyond permutation and rescaling symmetries, overparameterisation can induce continuous non-identifiability manifolds through redundant neuron splitting; for one-hidden-layer ReLU networks, both the represented function and the quadratic Gaussian-prior penalty remain unchanged along these manifolds, so that parameter uncertainty can persist even when the underlying function is fully identified [908, 1439].

Empirical evidence suggests that, for sufficiently large models, there is a sizeable manifold of good solutions that is connected save for permutation symmetries [15, 1517]. We thus have two forces potentially acting on



(a) Underparameterised

(b) Overparameterised

Figure 33.1: Eight million posterior samples for two different networks. The plots show the bivariate marginal weight distribution of two randomly picked weights from the same hidden layer. The left panel corresponds to a single hidden layer network with three neurons. The right panel corresponds to a three hidden layer network with sixteen units per layer. In the underparameterised setting, a clear symmetric structure is visible, which poses a challenging sampling problem. In the overparameterised setting, the strong influence of the chosen standard Gaussian prior yields almost perfect marginal prior conformity. The figure is adapted from [908] and sampling is performed using MILE [1535]

the degree of multi-modality in the loss landscape: permutation symmetries creating disconnected local optima, and rescaling symmetries leading to connected constant-loss volumes. In contrast to permutation symmetries, non-uniform priors that favour certain parameters over others may break *rescaling* symmetries in the posterior landscape. Zero-mean Gaussians in particular prefer parameter vectors with small norm and assign progressively less weight to parameters of larger magnitude.

33.1.3 Prior Influence

Eq. (33.2) shows that the posterior density is controlled by a combination of the loss (or likelihood) and the prior. Symmetries, depending on their type and extent, can introduce multi-modality in the loss landscape. The commonly used prior densities (especially of the Gaussian sort), on the other hand, are convex and uni-modal. The relative influence of prior and likelihood depends on the degree of *overparameterisation* and on the informativeness of the data. Increasing the number of parameters for a fixed dataset often increases the relative influence of the prior, because the likelihood becomes more diffuse across directions unconstrained by the data [904, 908, 1439].

Fig. 33.1 illustrates the effect of prior influence. The left scenario belongs to the underparameterised regime and displays strong symmetric patterns in the posterior density. The overparameterised case on the right shows an entirely different picture, with the marginal posterior density closely matching the chosen isotropic standard Gaussian prior. It is easy to imagine that the left-hand scenario poses more of a challenge to sampling-based inference. In this situation, the posterior hypersurface is much less convex and connected, yet highly structured by visible symmetries.

The interplay between the prior and the data in the light of symmetries and overparameterisation affects the posterior in non-trivial ways. Paradoxically, *larger* models might be more amenable to sampling-based inference because the prior—at least, for the popular Gaussian variant or similarly smooth alternatives—has a softening effect on the posterior landscape [908]. Overparameterisation seems to be similarly beneficial to the curvature by promoting flatter optima [1328]. In practice, prior choice and model selection remain the key design challenges of the modeling process and need to be evaluated carefully with measures of calibration and predictive performance.

33.2 Sampling Strategies

How can the insights about the posterior landscape be used to devise good sampling strategies? The functional equivalence of symmetric parameter states begs the question of whether samplers actually need to visit *all* the modes scattered through the parameter space. Early works in the 1990s that first pointed out the existence of functionally equivalent parameter states in the loss landscape implied the existence of *minimally sufficient search sets* [265, 724]. These sets contain representatives from each functionally diverse parameterisation, of which all other states are mere symmetric replicates.

In theory, confining inference to the sufficient sets and re-scaling posterior dispersion appropriately should serve to cover all functionally relevant parameters. For discrete symmetries (in particular, those caused by parameter permutations), this can be translated to the following, modified version of the posterior predictive density [1748]:

$$\begin{aligned} p(y_*|x_*, \mathcal{D}) &= \int_\Theta p(y_*|x_*, \theta) p(\theta|\mathcal{D}) \, \mathrm{d}\theta \\ &\approx |\mathbb{T}| \int_\mathcal{S} p(y_*|x_*, \theta) p(\theta|\mathcal{D}) \, \mathrm{d}\theta, \end{aligned} \tag{33.3}$$

assuming a finite $\mathbb{T}$-invariant posterior with $\mathbb{T}$ the set of output-preserving transformations,

$$\mathbb{T} = \{\mathcal{T} : \Theta \to \Theta \mid f(x;\theta) = f(x;\mathcal{T}(\theta)) \quad \forall x \in \mathcal{X}\}, \tag{33.4}$$

and a disjoint tiling of Θ by the orbits (see Def. 32.7 in the previous chapter) of $\mathbb{T}$ up to null sets. It should be noted that, for general symmetries, the transformations $\mathbb{T}_\theta$ that preserve the network output depend on θ. Lastly, $\mathcal{S}$ denotes the minimally sufficient search set with $|\mathcal{S}| \ll |\Theta|$. Defining $\mathcal{S}$ explicitly can be difficult in practice. More importantly, there is no straightforward method to restrict sampling to this minimally sufficient search set. That said, several approaches exist to sample some—small and accessible—subspace of the entire parameter space (see 33.2.2).

33.2.1 Symmetry Removal

Eq. (33.3) and the concept of minimally-sufficient search sets suggest that the *removal* of symmetries might facilitate the inference problem on a great scale—with the number of permutations increasing exponentially in higher dimensions, the amount of symmetric copies usually exceeds the number of functionally diverse solutions by far. Similar considerations follow from the insights on mode connectivity, suggesting that breaking permutation symmetries will leave us with a connected, easy-to-traverse hypersurface. Several ideas exist to remove symmetries [1328, 1564, 1748, see also Chapter 32]. As symmetries are vastly complex and hard to characterise for arbitrary architectures, especially when overlapping in their effects, it is challenging to design general-purpose algorithms to remove them fully.

33.2.2 Subspace Inference

A parameter space devoid of symmetries constitutes a *subspace* of the original. While recovering the symmetry-free subspace is hard, other ideas exist to find subspaces amenable to sampling (typically of lower dimension; see also Chapters 39, 48). Subspace inference can replace the full procedure if the subspace is chosen in a meaningful way and all functional diversity is captured. This can be achieved, for example, by finding the most important parameters as indicated by the variability along the respective dimension in a loss-based optimisation procedure [1116]. As an alternative, we can use low-loss tunnels resulting from mode connectivity approaches, which are typically low-dimensional by design [436, 825].

33.2.3 Multi-Start Sampling

Other approaches propose to sample the original weight space but in a clever way. Most practical problems have posteriors with multiple basins of attraction. A simple yet effective approach is then to run several samplers in parallel [1534, 1535, 1765]. Even when choosing the starting locations for the chains at random, this can potentially improve performance as a single-chain sampler can get stuck or start exploring irrelevant areas of the parameter space. Such a multi-start sampling approach can be further improved by smartly choosing the initial chain locations. Successful approaches use, for instance, deep ensembles to warm-start sampling-based inference. For a detailed discussion of such hybrid inference methods, see Chapter 20. Other than choosing favourable starting locations, we can incentivise samplers to explore the search space more effectively by enforcing restarts during the

sampling stage. *Cyclical MCMC* [1854, Chapter 2] does so by scheduling the learning rate to alternate between exploration (large moves to cross the space in search of different modes) and exploitation (small steps to cover the local area around a single mode). A similar idea exists in non-Bayesian deep learning via so-called *symmetry teleportation*, allowing the optimiser to jump towards solutions with identical loss but steeper gradient by means of output-preserving parameter transformations [1864].

Generally speaking, the impact of parameter symmetries on sampling-based inference varies across different practical situations—underparameterised problems may exhibit pronounced symmetric patterns, while overparameterised models lean towards smoother, more prior-dominated posteriors. The need and benefit of accounting for symmetries in the inference procedure thus depends strongly on the problem at hand.

33.3 Sampling diagnostics in the presence of symmetries

Assessing the convergence of MCMC samplers is a cornerstone of the practical Bayesian workflow and has received renewed interest in the light of modern sampling applications [964, 1131, 1195]. We aim to run the sampler long enough for it to reach its stationary distribution, so that the resulting samples can be used to approximate the posterior reliably. However, the presence of weight-space symmetries in Bayesian neural networks violates the assumptions underpinning classical MCMC diagnostics, which can lead to misleading conclusions about convergence. This section examines why these standard measures often fail and discusses the challenges of diagnosing convergence in non-identifiable models.

33.3.1 Diagnostics in Parameter Space

The two most common diagnostics for MCMC convergence are the potential scale reduction factor, $\widehat{R}$, and the effective sample size (ESS), both of which have been introduced in Chapter 1. However, $\widehat{R}$ and ESS have been designed for models with identifiable parameters, a condition that does not hold for BNNs.

The Potential Scale Reduction Factor ($\widehat{R}$)

The $\widehat{R}$ diagnostic is designed to determine whether multiple, parallel MCMC chains have converged to the same distribution. To this end, the diagnostic compares the variance of samples *between* the chains and the variance *within* each chain. More formally, for K chains of length S, the statistic is based on the between-chain variance (B) and the within-chain variance (W):

$$B = \frac{S}{K-1}\sum_{k=1}^{K}\left(\bar{\theta}^{(k,\cdot)} - \bar{\theta}^{(\cdot,\cdot)}\right)^2, \quad W = \frac{1}{K}\sum_{k=1}^{K}\mathbb{V}(\theta^{(k,\cdot)}).$$

$\bar{\theta}^{(k,\cdot)}$ denotes the sample mean of the k-th chain, and $\bar{\theta}^{(\cdot,\cdot)}$ is the overall mean of samples across chains. The potential scale reduction factor is then given by:

$$\widehat{R} = \sqrt{\frac{\widehat{\mathbb{V}}^{+}(\theta|\mathcal{D})}{W}} := \sqrt{\frac{\frac{S-1}{S}W + \frac{1}{S}B}{W}}. \tag{33.5}$$

Here, $\widehat{\mathbb{V}}^{+}(\theta|\mathcal{D})$ is an estimate of the marginal posterior variance. The '+' in $\widehat{\mathbb{V}}^{+}$ signifies that it is an intentionally overestimated (and thus more conservative) estimate, created by combining the within-chain (W) and between-chain (B) variances. If the chains have not fully explored the posterior, W may be an underestimate of the true variance, and this combined measure helps correct for that.

For greater robustness, modern implementations compute this statistic on modified samples [1672]. First, the calculation is performed on folded chains, meaning each chain is split into subchains. This effectively increases the number of chains considered in the calculation and allows the diagnostic to better detect non-stationarity (i.e., if a chain's first and second halves explore different regions). Second, the statistic is computed on rank-normalised samples to further increase robustness.

A value of $\widehat{R}$ close to 1.0 (e.g., Vehtari et al. [1672] suggest a threshold of < 1.01) is taken to indicate convergence, as it implies that the between-chain variance (B) is negligible compared to the within-chain variance (W).

The rationale of $\widehat{R}$ breaks down in the presence of symmetries. Consider that we are using two chains to sample from a BNN posterior. Due to different random initialisations, they will likely explore different but potentially

functionally equivalent modes. For instance, one chain might explore a mode centred around a parameter θ_A, while the second explores a symmetric mode centred at θ_B. Even if both chains have perfectly converged locally and are stationary within their respective modes (thus also functionally), the parameter means $\bar{\theta}_A$ and $\bar{\theta}_B$ will be different. This results in a large and persistent between-chain variance B, causing $\widehat{R}$ to remain bounded away from one even after very long runs. The diagnostic will therefore falsely signal non-convergence, when in reality, the chains have successfully identified the local posterior geometry. Consequently, applying the standard multi-chain $\widehat{R}$ to the parameters of a BNN is inappropriate.

Effective Sample Size (ESS)

The effective sample size (ESS) estimates the number of independent samples that are information-theoretically equivalent to the autocorrelated samples produced by an MCMC chain. With $K \cdot S$ total samples, possibly gathered from K parallel-running chains of length S, it can be defined as:

$$\text{ESS} = \frac{KS}{1 + 2\sum_{t=1}^{\infty} \rho_t}, \tag{33.6}$$

where ρ_t is the autocorrelation of the samples at lag t. Most practical implementations use Geyer corrections to stabilise the ESS computation, as the estimate can become noisy for high t (only few samples are separated by such large lags and can contribute to the autocorrelation estimation). In sampling, high autocorrelation is undesirable and leads to a low ESS, suggesting inefficient exploration.

In BNNs, symmetries create continuous manifolds, or "ridges", in the posterior landscape where the probability density is constant. A sampler moving along such a ridge may exhibit random-walk behaviour, which induces high autocorrelation in the parameter traces. This can result in very low ESS values even if the sampler is efficiently traversing the functionally important parts of the space. Therefore, like $\widehat{R}$, a low ESS can have various causes: it may indicate that the sampler has not yet converged, or a genuinely "stuck" sampler unable to escape from a local basin, or simply reflect the sampler's movement along a symmetric manifold.

33.3.2 Diagnostics in Function Space

Given the aforementioned issues in parameter space, one might hope that assessing convergence in function space—by analysing the model's outputs—would be more meaningful. The general idea is to apply diagnostics not to the parameters θ, but to some function of them, such as the predictions on a held-out dataset or a performance metric like the log-likelihood. However, this approach faces a similar, albeit more subtle, conceptual problem. While symmetries create functionally identical modes, BNN posteriors are also known to contain *functionally distinct* modes [1748]. It is common for different MCMC chains to stabilise at different but locally consistent performance levels [1534]. While these solutions all have high posterior probability, they produce functionally different, high-performance models. If we compute a functional $\widehat{R}$ on the predictions of these chains, we again penalise between-chain disagreement. A high $\widehat{R}$ value might simply indicate that the chains have successfully discovered different, competing functional modes. From a Bayesian perspective, capturing this functional diversity is often a primary goal of the analysis, as it reflects genuine model uncertainty. A diagnostic that punishes the discovery of good but different solutions therefore creates the wrong incentive; it mistakes successful exploration of a complex function space as a failure and rewards sampling redundant modes. This issue persists for both pointwise predictions and summary statistics like the root mean squared error or the log pointwise likelihood.

33.3.3 Towards More Meaningful Diagnostics

The failure of classical diagnostics necessitates a shift in perspective. Rather than asking "Have all chains converged to the same answer?", a more appropriate set of questions for BNNs would be:

1. Has each individual chain reached a stationary state?
2. How diverse is the ensemble of solutions found by the different chains?

To address the first question, diagnostics can be adapted to operate on a **chain-wise** basis. For example, one can split a single chain into multiple segments and compute a "chain-wise" $\widehat{cR}$ to check for internal stationarity as proposed in Sommer et al. [1534] and closely connected to the rationale presented by Vehtari et al. [1672]. For the second question, one can monitor the evolution of a chain-wise performance metric, such as an expanding-window estimate of the log posterior predictive density. This allows checking whether each chain has stabilised in terms of its predictive performance and quantifying the disagreement between the final performance levels of the chains.

It is crucial to recognise, however, that these adapted methods are not fail-safe. A chain can appear stationary while being stuck in a small, suboptimal region of a posterior mode. At present, developing robust, scalable, and theoretically sound diagnostics for non-identifiable models remains an open and critical research challenge in the field of Bayesian deep learning.

Chapter 34

Learning inductive bias and symmetries

Backpropagation is the cornerstone of current deep learning practice. If you specify a neural network architecture and a dataset to train on, backpropagation is used to find weights that make good predictions. However, designing an architecture is currently still a laborious process, which relies on manual design, trial-and-error, and crossvalidation. In this chapter, we investigate how Bayesian inference can help automate this procedure, with the goal of having adaptable architectures that can be learned as easily as the weights.

Having such a training procedure would give numerous advantages. The most straightforward is that, when faced with a new dataset, training would need less human design and supervision. A fully automatic method would mean that practitioners would no longer need to worry about selecting the hyperparameters that control the architecture, such as the number of layers, widths of layers, pooling structures etc. Indeed, making learning automatic is one of the main points of machine learning in the first place! Such procedures could also be more computationally efficient, as current model adaptation requires many iterations of training from scratch. There could be significant computational savings if these could be combined in a single training run, even if this one run is more expensive than a single run on a fixed architecture.[1]

We formulate our goal in terms of **three desiderata**:

1. We want our procedure to work well over as wide a range of tasks and settings as possible (i.e. reliably), and with as little built-in prior knowledge and human input as possible (i.e. autonomously).

2. We want learning of hyperparameters (e.g. controlling architecture) to be as easy as learning the weights. This means that only a single training dataset should be needed (rather than train/validation splits), and that a single procedure should determine both the weights and hyperparameters (e.g. unified training objective).

3. We want the generalisation performance of our resulting model to be as good as possible. Crossvalidation is a good baseline that we do not want to sacrifice too much performance from.

Currently, there is no method that perfectly fulfils all these desiderata. However, these desiderata do help assess methods, and guide research towards improvements.

A learning procedure for neural network architecture, like all learning procedures, will need to consist of two ingredients: **(i)** a space of candidate solutions, and **(ii)** a procedure for selecting within this space. To make ingredient **(i)** manageable in practice, we will take inspiration from Geometric Deep Learning [210, 318, 914, 1735], which studies how to create architectures with vastly different inductive biases by enforcing symmetries in the functions that neural networks encode. Based on these ideas, we will parameterise architectures that interpolate between different symmetries. This allows us to build architectures that have the ability to move between different architectures during training, where they would normally need be manually designed and fixed.[2] For ingredient **(ii)**, we will show how this is a *model selection* problem that Bayesian hierarchical inference can help with. Bayesian reasoning directly provides a pathway to selecting a good architecture, that is much more robust than the normal training loss, and works towards satisfying our desiderata more than crossvalidation.

Put together, we obtain a single architecture that can automatically adapt its inductive bias to the dataset that it is trained on. We show that if the *same* adaptive method is run on datasets with different characteristics,

[1] A reasonable question within the current paradigm of scaling laws, is whether such tuning is necessary. The strategy of picking a single architecture and scaling it is currently leading to continuous improvements. At the end of this chapter we will acknowledge these trends, and motivate future research directions.

[2] Our space of architectures does not include all symmetries or architectures, so the lofty goal from recognising symmetries or building architectures completely from scratch is still open. Parameterising a space of architectures that is wide enough to include symmetries, but is not too big to make learning infeasible is still an open research problem, as we will discuss later.

the architecture will adapt its inductive bias to respect the symmetry that is present in the dataset. Crucially, it does this using only a training dataset, without any nested training procedures, in a single run of training, and so making progress on obtaining a method that satisfies our three desiderata.

In the course of this chapter, we will:

- formalise our problem and review why the regular training loss is not used to learn architecture (Section 34.1),
- discuss other approaches to adapting inductive bias in DNNs, paying special attention to Neural Architecture Search, and assess to what degree these frameworks can satisfy our goals,
- review Bayesian model selection, how it has been proposed in the past to learn inductive bias, and its possibilities if it could be applied to neural networks,
- explain a line of work that uses (approximate) Bayesian model selection to automatically adapt neural network architecture, and finally
- discuss the limitations of Bayesian model selection, and the relevance of adaptive architectures to the current trend of large, scalable models.

Throughout this chapter, one question will recur: Does the Bayesian approach offer benefits beyond what can be achieved with point estimates? This question is important, since developing Bayesian methods adds computational cost and complication to a deep learning system[3], and the research endeavour is only worth it if sufficient benefits can be obtained. Whether this is possible in practical deep learning settings is an open research question.

This chapter covers the current evidence for answering this question from two perspectives: **1)** what Bayesian inference promises to deliver *if the requisite quantities can be computed*, and **2)** what is achieved by currently implementable methods. The first perspective has long been discussed in work on Bayesian model selection, hierarchical Bayesian inference, and Occam's razor. We discuss the ongoing debate, and provide our view is that the framework that was set up in the past is mostly correct, and that the greatest challenges lie in actually calculating the necessary quantities. To provide evidence from the second perspective, we discuss a line of work [812, 815, 1658, 1660–1662] that approximates what Bayesian model selection prescribes to do, and compare it to alternative approaches.

We, the authors of this chapter, are co-authors of this line of work. We believe that the results are encouraging: Bayesian thinking can lead to new capabilities in deep learning, and the closer we approximate what Bayes prescribes, the better our methods work. However, many more steps are needed to make these methods ready to be routinely applied in deep learning practice. In this chapter, we summarise the existing evidence and, while acknowledging that the research question remains open, argue that this direction warrants further exploration, with the hope of inspiring others to join the investigation.

34.1 The Problem Setting

To begin, let's review why architecture selection is currently performed. Before investigating any methods, it is helpful to first review how the problem is currently approached, to highlight challenges that any potential method will need to deal with.

Regardless of whether we are solving toy 1D regression examples, classifying MNIST or ImageNet, or predicting the next token in an LLM, neural networks solve the same problem: function estimation. From a dataset of input-output pairs $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ (with $x_n \in \mathcal{X}, y_n \in \mathcal{Y}$), we aim to find a function $f : \mathcal{X} \to \mathcal{Y}$ that makes good predictions at new inputs.[4] "Good" is typically defined by achieving a low loss on future data, where future data can arrive i.i.d. from the same distribution as the training data $\mathcal{D}$, or from some kind of new process, as is the case in the distribution shift or out-of-distribution (OoD) settings.

Deep learning works by first selecting a neural network architecture, which is controlled by various hyperparameters, for example, number of layers, type of layer (e.g. fully connected vs convolutional), connectivity pattern (e.g. stride or window size) width, regularisation, skip connections, etc. The architecture can be seen as a map from the weights to a function from inputs to outputs which we can denote as

$$f^\theta : \mathbb{R}^{D(\theta)} \times \mathcal{X} \to \mathcal{Y}, \tag{34.1}$$

$$f_w^\theta : \quad \mathcal{X} \to \mathcal{Y}, \tag{34.2}$$

[3] In particular, we will see that hierarchical Bayesian inference requires approximating integrals that quantify *volume* of the parameters that fit the training set, rather than just keeping track of a single parameter that fits the training set.

[4] Typically $\mathcal{X} = \mathbb{R}^D$, and for discrete classes/tokens as outputs we have $\mathcal{Y} = \{0, 1\}^C$, while for our visualisations we take $\mathcal{Y} = \mathbb{R}$.

with θ controlling the architecture, and $w \in \mathbb{R}^{D(\theta)}$ being the weights (with dimension potentially depending on the architecture). After specifying an architecture, the weights are chosen by (approximately) minimising some loss w.r.t. the parameters

$$w_\theta^* \approx \arg\min_w \sum_{n=1}^{N} \ell(f_w^\theta(x_n), y_n) \qquad x_n, y_n \in \mathcal{D}\,, \tag{34.3}$$

which is usually performed by running some kind of (stochastic) gradient descent for certain number of iterations. We write w_θ^* to make the dependence of the optimal weights on the hyperparameters θ explicit.

Given that the weights are optimised on the training data, sufficiently flexible architectures can often achieve similar training losses,

$$\sum_{n=1}^{N} \ell\Big(f_{w_\theta^*}^\theta(x_n), y_n\Big) \approx \sum_{n=1}^{N} \ell\Big(f_{w_{\theta'}^*}^{\theta'}(x_n), y_n\Big)\,. \tag{34.4}$$

Particularly for flexible architectures and low-noise datasets, training losses can be near zero, making this very close to equality. However, there are many different functions that go near the training data, but which take on different values for other inputs, as shown in Fig 34.1. *Any* prediction method must choose which of the many functions that minimise the loss to pick. Which function is chosen is determined by the *inductive bias*.[5] In neural networks, the architecture influences the inductive bias, given that it determines how weights map to an entire function (Eq 34.1).

This leads to the question: How should inductive bias (and therefore architecture) be selected? This is the central question of this chapter, and our goal is to find a procedure that makes this selection as easy as selecting the weights.

34.2 Potential Methods

34.2.1 Minimising the Training Loss: Flexible Models Fit Any Data, But Generalise Differently

To start, we can ask why minimising the training loss, which works so well for the weights, is not also used to learn hyperparameters. One could easily perform the minimisation

$$\theta^*, w^* \approx \arg\min_{\theta,w} \sum_{n=1}^{N} \ell(f_w^\theta(x_n), y_n) + n_\theta(w) \qquad x_n, y_n \in \mathcal{D}\,, \tag{34.5}$$

through some (stochastic) gradient descent method, perhaps relying on some additional regularisation term $r_\theta(w)$. If this were possible, adjusting architecture *would* be as easy as learning the weights!

The reasons for why this method does not work in general are often discussed in the earliest stages of an ML course [160, p. 11], and show why validation sets are needed. Nevertheless, in specific cases it has sometimes been used to learn hyperparameters [e.g. 120, 1418, 1657], and there are different mechanisms that cause it to succeed or fail (see Section 34.2.3).

The core reason for not using the training loss to learn hyperparameters is that, inductive biases that help generalisation do not necessarily improve the training loss. The regularisation term $n_\theta(w)$ is an additional loss, which can be removed by reducing the hyperparameter controlling its weight. In addition, architectures with different inductive biases are still flexible enough to fit the training data, so the training loss cannot distinguish them even though they generalise differently.

An example of this is shown in Fig 34.1, which compares the train and test loss of two models with and without a periodic inductive bias, on a dataset with periodicity. Both models were chosen to be overparameterised, in accordance with the regime that deep learning operates in.[6] In addition, the training set was taken to be noise-free,

[5] Jumping ahead a bit, it may seem that Bayesian machine learning avoids choosing a function to pick, by instead selecting a distribution over possible functions. It would be a mistake to conclude that Bayesian methods are therefore less reliant on inductive biases. There are still many distributions over functions that fit the training data, which all predict differently, and the inductive bias determines which of these distributions a Bayesian method picks.

[6] The idea that models should have more parameters than datapoints was considered puzzling in deep learning. Belkin et al. [110] coined the term "double descent", although this behaviour was also noted and explained earlier (e.g. Bös and Opper [189]). Using more parameters than datapoints is no surprise in Bayesian nonparametrics though, e.g. in Gaussian processes where the number of weights is infinite [1380, 1382].

and hyperparameters controlling e.g. the smoothness were set to values that were optimal for generalisation. We see that the periodic model performs much better on the test set, but has an effectively identical training loss as the non-periodic model, which performs much worse on the test set. This demonstrates that the training loss does not correctly distinguish the right model in this situation.

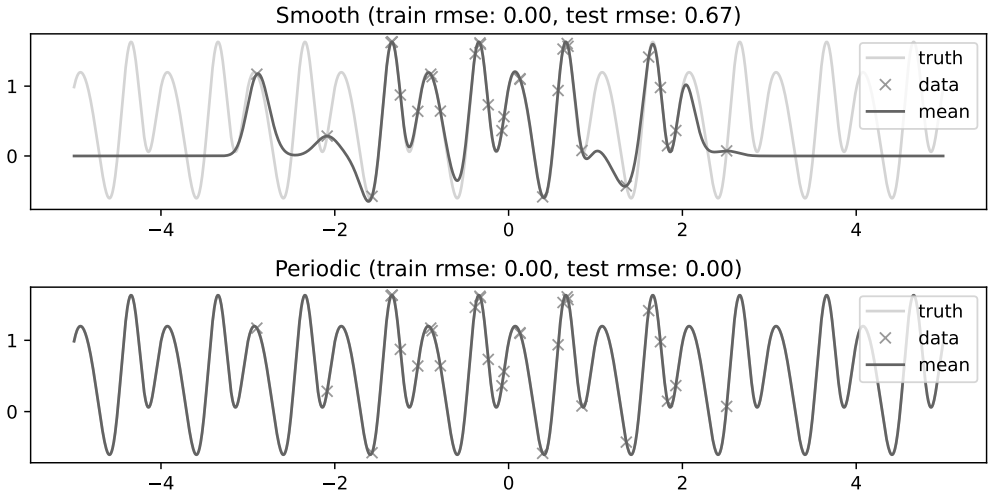


Figure 34.1: Learning a function that has a periodicity in it, from noise-free data. We show two models with different inductive biases: smoothness (top), and smoothness *plus* periodicity (bottom). Both models are overparameterised and cannot be distinguished according to their fit on the training data, but test loss is much lower for the model with the appropriate inductive bias. Both models are single-layer finite basis-function models, with weights minimising the training loss without regularisation, and the hyperparameters (e.g. lengthscale of the smoothness) set to values that lead to optimal test loss. While the training losses are best described as indistinguishable, it is worth noting that the loss for the smooth model ($8.6 \cdot 10^{-6}$) is smaller than that of the periodic model ($2.8 \cdot 10^{-5}$).

This effect is made worse the larger the variation in train-test loss gap between models. Uncontrolled overfitting causes this, and the experimental setting was chosen to minimise this overfitting by selecting good hyperparameters for each model, and using a low-noise dataset. Noise only increases this effect, since noise is very unlikely to fit with inductive biases, leading to the best-fitting model being the most flexible one.

34.2.2 Crossvalidation: The Common Solution

From the first classes in ML, crossvalidation is proposed as a solution to the problems with the training loss. The method is justified by selecting the inductive bias using a direct estimate of the performance on future data, following our original goal. We estimate this "validation loss" for several hyperparameters θ on a "validation" dataset that was unseen during training, and select the inductive bias with the best estimated performance:

$$\theta^* = \arg\min_{\theta} \sum_{n=1}^{N_{\text{test}}} \ell(f^{\theta}_{w^*_\theta}(x_n), y_n)\,, \tag{34.6}$$

noting that w^*_θ is still selected by minimising the training loss according to Eq. 34.3. We perform this minimisation by evaluating the validation loss for several different hyperparameter values, and then selecting the best one. Since the weights depend on the hyperparameter, this requires retraining the weights separately for each hyperparameter value that is considered. This whole procedure is known as (1-fold) crossvalidation, and is effectively trial-and-error.

Provided that the validation set size is large enough to fit the desired number of hyperparameters[7], crossvalidation is highly reliable and near-ubiquitously applied in machine learning. The machine learning community as a whole can also be seen as a large crossvalidation process, with researchers across the world suggesting new architectures, and publishing those that perform better on a test set.

[7]If a small validation set is used to select a large number of hyperparameters, crossvalidation can also end up overfitting [248, 1234]. Other variants of crossvalidation (e.g. K-fold even leave-one-out) can reduce this overfitting [830, §5.1], at the cost of needing to re-train the w^*_θ multiple times. However, the number of hyperparameters is typically small.

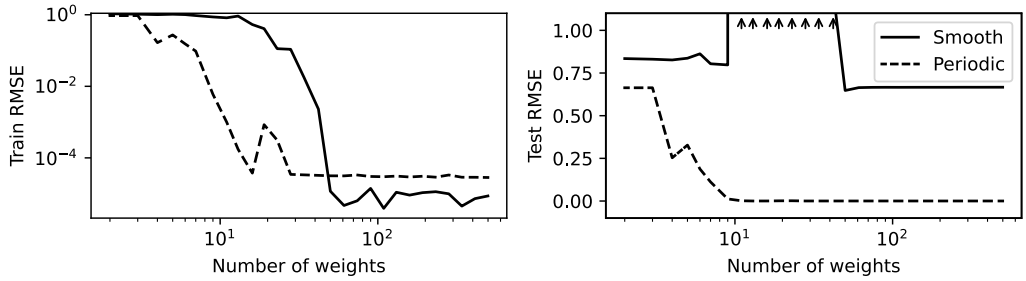


Figure 34.2: Training and test losses for smooth and periodic basis function models as the number of weights increases. With limited capacity, the periodic inductive bias improves both training and test loss. Once both models are sufficiently overparameterized, their training losses become indistinguishable while their test losses remain different, so training loss no longer identifies the better inductive bias.

Despite the ubiquity and success of this process, it requires humans to design new architectures and a use a multi-level training procedure, which breaks desiderata (1) and (2). Nevertheless, the reliability and performance of crossvalidation's results make it the current gold standard. Any method for selecting inductive bias cannot sacrifice too much performance, or it will not be used, as described in desideratum (3).

34.2.3 When Training Losses *do* Identify Inductive Bias

The desire to avoid the inconvenience and computational cost of using validation sets has led to attempts to still use the training loss to provide signal for learning hyperparameters (e.g. Romero and Lohit [1418]). Some success has been reported, in the sense that training over some additional hyperparameters leads to performance gains over fixed hyperparameters. These results are certainly better than what is implied by Fig 34.1, where selecting hyperparameters in this way would lead to an actively worse model. Given these results, we must ask: In what situations do training losses provide useful signal for learning hyperparameters?

One mechanism that can cause this method to work, is implicit regularisation of the solution by the initialisation and the optimisation dynamics. Consider a model that can select between the smooth and periodic models of Fig. 34.1:

$$f_w^\theta(x) = e^{\theta_1} f_w^{\text{smooth}}(x) + e^{\theta_2} f_w^{\text{periodic}}(x) \tag{34.7}$$

$$\theta^*, w^* = \arg\min_{\theta,w} \sum_{n=1}^{N} \ell(f_w^\theta(x_n), y_n) \; - c_1 e^{\theta_2} + c_2 e^{\theta_1} \, . \tag{34.8}$$

If the model is initialised with the most restrictive inductive bias (e.g. a large θ_2 and small θ_1 in the equation above), then there will be encouragement to move to a more flexible model if the inductive bias is inappropriate (and significantly hurts training loss), while the inductive bias may remain largely unchanged if it is appropriate (and still allows the data to be fit), particularly if there is some early stopping.

While this approach *can* work, it is brittle, and relies on a-priori input or human tuning to determine what initialisation and learning rate to use, or what the right amount of early stopping is. Both of these may also require some trial-and-error (or crossvalidation) to get right.

Another mechanism is the interaction between model size and inductive bias. There are settings where an appropriate inductive bias actually helps a small model to fit the data. In these cases, if the model size is constrained, then the training loss will reliably prefer the model that generalises well.[8] We ran the example from Fig 34.1 for different model sizes, and plotted the train and test losses in Fig 34.2. We see that when the number of weights is constrained, the training loss is lowest with the periodic inductive bias, leading to it preferring the model that generalises well. On the one hand, this can be seen as a good thing: Surely we want small models that generalise well?

However, for this approach to be reliable, it requires *an appropriately small model size to be set a priori.* The size must be large enough to not limit performance of the model with the good inductive bias, but small enough to prevent the training loss from preferring the more flexible model with the worse inductive bias. This "Goldilocks" is difficult to know before training, and therefore may again need crossvalidation to determine. Moreover, models

[8] The effect of model size influencing inductive bias has also been noted in transformers [1070].

are typically trained in the overparameterised regime, meaning that this mechanism for the training loss to work is brittle in exactly the regime where it matters.

The reliance of both these mechanism on a-priori knowledge or tuning means we do not consider them robust enough to satisfy desiderata (1) and (2). However, the existence of these mechanisms does provide a difficulty for evaluating such methods. If we see a method based on training loss working, we should carefully assess whether

1. we are perhaps using an intialisation based on information that is not available when the method is applied in practice, and
2. the model size is constrained and too small for allowing the architecture to predict optimally,

before concluding a method is generally robust. Although, the converse warning also holds, in that many heuristics have been remarkably successful in spite of earlier theoretical objections.

34.2.4 Explicit Regularisation of Hyperparameters

So far, we have noted various ways in which the training loss of Eq 34.3 can lose or give poor training signal to select inductive biases, in models that are sufficiently flexible to fit the training data. A natural idea to combat this is to add an *explicit regularisation* term $r(\theta)$ to directly encourage the selection of models that are known to generalise better:[9]

$$\theta^*, w^* \approx \arg\min_{\theta,w} \sum_{n=1}^{N} \ell(f_w^\theta(x_n), y_n) + n_\theta(w) + c \cdot r(\theta) \qquad x_n, y_n \in \mathcal{D}\,, \tag{34.9}$$

For the example in Fig 34.1, we could have created a model that mixed smooth and periodic components, and used a training loss that encouraged periodicity:

$$f_w^\theta(x) = e^{\theta_1} f_w^{\text{smooth}}(x) + e^{\theta_2} f_w^{\text{periodic}}(x) \tag{34.10}$$

$$\theta^*, w^* = \arg\min_{\theta,w} \sum_{n=1}^{N} \ell(f_w^\theta(x_n), y_n) \; - c_1 e^{\theta_2} + c_2 e^{\theta_1}\,. \tag{34.11}$$

This explicitly encourages the function to include a large periodic component. If this periodic component does not hurt the fit to the data too much, it is preferred.Many symmetry discovery methods use an objective function of this form, including Augerino [120] for learning affine invariances, Residual Pathway Priors [521], learning local symmetries in [149], symbolic differential equations [1799], explicitly learned weight-sharing in [1657], and bispectral networks [1457].

This strategy does come with several drawbacks:

1. A-priori knowledge is needed of which inductive biases will work on the task at hand, so the correct encouragement and discouragement can be encoded in $r(\theta)$.
2. The regularisation depends on the parameterisation of the model, and changes to the parameterisation can completely change a regulariser's behaviour (e.g. $\theta' = 1/\theta$). As a consequence, regularisers must be tailored to a particular model.
3. Additional hyperparameters (e.g. such as the strength of the regularisation c above) are introduced, which may require tuning in itself.

The a-priori knowledge that is required makes this approach task dependent, and reliant on an external/human source of this knowledge, breaking desideratum (1). In addition, any need for additional tuning of hyperparameters breaks desideratum (2).

The *degree* to which these drawbacks are a problem in practice depends on the sensitivity to the above factors, which can be determined empirically. If a single regularisation gives improved results over a wide range of tasks, it will still be useful. Some evidence of success exists in the literature: When the loss landscape contains broad, near-flat basins of almost constant training loss the sensitivity to the tuning of the hyperparameter regulariser $c \cdot r(\theta)$ may be low [119, 521, 1659]. However, empirical examples of the failure modes also exist (see Sec. 34.8.5, or App. C in [815]). Ultimately, the only way to determine the sufficiency of a method in practice, is an experimental comparison.

[9] Probabilistically speaking, this can be interpreted as placing a hyperprior $p(\theta) \propto \exp[-c \cdot r(\theta)]$ on the inductive-bias parameters, when the loss is properly scaled and the resulting density is normalisable.

34.2.5 Differentiable Validation Losses and Neural Architecture Search

Overall, of the methods discussed so far, crossvalidation remains the most widely applied, at the cost of requiring retraining models many times. Another line of inquiry focusses on reducing this cost, by estimating gradients of the validation loss w.r.t. hyperparameters. Notable examples are in the field of neural architecture search (NAS). Notably, Darts [1060] does this by differentiating the validation loss through a finite sequence of updates on weights based on training data. This requires backpropagation through the optimization trajectory, which involves Hessian-vector products. Alternatively, this can be done by implicit differentiation at a converged inner optimum involves an inverse-Hessian-vector product [1080]. Both approaches require approximations to allow application in practice. Methods based on neural architecture search also often use evolutionary algorithms [490, 1384] or reinforcement learning [1885]. Applied to symmetry discovery, notably [1869] differentiate validation loss to learn convolutional weight-sharing structure and AutoAugment [348] used reinforcement learning outer loops. Other approaches to symmetry discovery also exist, for example through generative adversarial networks in [1798]. Approaches that rely on the validation set inherit the robustness of this method (up to any approximations), and avoid the need for knowing what kind of solutions are good a priori, but do introduce different procedures and datasets for weights and hyperparameters, breaking desideratum (2).

34.2.6 The Need for Something Better

In the past sections we have seen that several classes of methods have been developed with the aim of simplifying hyperparameter selection, and with various successes and compromises on our desiderata. Methods based on validation sets inherently rely on using different procedures for parameters and hyperparameters, which introduces complexity we would rather avoid (desideratum 2), while methods relying only on training data relied on parameters that needed to be tuned just right to make the method work (breaking desideratum 1). We argued that ultimately, empirical results are the only way to determine the success of an overall method.

For methods based on training sets only (Sections 34.2.3, 34.2.4), we saw that implicit regularisation could lead to selecting models that generalised well, even if the objective function itself did not prefer them. We argued that these implicit regularisers (initialisation, training dynamics, early stopping, using small model sizes) could be brittle and break when placed in a realistic scenario. The interaction between inductive bias and model size was a particularly strong example of this. Small model sizes could make the training loss prefer a constraining inductive bias, that did help generalisation performance. However, this effect can disappear when applied to models of sizes that are not artificially restricted, which is exactly the setting that deep learning cares about!

We draw two conclusions from this review. **Firstly**, that empirical evaluation of hyperparameter learning methods has potential pitfalls that we should be aware of. Empirical evaluations should be aware of implicit regularisation that may exist, and establish how widely this can be relied on. **Secondly**, we saw that every method broke some of our desiderata, or had failure modes.

The key trade-off came from the objective functions we considered. On the one hand, we had losses evaluated on validation sets, which robustly evaluated which methods generalised without relying on implicit or explicit regularisation, but which required awkward multi-stage optimisation processes. On the other hand, we had losses evaluated only on training sets, which were convenient to optimise, but which failed to distinguish between hyperparameters that performed differently, but which fit the data equally well.

In the following, we argue that Bayesian inference can provide us with a strong principle for developing procedures for learning hyperparameters.[10] In particular, it leads to an objective function that *does* distinguish between models that generalise differently, while also satisfying our desiderata. The difficulty with the Bayesian approach is that it prescribes *what* should be computed to solve a particular problem without giving any hints on *how* this should be done. However, knowing what quantity to compute provides a helpful way to guide the development of methods, since efforts can focus on approximating this quantity accurately.

Currently, it is still an open problem for how to *accurately* approximate Bayesian inference in neural networks, despite much work that has already been carried out. We therefore review the capabilities we hope to achieve by continuing this line of work, and argue that it is worth to continue research in this direction.

[10] We do not argue that any (strictly) Bayesian approach is the *only* way to solve these problems! There is are many other theories for assessing what models will generalise well without relying on validation data. While these methods are currently mostly used for explaining why deep learning works, they could also be investigated as principles for improving training procedures. It is also interesting to note that learning theorists focus on providing mathematically precise statements about generalisation. Bayesian inference does not. It only prescribes a procedure. Or alternatively, "Bayes tells us what to do, but not why it works".

34.3 From Bayesian Hierarchical Inference to Model Selection

The general Bayesian procedure for inferring an unknown quantity is to put a prior on it, and then calculating the posterior distribution given the training data. One early hint that Bayes may lead us to a method that fits with our desiderata, is that *any* strictly Bayesian procedure only depends on *training data*, which was one of the requirements laid out in the introduction. Here, we review which quantities Bayesian inference *prescribes* us to compute, leaving concerns of how to compute practical approximations to later.

34.3.1 Uncertainty over the Weights: The Typical Bayesian Deep Learning Case

We begin with the problem that Bayesian deep learning is typically known for: quantifying uncertainty over the weights. We specify a model and its assumptions through its *probability of everything* [1111, p. 61], by specifying a prior on weights w and a likelihood given the data (x_n, y_n):

$$p(\mathbf{y}, w|\theta, \mathbf{x}) \stackrel{\text{def}}{=} \prod_{n=1}^{N} p(y_n|w, x_n, \theta)p(w|\theta) \tag{34.12}$$

Typically, we have a normal distribution on weights, possibly with per-layer variances, with our likelihood depending only on the weights and hyperparameters through the neural network function $f_w^\theta(x_n)$, so we can write:

$$p(y_n|w, x_n, \theta) = p(y_n|f_w^\theta(x_n))\,, \qquad p(w|\theta) = \prod_{\ell=1}^{L} \mathcal{N}(w_\ell; 0, \sigma_\ell^2)\,. \tag{34.13}$$

We merge all hyperparameters into θ, so they control both the architecture, as well as the prior variances for each layer σ_ℓ^2.

In the typical Bayesian deep learning case, hyperparameters are considered to be fixed, and they are typically found through trial-and-error/crossvalidation. This means that in order to make predictions, we can focus on finding the posterior on the weights:

$$p(w|\mathbf{x}, \mathbf{y}, \theta) = \frac{\prod_{n=1}^{N} p(y_n|w, x_n, \theta)p(w|\theta)}{p(\mathbf{y}|\theta, \mathbf{x})}\,. \tag{34.14}$$

Actually calculating a good approximation to this posterior is a significant challenge in itself, and many papers have been written on this topic.

34.3.2 Inferring Hyperparameters through Hierarchical Bayes

Bayes is a helpful starting point for solving inference problems, because it *prescribes* what should be computed to solve the problem. If we want to infer hyperparameters as well as weights, we can simply follow the previous procedure, and specify a "probability of everything" that includes the hyperparameters:

$$p(\mathbf{y}, w, \theta|\mathbf{x}) \stackrel{\text{def}}{=} \prod_{n=1}^{N} p(y_n|w, x_n, \theta)p(w|\theta)p(\theta)\,. \tag{34.15}$$

To make predictions, all we need to do is find the posterior over both the weights w and hyperparameters θ. By grouping terms together, we can see that this posterior consists of the posterior over the weights from earlier (Eq 34.14) and an additional posterior over the hyperparameters:

$$p(w, \theta|\mathbf{y}, \mathbf{x}) = \frac{\prod_{n=1}^{N} p(y_n|w, x_n, \theta)p(w|\theta)p(\theta)}{p(\mathbf{y}|\mathbf{x})} \tag{34.16}$$

$$= \underbrace{\frac{\prod_{n=1}^{N} p(y_n|w, x_n, \theta)p(w|\theta)}{p(\mathbf{y}|\theta, \mathbf{x})}}_{=p(w|\mathbf{y},\theta,\mathbf{x})} \cdot \underbrace{\frac{p(\mathbf{y}|\theta, \mathbf{x})p(\theta)}{p(\mathbf{y}|\mathbf{x})}}_{=p(\theta|\mathbf{y},\mathbf{x})}\,, \tag{34.17}$$

$$\text{with } p(\mathbf{y}|\theta, \mathbf{x}) = \int \prod_{n=1}^{N} p(y_n|w, x_n, \theta)p(w|\theta)\mathrm{d}w\,. \tag{34.18}$$

The structure of this posterior is shared with that of any hierarchical model. Hierarchical modelling is a powerful technique that goes far back, and understanding its characteristics is very helpful in understanding why model selection gives benefits over non-hierarchical independent priors. Chapter 28 in MacKay [1111] gives an excellent overview of how to apply hierarchical modelling and Bayesian model selection, and the mechanisms by which it works (a measurement of simplicity, and an automatic Occam's razor effect that removes unnecessary complexity). In-depth discussions can also be found in the statistics literature, e.g. chapter 5 and 15 of Gelman et al. [602] or Gelman [595]. Interestingly, there are different opinions on when hierarchical modelling is a valid thing to do, which we will discuss later. Nevertheless, in principle, Bayes gives a unified training procedure for weights and hyperparameters that also only depends on training data. As before, approximations are needed in practice, and research continues on how best to do this.

Leaving the *how* aside for the moment, we can already ask questions about how this *prescribed* procedure compares to the approaches discussed in Section 34.2, in terms of what information needs to be specified by the user. All terms that need to be specified are in Eq. 34.15. After taking the logarithm and comparing to Eq. 34.5, we notice that the likelihood $p(y_n|w, x_n, \theta)$ can be identified with the loss function, and $p(w|\theta)$ with the weight regulariser. Moreover, we can notice that the prior on the hyperparameters $p(\theta)$ can be identified with the explicit regulariser in Eq. 34.9.

The requirement to specify a loss and a regulariser is no more a-priori knowledge than what is required in "typical" deep learning. However, in Section 34.2.4 we criticised the need for explicit encouragement of solutions that were known to be good, which breaks desideratum (1). Can we determine that Bayes suffers from the same problem? To answer this question, we must investigate the hyperparameter posterior $p(\theta|\mathbf{y}, \mathbf{x})$ (Eq. 34.17). It is true that Bayes prescribes that we need to specify *some* prior $p(\theta)$, but the problem in Section 34.2.4 was that the hyperparameter regulariser was the *only* mechanism for preferring some solutions over others. In contrast, the posterior $p(\theta|\mathbf{y}, \mathbf{x})$ uses $p(\mathbf{y}|\theta, \mathbf{x})$ as the likelihood. We will discuss later (Section 34.4) that this behaves very differently to normal likelihoods, and includes an automatic Occam's razor behaviour that also distinguishes models in a way that is related to their generalisation, allowing the method to work even with a flat prior $p(\theta)$.

34.3.3 Maximum Marginal Likelihood as a Practical Approximation

In both cases described above, we mentioned that approximations are needed. One general practical difficulty with applying Bayesian inference comes from needing to approximate analytically-intractable integrals in the posteriors and predictive distributions. *Any* approximation has to deal with the integration in one way or another.[11]

Strictly speaking, Bayesian inference in hierarchical models *prescribes* us to make predictions using the full posterior over weights w *and* hyperparameters θ, which brings additional challenges over just computing posteriors over weights alone. The posterior over hyperparameters $p(\theta|\mathbf{y}, \mathbf{x})$ is often a lot harder to compute than of the level below in the hierarchy (in our case w). For example, in (hierarchical) Bayesian linear regression [1212, §11.6] or Gaussian processes [1382] the posterior that corresponds to w can be computed in closed form, while the posterior over hyperparameters cannot. More generally, integrating over the additional hyperparameter variables is harder than simply integrating over the weights w, and hierarchical structure makes this notoriously hard [146]. Neural networks share these properties, and moreover, integrating over multiple hyperparameters would mean keeping around multiple architectures, which is a large cost. Can Bayes still help to get adaptive architectures, without needing to directly ensemble over the whole architecture?

The *Maximum Marginal Likelihood* method, also known as *Type-II Maximum Likelihood*, suggests to represent only the mode of the posterior over *hyperparameters* $p(\theta|\mathbf{y}, \mathbf{x})$, instead of the whole distribution.[12] The key justification is that if the full hyperparameter posterior $p(\theta|\mathbf{y}, \mathbf{x})$ is highly peaked, integrals over it can be approximated by placing all its mass at its mode

$$p(\theta|\mathbf{y}, \mathbf{x}) \approx \delta(\theta - \theta^*) \tag{34.19}$$

$$\theta^* = \arg\max_\theta \log p(\mathbf{y}|\theta, \mathbf{x}) + \log p(\theta)\,. \tag{34.20}$$

Additionally, if the prior on θ is flat, we can even disregard the hyperparameter prior $p(\theta)$ and seek out the maximum of the *marginal likelihood* only:

$$\theta^* = \arg\max_\theta \log p(\mathbf{y}|\theta, \mathbf{x})\,. \tag{34.21}$$

[11] For discussions on the mathematical reasons for why this is hard, see MacKay [1111, §29.1], or Betancourt [144, §1] and its accompanying talk [145]).

[12] For examples in various models, see §7.5 in Murphy [1212], or §5.2 in Rasmussen and Williams [1382] for an application in Gaussian processes.

This approximation makes predictions with the usual weight posterior $p(w|\mathbf{y}, \mathbf{x}, \theta)$, but with the hyperparameters θ being set by the above point estimate. This still allows adaptability of the architecture, but without the need to consider multiple architectures.

The only quantity that is needed compared to the typical Bayesian deep learning case, is the marginal likelihood $p(\mathbf{y}|\theta, \mathbf{x})$. Interestingly, this quantity is computed directly when computing the weight posterior $p(w|\mathbf{x}, \mathbf{y}, \theta)$ (Eq. 34.14). In addition, approximate inference methods often also provide an estimate of the marginal likelihood as a byproduct of approximating the weight posterior, with the approximation quality of both being related (e.g. in variational inference [170] and annealed importance sampling [1228] a more accurate distribution approximation leads to a more accurate marginal likelihood). So, there may be reason to hope that this adaptivity can be gained without too much additional overhead compared to Bayesian learning of the weights alone.

So overall, we see that by starting from what Bayesian inference prescribes for inferring hyperparameters and applying a simple approximation, we arrive at a procedure that selects a single model. We can see that this procedure solves the same *model selection* problem as crossvalidation. Because of this, we will interchangeably refer to this procedure for selecting (mostly continuous) hyperparameters as Bayesian model selection, along with Maximum Marginal Likelihood, and Type-II Maximum Likelihood. This approximation has a long history, with early mentions of the phrase "type-II maximum likelihood" being in Good [648] and Berger [123].

The reasoning that we discussed so far followed Bayesian inference as a *prescriptive* procedure. We have simply followed the Bayesian prescription of solving inference problems by finding a posterior, and then applied one simplification to avoid the computational difficulty of integrating over the hyperparameters. The outcome of this path, is a procedure that can be directly applied to models where the weight posterior can be computed directly (e.g. linear models [1212, Chapter 9] or Gaussian processes [1382]), which is not the case for neural networks. And so, we will discuss further approximations in upcoming sections.

34.3.4 Practical Approximations

For Bayesian model selection we require the marginal likelihood $p(\mathbf{y} \mid \mathbf{x}, \theta)$ (or its log) and, more generally, the posterior $p(w \mid \mathbf{x}, \mathbf{y}, \theta)$. In neural networks these quantities are typically intractable because the likelihood $p(\mathbf{y} \mid \mathbf{x}, w, \theta)$ depends on w through a *nonlinear* mapping $w \mapsto f_w^\theta(\cdot)$ induced by the network (and because w is high-dimensional). As a result, the marginal likelihood (evidence) integral cannot be computed in closed form, motivating approximate inference.

Two closely related approximations are *variational inference* (VI) [170, 174, 660] and the *Laplace approximation* [1105]. Both typically approximate the posterior with a Gaussian distribution $q(w)$ and thereby induce a tractable approximation to the marginal likelihood. VI treats the mean and covariance of this Gaussian as variational parameters and optimises them through the evidence lower bound (ELBO), yielding a unified training criterion that can be differentiated with respect to both the parameters of $q(w)$ and the hyperparameters θ. The ELBO is a global, distribution level objective that balances expected data fit and prior complexity across $q(w)$. In contrast, Laplace restricts $q(w)$ to Gaussians centred at local MAP estimates, with covariance fixed by local posterior curvature. For the same Gaussian structure, the set of approximations attainable by Laplace is therefore a strict subset of the variational family available to VI. Consequently, the optimal VI ELBO is at least as high as the ELBO of any Laplace solution, and is strictly higher whenever the Laplace approximation is not itself ELBO optimal.

Regardless, the Laplace approximation can be very practical: it avoids solving an additional optimisation problem for $q(w)$ and instead constructs a Gaussian approximation from a point estimate (often the MAP weights) and local curvature information. This can make Laplace-based evidence approximations attractive when a good point estimate is readily available and when scalable curvature approximations are feasible.

In practice, both VI and Laplace often introduce additional structural approximations to make covariances (or precisions) computationally manageable at scale. Common choices include diagonal covariances, Kronecker-factored [812, 1136], or equivalently matrix-normal parameterisations [1083] in VI. These structural assumptions trade off fidelity and scalability and can be used within either approach: in VI they parameterise $q(w)$ directly, while in Laplace they parameterise or approximate the curvature used to form the Gaussian $q(w)$.

In literature, these approximations have provided differentiable surrogates of the marginal likelihood $\log p(\mathbf{y} \mid \mathbf{x}, \theta)$ for model selection and tuning hyperparameters θ (including symmetries) using only training data. Immer et al. [812] was the first to demonstrate Bayesian model selection at scale in large neural networks using the Laplace approximation, relying on Kronecker-factored approximations of the Generalized-Gauss Newton [191, 1136]. This was extended in [815] to allow differentiable Laplace-based evidence objectives for online learning of simple affine invariances using mini-batched training. In [1658] such scalable online Laplace approximations were used to learn layer-wise equivariance symmetries. A last-layer variational objective similar to deep kernel learning [1766] was

used for learning affine invariances in [1480, 1661], but the lower bound was only found tight enough to scale to single hidden-layer neural networks. In [1660], variational inference was applied to deeper networks for learning conservation laws and symmetries in Hamiltonian systems.

34.3.5 Applying Bayesian Model Selection

To get an idea of what capabilities Bayeisan model selection can potentially bring to neural networks, we first illustrate its behaviour on a simple example where all the required calculations can be performed in closed form. We return to the example of Fig. 34.1 and consider training a model that can interpolate between two inductive biases as in Eq. 34.10: purely smooth, and periodic. We parameterise our space of functions using a feature map $\phi_\ell : \mathbb{R}^D \to \mathbb{R}^M$ consisting of many local "bump" basis functions arranged in a dense grid, with width ℓ.[13] We use this to construct one space of functions $f_w^{\text{smooth}}(x)$ that just has a smoothness constraint, and another $f_w^{\text{periodic}}(x)$ which is also constrained to be periodic by first passing the input x through the map $u_p(x) = [\cos(2\pi x/p), \sin(2\pi x/p)]^T$. We specify our "probability of everything" as:

$$w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \qquad \theta = \{\ell_1, \ell_2, p, \theta_1, \theta_2, \sigma^2\} \tag{34.22}$$

$$f_{w,\theta}^{\text{smooth}}(x) = \phi_{\ell_1}(x)^T w_1 \,, \tag{34.23}$$

$$f_{w,\theta}^{\text{periodic}}(x) = \phi_{\ell_2}(u_p(x))^T w_2 \,, \tag{34.24}$$

$$f_w^\theta(x) = f_w^{\text{smooth}}(x) + f_w^{\text{periodic}}(x) \,, \tag{34.25}$$

$$p(w|\theta) = \mathcal{N}\left(w; 0, \begin{bmatrix} e^{\theta_1} \cdot I_M & 0 \\ 0 & e^{\theta_2} \cdot I_m \end{bmatrix}\right) \,, \tag{34.26}$$

$$p(y_i|w, \theta, x_i) = \mathcal{N}(y_i; f_w^\theta(x_i), \sigma^2) \tag{34.27}$$

We now have hyperparameters θ, which would need to be learned by crossvalidation if taking a point estimate for w to avoid the pitfalls of Sections 34.2.3, 34.2.4. Instead, we find a point estimate for θ according to Maximum Marginal Likelihood (Eq. 34.21) and find that after only a small number of datapoints, it strongly favours the periodic model (see Fig. 34.3).

We note several advantages over method that rely on (regularised) training fits:

- The marginal likelihood could distinguish between models that fit the data equally well, in contrast to the training loss (Fig. 34.1).
- Moreover, the marginal likelihood preferred the more constrained inductive bias, that allowed better generalisation on this problem.
- This preference came from the marginal likelihood directly, since we effectively used a flat prior $p(\theta)$. We did not need to use a prior $p(\theta)$ that encouraged inductive biases that were known to be good for the problem, as in Section 34.2.4.
- *All* hyperparameters were selected using the marginal likelihood, including those hat controlled the weight prior $p(w|\theta)$.

In this example, Bayesian model selection provided a principle for finding a good inductive bias, and the right regularisation parameters that controlled the prior $p(w|\theta)$, while only relying on the training set. Moreover, it did not rely on imposing additional regularisers, that did not already exist in the typical deep learning formulation (Eq. 34.5). This demonstrates, in this simple example, capabilities that the other potential methods (Section 34.2) did not have, that get far closer to satisfying our three desiderata. The broader research goal is to get these benefits that exist for simple models, even in deep neural networks.

34.4 Why & When Bayesian Model Selection Works

Before we discuss practical approximations, we first return to the question that was raised at the end of Section 34.3.2: *Why* does this Bayesian procedure work? Bayes has told us *what* to do, but it has not explained *why* it

[13] We are following the weight-space parameterisation of Gaussian processes with Squared Exponential [1382, §4.2.1] and periodic [1113, 1382, §4.2.3] kernels. These are both single-layer fixed-basis function neural networks.

works. We can ask this question both about the exact procedure that Bayes has specified (Section 34.3.2) and the approximation we derived (Section 34.3.3). Answering this question will also provide insight into any limitations. In our discussions, we will take care to understand whether the problems we encounter are **a)** fundamentally hard problems about statistics (and so whether *any* method could solve it), **b)** difficulties with Bayesian inference in general, or **c)** problems that are introduced through the point estimate approximation of the full posterior $p(\theta|\mathbf{y}, \mathbf{x})$.[14]

As discussed in the previous section, Bayes prescribes that we find the hyperparameters through the posterior

$$p(\theta|\mathbf{y}, \mathbf{x}) = \frac{p(\mathbf{y}|\theta, \mathbf{x})p(\theta)}{p(\mathbf{y}|\mathbf{x})}\,, \tag{34.28}$$

and integrate them out when making predictions, with our practical approximation choosing the point estimate

$$\theta^* = \arg\max_{\theta} \log p(\mathbf{y}|\theta, \mathbf{x})\,. \tag{34.29}$$

In both cases, the key quantity that determines how the *data* influences the estimate is the *marginal likelihood* $p(\mathbf{y}|\theta, \mathbf{x})$. This is the key quantity that needs to be understood to understand both how the full hyperparameter posterior changes from the prior $p(\theta)$, and how our point estimate approximation behaves.

We can derive the marginal likelihood in terms of the model specification in the "probability of everything" (Eq. 34.15) as

$$p(\mathbf{y}|\mathbf{x}, \theta) = \int \prod_{n=1}^{N} p(y_n|w, x_n, \theta)p(w|\theta)\mathrm{d}\theta\,. \tag{34.30}$$

From this, we can immediately see that the integration over the global parameters $p(w|\theta)$ lead $p(\mathbf{y}|\mathbf{x}, \theta)$ to no longer be i.i.d. over $\mathbf{y}$, in contrast to the likelihood *given* the weights $p(\mathbf{y}|w, \mathbf{x}, \theta) = \prod_{n=1}^{N} p(y_n|w, x_n, \theta)$. It is this integration over the global parameters that gives the marginal likelihood its important Occam's razor property, which is crucial for using it to select inductive hyperparameters that control inductive bias. The fact that it is a joint distribution is also the cause of it behaving fundamentally differently from methods that rely on the fit to the training set discussed in Sections 34.2.3 and 34.2.4. In the following, we investigate further properties of this distribution that explain why the marginal likelihood correlates more with generalisation than (regularised) training losses, where its Occam's razor property comes from, and how it relates to other frameworks.

34.4.1 Losses During On-Line Prediction

One way to intuitively understand why the marginal likelihood is sensitive to how a model generalises, regardless of how it ends up fitting, is to apply the product rule of probability. By selecting an arbitrary ordering of our datapoints, we can rewrite the marginal likelihood as:

$$\log p(\mathbf{y}|\theta, \mathbf{x}) = \log p(y_1|x_1, \theta) + \log p(y_2|y_1, x_{:2}, \theta) + \log p(y_3|y_{:2}, x_{:3}, \theta) + \dots \tag{34.31}$$

$$= \sum_{n=1}^{N} \log p(y_n|y_{:n-1}, x_{:n}, \theta)\,. \tag{34.32}$$

We can view each term in the sum as the utility that is obtained when the model makes a prediction based on all $n-1$ past datapoints, and the nth datapoint is the actual outcome, when the log probability is used as the utility function.[15] The marginal likelihood is therefore the total utility obtained if this prediction procedure were played as all datapoints arrive in an on-line manner.[16]

This firstly explains why models that have the same fit to the training dataset can have vastly different marginal likelihoods. Models where past data predict future data well have high marginal likelihoods, whereas model fit only measures how a model memorises the training data. Secondly, this also makes a link to generalisation. Each term in the sum measures how well the model generalises on the nth datapoint. If the model has generalised well for many datapoints, the credibility that it will predict well on the next one increases too (see 34.4.3).

[14] In practical applications, the marginal likelihood itself will need to be approximated. This also introduces inaccuracies, and that gives an additional factor to be disentangled. A failure of an approximation to the marginal likelihood failing does not necessarily invalidate the maximum marginal likelihood method in general.

[15] With utility simply being the negative of loss, to keep the optimistic sign convention.

[16] See similar discussions in MacKay [1111, p. 353], and MacKay [1110].

We have visualised this process in Fig. 34.3 for the example discussed in Sections 34.2.1 and 34.3.5. We see that although the fit to the training data ends up being very similar for both models (right column), the marginal likelihood for the periodic model ends up being much higher, as the inductive bias allows for good predictions for unseen data with far less data than for the smooth model. This also indicates the better generalisation of the periodic model.

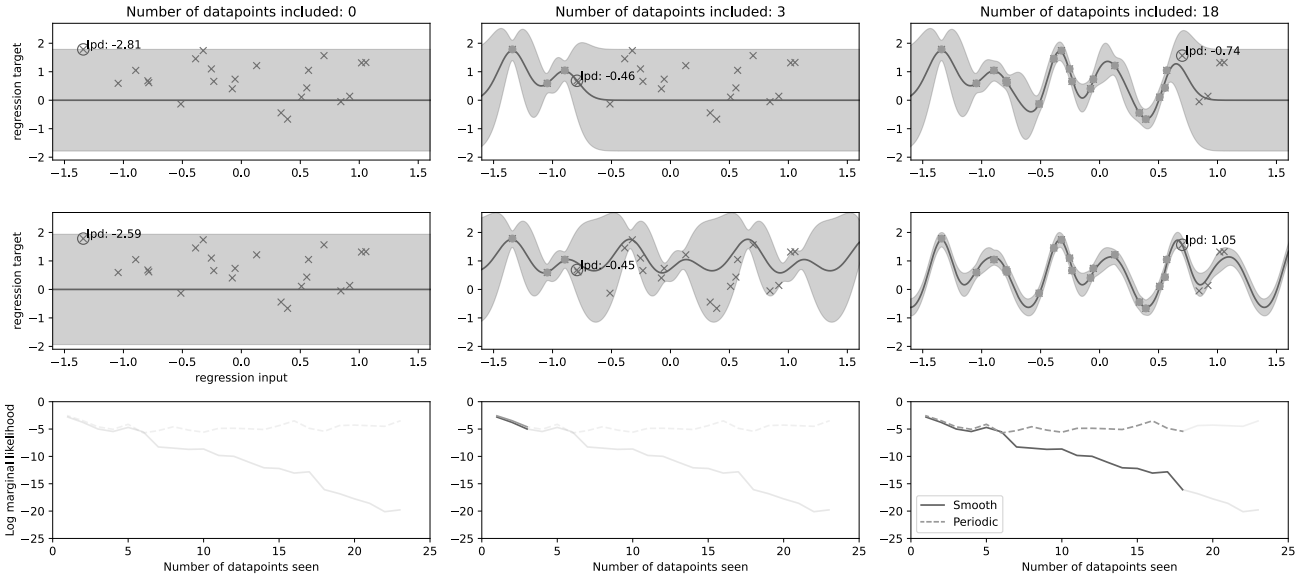


Figure 34.3: Regression predictions (with mean and 95% credible interval indicated) on a periodic dataset, with the smooth (top row) and periodic (middle row) models described in Section 34.3.5. All hyperparameters are optimised using the marginal likelihood, with the exception of the smooth model's periodic component, which is clamped to zero. The columns differ in the number of datapoints given to the predictive posterior. The marginal likelihood is the sum of all predictive log likelihoods given past data points (indicated by a circle). The evolution of the marginal likelihood as more datapoints are added is plotted in the bottom row. For the periodic model, past parts of the dataset predict future parts well, and the model start predicting future datapoints well much earlier, indicating better generalisation.

34.4.2 Occam's Razor

Another view on the marginal likelihood has been that it automatically embodies an "Occam's razor" property that penalises model complexity. The key observation is that $p(\mathbf{y}|\theta, \mathbf{x})$ has to normalise over all possible datasets $\mathbf{y}$. Models that can fit many datasets must spread their probability mass thinly (and could be called more "complex"), while models that fit fewer datasets can assign higher probability to each (and could be called "simpler"). See Chapter 28 in MacKay [1111] or Rasmussen and Ghahramani [1380] for deeper discussions. An common illustration of this is depicted in Fig. 34.4, with [1214] providing a very helpful realistic depiction for a toy model.

34.4.3 Relationship to Other Frameworks: Crossvalidation, Generalisation Bounds, Compression

In addition to intuitive perspectives, other mathematical frameworks also investigate or prescribe the use of quantities that are similar to the marginal likelihood. The relationship to generalisation was made theoretically rigorous by Germain et al. [610] who noted that a monotonic relationship between a generalisation bound on samples from the posterior and the marginal likelihood of the model. However, this was under the constraint that no fitting of (hyper)parameters was done using the marginal likelihood, meaning that no generalisation bound for the Maximum Marginal Likelihood procedure (Section 34.3.3 is provided by this work. Nevertheless, the PAC-Bayesian framework can be extended to give a valid generalisation bound when hyperparameters are adapted, but this then required a full posterior over hyperparameters, *just as Bayes actually prescribes* (Section 34.3.2). This serves as a reminder that Maximum Marginal Likelihood is justified by the Bayesian framework only as an approximation when the posterior is peaked. When it is not, Bayes prescribes that hyperparameters are integrated.

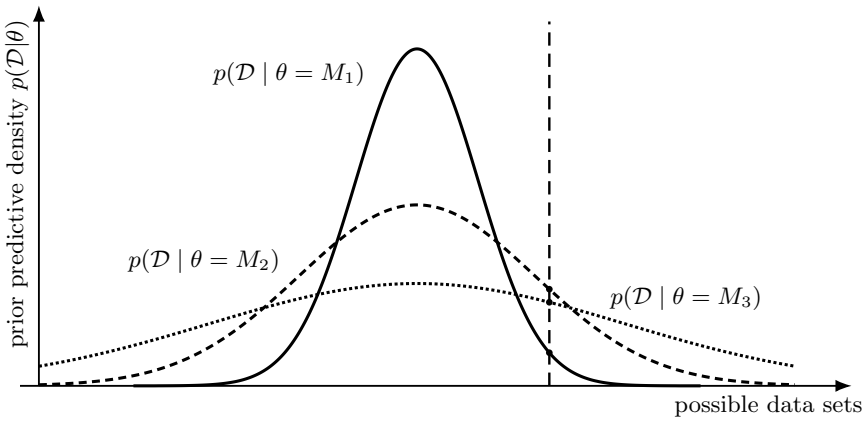


Figure 34.4: Cartoon illustrating that models must have distributions over datasets $p(\mathbf{y}|\theta, \mathbf{x})$ that normalise. Simple models spread their mass narrowly, and can give high marginal likelihoods to a small set of datasets, while complex models spread their mass widely, but cannot assign high marginal likelihood to all of these datasets. This property of the marginal likelihood is the mechanism for automatically penalising model complexity.

Another perspective is that the log marginal likelihood $\log p(\mathbf{y}|\theta, \mathbf{x})$ actually quantifies the coding length of how many bits are required to encode both the training data $\mathbf{y}$ and the model parameters $\mathbf{w}$. Compression is key to generalisation [173], and the Minimum Description Length (MDL) community [676, 1399, 1698] has argued for selecting models based on which compresses most. In MDL terminology, the marginal likelihood corresponds to a one-part Bayesian mixture code, and the sequential decomposition in Section 34.4.1 corresponds to a *prequential* code. Again, just like the Bayesian prescription, and the PAC-Bayesian analysis, MDL also notes that maximising the marginal likelihood w.r.t. hyperparameters is an approximation, given that it ignores the coding cost of the hyperparameters themselves. Again, the solution is to introduce a posterior over hyperparameters.

Finally, there is also a relationship between the marginal likelihood with the model selection method that we consider our "gold standard". Fong and Holmes [537] showed that the marginal likelihood is equivalent to a crossvalidation scheme that averaged over all splits between training and testing sets. This correspondence shows that there is nothing magical about the marginal likelihood, in that it has equivalences with established methods, and therefore shares strengths and weaknesses. One key benefit of this mathematical relationship, is that formulation of the marginal likelihood as an integral may actually allow for approximations that are relatively computationally efficient. Crossvalidation is actually an incredibly expensive method, that requires retraining a whole machine learning method many times. While evaluating the integral of the marginal likelihood is certainly not easy, it does open up the possibility for applying a whole host of approximation techniques that are not available when viewing crossvalidation the "retrain-evaluate" perspective.

34.5 Problems with Bayesian Model Selection

Bayesian model selection in general, and maximising the marginal likelihood in particular have been discussed many times over the years. Sometimes, the critiques have even strongly argued against the use of this method. In many ways, our exposition in previous sections has followed exactly what Bayes straightforwardly prescribes, and which has been explained by many researchers who came before us. We take the position that this framework is mostly correct, and that any problems with the Bayesian framework as a whole, and Maximum Marginal Likelihood specifically, are small compared to the unrealised benefits that they offer. The challenge is in how compute the required quantities, and how to specify good models. Whether this is possible is an open research problem, with at most indirect evidence for and against.

Nevertheless, it is good to be aware of the limitations of the framework that we base our methods on. Three limitations stand out.

34.5.1 Sensitivity to Weight Prior

The marginal likelihood has its name because it is found by marginalising over the parameters of a model (Eq. 34.30), so its value certainly depends on the choice of the weight prior. The problem is that once the dataset is large enough, the predictions of the posterior may depend only weakly on the prior, while the value

of the marginal likelihood is very sensitive to it. This means that there exist ways to select priors for models so that they predict well, but have a poor marginal likelihood. This may cause worse models to be selected. Gelman [598] and Mackay had a frequently cited discussion on whether the Occam's razor behaviour of Bayesian model selection and the interplay with priors was desirable or not. The resolution we subscribe to follow the explanation of Occam's razor of MacKay [1111], but to follow Gelman's advice on how to *apply* Bayesian model selection. In short, Gelman [598] argues against using Bayesian model selection over a discrete set of models. Instead, Gelman [594] argues for "bridging" these models using some continuous parameterisation. We followed exactly this advice in Section 34.3.5, where we allowed a continuous interpolation between the two extreme inductive biases of only smoothness or full periodicity. We follow this same advice in our later architecture search.

Lotfi et al. [1081] also raise the issue with the prior, and argue that using highly dispersed priors can really hurt the marginal likelihood, while still generalising well. The paper includes experiments where changing the variance of the weigh prior $p(w|\theta)$ can break correlation between the marginal likelihood and generalisation. This effect certainly exists, and can cause the selection of a sub-optimal model. Luckily, properties of the prior can be adjusted using the marginal likelihood (see Section 34.3.5 for an example), so long as the number of hyperparameters does not grow so large that the point estimate approximation becomes too poor. In our work on hyperparameter selection, this is how we deal with this problem.

Lotfi et al. [1081] suggest a "conditional log marginal likelihood" as an alternative solution, which removes the first few terms in the log marginal likelihood sum of Eq. 34.32. This can help remove some prior dependence, but it comes at the price of also needing to tune how many terms to remove, while also introducing another split within the training set, both of which we are aiming to avoid according to our desiderata.

34.5.2 Overfitting when Fitting Many Hyperparameters

Increasing the number of parameters that are to be estimated using the marginal likelihood also comes with problems. It is well-known [1382, p. 109] that overfitting can even occur for the marginal likelihood when too many hyperparameters are added relative to the dataset size (e.g. when putting all neural network parameters in the hyperparameters [1253]). This behaviour is also discussed by more recently Lotfi et al. [1081] as an argument against the marginal likelihood.

When assessing this problem, it is important to keep two things in mind. Firstly, that there are limits to how many models we can select between from a finite data. Even crossvalidation can end up overfitting in this way [248, 1234]. Secondly, that there are ways to mitigate against these problems. The Bayesian prescription (Section 34.3.2) is to fall back on the prior when the marginal likelihood is not peaked enough, and that we need to perform integration over hyperparameters. Another solution is to be careful to not include too many hyperparameters to optimise over, and leave it as an empirical question to formulate a procedure that can be shown to be generally applicable. We take the latter approach.

34.5.3 Architectures are Defined

One final criticism about this approach, is the degree to which we are really *learning* inductive biases. In all our approaches, we always have to specify a set of inductive biases to select from. In our example from section 34.3.5, our model could select various amounts of periodicity. One objection that could be raised is that the method only worked because the inductive bias (e.g. periodicity) was built in! If the dataset did not contain periodicity (or other properties that fit an inductive bias in our set), the inductive bias cannot be learned.

Again, this is true. However, there are two responses to this. Firstly, we must compare against the alternative: not adapting inductive biases. In the example from section 34.3.5, we would be stuck with just using the smooth function. With hyperparameter adaptation, we can gain the advantage of an inductive bias in our set (e.g. periodicity) if we are lucky enough to encounter dataset that is suitable for it. If the dataset at hand does not have exploitable structure, we are no worse of. Secondly, we should keep in mind that every machine learning method must specify a candidate set of solutions that one is chosen from. Neural networks themselves parameterise spaces of functions. If the function that is to be learned is outside this space, it cannot be learned. Developing a space of inductive biases is an active research direction, and geometric deep learning [210] is a rich area that is exploring this space. Developing Bayesian model selection further would be a good counterpart to geometric deep learning research, since it would allow automatically selecting an inductive bias within this increasingly rich set.

34.6 Exact symmetry learning in basis function model

To make the role of marginal likelihood concrete, we begin with a tractable shallow model: Bayesian linear regression with fixed basis functions. This allows us to compute the marginal likelihood in closed form and to see explicitly how hyperparameters θ can shape the inductive bias. As discussed in the previous section, the symmetry is not embedded explicit prior on weights, but in the parameterization of the function class, here through the choice of basis functions. We assume inputs $x \in \mathbb{R}^{D_{\text{in}}}$ and a vector of basis functions $\phi(x) \in \mathbb{R}^M$. The model is

$$y = w^\top \phi(x) + \epsilon, \tag{34.33}$$

where $w \in \mathbb{R}^M$ are weights and $\epsilon \sim \mathcal{N}(0, \sigma^2)$. We place a standard isotropic Gaussian prior on the weights:

$$p(w) = \mathcal{N}(w \mid 0, \alpha^{-1} I), \tag{34.34}$$

with precision hyperparameter $\alpha > 0$. Given data $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$, the likelihood is

$$p(y \mid w, \sigma^2) = \mathcal{N}(y \mid \Phi w,\, \sigma^2 I), \tag{34.35}$$

with design matrix $\Phi = [\phi(x_1)^\top; \ldots; \phi(x_N)^\top]$. Because both prior and likelihood are Gaussian, the marginal likelihood has a closed form [160, 1105, 1382]:

$$p(y \mid \alpha, \sigma^2) = \int p(y \mid w, \sigma^2)\, p(w \mid \alpha)\, dw \tag{34.36}$$

$$= \mathcal{N}\big(y \mid 0,\, \sigma^2 I + \tfrac{1}{\alpha}\Phi\Phi^\top\big)\,. \tag{34.37}$$

The corresponding log marginal likelihood is

$$\log p(y \mid \alpha, \sigma^2) = -\tfrac{1}{2} y^\top C^{-1} y - \tfrac{1}{2}\log|C| - \tfrac{N}{2}\log(2\pi), \tag{34.38}$$

with covariance

$$C = \sigma^2 I + \tfrac{1}{\alpha}\Phi\Phi^\top. \tag{34.39}$$

Here the hyperparameters θ correspond to choices in the basis functions $\phi(x;\theta)$, which parameterize the prior over functions. By maximizing the marginal likelihood,

$$(\hat\theta, \hat\alpha, \hat\sigma^2) = \arg\max_{\theta,\, \alpha>0,\, \sigma^2>0} \log p(y \mid \alpha, \sigma^2, \theta), \tag{34.40}$$

we jointly learn the symmetry parameter, weight-prior precision, and observation-noise variance from the marginal likelihood. Additional basis hyperparameters, such as length scales, can be included in the same optimization. Thus, even in this shallow example, marginal likelihood provides a clear, analytic demonstration of how inductive bias, encoded via parameterization of the basis functions, can be selected in a principled way.

34.6.1 Example: Flip symmetry.

Consider one-dimensional inputs $x \in \mathbb{R}$ and let $\phi(x)$ consist of basis functions, such as radial basis functions RBFs [708], centered at a set of locations with fixed length scales. A flip symmetry $x \mapsto -x$ can be embedded by symmetrizing the basis (through relaxed group averaging):

$$\phi'(x;\theta) = (1-\theta)\,\phi(x) + \theta\,\tfrac{1}{2}\big[\phi(x) + \phi(-x)\big], \tag{34.41}$$

where $\theta \in [0,1]$ controls the degree of symmetry. For $\theta = 0$, the basis is unconstrained; for $\theta = 1$, the basis is fully flip-symmetric; and intermediate values interpolate between the two. In this way, θ plays the role of the hyperparameter that governs symmetry in the prior over functions. Equivalently, decomposing the original basis into its symmetric and antisymmetric components gives

$$\phi'(x;\theta) = \phi_{\text{sym}}(x) + (1-\theta)\phi_{\text{asym}}(x), \tag{34.42}$$

and therefore

$$\phi'(x;\theta) - \phi'(-x;\theta) = (1-\theta)\big[\phi(x) - \phi(-x)\big]. \tag{34.43}$$

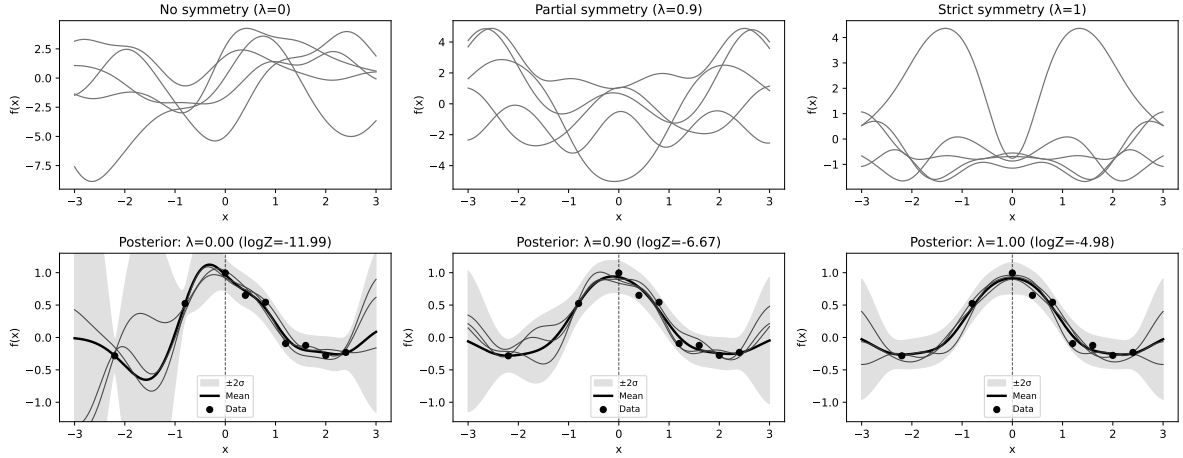


Figure 34.5: Priors and posteriors as θ suppresses antisymmetric variation; exact flip symmetry occurs at $\theta = 1$.

Thus, $\theta = 1$ enforces exact flip symmetry, while intermediate values continuously shrink, but do not eliminate, antisymmetric variation.

Figure 34.5 shows how the prior over functions and resulting posteriors change as θ increases. Larger θ values increasingly concentrate the function prior near the flip-symmetric subspace by shrinking its antisymmetric component. Exact symmetry is enforced only at $\theta = 1$, whereas smaller values retain varying degrees of asymmetric flexibility. The choice of θ is therefore central to determining how strongly symmetry is expressed in the posterior.

As illustrated in Figure 34.6, the marginal likelihood favors different degrees of symmetry for the two displayed datasets. For the symmetric case, the evidence curve favors large θ, and the posterior mean is approximately flip-symmetric. For the asymmetric case, the evidence peaks near $\theta = 0$, allowing the posterior to retain asymmetric variation. Taken together, these figures demonstrate that even in this simple setting, the marginal likelihood provides a principled mechanism for inferring whether symmetries should be imposed, and to what degree. Instead of manually building the inductive bias into the architecture or basis—which can improve generalisation when the bias is correct—the marginal likelihood can select the appropriate symmetry strength from the training data.

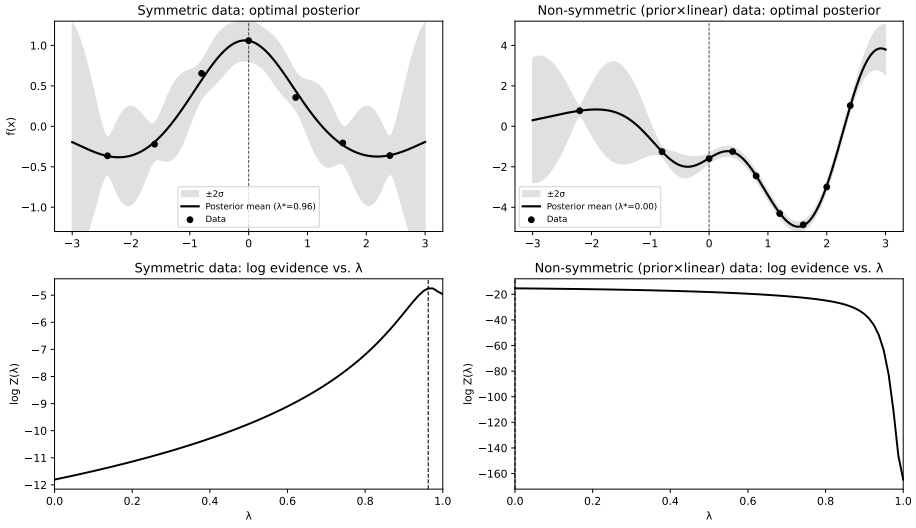


Figure 34.6: Effect of θ on posterior inference and the log marginal likelihood. **Left:** symmetric data. **Right:** asymmetric data. Top: data and posterior means at the evidence-optimal θ. Bottom: $\log Z(\theta)$.

34.7 Learning invariance in a shallow network

We now move beyond closed-form basis-function models to shallow neural-network classifiers, for which the non-conjugate likelihood makes the marginal likelihood intractable. We therefore use a variational bound whose complexity-sensitive objective can guide both weight and symmetry learning [1661]. The construction also connects group averaging of inputs to transformations and sharing of first-layer weights, as used in convolutional architectures [318, 914, 994].

34.7.1 Transformation-averaged function construction

We begin with a non-invariant base network

$$g_w : \mathbb{R}^{D_{\text{in}}} \to \mathbb{R}^K, \tag{34.44}$$

parameterised by w. A straightforward way to achieve (aprpoiximate) invariance is to average its outputs over a learnable transformation distribution $p_\theta(T)$:

$$f_w^\theta(x) = \mathbb{E}_{T\sim p_\theta(T)}[g_w(Tx)]\,. \tag{34.45}$$

Concentration at the identity recovers g_w, while coverage of more relevant transformations, rather than merely larger coordinate variance, typically promotes stronger approximate invariance. Strict invariance is the special case in which $p_\theta(T)$ is uniform over the relevant finite or compact group (its normalised Haar measure), so that the function remains exactly unchanged under further group actions.

Combined with a weight prior $p(w)$, this construction induces the function prior

$$p(f \mid \theta) = \int \delta(f - f_w^\theta) p(w)\,\mathrm{d}w, \tag{34.46}$$

so θ controls the type and strength of invariance. In practice, the transformation integral is approximated by

$$f_w^\theta(x) \approx \frac{1}{M}\sum_{m=1}^{M} g_w(T^{(m)}x), \qquad T^{(m)} \sim p_\theta(T). \tag{34.47}$$

Gradient-based learning of the inductive-bias hyperparameters requires both a differentiable marginal-likelihood objective and a differentiable sampling procedure for $T \sim p_\theta(T)$. The next section constructs such a procedure for affine transformations.

34.7.2 Example: affine transformations of 2D images

To make symmetry learning concrete, we parameterise local transformations from the affine group acting on 2D images. Using infinitesimal *generators* $G_i \in \mathbb{R}^{3\times 3}$, a transformation is represented by the 3×3 homogeneous coordinate matrix

$$A(\alpha) = \exp\Big(\sum_{i=1}^{6} \alpha_i G_i\Big) = \begin{pmatrix} a_{11}(\alpha) & a_{12}(\alpha) & t_1(\alpha) \\ a_{21}(\alpha) & a_{22}(\alpha) & t_2(\alpha) \\ 0 & 0 & 1 \end{pmatrix}. \tag{34.48}$$

Here, $\alpha = (\alpha_1, \ldots, \alpha_6)$ contains generator coordinates for rotation, scaling, shear, and translation. The matrix entries $a_{jk}(\alpha)$ and $t_j(\alpha)$ are the resulting functions of these coordinates after exponentiation. To obtain a bounded distribution over transformations that matches the range parameterisation used in the experiments, we sample

$$\epsilon_i \sim \text{Uniform}(-1, 1), \tag{34.49}$$

$$\alpha_i = \theta_i \epsilon_i, \qquad \theta_i \geq 0. \tag{34.50}$$

The parameter θ_i is therefore the maximum magnitude of the coefficient associated with generator G_i. Using the *reparameterisation trick* [892], the sampled transformation is

$$A(\theta, \epsilon) = A\big(\alpha(\theta, \epsilon)\big) = \exp\Big(\sum_{i=1}^{6} \theta_i \epsilon_i G_i\Big). \tag{34.51}$$

When $\theta = 0$, the transformation is the identity. Increasing a component θ_i expands the range of transformations along the corresponding generator, typically promoting stronger approximate invariance in that direction. This construction allows gradients to flow through both w and θ, enabling joint optimisation of the network parameters and the invariance parameters. It is commonly used in invariance learning [120, 815, 1661].

The matrix $A(\alpha)$ transforms pixel coordinates in homogeneous form. Writing $u = (u_1, u_2)$ for a pixel location and $\tilde{u} = [u_1, u_2, 1]^\top$, the induced action on an image x is the pullback

$$(T_\alpha x)(u) = x\big(\pi\big(A(\alpha)^{-1}\tilde{u}\big)\big), \tag{34.52}$$

where $\pi : \mathbb{R}^3 \to \mathbb{R}^2$ returns the first two coordinates. Off-grid arguments are evaluated by bilinear or bicubic interpolation. For fixed α, this interpolation is linear in x, so T_α may be represented as a matrix in $\mathbb{R}^{D_{\text{in}}} \times D_{\text{in}}$; discretisation makes the group relations only approximate.

The same construction has a weight-space reading. Let $W_1 \in \mathbb{R}^{H \times D_{\text{in}}}$ be the first-layer weight matrix, so the linear map is $x \mapsto W_1 x$. Identifying T with its matrix representation in $\mathbb{R}^{D_{\text{in}}} \times D_{\text{in}}$,

$$W_1(Tx) = (W_1 T)\, x, \tag{34.53}$$

and therefore

$$\mathbb{E}_{T \sim p_\theta(T)}\big[W_1(Tx)\big] = \Big(\mathbb{E}_{T \sim p_\theta(T)}[W_1 T]\Big)x. \tag{34.54}$$

Averaging over input transformations is thus equivalent to averaging the orbit of transformed first-layer weights $\{W_1 T\}$. For translations, that orbit is implemented by convolution followed by pooling [318, 994]; Section 34.9.1 revisits this connection in the context of learning equivariances.

34.7.3 Variational objective for a shallow network

We now turn from constructing the transformation-averaged model to learning its invariance parameters. For classification we apply a softmax to the averaged logits,

$$p(y \mid x, w, \theta) = \operatorname{softmax}\big(f_w^\theta(x)\big)_y, \tag{34.55}$$

and select θ by maximising the marginal likelihood $\log p(\mathcal{D} \mid \theta)$, as in the basis-function setting. Because this integral is intractable for neural classifiers, we use a last-layer variational bound: a posterior over the final-layer weights, with the first-layer weights and invariance parameters treated as deterministic hyperparameters [1661].

Concretely, take the shallow logit network

$$g_w(x) = W_2\, \sigma(W_1 x), \tag{34.56}$$

with the same transformation average $f_w^\theta(x) = \mathbb{E}_{T \sim p_\theta(T)}[g_w(Tx)]$. Treating W_1 as a model hyperparameter and placing a prior only on W_2 yields the conditional function prior

$$p(f \mid W_1, \theta) = \int \delta\big(f - f_{W_1, W_2}^\theta\big)\, p(W_2)\, \mathrm{d}W_2 \tag{34.57}$$

and marginal likelihood

$$p(y \mid X, W_1, \theta) = \int p(y \mid X, W_1, W_2, \theta) p(W_2)\, \mathrm{d}W_2. \tag{34.58}$$

We use a Gaussian prior $p(W_2)$ and a Gaussian variational posterior $q_\Omega(W_2)$, with a separate covariance block for each output class, while optimising W_1 and θ as deterministic hyperparameters. The transformation-exact ELBO is

$$\mathcal{L}_{\text{exact}}(W_1, \theta, \Omega) = \sum_{n=1}^{N} \mathbb{E}_{q_\Omega(W_2)}\Big[\log \operatorname{softmax}\big(\mathbb{E}_{T \sim p_\theta(T)}[g_w(Tx_n)]\big)_{y_n}\Big] \tag{34.59}$$

$$- \operatorname{KL}\big(q_\Omega(W_2) \,\|\, p(W_2)\big). \tag{34.60}$$

For a finite number M of transformation samples, define

$$\mathcal{L}_M(W_1, \theta, \Omega) = \sum_{n=1}^{N} \mathbb{E}_{\substack{W_2 \sim q_\Omega(W_2), \\ T_n^{(m)} \overset{\text{iid}}{\sim} p_\theta(T),\ m=1,\dots,M}} \left[\log \operatorname{softmax}\left(\frac{1}{M}\sum_{m=1}^{M} W_2 \sigma(W_1 T_n^{(m)} x_n)\right)_{y_n}\right] \tag{34.61}$$

$$- \operatorname{KL}\big(q_\Omega(W_2) \,\|\, p(W_2)\big). \tag{34.62}$$

	Test accuracy (%)			
Model	Fully rotated MNIST	Translated MNIST	Scaled MNIST	Original MNIST
MLP	90.35	89.34	96.61	98.10
MLP + Rotation (VI)	98.05	94.08	97.62	98.64
MLP + Translation (VI)	93.59	**97.87**	97.98	98.76
MLP + Scale (VI)	93.80	94.30	98.06	98.35
MLP + Affine (VI)	**98.14**	97.66	**98.31**	**98.93**

Table 34.1: Test accuracy (%) of shallow ReLU multilayer perceptrons (MLPs) with different candidate invariance families on four MNIST variants. Adapted from van der Ouderaa and van der Wilk [1661].

Since $\log \operatorname{softmax}(z)_y$ is concave in z, Jensen's inequality gives

$$\mathcal{L}_M(W_1, \theta, \Omega) \leq \mathcal{L}_{\text{exact}}(W_1, \theta, \Omega) \leq \log p(y \mid X, W_1, \theta). \tag{34.63}$$

The first inequality is the finite-transformation Jensen gap, whereas the second is the variational gap. For $M = 1$, this recovers the single-transformation bound; under the usual integrability conditions, $\mathcal{L}_M$ converges to $\mathcal{L}_{\text{exact}}$ as $M \to \infty$. Optimising W_1 jointly with θ and Ω corresponds to empirical Bayes optimisation of the deterministic model hyperparameters alongside approximate inference over W_2.

Let $W_2^{(\ell)} \sim q_\Omega(W_2)$ and $T_n^{(\ell,m)} \sim p_\theta(T)$. An unbiased estimator of $\mathcal{L}_M$ based on L weight samples is

$$\widehat{\mathcal{L}}_M(W_1, \theta, \Omega) = \frac{1}{L} \sum_{\ell=1}^{L} \sum_{n=1}^{N} \log \operatorname{softmax}\left(\frac{1}{M} \sum_{m=1}^{M} W_2^{(\ell)} \sigma(W_1 T_n^{(\ell,m)} x_n) \right)_{y_n} \tag{34.64}$$

$$- \operatorname{KL}\big(q_\Omega(W_2) \,\|\, p(W_2)\big). \tag{34.65}$$

This estimator provides a practical stochastic objective for joint training. With minibatches, the sum over data points is replaced by a suitably rescaled minibatch sum. Reparameterising samples from $q_\Omega(W_2)$ and $p_\theta(T)$ provides pathwise gradients for Ω and θ, while W_1 is differentiated directly. Thus, W_1, Ω, and θ can be optimised jointly using standard stochastic-gradient methods. Increasing L reduces estimator variance, while increasing M improves the finite-transformation approximation. This is the objective used in the experiments below.

34.7.4 Experiments: learning useful invariances

We now evaluate whether the variational objective (i) improves predictive performance, (ii) learns interpretable transformation distributions, and (iii) provides a useful symmetry-learning signal beyond point-estimate cross-entropy.

Predictive performance

We compare learned-invariance models with a non-invariant multilayer perceptron (MLP). Each model is trained separately on fully rotated, translated, scaled, and original MNIST; Table 34.1 reports the resulting test accuracies (see 1661 for experimental details and corresponding CIFAR-10 results).

The learned-invariance models outperform the baseline in every setting. The translation-specific model performs best on translated MNIST, while the affine model performs best on the rotated, scaled, and original variants and has the highest average accuracy.

Inspecting learned transformations

The weight-space view also lets us inspect what the model has learned. Figure 34.7 shows samples from the first-layer filter orbits introduced above; variation within each orbit reflects the transformation distribution $p_\theta(T)$ learned for that dataset.

Role of the variational objective

Finally, we isolate the role of the training objective. We compare the VI-based model with the same transformation-averaged architecture trained using point-estimate cross-entropy: $q(W_2)$ is replaced by a point estimate, W_1 and

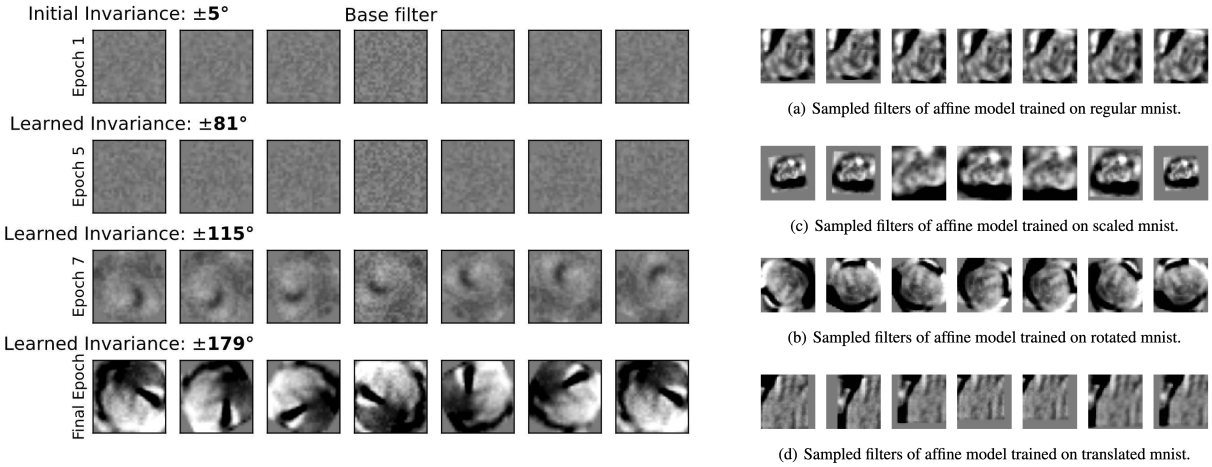


Figure 34.7: Filter-bank samples obtained by transforming the rows of W_1 under the learned distribution $p_\theta(T)$. Adapted from [1661].

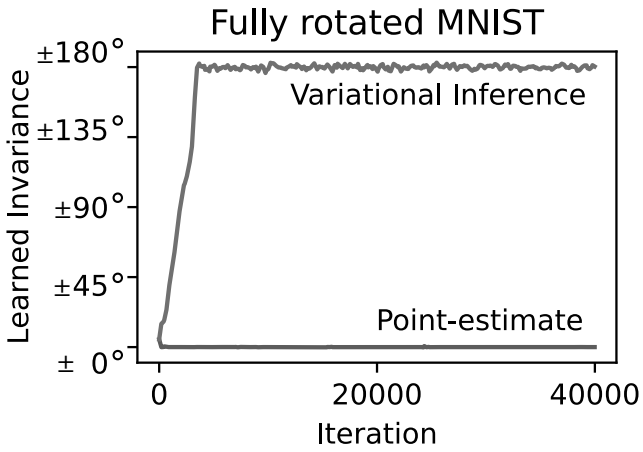


Figure 34.8: Predicted invariance over training iterations on fully rotated MNIST under point-estimate cross-entropy and variational inference. Adapted from [1661].

W_2 are optimised directly, and the KL term is removed. Figure 34.8 tracks the predicted invariance during training on fully rotated MNIST.
Point-estimate cross-entropy fails to recover the target invariance in this experiment, whereas the VI-based approach succeeds. This does not imply that maximum likelihood can never learn symmetry. Rather, when a sufficiently flexible network already fits the training data, the likelihood may provide little signal for selecting the invariance parameter. The variational objective adds a complexity-sensitive signal through posterior averaging and the KL term.

Together, these experiments show that the variational objective can learn useful, interpretable invariances in a shallow network. In the next chapter, we retain the same transformation-averaged construction but replace last-layer VI with a scalable Laplace approximation to allow invariances to be learned through approximate Bayeisan model selection in *deep* networks.

34.8 Learning invariances in deep neural networks

We now turn to learning invariances in *deep neural networks*. The overall construction of obtaining (approximate/learnable) invariance through group averaging is unchanged, but integrating over all weights to obtain the marginal likelihoood requires a scalable approximation. We use a Laplace approximation together with structured curvature approximations that make learning the invariance parameters θ tractable in deep architectures [812, 1402].

34.8.1 Differentiable marginal likelihood via the Laplace approximation

The shallow model combined transformation-averaged logits and a finite-sample Jensen bound with last-layer variational inference. For deep networks, we retain the same likelihood construction but integrate over all weights w under a prior $p(w \mid \lambda)$, where λ denotes its hyperparameters. With S transformation samples per input,

$$\widehat{f}^{\theta}_{S,w}(x_n) = \frac{1}{S}\sum_{s=1}^{S} g_w(T_n^{(s)} x_n), \qquad T_n^{(s)} \overset{\text{iid}}{\sim} p_\theta, \tag{34.66}$$

and the finite-S expected log likelihood is

$$\ell_S(w,\theta) = \sum_{n=1}^{N} \mathbb{E}_{T_n^{(1:S)}}\left[\log \operatorname{softmax}\left(\widehat{f}^{\theta}_{S,w}(x_n)\right)_{y_n}\right]. \tag{34.67}$$

By the same concavity argument as in the shallow model, ℓ_S lower-bounds the exact transformation-averaged log likelihood pointwise in w and converges to it as S increases.

This finite-S bound induces the evidence surrogate

$$Z_S(\theta,\lambda) = \int \exp\big(\ell_S(w,\theta)\big) p(w \mid \lambda)\, \mathrm{d}w \le p(y \mid X, \theta, \lambda). \tag{34.68}$$

In practice, neural-network optimisation selects a local mode of the finite-S surrogate log joint and evaluates the posterior curvature there:

$$\hat{w}_S(\theta,\lambda) \in \operatorname*{local\,argmax}_{w} \left[\ell_S(w,\theta) + \log p(w \mid \lambda)\right], \tag{34.69}$$

$$H_S = \nabla_w^2 \left[-\ell_S(w,\theta) - \log p(w \mid \lambda)\right]\big|_{w=\hat{w}_S}. \tag{34.70}$$

Treating this mode as dominant and assuming its curvature is positive definite, the single-mode *Laplace approximation* [815] gives

$$\log Z_S(\theta,\lambda) \approx \ell_S(\hat{w}_S,\theta) + \log p(\hat{w}_S \mid \lambda) - \tfrac{1}{2}\log|H_S| + \tfrac{P}{2}\log(2\pi), \tag{34.71}$$

where $P = \dim(w)$. A positive prior precision or explicit damping ensures positive definiteness for the curvature approximation used below. Although $\log Z_S$ lower-bounds the exact marginal likelihood, the subsequent Laplace and curvature approximations need not preserve this bound. Together, however, they provide a tractable differentiable approximation to the marginal likelihood that can be used to learn invariance and prior hyperparameters in deep networks.

34.8.2 Scalable curvature approximations

The remaining computational bottleneck is the curvature term $\log|H_S|$. We approximate it in two stages: first replacing the exact likelihood Hessian by the Generalized Gauss–Newton (GGN) matrix, and then imposing Kronecker-factored structure on each layer block.

GGN for transformation-averaged logits

The GGN drops second derivatives of the network outputs and retains the positive-semidefinite term $J^\top H_L J$ [191]. For one sampled set of transformations, let $J_{n,s} = J_g(T_n^{(s)} x_n; \hat{w}_S)$ and define the averaged Jacobian

$$\overline{J}_n = \left.\frac{\partial \widehat{f}^{\theta}_{S,w}(x_n)}{\partial w^\top}\right|_{w=\hat{w}_S} = \frac{1}{S}\sum_{s=1}^{S} J_{n,s}, \tag{34.72}$$

where J_g is the Jacobian of the base-network logits. The sampled GGN likelihood curvature is

$$\widehat{H}_{\mathrm{GGN},S} = \sum_{n=1}^{N} \overline{J}_n^\top H_{L,n} \overline{J}_n, \tag{34.73}$$

where $H_{L,n}$ is the negative log-likelihood Hessian with respect to the averaged logits. Because the averaged Jacobian enters quadratically, this curvature retains cross-transformation contributions. Current transformation samples provide a stochastic estimate that remains differentiable with respect to θ; the prior curvature is added below.

Kronecker factorisation

KFAC makes the sampled GGN scalable by using a block-diagonal approximation across layers and Kronecker factors within each block [1136, 1402]. For a fully connected layer, $J_{n,s}^{(\ell)} = {a_{n,s}^{(\ell-1)}}^\top \otimes B_{n,s}^{(\ell)}$, where $a_{n,s}^{(\ell-1)}$ are the bias-augmented input activations and $B_{n,s}^{(\ell)}$ is the output-to-preactivation Jacobian. We approximate

$$\overline{J}_n^{(\ell)} \approx {\overline{a}_n^{(\ell-1)}}^\top \otimes \overline{B}_n^{(\ell)}, \tag{34.74}$$

where bars denote averages over the S transformation samples. This replaces an average of Kronecker products by a Kronecker product of averages and therefore discards activation–Jacobian correlations across samples. Applying the usual KFAC factorisation across data points gives

$$\widehat{H}^{(\ell)}_{\mathrm{GGN},S} = \sum_{n=1}^{N} {\overline{J}_n^{(\ell)}}^\top H_{L,n} \overline{J}_n^{(\ell)} \approx N A^{(\ell)} \otimes G^{(\ell)}, \tag{34.75}$$

with

$$A^{(\ell)} = \frac{1}{N}\sum_{n=1}^{N} \overline{a}_n^{(\ell-1)} {\overline{a}_n^{(\ell-1)}}^\top, \qquad G^{(\ell)} = \frac{1}{N}\sum_{n=1}^{N} {\overline{B}_n^{(\ell)}}^\top H_{L,n} \overline{B}_n^{(\ell)}. \tag{34.76}$$

For convolutional layers, KFAC additionally factorises over spatial sites.

34.8.3 Practical differentiable Laplace objective

We now combine this curvature approximation with a separate Gaussian prior precision $\lambda_\ell > 0$ for each layer. Let $P_\ell = \dim(w_\ell)$ and $p(w_\ell \mid \lambda_\ell) = \mathcal{N}(w_\ell \mid 0, \lambda_\ell^{-1} I)$. Its normalised log density is

$$\log p(w_\ell \mid \lambda_\ell) = \tfrac{P_\ell}{2}\log\lambda_\ell - \tfrac{\lambda_\ell}{2}\|w_\ell\|^2 - \tfrac{P_\ell}{2}\log(2\pi). \tag{34.77}$$

The prior curvature adds $\lambda_\ell I$ to the sampled KFAC block:

$$\widehat{H}^{(\ell)}_{\mathrm{KFAC},S} = N A^{(\ell)} \otimes G^{(\ell)} + \lambda_\ell I_{P_\ell}. \tag{34.78}$$

If $a_i^{(\ell)}$ and $g_j^{(\ell)}$ are the eigenvalues of $A^{(\ell)}$ and $G^{(\ell)}$, the block-diagonal log determinant is

$$\log|\widehat{H}_{\mathrm{KFAC},S}| = \sum_{\ell=1}^{L_{\mathrm{layers}}} \sum_{i=1}^{D_\ell} \sum_{j=1}^{K_\ell} \log\Big(N a_i^{(\ell)} g_j^{(\ell)} + \lambda_\ell\Big), \tag{34.79}$$

where L_{layers} is the number of layers, $D_\ell = \dim(\overline{a}_n^{(\ell-1)})$, and K_ℓ is the preactivation dimension, so that $P_\ell = D_\ell K_\ell$ when biases are represented by augmented activations.

Substituting this log determinant into the Laplace expression yields

$$\log \widehat{Z}_S^{\mathrm{KFAC}}(\theta,\lambda) = \ell_S(\hat{w}_S,\theta) + \log p(\hat{w}_S \mid \lambda) - \tfrac{1}{2}\log|\widehat{H}_{\mathrm{KFAC},S}| + \tfrac{P}{2}\log(2\pi). \tag{34.80}$$

This is the differentiable Laplace (*Diff. Laplace*) objective optimised in practice. The factors $A^{(\ell)}$ and $G^{(\ell)}$ depend on θ through the sampled transformations, while the prior terms depend directly on λ. Training alternates between maximising $\ell_S(w,\theta) + \log p(w \mid \lambda)$ over w and updating θ and λ through the objective above.

The log determinant does not decompose over data points, so naively differentiating it would require retaining the full-data computation graph. For any positive-definite curvature block $H(\theta)$,

$$\nabla_\theta \log|H(\theta)| = \mathrm{vec}\big(H(\theta)^{-1}\big)^\top \nabla_\theta \,\mathrm{vec}\big(H(\theta)\big). \tag{34.81}$$

This separates a preconditioner from a vector–Jacobian product. In practice, the full-data KFAC factors are first accumulated batch by batch without retaining a computation graph; their eigendecompositions are then computed and held fixed as the preconditioner. The remaining derivatives of $A^{(\ell)}$ and $G^{(\ell)}$ are sums over data points, so, conditional on this preconditioner, they admit unbiased minibatch estimates [815]. This gives a memory-efficient stochastic gradient of the KFAC–Laplace surrogate. During each hyperparameter update, $\hat{w}_S$ is also treated as fixed; accounting for its implicit dependence on (θ,λ) would require implicit differentiation or unrolling.

34.8.4 Experiments: learned input invariance

We next test whether Diff. Laplace recovers known transformations in controlled benchmarks and remains practical for larger convolutional networks [815].

Recovering dataset transformations

We train on rotated, translated, and affine-transformed variants of MNIST and monitor the learned invariance parameters θ. In Figure 34.9, parameters associated with the transformations used to generate each dataset increase toward their target ranges, while irrelevant candidates remain near zero.

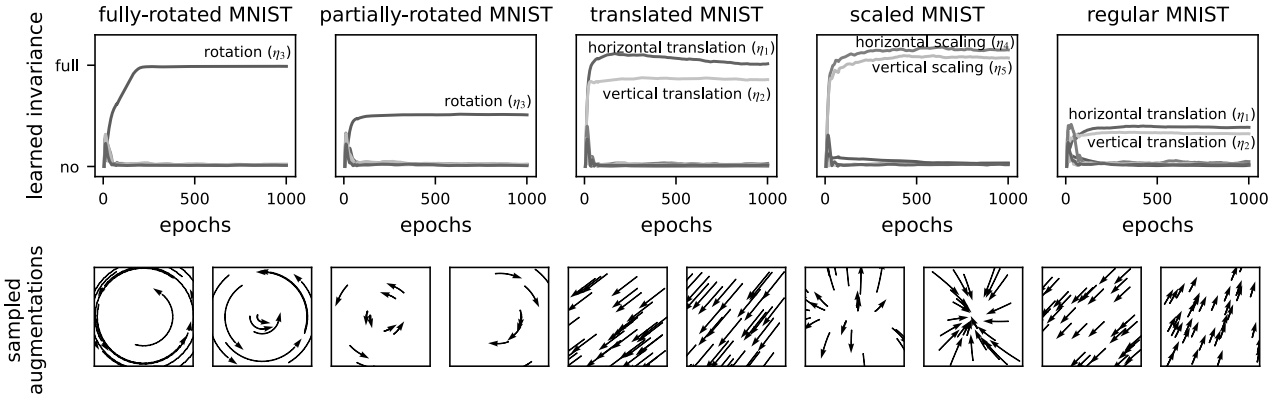


Figure 34.9: Trajectories of affine-invariance parameters $\theta = (\theta_1, \ldots, \theta_6)$, corresponding to horizontal translation, vertical translation, rotation, horizontal scaling, vertical scaling, and shear, on transformed MNIST variants. Two samples from each learned transformation distribution are shown as flow fields. Adapted from Immer et al. [815].

Dataset	Network	Method	Fully Rotated	Partially Rotated	Translated	Scaled	Original
CIFAR-10	ResNet	non-invariant	54.16 ± 0.40	59.90 ± 0.12	69.65 ± 0.16	66.06 ± 0.13	74.13 ± 0.51
		Augerino	75.40 ± 0.19	74.76 ± 0.34	73.71 ± 0.31	72.07 ± 0.09	79.03 ± 1.04
		Diff. Laplace (**ours**)	$\mathbf{79.50 \pm 0.62}$	$\mathbf{77.71 \pm 0.46}$	$\mathbf{79.21 \pm 0.17}$	$\mathbf{76.03 \pm 0.15}$	$\mathbf{84.19 \pm 0.76}$

Table 34.2: Test accuracy (%) of ResNet models on transformed CIFAR-10 variants, reported as mean ± standard error over three seeds. Adapted from Immer et al. [815].

Method	Test accuracy (%)
non-invariant	85.17 ± 0.39
Augerino	87.67 ± 0.08
Diff. Laplace (**ours**)	$\mathbf{91.98 \pm 0.04}$

Table 34.3: CIFAR-10 test accuracy (%) for Wide ResNet input-invariance learning with KFAC. Adapted from Immer et al. [815].

Large ResNet models

Table 34.2 compares Diff. Laplace with a non-invariant ResNet and Augerino across five CIFAR-10 variants. Diff. Laplace obtains the highest mean test accuracy in every column, providing evidence that the method can learn useful input-level invariances in this architecture.

The Wide ResNet result in Table 34.3 provides a second scalability example using KFAC curvature. Here the transformation distribution is shared by the network as a whole and influences all layers through training; the next section instead assigns symmetry-controlling priors to individual layers.

Together, these experiments show that the same approximate model-selection objective can learn input-level invariances in both shallow and deep networks, including the ResNet architectures considered here.

34.8.5 Empirical comparison with explicit regularisation

We now compare Diff. Laplace with explicit regularisation on partially rotated MNIST. A representative method is Augerino [120], which optimises prediction loss under a learnable augmentation distribution and favours broader transformation ranges through

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{CE}} + \beta\, R(\theta), \tag{34.82}$$

where $\mathcal{L}_{\text{CE}}$ is the summed cross-entropy, $R(\theta)$ decreases as the encouraged transformation ranges broaden, and β controls the trade-off. If mean rather than summed cross-entropy is used, its factor of $1/N$ must be absorbed into β. Both approaches require a candidate transformation family, but explicit regularisation additionally requires the design of R and the choice of β.

Figure 34.10 shows that the invariance learned with explicit regularisation is sensitive to β, with some settings producing unstable runs. Diff. Laplace instead learns the target invariance without a separate regularisation coefficient.

The second comparison tests sensitivity to parameterisation. Let $\boldsymbol{\eta} = (\eta_1, \ldots, \eta_6) \in \mathbb{R}^6_{>0}$ denote the same affine-range parameters previously represented by θ, and define the reciprocal coordinates $\zeta_i = 1/\eta_i$ componentwise. These coordinate systems describe the same strictly positive transformation ranges, with zero-range boundaries approached as limits.

Figure 34.11 shows that the explicit regulariser behaves differently after this coordinate change. By contrast, the exact marginal likelihood is unchanged by a one-to-one reparameterisation that preserves the induced probabilistic model. These experiments expose the two practical drawbacks of explicit regularisation discussed earlier: sensitivity to both the regularisation strength and the chosen coordinates. Diff. Laplace avoids a separate penalty, although its numerical approximation and optimisation can still exhibit coordinate dependence.

34.9 Learning layer-wise equivariances in deep neural networks

The previous sections learned input-level *invariance* by averaging predictions over transformations. Intermediate representations can instead be made *equivariant*: transforming the input then produces a corresponding transformation of the features. Such constraints can improve data efficiency and generalisation [210, 318, 914]. This raises a natural question: can we also learn *equivariances* in a principled, data-driven way?

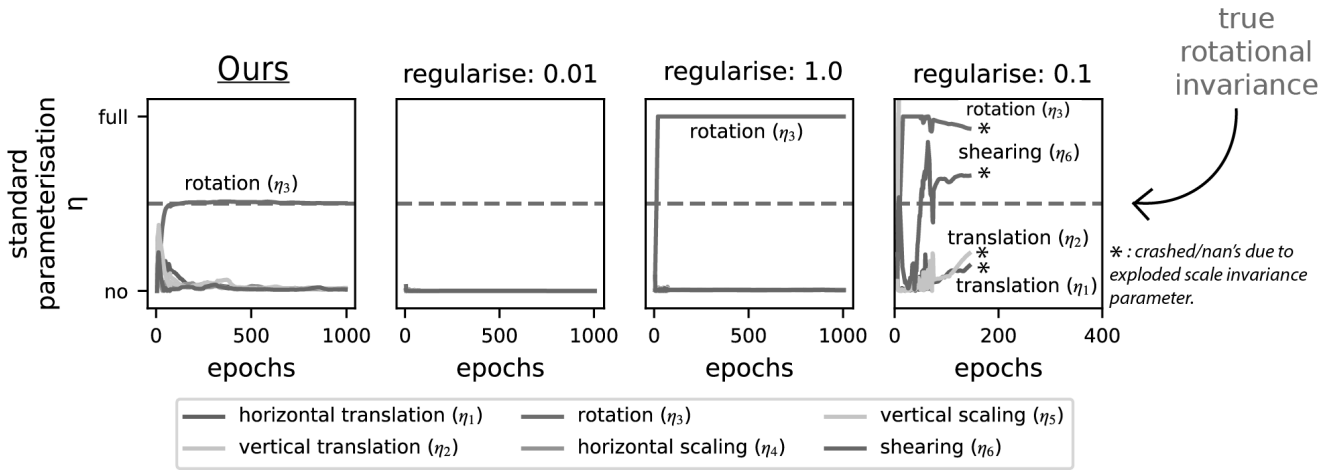


Figure 34.10: Learned affine-invariance parameters on partially rotated MNIST for Diff. Laplace and three explicit-regularisation strengths β. The dashed line marks the target rotational invariance; asterisks mark unstable runs. Adapted from Immer et al. [815].

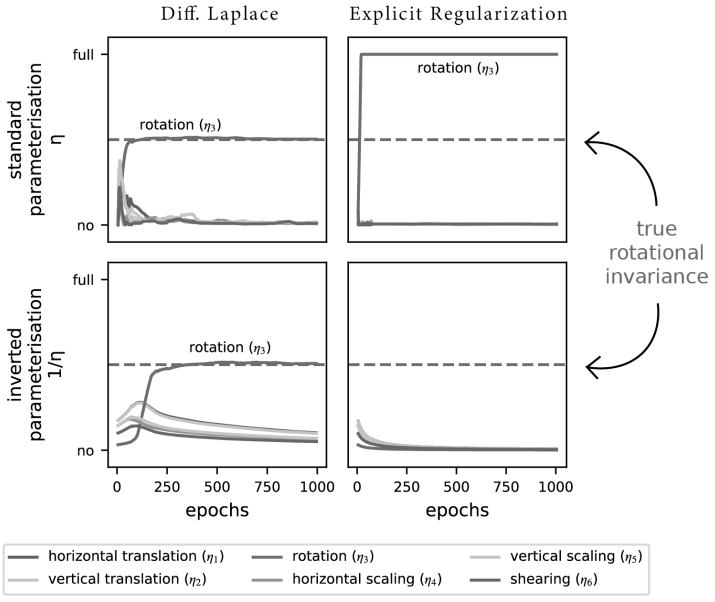


Figure 34.11: Learned affine-invariance parameters on partially rotated MNIST under the standard coordinates $\boldsymbol{\eta}$ (top) and componentwise reciprocal coordinates $\boldsymbol{\zeta} = 1/\boldsymbol{\eta}$ (bottom). Diff. Laplace is shown on the left and explicit regularisation on the right; dashed lines mark the target rotational invariance. Adapted from Immer et al. [815].

To make the degree of equivariance learnable, the model family must contain both equivariant and non-equivariant functions. We first review exact equivariance and two ways to relax it, then use Diff. Laplace to learn the relative prior strength of equivariant and non-equivariant pathways in each layer.

34.9.1 Exact equivariance and group convolution

Let $f(x) = Wx$, with $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, be a linear layer. Suppose $g \in G$ acts on the input and output spaces through T^g_{in} and T^g_{out}, respectively. Equivariance means that f intertwines these actions [319]:

$$f(T^g_{\text{in}} x) = T^g_{\text{out}} f(x) \;\; \forall x \quad \Longleftrightarrow \quad W T^g_{\text{in}} = T^g_{\text{out}} W, \qquad \forall\, g \in G. \tag{34.83}$$

If the same representation acts on both spaces, this becomes the commutation constraint

$$W T^g = T^g W, \qquad \forall\, g \in G, \tag{34.84}$$

which defines a linear subspace of equivariant weight matrices.

Example: translations in 2D

Consider a feature field x indexed by $u \in \mathbb{Z}^2$, with channel indices suppressed. A translation by $t \in \mathbb{Z}^2$ acts as

$$(T^t x)[u] = x[u - t]. \tag{34.85}$$

On a finite periodic grid, the commutation constraint (34.84) requires the weights to depend only on relative displacement [319]:

$$W[u, u'] = \kappa[u' - u], \qquad (Wx)[u] = \sum_{u'} \kappa[u' - u] x[u']. \tag{34.86}$$

Thus, convolution is the most general linear translation-equivariant map under these assumptions. Equivariance enforces weight sharing but not locality; ordinary convolutional networks add locality by restricting κ to finite support.

Equivariance to general groups

Let the input and output feature fields be indexed by elements of G and carry the left regular action

$$(T^h x)(g) = x(h^{-1} g), \qquad h, g \in G. \tag{34.87}$$

For finite G, every equivariant linear map between such fields is a *group convolution*. In a notation that also covers locally compact continuous groups,

$$(Wx)(g) = \int_G \kappa(g^{-1} g') x(g') \, \mathrm{d}\mu(g'). \tag{34.88}$$

Here μ is a left Haar measure; for finite or countable groups, the integral is interpreted as a sum with counting measure. The kernel depends only on the relative group element $g^{-1} g'$, and left invariance of μ makes the operator equivariant. For continuous groups, the kernel and features are assumed sufficiently integrable for the operator to be well defined. With multiple channels, κ is matrix-valued.

Example: roto-translations in 2D

To include rotations, consider

$$G = \mathbb{R}^2 \rtimes C_N, \tag{34.89}$$

where C_N is a finite cyclic rotation group. An image $x_0(u)$ is initially indexed only by position. A lifting layer correlates it with rotated filters to produce a group-indexed feature field $x(u, r)$, where $u \in \mathbb{R}^2$ and $r \in C_N$. A group element $(t, \rho) \in G$ then acts as

$$(T^{(t,\rho)} x)(u, r) = x\big(R_\rho^{-1}(u - t),\, \rho^{-1} r\big). \tag{34.90}$$

Here, R_ρ is the planar rotation associated with ρ.

Replacing C_N by continuous rotations gives $\mathrm{SE}(2) = \mathbb{R}^2 \rtimes \mathrm{SO}(2)$. On a pixel grid, either action requires discretisation and, for off-grid coordinates, interpolation. The resulting numerical equivariance is therefore approximate.

Substituting this action into the general group-convolution form gives

$$(Wx)(u, r) = \sum_{r' \in C_N} \int_{\mathbb{R}^2} \kappa\big(R_r^{-1}(u' - u),\, r^{-1}r'\big)\, x(u', r')\, \mathrm{d}u'. \tag{34.91}$$

The kernel depends only on displacement in the output orientation frame and on relative orientation. It therefore shares weights jointly across positions and orientations, giving exact equivariance in the continuous-domain model and approximate equivariance after discretisation.

34.9.2 Relaxation through nonstationary kernels

Strict group convolutions depend only on the relative element $g^{-1}g'$. To relax equivariance, we additionally allow the kernel to depend on the absolute output coordinate g [1659]. Suppressing channel indices, a general kernel operator between group-indexed features is

$$(Wx)(g) = \int_G k(g, g')x(g')\, \mathrm{d}\mu(g'). \tag{34.92}$$

For finite or countable groups, the integral is replaced by a sum. Since the change from (g, g') to the relative–absolute coordinates $(g^{-1}g', g)$ is one-to-one, we may write

$$k(g, g') = \tilde{k}(g^{-1}g', g), \tag{34.93}$$

$$(Wx)(g) = \int_G \tilde{k}(g^{-1}g', g)x(g')\, \mathrm{d}\mu(g'). \tag{34.94}$$

If $\tilde{k}$ is independent of its second argument, this reduces to an exactly equivariant group convolution. Dependence on g permits symmetry-breaking variation across absolute output coordinates. A practical parameterisation combines relative and absolute embeddings through a mixer:

$$\tilde{k}(g^{-1}g', g) = m\big(\psi_{\mathrm{rel}}(g^{-1}g'), \psi_{\mathrm{abs}}(g)\big)\,, \tag{34.95}$$

where ψ_{rel} and ψ_{abs} are learned embeddings and m is a mixing function. Exact equivariance is recovered when m ignores its second argument. The expressivity of the resulting nonstationary kernel depends on the embeddings and mixer.

Smooth relative and absolute features

Random Fourier features provide a smooth parameterisation of the relative and absolute embeddings [1363]. Let $\nu_r \sim \mathcal{N}(0, I_d)$, $r = 1, \ldots, R$, be fixed random projection directions and let $\xi : G \to \mathbb{R}^d$ be a coordinate embedding. For a frequency scale ω, define

$$\boldsymbol{\varphi}_\omega(g) = \frac{1}{\sqrt{R}}\big[\cos(\omega\nu_1^\top \xi(g)),\ \sin(\omega\nu_1^\top \xi(g)),\ \ldots,\ \cos(\omega\nu_R^\top \xi(g)),\ \sin(\omega\nu_R^\top \xi(g))\big]^\top. \tag{34.96}$$

Convenient coordinate maps for translations, rotations, and roto-translations are

$$\xi(t) = t, \qquad \xi(\rho) = [\cos\rho, \sin\rho]^\top, \tag{34.97}$$

$$\xi(t, \rho) = [t_1, t_2, \cos\rho, \sin\rho]^\top. \tag{34.98}$$

The periodic angle coordinates ensure that the features are well defined on $\mathrm{SO}(2)$. Other groups may require group-specific coordinate maps or bases [318, 520, 914].

Suppressing input–output channel indices, write

$$\psi_{\mathrm{rel}}(g^{-1}g') = \boldsymbol{w}_{\mathrm{rel}}^\top \boldsymbol{\varphi}_{\omega_{\mathrm{rel}}}(g^{-1}g'), \tag{34.99}$$

$$\psi_{\mathrm{abs}}(g) = \boldsymbol{w}_{\mathrm{abs}}^\top \boldsymbol{\varphi}_{\omega_{\mathrm{abs}}}(g). \tag{34.100}$$

The random directions are shared across channels, while separate coefficients $\boldsymbol{w}^{(\mathrm{rel})}_{c_{\mathrm{out}},c_{\mathrm{in}}}$ and $\boldsymbol{w}^{(\mathrm{abs})}_{c_{\mathrm{out}},c_{\mathrm{in}}}$ are learned for each channel pair.

The two scales have different roles. Changing ω_{rel} changes the smoothness of the filter as a function of $g^{-1}g'$, while preserving equivariance. The absolute features can instead break equivariance: when $\omega_{\mathrm{abs}} = 0$, $\boldsymbol{\varphi}_{\omega_{\mathrm{abs}}}(g)$ is constant in g, whereas nonzero values allow the kernel to vary across absolute output coordinates. The realised amount of symmetry breaking also depends on the absolute coefficients and the mixer m; exact equivariance is recovered whenever the resulting kernel is independent of its absolute argument.

Additive implementation and its limitation

For computational efficiency, the mixer can be replaced by the additive special case $\tilde{k}(g^{-1}g', g) = \psi_{\mathrm{rel}}(g^{-1}g') + \psi_{\mathrm{abs}}(g)$. For input feature $x_{c_{\mathrm{in}}}(g')$ and output feature $y_{c_{\mathrm{out}}}(g)$, this gives

$$y_{c_{\mathrm{out}}}(g) = \sum_{c_{\mathrm{in}}=1}^{C_{\mathrm{in}}} \int_G \Big[\boldsymbol{w}^{(\mathrm{rel})\top}_{c_{\mathrm{out}},c_{\mathrm{in}}} \boldsymbol{\varphi}_{\omega_{\mathrm{rel}}}(g^{-1}g') + \boldsymbol{w}^{(\mathrm{abs})\top}_{c_{\mathrm{out}},c_{\mathrm{in}}} \boldsymbol{\varphi}_{\omega_{\mathrm{abs}}}(g) \Big] x_{c_{\mathrm{in}}}(g')\, \mathrm{d}\mu(g'). \tag{34.101}$$

The relative term is equivariant, while the absolute term can break the symmetry. Because its basis does not depend on the input element g', the absolute contribution factorises as

$$\sum_{c_{\mathrm{in}}=1}^{C_{\mathrm{in}}} b_{c_{\mathrm{out}},c_{\mathrm{in}}}(g) \int_G x_{c_{\mathrm{in}}}(g')\, \mathrm{d}\mu(g'). \tag{34.102}$$

Here $b_{c_{\mathrm{out}},c_{\mathrm{in}}}(g) := \boldsymbol{w}^{(\mathrm{abs})\top}_{c_{\mathrm{out}},c_{\mathrm{in}}} \boldsymbol{\varphi}_{\omega_{\mathrm{abs}}}(g)$. This is a low-rank global correction: it can break equivariance, but it cannot represent a general nonstationary map. Full flexibility requires joint relative–absolute features, for example a tensor-product basis depending on both $g^{-1}g'$ and g. Figure 34.12 illustrates the broader family of joint nonstationary translation kernels; the additive construction above represents only a restricted subset.

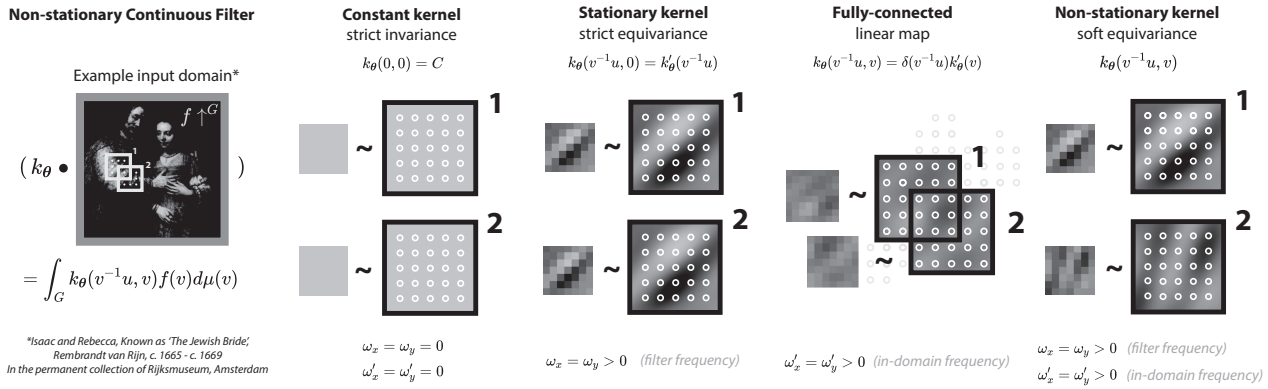


Figure 34.12: Translation kernels obtained by varying the relative (filter) and absolute-coordinate (output-domain, in our convention) frequencies. For the general joint nonstationary kernel, the limiting cases include a constant invariant kernel, a stationary equivariant kernel, a fully connected linear map, and a softly equivariant kernel. The additive parameterisation above is a restricted special case. Adapted from van der Ouderaa et al. [1659].

34.9.3 Relaxation through residual pathways

Nonstationary kernels relax equivariance through absolute-coordinate dependence. An alternative—and the one used in the experiments below—is to add a non-equivariant residual pathway to an equivariant one [521]:

$$y^{(\ell)} = W^{(\ell)}_{\mathrm{eq}} x^{(\ell-1)} + W^{(\ell)}_{\mathrm{non}} x^{(\ell-1)}. \tag{34.103}$$

For a finite group, $W^{(\ell)}_{\mathrm{eq}}$ has a group-convolutional kernel $\boldsymbol{w}_{\mathrm{eq}}[g^{-1}g']$, while $W^{(\ell)}_{\mathrm{non}}$ may use an unrestricted kernel $\boldsymbol{w}_{\mathrm{non}}[g, g']$. For translations, these are convolutional and fully connected pathways, respectively.

Independent zero-mean Gaussian priors on the two parameter blocks define a *residual pathway prior*. Let $\lambda_{\ell,\mathrm{non}}$ denote the precision of the non-equivariant parameters. Increasing this precision suppresses the residual

pathway, with exact equivariance recovered in the infinite-precision limit. The original method fixes or tunes the relative prior variances and trains with a MAP objective; here, Diff. Laplace instead learns the layer-wise precisions through approximate marginal likelihood.

An unrestricted residual pathway has $\mathcal{O}(C_{\text{in}}C_{\text{out}}|G|^2)$ parameters, or $\mathcal{O}(D^4)$ on a $D \times D$ grid. The experiments therefore use factorised or sparsified variants that retain non-equivariant flexibility with far fewer parameters [1658]. Other approaches also relax equivariance [1456, 1600, 1657, 1710], but the residual decomposition is particularly convenient here because separate priors can directly control the two pathways.

34.9.4 Experiments: learned layer-wise equivariance

Using these scalable residual pathways, we ask whether marginal-likelihood optimisation can mitigate symmetry misspecification, select an architecture from training data that generalises well, reveal where layer-wise symmetry constraints are retained, and choose among candidate equivariant groups.

Mitigating symmetry misspecification

To isolate symmetry misspecification, digits are placed in different image quadrants. In the first task, the target is digit identity and is invariant to position. In the second, the target contains both the digit and its quadrant, so enforcing position invariance discards information required for prediction. Conversely, a fully connected model retains position but fails to exploit the genuine symmetry of the first task. Table 34.4 compares these fixed alternatives with a model whose pathway precisions are selected by Diff. Laplace.

Example:

(3, upper left)

		MAP		Diff. Laplace		
		FC	CONV	FC	CONV	Learned
Task symmetry	Prediction target	112.7 M	0.4 M	112.7 M	0.4 M	1.8 M
Strict symmetry	*digit*	95.73	99.38	97.04	99.23	98.86
Partial symmetry	*digit, quadrant*	95.10	24.68	95.46	24.50	99.00

Table 34.4: Test accuracy (%) of fully connected (FC), convolutional (CONV) with an invariant readout, and learned-equivariance models on digit-only and digit-and-quadrant prediction tasks. Parameter counts appear below the model names. Adapted from van der Ouderaa et al. [1658].

The convolutional model performs well when the target is position invariant but falls to near-chance accuracy when the quadrant must be predicted. The fully connected model avoids this misspecification but does not exploit the symmetry as effectively. By adjusting the non-equivariant pathway precision, Diff. Laplace achieves high accuracy on both tasks.

Selecting a generalising architecture from training data

On CIFAR-10, we compare non-equivariant, strictly convolutional, and adaptive FC+CONV architectures under MAP and Diff. Laplace training. The pathway precisions are selected using only the training data; the test metrics in Table 34.5 are used afterward to assess how well the selected architecture generalises.

	Learnable		MAP		Diff. Laplace	
Layer	equivariance	# Params	Test NLL (↓)	Test Acc. (↑)	Test NLL (↓)	Test Acc. (↑)
Non-equivariant (FC path)		0.8 M	5.614	50.21	1.496	46.18
Strict equivariance (CONV path)		0.2 M	**0.604**	**82.53**	**0.562**	**81.29**
Learned equivariance (FC+CONV paths)	x	1.0 M	5.485	58.80	**0.538**	**81.55**

Table 34.5: CIFAR-10 test negative log likelihood (NLL) and accuracy (%) for non-equivariant, strictly equivariant, and learned-equivariance models under MAP and Diff. Laplace. Adapted from van der Ouderaa et al. [1658].

Under Diff. Laplace, the adaptive model improves test NLL from 0.562 to 0.538 and accuracy from 81.29% to 81.55% relative to strict equivariance. The same FC+CONV parameterisation performs substantially worse under MAP. Thus, its success comes from the marginal-likelihood objective selecting an effective architecture from the training data, rather than merely from making both pathways available.

Inspecting the learned layer-wise architecture

We next inspect which symmetry constraints the marginal likelihood selected in each layer. The learned non-equivariant-path precisions $\lambda_{\ell,\text{non}}$ exceed 10^6 in most layers, indicating strong shrinkage. Figure 34.13 visualises the resulting architecture using relative effective parameter counts, a curvature-based diagnostic of how much each pathway remains active after prior shrinkage [1105]. This diagnostic is not itself a direct measure of functional contribution. Convolutional pathways dominate until the spatial resolution reaches 1×1; at that resolution, an FC pathway is equivalent to a pointwise convolution and does not necessarily indicate broken spatial equivariance.

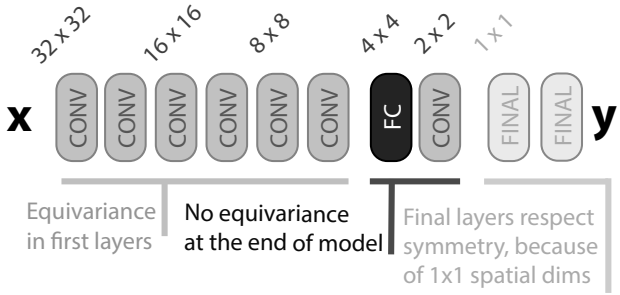


Figure 34.13: Dominant pathway across the depth of a learned FC+CONV network, determined from relative effective parameter counts. Convolutional paths dominate until the spatial resolution reaches 1×1. Adapted from van der Ouderaa et al. [1658].

Selecting among candidate symmetry groups

The preceding experiments supplied one equivariant pathway and one unrestricted alternative. More generally, each layer can contain several candidates with separately learned prior strengths. Here, FC is unrestricted, CONV is translation equivariant, and GCONV is a group-convolutional pathway equivariant to translations and the discrete rotations C_N, corresponding to $\mathbb{R}^2 \rtimes C_N$. Table 34.6 reports predictive performance, approximate evidence, and the relative effective parameter counts used to inspect which pathways remain active.

		MAP		Learned with Diff. Laplace			Relative effective parameter count					
Dataset	# Params	Test NLL (↓)	Test accuracy (%)	Test NLL (↓)	Test accuracy (%)	$\widehat{-\log p(\mathcal{D})}/N$ (↓)	FC (%)		CONV (%)		GCONV (%)	
MNIST	1.2 M	0.172	97.59	0.023	**99.21**	0.328	10	(0-46)	15	(0-98)	75	(1-100)
Translated MNIST	1.2 M	0.812	90.78	0.053	**98.27**	0.216	0	(0-0)	23	(0-99)	77	(1-100)
Rotated MNIST	1.2 M	0.819	91.02	0.136	**95.55**	0.896	8	(0-20)	8	(0-47)	83	(47-100)
CIFAR-10	1.2 M	3.540	68.33	0.552	**80.94**	0.926	0	(0-1)	44	(0-99)	56	(0-100)
Rotated CIFAR-10	1.2 M	5.953	48.30	1.236	**55.68**	1.630	4	(0-22)	14	(0-41)	82	(58-99)

Table 34.6: Selection among fully connected (FC), convolutional (CONV), and rotation-equivariant group-convolutional (GCONV) pathways. In $\widehat{-\log p(\mathcal{D})}/N$, the hat denotes the Laplace approximation and division by N normalises the value per datum; the final columns give the mean (minimum–maximum) relative effective parameter count across layers. Adapted from van der Ouderaa et al. [1658].

The small FC effective counts indicate that non-equivariant flexibility is strongly suppressed, particularly on translated MNIST and CIFAR-10. GCONV receives the largest allocation throughout and is most dominant on the rotated datasets, while CONV remains active on several tasks. These diagnostics are consistent with the method selecting among the supplied pathways and adjusting their strengths by layer; they do not imply discovery of groups outside the candidate architecture.

Together, the experiments show that marginal-likelihood optimisation can relax a misspecified symmetry, select a generalising architecture using only training data, and reveal where different symmetry constraints remain active within that architecture.

34.10 Conclusion and Outlook

In this chapter, we have argued that without Bayesian model selection, Bayesian deep learning is missing out on one of its key abilities: the ability to improve generalisation by selecting an appropriate inductive bias. We have reviewed the Bayesian approach to this problem, and argued that the challenges in making this work are not with

the conceptual Bayesian framework, but instead with finding usable approximations to the prescribed quantities. We could make progress towards creating adaptive neural network architectures, even with crude approximations to the marginal likelihood.

Whether adaptive architectures can provide benefits at the scale of modern foundation models, and with modern architectures like transformers, remains an open question. Perhaps model size is the next important question to tackle [1122, 1189] and perhaps Bayesian approaches may yet again have something to offer [1315].

Chapter 35

Interpretable and identifiable deep generative models via Bayesian pyramids and their variants

Deep generative models (DGMs) have become central tools for representation learning and achieved great empirical success across many applications. However, the theoretical foundations for parameter and representation identifiability, which is a prerequisite for interpretability and reliable Bayesian inference, still remain limited. This chapter develops a unified perspective on *identifiable and interpretable deep generative models with discrete latent layers*, synthesising several recently proposed models. These frameworks establish explicit structural and algebraic conditions under which the multi-layer latent structure and its parameters are uniquely determined from the observable distribution. Such identifiability guarantees provide solid foundations for reliably applying these models in high-stakes applications, such as education, medicine, and health care. We outline the theoretical results, sketch intuition via algebraic and tensor-based reasoning, and discuss implications for interpretable Bayesian deep learning.

35.1 Introduction

DGMs such as deep belief networks, variational autoencoders, and generative adversarial networks have achieved striking empirical performance in modelling complex data. Yet, from a statistical perspective, most existing DGMs are *nonidentifiable*: distinct parameter values can induce the same distribution of observables. This ambiguity impedes interpretability, obstructs posterior consistency, and causes label-switching and other pathologies in Bayesian inference.

Identifiability roughly means that unknown quantities, such as parameters and latent structures, can be uniquely identified from the observed data. It is a crucial prerequisite for reliable scientific discoveries and reproducible statistical analyses. Although classical latent variable models such as factor analysis and latent class models have a well-established identifiability theory, their deep counterparts remain poorly understood.

This chapter presents several recently proposed identifiable DGMs based on discrete latent structures: the *Bayesian Pyramid* model for multivariate categorical data, Bayesian latent class regression with interpretable binary profiles, Bayesian deep generative models for multiplex networks, and *Deep Discrete Encoders* with many discrete latent layers for rich data types coming from flexible distributions. Together, these models demonstrate that Bayesian deep learning with many random latent layers can be both *interpretable and identifiable* under explicit structural conditions.

35.2 Bayesian Pyramids

Multivariate categorical data are routinely collected in biomedical and social sciences. It is of great importance to build interpretable models that perform dimension reduction and uncover meaningful latent structures from such discrete data. Identifiability is a fundamental requirement for interpretable modelling and inference in such scenarios, yet is challenging to address when there are complex latent structures. *Bayesian Pyramids* proposed by Gu and Dunson [679] are a class of identifiable multilayer/deep discrete latent structure models for multivariate

categorical data.

35.2.1 Model Structure and Identifiability Results

The Bayesian Pyramid is a directed graphical model for multivariate categorical data. In a Bayesian Pyramid, the bottom layer is the observed data vector $\boldsymbol{y} \in [d_1] \times [d_2] \times \cdots \times [d_p]$, and many binary latent variables are organised in multiple layers above the observed layer, with the shallowest layer being $\boldsymbol{\alpha}^{(1)}$ and the deepest layer being $\boldsymbol{\alpha}^{(D)}$. Directed arrows occur only between adjacent layers in the top-down direction: $\boldsymbol{\alpha}^{(D)} \to \boldsymbol{\alpha}^{(D-1)} \to \cdots \to \boldsymbol{\alpha}^{(1)} \to \boldsymbol{y}$. For a random variable X, define $\mathrm{Pa}(X)$ to be the index set of the variables with directed arrows pointing to X (called the *parents* of X). Each layer $\boldsymbol{\alpha}^{(\ell)} \in \{0,1\}^{K_\ell}$ represents binary latent attributes. Formally, for any subject i, the joint distribution of all random variables factorises according to the graphical model as follows:

$$\mathbb{P}(\boldsymbol{y}_i, \{\boldsymbol{\alpha}_i^{(m)}\}, z_i^{(D)}) = \mathbb{P}(\boldsymbol{y}_i \mid \boldsymbol{\alpha}_i^{(1)}) \prod_{m=1}^{D-2} \mathbb{P}(\boldsymbol{\alpha}_i^{(m)} \mid \boldsymbol{\alpha}_i^{(m+1)}) \mathbb{P}(\boldsymbol{\alpha}_i^{(D-1)} \mid z_i^{(D)}) \mathbb{P}(z_i^{(D)}); \tag{35.1}$$

$$\mathbb{P}(\boldsymbol{y}_i \mid \boldsymbol{\alpha}_i^{(1)}) = \prod_{j=1}^{p} \mathbb{P}\Big(y_{i,j} \,\Big|\, \boldsymbol{\alpha}_{i,\mathrm{Pa}(j)}^{(1)}\Big), \quad \mathbb{P}(\boldsymbol{\alpha}_i^{(m)} \mid \boldsymbol{\alpha}_i^{(m+1)}) = \prod_{k=1}^{K_m} \mathbb{P}\Big(\alpha_{i,k}^{(m)} \,\Big|\, \boldsymbol{\alpha}_{i,\mathrm{Pa}(k^{(m)})}^{(m+1)}\Big). \tag{35.2}$$

Importantly, each variable (either in the observed layer or in any latent layer) connects only to a small subset of latent nodes above it. To summarise such parent–child dependence relationships, we introduce *graphical matrices* $\mathbf{G}^{(1)}, \ldots, \mathbf{G}^{(D-1)}$. Here, $\mathbf{G}^{(1)} = (g_{jk}^{(1)}) \in \{0,1\}^{p \times K_1}$ encodes the bipartite graph between the $\boldsymbol{y}$ layer and the $\boldsymbol{\alpha}^{(1)}$ layer, where $g_{jk}^{(1)} = 1$ if y_j is a child of $\alpha_k^{(1)}$ and 0 otherwise. Similarly, for each $m = 2, \ldots, D-1$, the matrix $\mathbf{G}^{(m)} = (g_{k\ell}^{(m)}) \in \{0,1\}^{K_{m-1} \times K_m}$ describes the bipartite graph between the layers $\boldsymbol{\alpha}^{(m-1)}$ and $\boldsymbol{\alpha}^{(m)}$, where $g_{k\ell}^{(m)} = 1$ if $\alpha_k^{(m-1)}$ is a child of $\alpha_\ell^{(m)}$ and 0 otherwise. Bayesian Pyramids are identifiable under the following transparent algebraic conditions on the shapes of these matrices $\{\mathbf{G}^{(m)}\}$.

Theorem 35.1 (Theorem 4 in Gu and Dunson [679]). *Consider a Bayesian Pyramid where the numbers $K_1, \ldots, K_{D-1}$ are known. Suppose each binary graphical matrix $\mathbf{G}^{(m)}$ of size $K_{m-1} \times K_m$ (size $p \times K_1$ if $m = 1$) takes the following form after some row permutation,*

$$\mathbf{G}^{(m)} = \begin{pmatrix} \mathbf{I}_{K_m} \\ \mathbf{I}_{K_m} \\ \mathbf{I}_{K_m} \\ \mathbf{G}^{(m),\star} \end{pmatrix}, \quad m = 1, \ldots, D-1, \tag{35.3}$$

where $\mathbf{G}^{(m),\star}$ denotes a submatrix of $\mathbf{G}^{(m)}$ with an arbitrary form. Then the following parameters are identifiable up to a latent variable permutation within each layer: the distribution of the deepest latent variable $z^{(D)}$, the conditional probability table of each variable (including observed and latent) given its parents, and also the graphical matrices $\{\mathbf{G}^{(m)}; m = 1, \ldots, D-1\}$.

A natural consequence of identifiability is posterior consistency, presented in the next theorem.

Theorem 35.2 (Theorem 3 in Gu and Dunson [679]). *Denote the collection of model parameters by Θ. Suppose the prior distributions for the parameters all have full support around the true values. If the parameters satisfy the proposed strict identifiability conditions, then*

$$\mathbb{P}(\Theta \in \mathcal{N}_\epsilon^c(\Theta^0) \mid \boldsymbol{y}_1, \ldots, \boldsymbol{y}_n) \to 0 \text{ almost surely,}$$

where $\mathcal{N}_\epsilon^c(\Theta^0)$ is the complement of an ϵ-neighbourhood of the true Θ^0 in the parameter space.

Theorem 35.2 implies that under an identifiable model and with appropriately specified priors, the posterior concentrates around the true parameters rather than a manifold of equivalent parameterisations. This property guarantees reproducible inference and interpretability of the latent structure – a feature absent in many neural network-based DGMs.

35.2.2 Bayesian Inference for Two-layer Bayesian Pyramids

Here $z_i \in [B]$ denotes the subject-specific deepest latent class, with $\mathbb{P}(z_i = b) = \tau_b$, and $\eta_{k,b} = \mathbb{P}(\alpha_{i,k}^{(1)} = 1 \mid z_i = b)$. In this section, as an important special case, we focus on a two-latent-layer model as illustrated in Figure 35.1

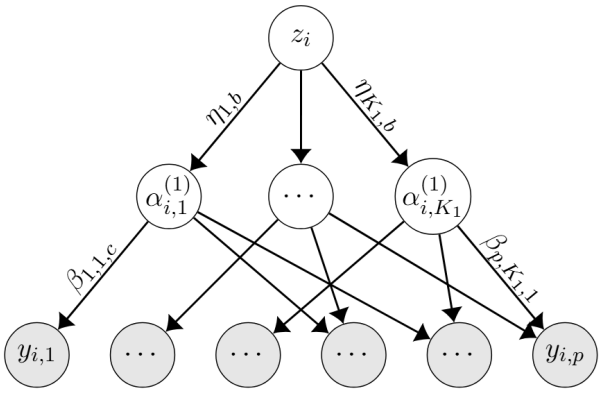


Figure 35.1: Two-latent-layer model. Latent variables $z_i, \alpha_{i,1}, \ldots, \alpha_{i,K_1}$ and observed variables $y_{i,1}, \ldots, y_{i,p}$ are subject-specific, and model parameters $\mathbf{G}^{(1)}, \boldsymbol{\beta}, \boldsymbol{\tau}, \boldsymbol{\eta}$ are population quantities.

with depth $D = 2$ and use a Bayesian approach to infer the latent structure and model parameters. Assume

$$\mathbb{P}(y_{i,j} = c \mid \boldsymbol{\alpha}_i^{(1)} = \boldsymbol{\alpha}) = \frac{\exp\left(\beta_{j,c,0} + \sum_{k=1}^{K_1} \beta_{j,c,k} g_{j,k}^{(1)} \alpha_k\right)}{\sum_{m=1}^{d_j} \exp\left(\beta_{j,m,0} + \sum_{k=1}^{K_1} \beta_{j,m,k} g_{j,k}^{(1)} \alpha_k\right)}, \quad j \in [p],\ c \in [d_j];$$

$$\mathbb{P}(\boldsymbol{\alpha}_i^{(1)} = \boldsymbol{\alpha}) = \sum_{b=1}^{B} \tau_b \prod_{k=1}^{K_1} \eta_{k,b}^{\alpha_k} (1 - \eta_{k,b})^{1-\alpha_k}, \quad \boldsymbol{\alpha} \in \{0,1\}^{K_1}. \tag{35.4}$$

Here we assume $\beta_{j,d_j,0} \equiv \beta_{j,d_j,k} \equiv 0$ for all $k \in [K_1]$. The parameter $\beta_{j,c,k}$ can be viewed as the weight associated with the potential directed edge from latent variable $\alpha_k^{(1)}$ to observed variable y_j, for response category c. The $\beta_{j,c,k}$ only impacts the likelihood if there is an edge from $\alpha_k^{(1)}$ to y_j with $g_{j,k}^{(1)} = 1$. Denote the collection of β parameters as $\boldsymbol{\beta}$. (35.4) specifies a usual latent class model in the second latent layer with B latent classes for the K_1-dimensional latent vector $\boldsymbol{\alpha}^{(1)}$. This layer of the model has latent class proportion parameters $\boldsymbol{\tau} = (\tau_1, \ldots, \tau_B)^\top$ and conditional probability parameters $\boldsymbol{\eta} = (\eta_{k,b})_{K_1 \times B}$. The full set of model parameters across layers is $(\mathbf{G}^{(1)}, \boldsymbol{\beta}, \boldsymbol{\tau}, \boldsymbol{\eta})$.

In addition to the previous *strict identifiability* result in Theorem 35.1 that holds for all parameters in the parameter space, for model (35.4) we can also establish a slightly weaker notion of *generic identifiability* under much more relaxed conditions on $\mathbf{G}^{(1)}$. Roughly speaking, generic identifiability means that there exists a measure-zero subset in the parameter space in which non-identifiability may occur. This notion was popularised by Allman et al. [22] and usually suffices for real data analysis purposes. Before stating the identifiability result, we formally define the allowable constrained parameter space for $\boldsymbol{\beta}$ under a graphical matrix $\mathbf{G}^{(1)}$ as

$$\Omega(\boldsymbol{\beta};\, \mathbf{G}^{(1)}) = \{\beta_{1:p,\,1:K_1,\,1:(d_j-1)};\ \beta_{j,c,k} \neq 0 \text{ if } g_{j,k}^{(1)} = 1;\ \text{and } \beta_{j,c,k} = 0 \text{ if } g_{j,k}^{(1)} = 0\}. \tag{35.5}$$

Proposition 35.1 (Proposition 3 in Gu and Dunson [679]). *Consider model* (35.4) *with* $\boldsymbol{\beta}$ *belonging to* $\Omega(\boldsymbol{\beta};\, \mathbf{G}^{(1)})$ *in* (35.5). *Suppose* $K_1 \geq 2\lceil \log_2 B \rceil + 1$ *and the graphical matrix* $\mathbf{G}^{(1)}$ *can be rewritten as* $\mathbf{G}^{(1)} = (\mathbf{G}_1, \mathbf{G}_2, \mathbf{G}_3, (\mathbf{G}^\star)^\top)^\top$, *where each* $\mathbf{G}_m$ *has size* $K_1 \times K_1$ *and*

$$\mathbf{G}_m = \begin{pmatrix} 1 & * & \cdots & * \\ * & 1 & \cdots & * \\ \vdots & \vdots & \ddots & \vdots \\ * & * & \cdots & 1 \end{pmatrix}, \quad m = 1, 2, 3;$$

i.e., each of $\mathbf{G}_1, \mathbf{G}_2, \mathbf{G}_3$ *has all diagonal entries equal to one while any off-diagonal entry can be either one or zero. Then* $(\mathbf{G}^{(1)}, \boldsymbol{\beta}, \boldsymbol{\tau}, \boldsymbol{\eta})$ *are generically identifiable.*

Bayesian Inference Procedure. We first describe the case where the true number of binary latent variables K_1 in the middle layer is fixed. We impose a Dirichlet prior on $\boldsymbol{\tau}$ and Beta priors on entries of $\boldsymbol{\eta}$. Inferring the latent sparse graph $\mathbf{G}^{(1)}$ is equivalent to inferring the sparsity structure of the continuous parameters β_{jkc}'s. Let

the prior for $\beta_{j,k,0}$ be $N(\mu_0, \sigma_0^2)$, where hyperparameters μ_0, σ_0^2 can be set to give weakly informative priors for the logistic regression coefficients. Specify priors for other parameters as

$$\text{(i) for fixed } K_1,\ k = 1, \ldots, K_1: \quad \begin{aligned} \beta_{jck} \mid (\sigma_{ck}^2,\ g_{j,k} = 1) &\sim N(0, \sigma_{ck}^2); \\ \beta_{jck} \mid (\sigma_{ck}^2,\ g_{j,k} = 0) &\sim N(0,\ \sigma_0^2); \\ \sigma_{ck}^2 &\sim \text{InvGa}(b_{1\sigma}, b_{2\sigma}); \\ \mathbb{P}(g_{j,k} = 1) = 1 - \mathbb{P}(g_{j,k} = 0) &= \gamma; \end{aligned} \tag{35.6}$$

Here σ_0 is a small positive number specifying the "pseudo"-variance, and we take $\sigma_0 = 0.1$ in the numerical studies. Our adoption of a prior variance for $\beta_{j,c,k}$ when $g_{j,k} = 0$ follows a similar spirit as the "pseudo-prior" approach in the Bayesian variable selection literature.

Next, we describe the prior specifications with an unknown number of binary latent variables K_1. Consider an upper bound K_{upper} for K_1. Based on the identifiability conditions in Theorem 35.1 about the shape of $\mathbf{G}^{(1)}_{p \times K_1}$, K_1 is naturally constrained to be at most $p/3$, therefore K_{upper} can be set to $p/3$ or smaller in practice. We adopt a prior that detects redundant binary latent variables by increasingly shrinking the variance of β_{jck}'s as k grows from 1 to K_{upper}. Specifically, letting $\beta_{jck} \sim N(0, \sigma_{ck}^2)$, we put a Cumulative Shrinkage Prior (CUSP) prior [1009] on the variances $\{\sigma_{c1}^2, \sigma_{c2}^2, \ldots, \sigma_{cK_{\text{upper}}}^2\}$ for each category $c \in [d-1]$, where σ_∞^2 is a prespecified small positive number indicating the spike variance for redundant binary latent variables:

$$\text{(ii) for unknown } K_1 < K_{\text{upper}},\ k = 1, \ldots, K_{\text{upper}}:$$

$$\sigma_{ck}^2 \mid \pi_k \sim (1 - \pi_k)\text{InvGa}(b_{1\sigma}, b_{2\sigma}) + \pi_k \delta_{\sigma_\infty^2}; \tag{35.7}$$

$$\pi_k = \sum_{\ell=1}^{k} v_\ell \prod_{m=1}^{\ell-1} (1 - v_m), \tag{35.8}$$

where $\text{InvGa}(b_{1\sigma}, b_{2\sigma})$ refers to the inverse gamma distribution with shape $b_{1\sigma}$ and scale $b_{2\sigma}$, and π_k has a stick-breaking representation as in (35.8) with $v_1, v_2, \ldots, v_{K_{\text{upper}}-1}$ independently following the Beta distribution $\text{Beta}(1, \alpha_0)$. We set $v_{K_{\text{upper}}} \equiv 1$ to truncate the stick-breaking representation at K_{upper}, similarly to Legramanti et al. [1009]. Posterior computation is performed with the data-augmented Gibbs sampler developed in Gu and Dunson [679].

35.2.3 Simulation Results and Real Data Application

Simulation Studies. We perform simulation studies to empirically verify the identifiability results and evaluate the performance of our algorithm. Consider a two-layer Bayesian Pyramid with $p = 20$ observed variables, $d = 4$ response categories for each observed variable, and $K_1 = 4$ binary latent variables in the middle layer and one binary latent variable in the deep layer. Let the true graphical matrix be $\mathbf{G}^{(1)} = (\mathbf{I}_4; \mathbf{I}_4; \mathbf{I}_4; 1100; 0110; 0011; 1001; 1010; 1001; 0101; 1$ which satisfies the strict identifiability condition in Theorem 35.1. Let the intercepts for categories $c = 1, 2, 3$ be $(\beta_{j,1,0}, \beta_{j,2,0}, \beta_{j,3,0}) = (-3, -2, -1)$ for each j; and for any $g_{j,k}^{(1)} = 1$, let the corresponding true main-effect parameters be $\beta_{j,c,k} = 3$ if variable y_j has a single parent and $\beta_{j,c,k} = 2$ if y_j has multiple parents. See Figure 35.2(c) for a heatmap of the sparse matrix of the main-effect parameters $(\beta_{j1k};\ j \in [p],\ k \in [K_1])$ for category $c = 1$.

We use the method with the CUSP prior and specify $K_{\text{upper}} = 7$, because $\lceil p/3 \rceil = 7$ is a natural upper bound for K_1 to satisfy the identifiability condition in Theorem 35.1. As for the hyperparameters in the CUSP prior, we mainly follow the default setting and suggestion in Legramanti et al. [1009] and set $\alpha_0 = 5$ and $\sigma_\infty^2 = 0.07$. First, Figure 35.2 presents posterior means of the main-effect parameters $(\beta_{j1k};\ j \in [p],\ k \in [K_1])$ for category $c = 1$, averaged across simulation replications. This figure demonstrates that the number of latent variables K_1 can be correctly inferred from our algorithm. Furthermore, Figures 35.3 and 35.4 show that as sample size n grows, both the discrete $\mathbf{G}$ and the continuous parameters are estimated consistently. These results empirically confirm the posterior consistency in Theorem 35.2.

Application to the DNA Nucleotide Sequence Data. We apply the proposed method for two-layer Bayesian Pyramids and analyse a real dataset of DNA nucleotide sequences of A, G, C, T: the splice dataset [455] with $n = 3175$ nucleotide sequences, each belonging to one of three types: Exon–Intron junction ("EI"); Intron–Exon junction ("IE"); and Neither EI nor IE ("N"). Figure 35.5 summarises the results and shows our method's impressive ability to extract biologically meaningful latent features. Here, we specify a relatively large number of deep latent classes with $B = 5$ in the Bayesian pyramid and impose a Dirichlet prior with hyperparameters $(0.01, 0.01, \ldots, 0.01)$.

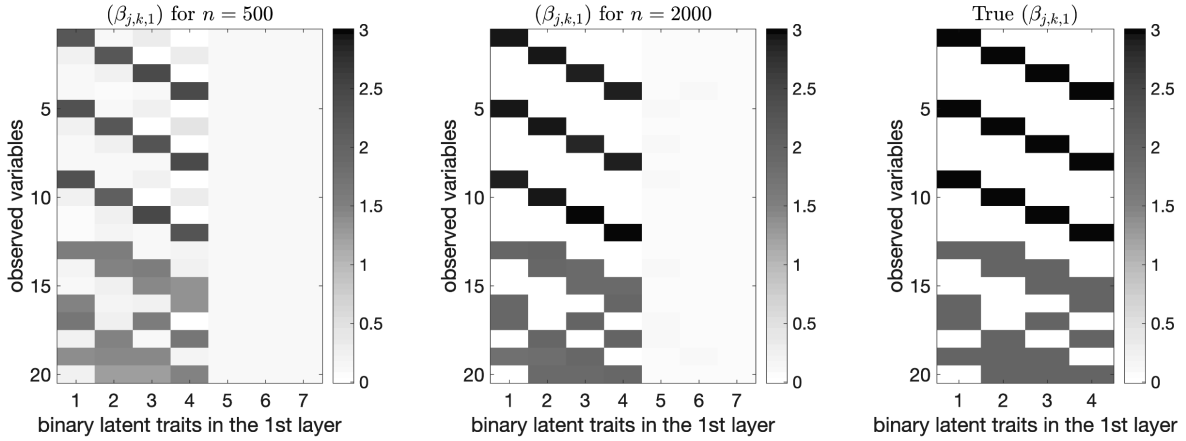


Figure 35.2: Posterior means of the Bayesian Pyramid parameters $\{\beta_{j1k}\}$ for response category $c = 1$ averaged across 50 independent replications, for $n = 500$ or 2000. The value $K_{\text{upper}} = 7$ is used in the CUSP prior for posterior inference, while the true K_1 is 4 as shown in the rightmost plot.

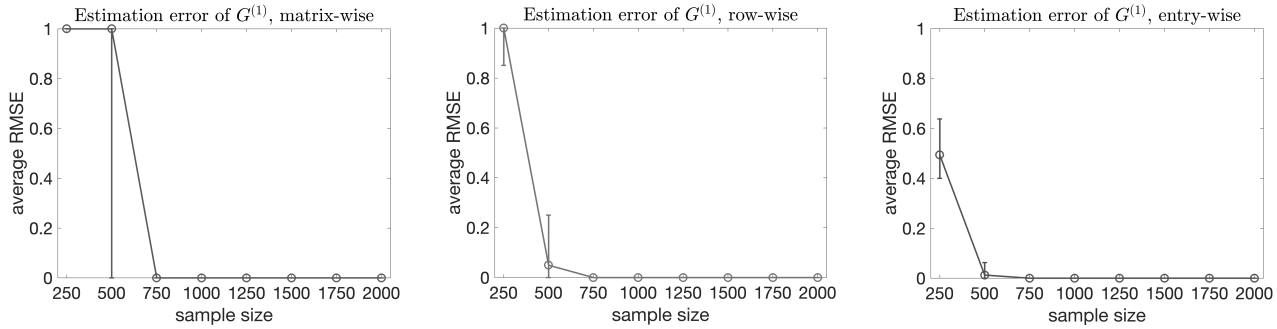


Figure 35.3: Mean estimation errors of the graphical matrix $\mathbf{G}^{(1)}$ at the matrix level (in (a)), row level (in (b)), and entry level (in (c)). The median, 25% quantile, and 75% quantile based on the 50 independent simulation replications are shown by the circle, lower bar, and upper bar, respectively.

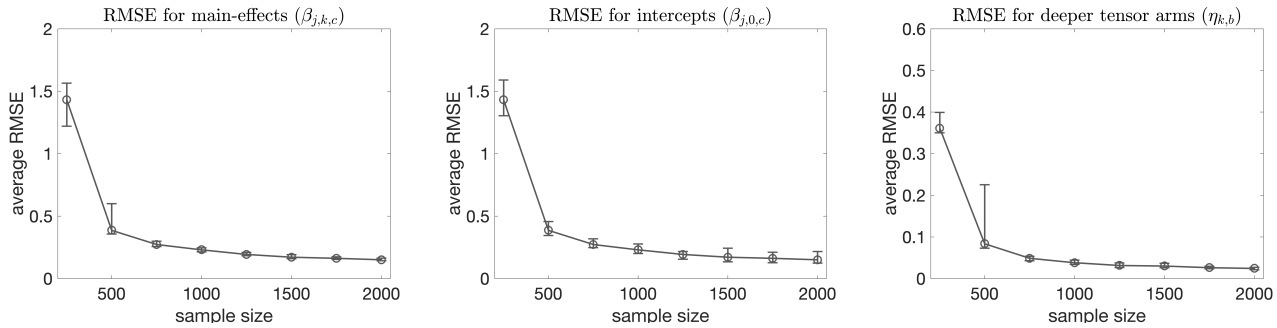


Figure 35.4: Average root-mean-square errors (RMSE) of the posterior means of the main effects $\boldsymbol{\beta}$, intercepts $\boldsymbol{\beta}_0$, and deeper-layer conditional Bernoulli probability parameters $\boldsymbol{\eta}$ versus sample size $n = 250 \cdot i$ for $i \in \{1, 2, \ldots, 8\}$. The median, 25% quantile, and 75% quantile based on the 50 independent simulation replications are shown by the circle, lower bar, and upper bar, respectively.

Interestingly, in this "overfitted" mixture setting, our algorithm is able to effectively empty out three out of five latent classes in the posterior, leading to $B_{\text{eff}} = 2$ meaningful latent classes that separate the EI type from the two other types. By visual inspection, one can already see the learned $\boldsymbol{\alpha}$-layer and z-layer latent features (left two panels of Figure 35.5) match the held-out sequence labels "EI", "IE", "N" provided by biologists (third panel) very well. A downstream classification based on a simple decision tree indeed yields very high accuracy, as shown in the confusion matrix in Figure 35.5. Such accuracy matches that given by fine-tuned convolutional neural networks

in Nguyen et al. [1244].

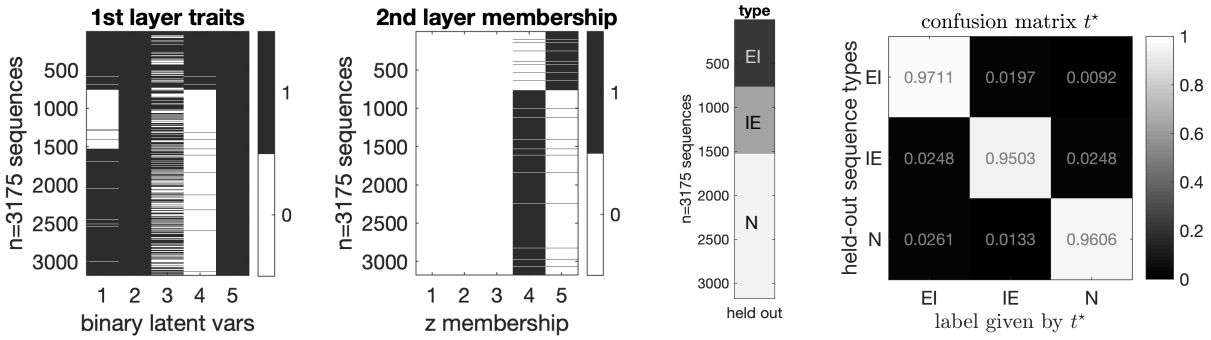


Figure 35.5: Application of two-latent-layer Bayesian Pyramid to DNA nucleotide sequence data [455]. Left to right: $\boldsymbol{\alpha}$-layer latent features; z-layer latent classes; held-out sequence labels; confusion matrix showing the downstream classification results.

35.3 Bayesian Latent Class Regression with Interpretable Binary Profiles

High-dimensional categorical data are prevalent in diverse scientific domains, including ecology, genetics, and the social sciences. For example, species co-occurrence across sampling sites is often represented by binary presence–absence vectors [1275, 1611], DNA sequences can be encoded as categorical strings of nucleotides [1883], and survey responses frequently capture high-dimensional patterns of individual choices and opinions [183]. These datasets are typically accompanied by covariates such as environmental conditions [1721], biological characteristics [28], or demographic information [1676], motivating the development of statistical models that jointly account for categorical outcomes and covariates. As one of the recent advances, Bayesian latent class regression with interpretable binary profiles [1877] leverages multilayer network structures to achieve theoretical guarantees on model identifiability and robust performance in high-dimensional settings. In an application to joint species distribution modelling in ecology, the model also exhibits performance comparable to that of state-of-the-art methods while improving the interpretability of the results.

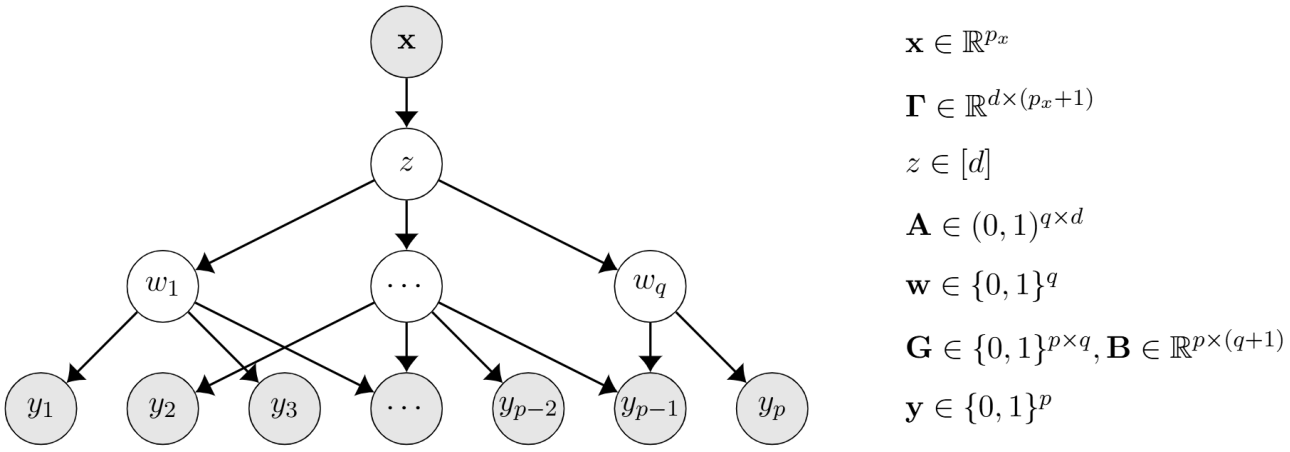


Figure 35.6: Bayesian latent class regression with interpretable binary profiles. Shading/unshading represents observed/latent variables. Covariates $\mathbf{x}$, deep latent class z, binary latent attributes $\mathbf{w}$, and observed variables $\mathbf{y}$ are specific to each sample. Parameters $\mathbf{A} = (\alpha_{j,z})$, $\mathbf{B} = (\beta_{i,j})$, $\boldsymbol{\Gamma} = (\gamma_{h,j})$, and $\mathbf{G} = (g_{i,j})$ are shared across the population and respectively encode the latent-attribute probabilities, observed-layer regression coefficients, covariate effects on the deep classes, and sparse dependency structure.

35.3.1 Model Structure and Identifiability Results

Compared to latent class regression [306, 382, 586, 651, 1121], Bayesian latent class regression with interpretable binary profiles (BLIP) introduces an additional layer of binary latent attributes $\mathbf{w}$ representing interpretable fine-grained traits, as illustrated by the network structure in Figure 35.6. The layer of binary latent attributes $\mathbf{w}$ lies between the layer of observed categorical outcomes $\mathbf{y}$ and the layer of deep latent classes z that captures broader population-level clusters influenced by covariates $\mathbf{x}$.

We let each observed categorical outcome dimension y_i be conditionally independent given the binary attributes $\mathbf{w} = (w_1, \ldots, w_q)$, and further restrict each y_i to depend only on a subset of attributes w_j. This sparse dependency is introduced to improve model interpretability and characterised by a binary matrix $\mathbf{G}$, with its entry $g_{i,j} = 1$ indicating the dependency of y_i on w_j. Using a logit link function, the conditional distribution is modelled as

$$\log \frac{\mathbb{P}(y_i = 1 \mid \mathbf{w}, \mathbf{G}, \mathbf{B})}{\mathbb{P}(y_i = 0 \mid \mathbf{w}, \mathbf{G}, \mathbf{B})} = \beta_{i,0} + \sum_{j=1}^{q} g_{i,j} w_j \beta_{i,j}, \tag{35.9}$$

for logistic regression intercepts $\beta_{i,0}$ and coefficients $\beta_{i,j}$, and $\mathbf{B} := (\beta_{i,j})$. Building on this foundation, we structure the latent attributes $w_1, \ldots, w_q$ by conditional independence on a deeper latent class $z \in \{1, \ldots, d\}$ with Bernoulli distributions:

$$\mathbb{P}(\mathbf{w} \mid z, \mathbf{A}) = \prod_{j=1}^{q} \alpha_{j,z}^{w_j} (1 - \alpha_{j,z})^{1 - w_j}, \tag{35.10}$$

for Bernoulli distribution parameters $\alpha_{j,z} \in (0, 1)$ and $\mathbf{A} := (\alpha_{j,z})$. To incorporate the influence of covariates $\mathbf{x}$, we further allow the deep latent class z to depend on $\mathbf{x}$ through a multinomial logistic regression with class probabilities

$$\mathbb{P}(z \mid \mathbf{x}, \mathbf{\Gamma}) = \frac{\exp(\gamma_{z,0} + \boldsymbol{\gamma}_z^\top \mathbf{x})}{\sum_{h=1}^{d} \exp(\gamma_{h,0} + \boldsymbol{\gamma}_h^\top \mathbf{x})}, \tag{35.11}$$

where $\gamma_{h,0}$ and $\boldsymbol{\gamma}_h$ serve as the regression intercept and coefficients for class h, respectively, and $\mathbf{\Gamma} := (\gamma_{h,0}, \boldsymbol{\gamma}_h)$. To ensure identifiability, class d is designated as the baseline class with its regression intercept and coefficients fixed at zero.

The multilayer model structure improves model expressiveness compared to latent class regression while maintaining clear interpretability. The deep latent class z groups observations into d clusters, while the latent attributes w_j capture the presence or absence of fine-grained latent traits. According to the q latent attributes, the observations can be further partitioned into at most 2^q groups, providing a flexible and expressive cluster structure.

Although resembling the structure of a multilayer neural network, our model distinctively enjoys the property of identifiability. The sparsity $\mathbf{G}$ on the conditional dependency of y_i on w_j plays a pivotal role in establishing identifiability. As in latent class models, our model exhibits a trivial form of non-identifiability caused by label switching [693], arising from permutations over the d deep latent classes and permutations over the q latent attributes. A common practice to handle the label switching issue is through post-processing methods [1562].

Theorem 35.3. *(1) The model parameters are identifiable up to label switching, if the matrix $\mathbf{G}$ contains three distinct identity blocks $\mathbf{I}_q$ and the continuous parameters $\mathbf{A}, \mathbf{B}$ satisfy certain mild conditions.*

(2) The model parameters are almost surely identifiable up to label switching, if $\mathbf{G}$ has distinct columns and contains three distinct blocks, two of which are $q \times q$ with all-ones diagonals and the third has no all-zeros columns.

Theorem 35.3 establishes the identifiability of BLIP model under two notions: The stronger notion in (1) requires all parameters within the parameter space to be identifiable, while the weaker notion in (2) relaxes this requirement to almost every parameter in the parameter space with respect to the base measure. The weaker notion enjoys much weaker conditions on $\mathbf{G}$ while being sufficient for statistical inferences. To distinguish them, the stronger notion of identifiability is typically referred to as strict identifiability, whereas the weaker notion is referred to as generic identifiability. Mathematical definitions of strict/generic identifiability for BLIP model, together with a complete statement of Theorem 35.3 and its proof, can be found in Zhou et al. [1877].

35.3.2 Bayesian Inference

To perform Bayesian inference, we adopt independent weakly informative priors [596] by default, though structured priors can be applied in specific cases. Notably, using established identifiability results, the model is posterior

consistent under mild conditions on the prior distribution and parameter space, providing theoretical guarantees. A simple Gibbs sampler with Pólya–Gamma data augmentation [1337] can be used for posterior computation.

We have incorporated observation-specific covariates $\mathbf{x}$ that impact the distribution of the deep latent class z in (35.11). In many applications, there exists another type of covariates, which we refer to as *meta covariates* $\mathbf{t}_i$, which are associated with each dimension y_i of the observed data [1248]. For example, in ecology, each species has its biological traits; in genetics, each DNA nucleotide has a corresponding location; and in survey design, each question has its semantic features.

We design a hierarchical prior distribution on the sparse dependency matrix $\mathbf{G}$ that encourages y_i with similar meta covariates $\mathbf{t}_i$ to exhibit similar dependency structures on $\mathbf{w}$:

$$g_{i,j} \mid \mathbf{t}_i, \theta_{j,0}, \boldsymbol{\theta}_j \overset{ind}{\sim} \text{Bernoulli}\left(\frac{\exp(\theta_{j,0} + \boldsymbol{\theta}_j^\top \mathbf{t}_i)}{1+\exp(\theta_{j,0} + \boldsymbol{\theta}_j^\top \mathbf{t}_i))}\right), \quad \theta_{j,0} \overset{iid}{\sim} N(0,1), \quad \boldsymbol{\theta}_j \overset{iid}{\sim} N(0, \mathbf{I}). \tag{35.12}$$

Intuitively, we can interpret each $\boldsymbol{\theta}_j$ as a summary of the effects of the meta covariates relevant to w_j, such that a data entry y_i has a higher prior probability of depending on w_j when $\boldsymbol{\theta}_j^\top \mathbf{t}_i$ is larger.

35.3.3 Limiting Behaviour in High-dimensional Clustering

Although influential, traditional latent class models and their extensions such as mixed membership models [680, 1442] struggle in the high-dimensional regime, where these models are prone to over-fragmentation or collapsing to trivial clusters, a manifestation of the curse of dimensionality. Recent theoretical work shows that even well-specified Bayesian mixture models can asymptotically degenerate into singleton clusters as the data dimension grows [256, 1877]. For a motivating example, we consider the Bernoulli mixture model

$$\mathbb{P}_{\text{mix}}(\mathbf{y}^{(1:N)} \mid z^{(1:N)}, \boldsymbol{a}) = \prod_{n=1}^{N}\prod_{i=1}^{p} \mathbb{P}_{\text{mix}}(y_i^{(n)} \mid z^{(n)}, a_{i,z^{(n)}}) = \prod_{n=1}^{N}\prod_{i=1}^{p} a_{i,z^{(n)}}^{y_i^{(n)}}(1-a_{i,z^{(n)}})^{1-y_i^{(n)}} \tag{35.13}$$

with uniform prior distribution $a_{i,z} \sim \text{Uniform}(0,1)$, and let the observations $\mathbf{y}^{(1:N)}$ be independent realisations from a single-cluster Bernoulli mixture model with parameters $a_{i,1} = \frac{1}{2}$.

Theorem 35.4 (Proposition 4.1 in Zhou et al. [1877])**.** *Let the number of observations N be fixed, and $z^{(1:N)} = \{\{1\},\{2\},\ldots,\{N\}\}$ denote the partition into N separate clusters. As $p \to \infty$, we have $\mathbb{P}_{\text{mix}}(z^{(1:N)} = \{\{1\},\{2\},\ldots,\{N\}\} \mid \mathbf{y}^{(1:N}$ 1 almost surely.*

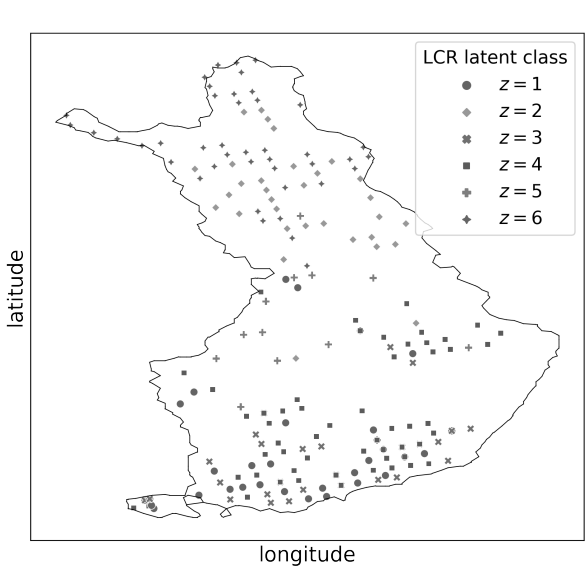
As shown by Theorem 35.4, our example of a well-specified Bernoulli mixture model with a single true cluster tends to assign each observation to a separate cluster when observations are high-dimensional. Multiple approaches can be designed to address this challenge faced by latent class regression and mixture models. BLIP model escapes the curse of dimensionality through the introduced layer of latent attributes of fixed dimension q. Intuitively, given high-dimensional observations $\mathbf{y}^{(1:N)}$, the posterior distribution of the deep latent classes $z^{(1:N)}$ is asymptotically equivalent to its conditional distribution given the true latent attributes $\mathbf{w}_*^{(1:N)}$ (referred to as the *Bayesian oracle clustering probability*).

Theorem 35.5 (Theorem 4.6 in Zhou et al. [1877])**.** *Under some mild conditions on well-specified data generating distribution and model prior distribution, we have $\lim_{p\to\infty} \mathbb{P}(z^{(1:N)} \mid \mathbf{x}^{(1:N)}, \mathbf{y}^{(1:N)}) = \mathbb{P}(z^{(1:N)} \mid \mathbf{x}^{(1:N)}, \mathbf{w}_*^{(1:N)})$ almost surely.*

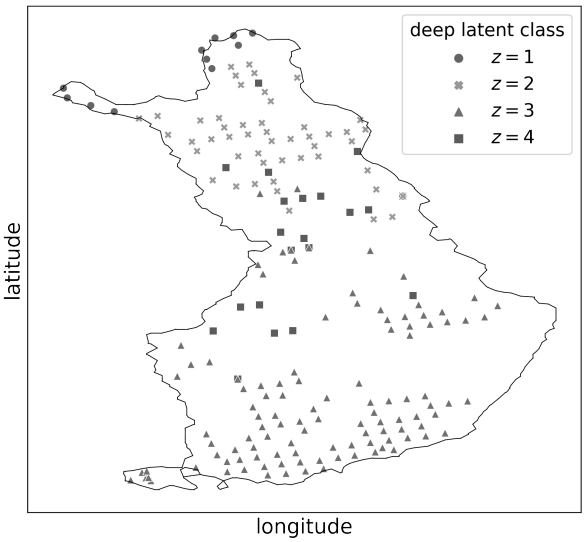
The oracle probability $\mathbb{P}(z^{(1:N)} \mid \mathbf{x}^{(1:N)}, \mathbf{w}_*^{(1:N)})$ remains invariant and reliable regardless of the observed data dimensionality. Thus, for clustering high-dimensional binary data, Theorem 35.5 provides a theoretical guarantee of the reliability of posterior inference for deep latent classes $z^{(1:N)}$.

35.3.4 Application to Joint Species Distribution Modelling

As an application, we compare the traditional latent class regression (LCR) with our model (BLIP) in the setting of joint species distribution modelling (JSDM). JSDMs model the joint distribution of the occurrences of multiple species and their dependence on environmental covariates [1276–1279]. A widely used benchmark in JSDM research is the Finnish bird dataset [1275], which records the presence or absence of 50 bird species in various sampling locations in Finland over multiple years and includes environmental covariates for each sampling location, such as the mean temperature in April and May and habitat type, classified into broadleaf forests, coniferous forests,



(a) Latent class regression (LCR).



(b) Bayesian latent class regression with interpretable binary profiles (BLIP).

Figure 35.7: Spatial distributions of bird species sampling locations on the map of Finland, grouped by their inferred LCR latent class z or BLIP deep latent class z. In ecology, such clusters are referred to as regions of common profile.

open habitats, urban habitats, and wetlands. The dataset also has meta (species-specific) covariates including biological traits of each bird species, such as the log-transformed typical body mass and migration type (classified as long term, short term, and resident).

We used data recorded from 2011 to 2013 as in-sample observations and data recorded in year 2014 as out-of-sample observations, comprising 363 and 137 sampling locations, respectively. Using the Watanabe–Akaike information criterion [WAIC, 1724], we select the optimal model structure for BLIP with $q = 5$ latent attributes and $d = 4$ deep latent classes, and the optimal model structure for LCR with $d = 6$ latent classes. We visualise the (deep) latent class assignments of LCR and BLIP in Figure 35.7, plotted on the longitude–latitude plane of the corresponding sampling locations. Although both models do not incorporate spatial information, compared to LCR, the inferred deep latent classes in BLIP align much better with Finland's geography, effectively capturing distinct ecological zones. Class $z = 1$ represents a borderline arctic tundra habitat, characterised by high mountains, the absence of trees, and a unique assemblage of bird species found nowhere else in the country. Class $z = 2$ corresponds to the northern boreal forest, which harbours Lapland forest species along with other old-growth forest species. Class $z = 3$ spans southern Finland, where a greater diversity of southern bird species is present. Finally, class $z = 4$ is concentrated along a small southwestern coastal region, which supports southern broadleaf forest species and potentially includes habitats for mire or peatland species.

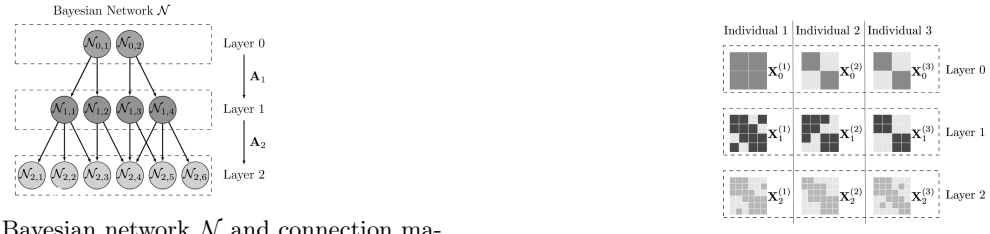
35.4 Bayesian deep generative models for multiplex networks

Networks are a central object in many disciplines, from neuroscience and ecology to sociology and economics. An undirected network with p nodes is commonly represented by a symmetric binary adjacency matrix $\mathbf{X}$, where $X_{i,j} = 1$ if nodes i and j are connected. In many modern applications, however, we observe not just a single network but a collection of networks over the same set of nodes, so-called multiplex networks. Examples include brain connectivity networks for multiple individuals, trade networks between countries across different commodities, soccer passing networks across matches, or contact networks among students observed at different times.

Existing approaches often model a shared connection probability matrix with hierarchical factorisation, but rarely address simultaneous inference of node communities, low-dimensional network embeddings, and clustering of individual networks. Leveraging the idea of a multilayer discrete latent structure in a Bayesian pyramid, we designed a family of Bayesian deep generative models for multiplex networks [1876]. These models can simultaneously (1) recover hierarchical community structure of nodes at the population level, (2) capture heterogeneity across individual networks, and (3) provide multi-resolution, overlapping clustering representations of both nodes

and networks. Despite their multifunctionality, they are highly interpretable and enjoy identifiability properties established through recently introduced theoretical tools.

35.4.1 Model Structure and Identifiability Results



(a) Population Bayesian network $\mathcal{N}$ and connection matrices $\mathbf{A}_k$ between layers $(k-1)$ and k.

(b) Individual adjacency matrices $\mathbf{X}_k^{(1:N)}$ at each layer k.

Figure 35.8: An illustration of Bayesian deep generative model for multiplex networks, where the Bayesian network $\mathcal{N}$ in (a) has $(K+1) = 3$ layers with $p_k = 2(k+1)$ nodes in each layer k, and the sample in (b) has $N = 3$ individuals. Connection matrices: $\mathbf{A}_1 = (1,0;\ 1,0;\ 0,1;\ 0,1)$, and $\mathbf{A}_2 = (1,0,0,0;\ 1,0,0,0;\ 1,1,0,0;\ 0,1,1,1;\ 0,0,1,1;\ 0,0,0,1)$.

In this section, uppercase $\mathbf{X}$ denotes an adjacency matrix and is distinct from the lowercase covariate vector $\mathbf{x}$ used in the preceding section. In Figure 35.8, $\mathbf{X}_k^{(n)}$ denotes the adjacency matrix for individual n at layer k. We propose a multi-layer latent graphical model (Figure 35.8) that extends the Stochastic Block Model (SBM) framework to multiplex networks while also including a multiresolution structure. The key innovation is the introduction of latent adjacency matrices at successive resolutions:

- Population layer: nodes are grouped into communities via a binary membership matrix $\mathbf{A}$.
- Individual latent layers: for each network, a latent adjacency matrix $\tilde{\mathbf{X}}^{(n)}$ (introduced below) encodes which communities interact, allowing flexible variation across individuals.
- Observed layer: each network's adjacency matrix $\mathbf{X}^{(n)}$ is generated conditional on its latent adjacency structure.

This hierarchy can be repeated across multiple layers, forming a Bayesian pyramid in which higher layers capture the coarse community structure and lower layers capture finer details. Importantly, the directed edges in the Bayesian network represent statistical dependence, distinct from the observed network edges. The model naturally accommodates overlapping communities: each node can belong to multiple communities, and the probability of an edge between two nodes increases if they share communities or if their communities are adjacent in the latent structure. This leads to interpretable additive effects that distinguish shared versus cross-community interactions.

We focus on the two-layer version of the model and provide intuition by generalising step by step from a stochastic block model (SBM) [768]. A simple SBM for a single binary adjacency matrix $\mathbf{X} = \{X_{i,j}\}$ is as follows:

$$\mathbb{P}(X_{i,j} = 1 | \mathbf{A}, C, \gamma) = \frac{\exp\left(C + \gamma \sum_{s=1}^{\tilde{p}} A_{i,s} A_{j,s}\right)}{1 + \exp\left(C + \gamma \sum_{s=1}^{\tilde{p}} A_{i,s} A_{j,s}\right)}, \tag{35.14}$$

where $C, \gamma \in \mathbb{R}$ are parameters and the binary matrix $\mathbf{A} \in \mathbb{R}^{p \times \tilde{p}}$ denotes the community membership of the p nodes, so that node i belongs to the community s if and only if $A_{i,s} = 1$. For multiplex networks with N observed adjacency matrices $\mathbf{X}^{(1:N)} := \{\mathbf{X}^{(n)}\}_{n=1}^N$, the above model can be easily extended if we assume that the N networks share common community memberships $\mathbf{A}$ and that the networks are conditionally independent given these memberships. We leverage information from multiplex networks to improve flexibility over SBMs while characterising differences across the networks. With this in mind, we introduce *latent adjacency matrices* for each network, which characterise connections between communities: $\tilde{\mathbf{X}}^{(1:N)} := \{\tilde{\mathbf{X}}^{(n)}\}_{n=1}^N$. Each $\tilde{\mathbf{X}}^{(n)} = (\tilde{X}_{s,t}^{(n)})$ is a binary $\tilde{p} \times \tilde{p}$ matrix, where $\tilde{X}_{s,t}^{(n)} = 1$ or 0 indicates whether the latent community s and the latent community t are adjacent. In (35.14), the basic SBM assumes that the nodes are more likely to be adjacent if they belong

to the same community. We additionally assume that nodes in the nth individual network are more likely to be adjacent if they belong to adjacent communities in $\tilde{\mathbf{X}}^{(n)}$:

$$\mathbb{P}(X_{i,j}^{(n)} = 1 | \mathbf{A}, C, \mathbf{\Gamma}) = \frac{\exp\left(C + \mathbf{a}_i^\top (\tilde{\mathbf{X}}^{(n)} * \mathbf{\Gamma}) \mathbf{a}_j\right)}{1 + \exp\left(C + \mathbf{a}_i^\top (\tilde{\mathbf{X}}^{(n)} * \mathbf{\Gamma}) \mathbf{a}_j\right)}, \tag{35.15}$$

where $\mathbf{a}_i^\top \in \mathbb{R}^{1 \times \tilde{p}}$ denotes the ith row of $\mathbf{A}$, $*$ denotes the Hadamard product, and $\mathbf{\Gamma}$ is a symmetric $\tilde{p} \times \tilde{p}$ matrix with positive entries. As the latent adjacency matrices $\tilde{\mathbf{X}}^{(1:N)}$ form a sample of multiplex networks, the same approach of this subsection can be applied to model the distribution of the latent $\tilde{\mathbf{X}}^{(1:N)}$ and our model can be formulated hierarchically across multiple layers, forming a deep generative model for multiplex networks.

Interpretability hinges on identifiability. Similarly to Bayesian pyramids and Bayesian latent class regression with interpretable binary profiles, we establish two notions of identifiability for the family of Bayesian deep generative models for multiplex networks, notably strict identifiability ensuring that all parameters within the parameter space are identifiable up to label switching at each layer of the Bayesian network, and generic identifiability ensuring that almost every parameter within parameter space is identifiable up to label switching.

Theorem 35.6. *(1) The model parameters are identifiable up to label switching if the binary membership matrix* $\mathbf{A}$ *at each layer of the Bayesian network contains two distinct identity blocks.*

(2) The model parameters are almost surely identifiable up to label switching if the binary matrix $\mathbf{A}$ *at each layer of the Bayesian network contains two distinct blocks with all one diagonals and some additional mild conditions are satisfied.*

Formal statements of Theorem 35.6 and its proof can be found in Zhou et al. [1876]. While the proof uses Kruskal's theorem on uniqueness of three-way CANDECOMP/PARAFAC (CP) tensor decomposition [937], as is standard for discrete latent variable models, we have also introduced a novel proof technique from complex analysis, which generalises existing techniques for proving identifiability and is of independent interest.

Theorem 35.7. *Let* $f(z_1, \ldots, z_n)$ *be a holomorphic function with domain* $\mathbb{C}^n$*. If its set of zeros in* $\mathbb{R}^n$ *has positive Lebesgue measure, i.e.*

$$\lambda\left(\{x \in \mathbb{R}^n : \ f(x) = 0\}\right) > 0,$$

then $f \equiv 0$ *on* $\mathbb{C}^n$*.*

The new proof technique involves two steps, first establishing the set of non-identifiable parameters to be contained in the zero set of a holomorphic function (e.g. polynomial, rational, exponential, trigonometric functions, and their compositions), then showing the holomorphic function to be non-zero by finding a specific parameter that is identifiable.

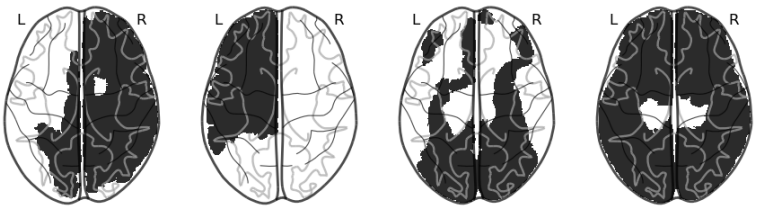
35.4.2 Bayesian Inference

We impose conditionally independent and weakly informative prior distributions on the model parameters for simplicity. To encourage statistical parsimony and prevent each node from belonging to too many communities, we enforce sparsity in the binary membership matrices $\mathbf{A}$ through a hyperparameter S controlling the maximum number of communities allowed for each node. We recommend choosing S as 1 or 2 for interpretability and computational efficiency. Building on the identifiability results of the model, posterior consistency can also be established using classical Bayesian asymptotics theories [442, 1477], ensuring reliable inference as the sample size grows.

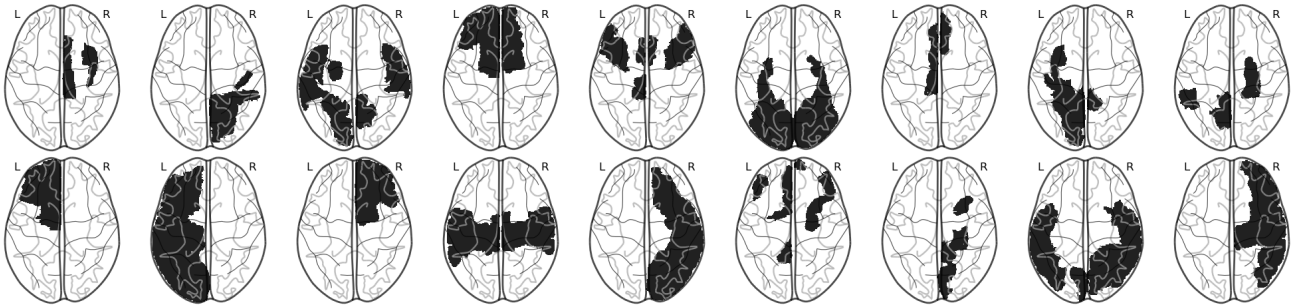
Posterior inference can be conducted through a Gibbs sampler with Pólya–Gamma data augmentation [1337]. To handle large-scale datasets, multiple techniques can be applied to accelerate posterior computation, including approximate MCMC methods such as a subsampling Gibbs sampler for speeding up the sampling step, and spectral initialisation methods leveraging the Mixed-SCORE algorithm for the degree-corrected mixed-membership (DCMM) model [845]. We treat the number of nodes in each layer of the Bayesian network as unknown hyperparameters and determine them through model selection based on the Watanabe-Akaike information criterion (WAIC) in Watanabe [1724].

35.4.3 Applications to Brain Connectivity Networks

We apply our model to brain structural connectome data from the Human Connectome Project (HCP) [487, 1664]. Following the Desikan atlas [416], the brain is divided into 68 anatomical regions of interest (ROI), and the observed data are the 68×68 adjacency matrices for each of the 1065 individuals.



(a) Visualisation of $p_0 = 4$ higher-level clusters of ROIs on the cerebral cortex.



(b) Visualisation of $p_1 = 18$ lower-level clusters of ROIs on the cerebral cortex.

Figure 35.9: Posterior distributions of the higher-level and lower-level clusters given by connection matrices $\mathbf{A}$, under sparsity hyperparameter $S = 2$ and number of nodes in each Bayesian network layer $p_0 = 4, p_1 = 18, p_2 = 68$.

We consider applying the model with a three-layer Bayesian network, with the 68 nodes at the bottom layer being the brain regions of interest (ROIs). Under the sparsity parameter $S = 2$, the optimal model has 18 nodes in the middle layer and 4 nodes in the top layer. The 18 nodes in the middle layer can be viewed as communities for the 68 ROIs, and the 4 nodes in the top layer can be viewed as communities for the 18 nodes in the middle layer. The binary membership matrices $\mathbf{A}_1, \mathbf{A}_2$ indicate the memberships of these community assignments, forming a hierarchical overlapping clustering of the 68 ROIs.

We visualise the learned hierarchical overlapping clustering of ROIs on the brain cortex in Figure 35.9. The learned lower-level and higher-level clusters are highly interpretable, despite the model having no prior knowledge of where each ROI lies on the brain cortex. The lower-level clusters in Figure 35.9b form 18 overlapping regions on the cerebral cortex roughly corresponding to the frontal, parietal, temporal, and occipital lobes of the brain. Some of clusters are restricted to one hemisphere, while others include regions that are symmetric in the left and right hemispheres. In Figure 35.9a, among the four higher-level groups, two of them roughly correspond to the left and right hemispheres, while the remaining two are symmetric across the hemispheres roughly representing the temporal and occipital lobes, and all regions excluding the parietal lobe. We find that these clusters are roughly consistent with the known anatomical structure of the human brain even though we infer these spatially contiguous regions in a fully unsupervised manner without including information on spatial proximity or anatomy.

35.5 Going beyond Categorical Data to Arbitrary Data Types: Deep Discrete Encoders

Interestingly and somewhat surprisingly, the Bayesian Pyramids framework can be substantially generalised beyond categorical data to model *arbitrary types of data* while still enjoying identifiability. The Deep Discrete Encoders [DDEs, 1007] extend Bayesian Pyramids to model very rich types of data by allowing the observed data layer to follow any parametric distribution, while remaining fully identifiable under essentially the same structural conditions on the between-layer graphical matrices. A scalable two-stage computational pipeline is proposed in Lee and Gu [1007] to enable scaling DDEs to high-dimensional data and truly deep latent architectures with $D = 5$ latent layers. We apply DDEs to diverse real-world datasets with different data types (images, texts, response times) to perform hierarchical topic modelling, image representation learning, and multimodal modelling in educational testing. These applications demonstrate DDEs' flexibility, interpretability, representation power, and downstream prediction accuracy.

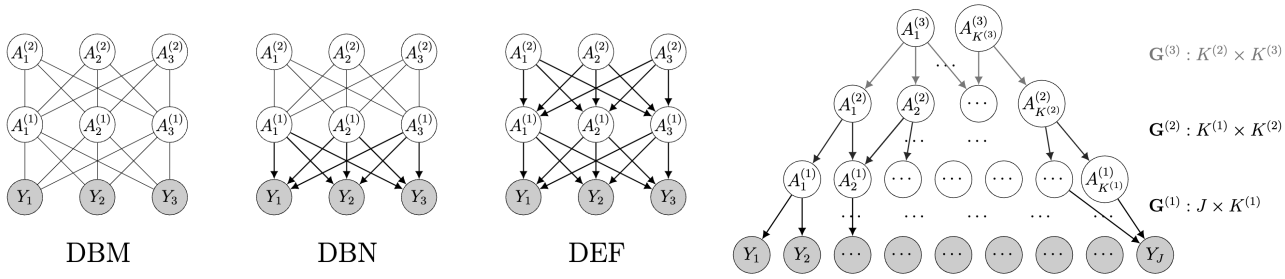


Figure 35.10: Graphical structures of DDEs and related existing DGMs: a deep Boltzmann machine (DBM), with binary data and undirected binary latent layers; a deep belief network (DBN), with binary data and directed binary latent layers apart from its top layer; a deep exponential family (DEF), with exponential-family conditional distributions; and a DDE (rightmost), with general responses Y_j and shrinking-ladder-shaped binary latent layers.

35.5.1 Model Setup of DDEs

DDEs have a flexible data layer at the bottom of the directed graphical model, but share the same key latent architecture as Bayesian Pyramids in that the discrete latent layers have progressively smaller sizes as they go deeper. Figure 35.10 illustrates DDEs together with some related DGMs. To distinguish the notation used for the two model families, Bayesian Pyramid latent layers are denoted by $\boldsymbol{\alpha}^{(m)}$, whereas DDE latent layers are denoted by $\mathbf{A}^{(d)}$.

We next describe a DDE with D latent layers. The top layer latent variables are assumed to be independent Bernoullis with parameters $\boldsymbol{p} = (p_1, \ldots, p_{K^{(D)}})$ to promote statistical parsimony:

$$\mathbb{P}(\mathbf{A}^{(D)} = \boldsymbol{\alpha}^{(D)}) = \prod_{k=1}^{K^{(D)}} \mathbb{P}(A_k^{(D)} = \alpha_k^{(D)}) = \prod_{k=1}^{K^{(D)}} p_k^{\alpha_k^{(D)}} (1-p_k)^{1-\alpha_k^{(D)}}, \quad \forall \boldsymbol{\alpha}^{(D)} \in \{0,1\}^{K^{(D)}}. \tag{35.16}$$

For each $d > 1$, given $\mathbf{A}^{(d)}$, we define the $(d-1)$th layer by assuming the conditional independence of $A_1^{(d-1)}, \ldots, A_{K^{(d-1)}}^{(d-1)}$ given $\mathbf{A}^{(d)}$. Each conditional distribution is

$$A_k^{(d-1)} \mid \mathbf{A}^{(d)} \sim \text{Bernoulli}\Big(g_{\text{logistic}}\big(\beta_{k,0}^{(d)} + \sum_{l=1}^{K^{(d)}} g_{k,l}^{(d)} \beta_{k,l}^{(d)} A_l^{(d)}\big)\Big), \tag{35.17}$$

where $g_{\text{logistic}}(x) = 1/(1+e^{-x})$; one may use alternative link functions $g : \mathbb{R} \to [0,1]$, such as the probit link. Here, $g_{k,l}^{(d)} = 1$ or 0 is the binary indicator of whether $A_l^{(d)}$ is a parent of $A_k^{(d-1)}$. Similarly as in Bayesian Pyramids, we collect these indicators into a graphical matrix $\mathbf{G}^{(d)} \in \{0,1\}^{K^{(d-1)} \times K^{(d)}}$ for each d, which summarises the bipartite graph structure between the $(d-1)$th and dth latent layer. Collect the $\beta_{k,l}$-parameters in a $K^{(d-1)} \times (K^{(d)} + 1)$ matrix $\mathbf{B}^{(d)}$, whose first column is the intercepts $\big(\beta_{k,0}^{(d)}\big)_{k \in [K^{(d-1)}]}$ and remaining parts are $\big(\beta_{k,l}^{(d)}\big)_{k \in [K^{(d-1)}], l \in [K^{(d)}]}$.

Finally, we model the bottom layer for the observed data. The observed variables $\mathbf{Y} = (Y_1, \ldots, Y_J)$ are modelled by assuming the conditional independence of $Y_1, \ldots, Y_J$ given $\mathbf{A}^{(1)}$. As the data are not necessarily categorical, we consider a general parametric family of the form

$$Y_j \mid \mathbf{A}^{(1)} \sim \text{ParFam}_j\Big(g_j\Big(\beta_{j,0}^{(1)} + \sum_{k=1}^{K^{(1)}} g_{j,k}^{(1)} \beta_{j,k}^{(1)} A_k^{(1)}, \gamma_j\Big)\Big). \tag{35.18}$$

Here, ParFam_j denotes an arbitrary parametric family, and H_j denotes its parameter space. Let p_j be the probability mass/density function of ParFam_j. The known link function $g_j : \mathbb{R} \times [0,\infty) \to H_j$ maps the linear predictor and, when present, the dispersion parameter $\gamma_j > 0$ to the parameters of the given family; the dispersion argument is omitted for families that do not use it. Similarly, denote $\mathbf{G}^{(1)} = (g_{j,k}^{(1)})_{J \times K^{(1)}}$ as the graphical matrix between the observed layer and the shallowest latent layer.

Next, we give some examples of the various response types $\mathcal{Y}_j$ allowed in the DDE framework, along with the corresponding link functions g and parametrisations for (35.18). Consider three types of responses for example: (i) binary, (ii) count, and (iii) continuous. We can model each of these responses using (i) Bernoulli with $g = g_{\text{logistic}}$, (ii) Poisson with an exponential link $g(x) = e^x$, and (iii) Normal with an identity link $g(x,y) = (x,y)$, respectively. Modelling other data types is also straightforward. We remark that the DDE framework can easily handle the case where the observed variables at the bottom layer have different data types and follow different distributions.

35.5.2 Identifiability and Estimation Consistency of DDEs

Lee and Gu [1007] show that DDEs are identifiable under essentially the same structural conditions on the graphical matrices $\mathbf{G}^{(1)}, \ldots, \mathbf{G}^{(d)}$ as Bayesian Pyramids, even under challenging and flexible settings with arbitrary parametric distribution families employed in the bottom layer. The following theorem establishes the identifiability of DDEs.

Theorem 35.8 (Theorem 1 in Lee and Gu [1007]). *Consider a D-latent-layer DDE where the number of latent variables is given. Suppose that for all $d \leq D-1$, the true graphical structures $\mathbf{G}^{(d)\star}$ and parameters $\mathbf{B}^{(d)\star}$ satisfy the following conditions:*

A. Each latent variable $A_k^{(d)}$ has at least two pure children $A_{j_{k,1}}^{(d-1)}, A_{j_{k,2}}^{(d-1)}$ according to $\mathbf{G}^{(d)\star}$.

B. For any $\boldsymbol{\alpha}^{(d)} \neq \boldsymbol{\alpha}'^{(d)} \in \{0,1\}^{K^{(d)}}$, there exists $j \in [K^{(d-1)}] \setminus \cup_{k=1}^{K^{(d)}} \{j_{k,1}, j_{k,2}\}$ such that $\sum_{k=1}^{K^{(d)}} \beta_{j,k}^{(d)\star}(\alpha_k^{(d)} - \alpha_k'^{(d)}) \neq 0$.

Then, the model is strictly identifiable.

The key insight behind proving the identifiability of DDEs for rich data types is that, with discrete latent layers, it suffices to establish identifiability for a one-latent-layer model and proceed in a layer-wise manner inductively. To elaborate, consider a "collapsed" DDE where the latent layers indexed by $d = 2, \ldots, D$ are marginalised out to give a probability mass function (pmf) for the first latent layer: $\{\mathbb{P}(\mathbf{A}^{(1)} = \boldsymbol{\alpha}^{(1)}) : \boldsymbol{\alpha}^{(1)} \in \{0,1\}^{K^{(1)}}\}$. If the collapsed one-layer model is proven to be identifiable, that means the conditional distributions $\mathbb{P}(Y_j \mid \mathbf{A}^{(1)})$ and the marginal distribution $\mathbb{P}(\mathbf{A}^{(1)})$ can be uniquely identified from the data distribution $\mathbb{P}(\mathbf{Y})$. Then, we can theoretically treat the shallowest latent layer $\mathbf{A}^{(1)}$ *as if it were observed*, because its pmf is now identified and known. Then, viewing this pmf as the marginal distribution of $K^{(1)}$-dimensional observed variables of a $(D-1)$-latent-layer Bernoulli-DDE, we can inductively identify all parameters via a layerwise argument. Notably, with a flexible data layer in DDEs, the first step of proving the collapsed model to be identifiable involves the following key technical step: carefully categorising the sample spaces of the general responses Y_j and defining surrogate categorical responses $\tilde{Y}_j$, whose joint distribution can be represented by a probability tensor. Then we invoke the classical multilinear algebra result on the uniqueness of tensor decompositions [937] to establish identifiability.

We next state the generic identifiability result for DDEs. In graph theory, a bipartite graph is said to have a "perfect matching" if it contains a set of edges without common vertices that covers every vertex of the graph [703].

Theorem 35.9 (Theorem 2 in Lee and Gu [1007]). *Consider the D-latent-layer DDE where the number of latent variables $\mathcal{K}$ is given, and all parametric families and link functions g_j are analytic. Let $\mathcal{G}^\star$ denote the true graphical matrices and suppose the true parameter lives in the graph-constrained parameter space $\Omega_{\mathcal{K}}(\boldsymbol{\Theta}; \mathcal{G}^\star)$. Suppose that for all $d \leq D-1$, the true graphical structure $\mathbf{G}^{(d)}$ satisfies the following condition C:*

C. There exists a partition of the $(d-1)$th layer variable indices $[K^{(d-1)}] = \mathcal{I}_1 \cup \mathcal{I}_2 \cup \mathcal{I}_3$ that satisfies the following properties: (i) for $a = 1, 2$, there is a perfect matching in the bipartite graph between $\mathbf{A}_{\mathcal{I}_a}^{(d-1)}$ and $\mathbf{A}^{(d)}$, (ii) each latent variable $A_k^{(d)}$ has at least one child in $\mathbf{A}_{\mathcal{I}_3}^{(d-1)}$.

Then, the model components $(\boldsymbol{\Theta}, \mathcal{G})$ are generically identifiable.

Estimation Consistency. We propose a penalised maximum likelihood estimation method for DDEs and show that the estimator is consistent for both the continuous parameters and the discrete graph structures. Suppose that the numbers of latent variables in all layers are known. We maximise the following objective function to estimate parameters $\boldsymbol{\Theta} = (\boldsymbol{p}, \mathcal{B}, \boldsymbol{\gamma})$:

$$\hat{\boldsymbol{\Theta}} \in \arg\max_{\boldsymbol{\Theta}} \left[\ell(\boldsymbol{\Theta} \mid \mathbf{Y}) - \sum_{d=1}^{D} p_{\lambda_N, \tau_N}(\mathbf{B}^{(d)}) \right], \tag{35.19}$$

where $\ell(\boldsymbol{\Theta} \mid \mathbf{Y}) = \sum_{i=1}^{N} \log \mathbb{P}(\mathbf{Y}_i \mid \boldsymbol{\Theta})$ denotes the marginal log-likelihood function given a sample $\mathbf{Y}_1, \ldots, \mathbf{Y}_N$ of size N, defined based on the marginal distribution of $\mathbf{Y}$. The term $p_{\lambda_N, \tau_N} : \mathbb{R} \to [0, \infty)$ denotes a sparsity-inducing penalty, such as the truncated lasso penalty (TLP) [1509] and smoothly clipped absolute deviation (SCAD) penalty [504]. We slightly abuse notation and let $\mathbf{Y}$ denote the $N \times J$ data matrix including $\mathbf{Y}_1, \ldots, \mathbf{Y}_N$ as rows. Using

the estimated coefficients in $\hat{\mathcal{B}} = (\hat{\beta}_{l,k}^{(d)})$, the layer-wise graphical matrices in $\mathcal{G}$ can be estimated by reading off the sparsity pattern of $\hat{\mathcal{B}}$:

$$\hat{g}_{l,k}^{(d)} := 1_{\hat{\beta}_{l,k}^{(d)} \neq 0} \quad \text{for all} \quad d \in [D],\ k \in [K^{(d)}],\ l \in [K^{(d-1)}]. \tag{35.20}$$

Assuming a compact parameter space with bounded coefficients $\mathbf{B}^{(d)}$, the next theorem shows that the estimator defined in (35.19) and (35.20) results in consistent estimation.

Theorem 35.10 (Theorem 3 in Lee and Gu [1007]). *Consider a D-latent-layer DDE with true parameters $\mathbf{\Theta}^\star, \mathcal{G}^\star$ and known $\mathcal{K}$. Assume that the model at $\mathbf{\Theta}^\star$ is identifiable, has a non-singular Fisher information, and all entries of $\{\mathbf{B}^{(d)}\}_{d=1}^D$ are bounded. Then, the estimator $\hat{\mathbf{\Theta}}$ in* (35.19) *is $\sqrt{N}$-consistent in the sense that there exists some $\tilde{\mathbf{\Theta}} \sim_{\mathcal{K}} \hat{\mathbf{\Theta}}$ such that $\|\tilde{\mathbf{\Theta}} - \mathbf{\Theta}^\star\| = O_p(1/\sqrt{N})$. Here, $\sim_{\mathcal{K}}$ denotes equivalence up to within-layer permutations of the latent variables, and $\|\cdot\|$ denotes the vectorised L^2 norm. Additionally, the graphical matrices are consistently estimated: for $\tilde{\mathcal{G}}$ resulting from $\tilde{\mathbf{\Theta}}$ according to* (35.20)*, we have $\mathbb{P}(\tilde{\mathcal{G}} \neq \mathcal{G}^\star) \to 0$.*

35.5.3 Scalable Computation Pipeline

We present a two-stage scalable computational pipeline to compute the penalised maximum likelihood estimator in (35.19). This method builds upon the penalised expectation–maximisation (EM) algorithm by including a spectral initialisation and stochastic approximation. For notational simplicity, we describe the method for $D = 2$, which straightforwardly extends to an arbitrary number of latent layers.

Stage One: Layerwise Double-SVD Initialisation. The multiple layers of nonlinearity in DDEs lead to a highly nonconvex optimisation landscape with potentially exponentially many local optima. Hence, it is critical to initialise the optimisation algorithm wisely. Spectral methods have been widely used for initialising linear low-rank matrix factorisation, but existing approaches are not applicable for DDEs with a deep nonlinear structure. To address this, we propose a novel layerwise nonlinear spectral initialisation strategy, outlined in Algorithm 29.

Algorithm 29: Outline of the Layerwise Double-SVD Initialisation

Data: Data matrix $\mathbf{Y}_{N\times J}$, latent dimensions $\mathcal{K}$.
Result: Initial layerwise estimates $\{\mathbf{B}^{(d)}, \mathbf{A}^{(d)}\}_{d=1}^D$.

1. De-noise the data matrix $\mathbf{Y}$ using a first SVD, and linearise this matrix by applying the inverse-link function $(\mu \circ g)^{-1}$ elementwise. Let $\hat{\mathbf{Z}}$ denote the inverted matrix.
2. Let $\hat{\mathbf{Z}}_0$ be the column-centred version of $\hat{\mathbf{Z}}$, and compute its rank-$K^{(1)}$ approximation by a second SVD: $\hat{\mathbf{Z}}_0 \approx \mathbf{U}_{N\times K^{(1)}} \mathbf{\Sigma}_{K^{(1)}\times K^{(1)}} \mathbf{V}_{J\times K^{(1)}}^\top$.
3. Rotate $\mathbf{V}$ according to the Varimax criteria, and denote it as $\hat{\mathbf{B}}^{(1)}$.
4. Use the relationship $\hat{\mathbf{Z}} \approx [\mathbf{1}_N, \mathbf{A}^{(1)}]\mathbf{B}^{(1)\top}$ to estimate $\mathbf{A}^{(1)}$.
5. Now, suppose that the estimated $\hat{\mathbf{A}}^{(1)}$ is the "observed data" of a one-latent-layer DDE. Repeat steps 1-4 to estimate $\mathbf{B}^{(2)}$ and $\mathbf{A}^{(2)}$.

The main idea of Algorithm 29 is to view the responses $\mathbf{Y}$ as a perturbation of a population expectation $\mathbb{E}(\mathbf{Y} \mid \mathbf{A}^{(1)}, \mathbf{B}^{(1)}) = (\mu \circ g)\left([\mathbf{1}_N, \mathbf{A}^{(1)}]\mathbf{B}^{(1)\top}\right)$, which is an elementwise (nonlinear) transformation of a low-rank matrix. Here, $\mu : H \to \mathcal{Y}$ is the known mean function of the observed-layer parametric family in (35.18) and $g : \mathbb{R} \to H$ is the link function. The function $(\mu \circ g)$ is g_{logistic} for Bernoulli responses, the exponential function for Poisson, and the identity function for Normal. When $(\mu \circ g)$ is nonlinear, we use the double SVD procedure [1846]. This procedure applies a first SVD to de-noise the data, then linearises the data by inverting the link function $(\mu \circ g)$, and finally performs a second SVD to find the low-rank matrix $\hat{\mathbf{Z}} \approx [\mathbf{1}_N, \mathbf{A}^{(1)}]\mathbf{B}^{(1)\top}$. Next, noting that the true coefficient matrix $\mathbf{B}^{(1)}$ is sparse, we estimate it by seeking a sparse rotation of the right singular subspace of $\hat{\mathbf{Z}}$, using the popular Varimax criterion [860, 1416]. This procedure also provides an estimate of the latent $\mathbf{A}^{(1)}$, which can be treated as the "observed data" to initialise the deeper layer's $\mathbf{B}^{(2)}$ and $\mathbf{A}^{(2)}$ in a similar fashion as described above. This layer-by-layer algorithm readily generalises to deeper models and is reminiscent of the greedy learning strategy for DBNs [752].

Stage Two: Penalised SAEM Algorithm. Here t indexes the iterations, so $\boldsymbol{\Theta}^{[t]}$ denotes the parameter estimate at iteration t. For DDEs with a large number of latent variables, the E-step in standard EM algorithms computes all conditional probabilities $\mathbb{P}(\mathbf{A}_i^{(1)} = \boldsymbol{\alpha}^{(1)}, \mathbf{A}_i^{(2)} = \boldsymbol{\alpha}^{(2)} \mid \mathbf{Y}; \boldsymbol{\Theta}^{[t]})$ for all $\boldsymbol{\alpha}^{(1)} \in \{0,1\}^{K^{(1)}}$ and $\boldsymbol{\alpha}^{(2)} \in \{0,1\}^{K^{(2)}}$. This requires computing and storing $O(N \times 2^{\sum_{d=1}^{D} K^{(d)}})$ terms. The exponential dependency in $K^{(d)}$ is concerning even for moderately large latent dimensions, say $K^{(d)} = 10$, and quickly becomes prohibitive for larger $K^{(d)}$. Therefore, we propose a penalised stochastic approximation expectation–maximisation (SAEM) algorithm [see 406] by modifying both the E-step and M-step to more scalable versions using approximate sampling. This method has computing-time and memory costs that depend linearly on $\sum_{d=1}^{D} K^{(d)}$.

We elaborate on the details on deriving the SAEM. *First*, we replace the standard E-step with a simulation step, which simulates only a small number (denoted as C) of posterior samples of the latent variables. As exact sampling from the joint distribution $\mathbb{P}(\mathbf{A}_i^{(1)}, \mathbf{A}_i^{(2)} \mid \mathbf{Y}; \boldsymbol{\Theta}^{[t]})$ is expensive, we sample each latent variable from their complete conditionals: sample each $A_{i,k}^{(1),[t+1]}$ from the Bernoulli distribution $\mathbb{P}(A_{i,k}^{(1)} \mid (-), \boldsymbol{\Theta}^{[t]})$, where $(-)$ denotes the estimates of all other latent variables from the tth iteration. This simulation-based E-step drastically reduces the exponential complexity to a linear complexity by computing and storing only $O(N \times \sum_{d=1}^{D} K^{(d)})$ terms. We empirically find that taking $C = 1$ sample in each iteration is computationally efficient without sacrificing much accuracy. *Second*, we modify the standard M-step objective function (expected complete data log-likelihood) by (i) replacing the exact conditional probability values with sample-based quantities, (ii) stochastically averaging the objective functions, and (iii) including the sparsity-inducing penalties in (35.19). Algorithm 30 summarises the SAEM algorithm with $C = 1$. Here, the detailed M-step updates can be written in terms of low-dimensional maximisations for each row of the coefficient matrices.

Although the above two-stage pipeline is not a Bayesian inference procedure, one could easily develop a Bayesian version of DDEs and adapt the full conditional distributions derived in the SAEM Algorithm 30 to develop a Gibbs sampler and apply it if there is enough computing budget in high-dimensional scenarios. In this case, one can still first run the layerwise nonlinear spectral initialisation in Algorithm 29 to initialise the MCMC chain and speed up convergence.

Algorithm 30: Penalised SAEM algorithm for the two-latent-layer DDE

Data: $\mathbf{Y}, \mathcal{K}$, tuning parameters λ_N, τ_N.
Initialise $\mathbf{A}^{(1),[0]}, \mathbf{A}^{(2),[0]}$ and $\boldsymbol{\Theta}^{[0]}$ using Algorithm 29.
while $\|\boldsymbol{\Theta}^{[t]} - \boldsymbol{\Theta}^{[t-1]}\|$ *is larger than a threshold* **do**
  In the tth iteration, `// Simulation-step` Sample each $A_{i,k}^{(1),[t+1]}, A_{i,l}^{(2),[t+1]}$ from the complete conditionals using the previous parameter estimates $\boldsymbol{\Theta}^{[t]}, \mathbf{A}^{[t]}$ `// Stochastic approximation` `M-step` update the parameters $\boldsymbol{\Theta}^{[t+1]}$ by maximising the stochastically averaged objectives
Estimate $\{\mathbf{G}^{(d)}\}_{d\in[D]}$ based on the sparsity structure of $\hat{\mathbf{B}}$ according to (35.20).
Output: Estimated continuous parameters $\hat{\boldsymbol{\Theta}}$, estimated graphical matrices $\mathcal{G}$.

35.5.4 Simulation Results

For relatively shallow DDEs with $D = 2$ latent layers, the simulation results deliver a similar message as those for Bayesian Pyramids – they empirically confirm the identifiability theory and demonstrate the excellent performance of the proposed algorithms. We thus omit the simulation results in those standard regimes, but present results under deeper models and ablation studies.

Simulation studies under deep models. We assess the scalability and estimation accuracy of the two-stage computational pipeline for *deeper models* (with $D = 3, 4, 5$ latent layers) with potentially *high-dimensional responses* (with the observed data dimension set to $J = 486$ in the $D = 5$ case). For all settings, we consider Normal observations with $K^{(D)} = 2$ latent variables for the top (deepest) layer and $K^{(d-1)} = 3K^{(d)}$ variables for the dth layer for each $d = D, \ldots, 2$.

Table 35.1 summarises the estimation accuracy of the graphs in each layer and the average runtime for DDEs with $D = 5$ latent layers. The computation time illustrates that our method is scalable to deeper models. Specifically, the $D = 3, 4$ cases only took a few minutes (tables omitted here), and the most challenging case with $D = 5$ and $N = 16,000$ also took less than an hour on a personal laptop. The estimation accuracy is higher for shallower layers than for deeper layers. This results from the accumulation of uncertainty for inferring deeper

layers, which is inevitable in hierarchical models as each layer consists of stochastic latent variables. Notably, even for deep models with $D = 5$, the shallow structures $\mathbf{G}^{(1)}, \mathbf{G}^{(2)}$ are still recovered with high accuracy. This indicates that one can include additional latent layers to increase the models' representational power, without sacrificing the accuracy of learning graph structures closer to the data layer.

Layer $\backslash N$	500	1000	2000	4000	8000	16000
$\mathbf{G}^{(1)}$	1.00	1.00	1.00	1.00	1.00	1.00
$\mathbf{G}^{(2)}$	0.81	0.90	0.95	0.97	0.98	0.98
$\mathbf{G}^{(3)}$	0.77	0.83	0.86	0.88	0.90	0.92
$\mathbf{G}^{(4)}$	0.61	0.62	0.65	0.67	0.66	0.74
$\mathbf{G}^{(5)}$	0.60	0.59	0.62	0.62	0.65	0.64
runtime (s)	220	259	326	563	893	2925

Table 35.1: Average entrywise-accuracy of estimating the graphical matrices and runtime in seconds for DDEs with $D = 5$ latent layers.

Ablation studies. As DDEs closely resemble DBNs [752], we compare DDEs with DBNs. For a fair comparison, we considered two settings: (a) data generated from a Bernoulli-DDE, and (b) data generated from a DBN with identical coefficients. The results in Figure 35.11 demonstrate that our proposed algorithm has better estimation accuracy for both the graphical structure and the continuous parameters under both well-specified and misspecified settings, even when data are generated from a DBN with undirected edges in the top layer. This is because the DBN algorithm does not learn sparse coefficients and fundamentally suffers from local optima due to random initialisation. The ablation studies indicate that the current implementations of DDEs are slower than DBNs. We believe that our DDE algorithms can be further scaled up by replacing the M-step in the SAEM algorithm with first-order optimisation methods (e.g. gradient ascent or stochastic gradient ascent) in future studies.

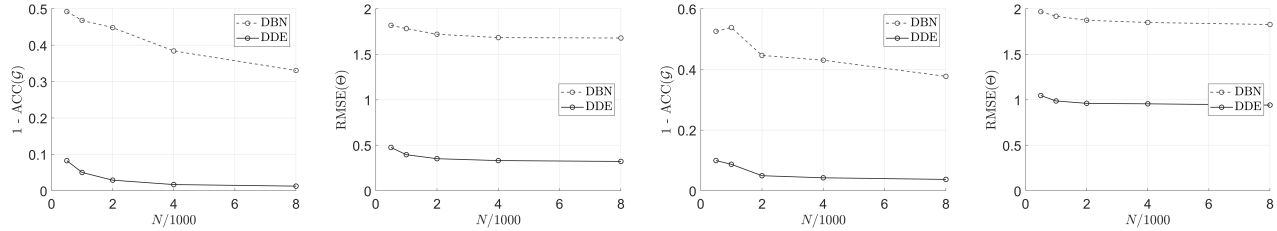


Figure 35.11: Comparison of Bernoulli-DDE versus DBN. True data are generated from a **DDE** in (a)–(b), and from a **DBN** in (c)–(d). Dashed and solid lines show results obtained by the DBN algorithm and our DDE algorithm, respectively. (a) and (c): estimation error for the graphs $\mathcal{G}$. (b) and (d): estimation error for continuous parameters Θ. Smaller values are better.

35.5.5 Diverse Real-world Applications: Images, Texts, Multimodal Data

Binary Data: Bernoulli-DDE for MNIST Handwritten Digits. The MNIST dataset for handwritten digits is very popular for classification as well as unsupervised learning [409]. We fit the two-latent-layer DDE with binary responses (Bernoulli-DDE), where the observed layer distributions are $Y_j \mid \boldsymbol{\alpha}^{(1)} \sim \text{Ber}\big(g_{\text{logistic}}(\beta^{(1)}_{j,0} + \sum_{k\in[K^{(1)}]} \beta^{(1)}_{j,k}\alpha^{(1)}_k)\big)$. This resembles existing generative models for images such as DBNs and DBMs, but we instead consider an identifiable, much lower-dimensional, shrinking-ladder-shaped latent structure. For easier presentation, we consider the subset of images whose true digit labels are 0, 1, 2, and 3. After preprocessing, our training set consists of $N = 20,679$ images, each with $J = 264$ binary pixels, representing challenging and relatively high-dimensional binary data. Compared to many existing works analysing MNIST, we consider a more challenging fully unsupervised setting by holding out all other information about the images, such as the true labels, number of classes, and the spatial location among the pixels.

We fit the two-latent-layer DDE with the latent dimensions set to $K^{(1)} = 5$ and $K^{(2)} = 2$. Figure 35.12 displays the first-layer coefficients $\mathbf{B}^{(1)}$ by rearranging each column into the original 28×28 grid. Note that the value of $\mathbf{B}^{(1)}$ is on the logit scale, so the negative coefficients make the corresponding pixel more likely to be zero. From

the reshaped columns $\boldsymbol{\beta}_{0,0}^{(1)}, \boldsymbol{\beta}_{0,1}^{(1)}, \ldots, \boldsymbol{\beta}_{0,5}^{(1)}$ of $\mathbf{B}^{(1)}$, we can interpret the meaning of each latent variable: $A_1^{(1)} = 1$ indicates a zero-like shape, $A_5^{(1)} = 1$ indicates symmetric curves on the upper-left and bottom-right corners, and the other latent variables represent different rotations. Additionally, using the estimated deeper graphical matrix $\mathbf{G}^{(2)}$, we can interpret the top layer latent variables as broader information about the images: e.g., $A_1^{(2)}$ indicates large pixel density and $A_2^{(2)}$ indicates symmetry with respect to the x-axis.

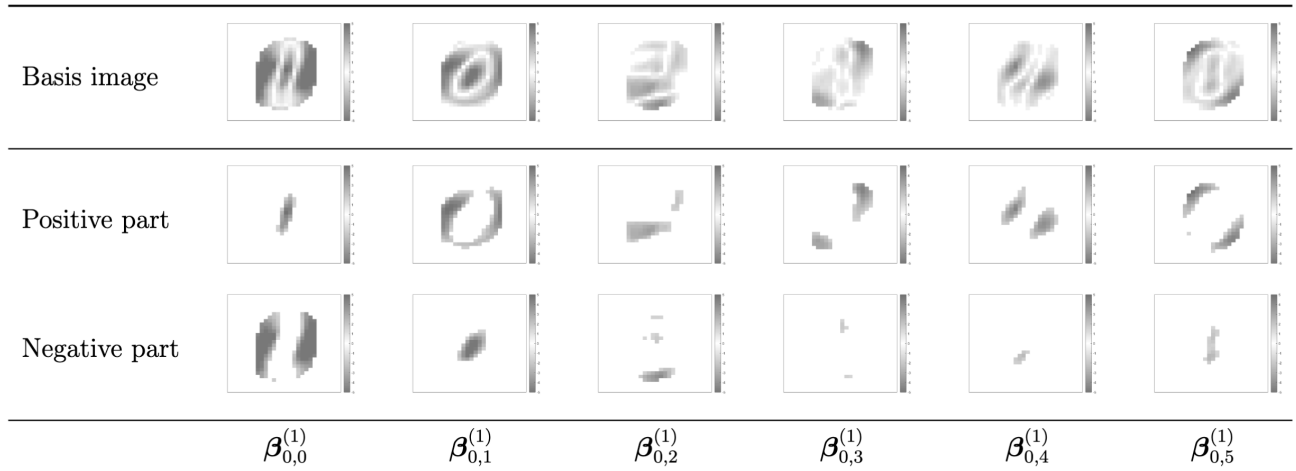


Figure 35.12: DDE-estimated basis images (reshaped from the estimated $\boldsymbol{\beta}_{0,k}^{(1)}$) from the MNIST data, which define sparse subregions in the 28×28 image. The second and third rows show the most significant positive and negative parts of the basis images after thresholding at ± 1.5.

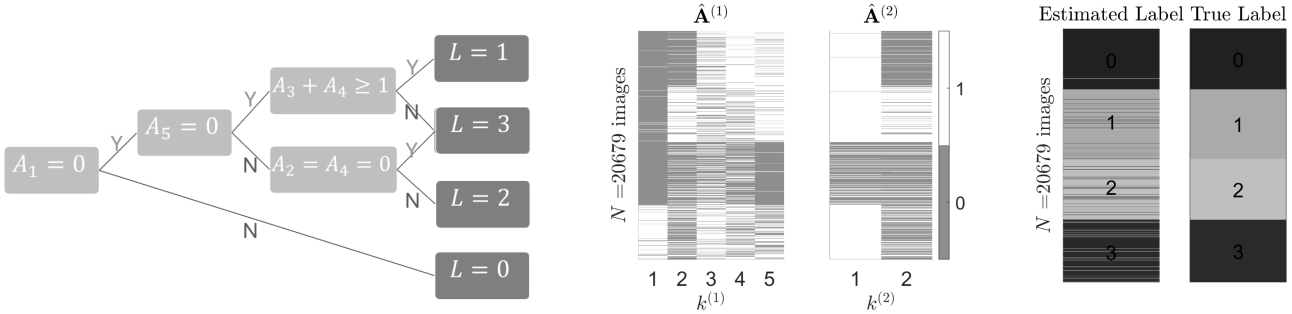
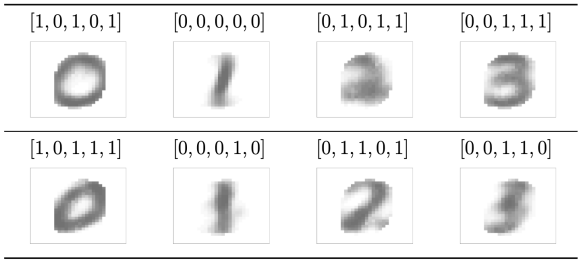


Figure 35.13: **Left**: Decision tree to estimate the digit $L = 0, 1, 2, 3$. **Centre**: Estimated latent representations $\hat{\mathbf{A}}^{(1)}, \hat{\mathbf{A}}^{(2)}$. **Right**: Estimated and true digits in the training set.



Accuracy	LCM	1-DDE	**2-DDE**	2-DBN	VAE
Train classif. (%)	89.5	84.8	**92.0**	89.3	97.1
Test classif. (%)	90.9	86.2	**92.6**	90.6	92.0
Train recon. (%)	48.2	79.5	**79.6**	76.4	82.5
Test recon. (%)	48.5	79.1	**79.9**	77.0	82.7

Figure 35.14: **Left**: Example images generated from DDE and their latent representations. **Right**: Classification accuracy and pixel-wise reconstruction accuracy for the MNIST dataset.

The learned shallower and deeper latent representations $(\hat{\mathbf{A}}^{(1)}, \hat{\mathbf{A}}^{(2)})$ are evaluated under two measures of performance: *classification accuracy* and *reconstruction accuracy*. Then, a decision tree that classifies the binary

latent representations into the categorical labels is built using misclassification error as the splitting criterion; see the left panel of Figure 35.13. The centre and right panels of Figure 35.13 display the estimated latent traits and labels. Our classifier leads to a high train/test accuracy of 92.0%/92.6%, even though our model is not fine-tuned for image classification. Although state-of-the-art machine-learning methods can achieve an accuracy as high as 97% [1198], the main goal here is interpretability and parsimony rather than classification; the DDE uses more limited information and is a less complex but more interpretable model that provides a generative process for the image. In the right panel of Figure 35.14, we compare the classification accuracy and pixel-wise reconstruction accuracy of the two-latent-layer DDE with alternative interpretable models (a latent class model (LCM) and a one-latent-layer DDE), as well as popular unsupervised machine-learning models (a two-latent-layer DBN and a variational autoencoder (VAE)). The results show that DDE performs well for both measures. We also display example images generated from DDE alongside their latent representations in the left panel of Figure 35.14, which illustrate various handwriting styles for each digit.

Count Data: Poisson-DDE Hierarchical Topic Modelling Next, we apply DDEs to learn hierarchical latent topics from text documents. In topic modelling, it is natural for the topics to be correlated with each other [960], and hierarchical topic modelling is often adopted [253, 672, 1283]. While many existing works assume a tree-structured hierarchy, DDEs flexibly allow multiple parents for each variable.

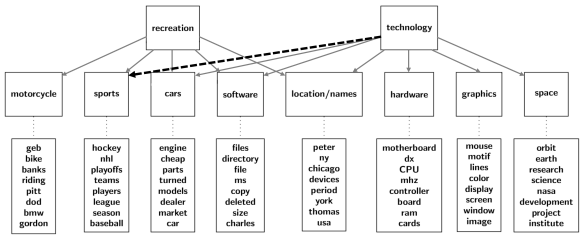


Figure 35.15: DDE estimated from the 20 Newsgroups dataset. For each shallow-layer latent variable, we display the top eight representative words. The width of each upper-layer arrow is proportional to the corresponding coefficient; the arrow from "technology" to "sports" indicates a negative value.

We analyse the 20 Newsgroups dataset [971]. After preprocessing and focusing on 12 newsgroups, the dataset consists of $N = 5,883$ documents and $J = 653$ words, representing a high-dimensional count-valued dataset. We fit the two-latent-layer DDE with a Poisson-distributed data layer (Poisson-DDE) $Y_j \mid (\mathbf{A}^{(1)} = \boldsymbol{\alpha}^{(1)}) \sim \text{Poi}\big(\exp\big[\beta_{j,0}^{(1)} + \sum_{k\in[K^{(1)}]} \beta_{j,k}^{(1)} \alpha_k^{(1)}\big]\big)$ and latent dimensions $K^{(1)} = 8, K^{(2)} = 2$, and display the estimated latent structure in Figure 35.15. To better interpret individual latent variables, we define representative words for each topic k based on the discrepancy between $\beta_{j,k}$ and all other coefficients, $(\beta_{j,l})_{l\neq k}$. That is, for each index k, we choose the words j with the largest values of $\max\{\min\{\beta_{j,k} - \beta_{j,l} : l \neq k\}, 0\}$. We display the representative words for each latent variable in the bottom row of Figure 35.15. Here, each latent variable is named based on the representative words and the held-out newsgroup categories. Compared to the held-out tree structure of the 12 newsgroup labels, DDE discovers a more flexible DAG structure in Figure 35.15, allowing multiple parents for each topic to effectively model the complex label dependence. For example, 'cars', 'software', 'location/names' have both 'recreation' and 'technology' as parents, and many bottom-layer words are assigned to multiple topics.

Model	Latent dimensions	Train perplexity	Test perplexity	Neg. coherence	Similarity
LDA	8	499	512	276	43
LDA	256	269	515	321	(41450)
DPFA	$8-2$	289	499	211	5
DPFA	$128-64$	175	232	280	(3378)
DDE	$8-2$	322	398	270	0
DDE	$8-3$	322	399	275	0

Table 35.2: Train and test perplexity, negative coherence, and similarity scores of different models on the 20 Newsgroups dataset. For all measures, smaller values are better. We parenthesise similarity scores for models with different dimensions because the measure is not normalised.

We also compare the fit of DDE to other directed graphical models with matching latent dimensions: LDA [169] and DPFA-SBN [570]. LDA has a single latent layer with mixed membership scores, and DPFA-SBN is a

multilayer model with binary latent variables similar to DDEs. We consider the following three metrics widely used in topic modelling to measure different aspects of fit [281]: *perplexity, coherence, and similarity* (see definitions in Lee and Gu [1007]). Table 35.2 summarises the results and shows the promising fit of DDE. Compared to other models with the same latent dimensions, DDEs have better test perplexity and similarity.

Multimodal Educational Data: Bernoulli-Lognormal-DDE. We apply the DDE to an educational assessment dataset from the Trends in International Mathematics and Science Study (TIMSS) [523]. As the assessment is electronically conducted, multiple modalities of information are recorded. Here, we focus on two important modalities: *binary* response accuracy and *continuous* response time. For each individual student, our data consist of response accuracy (whether the student gave a correct answer) and response time (how long the student took) for each of the $J/2 = 29$ items. We use the same latent variables to model both data modalities and model the binary response accuracy via Bernoulli distributions, and model the continuous positive response times via Lognormal distributions: $Y_j \mid \boldsymbol{\alpha}^{(1)} \sim \text{lognormal}\big(\beta_{j,0}^{(1)} + \sum_{k\in[K^{(1)}]} \beta_{j,k}^{(1)} \alpha_k^{(1)}, \gamma_j\big)$.

Response \ Latent skill	$A_1^{(2)}$	$A_1^{(1)}$: Number	$A_2^{(1)}$: Algebra	$A_3^{(1)}$: Geometry	$A_4^{(1)}$: Data and Prob
Agree a lot	0.62	0.59	0.62	0.42	0.57
Agree a little	0.50	0.47	0.52	0.34	0.43
Disagree a little	0.37	0.33	0.38	0.31	0.37
Disagree a lot	0.29	0.29	0.32	0.22	0.29

Table 35.3: Average latent variable estimates for each response category for the question "*Mathematics is one of my favorite subjects*". The table shows the higher-order latent variable and four representative first-layer skills.

We fit the two-latent-layer DDE with $K^{(1)} = 7$, $K^{(2)} = 1$, and estimate each student's latent skills by the latent attribute profile with the largest posterior probability. We compare the estimated latent variables with the held-out information of each student's categorical response to a survey question: "*Mathematics is one of my favorite subjects*", and display the results in Table 35.3. The first column shows that the higher-order latent variable, $A_1^{(2)}$, is highly correlated with the extent to which students like mathematics. This suggests that $A_1^{(2)}$ can be interpreted as a general indicator of the students' interest in mathematics, while the fine-grained latent variables $\mathbf{A}^{(1)}$ represent students' specific skill mastery profiles. In addition, we observe that the students who enjoy math tend to have a higher probability of mastering specific skills as well. This is coherent with the fact that the estimated $\mathbf{B}^{(1)}$-coefficients are nonnegative for both modalities.

35.6 Concluding Remarks

In summary, identifiability provides the theoretical foundation for interpretable and reproducible Bayesian deep learning. This chapter unified recently proposed identifiable deep generative frameworks and outlined their key theoretical guarantees. By embedding algebraic identifiability conditions as graphical constraints within hierarchical Bayesian modelling, these approaches demonstrate that deep representation learning can be both *statistically sound and practically interpretable.*

Chapter 36

Bayesian mixture of experts in BDL

Mixture-of-experts (MoE) models, originally proposed by Jacobs et al. [828], extend traditional mixture models in the context of regression, classification, and beyond. In the context of modern deep learning, there has been a recent resurgence in interest in such models, as they show promise in scaling computation while retaining model expressiveness. The aim of this chapter is to study a particular version of MoE models, those featuring *softmax gating*, which has been popularized in deep learning applications and which naturally lends itself to a treatment in the Bayesian learning framework.

36.1 Introduction

At a high level, MoE models aggregate multiple specialized submodels, known as *experts*, and employ a learnable *gating function* or *router* to determine the weight of (or probability of assignment to) each expert [654]. In contrast to traditional mixture models with static mixing proportions, the gating function in MoE models flexibly assigns input-dependent weights to experts, in such a way that experts that are more relevant to the input receive larger weights than others. As a result, MoE models are more expressive and adaptive than mixture models when one wishes to model the conditional distribution of an output given inputs. Specifically, MoE can lead to more accurate estimation than conventional mixture models in regression/classification settings where the conditional distribution of the response Y given the predictor vector X varies significantly as a function of X.

The basic MoE paradigm has been extended in several directions. Jordan and Jacobs [850] devised hierarchical mixtures-of-experts (HME), allowing experts to be organized in a tree structure for better expert specialization. Recently, authors in [1508] introduced a sparse version of MoE in which only a subset of experts is activated per input during training. This conditional-computation strategy helps scale up the model capacity significantly without a proportional increase in computation. Thanks to their scaling power, there has been a surge of interest in using sparse MoE models in a wide range of fields in deep learning, including language modelling [392, 843], computer vision [1398], multi-task learning [717], multi-modal learning [704], domain adaptation [1022, 1242], continual learning [989] and parameter-efficient fine-tuning [425, 990]. Despite their success, these implementations rely on maximum likelihood estimation, often via the Expectation-Maximization algorithm, which exposes them to issues of overfitting and fixed model complexity [1143].

If uncertainty quantification is desired, adapting the Bayesian paradigm to MoE models is a sensible choice. Authors in [1313] developed a fully Bayesian MoE model using MCMC methods, which allowed inference over both expert parameters and router parameters. Simultaneously, authors in [1728] proposed a Bayesian framework for inferring the parameters of an MoE model based on ensemble learning by variational free energy minimization, enabling more scalable inference by optimizing a lower bound on the marginal likelihood. These contributions marked a shift from point estimates to full posterior distributions, improving generalization and interpretability. Bayesian MoE models bring together the strengths of modular learning and Bayesian probabilistic inference: by decomposing complex prediction tasks into simpler subtasks solved by specialized experts and coordinated through a gating mechanism, MoE models provide flexibility and interpretability; the Bayesian formulation further enhances this framework by addressing overfitting, model selection, and uncertainty quantification, combining the benefits of both paradigms.

A key theoretical advancement in the development of Bayesian MoE models came with the work of Ueda and Ghahramani [1645], who proposed a variational Bayes framework capable of performing automatic model selection. Unlike traditional MoE models that require the number of experts to be specified a priori, their method treated this quantity as a latent random variable, enabling the model to infer an appropriate level of complexity

from data. By optimizing an evidence lower-bound (ELBO), their approach facilitated the adaptive pruning of redundant or unnecessary experts, thus balancing model fit and parsimony. Building on this foundation, authors in [1381] introduced a nonparametric Bayesian formulation of MoEs by incorporating Dirichlet Processes (DPs) for model selection and Gaussian Processes (GPs) as flexible expert functions. This allows the number of mixture components to grow dynamically with the complexity of the data, removing the need to fix the number of experts in advance. Moreover, by employing GPs, the model can capture highly nonlinear relationships within each expert domain while retaining a coherent probabilistic framework.

Inference in Bayesian MoE models, however, remains a non-trivial task due to the complexity of the posterior distribution, particularly when dealing with non-conjugate priors or hierarchical model structures. Traditional Markov Chain Monte Carlo (MCMC) methods, such as those discussed in [1313], provide asymptotically exact samples from the posterior, but they are often computationally expensive and slow to converge, especially in high-dimensional or large-scale settings. As an alternative, variational inference techniques have been developed to approximate the posterior more efficiently. Notably, Bishop and Svensén [162] applied variational methods to MoEs, demonstrating that such approximations could yield accurate predictions with significantly reduced computational costs. Variational inference also provides a natural framework for model comparison and selection via the ELBO, further enhancing its utility. In recent years, the rise of probabilistic programming languages such as Stan [240] and Pyro [158] has dramatically simplified the implementation of Bayesian MoEs. These frameworks support advanced inference algorithms, including Hamiltonian Monte Carlo (HMC), No-U-Turn Sampler (NUTS) [762], and stochastic variational inference [763], which enable more scalable and expressive models for the expert functions and the gating functions. The modular design of probabilistic programming also allows seamless integration with deep learning architectures, facilitating the construction of hybrid models that leverage both probabilistic reasoning and the representational power of neural networks for MoE.

Bayesian MoE models have demonstrated their utility across a broad range of application domains. In time-series forecasting, they offer robustness to regime shifts and non-stationary patterns by allowing expert specialization over different temporal segments [1728]. In personalized medicine, Bayesian MoEs have been used to capture patient heterogeneity, account for latent subpopulations, and improve treatment effect estimation in heterogeneous clinical cohorts [1784]. In control and robotics, they enable interpretable mode switching in dynamical systems, allowing agents to transition smoothly between operational modes in response to changing environmental conditions [81]. Despite the growing dominance of non-Bayesian deep learning methods, recent work at the intersection of Bayesian inference and large-scale neural architectures is reviving interest in probabilistic MoEs.

36.2 Softmax-gated mixture of experts models

In this section, we introduce simple softmax MoE models for regression and classification, and provide intuition on the generative mechanism they imply.

36.2.1 Regression

Consider an iid sample $(Y_i, X_i)_{i=1}^n$, where each observation i consists of a response variable $Y_i \in \mathbb{R}$ and an input vector $X_i \in [0,1]^d$. As in classical regression tasks, the goal is to learn the conditional distribution of the response given the inputs. To that end, we assume that the sample is generated as follows: $(Y_i, X_i) \overset{\text{iid}}{\sim} g_{G^\star}$, where the joint distribution $g_{G^\star}$ factorizes as

$$g_{G^\star}(Y, X) = f_{G^\star}(Y \mid X) \times p(X).$$

Here, because we are not interested in learning the marginal distribution of the input vector, we simply assume p to be known and to coincide with the uniform density on $[0,1]^d$. As for the conditional density $f_{G^\star}(\cdot \mid X)$, setting

$$G^\star := \sum_{j=1}^{K^\star} \exp(\alpha_{0j}^\star) \delta_{(\alpha_{1j}^\star, \beta_j^\star, \sigma_j^{\star 2})},$$

we assume the form

$$f_{G^\star}(Y \mid X) := \sum_{j=1}^{K^\star} \frac{\exp(\alpha_{0j}^\star + X^\top \alpha_{1j}^\star)}{\sum_{\ell=1}^{K^\star} \exp(\alpha_{0\ell}^\star + X^\top \alpha_{1\ell}^\star)} \mathcal{N}(Y \mid \zeta(X, \beta_j^\star),\, \sigma_j^{\star 2}). \tag{36.1}$$

where $\alpha_{0j}^\star \in \mathbb{R}$, $\alpha_{1j}^\star \in \mathbb{R}^d$, $\zeta : [0,1]^d \times \mathbb{R}^E \to \mathbb{R}$ is a mean (or *expert*) function parametrised by $\beta \in \mathbb{R}^E$, and $\mathcal{N}(\cdot \mid \mu, \sigma^2)$ stands for the normal density with mean $\mu \in \mathbb{R}$ and variance $\sigma^2 > 0$. A convenient way to parametrise

this conditional density is through the measure[1] $G^\star$ defined above, which collects all of the model parameters and which, as we will see later in the chapter, may be a useful object for the study of inferential procedures.

Let us now interpret the structure of this Softmax-gated Mixture-of-Gaussian-Experts (SMoGE) generative model. Given a value $X \sim p$ of the inputs, the response Y is generated according to a convex combination of $K^\star$ normal distributions, termed *experts*, each with its own input-dependent mean $\zeta(X, \beta_j^\star)$ and variance $\sigma_j^{\star 2}$. Here the expert function ζ can be as simple as an affine transformation of X (so β is a linear coefficient vector together with a bias term), or a more flexible mapping based, for instance, on a deep neural network (with β collecting the network weights).

An important feature, which sets apart the MoE model from traditional mixtures, is that the weight assigned to each expert depends on the input value X (rather than being a constant). In particular, this dependence is modelled with the softmax function

$$\begin{aligned} S : \mathbb{R}^{K^\star} &\to \Delta_{K^\star} \\ \boldsymbol{z} &\mapsto \frac{\exp(\boldsymbol{z})}{\mathbf{1}^\top \exp(\boldsymbol{z})}, \end{aligned}$$

where $\Delta_{K^\star} := \{\boldsymbol{w} \in \mathbb{R}^{K^\star} : \boldsymbol{w} \geq \mathbf{0},\, \mathbf{1}^\top \boldsymbol{w} = 1\}$ is the $(K^\star - 1)$-dimensional simplex, and the function exp is applied element-wise to each coordinate of $\boldsymbol{z} \in \mathbb{R}^{K^\star}$. In this case, the softmax function takes as input the vector of linear combinations[2] $(\alpha_{0j}^\star + X^\top \alpha_{1j}^\star)_{j=1}^{K^\star}$ and outputs the weights corresponding to the $K^\star$ experts.

To further understand the structure of the model, it is useful to introduce a set of latent *gating* variables $Z_1, \ldots, Z_n \in \{1, \ldots, K^\star\}$ and rewrite the model in hierarchical form as follows:

$$\begin{aligned} Y_i \mid X_i, Z_i &\overset{\text{ind}}{\sim} \mathcal{N}\left(\cdot \mid \zeta(X_i, \beta_{Z_i}^\star),\, \sigma_{Z_i}^{\star 2}\right), \\ Z_i \mid X_i &\overset{\text{ind}}{\sim} \mathcal{C}\left(\cdot \mid (1, ..., K^\star),\, S\left((\alpha_{0j}^\star + X_i^\top \alpha_{1j}^\star)_{j=1}^{K^\star}\right)\right), \end{aligned}$$

where $Z \sim \mathcal{C}(\cdot \mid \boldsymbol{a}, \boldsymbol{w})$, for $\boldsymbol{a} \in \mathbb{R}^{K^\star}$ and $\boldsymbol{w} \in \Delta_{K^\star}$, means $Z = a_j$ with probability w_j for all $j = 1, \ldots, K^\star$.

In other words, in the SMoGE model the conditional generation of the response variable Y_i can be interpreted as follows: given a certain input value $X_i \sim p$, first an indicator Z_i is generated, according to X_i-dependent softmax probabilities, which "gates" observation i to expert $Z_i \in \{1, \ldots, K^\star\}$. Then, given this assignment, the output Y_i is generated by a normal distribution with X_i-dependent mean parametrised by ζ. This generative view further clarifies a key source of added flexibility of SMoGE models compared to simple mixtures: SMoGE models allow for output observations whose distribution depends on inputs not only in terms of the mean structure, but also in terms of the probability of being assigned to a specific mean structure itself.

36.2.2 Classification

If the output belongs to a finite set of classes $Y_i \in \{1, \ldots, C\}$ (instead of being continuous as in a regression problem), then we face a classification problem—say, to determine whether a certain image X_i depicts a certain type of animal Y_i among a list labelled 1 through C [1239]. Similarly to before, this can be tackled by specifying the following MoE generative model expressed through the conditional probability mass function of Y given $X \sim p$:

$$\begin{aligned} &f_{G^\star}(Y \mid X) := \\ &\quad \sum_{j=1}^{K^\star} \frac{\exp(\alpha_{0j}^\star + X^\top \alpha_{1j}^\star)}{\sum_{\ell=1}^{K^\star} \exp(\alpha_{0\ell}^\star + X^\top \alpha_{1\ell}^\star)} \times \mathcal{C}\left(Y \mid \{1, \ldots, C\},\, S\left(\zeta(X, \beta_{j1}^\star), \ldots, \zeta(X, \beta_{jC}^\star)\right)\right). \end{aligned}$$

The implied generative model is analogous to the regression case, and can be similarly rewritten in hierarchical form by introducing suitable latent gating variables. The only difference is that, once an observation has been gated to expert j according to input-dependent probabilities, the output is generated according to an expert-specific categorical distribution on $\{1, \ldots, C\}$, with probabilities parametrised by a softmax transformation of the expert functions $\zeta(X, \beta_{j1}), \ldots, \zeta(X, \beta_{jC})$. Please take a moment to notice that, while the softmax function appears twice in this model specification, it serves two distinct purposes in each case: in the first instance, to gate observations to a certain expert; in the second one, to assign each observation to one of the C classes under consideration.

[1] Please note that $G^\star$ is not a probability measure, as its weights, while positive, are not guaranteed to sum to 1.

[2] Please note that we force the softmax gating to depend linearly on the input vector for a matter of convenience. This linear specification can readily be replaced by more flexible functions, if needed.

36.3 Fitting MoE models

Having defined the basic MoE models, we now turn to the question of how to fit them in practice using Bayesian methods.

A first basic issue to deal with is the fact that the number $K^\star$ of experts is not in general known in advance. To address this, there are two basic strategies at the disposal. First, we can simply specify an MoE model with $K \in \mathbb{N}$ experts, where K is large enough so that $K \geq K^\star$. In this case, we talk about an *overspecified* MoE model. Alternatively, as we will be working in the Bayesian framework, one can treat the number of experts K as yet another parameter to estimate with the available data by placing a prior on it (say, a shifted Poisson distribution with support on the positive integers) and making posterior inference on it together with the other parameters. Each strategy has its advantages and disadvantages. Overspecified models are, on the one hand, relatively simple to fit (see below) and are capable of approximating the true model—just think of a sequence of MoE density with gating parameters that make the gating weights of the extra experts closer and closer to 0; on the other hand, in real world situations where data is limited and careful modelling is essential for good inference and prediction, specifying too high a value of K may result in poor performance. Instead, placing a prior for K leads to principled inference on this unknown quantity, and allows to learn it automatically with the other parameters. However, notice that, as K varies, so does the dimension of the parameter space, as more (or fewer) gating and expert parameters are included in the model. This complicates significantly the computational burden of fitting the model, as ad hoc procedures that take into account the *transdimensional* nature of the inference problem are needed [662]. To keep the exposition clear, we focus on the first choice, i.e., we fix K and we assume that it has been selected large enough so as to subsume the true underlying MoE model.[3]

Given the model[4] we specified in the previous section, we are interested in performing inference on two distinct sets of parameters: the gating parameters $(\alpha_{0j}, \alpha_{1j})_{j=1}^K$, and the expert parameters $(\beta_j)_{j=1}^K$. According to the standard Bayesian pipeline, then, we assign a prior distribution Π to these parameters. To keep the notation compact, recall that the measure $G = \sum_{j=1}^K \exp(\alpha_{0j})\delta_{(\alpha_{1j},\beta_j,\sigma_j^2)}$ contains all the information about the MoE parameters, so we will think of the prior as being placed on the space of all such measures G. While it may seem abstract, this is just to avoid notational clutter, and all we mean by $\Pi(G)$ is that a prior is placed on the finite-dimensional (Euclidean) space of MoE model parameters.

To make inference, we compute the posterior distribution

$$\Pi(G \mid Y_{1:n}, X_{1:n}) \propto \Pi(G) \times \prod_{i=1}^n f_G(Y_i \mid X_i). \tag{36.2}$$

If we care about predicting a new value Y_{n+1} of the output given a certain input value X_{n+1}, we may also compute the posterior predictive distribution

$$\hat{f}_n(Y_{n+1} \mid X_{n+1}) := \int f_G(Y_{n+1} \mid X_{n+1})\, \Pi(\mathrm{d}G \mid Y_{1:n}, X_{1:n}). \tag{36.3}$$

It should be clear that, given the rather complicated density $f_G(Y \mid X)$ associated to MoE models, it is not possible to compute exactly the posterior distribution (36.2) nor the predictive (36.3). Therefore, we now briefly examine two popular options to perform computation: Markov Chain Monte Carlo (MCMC) sampling and variational inference (VI).

36.3.1 Markov Chain Monte Carlo sampling

A first popular strategy is to use MCMC sampling to approximate the posterior distribution. As discussed in depth in other parts of the book, this involves simulating a Markov chain having as state space the parameter space and as a stationary distribution the posterior distribution, and then treating the realizations of the chain (after discarding an appropriate burn-in initial segment to ensure convergence of the chain to stationarity) as samples from the posterior.

[3] Two further subtle points, related to the problem of selecting K, are the following: first, while a true data-generating process may exist, the latter may not be expressible as an MoE model, so a true $K^\star$ may not exist, and one may rather want to select a value of K that ensures good predictive performance; second, if one takes the subjective Bayesian perspective seriously, no "true" data-generating process—in the usual iid sense—may exist at all (let alone a true number of experts). Therefore, although convenient to study the theoretical properties of the models, the implications of choosing a "correct" value of $K^\star$ should not be over-interpreted.

[4] For the sake of convenience, we focus on MoE models for regression, although the treatment easily extends to the classification setting as well.

The smooth parametrisation of MoE models lends them to any state of the art MCMC sampler, such as Hamiltonian Monte Carlo and Langevin Dynamics. We therefore omit a detailed treatment of this aspect, and refer the readers to the relevant chapters within the book. Nevertheless, a couple of observations, which are specific to MoE models, are in order.

First, the similarity with traditional mixture models may lead one to expect to have access to full conditional distributions that would allow to perform Gibbs updates for the gating and expert parameters. This is unfortunately not the case in general, at least for two reasons:

1. The softmax gating parametrisation, while intuitive and very flexible, does not allow for conditionally conjugate priors, especially as the input-dependent weights interact with the expert densities when one wishes to perform MCMC updates;

2. The expert function ζ is rarely of simple enough form (e.g., linear) so as to allow Gibbs updates.

Nevertheless, in simple cases such as in the in the context of linear regression with conditionally conjugate priors, it may be possible to perform Gibbs updates for at least some of the parameters.

Second, MoE models naturally lead to a model-based clustering of the observations. Recall that, by introducing the latent expert assignment variables Z_i, one can interpret the generative model as first assigning observations to a certain cluster (expert), then generating a response according to a conditional density that is specific to the cluster. Therefore, by making inference on the latent Z_i's, one can learn useful features of the clustering structure of the observed sample. It turns out that many useful MCMC algorithms do rely on this augmentation strategy and, as a consequence, allow to obtain posterior summaries of the data clustering.

36.3.2 Variational inference

A second approach, which is standard in the BDL literature, is to fit MoE models via variational inference. The basic idea behind this strategy is simple: instead of sampling from the posterior distribution (asymptotically) exactly, find the distribution that best approximates the posterior within a tractable family of distribution (the so called variational family). This methodology is also standard and explored in depth elsewhere in the book, therefore we omit a detailed treatment of the topic.

The following, however, are two important remarks. First, variational inference is to be preferred, in terms of scalability, to MCMC, especially in situations where the expert functions are highly non-linear (for instance, deep neural networks) and where the parameter space is very high-dimensional: in these cases, Metropolis-Hastings adjustment will be most likely needed, a feature that, paired with high-dimensionality, is very likely to lead to a high computational cost of MCMC. As usual, however, depending on the chosen variational family and the nature of the data, variational inference may suffer from lower posterior approximation accuracy compared to MCMC (which is asymptotically exact), especially in terms of posterior variance under-estimation. Therefore, if accurate posterior uncertainty quantification is essential, MCMC may be preferable even at the expense of higher computational costs.

Second, the complicated likelihood associated to MoE models makes it infeasible to employ simple variational inference algorithms based on conjugate priors (e.g., coordinate ascent variational inference). Instead, more flexible methods are necessary, such as *black-box variational inference* [939, 1377]. The latter is a powerful technique that, by approximating the gradient of the evidence lower-bound (the objective we optimize in variational inference) with an average of terms with respect to Monte Carlo samples from the variational family, allows to fit elaborate models like the MoE ones. Key strengths of this strategy are that (i) it allows to take advantage of the powerful automatic differentiation tools available within routinely used modern deep learning libraries, and (ii) it easily allows for data mini-batching, which is essential if n is large.

36.4 Asymptotic properties

In this section, we study some asymptotic consistency properties of the softmax-gated Gaussian mixture of experts, namely the convergence behavior of density estimation and parameter estimation. To give some preliminary background, asymptotic consistency is a key criterion for evaluating statistical estimation procedures, and its study in the context of Bayesian posterior distributions has a long history. Heuristically, one concludes that the posterior distribution in a given model is consistent if the mass it assigns to any neighbourhood (no matter how small) of the data-generating parameter converges to 1 almost surely (or in probability) as the sample size grows to infinity. The first results in this direction trace back to Doob [442] establishing, under mild conditions, posterior consistency on a set of data-generating parameters of prior measure 1. Later developments focused on

parametric models for iid data, such as Walker [1695] and Berk [126], building on frequentist consistency results for maximum likelihood estimators (however see Bariletto et al. [96] for a recent perspective on parametric consistency independent of frequentist techniques). The focus has since mostly shifted towards consistency in nonparametric models [100, 616, 1476, 1696].

In what follows, instead of looking at consistency, we look at the related concept of contraction rates: rather than making sure that the MoE posterior distribution consistently estimates the true MoE density and parameters by placing vanishing mass outside of any neighbourhood of the true object of inference, we ask *how fast* we can shrink such neighbourhoods and still obtain posterior concentration within them. To simplify the exposition, we will only focus on regression, although quite similar results can be stated in the context of classification.

36.4.1 Density estimation

As described in Section 36.2.1, a primary objective in classical regression tasks is to learn the conditional distribution of the response given the input, which is characterized by its conditional density function. Having access to this knowledge allows us to capture the convergence rate of density estimation, expressed as a function of the sample size. As a consequence, we are able to determine the amount of data necessary to approximate a softmax-gated MoE density within a given estimation error. In particular, let us recall that the iid sample of size n, $(Y_i, X_i)_{i=1}^n$, is assumed to have been generated from the joint distribution with the density function $g_{G^\star}(Y, X) = f_{G^\star}(Y \mid X) \times p(X)$, where the conditional density $f_{G^\star}(\cdot \mid X)$ is given by

$$f_{G^\star}(Y \mid X) := \sum_{j=1}^{K^\star} \frac{\exp(\alpha_{0j}^\star + X^\top \alpha_{1j}^\star)}{\sum_{\ell=1}^{K^\star} \exp(\alpha_{0\ell}^\star + X^\top \alpha_{1\ell}^\star)} \cdot \mathcal{N}(Y \mid \zeta(X, \beta_j^\star),\, \sigma_j^{\star 2}). \tag{36.4}$$

A question arising here is how accurately we can estimate the joint density function $g_{G^\star}(Y, X)$ using the iid sample $(Y_i, X_i)_{i=1}^n$. To answer this question, it is necessary to choose an appropriate metric to measure the distance between the estimated and the ground-truth density among the following three popular choices:

(i) Hellinger distance. For any two probability density functions p and q defined on a Borel set $S \subseteq \mathbb{R}^m$, the Hellinger distance is defined as

$$d_H(p, q) := \left(\frac{1}{2} \int_S \left(\sqrt{p} - \sqrt{q}\right)^2 \mathrm{d}\lambda \right)^{1/2},$$

where λ denotes a σ-finite measure on S;

(ii) Total Variation (TV) distance. The TV distance is given by

$$d_{TV}(p, q) := \int_S |p - q| d\lambda;$$

(iii) Kullback-Leibler (KL) divergence. The KL divergence is computed as

$$\mathrm{D}_{\mathrm{KL}}(p \| q) := \int_S \log(p/q) p \mathrm{d}\lambda.$$

In the Bayesian asymptotics literature, the TV distance and Hellinger distance are usually preferred over the KL divergence because they provide more stable and decision-theoretically meaningful measures of density discrepancies (though they are topologically weaker than the KL divergence). In particular, the KL divergence can be infinite when estimated densities do not admit the same support as the ground-truth density. By contrast, both the TV and Hellinger distances remain finite and bounded, making them more robust to support mismatches and better behaved for asymptotic analysis. Because the Hellinger and TV metrics are well known to be topologically equivalent, we will measure density convergence in the Hellinger distance without loss of generality. Nevertheless, the KL divergence will be employed to measure prior concentration around the true density function.

Before presenting the main result on posterior contraction rates for density estimation, we need to introduce some mild conditions on the parameter space. In particular, we will work under the assumption that all K experts share the same parameter space Θ, i.e. $\theta = (\alpha_{0j}, \alpha_{1j}, \beta_j, \sigma_j^2) \in \Theta$, and we will denote by H the subspace of expert parameters $\eta = (\beta_j, \sigma_j^2) \in H$. For any $K \in \mathbb{N}$, denote by $\mathcal{G}_K$ the set of all K-expert mixing measures, that is,

$$\mathcal{G}_K := \left\{ G = \sum_{j=1}^{K} \exp(\alpha_{0j})\, \delta_{(\alpha_{1j}, \beta_j, \sigma_j^2)} \;:\; (\alpha_{0j}, \alpha_{1j}, \beta_j, \sigma_j^2) \in \Theta,\ j = 1, \ldots, K \right\}.$$

Assumption 36.1. *The parameter space Θ is compact. In particular, the variance space is bounded away from 0.*

Assumption 36.2. *The prior Π satisfies*

$$\Pi\left(\mathrm{D}_{\mathrm{KL}}(g_{G^\star}\|g_G) \leq A\frac{\log t}{t}\right) \geq \frac{B}{t} \tag{36.5}$$

for some $A, B > 0$ and all large $t > 0$.

Assumption 36.3. *There exists $L > 0$ such that, for all $x \in \mathcal{X}$ and $\beta, \beta' \in \mathbb{R}^E$ in the parameter space,*

$$|\zeta(x,\beta) - \zeta(x,\beta')| \leq L\|\beta - \beta'\|. \tag{36.6}$$

Assumption 36.1 is a standard compactness assumption that simplifies theoretical analysis, while 36.2 requires that enough prior mass be placed around the data-generating parameters [619]; see also Bariletto et al. [97] for more details on the second requirement for MoE models. Additionally, Assumption 36.3 allows the expert function to be Lipschitz continuous. Given these three assumptions, we are ready to state main results about the convergence rate of density estimation.

Theorem 36.1. *Consider a SMoGE model. Under Assumptions 36.1-36.3, the following holds for any sequence $M_n \to \infty$:*

$$\lim_{n\to+\infty} \Pi\left(\left\{G \in \mathcal{G}_K : d_H(g_G, g_{G^\star}) \gtrsim M_n\sqrt{\log(n)/n}\right\} \mid (X_i, Y_i)_{i=1}^n\right) = 0$$

in $g_{G^\star}^n$-probability, where $\gtrsim$ denotes an inequality up to a constant.

From Theorem 36.1, we learn that under the Hellinger distance, the posterior distribution over MoE densities contracts around the ground-truth density function at a rate of order $\mathcal{O}_P(\sqrt{\log(n)/n})$, which is parametric in the sample size n.

Now, it is natural to ask about the consistency of the above posterior convergence on the space of densities, or equivalently, the identifiability of softmax-gated Gaussian MoE (SMoGE). In particular, assume that there are two conditional density functions corresponding to two softmax-gated Gaussian MoE models that admit the same values almost everywhere. Then, in Proposition 36.1, we will determine if these two models share the same parameter set. For that purpose, it is necessary to introduce an identifiability assumption on the expert function and a distinction assumption on the model parameters.

Assumption 36.4. *The expert function $\zeta : [0,1]^d \times \mathbb{R}^E \to \mathbb{R}$ is identifiable, that is, if for each $K \in \mathbb{N}$, the following holds: if there exist distinct parameters $(\beta_1, \beta_2, \ldots, \beta_K)$ and $(\beta_1', \beta_2', \ldots, \beta_K')$ such that for almost every $x \in \mathcal{X}$, we can find permutation function $\sigma_x : \{1, 2, \ldots, K\} \to \{1, 2, \ldots, K\}$ such that $\zeta(x, \beta_{\sigma_x(i)}) = \zeta(x, \beta_i')$ for all $1 \leq i \leq K$, then $\{\beta_1, \beta_2, \ldots, \beta_K\} \equiv \{\beta_1', \beta_2', \ldots, \beta_K'\}$.*

For example, it can be validated that for any injective function $\varphi : \mathbb{R} \to \mathbb{R}$, an expert function of the form $\zeta(x, (\beta_1, \beta_0)) = \varphi(\beta_1^\top x + \beta_0)$, where $\beta_1 \in \mathbb{R}^d$ and $\beta_0 \in \mathbb{R}$, is identifiable.

Assumption 36.5. *The discrepancies between gating parameters $\alpha_{1i}^\star - \alpha_{1j}^\star$, for $1 \leq i < j \leq K^\star$, are pairwise distinct. In addition, expert parameters $(\beta_j^\star, \sigma_j^{\star 2})$, for $1 \leq j \leq K^\star$, are also pairwise different. The same holds for the parameter configuration associated to Π-almost every mixing measure G.*

Proposition 36.1. *Under Assumptions 36.4–36.5, if there exist two mixing measures $G = \sum_{j=1}^K \exp(\alpha_{0j})\delta_{(\alpha_{1j},\beta_j,\sigma_j^2)}$ and $G^\star = \sum_{j=1}^{K^\star} \exp(\alpha_{0j}^\star)\delta_{(\alpha_{1j}^\star,\beta_j^\star,\sigma_j^{\star 2})}$ on Θ such that $g_G(y,x) = g_{G^\star}(y,x)$ for almost every (y,x), then we obtain $K = K^\star$ and*

$$G \equiv G_{t_0,t_1}^\star := \sum_{j=1}^{K^\star} \exp(\alpha_{0j}^\star + t_0)\delta_{(\alpha_{1j}^\star + t_1, \beta_j^\star, \sigma_j^{\star 2})},$$

for some $t_0 \in \mathbb{R}$ and $t_1 \in \mathbb{R}^d$.

The result of Proposition 36.1 indicates that the softmax-gated Gaussian MoE is identifiable up to the translation of gating parameters. This dependence on the gating parameter translation comes from the softmax gating structure in equation (36.4). More specifically, the values of softmax weights will remain unchanged if we translate the gating parameters $\alpha_{0j}^\star$ and $\alpha_{1j}^\star$ to $\alpha_{0j}^\star + t_0$ and $\alpha_{1j}^\star + t_1$, respectively, for some $t_0 \in \mathbb{R}$ and $t_1 \in \mathbb{R}^d$, which is mathematically presented as

$$\frac{\exp(\alpha_{0j}^\star + t_0 + X^\top(\alpha_{1j}^\star + t_1))}{\sum_{\ell=1}^{K^\star} \exp(\alpha_{0\ell}^\star + t_0 + X^\top(\alpha_{1\ell}^\star + t_1))} = \frac{\exp(\alpha_{0j}^\star + X^\top \alpha_{1j}^\star)}{\sum_{\ell=1}^{K^\star} \exp(\alpha_{0\ell}^\star + X^\top \alpha_{1\ell}^\star)}, \tag{36.7}$$

for all $j = 1, 2, \ldots, K^\star$. It should be noted that if we fix a pair of gating parameters to be zero, e.g., $\alpha_{0K^\star}^\star = 0$ and $\alpha_{1K^\star}^\star = 0_d$, then the numerator of the last softmax weight will be equal to one, that is, $\exp(\alpha_{0K^\star}^\star + X^\top \alpha_{1K^\star}^\star) = 1$. Therefore, the softmax weights will no longer be invariant to the translation of their parameters. As a consequence, the softmax-gated Gaussian MoE will become identifiable. Then, we can say that the posterior convergence on the space of densities in Theorem 36.1 is consistent.

36.4.2 Parameter estimation

In this section, we examine the convergence properties of parameter estimation within the SMoGE framework. Recall that during the training of MoE architectures in deep learning, each expert is expected to specialize in distinct subproblems or latent regions of the data distribution. For example, in large-scale language models based on the MoE paradigm [392, 603, 843], certain experts will capture syntax, another might focus on semantics, while the rest concentrate on context.

From a Bayesian perspective, this specialization can be viewed as a form of posterior concentration, where each expert's parameters adapt to a particular mode of the underlying data generation process. The rate at which this concentration occurs—i.e., the rate of expert specialization—is therefore a key quantity of interest [356, 1241, 1257]. A natural approach to studying this phenomenon is through the convergence behavior of parameter estimation, as the asymptotic or finite-sample convergence rates of the model parameters directly govern the rate at which experts specialize.

Intuitively, a faster convergence rate indicates that an expert requires fewer data samples to effectively capture its designated region of the input space, given a desired approximation accuracy. By contrast, slower convergence suggests higher sample complexity and delayed specialization. Consequently, analyzing the convergence of parameter estimation not only provides theoretical insights into the dynamics of expert learning but also informs strategies to improve the sample efficiency and inductive bias of MoE models.

Notably, the posterior convergence on the space of densities in Theorem 36.1 plays a vital role in establishing the convergence rate of parameter estimation. In particular, if there exists a metric to capture the discrepancies between parameter estimation and ground-truth parameters denoted by $\mathcal{L}(G, G^\star)$ such that it is lower bounded by the Hellinger distance between density estimation and the ground-truth density, that is,

$$d_H(g_G, g_{G^\star}) \gtrsim \mathcal{L}(G, G^\star), \tag{36.8}$$

for any feasible mixing measures G. Then, by combining the results of Theorem 36.1 and equation (36.8), we will obtain the convergence rate of parameter estimation. Therefore, in the sequel, we will focus on constructing the metric $\mathcal{L}(G, G^\star)$. To this end, we divide the analysis into two main settings referred to as *(i) exactly-specified setting* where the number of ground-truth experts $K^\star$ is known, and *(ii) over-specified setting* where $K^\star$ becomes unknown.

Exactly-specified setting. First, we perform the convergence analysis of parameter estimation when the number of ground-truth experts $K^\star$ is known. For simplicity, we set the last pair of gating parameters to zero to ensure that the softmax-gated Gaussian MoE is identifiable. Next, it should be noted that the mixing measure $G^\star = \sum_{j=1}^{K^\star} \exp(\alpha_{0j}^\star)\delta_{(\alpha_{1j}^\star, \beta_j^\star, \sigma_j^{\star 2})}$ is only identifiable up to a permutation of parameters. In particular, if there exist two equivalent mixing measures, $G \equiv G'$, then it only implies $(\alpha_{0j}, \alpha_{1j}, \beta_j, \sigma_j^2) = (\alpha_{0\phi(j)}^\star, \alpha_{1\phi(j)}^\star, \beta_{\phi(j)}^\star, \sigma_{\phi(j)}^{\star 2})$ for some permutation ϕ of the set $\{1, 2, \ldots, K^\star\}$. To identify that permutation, we employ the notion of Voronoi cells, which is defined as follows. Given an arbitrary mixing measure G with $K^\star$ components, we partition its components into the following Voronoi cells, which are generated by the components of ground-truth mixing measure $G^\star$:

$$\mathcal{V}_j \equiv \mathcal{V}_j(G) := \{i \in \{1, 2, \ldots, K^\star\} : \|\omega_i - \omega_j^\star\| \leq \|\omega_i - \omega_\ell^\star\|,\ \forall \ell \neq j\}, \tag{36.9}$$

where $\omega_i := (\alpha_{1i}, \beta_i, \sigma_i^2)$ and $\omega_j^\star := (\alpha_{1j}^\star, \beta_j^\star, \sigma_j^{\star 2})$ for any $j \in \{1, 2, \ldots, K^\star\}$. Notably, when the components of G are sufficiently close to those of $G^\star$, then each of these Voronoi cells will contain one element. As a result,

we can characterize the aforementioned permutation of parameters. Then, the corresponding Voronoi-based loss capturing parameter discrepancies is given by

$$\mathcal{L}(G, G^\star) := \sum_{j=1}^{K^\star} \Big| \sum_{i \in \mathcal{V}_j} \exp(\alpha_{0i}) - \exp(\alpha_{0j}^\star) \Big| + \sum_{j=1}^{K^\star} \sum_{i \in \mathcal{V}_j} \exp(\alpha_{0i}) \Big[\|\alpha_{1i} - \alpha_{1j}^\star\| + \|\beta_i - \beta_j^\star\| + |\sigma_i^2 - \sigma_j^{\star 2}| \Big], \tag{36.10}$$

where we denote Above, if a Voronoi cell $\mathcal{V}_j$ is empty, then we let the corresponding summation term be zero. Additionally, it can be checked that $\mathcal{L}(G, G^\star) = 0$ if and only if $G \equiv G^\star$. Thus, when $\mathcal{L}(G, G^\star)$ is sufficiently small, the differences $\alpha_{1i} - \alpha_{1j}^\star$, $\beta_i - \beta_j^\star$, and $\sigma_i^2 - \sigma_j^{\star 2}$ are also small. This property indicates that $\mathcal{L}(G, G^\star)$ is an appropriate loss function for measuring parameter discrepancies. However, since the loss $\mathcal{L}(G, G^\star)$ is not symmetric, it is not a proper metric. Lastly, computing the Voronoi loss function $\mathcal{L}(G, G^\star)$ is efficient as its computational complexity is at the order of $\mathcal{O}(K^{\star 2})$. Note that the key benefits of Voronoi-based loss functions are expressed more clearly in the overspecified setting where the rates for estimating overspecified parameters (parameters of $G^\star$ fitted by at least two parameters) are slower than those for estimating exactly-specified parameters (parameters of $G^\star$ fitted by one parameter) [1238]. In particular, Nguyen et al. [1238] have shown that Voronoi-based loss functions are capable of characterizing those rates, while the generalized Wasserstein used in Ho et al. [756] are incapable of capturing the rate difference.

Given the Voronoi-based loss in equation (36.10), we are ready to present results on the convergence rate of parameter estimation in the following theorem.

Theorem 36.2. *Suppose that Assumptions 36.1–36.5 hold true. When the number of ground-truth experts $K^\star$ is known, we find that*

$$d_H(g_G, g_{G^\star}) \gtrsim \mathcal{L}(G, G^\star), \tag{36.11}$$

for all mixing measures of the form $G = \sum_{j=1}^{K^\star} \exp(\alpha_{0j}) \delta_{(\alpha_{1j}, \beta_j, \sigma_j^2)}$. As a consequence, the Hellinger lower bound (36.11) *and the result of Theorem 36.1 together imply that for any sequence $M_n \to \infty$, we have*

$$\lim_{n \to +\infty} \Pi \left(\left\{ G \in \mathcal{G}_K : \mathcal{L}(G, G^\star) \gtrsim M_n \sqrt{\log(n)/n} \right\} \mid (X_i, Y_i)_{i=1}^n \right) = 0. \tag{36.12}$$

in $g_{G^\star}^n$-probability.

A few remarks regarding the results of Theorem 36.2 are in order.

(i) Convergence rate of parameter estimation: From equation (36.12) and the formulation of the Voronoi-based loss $\mathcal{L}(G, G^\star)$ in equation (36.10), it follows that the convergence rates of estimations of parameters $\exp(\alpha_{0j}^\star)$, $\alpha_{1j}^\star$, $\beta_j^\star$, and $\sigma_j^{\star 2}$ admit the same order of $\mathcal{O}_P(\sqrt{\log(n)/n})$, which is parametric on the sample size n.

(ii) Convergence rate of expert estimation: Assume that the expert function $\zeta(x, \beta)$ is Lipschitz continuous with respect to its parameter β, which yields

$$|\zeta(x, \beta_i) - \zeta(x, \beta_j^\star)| \lesssim \|\beta_i - \beta_j^\star\|,$$

for all $i \in \mathcal{V}_j$ and $j \in [K^\star]$. It should be noted that the above Lipschitz continuity condition is satisfied by several expert choices in practice, including two-layer feed-forward networks used in the Transformer architecture in large language models [1669]. Then, from the rate for estimating parameter $\beta_j^\star$ in part (i), we deduce that the estimators of experts $\zeta(x, \beta_j^*)$ also share the convergence rate of the parametric order $\mathcal{O}_P(\sqrt{\log(n)/n})$, for $j \in \{1, 2, \ldots, K^\star\}$. As a consequence, to estimate an expert $\zeta(x, \beta_j^\star)$ with a given error $\epsilon > 0$, we need a polynomial number of data points $\mathcal{O}(\epsilon^{-2})$.

Overspecified setting. Next, we extend the above analysis to the scenario in which the number of ground-truth experts $K^\star$ becomes unknown. A popular approach to this case is to over-specify the ground-truth model in equation (36.4) by a softmax-gated Gaussian mixture of K experts, where $K > K^\star$. Then, according to the pigeonhole principle, when this fitted model becomes sufficiently close to the ground-truth model, at least one of the ground-truth components $(\alpha_{11}^\star, \beta_1^\star, \sigma_1^{\star 2}), \ldots, (\alpha_{1K^\star}^\star, \beta_{K^\star}^\star, \sigma_{K^\star}^{\star 2})$ will be approximated by more than one fitted component. Thus, there exists a Voronoi cell $\mathcal{V}_j$, where $j \in \{1, 2, \ldots, K^\star\}$, having more than one element. As a result, the Voronoi-based loss in equation (36.10) cannot be applied to the overspecified setting, indicating that

the convergence rates of parameter and expert estimations will be affected. In particular, Nguyen et al. [1238] have shown that these rates depend on the cardinality of Voronoi cells, thereby becoming substantially slower than the parametric rate of order $\mathcal{O}_P(\sqrt{\log(n)/n})$ achieved in the exactly-specified setting. However, due to the technical complexity of this analysis, which is beyond the scope of this chapter, we would rather refer the reader to Nguyen et al. [1238] rather than present it in further detail here.

36.5 Model selection

An important open question in Bayesian MoE models is how to choose the number of experts K before training. As a rough rule of thumb, setting K large ensures either $K \geq K^\star$ (if the true data-generating process is MoE), so the true density is included, or, under misspecification, makes the model rich enough to approximate the unknown distribution.

However, selecting a very large K has drawbacks. First, the parameter space grows linearly with K, making training computationally expensive, especially with high-dimensional expert or gating functions. Second, over-specification harms parameter estimation accuracy. Third, in applications where inference and interpretability matter, fewer parameters aid both clarity and uncertainty quantification. Thus, it is desirable to have a *model selection* procedure to choose the "best" K, where "best" depends on the application.

A pragmatic approach is to define "best" as what works in practice. In deep learning, MoE layers scale Transformer models [392, 603, 843], but computational limits make selecting the optimal K critical. Ludziejewski et al. [1089] proposed a scaling law relating final loss, training data, model size, and K, validated through extensive experiments. This enables selecting K given other hyperparameters. Scaling laws for MoE-based LLMs are further explored in Abnar et al. [5], Tian et al. [1605].

Alternatively, under the assumption that the true model is MoE with $K^\star$ experts, one may attempt to recover $K^\star$ via statistical procedures. We present two such strategies next, discussing their advantages and limitations.

36.5.1 K as a learnable parameter

In the Bayesian framework we work with, it is natural to treat any aspect of the data-generating process we want to learn about as a parameter to be estimated for the data. In particular, quantities that are part of the likelihood model can be learned from data by including them in the Bayesian model to be trained, and in particular by assigning a prior to them and computing the resulting posterior distribution.

This simple principle can be readily applied to the number of experts K, as follows. Consider the usual likelihood model

$$Y_i \mid X_i, K, G \overset{\text{ind}}{\sim} f_G(\cdot \mid X_i),$$

recalling how, for notational convenience, we collect the MoE parameters into the measure $G = \sum_{j=1}^{K} \exp(\alpha_{0j})\delta_{(\alpha_{1j},\beta_j,\sigma_j^2)}$ with K support points, one for each expert. The key in the above expression is to explicitly recognize the functional dependence of the distribution of Y_i on K, by treating the latter as one more parameter in the model (as the ones collected by G). The next step is to assign priors, starting with G:

$$G \mid K \sim \Pi_K$$

where Π_K is a probability distribution on the set of positive measures with K support points on the space of expert parameters. Once again, while this formulation may sound abstract, it simply means that Π_K is a prior for the parameters of an MoE model with K experts. The crucial point is that we are now explicitly acknowledging the number of experts in the MoE model construction, while we left this information implicit when we were previously fixing K to an arbitrary but large number. Finally, because we want to learn K, we model it by assigning a prior ϕ (supported on $\mathbb{N}$) to it as well: $K \sim \phi$.

Having set up the model, we compute the posterior in the usual way as

$$\Pi(G, K \mid Y_{1:n}, X_{1:n}) \propto \phi(\{K\}) \times \Pi_K(G) \times \prod_{i=1}^{n} f_G(Y_i \mid X_i), \tag{36.13}$$

where the product of the first two terms on the right-hand side of the previous expression is simply the factorization of the joint prior for G and K implied by the hierarchical model. An important fact to notice is that the dimension of the support of the joint posterior distribution (36.13) varies with K: the larger K, the more parameters the model includes. This implies that, whatever learning algorithm one employs (whether MCMC or variational inference), this variable-dimensional nature of the parameter space should be taken care of appropriately. We do

not dwell on this aspect any further and refer the reader to the rich literature on the topic of a random number of mixture components (mostly developed with traditional mixtures in mind).

If we want to make inference on K, we can compute the marginal posterior distribution of K by marginalizing (36.13):

$$\Pi(K \mid Y_{1:n}, X_{1:n}) = \int \Pi(\mathrm{d}G, K \mid Y_{1:n}, X_{1:n}).$$

Notice that, in practice, the marginal posterior is usually available (approximately) as a byproduct of the training algorithm we employ to fit the model; for instance, if we use and an MCMC sampler where K is resampled at each iteration, we can simply take the empirical distribution of the marginal Markov Chain as an approximation to the marginal posterior of K. Be it as it may, once we have access to $\Pi(K \mid Y_{1:n}, X_{1:n})$, we can use it to make inference on K. For example, we can obtain a point estimate by computing the mode of the marginal posterior, and we can get uncertainty quantification by selecting as a credible set of values those that achieve the highest posterior probabilities which sum up to at least $1 - \kappa$ (where $\kappa \in (0, 1)$ is some small tolerance level).

An interesting question to ask is whether treating K as a parameter in the Bayesian model leads to good empirical performance, at least when enough data is collected. A way to study this aspect, as we did in the previous section, is to assume that the data is generated from a proper MoE model (36.1) with a fixed but unknown number of experts $K^\star$, and to study whether the posterior distribution concentrates on the correct value: that is, whether

$$\Pi(K^\star \mid X_{1:n}, Y_{1:n}) \overset{n\to\infty}{\longrightarrow} 1$$

in some appropriate probabilistic sense.

To facilitate the discussion, let's now fix some notation. Recall that Θ denotes the space of parameters relative to a single expert (i.e., the product of the space of gating parameters (α_0, α_1) and the space of expert parameters β, σ^2), and assume that the parameters for each different expert all belong to Θ. Therefore, when we look at an MoE model with k experts, the overall parameter space coincides with $\Theta^k := \Theta \times \cdots \times \Theta$ (k times). In our case, however, we are looking to place a prior that places positive probability on MoE models with any $K \in \mathbb{N}$, so the parameter space of interest is $\Theta_\infty := \bigcup_{k\in\mathbb{N}} \Theta^k$. Accordingly, the prior Π will be a probability measure on Θ_∞ specified hierarchically as follows: $\theta \sim \Pi$ if and only if $\theta \mid K \sim \Pi_K$, $K \sim \phi$, where Π_K is a probability measure on Θ^K. That is, to draw a random parameter θ from the prior Π, one first draws a random number of experts $K \sim \phi$ and then, conditional on this draw, one samples a K-expert MoE parameter from Π_K.

With this setting in mind, we are ready to state the next result about the asymptotic consistency of the posterior distribution on the number of experts.

Theorem 36.3. *Assume that Θ is compact, with the space of variances bounded away from 0. Moreover, assume the following:*

1. *The MoE model is identifiable;*
2. *Let Π_Θ be a continuous probability measure on Θ such that $\Pi_K = \Pi_\Theta^K$ (i.e., conditional on the number of experts K, the prior is of product form across experts);*

Then there exists a set $\Theta_\star \subseteq \Theta_\infty$ such that $\Pi(\Theta_\star) = 1$ and, for all $\theta_\star \in \Theta_\star$,

$$\lim_{n\to\infty} \Pi(K(\theta_\star) \mid X_{1:n}, Y_{1:n}) = 1$$

almost surely under $g^\infty_{G(\theta_\star)}$, where, for any $\theta \in \Theta_\infty$, $K(\theta)$ and $G(\theta)$ denote, respectively, the number of experts and the mixing measure associated to θ.

The proof of the last result, which we omit here but can be found in Bariletto et al. [97], is based on a fundamental result in Bayesian posterior asymptotics, namely Doob's consistency theorem. Loosely speaking, Doob's theorem says that, as long as a parameter of interest can be exactly recovered with infinite data, the posterior distribution is consistent for a set of such parameters that has prior mass 1. The result says, then, that this is the case for the number of experts in the MoE model (under the stated assumptions). Therefore, as long as the true generating MoE model lies in such a set of prior probability 1,[5] posterior inference on the number of experts will lead to the correct answer as the sample size grows to infinity.

[5]The fact that such a set has prior probability 1 means that, when looked from the perspective of the prior distribution, this set is large. However, this interpretation should be taken with some caution, as sets of prior mass 1 may anyway exclude large portions of the parameter space, especially in infinite-dimensional settings.

36.5.2 Selection via the ELBO

Another strategy to select the number of experts K relies on the output of the training process when variational inference is used. In particular, at the end of the training loop, on top of an optimized parameter vector one obtains (an estimate of) the evidence lower-bound of the trained model. As the name itself reminds us, the evidence lower-bound can be seen as a proxy of the evidence (or marginal likelihood) associated to the model, that is

$$M_n(\Pi) := \int \prod_{i=1}^{n} f_G(Y_i \mid X_i)\, \Pi(\mathrm{d}G).$$

Moreover, recall that $M_n(\Pi)$ is often used in Bayesian theory as a key quantity to compare prior models Π and select the best one based on how high the corresponding $M_n(\Pi)$. This makes intuitive sense because $M_n(\Pi)$ measures the likelihood computed at the training sample, integrated against the prior—so the higher is this quantity, the better is the fit of Π.

Piecing together the previous observations, a simple algorithm arises as a natural criterion for selecting the number of experts K:

1. Choose a set $\mathcal{K} \subseteq \mathbb{N}$ of candidate values of K, e.g., those that are more plausible given the application domain of the learning task;
2. For each $K \in \mathcal{K}$, fit an MoE model with K experts and record the value of the evidence lower-bound at the end of training, $\texttt{ELBO}(K)$;
3. Select the best K as the one maximizing the evidence lower-bound:

$$K_{\text{best}} = \arg\max_{K \in \mathcal{K}} \texttt{ELBO}(K).$$

Just like when we proposed placing a prior on K, we can ask ourselves if the above procedure produces a consistent estimate of $K^\star$, under the assumption that the data-generating process is an MoE. That is, we can ask if, as the sample size n grows to infinity, the estimator K_{best} converges to $K^\star$ in some appropriate probabilistic sense. A theoretical analysis of this procedure has been carried out in the context of simpler models, for instance mixtures [1703, 1862], but is still lacking for models with more complicated likelihoods such as MoE ones. Nevertheless, some preliminary empirical evidence is presented in Bariletto et al. [97].

36.6 Extensions and future directions

There are two potential future directions that can be extended from the above studies of Bayesian MoE models.

1. Bayesian sparse MoE. In large-scale deep learning applications such as large language models, MoE is often utilized to scale up the model capacity (the number of model parameters) while keeping the computational overhead nearly unchanged. In particular, Shazeer et al. [1508] proposed a sparse variant of MoE where only a few experts were activated per input. For that purpose, a sparse gating function or router was employed to evaluate the relevance of experts to an input, and then selected a subset of experts which had the highest similarity scores. Despite its huge success in practice, the sparse gating function induces several technical challenges in studying the asymptotic properties and the model selection problem of sparse MoE due to its discontinuous nature. From the perspective of frequentists, Nguyen et al. [1240, 1243] addressed these challenges by partitioning the input space into multiple subregions, each of which corresponded to a subset of experts activated per input in that region. Next, they characterized the asymptotic properties of sparse MoE in each region and then unified these results to capture the properties of sparse MoE in the whole region. On the other hand, the technical challenges caused by the discontinuity of the sparse gating function have remained unsolved from the Bayesian perspective.

2. Bayesian nonparametric MoE. At an abstract level, Bayesian mixtures of experts (MoEs) can be viewed as a special case of dependent nonparametric mixture models [1103], where the mixing measures depend on covariates. The key distinction lies in their goals: dependent nonparametric mixtures (e.g., the dependent Dirichlet process [1356]) are primarily designed for density estimation, whereas Bayesian MoEs are tailored to estimating expert models (e.g., regression functions). The two approaches also differ in how they construct mixture weights. Dependent nonparametric mixtures typically define weights through dependent stick-breaking priors, while Bayesian MoEs place a prior directly on the gating function (e.g., a softmax) that governs the mixture weights. In this sense, Bayesian MoEs are conceptually similar to the construction in Dunson et al. [464], where a kernel is used to define a data-dependent mixing measure. The main difference, however, is that in Dunson et al. [464] the kernel measures

similarity between data points, whereas in Bayesian MoEs the gating function computes similarity between a data point and model parameters.

A practical consideration for Bayesian MoEs is the choice of prior $\Pi(G, K)$. As discussed in Section 36.5.1, we factorize it as $\Pi(K)\Pi_K(G)$. Alternatively, one may specify a joint prior directly over (G, K). For example, a repulsive prior on G can be employed [1319] to discourage redundant experts. More generally, $\Pi(G, K)$ can be modelled via a determinantal point process [944, 1102], where experts are treated as stochastic points. The corresponding inference procedures under such specifications also merit further investigation, with reversible jump methods [662] being a conventional approach. For the gating function and expert functions, we can go beyond parametric construction by using Gaussian process [1381, 1631].

Part VIII

Scalability

Chapter 37

Scalable BDL by single forward pass inference

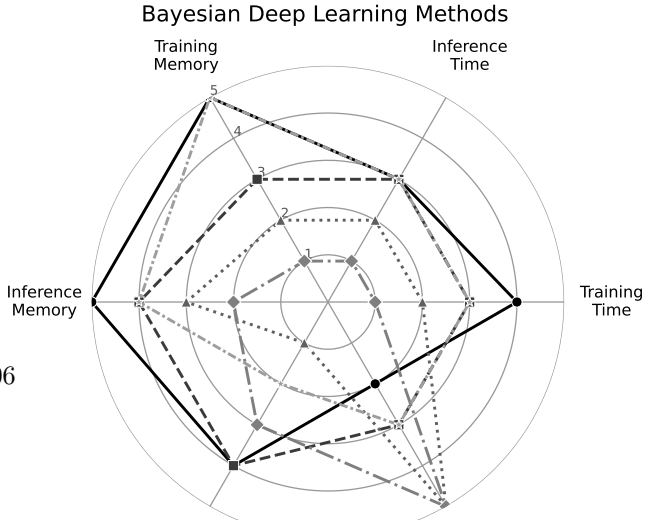
In the previous sections, we surveyed a broad range of technical methods that bridge the Bayesian framework with deep learning. This section takes a different angle, perhaps from a more practical standpoint, to investigate the following question: *How can we devise scalable methods for Bayesian Deep Learning?* We begin by motivating why thinking about scalability is essential for tackling problems of relevance with BDL. We then turn our attention to a specific class of scalable BDL methods that enable inference with a single forward pass through a deep network. These methods can be naturally understood as the composition of a *backbone* network, intended for representation learning, and a *predictive head*, which performs kernel approximation or density estimation for scalable Bayesian inference. Throughout this chapter, we discuss the challenges associated with each component and examine the distinct methods in the literature to address them.

37.1 Why do we need scalability in Bayesian Deep Learning?

So far, this book has presented a plethora of BDL methods, encompassing diverse approaches to enable Bayesian inference with deep learning models. These methods adhere to the formal language of Bayesianism, which offers a principled and self-consistent framework for probabilistic machine learning. They also provide important capabilities that non-Bayesian approaches typically lack, such as uncertainty estimation. However, Bayesian inference (whether exact or approximate) requires integrating over high-dimensional parameter spaces, which is computationally demanding. As a result, many BDL methods struggle to scale to the large models and datasets common in modern applications. Striking the right balance between computational efficiency, stability, and approximation quality remains a central challenge and an active area of research.

Over the past decade, deep learning research has been shaped by two defining characteristics. First, the prevalence of *large-scale datasets* from the Big Data era, composed of millions (or perhaps billions or even trillions) of data points. For instance, ImageNet [1441]—once a standard computer vision benchmark—contains more than 14 million data points; LAION-5B [1475], a recent multimodal dataset, contains 5.85 billion image-text pairs. Second, modern models are *highly overparameterised*, often containing millions to billions of parameters. Even relatively modest architectures, such as ResNet-18 [721], have 11.8 million parameters, while state-of-the-art foundation models, including LLMs/VLMs, can reach hundreds of billions or even trillions. Consequently, methods that fail to scale in memory or compute time may not be effective in practice.

We now turn to the memory and time complexities of the key methods discussed in this book. For clarity, our comparison focuses on their "vanilla" versions, acknowledging that subsequent refinements may improve scalability. Table 37.1 summarises time and memory complexities during training and inference stages, while Figure 37.1 extends the comparison along two additional dimensions: *ease of use*, which refers to how straightforward a method is for implementation, training, debugging, or tuning; and *approximation quality*, which refers to how well it captures posterior distri-



Method	Training Time	Inference Time	Training Memory	Inference Memory
Standard NN	$\mathcal{O}(N \cdot P)$	$\mathcal{O}(P)$	$\mathcal{O}(P)$	$\mathcal{O}(P)$
MC Dropout	$\mathcal{O}(N \cdot P)$	$\mathcal{O}(M \cdot P)$	$\mathcal{O}(P)$	$\mathcal{O}(P)$
Deep Ensembles	$\mathcal{O}(M \cdot N \cdot P)$	$\mathcal{O}(M \cdot P)$	$\mathcal{O}(M \cdot P)$	$\mathcal{O}(M \cdot P)$
Variational Inference	$\mathcal{O}(K \cdot N \cdot P)$	$\mathcal{O}(K \cdot P)$	$\mathcal{O}(P)$	$\mathcal{O}(P)$
MCMC	$\mathcal{O}(T \cdot N \cdot P)$	$\mathcal{O}(S \cdot P)$	$\mathcal{O}(S \cdot P)$	$\mathcal{O}(S \cdot P)$
Gaussian Processes	$\mathcal{O}(N^3)$	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$	$\mathcal{O}(N)$

Table 37.1: Complexity comparison of BDL methods, including a standard neural network baseline. Here, N and P denote the number of training points and model parameters. M, K, T and S correspond to the number of ensemble/dropout samples, VI samples, MCMC steps, and MCMC posterior samples, respectively. Typically, $T \gg S > K \gtrsim M$.

butions. Although this plot is inevitably subjective, it serves as a useful didactic illustration.

Methods originally designed with deep learning practice in mind (such as Monte Carlo (MC) Dropout and Deep Ensembles) present time and memory complexities that more closely resemble those of standard neural networks, suggesting better scalability. They are also easily integrable to deep learning pipelines, demonstrating good practicality. Yet, they present drawbacks. MC Dropout, despite being theoretically justified as a variational approximation (Chapter 14), assumes a restrictive variational family (a degenerate spike-and-slab distribution), which hinders its approximation quality and sometimes its predictive performance. Deep Ensembles (Chapter 19), on the other hand, often achieve strong empirical performance and reasonable uncertainty estimates, but incur significantly higher computational costs than a single deterministic network, as they require training and maintaining M independent model copies. Furthermore, their theoretical foundations in the Bayesian framework are still not completely understood (and a topic of disagreement among Bayesians), which leaves open the question of why they work and when their uncertainty estimates should be trusted.

Variational inference (VI) methods typically offer a compromise between computational efficiency and approximation quality. Compared to MC Dropout, they allow for richer variational families; yet, in practice, the distributions used remain highly restrictive to preserve tractability in the learning objective (e.g., mean-field VI). Unlike Deep Ensembles, they do not require training multiple copies of a model and can be integrated relatively easily into standard deep learning pipelines, which has made modern VI approaches both promising and widely adopted in BDL research. Nevertheless, training stability remains a significant challenge, often demanding careful hyperparameter tuning, especially for deeper or more complex architectures. Perhaps as a consequence, it is also unclear how to reconcile these methods with the emerging paradigm of Foundation Models.

Finally, we consider Bayesian methods that offer the most principled inference guarantees. Gaussian Processes (GPs) provide exact inference for a chosen kernel under certain likelihood models, and their closed-form analytical solutions are appealing to practitioners in many settings. However, their computational cost grows cubically with the dataset size, making direct application to modern deep learning problems infeasible. Sampling-based methods such as Markov chain Monte Carlo (MCMC) are asymptotically exact (that is, they theoretically converge to the true posterior in the limit) and thus provide the most faithful approximations available in high-dimensional problems. Yet, they are notoriously slow and memory intensive, requiring a large number of MCMC samples T for convergence and posterior samples S for inference, and are often tricky to implement and tune effectively.

In summary, a key takeaway is that *there is no silver bullet for Bayesian Inference in Deep Learning.* While the baseline methods presented here have their strengths, none of them really demonstrate the scalability and simplicity of standard deep networks. They exhibit a clear tension between algorithmic complexity (both in

terms of memory and time) and approximation quality or, more broadly, practicality. This is why developing scalable methods is crucial: to reconcile these trade-offs and make Bayesian approaches viable within modern deep learning pipelines and real-world applications. In this chapter, as well as the subsequent ones, we will explore various strategies to advance scalability in Bayesian Deep Learning.

37.2 Bayesian Inference by a single forward pass

Let us once again turn our attention to Table 37.1, particularly on parametric methods (we will explore GPs later). The first thing to notice is that the costs of all of them, including standard NNs, grows linearly with the number of model parameters. This cost looks non-negotiable, as we require to at least compute the model predictions by running over them. It is also not negligible, as the current model are already very large and the trend in the field is to gradually increase their size further. In this context, what can we do to improve scalability?

The natural answer is to focus on the other variables that affect their complexity. A common pattern among them is the presence of a multiplicative factor, which is related to the process of marginalisation. For instance, let us look at the process of predictive inference, which involves approximating the following expectation:

$$p(y \mid x, \mathcal{D}) = \int p(y \mid x, \theta)\, p(\theta \mid \mathcal{D})\, d\theta = \mathbb{E}_{\theta \sim p(\theta \mid \mathcal{D})}\big[p(y \mid x, \theta)\big]. \tag{37.1}$$

In most cases of BDL, the expectation above is approximated via Monte-Carlo sampling from a posterior approximation $q(\theta) \approx p(\theta \mid \mathcal{D})$. Each posterior sample θ is a *model hypothesis*: a different "version" that the neural network may assume. For instance, these are the M dropout masks in MC dropout, or the M models in Deep Ensembles, or the S posterior samples in MCMC, and so on. This means that most methods will require running multiples "versions" of an NN for a single inference, sometimes requiring storing a copy of each of them in memory. This explains why these methods are much more costly than standard NNs. Even a low number of samples (for instance, we often use $M = 5 \sim 10$) incurs high costs as the deep models are naturally very large.

We highlight two different strategies to circumvent these challenges. The first one is to assume a particular structure in the posterior distribution that allows computing it and approximating Equation 37.1 with lower cost, perhaps in a closed-form solution. One way of doing this is via a *Laplace approximation* (Chapter 7), which assumes a Gaussian posterior centred around the MAP estimate. With further approximations (diagonal approximation of Hessians, model linearisation), it is possible to achieve time and memory complexities equivalent to a standard NN. Naturally, these approximations affect the quality of inference. We explore scalable Laplace approximations in Chapter 40.

Another alternative, more relevant for this chapter, is to *accept being Bayesian only in a subspace of the model parameters*. Let $\theta = [\theta_b, \theta_h]$, where b and h refers to a model's *backbone* and *head*, respectively, where $P = \dim(\theta) = P_b + P_h$. For a Bayesian method with complexity $\mathcal{O}(M \cdot P)$, if we opt to be Bayesian only in the head parameters, we can leverage the output of the backbone as representations (via a single forward pass) and compute the marginalisation of Equation 37.1 in the Bayesian head. In this case, the complexity will be $\mathcal{O}(P_b + M \cdot P_h)$. For the case where $P_b \gg P_h$ and for small number of posterior samples M, $\mathcal{O}(P_b + M \cdot P_h) \approx \mathcal{O}(P_b) \approx \mathcal{O}(P)$. As an extreme case, we may consider a non-parametric head, such as a GP (over a small set of points), attaining complexity of $\mathcal{O}(P + N^3)$, again with $P \gg N$ so that is approximates $\mathcal{O}(P)$. This idea is explored both in this chapter as well as in Last-Layer Inference methods (Chapter 39).

37.2.1 Formalism

We now formalise the idea of BDL with a single forward pass inference. Let $\mathcal{X} \subseteq \mathbb{R}^d$ denote the input space and $\mathcal{Y} \subseteq \mathbb{R}^q$ the output space. We are given training data with N data points $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \mathbf{x}_i \in \mathcal{X},\ y_i \in \mathcal{Y}$, assumed to be generated from an unknown stochastic process $y \sim p(y \mid f^\star(\mathbf{x})), f^\star : \mathcal{X} \to \mathcal{Y}$, where $f^\star$ is the true (unknown) function underlying the observations. The learning objective is to approximate the predictive distribution $p(y_* \mid \mathbf{x}_*, \mathcal{D})$. A BDL method with single forward pass inference can be described by two components:

- A *backbone* network, defined as a parametric map $g_{\theta_b} : \mathcal{X} \to \mathbb{R}^p$, where θ_b represents the trainable parameters of this backbone (e.g., a deep neural network). It maps the input space (usually high-dimensional) to a low-dimensional representation space. The goal is to leverage the inductive biases and expressiveness of the deep network to learn high-level representations for probabilistic modelling.
- A *predictive head*, defined as a functional mapping $\mathcal{U}_{\theta_h} : \mathbb{R}^p \to \mathcal{P}(\mathcal{Y})$, where $\mathcal{P}(\mathcal{Y})$ denotes the set of probability distributions over $\mathcal{Y}$. As we mentioned previously, this mapping may be a parameterised function

(by θ_h), or assume a non-parametric form (as in the case of a GP head). This head governs predictive uncertainty, via a kernel function or a density estimator.

Two seminal works in the BDL world to apply these ideas are Deep Kernel Learning (DKL) [1769], discussed in more detail in Chapter 24, and Deterministic Uncertainty Quantification (DUQ) [1653]. In DKL, the backbone network extract features over which an inducing point GP is defined. We can think about it as defining a new kernel as:

$$k_{\theta_b}(\boldsymbol{x}_i, \boldsymbol{x}_j) \to \bar{k}(g_{\theta_b}(\boldsymbol{x}_i), g_{\theta_b}(\boldsymbol{x}_j)), \tag{37.2}$$

where $\bar{k}$ is a base kernel, such as the RBF kernel. DUQ is a "non-GP" counterpart to DKL: instead of modelling a GP on top of the deep kernel, it performs predictions and estimate uncertainties directly in the learned feature space via the kernelized distance (i.e., a RBF network).

37.3 The challenge of a good backbone

As described previously, the goal of the backbone is to leverage the expressive power of deep networks to learn good features that improve predictions. Nonetheless, learning this feature space is not a simple task, and naively following standard modelling assumptions sets up an underconstrained optimisation that potentially leads to pathological, trivial solutions, where the learned representations are degenerated. For instance, the backbone may learn to map dissimilar points to the same representation, a phenomenon called "feature collapse" [1653, 1654]. This is often the case for out-of-distribution data, where these learning objectives do not impose restrictions to points that are away of the training distribution.

Representation degeneracy poses a particular challenge for BDL methods because these methods rely on the kernel to encode prior assumptions about the function space. In practice, most kernels used in this setting are *geometry-aware*: they measure similarity between inputs through pairwise distances in the learned feature space, $d(\mathbf{x}, \mathbf{x}') = \|g_{\theta_b}(\mathbf{x}) - g_{\theta_b}(\mathbf{x}')\|$. If the feature space fails to represent meaningful geometry, the kernel matrix also degenerates. As a result, the probabilistic model can no longer provide reliable predictions and uncertainty estimates.

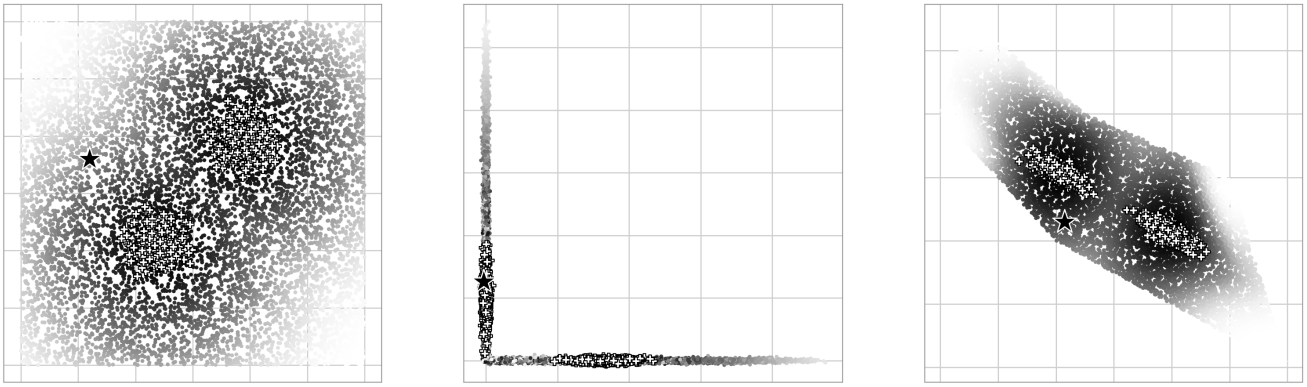
A concrete example where this pathology arises is in Deep Kernel Learning. DKL maximises the log marginal likelihood of a GP with prior $\mathcal{GP}(m, K)$, where K is the kernel matrix and takes the form $K_{i,j} = k_{\theta_b}(x_i, x_j)$, with k_{θ_b} as described in Equation 37.2. Mathematically, this objective is given by:

$$\begin{aligned} \log p(\mathbf{y}) &= \log \mathcal{N}\big(\mathbf{y} \mid \mathbf{0},\, K + \sigma_n^2 I_N\big) \\ &= \underbrace{-\tfrac{1}{2}\mathbf{y}^\top \big(K + \sigma_n^2 I_N\big)^{-1}\mathbf{y}}_{\text{data fit term}} \underbrace{-\tfrac{1}{2}\log\big|K + \sigma_n^2 I_N\big|}_{\text{complexity term}} \underbrace{-\tfrac{N}{2}\log(2\pi)}_{\text{constant}}, \end{aligned} \tag{37.3}$$

where σ_n is a learnable hyperparameter for the observation noise. In addition to the constant term, we identify two main components of this objective: the *data fit* term, a function of the observables $\boldsymbol{y}$ that optimises the kernel parameters to better explain the data; and a *complexity* term, which encourages simpler models.

At convergence, one can show that the data-fit term approaches $-N/2$ (this follows from the stationarity condition of the log marginal likelihood with respect to a global scale on $K + \sigma_n^2 I_N$: at the optimal scale, $\mathbf{y}^\top (K + \sigma_n^2 I_N)^{-1}\mathbf{y} = N$ exactly, so the data-fit term evaluates to $-N/2$). More interestingly, the complexity term encourages minimising the determinant $|K + \sigma_n^2 I_N|$. This determinant can be driven arbitrarily small by learning a kernel that maximally correlates the input representations without compromising the data fit—something feasible when the kernel has sufficient capacity, as in the case of deep backbone networks. A trivial solution arises when the kernel induces co-linear representations, driving the determinant toward zero and effectively collapsing the features. In this sense, the objective *encourages* feature collapse when using highly parameterised kernels.

We empirically illustrate this phenomenon in Figure 37.2, using a 2D classification task where the two classes are generated from Gaussian distributions. Figure 37.2a shows a grid of input points, coloured according to their log-probability under the data-generating distribution. In contrast, Figure 37.2b displays the learned representations obtained from an underconstrained model. Here, the points collapse into nearly co-linear representations, regardless of their original positions or likelihoods. This pathology leads to undesirable consequences, such as overfitting [1253] and poor uncertainty estimation [1654], even within a Bayesian framework.



(a) Input (b) Trained without constraint (c) Trained with constraint

Figure 37.2: A 2D classification task where the classes are two Gaussians (marked with white "+" symbols), and a grid of unrelated points, coloured according to their log-probability under the data generating distribution. A specific point is marked with a star. In (b), the features as computed by an unconstrained model. In (c), the features computed by a model with residual connections and spectral normalisation. The unconstrained model distorts the space, collapsing inputs onto a line regardless of their dissimilarities. In contrast, the constrained mapping preserves relative distances among points. Adapted from van Amersfoort et al. [1654].

37.4 The Antidote for the degeneracy: bi-Lipschitzness

How can we prevent representation degeneracy? One natural strategy is to return to the full Bayesian setting [1253] and place priors on all kernel parameters to prevent pathological solutions. Obviously, this approach defeats our purpose of making BDL more scalable, as discussed in Section 37.1. Instead, we can take an optimisation perspective and view feature collapse as the result of an *underconstrained problem*, and address it by imposing regularisation assumptions on the learned feature space to prevent such pathological solutions.

As we previously discussed, representation degeneracy is a major concern since kernel-based predictive heads usually rely on the geometry of the feature space, and, in particular, the induced distance between points $d(\mathbf{x}, \mathbf{x}') = \|g_{\theta_b}(\mathbf{x}) - g_{\theta_b}(\mathbf{x}')\|$. Therefore, preventing degeneracy encompasses learning a mapping that is *distance-preserving*, so that the distance in the feature space $d(\mathbf{x}, \mathbf{x}')$ has a meaningful correspondence to the distance $\|\mathbf{x} - \mathbf{x}'\|_{\mathcal{X}}$ in the data manifold [1061]. Here, $\|\cdot\|_{\mathcal{X}}$ denotes a semantically meaningful metric on the input data manifold, in contrast to the Euclidean norm in the raw observation space, which is a poor proxy for semantic similarity in high-dimensional inputs such as images.

We can decompose the distance-preserving property into two conditions. First, we require that different points $\mathbf{x}, \mathbf{x}'$ also present different representations: when the input changes, the features also change. This condition is called *sensitivity*, and one can express it mathematically as follows:

$$L_1 \leq \frac{\|g_{\theta_b}(\mathbf{x}) - g_{\theta_b}(\mathbf{x}')\|}{\|\mathbf{x} - \mathbf{x}'\|_{\mathcal{X}}}, \quad \forall \boldsymbol{x}, \boldsymbol{x'} \in \mathcal{X}, \boldsymbol{x} \neq \boldsymbol{x'} \tag{37.4}$$

where $0 < L_1 < 1$. Equation 37.4 is often referred as the lower Lipschitz condition, and L_1 is the lower Lipschitz constant. Intuitively, if we fix two distinct points $\boldsymbol{x}, \boldsymbol{x'}$, then this constraint prevents the numerator (the distance in feature space) to become too small.

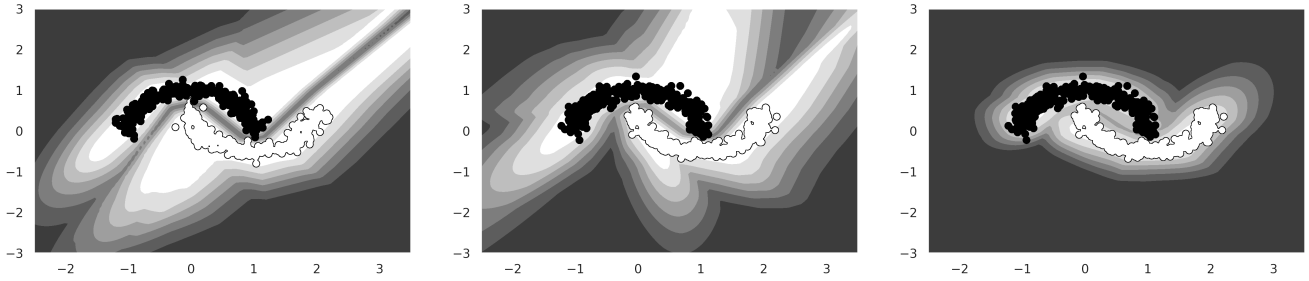
The second condition for a distance-preserving mapping is *smoothness*: small changes in the input should not lead to arbitrarily large changes in the representations. Smoothness complements sensitivity by imposing an upper Lipschitz condition:

$$L_2 \geq \frac{\|g_{\theta_b}(\mathbf{x}) - g_{\theta_b}(\mathbf{x}')\|}{\|\mathbf{x} - \mathbf{x}'\|_{\mathcal{X}}}, \quad \forall \boldsymbol{x}, \boldsymbol{x'} \in \mathcal{X}, \boldsymbol{x} \neq \boldsymbol{x'}, \tag{37.5}$$

where $L_2 > 1$ is the upper Lipschitz constant. Note that, by definition, $L_1 < L_2$. The composition of both conditions is called the bi-Lipschitz condition, and often expressed as:

$$L_1\|\mathbf{x} - \mathbf{x}'\|_{\mathcal{X}} \leq \|g_{\theta_b}(\mathbf{x}) - g_{\theta_b}(\mathbf{x}')\| \leq L_2\|\mathbf{x} - \mathbf{x}'\|_{\mathcal{X}}, \quad \forall \boldsymbol{x}, \boldsymbol{x'} \in \mathcal{X}, \boldsymbol{x} \neq \boldsymbol{x'}. \tag{37.6}$$

Next, we highlight different strategies to enforce bi-Lipschitzness during deep networks training.



(a) No gradient penalty (b) One-sided gradient penalty (c) Two-sided gradient penalty

Figure 37.3: Uncertainty quantification on the two moons dataset using an RBF network. Lighter shading indicates high certainty, while darker shading indicates uncertainty. Ideally, the model should be confident only near the training data. In (a), the model lacks any regularisation and remains overconfident far from the data. In (b) and (c), adding one-sided and two-sided penalties improves uncertainty estimates, with the bi-Lipschitz penalty in (c) giving the most reliable results. Adapted from Van Amersfoort et al. [1653].

37.4.1 Bi-Lipschitzness via Two-Sided Gradient Penalty

From the fundamental theorem of calculus, one can show that, for $f : \Omega \subset \mathbb{R}^d \to \mathbb{R}^k$ continuously differentiable on a convex set Ω:

$$\|f(a) - f(b)\| \leq \sup_{z \in [a,b]} \|J_f(z)\| \cdot \|a - b\|, \tag{37.7}$$

where J_f is the Jacobian. The Jacobian norm $\|J_f(z)\|$ provides a local bound on how much f can expand distances near z. Indeed, if $\|J_f(z)\| \leq L_2$ for all $z \in \Omega$, then we recover the upper Lipschitz condition. On the other hand, if the smallest singular value of the Jacobian is uniformly bounded below as $\sigma_{\min}(J_f(z)) \geq L_1 > 0$, we also recover the lower Lipschitz condition, since for any vector v, $\|J_f(z)v\| \geq \sigma_{\min}(J_f(z))\,\|v\|$. Together, these inequalities establish that the global Lipschitz constants of f can be controlled directly by regularising the Jacobian norm. A simple way to achieve this is to add a penalty term that explicitly constrains both the largest and the smallest singular values of the Jacobian:

$$\lambda \left(\max(0, \|J_f(z)\| - L_2)\right)^2 + \mu \left(\max(0, L_1 - \sigma_{\min}(J_f(z)))\right)^2, \tag{37.8}$$

where λ and μ are hyperparameters. This objective enforces that the singular values remain in the interval $[L_1, L_2]$, thereby directly encouraging the bi-Lipschitz property. In practice, however, previous works [686, 1653] adopt a simpler penalty of the form

$$\lambda \left(\|J_f(z)\|^2 - 1\right)^2. \tag{37.9}$$

While this penalty formally only controls the largest singular value, in practice it often has the effect of improving the overall conditioning of the Jacobian, discouraging collapse in the smaller singular values as well.

Figure 37.3 presents an example of the effectiveness of enforcing bi-Lipschitzness by gradient penalty. In this example from Van Amersfoort et al. [1653], a RBF network is trained on the two moons dataset, a 2D classification task, and we evaluate how well the model performs uncertainty quantification for points that are out-of-distribution. Ideally, the model should be confident only near the training points. In Figure 37.3a, without any penalty enforcing smoothness or sensitivity, the network remains highly confident even in regions far from the data. In Figure 37.3b, the penalty only encourages smoothness (upper Lipschitzness), which improves uncertainty estimates in comparison with (a) but still present areas of low uncertainty far from the training distribution. In Figure 37.3c, applying a two-sided penalty that enforces bi-Lipschitzness yields significantly better uncertainty estimates.

Although the two-sided gradient penalty appears effective in the example above, it also has important limitations. Computing Jacobians is expensive, and prior work has often relied on simplifications, such as using gradients of summary statistics (e.g., the gradient of the sum of kernel distances instead of the full Jacobian of the kernel distance vector). This difficulty restricts the method to using soft constraints, as in Equation 37.9, which not necessarily enforces bi-Lipschitzness. Furthermore, such soft constraints can introduce unintended side effects. For instance, causing residual connections to degenerate into identity mappings [1061].

37.4.2 Bi-Lipschitzness via Spectral Normalisation and Residual Connections

A faster alternative to Jacobian penalties for enforcing bi-Lipschitzness involves employing residual connections in the deep network and applying spectral normalisation to them. A residual connection is a neural network design where each layer learns only a residual mapping rather than a full transformation. Formally, for an input $\mathbf{x}$, a residual block is defined as $g_i(\mathbf{x}) = \mathbf{x} + h_i(\mathbf{x})$, where h_i is a learnable function (e.g., a neural network block). The term $\mathbf{x}$ is passed forward unchanged through a skip connection, while $h_i(\mathbf{x})$ models the residual. This simple addition helps stabilise optimisation, preserves distances under mild conditions, and enables training of very deep networks. Modern deep learning architectures are often composed of residual blocks, taking the form $g_{\theta_b}(\boldsymbol{x}) = g_{l-1} \circ g_{l-2} \circ \cdots \circ g_1(\boldsymbol{x})$, with $l-1$ referring to the number of residual blocks.

One can show that if all residual functions are α-Lipschitz, for $0 < \alpha \leq 1$, i.e.:

$$\|h_i(\mathbf{x}) - h_i(\mathbf{x}')\| \leq \alpha \|\mathbf{x} - \mathbf{x}'\|_{\mathcal{X}}, \quad \forall \boldsymbol{x}, \boldsymbol{x'} \in \mathcal{X}, \boldsymbol{x} \neq \boldsymbol{x'}, \tag{37.10}$$

then g_{θ_b} is also bi-Lipschitz with $L_1 = (1-\alpha)^{l-1}$ and $L_2 = (1+\alpha)^{l-1}$. Therefore, a sufficient condition is to ensure α-Lipschitzness in the residuals, which typically assume the form of $h_i = \sigma(\boldsymbol{W}_i \boldsymbol{x} + \boldsymbol{b}_i)$, where σ is a non-linear activation function (assumed 1-Lipschitz, e.g. ReLU or tanh), and $\boldsymbol{W}_i$ and $\boldsymbol{b}_i$ is the weight matrix and bias vector of the residual block i. We can achieve this by enforcing $\|\boldsymbol{W}_i\|_2 \leq 1$, since that:

$$\begin{aligned} \alpha = \sup_{\mathbf{x} \neq \mathbf{x}'} \frac{\|h_i(\mathbf{x}) - h_i(\mathbf{x}')\|}{\|\mathbf{x} - \mathbf{x}'\|_{\mathcal{X}}} &= \sup_{\mathbf{x} \neq \mathbf{x}'} \frac{\|\sigma(\mathbf{W}_i\mathbf{x} + \mathbf{b}_i) - \sigma(\mathbf{W}_i\mathbf{x}' + \mathbf{b}_i)\|}{\|\mathbf{x} - \mathbf{x}'\|_{\mathcal{X}}} \\ &\leq \sup_{\mathbf{x} \neq \mathbf{x}'} \frac{\|\mathbf{W}_i(\mathbf{x} - \mathbf{x}')\|}{\|\mathbf{x} - \mathbf{x}'\|_{\mathcal{X}}} = \|\mathbf{W}_i\|_2 \ \leq \ 1. \end{aligned} \tag{37.11}$$

The task, then, is to ensure that the spectral norm of each weight matrix $\mathbf{W}_i$ is properly controlled. We estimate this spectral norm using the power iteration method and apply the following normalisation:

$$\mathbf{W}_i = \begin{cases} c \cdot \mathbf{W}_i / \hat{\lambda}, & \text{if } \hat{\lambda} > c, \\ \mathbf{W}_i, & \text{otherwise,} \end{cases} \tag{37.12}$$

where $\hat{\lambda}$ is the estimated spectral norm and c is a hyperparameter that sets the desired upper bound. Since other architectural components can also influence the overall Lipschitz constant α, in practice we adjust c to ensure both smoothness and sensitivity.

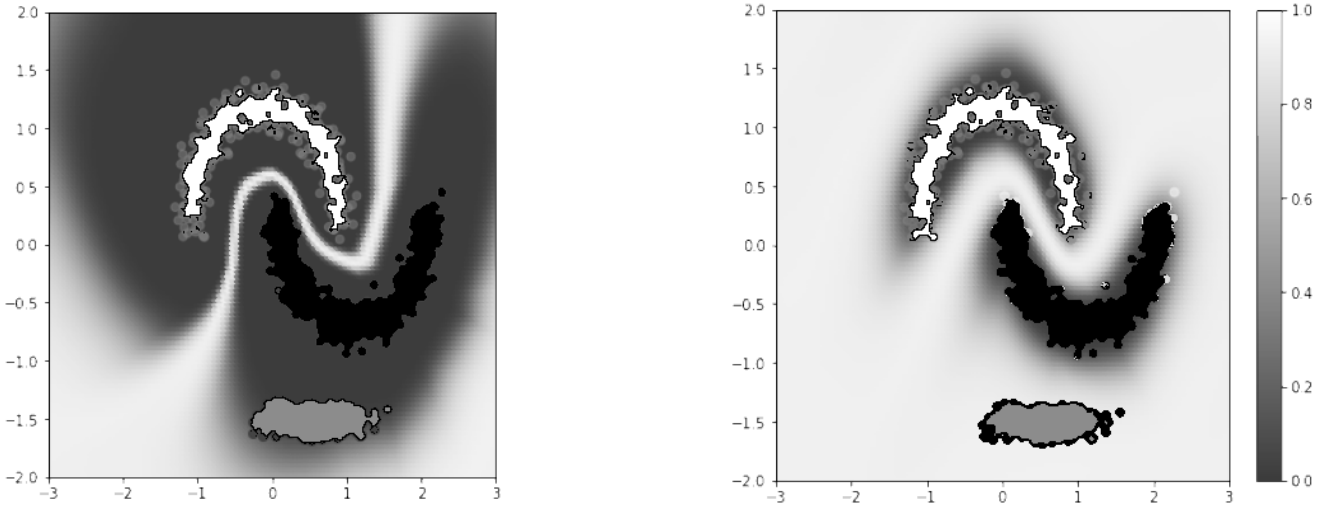
Figure 37.4 illustrates the effect of spectral normalisation in enforcing bi-Lipschitzness for a Gaussian Process with a deep backbone. In this case, lighter shading indicates regions of high uncertainty. Without spectral normalisation (Figure 37.4a), the model remains overconfident even far from the training data (black and white points) and near the out-of-distribution dataset (the far grey blob of points). By contrast, applying spectral normalisation leads the deep GP to produce more reliable uncertainty estimates, both around the training distribution and in OOD regions.

Finally, Table 37.2 compares the methods discussed in this section using experiments from Liu et al. [1061]. All approaches share the same ResNet backbone trained on CIFAR-10, and evaluation is performed both in-distribution (accuracy on the CIFAR-10 test set) and out-of-distribution (Area Under the Precision-Recall Curve, AUPR, on SVHN and CIFAR-100). DUQ [1653] denotes the RBF network with a double-sided gradient penalty (subsection 37.4.1), DNN-SN applies spectral normalisation without a predictive head, and Spectral-Normalised Neural Gaussian Process (SNGP) [1061] combines spectral normalisation with a GP head. Observe that DNN-SN outperforms DUQ in-distribution and matches its OOD performance, while achieving much lower latency. Adding a Bayesian predictive head (SNGP) further improves OOD performance, achieving the best overall results.

Overall, spectral normalisation offers a simple but effective way to enforce bi-Lipschitzness, but it also presents notable limitations. It requires architectures with residual blocks. Furthermore, the condition in Equation 37.11 assumes dense residual functions, and different inductive biases (e.g., attention, convolution) may lead to different conditions. Other regularisation techniques (e.g., batch normalisation, dropout) also affect the Lipschitz constant, requiring a careful tuning of the hyperparameter c. Lastly, spectral normalisation is still costly, requiring repeated matrix–vector products for each residual block, which results in a substantial latency gap between a deterministic and a spectral normalised network, as shown in Table 37.2.

37.5 The predictive head

As stated in Section 37.2, we define the predictive head as a parameterised functional mapping $\mathcal{U}_{\theta_h} : \mathbb{R}^p \to \mathcal{P}(\mathcal{Y})$, which takes as input the features learned by the deep backbone and maps them to a distribution over outputs. Its



(a) Without Spectral Normalisation

(b) With Spectral Normalisation

Figure 37.4: Uncertainty quantification on the two moons dataset using a Gaussian Process with a deep backbone network (ResNet). Black and white points denote positive and negative classes, while the far grey blob of points indicate OOD samples. Lighter shading indicates high uncertainty regions. Ideally, the model should be uncertain far from the training data. In (a), without enforcing bi-Lipschitzness, the model is overconfident in out-of-distribution regions. In (b), spectral normalisation enforces bi-Lipschitzness and yields more reliable uncertainty estimates. Adapted from Liu et al. [1061].

Method	Accuracy (ID)	OOD AUPR (↑)		Latency (↓)
	CIFAR-10	SVHN	CIFAR-100	(ms / example)
Deterministic	96.0 ± 0.01	0.781 ± 0.01	0.835 ± 0.01	**3.91**
DUQ[1]	94.7 ± 0.02	0.973 ± 0.01	0.854 ± 0.01	8.68
DNN-SN	96.0 ± 0.01	0.974 ± 0.01	0.859 ± 0.01	5.20
SNGP	95.9 ± 0.01	**0.990 ± 0.01**	**0.905 ± 0.01**	6.25

Table 37.2: Comparison of gradient-penalty (DUQ) and spectral-normalisation approaches (DNN-SN, SNGP) on CIFAR-10 with a ResNet backbone. Spectral normalisation (DNN-SN) achieves higher in-distribution accuracy and similar OOD performance to the gradient-penalty method (DUQ) while running with much lower latency. Adding a Bayesian predictive head (SNGP) further boosts OOD performance. Adapted from Liu et al. [1061].

primary purpose is to provide reliable predictions and uncertainty estimates. In this section, we present a range of approaches for modelling this mapping, starting from deterministic methods and progressing to scalable Bayesian approximations.

37.5.1 The standard neural network head

The simplest way to represent the mapping $\mathcal{U}_{\theta_h}$ is with a deterministic neural network. For example, one might use a few-layers dense network with nonlinear activations (e.g., ReLU) in the hidden layers, and either a linear output layer (for Gaussian likelihoods) or a softmax output (for categorical likelihoods). When combined with the backbone, this forms a single deep network capable of producing predictions over $\mathcal{Y}$. But can we *trust* these predictions?

A crucial requirement for reliable predictions is that the distribution over $\mathcal{Y}$ be meaningful—that is, it should reflect the model's uncertainty about the predicted variable. In other words, the model should *know what it does not know*, and express this through its predictive distribution. Standard neural networks, however, lack an inductive bias that enforces this property, and they can easily become overconfident on inputs far from the training data—precisely where we would expect low confidence. A simple illustration is a ReLU network, which one can show that it can produce outputs that grow without bound, leading to overconfident predictions. This pathology motivates the development of alternative heads that are uncertainty-aware by design.

37.5.2 The Radial Basis Function (RBF) head

A simple inductive bias that promotes uncertainty awareness is to explicitly model distances between input data and the training distribution in a representation space. Intuitively, if a point lies close to the training distribution, we expect its predictions to be consistent with those of nearby training examples, and the associated uncertainty should be low. Conversely, if a point is far from the training distribution in feature space, the model should express high uncertainty, as such inputs fall outside its domain of knowledge.

This concept is explicitly incorporated in *Radial Basis Function (RBF) networks*. These models construct predictions by measuring the similarity between an input and a set of reference points (often derived from the training data) in a feature space, where similarity is quantified in terms of Euclidean distances. Each reference point serves as the centre of a local receptive field, and the contribution of each centre to the final prediction decays with distance from the input. Formally, let $\{c_j \in \mathbb{R}^p : j = 1, \dots, m\}$ denote a collection of centres, and let $\phi : \mathbb{R}^p \times \mathbb{R}^p \to \mathbb{R}_{\geq 0}$ be a radial basis function, i.e. a function of the form

$$\phi(\boldsymbol{x}, c_j) = \varphi(\|g_{\theta_b}(\boldsymbol{x}) - c_j\|_2)\,, \tag{37.13}$$

where $\varphi : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$ is nonincreasing. A common choice is the Gaussian kernel,

$$\phi(\boldsymbol{x}, c_j) = \exp\Big(- \tfrac{1}{2\sigma^2} \|g_{\theta_b}(\boldsymbol{x}) - c_j\|_2^2 \Big). \tag{37.14}$$

An RBF network defines a mapping $f : \mathbb{R}^p \to \mathbb{R}^q$ of the form:

$$f(x) = \sum_{j=1}^{m} w_j \, \phi(\boldsymbol{x}, c_j), \tag{37.15}$$

where $w_j \in \mathbb{R}^q$ are trainable output weights. Thus, the prediction at x is expressed as a linear combination of local basis functions centred at the c_j. For classification tasks, the output of this linear combination is typically converted into class predictions either by applying a softmax to obtain probabilities or, in a prototype-based variant, by selecting the class whose centroid is closest to the input. The network is "local" in the sense that only the centres close to x make a significant contribution, explicitly encoding the principle that uncertainty grows as we move farther from the training distribution.

RBF networks have been used in prior work to quantify uncertainty in deep networks with a single forward pass, for example in DUQ [1653]. Nonetheless, these models are not truly probabilistic, and their notion of uncertainty is only implicit. Once the centres and weights are fixed, predictions reduce to deterministic point estimates. More importantly, they cannot disentangle epistemic from aleatoric uncertainty, which makes them less suitable for downstream tasks such as (Bayesian) active learning and experimental design.

37.5.3 The probabilistic head via generative classifiers

In the previous subsections, we described *discriminative heads*, which directly model $p(\boldsymbol{y} \mid \boldsymbol{x}, \theta_h)$. While discriminative classifiers are often effective for classification tasks, a key drawback is that they do not explicitly model the input distribution $p(\boldsymbol{x})$. This can lead to misleading uncertainty estimates: for example, in an RBF network that assigns labels by choosing the nearest centroid, the distance to that centroid can be small (suggesting low uncertainty) even if the point lies in a region where the true class has very low probability under its distribution. In other words, distance alone does not capture how concentrated or diffuse each class is.

An alternative is to use *generative classifiers*, which model the joint distribution $p(\boldsymbol{x}, \boldsymbol{y}) = p(\boldsymbol{x} \mid \boldsymbol{y})p(\boldsymbol{y})$ and recover predictions via Bayes' rule. By capturing the full class-conditional densities $p(\boldsymbol{x} \mid \boldsymbol{y})$ along with the prior over labels, generative classifiers can account for class-specific variance and better distinguish between regions of high and low support in the input space. This enables them to assign low confidence to points that are far from high-density regions of *any* class, resulting in qualitatively different and more reliable uncertainty estimates.

The simplest and most widely studied example of a generative classifier is *Gaussian Discriminant Analysis (GDA)*. In GDA, the class-conditional densities are modelled as multivariate normal distributions,

$$p(\boldsymbol{x} \mid \boldsymbol{y} = c) = \mathcal{N}(g_{\theta_b}(\boldsymbol{x});\, \mu_c, \Sigma_c), \tag{37.16}$$

and the prior over classes is often chosen to be Bernoulli in the binary case, or categorical in the multi-class case, i.e., $P(\boldsymbol{y} = c) = \frac{\beta_c}{\sum_{c'} \beta_{c'}}$, where β_c is the unnormalised prior for class c. A particularly interesting setting arises

when we assume the covariances are tied across classes: $\Sigma_c = \Sigma$ for all c. This corresponds to the classical *Linear Discriminant Analysis (LDA)* model. In this case, the posterior takes the following form:

$$P(\boldsymbol{y} = c \mid \boldsymbol{x}) = \frac{P(\boldsymbol{y} = c)\, p(\boldsymbol{x} \mid \boldsymbol{y} = c)}{\sum_{c'} P(\boldsymbol{y} = c')\, p(\boldsymbol{x} \mid \boldsymbol{y} = c')} = \frac{\exp(\mu_c^\top \Sigma^{-1} g_{\theta_b}(\boldsymbol{x}) - \frac{1}{2}\mu_c^\top \Sigma^{-1}\mu_c + \log \beta_c)}{\sum_{c'} \exp(\mu_{c'}^\top \Sigma^{-1} g_{\theta_b}(\boldsymbol{x}) - \frac{1}{2}\mu_{c'}^\top \Sigma^{-1}\mu_{c'} + \log \beta_{c'})}. \tag{37.17}$$

One can immediately recognise that this has the same form as a softmax classifier: the term $\mu_c^\top \Sigma^{-1}$ plays the role of the weight vector, while $-\frac{1}{2}\mu_c^\top \Sigma^{-1}\mu_c + \log \beta_c$ corresponds to a bias. This equivalence shows that training a softmax classifier implicitly corresponds to fitting a Gaussian model of the data, and moreover, that in principle we may recover the underlying densities from the discriminative formulation. To do so, we may compute the empirical class means and the tied covariance matrix as:

$$\hat{\mu}_c = \frac{1}{N_c} \sum_{i:y_i=c} g_{\theta_b}(\boldsymbol{x}_i), \qquad \hat{\Sigma} = \frac{1}{N} \sum_c \sum_{i:y_i=c} \big(g_{\theta_b}(\boldsymbol{x}_i) - \hat{\mu}_c\big)\big(g_{\theta_b}(\boldsymbol{x}_i) - \hat{\mu}_c\big)^\top, \tag{37.18}$$

where N_c is the number of training samples with label c. This procedure is equivalent to fitting the class-conditional Gaussian distributions with a tied covariance to the training features under the maximum likelihood estimator. With the class-conditional distributions estimated, prediction for a new input x reduces to measuring its distance from each class. A natural choice is the *Mahalanobis distance*,

$$D_c(\boldsymbol{x}) \;=\; \sqrt{(g_{\theta_b}(\boldsymbol{x}) - \hat{\mu}_c)^\top \hat{\Sigma}^{-1} (g_{\theta_b}(\boldsymbol{x}) - \hat{\mu}_c)}, \tag{37.19}$$

which accounts for the covariance structure of the data. The predicted label is the class with the smallest distance, while the distance itself serves as an uncertainty measure: small values indicate high density under class c, and large values correspond to low support, often signalling out-of-distribution inputs [1004]. Alternatively, we may directly evaluate the marginal likelihood of the feature representation as an uncertainty score:

$$p(\boldsymbol{x}) = \sum_c p(\boldsymbol{x} \mid \boldsymbol{y} = c)\, p(\boldsymbol{y} = c). \tag{37.20}$$

This principle is used in Deep Deterministic Uncertainty (DDU) [1206]. DDU directly fits a feature-density estimator using GDA without assuming tied covariances. It fits a single Gaussian per class by computing the empirical mean and covariance of the feature vectors for each of them. Then, it estimates epistemic uncertainty via Equation 37.20.

So far, the methods presented for functional heads only provide *proxies* for uncertainty. These are heuristics that resemble uncertainty estimates, as they capture the intuition that points far from the training distribution should be assigned higher uncertainty. However, they do not account for the fundamental source of epistemic uncertainty: our limited knowledge about the model parameter, which is formally captured via Bayesian posteriors. Next, we focus on truly Bayesian heads via Gaussian Processes, and discuss scalable methods for approximate inference.

37.5.4 The Bayesian head via Gaussian Processes

A natural progression from previous methods is to move toward a fully Bayesian treatment of function uncertainty. Gaussian Processes (GPs), covered in more depth in Chapter 23 and in the scalable-GP treatment of Chapter 38, provide such a framework: they define a prior over functions, and yield a posterior distribution that quantifies uncertainty in a principled way. GPs can be seen as a Bayesian analogue of kernel methods, where instead of heuristically measuring distances or fitting Gaussian densities in feature space, we directly model distributions over functions that explain the data.

Formally, a Gaussian Process is a distribution over functions $f : \mathcal{X} \to \mathbb{R}$ such that any finite collection of function values is jointly Gaussian,

$$f(\boldsymbol{x}) \sim \mathcal{GP}(m(\boldsymbol{x}), k(\boldsymbol{x}, \boldsymbol{x}')), \tag{37.21}$$

where $m(\boldsymbol{x})$ is the mean function (often set to zero) and $k(\boldsymbol{x}, \boldsymbol{x}')$ is the covariance kernel encoding smoothness and similarity assumptions. Given training data $\mathcal{D} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^N$ and assuming Gaussian observation noise, GP inference is analytically tractable. The predictive distribution at a new input $x^\star$ is Gaussian with mean and variance

$$\mu(\boldsymbol{x}^\star) = k(\boldsymbol{x}^\star, X)\,(K + \sigma^2 I)^{-1}\boldsymbol{y}, \quad \sigma^2(\boldsymbol{x}^\star) = k(\boldsymbol{x}^\star, \boldsymbol{x}^\star) - k(\boldsymbol{x}^\star, X)\,(K + \sigma^2 I)^{-1} k(X, \boldsymbol{x}^\star), \tag{37.22}$$

where K is the kernel matrix over the training inputs. Thus, predictions are accompanied by an explicit measure of uncertainty: high variance is assigned to regions far from observed data, reflecting epistemic uncertainty. Despite their conceptual elegance, GPs face important limitations: exact inference scales as $\mathcal{O}(N^3)$ in the number of data points N due to the inversion of K, and non-Gaussian likelihoods (e.g. for classification) render the posterior analytically intractable. In the following subsections, we will explore approximations that make Gaussian Processes practical in large-scale and non-Gaussian settings.

Approximated GP Inference via Random Fourier Features

Since all GP computations depend on the kernel matrix K, whose inversion costs $\mathcal{O}(N^3)$, replacing $k(x, x')$ with a fast approximation can dramatically reduce the complexity. Random Fourier Features (RFF) [1363] provide such an approximation for stationary kernels. By Bochner's theorem [176], any stationary kernel can be expressed as the Fourier transform of a positive measure, in particular:

$$k(\boldsymbol{x}, \boldsymbol{x'}) \;=\; \mathbb{E}_{\omega\sim p}\big[\cos\big(\omega^\top(\boldsymbol{x}-\boldsymbol{x'})\big)\big]\,, \tag{37.23}$$

where $p(\omega)$ is the spectral density of the kernel. Approximating this expectation with m Monte Carlo samples yields the feature map:

$$\phi(\boldsymbol{x}) \;=\; \sqrt{\tfrac{2}{m}}\big(\cos(\boldsymbol{\omega_1}^\top\boldsymbol{x}+b_1),\ldots,\cos(\boldsymbol{\omega_m}^\top\boldsymbol{x}+b_m)\big)^\top, \tag{37.24}$$

where $b_i \sim \text{Uniform}[0, 2\pi]$; the random phase b_i makes the estimator unbiased, since a product-to-sum expansion gives $\mathbb{E}_b[2\cos(\omega^\top\boldsymbol{x}+b)\cos(\omega^\top\boldsymbol{x'}+b)] = \cos(\omega^\top(\boldsymbol{x}-\boldsymbol{x'}))$ (the $\cos(\omega^\top(\boldsymbol{x}+\boldsymbol{x'})+2b)$ term integrates to zero over $b \sim \text{Uniform}[0, 2\pi]$). The kernel is then replaced by the inner product $k(\boldsymbol{x}, \boldsymbol{x'}) \;\approx\; \phi(\boldsymbol{x})^\top\phi(\boldsymbol{x'})$. With this approximation, the GP prior can be reformulated as a Bayesian linear model in the random feature space. We place a Gaussian prior on the weights, $\boldsymbol{w} \sim \mathcal{N}(0, I)$, and define the function as

$$f(\boldsymbol{x}) \;=\; \phi(\boldsymbol{x})^\top\boldsymbol{w}. \tag{37.25}$$

For training inputs $X = \{\boldsymbol{x_i}\}_{i=1}^N$, stacking the features into $\Phi \in \mathbb{R}^{N\times m}$ gives

$$\mathbf{f} = \Phi\boldsymbol{w}, \qquad \mathbf{f} \sim \mathcal{N}(0, \Phi\Phi^\top), \tag{37.26}$$

so that $\Phi\Phi^\top$ serves as a low-rank approximation to the exact kernel matrix K.

The computational advantage is immediate: instead of inverting the $N \times N$ kernel matrix, we only need to handle the $m \times m$ system arising from the weight posterior. This reduces the cost from $\mathcal{O}(N^3)$ to $\mathcal{O}(Nm^2 + m^3)$, with $m \ll N$. In practice, a few hundred features are often sufficient, making GP-style inference feasible even for datasets with tens of thousands of points. This is how SNGP [1061] enables BDL with a single forward pass in a deep backbone.

Nevertheless, a key limitation of RFF is that it sacrifices the non-parametric property of GPs. For any finite number of features, the GP becomes a parametric linear model in a fixed basis. As the dataset grows, the posterior variance can erroneously shrink to zero everywhere, even in regions far from the data, unlike a true GP which maintains uncertainty away from observations.

Approximated Inference via Variational Inducing Points

While Random Fourier Features approximate the kernel function itself, an alternative route is to retain the exact GP prior and instead approximate the posterior distribution in a scalable way [1615]. The key observation is that a GP is entirely specified by its values on any finite set of inputs, so one can summarise the function through a small number of carefully chosen pseudo-points. These inducing inputs provide a low-rank representation that can be optimised to best explain the data while maintaining consistency with the GP prior.

Concretely, let $Z = \{z_j\}_{j=1}^m$ be a set of inducing inputs with corresponding function values $\mathbf{u} = (f(z_1), \ldots, f(z_m))^\top$. For training inputs $X = \{x_i\}_{i=1}^N$, the joint GP prior over $\mathbf{f} = (f(x_1), \ldots, f(x_N))^\top$ and $\mathbf{u}$ is Gaussian:

$$\begin{bmatrix}\mathbf{f}\\ \mathbf{u}\end{bmatrix} \;\sim\; \mathcal{N}\!\left(\mathbf{0},\, \begin{bmatrix}K_{ss} & K_{sm}\\ K_{ms} & K_{mm}\end{bmatrix}\right), \tag{37.27}$$

where $K_{ss} = K_{XX}$, $K_{mm} = K_{ZZ}$, and $K_{sm} = K_{XZ}$. From standard Gaussian conditioning, the conditional distribution of $\mathbf{f}$ given $\mathbf{u}$ is:

$$p(\mathbf{f} \mid \mathbf{u}) \;=\; \mathcal{N}\big(K_{sm}K_{mm}^{-1}\mathbf{u},\, K_{ss} - Q_{ss}\big), \tag{37.28}$$

with $Q_{ss} = K_{sm}K_{mm}^{-1}K_{ms}$. Once we know the function values at the inducing points, the rest of the process is Gaussian with mean given by interpolation through Z and residual covariance measuring what cannot be explained by that interpolation.

The goal is to approximate the posterior over $\mathbf{f}$ given observations $\mathbf{y}$. The marginal likelihood of the model is:

$$p(\mathbf{y}) = \int p(\mathbf{y} \mid \mathbf{f})\, p(\mathbf{f} \mid \mathbf{u})\, p(\mathbf{u})\, d\mathbf{f}\, d\mathbf{u}, \tag{37.29}$$

but this integral is generally intractable. To address this, we introduce a variational distribution $q(\mathbf{u}) = \mathcal{N}(\mathbf{m}, \mathbf{S})$ and define:

$$q(\mathbf{f}, \mathbf{u}) \;=\; p(\mathbf{f} \mid \mathbf{u})\, q(\mathbf{u}). \tag{37.30}$$

This preserves the exact conditional structure of the GP and only approximates the distribution over inducing variables. Applying Jensen's inequality yields the evidence lower bound (ELBO):

$$\log p(\mathbf{y}) \;\geq\; \mathbb{E}_{q(\mathbf{f})}\big[\log p(\mathbf{y} \mid \mathbf{f})\big] - \mathrm{D}_{\mathrm{KL}}(q(\mathbf{u}) \| p(\mathbf{u})), \tag{37.31}$$

where $q(\mathbf{f}) = \int p(\mathbf{f} \mid \mathbf{u})\, q(\mathbf{u})\, d\mathbf{u}$ is Gaussian with mean $K_{sm}K_{mm}^{-1}\mathbf{m}$ and covariance $K_{ss} + K_{sm}K_{mm}^{-1}(\mathbf{S} - K_{mm})K_{mm}^{-1}K_{ms}$. This objective can be maximised with respect to the variational parameters $(\mathbf{m}, \mathbf{S})$, the inducing locations Z, and the kernel hyperparameters.

For Gaussian likelihoods, this bound can be evaluated in closed form, leading to a particularly elegant expression in which the log marginal likelihood of the Nyström approximation Q_{ss} is corrected by a trace term that penalises information lost outside the inducing subspace. For general likelihoods, the bound remains tractable through Monte Carlo estimates, and its factorised structure allows the use of minibatches. This makes the approach suitable for very large datasets.

The computational benefits are similar to those of RFFs: instead of inverting the full $N \times N$ kernel matrix, training reduces to operations involving the $m \times m$ covariance among the inducing points. This reduces complexity from $\mathcal{O}(N^3)$ to $\mathcal{O}(Nm^2 + m^3)$, with m typically in the hundreds. Unlike RFFs, however, the basis here is *not fixed in advance*. The inducing locations are variational parameters learned to maximize the evidence bound, adapting flexibly to the data distribution. Moreover, because the exact conditional structure of the GP is preserved, the predictive distribution retains good uncertainty estimation, with the trace correction ensuring that posterior variance does not collapse spuriously far from the data. Deterministic Uncertainty Estimation (DUE) [1654] uses this GP approximation method for BDL with a single forward pass.

Nevertheless, inducing point methods also have clear limitations. The quality of the approximation is bounded by the span of the inducing variables: if m is too small relative to the complexity of the function, the variational family may be too restrictive, leading to underfitting. Although increasing m makes the method arbitrarily close to the exact GP, this comes at the cost of higher $\mathcal{O}(m^3)$ operations, which can again become prohibitive. Optimising the inducing locations jointly with kernel hyperparameters is a non-convex problem and can be sensitive to initialisation. Finally, in very high-dimensional input spaces, it can be challenging to place inducing points effectively, since covering the relevant regions of the input space requires exponentially many points.

37.6 Outlook

This chapter introduced several methods that form the foundation of scalable BDL with a single forward pass. We now compare these approaches quantitatively to highlight how the different predictive heads and backbone regularisation schemes trade off predictive performance, uncertainty estimation, and computational efficiency.

Table 37.3 summarizes the performance of a range of methods that require only a single forward pass at inference, along with a standard deterministic neural network and Deep Ensembles. All models were trained on CIFAR-10 and evaluated both in-distribution (on the CIFAR-10 test set) and out-of-distribution (on SVHN). The results are adapted from Mukhoti et al. [1206], van Amersfoort et al. [1654].

Deep Ensembles deliver the strongest overall performance in both accuracy and uncertainty calibration but at a much higher computational cost—roughly five times that of a single model. They therefore serve as a useful reference point for assessing more efficient alternatives that aim to approach ensemble-level reliability at lower cost.

Among single-pass models, those employing spectral normalisation to enforce approximate bi-Lipschitzness, such as DDU and SNGP, perform best. Spectral normalisation provides an effective regularisation mechanism: it stabilises the learned representation and constrains the sensitivity of the predictive head, while keeping runtime roughly similar to that of a standard network. In contrast, Jacobian regularisation (as in DUQ) or the absence of any constraint tends to yield less stable uncertainty estimates and weaker predictive performance.

Method	Regulariser	Head	Runtime	Accuracy (↑)	AUROC^2 SVHN (↑)
Deterministic NN	-	Standard NN	1x	95.98 ± 0.02	94.44 ± 0.43
DUQ [1653]	Jacobian	RBF Network	3.5x	94.6 ± 0.16	93.71 ± 0.61
DDU [1206]	Spectral	GDA	1x	95.97 ± 0.03	$\mathbf{97.86 \pm 0.19}$
SNGP [1061]	Spectral	GP (RFFs)	1x	$\mathbf{96.04 \pm 0.09}$	94.0 ± 1.3
DUE [1654]	Spectral	GP (Inducing)	1x	95.6 ± 0.04	95.8 ± 0.005
Deep Ensembles [962]	-	Standard NN	5x	$\mathbf{96.59 \pm 0.02}$	97.73 ± 0.31

Table 37.3: Comparison of scalable Bayesian deep learning methods on CIFAR-10 (in-distribution) and SVHN (out-of-distribution). Spectral normalisation–based approaches (DDU, SNGP) achieve the best trade-off between predictive performance, uncertainty calibration, and efficiency, outperforming Jacobian-regularised and unregularised baselines. While Deep Ensembles remain the strongest overall, they do so at a substantially higher computational cost, highlighting the progress and remaining challenges in developing scalable single-pass uncertainty estimators. Adapted from Mukhoti et al. [1206] and van Amersfoort et al. [1654].

Within this family, DDU achieves the best out-of-distribution evaluation and matches SNGP in in-distribution accuracy. Fully Bayesian approaches—such as Gaussian Process heads with Random Fourier Features or inducing points—improve upon deterministic networks but still fall short of DDU in both accuracy and calibration. This gap suggests that, while Bayesian formulations provide a principled foundation for uncertainty estimation, practical implementations remain limited by computational constraints. Even with low-rank approximations, training and inference scale cubically with the number of features or inducing points, restricting the achievable model capacity.

Overall, the methods covered in this chapter have addressed key challenges in combining deep learning with Bayesian principles, leading to models capable of producing reliable, uncertainty-aware predictions. Nevertheless, there is still room for improvement, particularly for fully Bayesian predictive heads, to achieve state-of-the-art predictive power and uncertainty estimation at a reasonable computational cost.

Chapter 38

Scalable Gaussian processes

Gaussian processes [GPs; 1382], as introduced in Chapter 26, are gold standard methods for function space uncertainty quantification. Although they typically achieve accurate predictions with reliable uncertainty estimates on small datasets and relatively low-dimensional problems, it remains a challenge to scale them up to big datasets with high-dimensional inputs. The issue of the curse of dimensionality might be mitigated with better kernel design, e.g., deep kernel learning (introduced in Chapter 24), and we will not discuss it in detail in this chapter since it is prevalent in all kernel methods. Instead, we focus on reviewing methods that scale GPs to big data with a large number of data points. The main caveat of applying full GP in big data is the requirement of computing matrix inversion and matrix determinant involving the kernel matrix, $\mathbf{K}_{\mathbf{XX}}$, evaluated over all training inputs, $\mathbf{X}$, which scales cubically w.r.t the number of training instances. Therefore, approximations that bypass matrix inversion and matrix determinant computation for large matrices are indispensable.

In this chapter, we review two classes of structured approximation techniques commonly used in the context of GPs. The first class of approaches directly employ low-rank approximations for $\mathbf{K}_{\mathbf{XX}}$, and among them, Nyström approximation [1760] and random Fourier feature (RFF) [1363] are two prominent techniques. The second class of approaches, variational inducing points based approximations, are more tailored to GP models and we will discuss them in more detail. Specifically, a sparse variational Gaussian process [SVGP; 1528] constructs an approximate posterior process in a variational inference (VI) framework [851] (introduced in Chapter 12), and this approximate posterior is a structured GP fully determined by a small number of tunable inducing points, which only requires matrix inversion and determinant computation for the kernel matrix evaluated over the inducing points instead of the full set of training inputs. We also discuss decoupled SVGPs [285, 1454, 1513] and interdomain GPs [1663] as extensions of this framework. Furthermore, for streaming tasks where new data points keep being observed, storing all past data for retraining is expensive, and we resort to online or continual GP methods [218, 278] to bypass the requirement of storing all the data for retraining, but also to allow the model to maintain consistent performance across all the past tasks.

38.1 Direct approximations for $\mathbf{K}_{\mathbf{XX}}$

In Chapter 23, we see that the main computational cost of posterior inference in GP regression comes from the matrix inversion $(\mathbf{K}_{\mathbf{XX}} + \sigma^2\mathbf{I})^{-1}$ in the posterior predictive distribution, and the log-determinant evaluation $\log\det(\mathbf{K}_{\mathbf{XX}} + \sigma^2\mathbf{I})$ in the log-marginal-likelihood objective for tuning the kernel hyperparameters. Low-rank matrix approximation can be adopted to address this issue. Suppose $\mathbf{K}_{\mathbf{XX}} \in \mathbb{R}^{N\times N}$ admits a structured approximation of the form $\mathbf{K}_{\mathbf{XX}} \approx \mathbf{\Phi}^\top\mathbf{\Sigma}\mathbf{\Phi}$, where $\mathbf{\Phi} \in \mathbb{R}^{M\times N}$, $\mathbf{\Sigma} \in \mathbb{R}^{M\times M}$ and $M \ll N$. Then, by applying matrix inversion and matrix determinant calculations to the low-rank approximation, the original matrix inverse and determinant terms, which require $O(N^3)$ time complexity in computation, can be approximately computed with $O(NM^2 + M^3)$ time complexity:

$$(\mathbf{K}_{\mathbf{XX}} + \sigma^2\mathbf{I})^{-1} \approx \frac{1}{\sigma^2}\mathbf{I} - \frac{1}{\sigma^4}\mathbf{\Phi}^\top(\mathbf{\Sigma}^{-1} + \frac{1}{\sigma^2}\mathbf{\Phi}\mathbf{\Phi}^\top)^{-1}\mathbf{\Phi}, \tag{38.1}$$

$$\log\det(\mathbf{K}_{\mathbf{XX}} + \sigma^2\mathbf{I}) \approx 2N\log\sigma + \log\det(\mathbf{\Sigma}^{-1} + \frac{1}{\sigma^2}\mathbf{\Phi}\mathbf{\Phi}^\top) + \log\det(\mathbf{\Sigma}). \tag{38.2}$$

A few popular kernel approximation methods employ the structured form introduced above [1027, 1322, 1363, 1623, 1760], and we review Nyström approximation [1760] and random Fourier feature approximation [1363] as two examples.

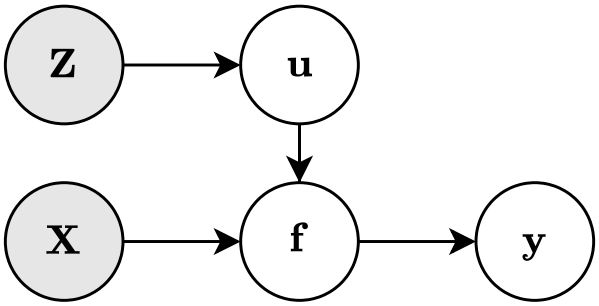


Figure 38.1: The graphical model for prediction with a sparse variational Gaussian process (SVGP). Shaded nodes ($\mathbf{X}$, $\mathbf{y}$) are observed and unshaded nodes are latent. Here $\mathbf{Z}$ denotes the inducing input locations (treated as hyperparameters), $\mathbf{u}$ the corresponding inducing function values, $\mathbf{f}$ the latent function values at the training inputs $\mathbf{X}$, and $\mathbf{y}$ the observed targets.

Nyström approximation. Nyström approximation [1760], which is closely related to the subset-of-regressors, DTC, and FITC approximations unified in Quiñonero-Candela and Rasmussen [1354], considers $\mathbf{\Phi}$ to be $\mathbf{K_{ZX}}$ and $\mathbf{\Sigma}$ to be $\mathbf{K_{ZZ}^{-1}}$, where $\mathbf{Z} \subset \mathbf{X}$ is a subset of M inputs randomly selected from the training set. Consequently, the kernel matrix evaluated on the training set is approximated as $\mathbf{K_{XX}} \approx \mathbf{K_{XZ}}\mathbf{K_{ZZ}^{-1}}\mathbf{K_{ZX}}$, and the matrix inverse and log-determinant terms are approximated as follows:

$$(\mathbf{K_{XX}} + \sigma^2\mathbf{I})^{-1} \approx \frac{1}{\sigma^2}\mathbf{I} - \frac{1}{\sigma^4}\mathbf{K_{XZ}}(\mathbf{K_{ZZ}} + \frac{1}{\sigma^2}\mathbf{K_{ZX}}\mathbf{K_{XZ}})^{-1}\mathbf{K_{ZX}}, \tag{38.3}$$

$$\log\det(\mathbf{K_{XX}} + \sigma^2\mathbf{I}) \approx 2N\log\sigma + \log\det(\mathbf{K_{ZZ}} + \frac{1}{\sigma^2}\mathbf{K_{ZX}}\mathbf{K_{XZ}}) - \log\det(\mathbf{K_{ZZ}}). \tag{38.4}$$

Random Fourier features. For stationary kernels (i.e., $k(\mathbf{x}, \mathbf{x}') = k(\|\mathbf{x} - \mathbf{x}'\|)$), Bochner's theorem [1433] provides a Fourier feature approximation:

$$\begin{aligned} k(\mathbf{x}, \mathbf{x}') &\approx \frac{1}{M}\sum_{m=1}^{M}\left[\cos\left(\mathbf{w}_m^\top\mathbf{x}\right)\cos\left(\mathbf{w}_m^\top\mathbf{x}'\right) + \sin\left(\mathbf{w}_m^\top\mathbf{x}\right)\sin\left(\mathbf{w}_m^\top\mathbf{x}'\right)\right] \\ &= \boldsymbol{\phi}(\mathbf{x})^\top\boldsymbol{\phi}(\mathbf{x}'), \quad \mathbf{w}_m \sim p(\mathbf{w}), \end{aligned} \tag{38.5}$$

where $\boldsymbol{\phi}(\mathbf{x}) = [\cos\left(\mathbf{w}_1^\top\mathbf{x}\right), \cdots, \cos\left(\mathbf{w}_M^\top\mathbf{x}\right), \sin\left(\mathbf{w}_1^\top\mathbf{x}\right), \cdots, \sin\left(\mathbf{w}_M^\top\mathbf{x}\right)]^\top$ and $p(\mathbf{w})$ is the spectral density of the stationary kernel. Based on this result, given M samples from the spectral density, $\{\mathbf{w}_m\}_{m=1}^M$, random Fourier feature (RFF) approach [1363] approximates $\mathbf{K_{XX}}$ as $\mathbf{K_{XX}} \approx \mathbf{\Phi}^\top\mathbf{\Phi}$, where $\mathbf{\Phi} = [\boldsymbol{\phi}(\mathbf{x}_1), \cdots, \boldsymbol{\phi}(\mathbf{x}_N)]^\top$. Again, RFF approximation admits the desired structured form by simply noticing that $\mathbf{\Sigma} = \mathbf{I}$ here. As a result, the matrix inversion and log-determinant terms are approximated as follows:

$$(\mathbf{K_{XX}} + \sigma^2\mathbf{I})^{-1} \approx \frac{1}{\sigma^2}\mathbf{I} - \frac{1}{\sigma^4}\mathbf{\Phi}^\top(\mathbf{I} + \frac{1}{\sigma^2}\mathbf{\Phi}\mathbf{\Phi}^\top)^{-1}\mathbf{\Phi} \tag{38.6}$$

$$\log\det(\mathbf{K_{XX}} + \sigma^2\mathbf{I}) \approx 2N\log\sigma + \log\det(\mathbf{I} + \frac{1}{\sigma^2}\mathbf{\Phi}\mathbf{\Phi}^\top). \tag{38.7}$$

For non-stationary kernels, more advanced Fourier feature techniques (e.g., Ton et al. [1623]) may be leveraged to construct the kernel approximation. Moreover, there exist Fourier features tailored to some specific kernels, such as the RBF/Gaussian kernel, that achieve more accurate approximations for high-dimensional inputs [1387, 1822].

38.2 Sparse variational Gaussian processes

This section reviews sparse variational Gaussian process [1528], a VI framework that extends GPs to big data. SVGP considers a structured GP as the approximate posterior process and its structure is fully determined by a small set (with size $M \ll N$) of pseudo input-function value pairs, $\{(\mathbf{z}_m, u_m := f(\mathbf{z}_m))\}_{m=1}^M$, which are commonly known as inducing points [737, 738, 1615].

With the additional inducing points, the prior model for the augmented set consisting of $\mathbf{u}$ and $\mathbf{f}$ (evaluated at arbitrary inputs $\mathbf{X}$), according to the prior GP $p(f) = \mathcal{GP}(0, k_\psi(\cdot,\cdot))$ with kernel hyperparameters ψ, is

$$p(\mathbf{f}, \mathbf{u}|\mathbf{X}, \mathbf{Z}) = \mathcal{N}(\mathbf{0}, \begin{pmatrix} \mathbf{K_{XX}} & \mathbf{K_{XZ}} \\ \mathbf{K_{ZX}} & \mathbf{K_{ZZ}} \end{pmatrix}). \tag{38.8}$$

SVGP then constructs an approximate posterior GP based on a Gaussian variational distribution for the set of inducing points, $q(\mathbf{u}) = \mathcal{N}(\mathbf{m_u}, \mathbf{S_u})$, where $\mathbf{m_u}$ and $\mathbf{S_u}$ are tunable variational mean and variational covariance parameters. Furthermore, we assume that the conditional distribution of function values given inducing variables in this approximate posterior is the same as the prior conditional distribution.

$$\text{(prior conditional matching)} \qquad q(\mathbf{f}|\mathbf{u}, \mathbf{Z}, \mathbf{X}) = p(\mathbf{f}|\mathbf{u}, \mathbf{Z}, \mathbf{X}). \tag{38.9}$$

With this prior conditional matching assumption, the induced approximate posterior for function values $\mathbf{f}$ is completely controlled by the variational distribution $q(\mathbf{u})$, and it can be shown to follow a structured Gaussian distribution

$$\begin{aligned} q(\mathbf{f}|\mathbf{X}, \mathbf{Z}) &= \int p(\mathbf{f}|\mathbf{u}, \mathbf{Z}, \mathbf{X}) q(\mathbf{u}) d\mathbf{u} \\ &= \mathcal{N}(\mathbf{f}; \mathbf{K_{XZ}}\mathbf{K_{ZZ}^{-1}}\mathbf{m_u}, \mathbf{K_{XX}} + \mathbf{K_{XZ}}\mathbf{K_{ZZ}^{-1}}(\mathbf{S_u} - \mathbf{K_{ZZ}})\mathbf{K_{ZZ}^{-1}}\mathbf{K_{ZX}}). \end{aligned} \tag{38.10}$$

Clearly, when $\mathbf{X}$ is the training set, the approximate posterior above reduces the $O(N^3)$ time complexity of full GP to $O(M^3 + NM^2)$ by noticing that the expensive matrix inversion is now applied to a smaller matrix $\mathbf{K_{ZZ}}$ (with $O(M^3)$ time complexity). The posterior predictive distribution of $\mathbf{y}$ is then obtained via the following integral

$$q(\mathbf{y}|\mathbf{X}, \mathbf{Z}) = \int\int p(\mathbf{y}|\mathbf{f}) p(\mathbf{f}|\mathbf{u}, \mathbf{Z}, \mathbf{X}) q(\mathbf{u}) d\mathbf{u} d\mathbf{f} = \mathbb{E}_{q(\mathbf{f}|\mathbf{X},\mathbf{Z})}[p(\mathbf{y}|\mathbf{f})]. \tag{38.11}$$

We plot the graphical model for the posterior prediction of SVGP in Figure 38.1. The flexibility of specifying tractable approximate posterior in the VI framework allows us to handle the cases where non-conjugate observation likelihoods are considered. Specifically, in the case of SVGP, although the posterior predictive distribution above has no closed form with non-Gaussian likelihoods, we can still use Monte Carlo to obtain an unbiased estimate since we can easily draw samples from $q(\mathbf{f}|\mathbf{X}, \mathbf{Z})$, which is a Gaussian (Eq. 38.10). In regression problems, where Gaussian observation likelihood is used, this posterior predictive distribution can be computed analytically as a Gaussian and the SVGP in this case is known as sparse Gaussian process regression [SGPR; 1615].

To ensure this approximate posterior to be close to the true posterior, we tune the inducing input locations $\mathbf{Z}$ (treated as model hyperparameters), the kernel hyperparameters and the variational parameters jointly by maximizing the evidence lower bound (ELBO). Specifically, with a training set $\mathcal{D} = (\mathbf{X}, \mathbf{y})$, the ELBO of SVGP is given as follows

$$\begin{aligned} \mathcal{L}_{ELBO} &= \mathbb{E}_{q(\mathbf{f},\mathbf{u}|\mathbf{Z},\mathbf{X})}[\log \frac{p(\mathbf{y}|\mathbf{f}) p(\mathbf{f}|\mathbf{u}, \mathbf{Z}, \mathbf{X}) p(\mathbf{u}|\mathbf{Z})}{q(\mathbf{f}, \mathbf{u}|\mathbf{Z}, \mathbf{X})}] \\ &= \underbrace{\mathbb{E}_{q(\mathbf{f}|\mathbf{X},\mathbf{Z})}[\log p(\mathbf{y}|\mathbf{f})]}_{\text{ELL}} - \underbrace{\mathrm{KL}(q(\mathbf{u})||p(\mathbf{u}|\mathbf{Z}))}_{\text{KL regulariser}} \end{aligned} \tag{38.12}$$

The above ELBO consists of two terms. The first term is commonly referred to as expected log-likelihood (ELL), which encourages better data fit, and the second term is a KL regulariser preventing the approximate posterior from significantly deviating from the prior. Since $q(\mathbf{u})$ and $p(\mathbf{u}|\mathbf{Z})$ are both Gaussian, the KL regulariser can be evaluated analytically as follows.

$$\mathrm{KL}[q(\mathbf{u})||p(\mathbf{u}|\mathbf{Z})] = \frac{1}{2}\mathbf{m_u}^\top \mathbf{K_{ZZ}^{-1}}\mathbf{m_u} + \frac{1}{2}\mathrm{Tr}[\mathbf{K_{ZZ}^{-1}}\mathbf{S_u} - \mathbf{I}] - \frac{1}{2}\log\det(\mathbf{K_{ZZ}^{-1}}\mathbf{S_u}). \tag{38.13}$$

Although the ELL term has no closed form with non-Gaussian likelihoods, we can again resort to Monte Carlo to estimate it. The main costs for the evaluation of the ELL and KL regulariser come from the computation of $q(\mathbf{f}|\mathbf{X}, \mathbf{Z})$ ($O(M^3 + NM^2)$), and the computation of the determinant and inversion of $\mathbf{K_{ZZ}}$ ($O(M^3)$), respectively, which are significantly lower than the $O(N^3)$ time complexity of full GP regression.

In SGPR where Gaussian observation likelihood is in use, the ELL term can also be computed analytically and the optimal variational mean and covariance parameters in $q(\mathbf{u})$ can be derived in closed forms as functions of inducing locations $\mathbf{Z}$ and kernel hyperparameters. As a result, the ELBO can be simplified to a collapsed bound [1615] for training $\mathbf{Z}$ and kernel hyperparameters only. This collapsed bound is particularly useful in practice as it eliminates the variational parameters entirely, reducing the number of optimisable parameters and providing a tighter lower bound on the log-marginal likelihood.

38.3 Decoupled sparse variational Gaussian processes

While standard SVGP considers direct optimisation of the variational parameters (i.e., mean and covariance of $q(\mathbf{u})$) for the inducing points, decoupled SVGPs [285, 1454, 1513] further improve the efficiency of sparse GP inference by imposing structures in the variational parameters, allowing structured decoupling of the inducing points into two complementary subsets, used to construct the posterior predictive mean and covariance, respectively.

Consider splitting the inducing points to obtain two complementary subsets: $(\mathbf{Z}, \mathbf{u}) = \{(\mathbf{Z}_\gamma, \mathbf{u}_\gamma), (\mathbf{Z}_\beta, \mathbf{u}_\beta)\}$, where the index sets $\gamma, \beta \subset \{1, ..., M\}$ satisfy $\gamma \cap \beta = \emptyset, \gamma \cup \beta = \{1, ..., M\}$. Decoupled SVGP [DSVGP; 285] then considers a structured variational distribution for the inducing points according to the split as follows

$$q(\mathbf{u}) = q(\mathbf{u}_\gamma)q(\mathbf{u}_\beta|\mathbf{u}_\gamma) = \mathcal{N}(\mathbf{u}_\gamma; \mathbf{0}, \mathbf{S}_\gamma)\mathcal{N}(\mathbf{u}_\beta; \mathbf{C}\mathbf{u}_\gamma + \mathbf{m}_\beta, \mathbf{B}), \tag{38.14}$$

where $\mathbf{B} = \mathbf{K}_{\mathbf{Z}_\beta\mathbf{Z}_\beta} - \mathbf{K}_{\mathbf{Z}_\beta\mathbf{Z}_\gamma}\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}\mathbf{K}_{\mathbf{Z}_\gamma\mathbf{Z}_\beta}$ and $\mathbf{C} = \mathbf{K}_{\mathbf{Z}_\beta\mathbf{Z}_\gamma}\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}$. The corresponding variational mean and covariance are:

$$\mathbf{m}_\mathbf{u} = \begin{pmatrix} \mathbf{0} \\ \mathbf{m}_\beta \end{pmatrix}, \quad \mathbf{S}_\mathbf{u} = \begin{pmatrix} \mathbf{S}_\gamma & \mathbf{S}_\gamma\mathbf{C}^\top \\ \mathbf{C}\mathbf{S}_\gamma & \mathbf{B} + \mathbf{C}\mathbf{S}_\gamma\mathbf{C}^\top \end{pmatrix}. \tag{38.15}$$

It can then be shown that:

$$\mathbf{K}^{-1}_{\mathbf{Z}\mathbf{Z}}\mathbf{S}_\mathbf{u} = \begin{bmatrix} \mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}\mathbf{S}_\gamma & (\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}\mathbf{S}_\gamma - \mathbf{I})\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}\mathbf{K}_{\mathbf{Z}_\gamma\mathbf{Z}_\beta} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}, \tag{38.16}$$

$$\mathbf{K}^{-1}_{\mathbf{Z}\mathbf{Z}}\mathbf{S}_\mathbf{u}\mathbf{K}^{-1}_{\mathbf{Z}\mathbf{Z}} - \mathbf{K}^{-1}_{\mathbf{Z}\mathbf{Z}} = \begin{bmatrix} \mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}\mathbf{S}_\gamma\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma} - \mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}. \tag{38.17}$$

Integrating this new design of the variational mean and covariance parameters into the posterior predictive distribution of SVGP (Eq. 38.10) allows us to obtain the posterior distribution $q(\mathbf{f}|\mathbf{X}, \mathbf{Z})$ for DSVGP, which is a Gaussian with the following mean $\mathbf{m}_\mathbf{f}$ and covariance $\mathbf{\Sigma}_{\mathbf{ff}}$:

$$\begin{aligned} \mathbf{m}_\mathbf{f} &= \mathbf{K}_{\mathbf{X}\mathbf{Z}_\beta}\mathbf{K}^{-1}_{\mathbf{Z}_\beta\mathbf{Z}_\beta}\mathbf{m}_\beta, \\ \mathbf{\Sigma}_{\mathbf{ff}} &= K_{\mathbf{X}\mathbf{X}} + \mathbf{K}_{\mathbf{X}\mathbf{Z}_\gamma}\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}(\mathbf{S}_\gamma - \mathbf{K}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma})\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}\mathbf{K}_{\mathbf{Z}_\gamma\mathbf{X}}. \end{aligned} \tag{38.18}$$

Moreover, the KL regulariser in the ELBO objective (Eq. 38.12) becomes

$$\mathrm{KL}[q(\mathbf{u})||p(\mathbf{u}|\mathbf{Z})] = \frac{1}{2}\mathbf{m}_\beta{}^\top\mathbf{K}^{-1}_{\mathbf{Z}_\beta\mathbf{Z}_\beta}\mathbf{m}_\beta + \frac{1}{2}\mathrm{Tr}[\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}\mathbf{S}_\gamma - \mathbf{I}] - \frac{1}{2}\log\det(\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}\mathbf{S}_\gamma). \tag{38.19}$$

In addition, since in both the posterior predictive distribution, $q(\mathbf{f}|\mathbf{X}, \mathbf{Z})$, and the KL regulariser, the variational mean parameter, $\mathbf{m}_\beta$, always appears together with a left matrix multiplication of $\mathbf{K}^{-1}_{\mathbf{Z}_\beta\mathbf{Z}_\beta}$, we can reparameterise the variational mean as $\mathbf{v}_\beta := \mathbf{K}_{\mathbf{Z}_\beta\mathbf{Z}_\beta}{}^{-1}\mathbf{m}_\beta$, which is then treated as a final variational parameter to optimise. As a result, the posterior predictive mean and the KL regulariser can be rewritten as

$$\begin{aligned} \mathbf{m}_\mathbf{f} &= \mathbf{K}_{\mathbf{X}\mathbf{Z}_\beta}\mathbf{v}_\beta, \\ \mathrm{KL}[q(\mathbf{u})||p(\mathbf{u}|\mathbf{Z})] &= \frac{1}{2}\mathbf{v}_\beta{}^\top\mathbf{K}_{\mathbf{Z}_\beta\mathbf{Z}_\beta}\mathbf{v}_\beta + \frac{1}{2}\mathrm{Tr}[\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}\mathbf{S}_\gamma - \mathbf{I}] - \frac{1}{2}\log\det(\mathbf{K}^{-1}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}\mathbf{S}_\gamma). \end{aligned} \tag{38.20}$$

Notice that with this reparameterisation, the expensive matrix inversion and determinant evaluation are now all computed for $\mathbf{K}_{\mathbf{Z}_\gamma\mathbf{Z}_\gamma}$ based on the second set of inducing points, $\{(\mathbf{Z}_\gamma, \mathbf{u}_\gamma)\}$, of size $M_\gamma := |\gamma| < M$, allowing further reduction of computation cost. On the other hand, one can consider using a larger number of inducing points $(\mathbf{Z}_\beta, \mathbf{u}_\beta)$ to ensure a more accurate posterior predictive mean, since now the time complexity of the posterior predictive distribution scales linearly w.r.t. M_β as shown in Table 38.1.

Salimbeni et al. [1454] further improved the optimisation of DSVGP by considering an alternative structured decoupling that admits an efficient natural gradient [26] update rule for faster convergence. The resulting approach, orthogonally decoupled SVGP (ODSVGP), considers the following decoupled variational distribution design:

$$\begin{aligned} q(\mathbf{u}) &= q(\mathbf{u}_\gamma)q(\mathbf{u}_\beta|\mathbf{u}_\gamma) \\ &= \mathcal{N}(\mathbf{u}_\gamma; \mathbf{m}_\gamma, \mathbf{S}_\gamma)\mathcal{N}(\mathbf{u}_\beta; \mathbf{C}\mathbf{u}_\gamma + \mathbf{B}\mathbf{K}_{\mathbf{Z}_\beta\mathbf{Z}_\beta}{}^{-1}\mathbf{m}_\beta, \mathbf{B}), \end{aligned} \tag{38.21}$$

which admits an alternative decoupled parameterisation of the variational parameters

$$\mathbf{m}_\mathbf{u} = \begin{pmatrix} \mathbf{m}_\gamma \\ \mathbf{C}\mathbf{m}_\gamma + \mathbf{B}\mathbf{K}_{\mathbf{Z}_\beta\mathbf{Z}_\beta}{}^{-1}\mathbf{m}_\beta \end{pmatrix}, \quad \mathbf{S}_\mathbf{u} = \begin{pmatrix} \mathbf{S}_\gamma & \mathbf{S}_\gamma\mathbf{C}^\top \\ \mathbf{C}\mathbf{S}_\gamma & \mathbf{B} + \mathbf{C}\mathbf{S}_\gamma\mathbf{C}^\top \end{pmatrix}. \tag{38.22}$$

Table 38.1: Complexity comparison for GP, SVGP, DSVGP, and ODSVGP. Here N denotes the number of training points, M the total number of inducing points, and M_β, M_γ the sizes of the two complementary inducing subsets used in the decoupled variants (with $M_\beta + M_\gamma = M$).

Model	Time complexity	Memory complexity
GP [1382]	$O(N^3)$	$O(N^2)$
SVGP [737, 738]	$O(NM^2 + M^3)$	$O(M^2)$
DSVGP [285]	$O(NM_\beta + NM_\gamma^2 + M_\gamma^3)$	$O(M_\beta + M_\gamma^2)$
ODSVGP [1454]	$O(NM_\gamma M_\beta + NM_\gamma^2 + M_\gamma^3)$	$O(M_\beta + M_\gamma^2)$

The posterior predictive covariance $\mathbf{\Sigma_{ff}}$ is still the same as in Eq. 38.18, but the posterior predictive mean is now computed based on all the inducing points. With the reparameterisation $\mathbf{v}_\beta := {\mathbf{K_{Z_\beta Z_\beta}}}^{-1}\mathbf{m}_\beta$ and $\mathbf{v}_\gamma := {\mathbf{K_{Z_\gamma Z_\gamma}}}^{-1}\mathbf{m}_\gamma$, the posterior predictive mean is computed as

$$\mathbf{m_f} = \mathbf{K_{XZ_\beta}}\mathbf{v}_\beta - \mathbf{K_{XZ_\gamma}}\mathbf{K}^{-1}_{\mathbf{Z_\gamma Z_\gamma}}\mathbf{K_{Z_\gamma Z_\beta}}\mathbf{v}_\beta + \mathbf{K_{XZ_\gamma}}\mathbf{v}_\gamma. \tag{38.23}$$

The KL regulariser becomes

$$\begin{aligned}\mathrm{KL}[q(\mathbf{u})||p(\mathbf{u}|\mathbf{Z})] = &\frac{1}{2}{\mathbf{v}_\beta}^\top[\mathbf{K_{Z_\beta Z_\beta}} - \mathbf{K_{Z_\beta Z_\gamma}}\mathbf{K}^{-1}_{\mathbf{Z_\gamma Z_\gamma}}\mathbf{K_{Z_\gamma Z_\beta}}]\mathbf{v}_\beta + \frac{1}{2}{\mathbf{v}_\gamma}^\top\mathbf{K_{Z_\gamma Z_\gamma}}\mathbf{v}_\gamma \\ &+\frac{1}{2}\mathrm{Tr}[\mathbf{K}^{-1}_{\mathbf{Z_\gamma Z_\gamma}}\mathbf{S}_\gamma - \mathbf{I}] - \frac{1}{2}\log\det(\mathbf{K}^{-1}_{\mathbf{Z_\gamma Z_\gamma}}\mathbf{S}_\gamma).\end{aligned} \tag{38.24}$$

The follow-up work by Shi et al. [1513] further proposed a generalised decoupled SVGP parameterisation, which recovers ODSVGP as a special case. In Table 38.1, we summarise the time and memory complexity for full GP, SVGP, DSVGP, and ODSVGP.

38.4 Interdomain Gaussian processes

Standard SVGP defines inducing variables as the function values evaluated at some inducing locations $\mathbf{Z}$ that live in the same space as the inputs $\mathbf{X}$. Interdomain Gaussian processes [987, 1663] further consider generalised inducing variables defined through transformations of the prior GP. Each inducing variable u_m is obtained by transforming the prior GP f through a linear operator $L_m[\cdot]$:

$$u_m := L_m[f], \quad p(f) = \mathcal{GP}(0, k(\cdot,\cdot)). \tag{38.25}$$

Since the variables obtained by transforming a GP with linear operators also follow a multivariate Gaussian distribution, the prior distribution of $\mathbf{u}$ is a Gaussian as in standard SVGP, enabling the extension of SVGP to leverage interdomain inducing variables.

A particularly popular class of linear transformations is linear projection using a basis function $\phi_m(\mathbf{x})$, which defines each inducing variable as

$$u_m := \int_{\mathcal{X}} f(\mathbf{x})\phi_m(\mathbf{x})d\mathbf{x}, \quad p(f) = \mathcal{GP}(0, k(\cdot,\cdot)). \tag{38.26}$$

Notice that standard SVGP can be recovered as a special case in this framework by considering $\phi_m(\mathbf{x}) = \delta_{\mathbf{z}_m}(\mathbf{x})$, where $\delta_{\mathbf{z}_m}(\cdot)$ is a Dirac-delta function, and therefore $u_m = \int f(\mathbf{x})\delta_{\mathbf{z}_m}(\mathbf{x})d\mathbf{x} = f(\mathbf{z}_m)$.

For general basis functions, the prior cross-covariance between the function values $f(\mathbf{x})$ and the inducing variables, and the prior inducing covariance between the inducing variables themselves are computed as integrals. To maintain the consistency of notations, we denote the prior cross-covariance between $f(\mathbf{x})$ and u_m, and the prior inducing covariance between u_n and u_m as $[\mathbf{K}_{f_\mathbf{x}\mathbf{u}}]_m$ and $[\mathbf{K_{uu}}]_{nm}$, respectively:

$$\begin{aligned}[\mathbf{K}_{f_\mathbf{x}\mathbf{u}}]_m &:= \mathrm{COV}(f(\mathbf{x}), u_m) = \int_{\mathcal{X}} k(\mathbf{x},\mathbf{x}')\phi_m(\mathbf{x}')\mathrm{d}\mathbf{x}', \\ [\mathbf{K_{uu}}]_{nm} &:= \mathrm{COV}(u_n, u_m) = \int_{\mathcal{X}}\int_{\mathcal{X}} k(\mathbf{x},\mathbf{x}')\phi_n(\mathbf{x})\phi_m(\mathbf{x}')\mathrm{d}\mathbf{x}\mathrm{d}\mathbf{x}'.\end{aligned} \tag{38.27}$$

Typically, the integration domain $\mathcal{X}$ is bounded to avoid exploding prior variances [740, 987].

Since we still impose the same structure in the approximate posterior as in standard SVGP (i.e., freely optimised variational distribution for the inducing variables and prior conditional matching), the form of posterior predictive distribution and the ELBO training objective for interdomain GPs remain exactly the same as the standard SVGP shown in Section 38.2. The only difference is that we replace the covariance matrices $\mathbf{K_{XZ}}$ and $\mathbf{K_{ZZ}}$ in standard SVGP with $\mathbf{K_{fu}}$ and $\mathbf{K_{uu}}$, respectively. The main technical contribution of most interdomain GP methods [476, 740, 987] is on finding basis functions that modulate the behaviour of GP approximations, as well as tractable computation of the integration-based covariance matrices $\mathbf{K_{fu}}$ and $\mathbf{K_{uu}}$.

Compared with standard SVGP which requires (iterative) optimisation of the inducing inputs $\mathbf{Z}$, interdomain GP relies on fixed basis functions for $\mathbf{K_{fu}}$ and $\mathbf{K_{uu}}$ computations. Therefore, interdomain GPs often incur less computational cost than standard SVGP, especially when the integrals are analytically tractable or can be computed efficiently [278]. Furthermore, with some combinations of kernels and basis functions, the prior inducing covariance $\mathbf{K_{uu}}$ may enjoy certain sparse structures to enable fast matrix inversion [223, 476, 740]. Notably, Dutordoir et al. [476] constructed a diagonal $\mathbf{K_{uu}}$ based on zonal kernels and spherical harmonics basis functions, so that its inversion can be computed in only $O(M)$ time.

38.5 Online sparse Gaussian processes

In these online or continual learning [250, 897, 1236, 1401] scenarios, we would like to update the model only based on the new batch of data (or the new task) without revisiting the past data, since storing all the past data and retraining the model on them become more and more expensive as more and more tasks are observed. However, the model typically suffers from catastrophic forgetting and cannot maintain consistent performance over the past data if we merely adapt it to the new task. Online or continual learning methods [347, 458, 1236] are proposed to mitigate the catastrophic forgetting issue so that the model can both adapt to the new task and also memorise the past tasks, without expensive retraining on (all the) past data.

Sequential Bayesian inference provides a principled framework for online learning where the old posterior is used as the prior knowledge for the new task. By adapting it to the new data through incorporating the corresponding new likelihood, we can then obtain the new posterior representing our updated knowledge. Approximation techniques, such as sequential variational inference (reviewed in Chapter 18), are required when exact sequential posterior inference is intractable.

For online learning with SVGPs, we sequentially update the inducing points and its variational parameters as new data arrive in the framework of sequential variational inference. Given the SVGP trained on the first task $\mathcal{D}_{t_1}$, $q_{t_1}(f) = \int p_{t_1}(f|\mathbf{u}_{t_1})q_{t_1}(\mathbf{u}_{t_1})d\mathbf{u}_{t_1}$ based on $(\mathbf{Z}_{t_1}, \mathbf{u}_{t_1})$, when the second task is observed, online SVGP [OSVGP; 218] obtain the new SVGP, $q_{t_2}(f) = \int p_{t_2}(f|\mathbf{u}_{t_2})q_{t_2}(\mathbf{u}_{t_2})d\mathbf{u}_{t_2}$, by updating the inducing points to $(\mathbf{Z}_{t_2}, \mathbf{u}_{t_2})$ as well as fitting the approximate posterior $q_{t_2}(\mathbf{u}_{t_2})$. Notice that the prior for the new SVGP can be updated as well (i.e., the kernel hyperparameters can be updated), so $p_{t_2}(f)$ can be different from $p_{t_1}(f)$. Ideally, in the case where revisiting the past data is allowed, we would like to minimise the full-batch KL-divergence $\mathrm{KL}\left(q_{t_2}(f)||p_{t_2}(f|\mathcal{D}_{t_1}, \mathcal{D}_{t_2})\right)$, which is equivalent to maximise the following ELBO:

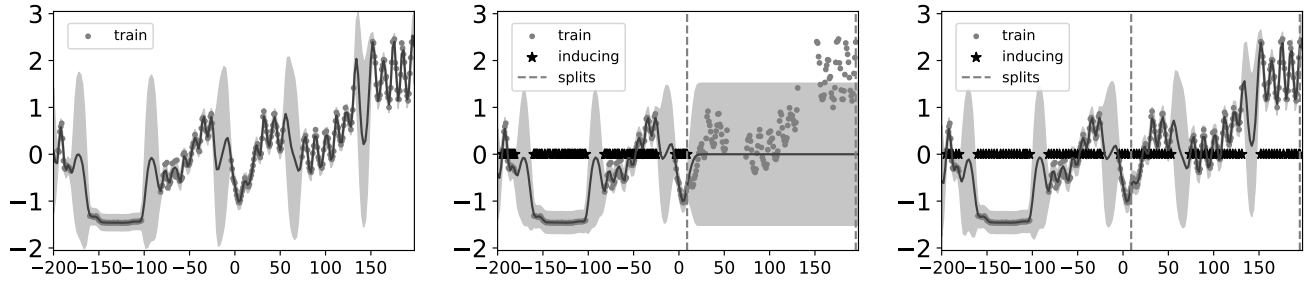
$$\begin{aligned}\mathcal{L}_{ELBO} &= \log \frac{p_{t_2}(\mathcal{D}_{t_1}, \mathcal{D}_{t_2})}{p_{t_1}(\mathcal{D}_{t_1})} - \mathrm{KL}\left(q_{t_2}(f)||p_{t_2}(f|\mathcal{D}_{t_1}, \mathcal{D}_{t_2})\right) \\ &= \int q_{t_2}(f) \log \frac{p_{t_2}(\mathbf{y}_{t_2}, \mathbf{y}_{t_1})}{p_{t_1}(\mathbf{y}_{t_1})q_{t_2}(f)} p_{t_2}(f|\mathbf{y}_{t_1}, \mathbf{y}_{t_2}) df \\ &= \int q_{t_2}(f) \log \frac{p_{t_2}(f)p_{t_2}(\mathbf{y}_{t_2}|f)p_{t_2}(\mathbf{y}_{t_1}|f)}{p_{t_1}(\mathbf{y}_{t_1})q_{t_2}(f)} df.\end{aligned} \tag{38.28}$$

Notice that $p_{t_1}(\mathcal{D}_{t_1})$ in the denominator of the first term is a constant since it only depends on the old kernel hyperparameters. Unfortunately, in an online setup, since we are not allowed to revisit the old data with the new model, $p_{t_2}(\mathbf{y}_{t_1}|f)$ is intractable and therefore the above ELBO is intractable. To get around this intractability, Bui et al. [218] proposed to approximate this term with

$$p_{t_2}(\mathbf{y}_{t_1}|f) \approx p_{t_1}(\mathbf{y}_{t_1}|f) = \frac{p_{t_1}(f|\mathbf{y}_{t_1})p_{t_1}(\mathbf{y}_{t_1})}{p_{t_1}(f)} \approx \frac{q_{t_1}(f)p_{t_1}(\mathbf{y}_{t_1})}{p_{t_1}(f)}. \tag{38.29}$$

Based on this approximation, the above ELBO in Eq. 38.28 can be approximated with the following "online ELBO":

$$\mathcal{L}^o_{ELBO} = \int q_{t_2}(f) \log \frac{p_{t_2}(f)p_{t_2}(\mathbf{y}_{t_2}|f)q_{t_1}(f)}{q_{t_2}(f)p_{t_1}(f)} df. \tag{38.30}$$



(a) GP (b) OSVGP after Task 1 (c) OSVGP after Task 2

Figure 38.2: OSVGP with 100 inducing points on an online time series regression problem consisting of two tasks. The dash lines in panels (b) and (c) indicate the split of data into two tasks.

Since $q_{t_1}(f)$ and $q_{t_2}(f)$ are SVGPs fully specified by inducing points $\mathbf{u}_{t_1}$ and $\mathbf{u}_{t_2}$, respectively, the objective can be further simplified to:

$$\begin{aligned}\mathcal{L}_{ELBO}^{o} =& \mathbb{E}_{q_{t_2}(\mathbf{f}_{t_2})}\left[\log p_{t_2}(\mathbf{y}_{t_2} \mid \mathbf{f}_{t_2})\right] + \mathrm{KL}\left(\tilde{q}_{t_2}(\mathbf{u}_{t_1}) \,\|\, p_{t_1}(\mathbf{u}_{t_1})\right) \\ & - \mathrm{KL}\left(\tilde{q}_{t_2}(\mathbf{u}_{t_1}) \,\|\, q_{t_1}(\mathbf{u}_{t_1})\right) - \mathrm{KL}\left(q_{t_2}(\mathbf{u}_{t_2}) \,\|\, p_{t_2}(\mathbf{u}_{t_2})\right),\end{aligned} \tag{38.31}$$

where $\tilde{q}_{t_2}(\mathbf{u}_{t_1}) := \int p_{t_2}(\mathbf{u}_{t_1}|\mathbf{u}_{t_2}) q_{t_2}(\mathbf{u}_{t_2}) \mathrm{d}\mathbf{u}_{t_2}$.

The time complexity of online SVGP is $O(BM^2 + M^3)$ per step, where B is the size of the streaming data batch (analogous to the batch size N for SVGP). Figure 38.2 shows the fit of OSVGP for online regression on the Solar Irradiance dataset [992]. The time series is split into two tasks and OSVGP learns these two tasks online. When learning the second task, even without retraining on the first task, OSVGP still maintains reliable predictive performance in this historical region and also adapts to the new task. The final prediction is similar to a full GP model trained jointly over all the data, illustrating OSVGP's efficiency.

Other approximate inference techniques, such as Laplace approximation [1118], expectation propagation [218, 347], and variational approximation enhanced with replay buffer, have also been proposed to bypass the intractability of exact online posterior inference [257].

Interdomain GPs can also be integrated into the online SVGP framework [278] and since the interdomain inducing variables are constructed without training the inducing locations, they can further reduce the computational cost. However, previous interdomain GPs typically construct inducing variables via integration based on a predefined measure (e.g., a uniform measure over a fixed interval [740] or a fixed Gaussian measure [987]) to prevent exploding prior variances. This requires the practitioner to pre-define an integration measure that covers all regions where the future tasks live, which is impractical for real-world online learning problems. Chen et al. [278] proposed online HiPPO-SVGP (OHSVGP), which bypasses this limitation by adopting adaptive basis functions in the HiPPO (High-order Polynomial Projection Operators) framework [678] (a renovated RNN architecture). These adaptive basis functions are constructed based on a time-dependent measure which keeps extending to the new input region as more tasks arrive, making them suitable for online inference. As a result, the prior covariance matrices, $\mathbf{K_{fu}}$ and $\mathbf{K_{uu}}$, are now adaptive as well, and they can be efficiently updated in a recurrence form when a new task arrives, similar to the evolution of the recurrent hidden state in the HiPPO framework.

38.6 Summary

This chapter reviews scalable GP methods. We have discussed scalable low-rank approximations of kernel matrix enabling faster computation of matrix inversion and determinant evaluation in full GP. Then we reviewed SVGPs based on variational inducing points, arguably the most representative technique for scaling up GP posterior inference. We also introduced decoupled SVGPs and interdomain GPs as extensions of standard SVGPs for better scalability and approximation quality. Lastly, we discussed online SVGP methods that enable scalable online learning with GP models, which further reduce the computational cost by avoiding retraining over historical data. This chapter is by no means exhaustive, but most methods included here are applicable to general setups, e.g., regression, classification, and latent variable models (e.g., Titsias and Lawrence [GP-LVM; 1616]). While we have focused on variational approaches to sparse GP inference, MCMC-based alternatives also exist. These methods place priors over the inducing variables or kernel hyperparameters, and sample from their posterior, and can offer improved uncertainty quantification at additional computational cost [739, 963, 1425, 1647]. Further improvements

of scalable GPs based on task-specific requirements are possible. For example, Markovian GPs [1460, Chapter 12] exploit their state-space SDE representation to reduce the time complexity to $\mathcal{O}(N)$, but these approaches are restricted to 1D input and Markovian kernels.

Chapter 39

Last-layer inference

In the previous chapter, we saw that Gaussian processes scale cubically with the number of data points, and one way to lower this cost is to reduce the effective number of data points via inducing points. An analogous strategy can be applied to Bayesian neural networks, where the computational bottleneck lies in the number of parameters. Instead of computing a posterior over all network weights, one can compute a posterior over the last-layer weights only and treat the remaining parameters as deterministic. This approach is called *last-layer inference* and is the topic of this chapter.

39.1 Motivation

Many drawbacks of deterministic neural networks can be addressed by treating all network weights as random variables and performing posterior estimation using Bayesian inference. In classification, Bayesian neural networks mitigate overconfidence and improve calibration of predictive class probabilities, whereas in regression they provide principled predictive densities that are not available for deterministic neural networks. Yet, such fully Bayesian neural networks are rarely used in practice. This is because they do not scale well to the large datasets and highly parameterised neural networks required for real-world applications. In the simplest fully Bayesian neural network, one not only has to learn the weights but also the corresponding posterior variances. Thus, one has to learn twice as many parameters as in a deterministic neural network. Ad-hoc approximate methods, such as Monte Carlo (MC) dropout, are a practical solution to make computation feasible, but require several forward passes during prediction and make it difficult to encode prior assumptions in a principled manner.

In some applications, practitioners are willing to trade off accurate inference (in the form of fully Bayesian neural networks) against computational speed in training and prediction. To that end, it is often sufficient to only perform Bayesian inference over a *subset* of layers or weights. A popular choice in that regard is to infer the posterior over the last-layer weights $w \in \mathbb{R}^{C \cdot H_{L-1}}$ and use point estimates for the remaining weights $\theta \in \mathbb{R}^{P - C \cdot H_{L-1}}$. This is called *last-layer inference*.

Assume we wish to predict a target $y_* \in \mathbb{R}^C$ from an input $x_* \in \mathbb{R}^D$ using a neural network $f_\vartheta : \mathbb{R}^D \to \mathbb{R}^C$ with parameters $\vartheta = (\theta^\top, w^\top)^\top \in \mathbb{R}^P$ trained on observed data $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$.

In last-layer inference, one approximates the posterior predictive distribution via

$$p(y_* \mid x_*, \mathcal{D}) \approx p(y_* \mid x_*, \mathcal{D}, \theta) = \int p(y_* \mid x_*, \mathcal{D}, \theta, w) p(w \mid \mathcal{D}, \theta) \mathrm{d}w. \tag{39.1}$$

This approach offers two primary benefits. Computationally, it reduces the dimensionality of the posterior inference task, which speeds up training and prediction compared to a fully Bayesian neural network. Statistically, it can be interpreted as performing Bayesian linear regression with learned deep feature projections as covariates. This is a setting where closed-form posterior solutions with conjugate priors exist, and efficient algorithms for various other prior choices are well established. However, it is important to note that the quality of uncertainty estimates produced by last-layer inference may, in some applications, depend on the representational capacity of the deep feature projections, and ignoring non-determinism from earlier layers may lead to different uncertainty estimates. We illustrate the main concept of last-layer inference in Fig. 39.1.

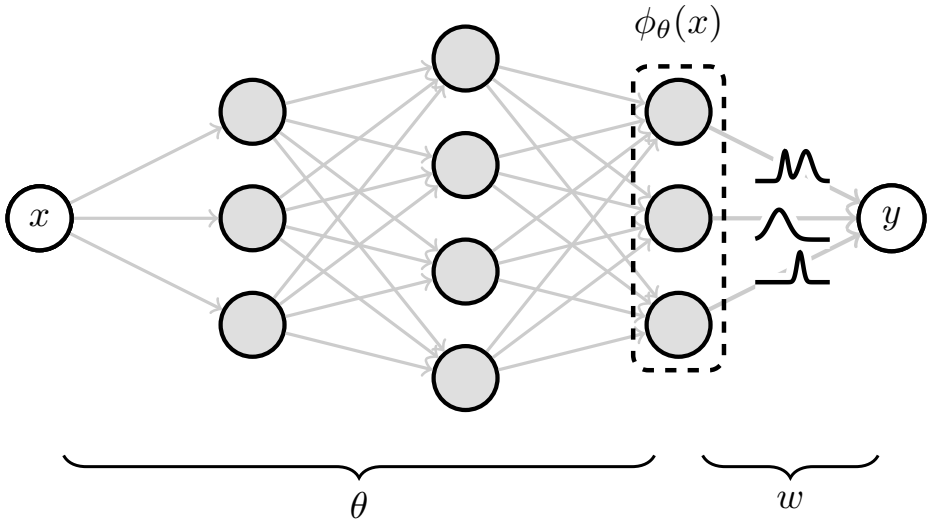


Figure 39.1: In last-layer inference, the parameters $\theta \in \mathbb{R}^{P-C \cdot H_{L-1}}$ up to and including the penultimate layer are treated as deterministic. Only the posterior $p(w \mid \mathcal{D})$ for the last-layer weights is computed using the deep feature projection ϕ_θ. Figure inspired by Blundell et al. [174].

39.2 Last-layer inference as a Bayesian linear model

Last-layer inference can be seen as a Bayesian linear regression model on deep feature projections of the inputs x. For simplicity, we focus on regression for scalar targets, such that $C = 1$ and $w \in \mathbb{R}^{H_{L-1}}$. To perform last-layer inference for classification, we refer to Chapter 10, which treats Laplace approximations for classification, to Gelman et al. [601] for an overview of Bayesian models for classification, as well as to Harrison et al. [709] for an example of last-layer inference based on a Dirichlet distribution.

Let $\phi_\theta : \mathbb{R}^D \to \mathbb{R}^{H_{L-1}}$ denote the sub-network of f up to the penultimate layer, which maps an input x to a deep feature projection $\phi_\theta(x)$. The network ϕ_θ is called a deep feature projector because it maps the input features to a lower-dimensional space, since often $H_{L-1} \ll D$. As shorthand notation, the deep feature projection at a specific training input x_n for $n = 1, \ldots, N$ is denoted as $\phi_n \coloneqq \phi_\theta(x_n)$.

The output of the neural network using a fully connected last layer with a linear activation can be formulated as a linear regression model with covariates ϕ_θ. For an arbitrary input x the neural network output can be written as

$$f_\vartheta(x) \coloneqq w^\top \phi_\theta(x), \tag{39.2}$$

where a constant is appended to $\phi_\theta(x)$ to account for the bias term.

While θ are assumed to be deterministic, w are interpreted as random variables with prior distribution $p(w)$. We are interested in inferring the respective posterior distribution $p(w \mid \mathcal{D}, \theta)$ after observing training data $\mathcal{D}$. The latter can be obtained by applying Bayes' rule

$$p(w \mid \mathcal{D}, \theta) = \frac{p(\mathcal{D} \mid \theta, w) p(w)}{p(\mathcal{D} \mid \theta)}, \tag{39.3}$$

where $p(\mathcal{D} \mid \theta, w)$ defines the likelihood and $p(\mathcal{D} \mid \theta)$ is the marginal likelihood or evidence. In Eq. (39.1), we assume that θ are deterministic when computing the posterior predictive distribution, that is, $p(w \mid \mathcal{D}) = p(w \mid \mathcal{D}, \theta)$. Therefore, we drop θ in the following and write the posterior as $p(w \mid \mathcal{D})$ and the evidence as $p(\mathcal{D})$ instead.

To perform Bayesian inference in the model (39.2), it is necessary to specify a likelihood, a prior on w, and to apply Bayes' rule to obtain the posterior distribution. We first discuss how to extend Eq. (39.2) to obtain a likelihood, give an overview of suitable priors w, and derive the resulting posterior distributions.

39.2.1 Likelihood

The likelihood in the canonical Bayesian linear regression model is a noisy version of expression (39.2), that is,

$$y_n = w^\top \phi_n + \varepsilon_n, \tag{39.4}$$

where $\varepsilon_n \sim \mathcal{N}_1(0, \sigma_\varepsilon^2)$ are i.i.d. Gaussian errors, and $\mathcal{N}_1(m, s)$ denotes a one-dimensional Gaussian distribution with mean m and variance s. More complex noise distributions are possible (see Sec. 39.5.1), but for simplicity only the homoscedastic case is considered throughout this chapter. In matrix form, the likelihood over the entire training set is

$$p(\mathcal{D} \mid w, \sigma_\varepsilon^2) \equiv p(Y \mid X, w, \sigma_\varepsilon^2) = \mathcal{N}\left(Y; \Phi w, \sigma_\varepsilon^2 I_N\right) = \prod_{n=1}^{N} \mathcal{N}\left(y_n; w^\top \phi_n, \sigma_\varepsilon^2\right), \tag{39.5}$$

where $\mathcal{N}(x; m, S)$ denotes the density of a Gaussian distribution with mean m and covariance S, the matrix I_N is the N-dimensional identity matrix, $\Phi = \left[\phi_1^\top, \ldots, \phi_N^\top\right] \in \mathbb{R}^{N \times H_{L-1}}$ are the vertically stacked row vectors of the deep feature projections at the training inputs $X = \left[x_1^\top, \ldots, x_N^\top\right]^\top \in \mathbb{R}^{D \times N}$, the vector $Y = (y_1, \ldots, y_N)^\top \in \mathbb{R}^N$ contains the training targets. To perform Bayesian inference in the model (39.4), we have to choose a prior for the last-layer weights w and the variance of the observational noise σ_ε^2.

39.2.2 Prior distributions

We focus here on the most common priors for w, how to use them to induce shrinkage and regularisation, and we present a default prior for σ_ε^2. Further details on priors in general can be found in Part VI (see in particular Chapters 28 and 29).

Prior for the last-layer weights w

We assume a Gaussian prior for the last-layer weights, that is,

$$w \sim \mathcal{N}(\mu_0, \Sigma_0) \tag{39.6}$$

with mean $\mu_0 \in \mathbb{R}^{H_{L-1}}$ and covariance $\Sigma_0 \in \mathbb{R}^{H_{L-1} \times H_{L-1}}$. The Gaussian prior is conjugate to the likelihood in Eq. (39.5), i.e., the posterior is of the same distributional family as the prior distribution, which is attractive from a computational point of view. In the absence of problem-specific prior knowledge, the mean μ_0 is commonly set to zero. The covariance can be assigned a hyperprior that induces shrinkage to reduce overfitting and separate signal from noise, and we now introduce two such designs.

The most common shrinkage prior for Σ_0 is a ridge prior of the form

$$\Sigma_0 = \tau^2 I, \quad \text{where} \quad \tau^2 \sim \mathcal{IG}(a_\tau, b_\tau), \tag{39.7}$$

where $\tau^2 > 0$ denotes a scalar variance parameter, $\mathcal{IG}$ denotes the inverse gamma distribution with shape $a_\tau > 0$ and scale $b_\tau > 0$. Appropriate choices for the parameters a_τ and b_τ of the hyperprior can be found in Gelman [596] and George and McCulloch [607].

The inverse gamma prior for τ^2 is conjugate, and encompasses weakly informative priors on the log-scale ($a_\tau = \epsilon, b_\tau = \epsilon$, $\epsilon \to 0$), as well as flat priors for the variance ($a_\tau = -1, b_\tau = 0$) and standard deviation ($a_\tau = -1/2, b_\tau = 0$) as degenerate cases.

However, there has been considerable debate about the suitability of the inverse gamma distribution due to overfitting [1516]. Thus, alternative choices such as half-Normal, half-Cauchy [1336] or scale-dependent priors [901] have been proposed. Fig. 39.2 depicts the densities of the different hyperpriors for the variance parameter.

A further popular prior choice that induces more finely controlled sparsity is the horseshoe prior [243]. The horseshoe prior introduces independent Gaussian priors for each last-layer weight

$$w_l \sim \mathcal{N}(0, \tau_l^2), \quad \text{for} \quad l = 1, \ldots, H_{L-1} \tag{39.8}$$

and a global-local shrinkage structure

$$\tau_l \mid \xi \sim \mathcal{HC}(0, \xi), \quad \text{with} \quad \xi \sim \mathcal{HC}(0, b_\xi), \tag{39.9}$$

where $\mathcal{HC}$ denotes the half-Cauchy distribution and $b_\xi > 0$ is a fixed hyperparameter. Consequently, the covariance matrix is diagonal $\Sigma_0 = \mathrm{diag}(\tau_1^2, \ldots, \tau_{H_{L-1}}^2)$, and the diagonal elements τ_l^2 allow for varying amounts of regularisation across the weight dimension. Choosing half-Cauchy distributions results in strong shrinkage, such that this choice is appropriate when, effectively, most of the weights should be shrunk towards zero [1123]. The heavy tails of the half-Cauchy distribution allow sufficiently large weights to remain un-shrunk [243]. This can, for instance, be helpful when the last layer is heavily overparameterised.

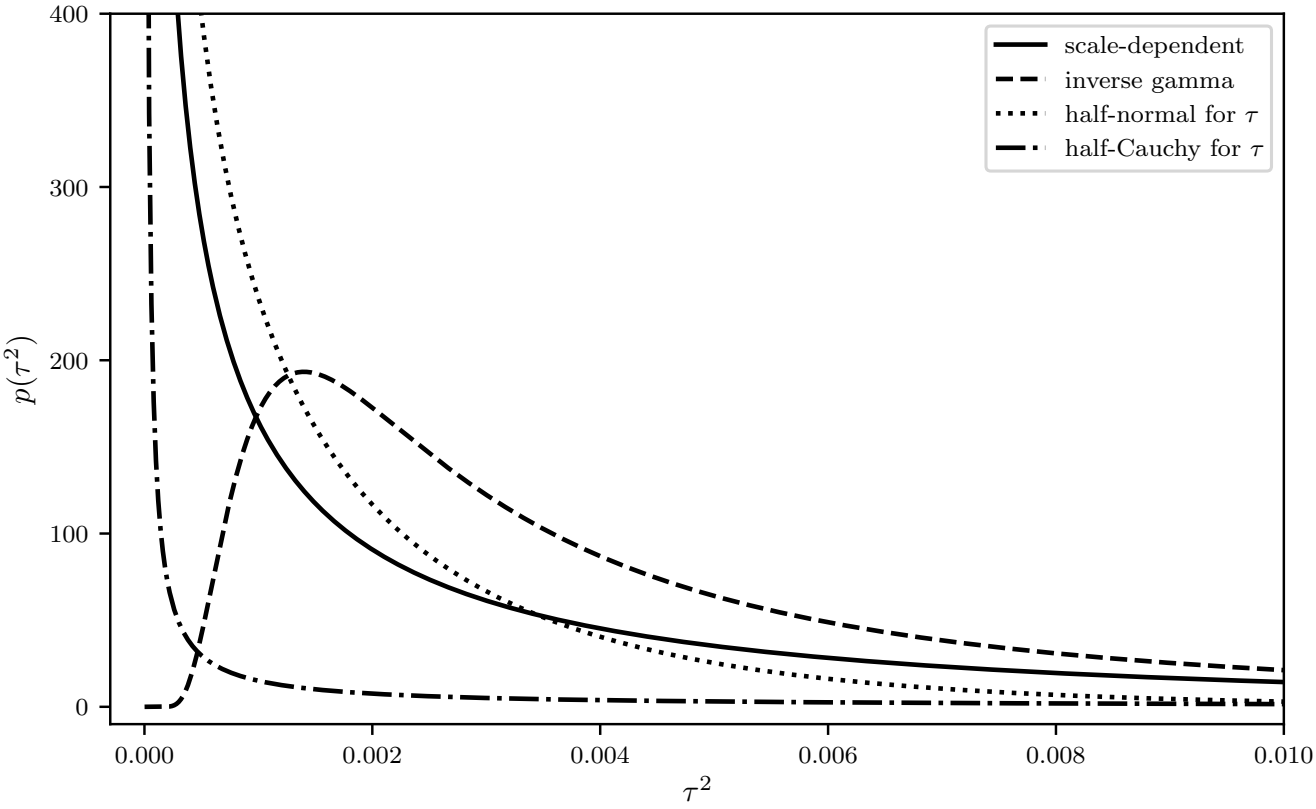


Figure 39.2: Densities for the hyperpriors on the variance parameter τ^2. The half-Cauchy prior on τ corresponds to a generalised beta prime prior on τ^2, and the half-normal prior for τ corresponds to a gamma prior for τ^2. The figure is adapted from Klein and Kneib [901].

Prior for the error noise σ_ε^2

In the standard Bayesian linear regression model, the variance of the error noise is assumed to have an inverse gamma prior

$$\sigma_\varepsilon^2 \sim \mathcal{IG}(a_\sigma, b_\sigma), \tag{39.10}$$

with $a_\sigma, b_\sigma > 0$ being the shape and scale parameters. Similar to the prior for the weights, this choice is conjugate and thus yields a closed-form (conditional) posterior distribution. It also ensures positivity of the variance due to the positive support of the inverse gamma distribution.

39.2.3 Posterior distributions

By Bayes' rule, the joint posterior $p(w, \sigma_\varepsilon^2, \Sigma_0 \mid \mathcal{D})$ is proportional to

$$p(w, \sigma_\varepsilon^2, \Sigma_0 \mid \mathcal{D}) \propto p(\mathcal{D} \mid w, \sigma_\varepsilon^2)\, p(w \mid \Sigma_0)\, p(\Sigma_0)\, p(\sigma_\varepsilon^2). \tag{39.11}$$

The $\propto$ symbol indicates proportionality up to factors not depending on the model parameters $w, \sigma_\varepsilon^2, \Sigma_0$. A key observation is that Bayesian inference usually builds upon the last equation such that the intractable evidence $p(\mathcal{D})$ is neglected in further computations. We now derive the conditional posteriors, which are, for example, used for the Gibbs sampler in Sec. 39.3.

Conditional posterior of w

The conditional posterior of the last-layer weights is

$$\begin{aligned} p(w \mid \mathcal{D}, \sigma_\varepsilon^2, \Sigma_0) &\propto p(\mathcal{D} \mid w, \sigma_\varepsilon^2)\, p(w \mid \Sigma_0) && (39.12)\\ &= \mathcal{N}\left(Y; \Phi w, \sigma_\varepsilon^2 I\right) \mathcal{N}\left(w; \mu_0, \Sigma_0\right) && (39.13)\\ &= \mathcal{N}\left(w; \mu_N, \Sigma_N\right) && (39.14) \end{aligned}$$

with posterior mean μ_N and covariance Σ_N given by

$$\mu_N = \Sigma_N(\Sigma_0^{-1}\mu_0 + \sigma_\varepsilon^{-2}\Phi^\top Y) \text{ and } \Sigma_N = (\Sigma_0^{-1} + \sigma_\varepsilon^{-2}\Phi^\top\Phi)^{-1}. \tag{39.15}$$

Conditional posterior of σ_ε^2

The conditional posterior of the observational noise is

$$p(\sigma_\varepsilon^2 \mid \mathcal{D}, w) \propto p(\mathcal{D} \mid w, \sigma_\varepsilon^2)\, p(\sigma_\varepsilon^2). \tag{39.16}$$

With the conditionally conjugate inverse gamma prior from Eq. (39.10), the conditional posterior is also inverse gamma distributed

$$\begin{aligned} p(\sigma_\varepsilon^2 \mid \mathcal{D}, w) &\propto \mathcal{N}\left(Y; \Phi w, \sigma_\varepsilon^2 I\right) \cdot \mathcal{IG}(\sigma_\varepsilon^2; a_\sigma, b_\sigma) && (39.17)\\ &\propto \mathcal{IG}(\sigma_\varepsilon^2; a_\sigma + \frac{N}{2}, b_\sigma + \frac{1}{2}\|Y - \Phi w\|^2), && (39.18) \end{aligned}$$

where $\mathcal{IG}(x; a, b)$ denotes the density of the inverse gamma distribution with shape a and scale b evaluated at x.

Conditional posterior of Σ_0

The conditional posterior for the covariance Σ_0 of the last-layer weight distribution is

$$p(\Sigma_0 \mid \mathcal{D}, w, \sigma_\varepsilon^2) \propto p(w \mid \Sigma_0)\, p(\Sigma_0), \tag{39.19}$$

using the conditional independence assumption $\Sigma_0 \perp (\mathcal{D}, \sigma_\varepsilon^2) \mid w$. With the ridge prior and setting $\mu_0 = 0$, this conditional posterior is the posterior of τ^2, which is also inverse gamma distributed

$$\begin{aligned} p(\tau^2 \mid \mathcal{D}, w, \sigma_\varepsilon^2) &\propto \mathcal{N}\left(w; 0, \tau^2 I_{H_{L-1}}\right) \mathcal{IG}(\tau^2; a_\tau, b_\tau) && (39.20)\\ &\propto \mathcal{IG}(\tau^2; a_\tau + \frac{H_{L-1}}{2}, b_\tau + \frac{1}{2}\|w\|^2). && (39.21) \end{aligned}$$

Let $\mathcal{HC}(x; 0, b)$ denote the density of the half-Cauchy distribution with location 0 and scale b evaluated at x. Then, with the horseshoe prior the conditional posterior of $\tau_1^2, \ldots, \tau_{H_{L-1}}^2, \xi$ is

$$p(\tau_1^2, \ldots, \tau_{H_{L-1}}^2, \xi \mid \mathcal{D}, w, \sigma_\varepsilon^2) \propto \prod_{l=1}^{H_{L-1}} \mathcal{N}\left(w_l; 0, \tau_l^2\right) \mathcal{HC}(\tau_l; 0, \xi)\, \mathcal{HC}(\xi; 0, b_\xi), \tag{39.22}$$

which does not correspond to a closed-form distribution.

Note that, except for the horseshoe prior, we can express all conditional posteriors in closed form. Thus, we can sample from the conditional posteriors, and we will see in the next section that this allows us to efficiently obtain exact samples of the posterior using a Gibbs sampler.

39.3 Posterior estimation and prediction

To estimate the posterior distribution $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$, we briefly review Markov chain Monte Carlo (MCMC) simulation as an exact sampling method and variational inference (VI) as an approximate, optimisation-based inference method that better scales to large datasets and highly parameterised models. A third alternative, which is described in detail in Chapter 40, is the Laplace approximation, which uses a local Gaussian approximation. We then describe how to conduct two-step and joint estimation of the posterior $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$ and deterministic neural network weights θ, and how to perform predictive inference for new inputs x_*.

39.3.1 Markov chain Monte Carlo

MCMC produces asymptotically exact samples for posteriors that are known up to a normalising constant by constructing a Markov chain whose stationary distribution is the posterior. The chain is initialised and run until convergence. Usually, draws from the pre-convergence phase (burn-in period) are discarded, and the remaining draws are used as exact samples from the posterior distribution. In principle, any MCMC method from Part I can be used for last-layer inference.

We can distinguish between standard MCMC methods and those that use stochastic gradients (SG-MCMC). The reason for separating these classes is that, with SG-MCMC, we can perform *full training*, that is, joint estimation of $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$ and θ (compare Sec. 39.3.3), whereas standard MCMC only allows to fit the posterior after the network parameters θ have been learned.

Standard MCMC

To estimate $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$, the training data is first passed through the deep feature projector. Then, an MCMC sampler is run with the feature projections as covariates. Appropriate samplers are chosen based on the specified model and the dimension of the last layer. For example, in the simple, conjugate case with a Gaussian prior on the weights $w \sim \mathcal{N}(0, \tau^2 I_{H_{L-1}})$, an inverse gamma prior on the error noise $\sigma_\varepsilon^2 \sim \mathcal{IG}(a_\sigma, b_\sigma)$, and an inverse gamma hyperprior on the variance parameter $\tau^2 \sim \mathcal{IG}(a_\tau, b_\tau)$, a Gibbs sampler can be used to sample from the joint posterior $p(w, \tau^2, \sigma_\varepsilon^2 \mid \mathcal{D})$. This sampler is fast and computationally efficient and summarised in Alg. 31.

Algorithm 31: Gibbs sampler

Input : Training data $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$
Output: Posterior samples of w, τ^2, σ_ε^2
for $s = 1, \ldots, S$ **do**
  Draw $w \mid \mathcal{D}, \tau^2, \sigma_\varepsilon^2 \sim \mathcal{N}\left(\mu_N, \Sigma_N\right)$
  Draw $\tau^2 \mid \mathcal{D}, w, \sigma_\varepsilon^2 \sim \mathcal{IG}\left(a_\tau + H_{L-1}/2, b_\tau + \frac{1}{2}\|w\|^2\right)$
  Draw $\sigma_\varepsilon^2 \mid \mathcal{D}, \tau^2, w \sim \mathcal{IG}\left(a_\sigma + N/2, b_\sigma + \frac{1}{2}\|Y - \Phi w\|^2\right)$
return $\{w^{[s]}, (\tau^2)^{[s]}, (\sigma_\varepsilon^2)^{[s]}\}_{s=S_{burn\text{-}in}+1}^S$

A general drawback of standard MCMC methods for last-layer inference is that they use the full training set in each iteration, which can become expensive for large datasets. An alternative is to employ mini-batches, which can, for example, be combined with Hamiltonian Monte Carlo (HMC) methods [315].

SG-MCMC

SG-MCMC naturally accommodates mini-batch training, making it well-suited to large datasets. It is attractive for last-layer inference as it is conceptually simple and requires only a small, computationally inexpensive modification to the parameter updates used to train deterministic neural networks. The general idea of SG-MCMC is to add noise to stochastic gradient iterates and gradually decrease the step size during training towards zero, a process known as *annealing*. Asymptotically, the iterates will converge to samples of the posterior. A general approach for converting standard MCMC samplers into SG-MCMC is shown in Ma et al. [1097], and Chapters 2 and 5 provide further details.

A popular example of SG-MCMC are the stochastic gradient Langevin dynamics [SGLD; 1737], which updates the last-layer weights according to

$$w^{[t+1]} = w^{[t]} + \frac{\eta^{[t]}}{2}\Big(\nabla_w \log p(w^{[t]}) + \frac{N}{|B|}\sum_{n\in B} \nabla_w \log p(y_n \mid w^{[t]}, x_n)\Big) + \nu^{[t]}, \tag{39.23}$$

where t denotes the iteration, $\eta^{[t]}$ is a sequence of learning rates or so-called *step sizes*, $\nu^{[t]} \sim \mathcal{N}(0, \eta^{[t]} I_{H_{L-1}})$ is injected noise, and $B \subset \{1, \dots, N\}$ denotes the indices of a mini-batch.

The key idea of SGLD is that for relatively large values of $\eta^{[t]}$, Eq. (39.23) performs standard stochastic gradient updates, but as $\eta^{[t]} \to 0$, the updates approach Langevin dynamics and yield samples of the posterior distribution. The sequence of step sizes typically needs to satisfy $\sum_{t=1}^{\infty} \eta^{[t]} = \infty$ and $\sum_{t=1}^{\infty} (\eta^{[t]})^2 < \infty$. The first condition allows high-probability regions to be reached regardless of the initialisation, while the second controls the rate at which the injected noise vanishes.

39.3.2 Variational inference

In VI, we approximate the intractable posterior $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$ with a tractable variational distribution with density $q_\lambda(w, \Sigma_0, \sigma_\varepsilon^2)$ that is parameterised by so-called variational parameters $\lambda \in \mathbb{R}^K$. We seek to find the parameter values λ^* that minimise a divergence measure between the posterior $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$ and the parametric variational density $q_\lambda(w, \Sigma_0, \sigma_\varepsilon^2)$. Part III provides an in-depth introduction to VI, and we briefly derive an appropriate VI objective for last-layer inference here.

Commonly, VI is based on the reverse Kullback-Leibler (KL) divergence (compare Chapter 8), which leads to the following optimisation problem

$$\lambda^* = \arg\min_{\lambda \in \mathbb{R}^K} \mathrm{D}_{\mathrm{KL}}\big(q_\lambda(w, \Sigma_0, \sigma_\varepsilon^2) \big\| p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})\big). \tag{39.24}$$

Rather than minimising the KL divergence, which is typically intractable, one can equivalently maximise the evidence lower bound (ELBO), given by

$$\begin{aligned} \mathrm{ELBO}(\lambda) &= \mathbb{E}_{q_\lambda(w,\Sigma_0,\sigma_\varepsilon^2)}\big[\log p(\mathcal{D}|w, \Sigma_0, \sigma_\varepsilon^2)\big] + \mathbb{E}_{q_\lambda(w,\Sigma_0,\sigma_\varepsilon^2)}\big[\log p(w, \Sigma_0, \sigma_\varepsilon^2)\big] \\ &\quad - \mathbb{E}_{q_\lambda(w,\Sigma_0,\sigma_\varepsilon^2)}\big[\log q_\lambda(w, \Sigma_0, \sigma_\varepsilon^2)\big] \\ &= \mathbb{E}_{q_\lambda(w,\Sigma_0,\sigma_\varepsilon^2)}\big[\log p(\mathcal{D}|w, \Sigma_0, \sigma_\varepsilon^2)\big] - \mathrm{D}_{\mathrm{KL}}\big(q_\lambda(w, \Sigma_0, \sigma_\varepsilon^2) \big\| p(w, \Sigma_0, \sigma_\varepsilon^2)\big) \end{aligned} \tag{39.25}$$

The ELBO can be maximised using mini-batches and stochastic gradient ascent (SGA), where the variational parameters λ at iteration $t+1$ are updated via

$$\lambda^{[t+1]} = \lambda^{[t]} + \eta^{[t]} \nabla_\lambda \widehat{\mathrm{ELBO}(\lambda^{[t]})}, \tag{39.26}$$

where $\nabla_\lambda \widehat{\mathrm{ELBO}(\lambda^{[t]})}$ denotes a Monte Carlo estimate of the gradient w.r.t. the variational parameters, and $\eta^{[t]}$ is the learning rate, which may be a scalar or vector and potentially adaptive as well.

Choosing the variational distribution

Gaussian variational distributions have become popular and are known to provide generally accurate approximations of posterior means [875]. The simplest choice is the Gaussian distribution with a diagonal covariance matrix [851]. The variance parameters need to be appropriately transformed to ensure positive support. We apply a logarithmic transformation so that the elements of Σ_0 and σ_ε^2 follow a log-normal distribution. The resulting

variational density is

$$q_\lambda(w, \log(\Sigma_0), \log(\sigma_\varepsilon^2)) = \mathcal{N}\left(\begin{bmatrix} w \\ \log(\Sigma_0) \\ \log(\sigma_\varepsilon^2) \end{bmatrix}; \mu_\lambda, \Sigma_\lambda\right),$$

where $\Sigma_\lambda = \text{diag}(\sigma_1^2, \ldots, \sigma_K^2)$ and $\log(\Sigma_0)$ denotes element-wise application of the logarithm. For the ridge prior $\log(\Sigma_0) = \log(\tau^2)$ and for the horseshoe prior $\log(\Sigma_0) = (\log(\tau_1^2), \ldots, \log(\tau_{H_{L-1}}^2))^\top$. For the ridge prior, the variational parameters have dimension $K = H_{L-1} + 2$, and for the horseshoe prior $K = 2H_{L-1} + 1$. In both cases, the computational cost of VI scales linearly in H_{L-1}, i.e. $\mathcal{O}(H_{L-1})$.

While linear scaling is attractive from a computational viewpoint, a diagonal covariance matrix implies posterior independence between the last-layer weights w, the elements of Σ_0, and the noise variance σ_ε^2, an assumption that is often too restrictive. On the other hand, specifying a full covariance matrix Σ_λ leads to quadratic scaling in H_{L-1}, i.e., $\mathcal{O}(H_{L-1}^2)$. As a trade-off, Ong et al. [1261] and others propose factorising the covariance matrix as

$$\Sigma_\lambda = BB^\top + D^2,$$

where $B \in \mathbb{R}^{K \times F}$ is a lower triangular, full rank matrix with $F \ll K$ and $D = \text{diag}(d_1, \ldots, d_K)$ is a diagonal matrix with nonnegative entries, $d_l > 0$, $l = 1, \ldots, K$.

With this variational density, $K = 2(H_{L-1} + 2) + \frac{F(F+1)}{2}$ for the ridge prior and $K = 2(2H_{L-1} + 1) + \frac{F(F+1)}{2}$ for the horseshoe prior. For both priors, VI scales with $\mathcal{O}(H_{L-1}\, F^2)$.

Closed-form loss

Assuming a Gaussian likelihood and a Gaussian prior, an advantage of choosing q_λ to be Gaussian is that $\mathbb{E}_{q_\lambda(w)}\left[\log p(y|x, w, \sigma_\varepsilon^2)\right]$ is of closed form, as shown in the following result by Titsias [1615]. Hence, no Monte Carlo sampling is required to obtain unbiased estimates of the expected log-likelihood during training [709].

Lemma 39.1 (Closed-form expected log-likelihood under Gaussian q_λ)**.** *Let $q_\lambda(w) = \mathcal{N}(w; \mu_w, \Sigma_w)$ with $w \in \mathbb{R}^{H_{L-1}}$, $\mu_w \in \mathbb{R}^{H_{L-1}}$, $\Sigma_w \in \mathbb{R}^{H_{L-1} \times H_{L-1}}$, and $p(y \mid x, w, \sigma_\varepsilon^2) = \mathcal{N}\left(y; w^\top \phi, \sigma_\varepsilon^2\right)$ with $\phi \coloneqq \phi_\theta(x) \in \mathbb{R}^{H_{L-1}}$ and $\sigma_\varepsilon^2 > 0$. Then, it holds*

$$\mathbb{E}_{q_\lambda(w)}\left[\log p(y \mid x, w, \sigma_\varepsilon^2)\right] = \log \mathcal{N}\left(y; \mu_w^\top \phi, \sigma_\varepsilon^2\right) - \frac{1}{2}\sigma_\varepsilon^{-2} \phi^\top \Sigma_w \phi. \tag{39.27}$$

The proof for uni- and multivariate targets ($C \geq 1$) is left as an exercise to the reader. VI is popular in practice because estimation is fast, and it is straightforward to check whether the ELBO has converged. However, a variational approximation can incur a so-called variational gap, which arises if the true posterior is not a member of the variational family, i.e., $\nexists \lambda^* : q_{\lambda^*}(w, \Sigma_0, \sigma_\varepsilon^2) = p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$. In addition, posterior standard deviations are often underestimated [170].

39.3.3 Estimation and prediction

To train $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$ and θ, one can either choose a *two-step* or a *joint* approach as we will discuss next.

Two-step estimation

Alg. 32 summarises the two-step approach where θ is estimated first and used as a plug-in estimate to perform posterior estimation.

Algorithm 32: Two-step estimation

Input : training data $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$
Output: network parameters θ, posterior distribution $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$
Train the network f_ϑ using $\mathcal{D}$ with a standard loss function (e.g. mean squared error)
Discard last-layer weights w and freeze the remaining parameters θ
Forward pass $\{x_n\}_{n=1}^N$ through the network to obtain feature projections $\phi_n \coloneqq \phi_\theta(x_n)$ for $n = 1, \ldots, N$
Estimate $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$ using the exact posterior, MCMC, or VI

The main advantage of the two-step estimation approach is that the cost of training ϕ_θ is completely independent of the posterior approximation scheme, and any previously trained deterministic network can be extended with a last-layer inference using Alg. 32.

Joint estimation

Estimating θ and $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$ jointly takes into account the uncertainty from the last-layer weights when learning θ. For VI, we interpret the ELBO in Eq. (39.25), as a joint function of θ and λ, following a similar approach to that of Harrison et al. [709], such that

$$\mathrm{ELBO}(\lambda, \theta) = \mathbb{E}_{q_\lambda(w, \Sigma_0, \sigma_\varepsilon^2)}\big[\log p(\mathcal{D} \mid \theta, w, \Sigma_0, \sigma_\varepsilon^2)\big] - \mathrm{D}_{\mathrm{KL}}\big(q_\lambda(w, \Sigma_0, \sigma_\varepsilon^2) \big\| p(w, \Sigma_0, \sigma_\varepsilon^2)\big),$$

and jointly maximise over θ, λ.

With SG-MCMC, the last-layer weights are updated according to Eq. (39.23), and the deterministic weights at iteration $t+1$ are updated, for example, with standard SGA

$$\theta^{[t+1]} = \theta^{[t]} + \eta^{[t]} \sum_{n \in B} \nabla_\theta \log p(y_n \mid x_n, \theta^{[t]}, w^{[t]}, \Sigma_0^{[t]}, (\sigma_\varepsilon^2)^{[t]}). \tag{39.28}$$

The main drawback of joint estimation is that fitting θ and $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$ simultaneously is challenging, because changes in θ continually alter the posterior.

39.3.4 Posterior predictive distribution

For a new input x_*, the posterior predictive distribution is given by

$$p(y_* \mid x_*, \mathcal{D}, \theta) = \int p(y_* \mid x_*, \mathcal{D}, \theta, w, \sigma_\varepsilon^2) p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D}, \theta) \mathrm{d}(w, \Sigma_0, \sigma_\varepsilon^2). \tag{39.29}$$

Assume we have point estimates $\hat{\theta}$ of the deterministic neural network weights and a sample $\{(w^{[1]}, \Sigma_0^{[1]}, (\sigma_\varepsilon^2)^{[1]}), \ldots, (w^{[S]}, \Sigma_0^{[S]}$ from the joint posterior of the last-layer weights, prior parameters in Σ_0 and noise variance σ_ε^2. The latter can be samples from the exact posterior (where available), MCMC samples, or samples from the variational approximation $q_{\lambda^*}(w, \Sigma_0, \sigma_\varepsilon^2)$. Then, Eq. (39.29) can be approximated by

$$p(y_* \mid x_*, \mathcal{D}, \theta) \approx \frac{1}{S} \sum_{s=1}^{S} p(y_* \mid x_*, \mathcal{D}, \hat{\theta}, w^{[s]}, (\sigma_\varepsilon^2)^{[s]}) \tag{39.30}$$

and used as an approximation to $p(y_* \mid \mathcal{D}, x_*)$ based on our assumption in Eq. (39.1).

39.3.5 Posterior predictive moments

Later in the applications, we will evaluate the posterior predictive mean

$$\mathbb{E}[Y_* \mid x_*, \mathcal{D}] = \int \mathbb{E}\big[Y_* \mid x_*, w, \sigma_\varepsilon^2\big]\, p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})\, \mathrm{d}(w, \Sigma_0, \sigma_\varepsilon^2), \tag{39.31}$$

and the posterior predictive variance

$$\begin{aligned} \mathbb{V}[Y_* \mid x_*, \mathcal{D}] =& \mathbb{E}_{p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})}\big[\mathbb{V}(Y_* \mid x_*, w, \sigma_\varepsilon^2)\big] \\ &+ \mathbb{V}_{p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})}[\mathbb{E}(Y_* \mid x_*, w, \sigma_\varepsilon^2)], \end{aligned} \tag{39.32}$$

where y_* is the realisation of the random variable Y_* and Σ_0 can be omitted from the conditioning set due to conditional independence $Y_* \perp \Sigma_0 \mid (x_*, w, \sigma_\varepsilon^2)$. In general, it holds that $\mathbb{E}\big[Y_* \mid x_*, w, \sigma_\varepsilon^2\big]$ and $\mathbb{V}\big[Y_* \mid x_*, w, \sigma_\varepsilon^2\big]$ are the first two moments of the conditional distribution of $Y_* \mid x_*, w, \sigma_\varepsilon^2$. In our case, these are the moments of the Gaussian distribution given by $\mathbb{E}\big[Y_* \mid x_*, w, \sigma_\varepsilon^2\big] = w^\top \phi_\theta(x_*)$ and $\mathbb{V}\big[Y_* \mid x_*, w, \sigma_\varepsilon^2\big] = \sigma_\varepsilon^2$. Hence, we obtain

$$\mathbb{E}[Y_* \mid x_*, \mathcal{D}] = \int w^\top \phi_\theta(x_*)\, p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})\, \mathrm{d}(w, \Sigma_0, \sigma_\varepsilon^2), \tag{39.33}$$

and by the law of total variance

$$\mathbb{V}[Y_* \mid x_*, \mathcal{D}] = \int \mathbb{V}\big[Y_* \mid x_*, w, \sigma_\varepsilon^2\big]\, p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})\, \mathrm{d}(w, \Sigma_0, \sigma_\varepsilon^2) \tag{39.34}$$

$$+ \int \big(\mathbb{E}[Y_* \mid x_*, w, \sigma_\varepsilon^2] - \mathbb{E}[Y_* \mid x_*, \mathcal{D}]\big)^2\, p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})\, d(w, \Sigma_0, \sigma_\varepsilon^2). \tag{39.35}$$

The posterior $p(w, \Sigma_0, \sigma_\varepsilon^2 \mid \mathcal{D})$ can be integrated out numerically in practice using S samples from the posterior obtained via MCMC or VI. Therefore, the first integral is approximated using Monte Carlo integration by $\frac{1}{S}\sum_{s=1}^{S}(\sigma_\varepsilon^2)^{[s]}$. The second integral can be approximated by first plugging in $\mathbb{E}[Y_* \mid x_*, \mathcal{D}] \approx \frac{1}{S}\sum_{s=1}^{S}(w^{[s]})^\top \phi_\theta(x_*)$, and then applying Monte Carlo integration using the expression for $\mathbb{E}\big[Y_* \mid x_*, w, \sigma_\varepsilon^2\big]$ from before.

39.4 Application

We now illustrate last-layer inference on two regression tasks with synthetic and real-world data. We compare last-layer inference methods under the different prior specifications from Sec. 39.2.2 against a fully Bayesian neural network (fBNN) and MC dropout [566] (see Chapter 14 for a detailed treatment) as baselines. The last-layer inference models are trained using both exact and approximate inference. The goal is to estimate the posterior predictive density $p(y_* \mid x_*, \mathcal{D})$, the posterior predictive mean $\mathbb{E}[Y_* \mid x_*, \mathcal{D}]$, and the posterior predictive variance $\mathbb{V}[Y_* \mid x_*, \mathcal{D}]$, derived previously.

We conduct the analysis under three objectives. Our first objective is to check visually whether last-layer inference performs similarly to fBNN and MC dropout. All models should provide accurate posterior predictive means for the targets, capture uncertainty within the training range, and yield higher uncertainty at input points that lie far from the training data, so-called "out-of-distribution", where the model is not expected to generalise.

In the second objective, we inspect the coverage rates of the prediction intervals. Following the definition of Pearce et al. [1305], a $1 - \alpha$ prediction interval is an interval $[\hat{y}_{n,\text{LB}}, \hat{y}_{n,\text{UB}}]$, such that at least a proportion $1 - \alpha$ of the observations fall within it

$$\mathbb{P}[\hat{y}_{n,\text{LB}} \leq y_n \leq \hat{y}_{n,\text{UB}}] \geq 1 - \alpha. \tag{39.36}$$

The coverage rates of the prediction intervals should be accurate so that the width of the prediction interval at a given level can be used to quantify uncertainty of the current prediction. In autonomous driving applications, for example, the width of a prediction interval for the steering angle at a prespecified prediction level can be used to perform model selection or switch to manual control [765].

Finally, in the third objective, we evaluate how well the posterior predictive mean fits the observed data, measured by the predictive root mean square error (RMSE). Let $\hat{y}_n$ for $n = N + 1, \ldots, N^*$ denote the posterior predictive mean of observation n in the test set, where N^* denotes the combined number of observations in the training and test set. The RMSE is then

$$\text{RMSE} = \sqrt{\frac{1}{N^* - N} \sum_{n=N+1}^{N^*} (\hat{y}_n - y_n)^2}, \tag{39.37}$$

where y_n is the respective observed target value.

39.4.1 Datasets

First, we consider a synthetic dataset generated by a noisy sine function

$$y = g(x) = \sin(x) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2), \tag{39.38}$$

where $\sigma_\varepsilon = 0.3$. To create a training and validation set, the function $g(x)$ is evaluated at 200 evenly spaced input values in the interval $[-4, 4]$. From these 200 samples, 100 are randomly selected as the training set and validation set with an 80/20 split. The remaining 100 samples form the test set.

As real-world data, we use the concrete compressive strength dataset [1814] from the UCI Machine Learning repository [878]. The goal is to predict how well the concrete can withstand compressive forces based on eight features, including the age of the concrete and its water content. The dataset comprises 1,038 samples. From these, 830 samples are used for training (from which 20 percent are used for validation during training) and 208 samples for testing. Both the features and targets are standardised using the training set statistics.

39.4.2 Implementation details

While most estimation methods for last-layer inference admit jointly fitting the last-layer weights and parameters of the deep feature projector ϕ_θ, we focus on two-step estimation throughout. This is the most common approach in practice, since it allows uncertainty estimation to be added to any deterministic neural network after training. The hyperparameters, architectures, and training settings for each model are as follows.

Last-layer inference

The network used as a basis for the deep feature projector ϕ_θ is a feedforward network with dense layers and ReLU activations in all layers except the output layer. For the synthetic data, the network has dimensions

$[1, 100, 100, 100, 10, 1]$, and for the concrete compressive strength data $[8, 200, 200, 100, 10, 1]$. It is trained for 100 epochs with the Adam optimiser [894] and a learning rate of 10^{-3}.

We perform last-layer inference with three different prior specifications: (1) a Gaussian prior on the weights combined with a Gaussian distribution for the observational noise and a fixed noise variance σ_ε^2, (2) the ridge prior, and (3) the horseshoe prior. For (1), we choose $\tau^2 = 1$ and $\sigma_\varepsilon = 0.3$. For (2) and (3), the noise is inverse gamma distributed $\sigma_\varepsilon^2 \sim \mathcal{IG}(a_\sigma, b_\sigma)$ with $a_\sigma = b_\sigma = 2$. For the ridge prior, we set $a_\tau = b_\tau = 2$. For the horseshoe prior, we set $\xi = 1$ to simplify computation, as inference over the global shrinkage parameter led to challenging posterior geometries in our case. This simplified horseshoe prior can provide local shrinkage but does not adapt the overall shrinkage level to the observed data. Alternative approaches for specifying and calibrating the prior for the global shrinkage parameter are discussed in Piironen and Vehtari [1325]. For each prior choice, we use both MCMC and VI with the following settings.

We employ different MCMC algorithms for each last-layer specification. We fit (1) with SG-MCMC. This is an attractive solution if there is a large number of observations, such that computing a closed-form posterior is computationally infeasible, but inference should be exact. We use the settings from Welling and Teh [1737], where the learning rate η is decayed from 0.1 to 0.001 during training via $\eta^{[t]} = a(b+t)^{-\gamma}$ with $\gamma = 0.55$ and appropriate choices for a and b, and where $t \geq 0$ denotes the iteration. The posterior predictive mean is computed based on a weighted average using the last S samples

$$\mathbb{E}[Y_* \mid x_*, \mathcal{D}] \approx \frac{\sum_{s=1}^{S} \eta^{[s]} \phi_\theta(x_*) w^{[s]}}{\sum_{s=1}^{S} \eta^{[s]}}. \tag{39.39}$$

as proposed in Welling and Teh [1737]. We run SG-MCMC for 2,000 iterations and discard the first 1,000 samples as burn-in. For (2), we employ a Gibbs sampler, which is run for 2,000 iterations, and the first 1,000 samples are discarded as burn-in. For the Gibbs sampler, the posterior predictive distribution is a Gaussian. Since (3) is a more complex model, we use HMC, which is known to be faster and less sticky than samplers based on random walks [1229]. We use different settings for each dataset, which we indicate in parentheses. We run the HMC sampler for 5,000 (*synthetic*) and 8,000 (*concrete*) iterations with 18 (*synthetic*) and 20 (*concrete*) leapfrog steps, and a step size of 0.0045 (*synthetic*) and 0.0015 (*concrete*). The parameters and number of iterations were chosen so that trace plots suggested convergence, and the HMC acceptance rate was about 65 percent. For the ridge prior with a Gibbs sampler, the effective sample size [ESS; 915] averaged across all weight dimensions is 958 (*synthetic*) and 1,939 (*concrete*), with a Gelman-Rubin diagnostic [600] of $\hat{R} \approx 1.03$ for both datasets. For the horseshoe prior with HMC, the chain is not well-mixed $\hat{R} > 1.5$, and the effective sample sizes are low (≈ 125). This highlights the challenge of choosing an appropriate MCMC scheme for more complex model specifications. Convergence may be improved by using more sophisticated samplers such as the No-U-Turn Sampler (NUTS) [762].

For VI, we use the three previously introduced schemes. We employ the closed-form expression from Lemma 39.1 for the Gaussian with a full covariance matrix in specification (1). For the ridge (2) and horseshoe (3) cases, we use a Gaussian variational distribution with both diagonal and factored covariance matrices, where for the latter we heuristically choose $F = 5$ (ridge) and $F = 15$ (horseshoe) factors. A more precise approach would be to perform a grid search and choose the number of factors that yield the best predictive performance on a held-out validation set. For all VI schemes, the learning rate is 10^{-3}, and we run the schemes until the ELBO has converged. This is checked visually, though automatic stopping criteria could also be employed. This results in 1,000 iterations for prior specification (1), 20,000 iterations for (2), as well as 30,000 (diagonal), and 50,000 (factored) iterations for (3).

MC dropout

For MC dropout, we use the same architecture and training hyperparameters as before. In addition, a so-called dropout probability p_{drop} has to be specified, which defines the probability of each neuron in the network being "dropped out" during training and prediction. In other words, certain weights and biases are randomly set to zero. During training, this acts as regularisation and thereby prevents overfitting. During prediction, each forward pass under a different dropout mask can be interpreted as a sample of the posterior predictive distribution. We choose $p_{\text{drop}} = 0.2$ based on [566].

Fully Bayesian neural network

For the fBNN, we place a Gaussian prior with mean 0 and variance 0.1 on each weight and bias. This prior variance is a common default choice, but one could also determine it by grid-search or scale the prior variance by the dimension of the input [1226]. The corresponding posterior is approximated with mean-field VI (see

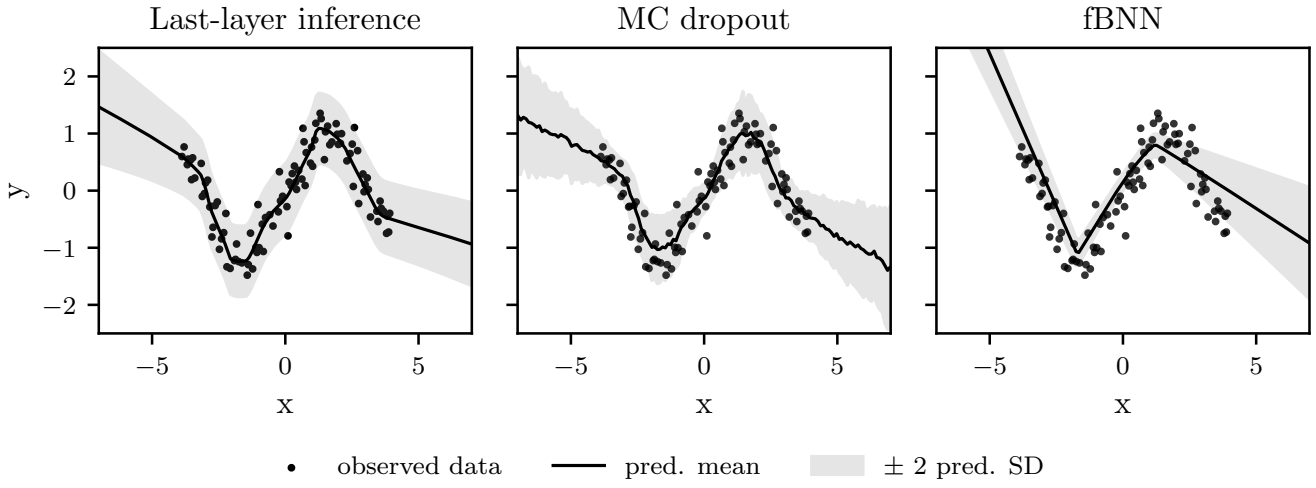


Figure 39.3: Comparison of last-layer inference with a Gaussian prior and closed form predictive posterior distribution (left), MC dropout (centre), and an fBNN (right) fitted to 100 points from the noisy sine function in Eq. (39.38).

Chapter 13) using the Bayes-by-Backprop algorithm by Blundell et al. [174] via the `blitz` package in Python [498], a KL weight of 10^{-3}, three MC samples per training step, and a learning rate of 10^{-2}. The fBNN is trained for 200 epochs and has hidden dimensions $[100, 100]$ (synthetic) and $[150, 150]$ (concrete) with ReLU activation in all layers except in the output layer.

All models are trained on the same training data using a batch size of 32 (synthetic) and 64 (concrete).

39.4.3 Results

Visualisation on synthetic data

Fig. 39.3 illustrates the posterior predictive means and pointwise intervals corresponding to two posterior predictive standard deviations on the synthetic dataset. We compare three models: the last-layer inference specification with a Gaussian prior (1; left), where the exact posterior is available in closed form, MC dropout (middle), and an fBNN (right).

Overall, all models capture the underlying sine function well in terms of the posterior predictive mean. MC dropout produces jagged standard deviation intervals across the input space, whereas last-layer inference and the fBNN yield smoothly varying intervals. For all models, predictive uncertainty increases for input values far from the training data, highlighting their potential to detect out-of-distribution observations. The visualisations for the ridge and horseshoe prior appear virtually identical to last-layer inference with the Gaussian prior and are therefore omitted.

Coverage of prediction intervals

Next, we inspect the coverage rates of the prediction intervals. Fig. 39.4 and Fig. 39.5 show the difference between the predicted and empirical coverage of the prediction intervals. We compare several last-layer approaches, as well as the fBNN and MC dropout. The x-axis measures the level of the prediction interval, and the y-axis measures the average difference between the predicted coverage and empirical coverage on the test set. A reliable probabilistic model would produce a horizontal line at a value of 0 on the y-axis, indicating that the predicted and empirical coverage coincide. If, at a given level, a model produces a value below zero, it is underconfident. Vice versa, if the value is greater than zero, it is overconfident. In Fig. 39.4, last-layer inference with a closed-form Gaussian predictive distribution (1; solid line), MC dropout (dashed line), and fBNN (dash-dotted line) produce similar coverage rates. On average, last-layer inference produces comparable or less reliable coverage for levels up to around 85 percent (synthetic) and 70 percent (concrete). For higher levels, last-layer inference performs better. Across all levels, the models predict too-narrow prediction intervals, resulting in overconfidence. In the worst case, the predicted coverage is up to 20 percentage points higher than the empirical coverage. Fig. 39.5 compares the coverage rates for last-layer inference with different prior specifications. Coverage varies considerably between models fitted using VI and MCMC. For clarity, we only show the models with the highest average coverage for

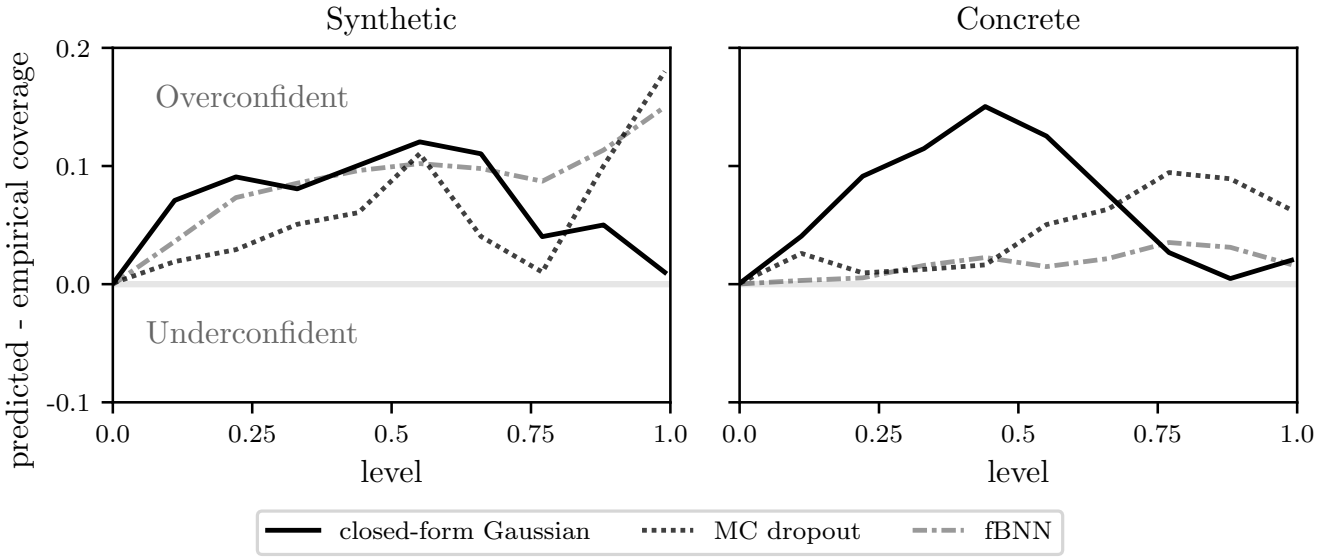


Figure 39.4: Prediction interval coverage rates for last-layer inference, fBNN and MC dropout on the test set for the noisy sine function from Eq. (39.38) (left) and concrete compressive strength data (right).

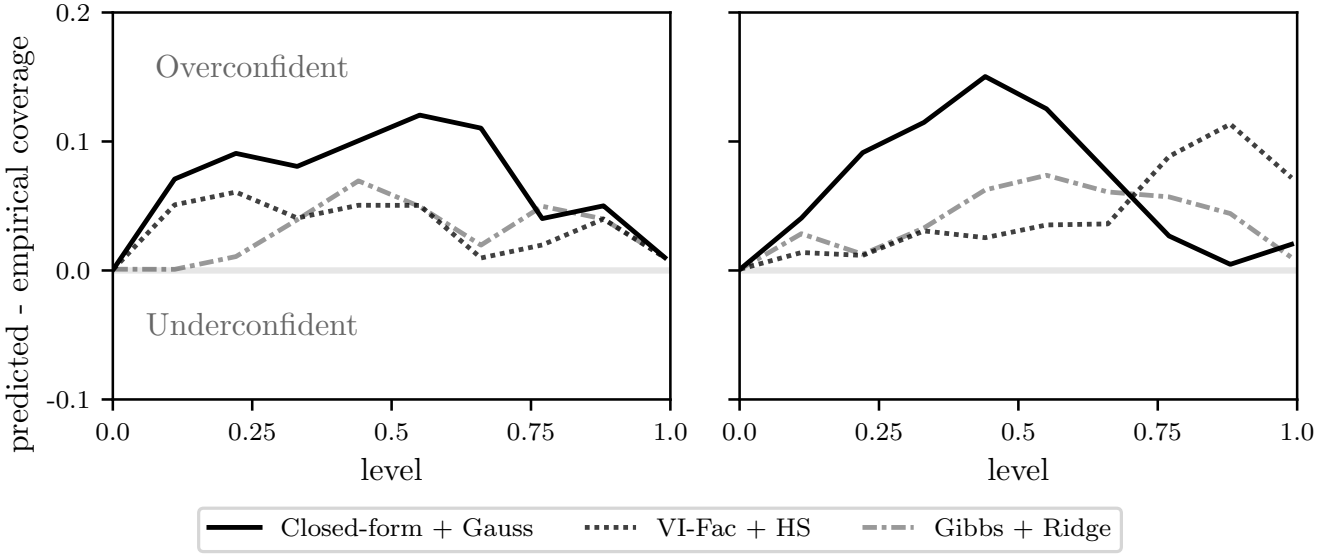


Figure 39.5: Prediction interval coverage rates for last-layer inference with different priors on the test set for the noisy sine function from Eq. (39.38) (left) and concrete compressive strength data (right).

each prior. For the synthetic data, the ridge (2; light dash-dotted line) and horseshoe priors (3; dotted line) deliver better coverage rates on average compared to a fixed variance Gaussian prior. This illustrates that a more sophisticated Bayesian treatment via a hierarchical model can be beneficial to obtain accurate uncertainty quantification. For the concrete data, the ridge and horseshoe settings provide more reliable coverage for levels up to 70 percent, and for higher levels, the closed-form Gaussian setting performs best.

Accuracy of posterior predictive means

To quantify predictive accuracy, Table 39.1 shows the RMSE on the test set for each method. Overall, the performance is similar across methods, with fully Bayesian approaches achieving the best performance. Among all last-layer inference models on the synthetic data, exact inference using SG-MCMC with a Gaussian prior results in the lowest RMSE, yet performs slightly worse than the fBNN. On the concrete data, the ridge prior combined with a Gibbs sampler performs best among the last-layer inference models, and MC dropout achieves the lowest RMSE overall. If the output layer is low-dimensional, the last-layer inference models are computationally cheaper during

prediction, because the posterior predictive mean can be computed analytically or numerically using posterior samples. In contrast, MC dropout and the fBNN require several stochastic forward passes.

Table 39.1: RMSE on the test set for different last-layer inference methods, fBNN and MC dropout on the noisy sine function from Eq. (39.38) and the concrete compressive strength data. Bold indicates the best RMSE across all methods per dataset.

Category	Type	Estimation	Prior	Synthetic	Concrete
Last-layer inference	exact	Closed-form	Gaussian	1.03	1.43
		SG-MCMC	Gaussian	0.98	1.50
		HMC	horseshoe	0.99	1.43
		Gibbs	ridge	1.02	1.42
	approx.	VI closed-form	Gaussian	1.03	1.43
		VI-Diag	ridge	1.02	1.43
		VI-Diag	horseshoe	1.03	1.43
		VI-Fac	ridge	1.03	1.43
		VI-Fac	horseshoe	1.02	1.43
Fully Bayesian	MC dropout	–	–	0.98	**1.37**
	fBNN	–	–	**0.96**	1.39

For the horseshoe and ridge priors, employing a full covariance matrix (VI-Fac) instead of a diagonal covariance matrix (VI-Diag) in the VI scheme does not lead to a substantial improvement in RMSE. While last-layer inference with SG-MCMC produces a low RMSE for the synthetic data, it exhibits convergence issues for the concrete data, resulting in poor fits and large RMSE compared to the other methods. This highlights the challenge of tuning the hyperparameters for (SG-)MCMC samplers in more complex scenarios.

Fig. 39.6 shows the posterior means and standard deviations for the last-layer weights w comparing exact with approximate Bayesian inference. Points on the diagonal indicate that the exact and approximate estimates coincide. Generally, both VI schemes provide good approximations for the posterior predictive means, with a few outliers for the horseshoe prior. The posterior predictive standard deviations suffer from over- and underestimation. This is particularly the case for the horseshoe specification (3), although this may reflect convergence issues with the HMC sampler rather than misestimation.

Impact of the feature projector's training procedure

Lastly, we look at the influence of the training procedure of the deep feature projector. Fig. 39.7 illustrates the posterior predictive mean and prediction intervals of last-layer inference for the specification with a ridge prior (2) with three different training regimes for the deep feature projector ϕ_θ. Each feature projector was trained with a mean squared error loss and an ℓ_2 regularisation on the network weights and biases, that is $\alpha/2 \cdot \|\vartheta\|_2^2$, where α controls the strength of the regularisation. This regularisation can be interpreted as a Gaussian prior with variance α^{-1} (compare ℓ_2 regularisation priors in Sec. 28.1.3). A high value of α, therefore, is equivalent to smaller prior variances when training ϕ_θ.

Fig. 39.7 illustrates that introducing a wider prior over the last-layer weights during the second estimation step cannot compensate for having previously trained the deep feature projector with strong weight regularisation. With increasing ℓ_2 regularisation (left to right), the entries of the posterior covariance matrix Σ_N remain close to zero. The effects of this are visible in the left and middle panels, where the predictive uncertainty does not vary with x.

39.5 Extensions and limitations

We now discuss some general properties, extensions, and further applications of last-layer inference.

39.5.1 Beyond canonical Bayesian linear regression models

Some last-layer models can capture multimodal predictive densities and ensure specific calibration properties. For example, Klein et al. [902] propose marginally calibrated deep distributional regression, which is a last-layer

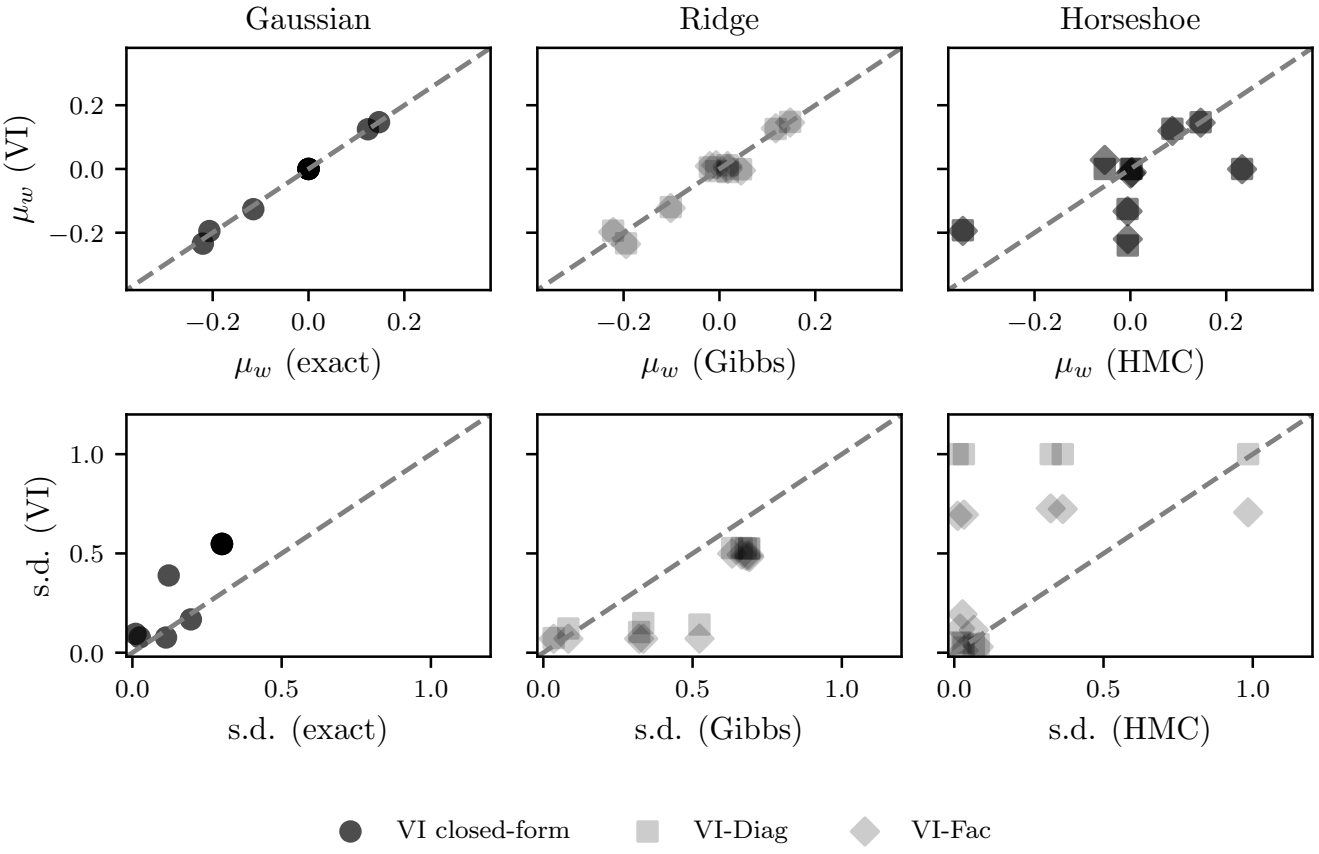


Figure 39.6: Comparison of the posterior means (top) and standard deviations (bottom) of w with exact and approximate estimation for different prior specifications (columns) with the synthetic data from Eq. (39.38).

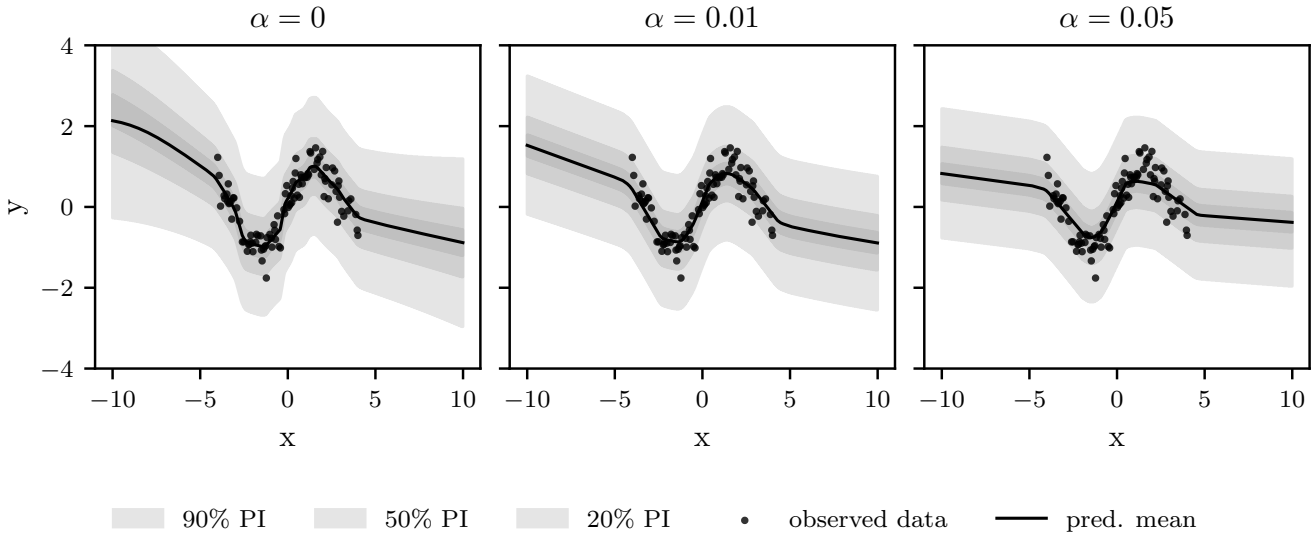


Figure 39.7: The effect of the ℓ_2 regularisation weight α used to train the deep feature projector ϕ_θ on predictive uncertainty of last-layer inference in a two-step estimation using the noisy sine function from Eq. (39.38).

inference approach based on a variable transformation model and a conditional Gaussian copula. This model produces flexible predictive densities exhibiting marginal calibration [640], that is, the asymptotic limits of the true marginal and average predictive distributions coincide. This approach was leveraged in Hoffmann and Klein [765] for autonomous driving, where training sets are too large to fit fBNNs.

Last-layer inference can also be extended beyond the standard Bayesian linear regression assumptions. In

Eq. (39.4), we assumed homoscedastic observational noise with $\varepsilon_n \sim \mathcal{N}(0, \sigma_\varepsilon^2)$, where the variance σ_ε^2 does not vary with the inputs x. In practice, the noise variance may depend on the inputs (heteroscedasticity). If this is expected, last-layer inference can be extended by allowing the noise variance to vary with the inputs. Harrison et al. [710] propose efficient estimation techniques for this model specification using VI.

39.5.2 Further applications

Last-layer inference has proven useful in several applied settings. One important example is Bayesian optimisation, which aims to optimise a target function that is expensive to evaluate. To avoid direct computation, a surrogate model approximates the target function, and optimisation is performed with the surrogate model instead. A good surrogate model quantifies uncertainty well and is cheap to evaluate. Last-layer inference fits both of these criteria and has proven competitive to other methods in benchmark studies [214, 1530].

In addition, last-layer inference has also been successfully applied to reinforcement learning, particularly in the widely studied contextual bandits problem (see Chapter 56). Riquelme et al. [1397] demonstrate that last-layer inference performs competitively with more complex Bayesian neural network approaches.

39.5.3 Drawbacks of last-layer inference

Last-layer inference becomes less effective when the output layer is high-dimensional. This situation arises, for instance, in image segmentation tasks with U-Net [1419] architectures, where the output dimension is high. In such cases, a more suitable approach is to perform Bayesian inference on an intermediate, lower-dimensional layer.

Finally, last-layer inference is not directly applicable to meta-learning frameworks such as the neural process family [582] because these models require task-dependent weight posteriors.

39.6 Final remarks

Last-layer inference is a powerful and computationally cheap method to estimate posterior predictive distributions with neural networks that can, in certain situations, produce similar performance to fully Bayesian approaches. There is a vast selection of Bayesian linear regression models that can be transformed into last-layer inference models, and the performance can be improved by employing appropriate prior structures.

Chapter 40

Scalable Laplace approximations

The previous chapters have established the linearised Laplace approximation as a principled and powerful framework for equipping neural networks with posterior uncertainty. By linearising the network around a point estimate (setting $\theta_0 \equiv \theta_*$ in Equation 8.1 of Chapter 8), we arrived at a Gaussian posterior $p(\theta \mid \mathcal{D}) \approx \mathcal{N}(\theta; \theta_*, \Psi^{-1})$ where the curvature Ψ from Equation 8.4 is often decomposed into the Generalised Gauss-Newton (GGN) matrix from Equation 8.22 along with the Hessian of the regulariser term. However, modern deep learning confronts us with formidable quantities in the network Jacobians and Hessians. The curvature matrix $\Psi \in \mathbb{R}^{P \times P}$ has a memory footprint of $\mathcal{O}(P^2)$ and requires $\mathcal{O}(P^3)$ computational effort for inversion if performed naively. Similarly, even if we approach the intractability of the curvature matrix by treating it as a linear operator, predictive posteriors and marginal likelihoods of Laplace approximations can be intractable or require instantiations of network Jacobians, which are matrices in $\mathbb{R}^{C \times P}$. For modern deep neural networks, where the number of parameters P can easily be in the millions or billions, the direct instantiation and manipulation of this matrix is utterly infeasible. In this chapter, we will address this challenge head-on by introducing a suite of techniques designed to make Laplace approximations tractable for large-scale models, making them a practical tool for the modern deep learning practitioner. We will explore three main avenues: (i) approximating the GGN matrix G with structured, computationally friendly alternatives; (ii) exploiting the function-space duality with Gaussian processes to perform inference over a reduced set of datapoints or inducing points; and (iii) sidestepping the explicit computation of the posterior covariance altogether, drawing posterior samples via iterative linear solvers. Table 40.1 summarises the methods covered in this chapter, organised by the space in which each approximation is made. Across these avenues, we must be careful to ensure that we maintain a principled approach to hyperparameter estimation using the marginal likelihood, and we will discuss these challenges in more detail.

40.1 Structured Approximations to the Curvature

The core difficulty of the Laplace approximation lies in the sheer size and lack of structure of the curvature matrix Ψ, which is mostly due to the GGN matrix component. The most direct path to scalability is therefore to replace G with an approximation $\tilde{G}$ that is easier to store, invert, and perform downstream calculations with. We briefly discussed some strategies in Section 8.4, and we expand upon them in the following section.

Table 40.1: Overview of the scalable Laplace approximations covered in this chapter, organised by the space in which the approximation is made.

Method	Space	Key approximation	Dominant cost
Diagonal	Parameter	Off-diagonal curvature = 0	$\mathcal{O}(P)$
KFAC	Parameter	Layer-wise Kronecker blocks	$\sum_i \mathcal{O}\big((D^{(i)})^3\big)$
Last-layer / subnetwork	Parameter	Dirac priors on θ_D	$\mathcal{O}(P_B^3)$
MoE LLA	Function	Disjoint data clusters	$\sum_i \mathcal{O}\big((N_i C)^3\big)$
VaLLA	Function	M inducing points	$\mathcal{O}(NM^2 + M^3)$
STO with CG/SGD	Sampling	Iterative solvers, MC predictive	$\mathcal{O}(KPN)$ per sample
Sampling-based EM	Sampling	Stochastic trace estimation	Matrix-vector products

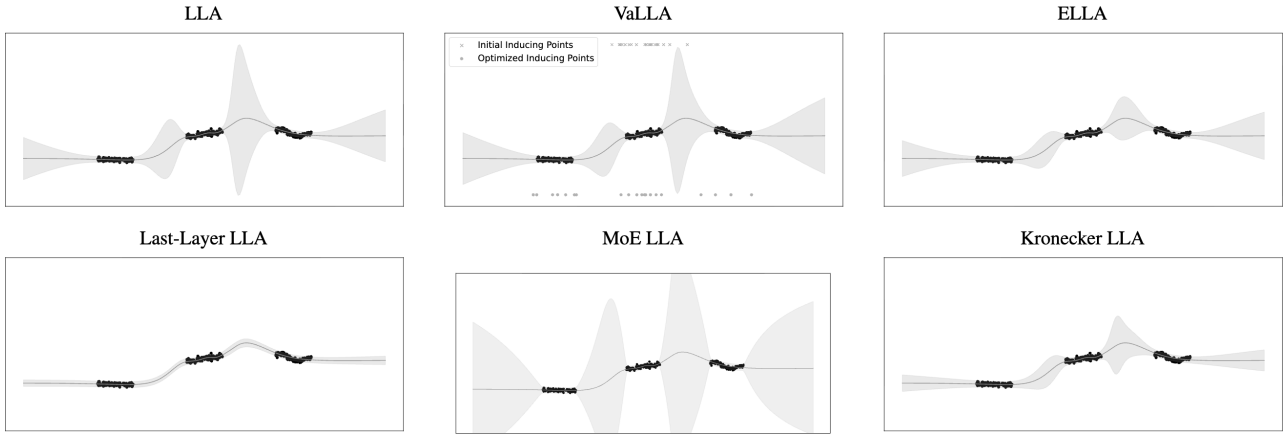


Figure 40.1: Predictive distributions (mean as a solid line, with the shaded region denoting two standard deviations) on a toy 1D regression dataset with a two-hidden-layer MLP with 50 units trained using back-propagation. The predictive distribution of the variational linearised Laplace approximation (VaLLA) is on par with or better than the last-layer and Kronecker-factored approximations, a mixture-of-experts LLA (MoE LLA) with 200 clusters, and the efficient low-rank approximation ELLA. The optimal values for the noise and prior are obtained by maximising the marginal log-likelihood estimate of the full LLA; VaLLA and ELLA reuse these values. VaLLA uses 20 inducing points for the predictive variances; ELLA uses 20 random locations and 20 features. Figure reproduced with permission from Ortega et al. [1265].

40.1.1 Diagonal approximations

The most drastic simplification is to assume that all off-diagonal elements of the curvature matrix are zero, i.e. $\tilde{G} = \text{diag}(G)$, which is equivalent to asserting that all parameters are independent *a posteriori*. The appeal of this method is its extreme computational efficiency. Inverting a diagonal matrix is a trivial operation with $\mathcal{O}(P)$ complexity, as it simply involves taking the reciprocal of each diagonal element. However, this efficiency comes at a significant cost in expressiveness. The assumption of parameter independence is strong and often unrealistic, ignoring the complex correlations that are learned during training. While it can be a useful baseline, the diagonal approximation often underestimates uncertainty in directions of strong parameter covariance, potentially leading to overconfident predictions [380].

40.1.2 Kronecker Factorisations

A more sophisticated approach seeks a middle ground between the full-rank GGN and the overly simplistic diagonal approximation, by assuming independence *between* layers while retaining correlations between parameters *within* layers. The Kronecker-factored approximation, often known as Kronecker-Factored Approximate Curvature (KFAC), exploits the layered structure of neural networks to construct a block-diagonal approximation of the GGN matrix. Reminding ourselves of the notation we previously established in Section 8.4, for a fully-connected neural network with $i = 1, \ldots, L$ layers with corresponding weights $W^{(i)} \in \mathbb{R}^{D_{\text{out}}^{(i)} \times D_{\text{in}}^{(i)}}$, the KFAC approximation represents each layer-wise block of the GGN matrix $G^{(i)}$ with a Kronecker product of two lower-dimensional matrices $\Phi^{(i)}$ and $B^{(i)}$, giving us

$$\tilde{G}^{(i)} \approx \Phi^{(i)} \otimes B^{(i)}, \qquad \Phi^{(i)} \in \mathbb{R}^{D_{\text{in}}^{(i)} \times D_{\text{in}}^{(i)}}, \quad B^{(i)} \in \mathbb{R}^{D_{\text{out}}^{(i)} \times D_{\text{out}}^{(i)}}. \tag{40.1}$$

The computational payoff comes from the identity $(\Phi^{(i)} \otimes B^{(i)})^{-1} = (\Phi^{(i)})^{-1} \otimes (B^{(i)})^{-1}$: by assuming this Kronecker-factored structural independence between the input and output sides of each layer, the inversion of a $D_{\text{in}}^{(i)} D_{\text{out}}^{(i)} \times D_{\text{in}}^{(i)} D_{\text{out}}^{(i)}$ block reduces to two far smaller inversions of cost $\mathcal{O}\big((D_{\text{in}}^{(i)})^3\big)$ and $\mathcal{O}\big((D_{\text{out}}^{(i)})^3\big)$. For each layer of the neural network, the Kronecker form is easy to store and perform curvature-vector products with, without ever needing to instantiate the dense $P \times P$ matrix; refer to Dangel et al. [367] for more details.

40.1.3 Inference over a subset of parameters

An alternative strategy for scalability is not to approximate the curvature matrix itself, but to reduce the number of parameters over which we perform full Bayesian inference. We partition the full parameter vector θ into a "Bayesian" subset θ_B and a "deterministic" subset θ_D. We then apply the Laplace approximation only to θ_B, treating θ_D as fixed at their point-estimate values. This is equivalent to placing an infinitely narrow (Dirac-delta) prior on the deterministic parameters. The posterior is then approximated as

$$p(\theta \mid \mathcal{D}) \approx p(\theta_B \mid \mathcal{D}) \prod_{\theta_d \in \theta_D} \delta(\theta_d - \theta_{d,*}), \tag{40.2}$$

where $p(\theta_B \mid \mathcal{D})$ is a Gaussian obtained from a Laplace approximation over the sub-space of θ_B. This idea is termed *variance pruning* by Daxberger et al. [381], since it effectively sets the variance of the deterministic set of parameters to zero, while maintaining the predictive capability of the model and attempting to closely approximate the predictive uncertainty of the full-scale Laplace approximation. Here, we discuss a few of these methods in further detail.

Last-Layer Laplace Approximations

As we established in Chapter 39, the last layer of neural networks can often be modelled probabilistically while treating the preceding layers as a fixed, non-linear feature extractor. Since the final layer is often a linear transformation from the dimensionality of the penultimate layer to the output dimensions ($W^{(L)} \in \mathbb{R}^{C \times D_{\text{in}}^{(L)}}$), the number of parameters P_L is much smaller than the total number of parameters P, drastically reducing the cost of the Laplace approximation from $\mathcal{O}(P^3)$ to a manageable $\mathcal{O}(P_L^3)$.

Subnetwork Inference

A more general approach to selecting a subset of weights to model probabilistically involves finding a subnetwork $\theta_S \subseteq \theta$, of size $P_S = |\theta_S|$, that produces the best approximate predictive posterior to that of the original neural network, but with a fraction of the weights. For making predictions themselves, i.e. the predictive mean, we still use the full neural network output $f(x, \theta)$ as before. While reasoning in the function space of the model outputs can be hard, it is easier to calculate discrepancy metrics in the weight space directly. Unfortunately, since we place Dirac delta distributions over the deterministic subset of weights, we cannot use common discrepancy metrics such as the KL divergence. Daxberger et al. [381] use the squared 2-Wasserstein distance instead, which remains well-defined between distributions without common support (unlike the KL divergence). We model the posterior distribution over the subnetwork weights using a full-covariance GGN-Laplace approximation of the form

$$p(\theta_S \mid \mathcal{D}) \approx \mathcal{N}(\theta_S; \theta_{S,*}, \Psi_S^{-1}), \qquad \Psi_S \in \mathbb{R}^{P_S \times P_S}, \tag{40.3}$$

where the GGN component of Ψ_S is given by

$$G_S = \sum_{n=1}^{N} J_{\theta_S} f(x_n, \theta)^\top \underbrace{\nabla_z \nabla_z^\top \ell\left(z, y_n\right)\big|_{z=f(x_n,\theta)}}_{=:H_n} J_{\theta_S} f\left(x_n, \theta\right). \tag{40.4}$$

Fortunately, the squared 2-Wasserstein distance between two Gaussian distributions is available in closed form, and by making a further assumption of independence between each parameter, we recover a simplified objective

$$W_2^2 \left(p(\theta \mid \mathcal{D}),\ p(\theta_S \mid \mathcal{D}) \prod_{\theta_d \in \theta_D} \delta(\theta_d - \theta_{d,\star}) \right) = \sum_{i=1}^{P} \sigma_i^2 (1 - m_i), \qquad \sigma_i^2 = [\Psi^{-1}]_{ii}, \tag{40.5}$$

where $m_i = 1$ if $\theta_i \in \theta_S$ and 0 otherwise. The objective in Equation 40.5 is trivially minimised by picking a subnetwork with the P_S highest marginal variances, which are easily approximated by a diagonal approximation to Ψ. While it might seem counter-intuitive that we must resort to diagonal approximations of the Hessian again, it is important to note that the diagonal assumption is made solely for the purpose of subnetwork selection, and we still perform full covariance inference over θ_S, allowing the rich correlations between parameters in the chosen subset to contribute meaningfully to the predictive uncertainty.

40.1.4 Inference over a subset of datapoints

In Chapter 8, we established that the linearised Laplace approximation turns any differentiable neural network into a Gaussian process (GP), with an induced Mercer kernel of the form

$$\hat{k}\left(x, x'\right) = J_\theta f\left(x, \theta_*\right) \Psi\left(\theta_*\right)^{-1} J_\theta f\left(x', \theta_*\right)^\top, \qquad \hat{k}(x, x') \in \mathbb{R}^{C \times C}, \tag{40.6}$$

where the Jacobian pushes the parameter-space uncertainty captured by Ψ into the function space. Note that for networks with multiple outputs the kernel is matrix-valued; scalar-kernel expressions in what follows extend blockwise. If we make an assumption about the prior distribution $p(\theta) = \mathcal{N}(0, A^{-1})$ for a prior precision matrix A, we can also write the induced Mercer kernel of the prior GP distribution as

$$k\left(x, x'\right) = J_\theta f\left(x, \theta_*\right) A^{-1} J_\theta f\left(x', \theta_*\right)^\top. \tag{40.7}$$

In function space, while working with GPs, it can be easier to consider quantities in the form of matrices, allowing us to represent tedious nested sums as matrix multiplications and simultaneously letting us collect quantities calculated over datasets $\mathcal{D}$ into matrices that imply their dimensionality more clearly. Here, we define the matrix $J \in \mathbb{R}^{NC \times P}$ as the vertical stacking of the per-datapoint Jacobians, $J = [J_\theta f(x_1, \theta)^\top, \ldots, J_\theta f(x_N, \theta)^\top]^\top$, and the Hessian of the loss $H \in \mathbb{R}^{NC \times NC}$ as the block-diagonal matrix where each block is given by $H_n = \nabla_z \nabla_z^\top \ell\left(z, y_n\right)\big|_{z=f(x_n,\theta)}$. With this notation, the posterior covariance follows from applying the Woodbury matrix identity to the parameter-space posterior precision $\Psi = G + A = J^\top H J + A$: inverting Ψ in the P-dimensional parameter space is exchanged for inverting an $NC \times NC$ matrix in function space,

$$\hat{k}(x, x') = k(x, x') - k(x, X) \underbrace{\left(H^{-1} + J A^{-1} J^\top\right)^{-1}}_{:=Q^{-1}} k(X, x'). \tag{40.8}$$

The function-space duality in Equation 40.8 allows us to easily reason about the linearised Laplace approximation as an equivalent GP for all intents and purposes, letting us trade off the cubic complexity in the parameter space with a cubic complexity in the number of datapoints times the number of output dimensions instead, which is often intractable itself in modern machine learning. In this section, however, we discuss a few approaches to circumvent this complexity by exploiting the rich literature on scalable Gaussian processes (see Chapter 38 for a dedicated treatment).

Mixture of Experts Laplace Approximations

As a simple approach, explored in Lee et al. [1002], we could consider creating disjoint subsets of the training data $\{\mathcal{D}_1, \ldots, \mathcal{D}_M\}$ with $\cup_{i=1}^M \mathcal{D}_i = \mathcal{D}$ and $\mathcal{D}_i \cap \mathcal{D}_j = \varnothing$ for $i \neq j$. For each of these subsets, we can fit individual GPs that would have a cubic complexity in $N_i C \ll NC$. This is a simple idea that is further improved by choosing datasets such that points within each cluster are highly correlated and clusters themselves have low correlation, and also incorporating the predictions of neighbouring GPs into the prior in order to make predictions on points at the border of clusters.

Variational Linearised Laplace Approximations

Instead of picking disjoint smaller subsets of the training data and pooling predictions from multiple GPs, we can follow the approach of variational GPs, which approximate the GP posterior using a small set of M inducing points Z with their associated function-space predictions $u = f(Z, \theta)$, following the sparse variational Gaussian process (SGP) framework introduced in Chapter 38. This is precisely the strategy adopted by Variational Linearised Laplace Approximations (VaLLA) [1265]. VaLLA bridges the gap between the linearised Laplace approximation and the SGP framework. The core idea is to construct a low-rank approximation to the GP posterior using M inducing points, where $M \ll N$. A key insight of VaLLA is to use a *decoupled* SGP parameterisation, which separates the treatment of the posterior mean and covariance. Given that the Laplace approximation is already centred at a high-quality point estimate, the MAP solution θ_*, VaLLA anchors its posterior mean function to the neural network's output, $f(x, \theta_*)$. It then uses the M inducing points solely to build a tractable, low-rank approximation of the posterior covariance. The parameters of this covariance (the inducing locations Z and an associated variational precision matrix V) are optimised by maximising a variational objective that drives the sparse approximation towards the true, intractable GP posterior; as we discuss in Section 40.3, VaLLA uses an α-divergence objective in place of the standard ELBO. This optimisation has a cost of $\mathcal{O}(NM^2 + M^3)$, which is significantly more manageable than the $\mathcal{O}((NC)^3)$ cost of the full function-space GP. Figure 40.1 compares the

resulting predictive distributions against other scalable approximations on a toy regression task. The resulting predictive covariance from VaLLA, given in Equation 40.9, takes a standard form from the sparse GP literature:

$$\hat{k}_{\text{VaLLA}}(x, x') = k(x, x') - k(x, Z)\left(V^{-1} + k(Z, Z)\right)^{-1} k(Z, x'), \tag{40.9}$$

where $k(x, x')$ is the prior kernel from Equation 40.7, $k(Z, Z)$ is the $M \times M$ prior kernel matrix evaluated at the inducing points, and V is a variational matrix optimised during training. While VaLLA is formulated in function space, it can also be interpreted as defining an implicit, low-rank approximation to the parameter-space curvature matrix Ψ. The full curvature $\Psi = A + G$ (for the prior precision matrix A defined above) is approximated by

$$\tilde{\Psi} = A + \tilde{G}, \tag{40.10}$$

where the full GGN matrix $G = J^\top H J$ (which depends on all N data points) is replaced by a low-rank approximation $\tilde{G}$ constructed from the M inducing points and the learned matrix V. This effectively substitutes the data-dependent term with a much smaller, tractable equivalent, achieving scalability while maintaining a principled variational approximation. For a detailed derivation, we refer the reader to Ortega et al. [1265].

40.2 Sampling via Iterative Solvers

The approaches discussed so far, from structured approximations like KFAC to function-space variational methods like VaLLA, all focus on constructing a tractable *approximation* to the curvature matrix Ψ or its inverse. An entirely different and powerful alternative for scalability is to sidestep the explicit approximation of Ψ altogether. Instead, this family of methods focuses on drawing samples from the posterior, $p(\theta \mid \mathcal{D}) \approx \mathcal{N}(\theta; \theta_*, \Psi^{-1})$, and then using these samples to compute posterior predictive distributions via Monte Carlo estimation. This strategy is particularly appealing because it leverages a key insight, often related to *Matheron's rule* for Gaussian conditionals [855, 1772], which has its roots in the perturb-and-optimise perspective of Papandreou and Yuille [1293] and has been operationalised as the *sample-then-optimise (STO)* principle in [36].

Definition 40.1. *(Matheron's Rule for Linear Models). Given a linear model with prior $p(\theta) = \mathcal{N}(0, A^{-1})$ and Gaussian likelihood $p(Y \mid \theta) = \mathcal{N}(J\theta, H^{-1})$, an exact sample from the posterior $p(\theta \mid Y) = \mathcal{N}(\theta_*, \Psi^{-1})$ can be constructed from a prior sample $\theta_0 \sim \mathcal{N}(0, A^{-1})$ and a noise sample $\epsilon \sim \mathcal{N}(0, H^{-1})$ as*

$$\theta_s = \theta_0 + A^{-1} J^\top \left(J A^{-1} J^\top + H^{-1}\right)^{-1} (Y - J\theta_0 - \epsilon),$$

where J is the Jacobian, H is the likelihood precision, and A is the prior precision.

While this formula provides a theoretical basis, its direct computation still involves a large matrix inverse. A more practical application of this principle is the "sample-then-optimise" method from Antorán et al. [36], which demonstrates that an exact sample from the high-dimensional Gaussian posterior can be obtained by solving a related, but randomly perturbed, convex optimisation problem.

40.2.1 Sampling by Optimising a Perturbed Objective

Let us recall our target posterior $\mathcal{N}(\theta; \theta_*, \Psi^{-1})$, where the full-rank curvature matrix is $\Psi = G + A$. Here, G is the GGN matrix and A is the Hessian of the regulariser, or equivalently the prior precision matrix (e.g., $A = \lambda I$ for an isotropic prior). The STO method shows that a sample θ_s from the posterior can be generated by finding the minimum of a randomly perturbed version of this same objective. The procedure to draw one sample θ_s is as follows:

1. *Sample from the prior:* Draw a random parameter perturbation from the prior distribution, $\epsilon_\theta \sim \mathcal{N}(0, A^{-1})$.
2. *Sample from the likelihood:* For each data point (x_n, y_n) in the training set $\mathcal{D}$, draw a random "target" perturbation from the likelihood's noise model, $\epsilon_{y,n} \sim \mathcal{N}(0, H_n^{-1})$, where $H_n = \nabla_z \nabla_z^\top \ell(z, y_n)\big|_{z=f(x_n,\theta_*)}$.
3. *Optimise the perturbed objective:* Solve for the parameter *delta* sample, $\delta\theta_s$, by minimising the following quadratic least-squares problem:

$$\delta\theta_s = \underset{\delta\theta \in \mathbb{R}^P}{\arg\min} \frac{1}{2}(\delta\theta - \epsilon_\theta)^\top A(\delta\theta - \epsilon_\theta) + \sum_{n=1}^{N} \frac{1}{2}(\epsilon_{y,n} - J_n\delta\theta)^\top H_n(\epsilon_{y,n} - J_n\delta\theta), \tag{40.11}$$

where J_n is the network Jacobian $J_\theta f(x_n, \theta_*)$ for data point n.

The final posterior sample is then constructed as $\theta_s = \theta_* + \delta\theta_s$. It can be shown that the solution θ_s to this procedure is an *exact sample* from the target Laplace posterior $\mathcal{N}(\theta; \theta_*, \Psi^{-1})$, and we refer to Antorán et al. [36] for additional details.

40.2.2 Iterative Solvers for Sampling

The true power of this method lies in the ability to optimise an objective instead of performing expensive matrix inversions. The objective in Equation 40.11 is a large, convex quadratic problem. Finding its minimum is equivalent to solving the linear system $\Psi\delta\theta = b$, where $b = A\epsilon_\theta + \sum_{n=1}^{N} J_n^\top H_n \epsilon_{y,n}$. This system can be solved efficiently with *iterative linear solvers*, avoiding any $\mathcal{O}(P^3)$ matrix inversions.

- *Conjugate Gradient (CG):* This is a classic iterative method for solving large linear systems $\Psi x = b$ where Ψ is symmetric and positive-definite (which it is for LLA). Conjugate gradient approaches simply require matrix-vector products of the form $\Psi v = (G + A)v$, and these "GGN-vector products" can be computed efficiently using automatic differentiation on the *entire dataset* without ever forming the $P \times P$ matrix. The total cost is $\mathcal{O}(KPN)$, where K is the number of iterations needed for CG to converge, often much smaller than P ($K \ll P$).
- *Stochastic Solvers (e.g., SGD):* For massive datasets where N is too large to process in each iteration, we can treat Equation 40.11 as a finite-sum objective and minimise it using standard stochastic optimisation. At each step, we sample a mini-batch $\mathcal{B}$ and compute a stochastic gradient of the objective. This requires a GGN-vector product *on the mini-batch* $\mathcal{B}$, and we obtain $\mathcal{O}(|\mathcal{B}|P)$ time-complexity per step. The total time is $\mathcal{O}(EPN)$ if E epochs are needed for convergence, which may be more than the full-batch CG, but it allows for trading compute time for memory.

40.2.3 Monte Carlo Predictive Distributions

Once we have a set of S posterior samples, $\{\theta_s\}_{s=1}^{S}$, obtained by running this procedure S times with different random perturbations, we can approximate the predictive distribution for a new input x_* using standard Monte Carlo approximations. The predictive mean of the *function* $f(x_*, \theta)$ is given by the sample mean:

$$\mathbb{E}[f(x_*, \theta) \mid \mathcal{D}] \approx \frac{1}{S} \sum_{s=1}^{S} f(x_*, \theta_s) =: \bar{f}(x_*). \tag{40.12}$$

The predictive covariance of the *function* is given by the sample covariance:

$$\mathrm{Cov}[f(x_*, \theta) \mid \mathcal{D}] \approx \left(\frac{1}{S} \sum_{s=1}^{S} f(x_*, \theta_s) f(x_*, \theta_s)^\top \right) - \bar{f}(x_*)\bar{f}(x_*)^\top, \tag{40.13}$$

where $\bar{f}(x_*)$ denotes the sample mean from the previous equation. This covariance captures the epistemic uncertainty (model uncertainty) in the function's output. This sampling-based approach therefore transforms the intractable problem of matrix inversion into a more manageable series of iterative optimisations, allowing us to trade off approximations for tractable computational complexity.

40.3 Model Selection in Scalable Laplace Approximations

A critical, yet challenging, component of any Bayesian method is the selection of hyperparameters. In the context of the Laplace approximation, this primarily involves the parameters of the prior distribution, $p(\theta \mid \varphi)$, which are encapsulated in the hyperparameter set φ. When we employ L2-regularisation of the form $\Omega(\theta) = \frac{1}{2}\theta^\top A\theta$, we are implicitly choosing a zero-mean Gaussian prior $p(\theta) = \mathcal{N}(0, A^{-1})$. The hyperparameters φ are thus the elements of the prior precision matrix A, which could be a single scalar precision λ (for an isotropic prior $A = \lambda I$), a set of layer-wise precisions, or a more complex structure.

40.3.1 The Marginal Likelihood Optimisation Framework

The most principled approach for setting these hyperparameters is *Type-II Maximum Likelihood*, also known as *Evidence Maximisation*. This framework treats the hyperparameters φ as parameters of a generative model for

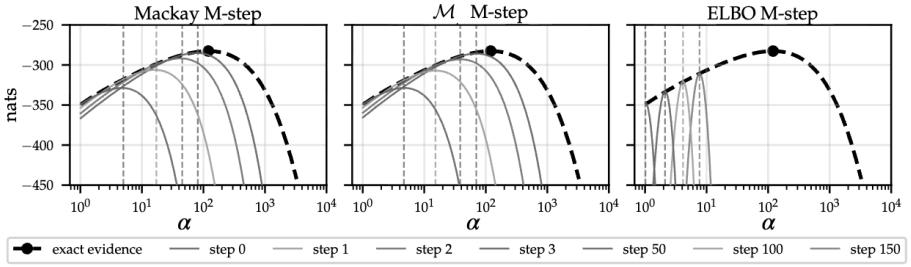


Figure 40.2: The rate of convergence of different approximate marginal likelihood algorithms compared with the exact marginal likelihood $\mathcal{M}$ (middle panel). MacKay's first-order updates (left panel) converge almost as quickly to the optimal solution $A = \lambda_\star I$, whereas the unconstrained ELBO (right panel) is very slow to converge.

the data, and seeks to find the values of φ that maximise the probability of the observed data $\mathcal{D}$. This probability is the marginal likelihood, or "evidence":

$$\mathcal{M}(\varphi) = \log p(\mathcal{D} \mid \varphi) = \log \int p(\mathcal{D} \mid \theta) p(\theta \mid \varphi) d\theta. \tag{40.14}$$

As we established in Section 11.2, the Laplace approximation provides a closed-form estimate for this intractable integral:

$$\mathcal{M}(\varphi) \approx \underbrace{\log p(\theta_* \mid \varphi) + \log p(\mathcal{D} \mid \theta_*)}_{\text{Log-joint at MAP}} - \frac{1}{2} \log\det \Psi(\theta_*) + \frac{P}{2} \log(2\pi), \tag{40.15}$$

where θ_* is the MAP estimate (which itself depends on φ) and the curvature $\Psi = G + A$ also depends on φ. The fundamental challenge of scalability is laid bare by this equation: the log-determinant term, $\log\det \Psi$, requires $\mathcal{O}(P^3)$ computation and $\mathcal{O}(P^2)$ memory, rendering it completely intractable for modern deep neural networks. The remainder of this section is dedicated to exploring the different strategies that scalable Laplace methods employ to circumvent this computational bottleneck.

40.3.2 Optimisation via the Evidence Lower Bound (ELBO)

One general strategy for dealing with an intractable marginal likelihood is to instead optimise a tractable lower bound. Variational Inference (VI), introduced in Chapter 12, posits an approximate posterior $q(\theta)$ from a simpler, tractable family (e.g., a Gaussian) and seeks to minimise the KL divergence between this approximation and the true posterior $p(\theta \mid \mathcal{D}, \varphi)$. This minimisation is equivalent to maximising the Evidence Lower Bound (ELBO), $\mathcal{M}_{\text{ELBO}}(q, \varphi)$:

$$\log p(\mathcal{D} \mid \varphi) \geq \mathcal{M}_{\text{ELBO}}(q, \varphi) = \mathbb{E}_q[\log p(\mathcal{D} \mid \theta)] - \mathcal{D}_{\text{KL}}(q(\theta) \,\|\, p(\theta \mid \varphi)). \tag{40.16}$$

This objective can be jointly optimised with respect to both the variational parameters of q and the hyperparameters φ of the prior. While this avoids the log-determinant of Ψ, it introduces its own complexities. Optimising with respect to a full-covariance $q(\theta) = \mathcal{N}(\theta_q, \Sigma_q)$ requires storing and updating Σ_q, which has $\mathcal{O}(P^2)$ parameters. This often leads to further simplifications, such as assuming a diagonal covariance $\Sigma_q = \text{diag}(\sigma_q^2)$, which is a common approach in "Variational Bayes" methods but, as we saw, can be an oversimplification.

40.3.3 Optimisation via the Expectation-Maximisation (EM) Algorithm

A second major strategy for maximising the marginal likelihood (Equation 40.15) is the *Expectation-Maximisation (EM) algorithm*. In the ELBO-based approach above, the variational mean θ_q must be optimised jointly with the covariance and the prior hyperparameters, coupling the two problems. The EM algorithm proposed by MacKay [1105] (see also MacKay [1111]) decouples them, alternating between setting the variational mean equal to the exact posterior mean of θ for a given choice of A (the E-step) and updating A (the M-step), until the pair (A, θ) converges to a locally optimal setting. Starting from an initial hyperparameter estimate A_k, the algorithm iterates between two steps:

- *(E-step)* Compute the posterior over the parameters under the current hyperparameters. In our setting, this is exactly the Laplace approximation,

$$q_k(\theta) = \mathcal{N}(\theta;\, \theta_*, \Psi_k^{-1}) \quad \text{where} \quad \Psi_k = G + A_k. \tag{40.17}$$

- *(M-step)* Update the hyperparameters by maximising the expected log-joint under this posterior,

$$A_{k+1} = \arg\max_A \mathbb{E}_{q_k}[\log p(\mathcal{D} \mid \theta) + \log p(\theta \mid A)]\,. \tag{40.18}$$

While this breaks the problem down, the M-step still involves computing the next estimate of the prior precision matrix A, which can be computationally intractable, requiring us to turn to approximate, first-order solutions. Figure 40.2 illustrates the convergence behaviour of these alternatives.

MacKay's First-Order Update

For the common case of an isotropic prior $A = \lambda I$, MacKay [1105] provided a simple and elegant first-order update rule that solves the M-step:

$$\lambda_{k+1} = \frac{\gamma_k}{\|\theta_*\|^2}. \tag{40.19}$$

Here, γ_k is the *"effective number of parameters"*, a crucial quantity defined as

$$\gamma_k = \mathrm{Tr}(G\Psi_k^{-1}) = \mathrm{Tr}\left(G(G + \lambda_k I)^{-1}\right). \tag{40.20}$$

This term γ_k intuitively measures the number of parameters in the model that are effectively "used" or constrained by the data, as opposed to being constrained by the prior. However, despite its elegance, this update rule is still intractable without further approximations, as calculating the trace of the matrix inverse of Ψ faces all the usual issues. Nevertheless, we mention MacKay's first-order update here, foreshadowing a computationally tractable version of it later in the chapter.

40.3.4 Practical Hyperparameter Tuning for Scalable Methods

We can now see how each of the scalable methods discussed in this chapter navigates these challenges.

Structured Approximations (Diagonal and KFAC)

These methods take the most direct route. They replace the intractable curvature Ψ with a structured approximation $\tilde{\Psi}$ (diagonal or KFAC) that is explicitly designed to be tractable. The core benefit is that $\log\det\tilde{\Psi}$ is cheap to compute. For a diagonal $\tilde{\Psi}$, it is $\mathcal{O}(P)$. For KFAC, it scales with the sum of cubes of layer dimensions, not $\mathcal{O}(P^3)$. These methods can therefore perform Type-II Maximum Likelihood by directly optimising the approximate marginal likelihood. This is typically done using gradient-based methods, computing $\nabla_\varphi \mathcal{M}(\varphi)$ and taking gradient steps on the prior hyperparameters (e.g., a single λ or layer-wise λ_i).

Subnetwork and Last-Layer Inference

These methods face a more fundamental problem. Partitioning the parameters into a Bayesian set θ_B and a deterministic set θ_D implicitly places a Dirac-delta prior on θ_D. A Dirac delta is not absolutely continuous with respect to the Lebesgue measure and therefore admits no density; evaluating the log-marginal likelihood (Equation 40.15) requires treating $\delta(\cdot)$ as a density, whose value diverges at the point mass, and any KL divergence between the continuous Gaussian component and the degenerate delta component is infinite. The failure is thus a measure-theoretic mismatch—a lack of absolute continuity—rather than merely "disjoint support". Lacking a principled Bayesian objective, these methods must fall back on the standard machine learning practice of *cross-validation*. The prior precision λ is treated as a regularisation hyperparameter to be tuned by minimising a score (e.g., negative log-likelihood) on a held-out validation set. For subnetwork inference, Daxberger et al. [381] also propose a heuristic to adjust the prior: if a full network of size P uses precision λ, a subnetwork of size P_S should use $\lambda_S = \lambda \cdot (P_S/P)$. This attempts to keep the function-space variance (the magnitude of the implied GP kernel) approximately constant.

Variational LLA (VaLLA)

Although VaLLA is formulated as a sparse variational GP, it does not optimise the standard ELBO. This is not because a fixed mean function is inherently incompatible with the ELBO—decoupled sparse GPs with fixed or pre-trained means can be trained with the standard variational objective—but because of how the fixed mean interacts with the non-conjugate likelihood once only the covariance is learned. With the mean anchored to $f(x, \theta_*)$, the

zero-forcing character of the KL term drives the posterior variances towards a degenerate collapse, systematically underestimating predictive uncertainty. To avoid this, VaLLA employs an alternative variational objective based on the α-*divergence* for $\alpha \in (0, 1]$:

$$\max_{q(f)} \sum_{i=1}^{N} \frac{1}{\alpha} \log \mathbb{E}_q \left[p\left(y_i \mid f\right)^{\alpha} \right] - \mathcal{D}_{\mathrm{KL}}(q \,\|\, p). \tag{40.21}$$

Compared with the ELBO, the objective in Equation 40.21 is mass-covering rather than zero-forcing: it penalises underestimated variances more heavily, preventing the collapse and enabling a stable, principled optimisation of the prior hyperparameters.

Sampling-Based EM Algorithm

Finally, the sampling-based approach from Section 40.2 provides a powerful way to implement the EM algorithm in a fully scalable manner. We want to use MacKay's update (Equation 40.19), but we must compute the intractable trace $\gamma_k = \mathrm{Tr}(G\Psi_k^{-1})$. We can thankfully rely on *stochastic trace estimation* as a solution. The trace of a matrix Λ can be written as an expectation $\mathrm{Tr}(\Lambda) = \mathbb{E}_{v \sim p(v)}[v^\top \Lambda v]$, where $p(v)$ is a distribution with zero mean and identity covariance. In our case, this can be cleverly rearranged to be an expectation over zero-mean posterior samples:

$$\gamma_k = \mathrm{Tr}(G\Psi_k^{-1}) = \mathbb{E}_{\bar{\theta} \sim \mathcal{N}(0, \Psi_k^{-1})} \left[\bar{\theta}^\top G \bar{\theta} \right]. \tag{40.22}$$

The identity in Equation 40.22 can be approximated with a Monte Carlo average. We can now combine all the pieces into a fully scalable algorithm:

1. Start with an initial hyperparameter λ_k.
2. *(E-step, implicitly)* Draw S zero-mean samples $\{\bar{\theta}_s\}_{s=1}^S$ from $\mathcal{N}(0, \Psi_k^{-1}) = \mathcal{N}(0, (G + \lambda_k I)^{-1})$ using the *sample-then-optimise (STO)* procedure from Section 40.2, which relies only on iterative linear solvers. Note that the minimiser of Equation 40.11 is the parameter *delta* $\delta\theta_s \sim \mathcal{N}(0, \Psi_k^{-1})$, which is exactly the zero-mean sample required here: we set $\bar{\theta}_s \equiv \delta\theta_s$ and do *not* add the MAP centre θ_*. Using the shifted posterior samples $\theta_s = \theta_* + \delta\theta_s$ instead would bias the trace estimator by $\theta_*^\top G \theta_*$.
3. *(M-step, stochastically)* For each sample $\bar{\theta}_s$, compute the GGN-vector product $v_s = G\bar{\theta}_s$. This is computationally cheap, with the same cost as the iterative solver steps. We then estimate the trace, $\hat{\gamma}_k = \frac{1}{S} \sum_{s=1}^S \bar{\theta}_s^\top v_s$, and finally apply MacKay's update: $\lambda_{k+1} = \hat{\gamma}_k / \|\theta_*\|^2$.
4. Repeat until convergence.

This sampling-based EM algorithm elegantly sidesteps the $\mathcal{O}(P^3)$ bottleneck, relying only on efficient matrix-vector products.

40.4 Concluding Remarks

In this chapter, we have explored a wide variety of techniques to scale the highly flexible Laplace approximation to very large neural networks on very large datasets. As we saw, there are multiple approaches to tackle the problem, ranging from approximations to the curvature matrix, to exploiting the function-space duality with sparse GPs, and finally to simplifying the task into stochastic estimates that can be approximated with samples from the posterior. Across all these methods, there are various trade-offs being made between time and space complexity, and the flexibility of the approximation being chosen. Depending on the problem at hand, the Laplace approximation is amenable to a veritable toolbox of scalable approaches for the practitioner.

Chapter 41

Learned Compression with Deterministic Codes

The ever-increasing amount of information that is being generated and transmitted globally demands increasingly powerful data compression algorithms. Although traditionally studied under information theory [1499] and signal processing [611], data compression is inherently a problem of probabilistic modelling and a fertile ground for the application of Bayesian deep learning methods. At its core, compression works by exploiting the structure and patterns inherent in the data, so that more predictable patterns can be described with fewer bits. It is then no surprise that statistical learning, which aims to understand structure in data, is built into such celebrated compression algorithms as adaptive arithmetic coding [1400] and Lempel-Ziv [1884] (the basis of extremely popular file formats such as `zip` and `gzip`). In the reverse direction, compression ideas have also profoundly influenced machine learning, starting from Occam's razor, to the principles of Minimum Description Length [676] and Information Bottleneck [1613, 1614], and to the current Large Language Models whose roots trace back to Shannon's landmark treatise on information theory [1499].

In recent years, the development of sophisticated statistical models (i.e., "deep generative models") such as variational autoencoders (VAEs) [892], generative adversarial networks (GANs) [650], and normalizing flows [1290] is having an increasing impact on data compression. Information theory tells us that any probability model for discrete data corresponds to a lossless compression algorithm [924, 1160], and various approaches have been developed for neural lossless compression based on (variational) autoencoders [1167, 1625], normalizing flows [754, 778, 1630], autoregressive models [779], and diffusion models [893]. Deep generative models have similarly reshaped lossy compression. Ideas for neural lossy compression can already be found in early work on hierarchical latent variable models [664], and were subsequently made competitive in rate-distortion performance by end-to-end training with an effective entropy model and approximation to quantization [83, 1596]. Subsequent methods further incorporate GANs [10, 1168] or diffusion models [624, 781, 1597, 1801, 1809] for enhanced realism or perceptual quality, allowing highly realistic reconstructions to be generated at extremely low bit-rates.

In this and the next chapter, we will introduce elements of information theory and the basics of data compression, and explore how ideas from deep probabilistic modelling are closely related to compression and can be used to build powerful compressors.

We start by considering lossless compression (Section 41.2), where the goal is to assign data points to strings of bits in a one-to-one manner, as efficiently as possible. A basic result in information theory guarantees that any probability model induces a valid assignment and vice versa. Therefore, we can (at least in principle) reduce the problem of lossless compression to that of estimating the data distribution. To simplify modelling high-dimensional discrete distributions, we consider either factoring our model distribution autoregressively, or formulating it as a latent variable model with a tractable joint distribution. Each modelling choice requires a different compression procedure. In the first case, an autoregressive model lends itself naturally to compression using an algorithm called *arithmetic coding* (Section 41.2.5). In the second case, we first infer the latent representation of a given data point, and then compress them both using their joint distribution under the model (Section 41.2.5). The resulting "two-part code" turns out to have a suboptimal code length, which is addressed by bits-back coding (Section 42.1, next chapter). In lossy compression (Section 41.3), a data point is first encoded into a compressed latent representation, from which a reconstruction is then later decoded. As we allow imperfect reconstructions, a trade-off must be made between the amount of acceptable *distortion* in the lossy reconstruction and the (bit-)*rate* of the compressed representation (as captured by the file size). Inspired by vector quantization and transform coding methods (Section 41.3.1), we build neural lossy compressors based on *nonlinear transform coding* (Section 41.3.2)

that are optimized end-to-end for rate-distortion performance. As we elaborate in Section 41.3.3, there is a strong parallel between a lossy compressor and a latent variable model, where the encoding procedure corresponds to variational inference and the decoding procedure corresponds to conditional data generation. This parallel can be made precise in the case of nonlinear transform coding, and later with the help of *Relative Entropy Coding* (Section 42.2, next chapter), which promises to turn any latent variable model into a lossy compressor.

In both lossless and lossy compression, a latent variable model has a role to play, and we can draw connections and inspirations from variational inference and learning. In lossless compression, the latent variables help define a probability model over the data space via marginalization, and the negative evidence lower bound (ELBO, Chapter 12), or "NELBO", represents an upper bound on the ideal coding cost under the marginal probability model (see Section 42.1.4). In lossy compression, the latent variables often represent a decorrelated representation of the data that can be more effectively quantized then compressed, and the NELBO represents a rate-distortion cost (see Section 41.3.3). In both cases, (approximate) variational inference produces the optimal encoding for a given latent variable model; training the model on the ELBO with amortized inference then corresponds to learning a data compressor end-to-end.

Although learning-based methods have consistently demonstrated strong compression performance in literature, building and deploying compression systems in practice faces many engineering challenges such as numerical issues [86] and computational efficiency [847, 1804]. We sweep these issues under the rug and focus on the conceptual foundations. Although we discuss learned compression in the data compression setting, the same techniques can also be adapted to compress the weights of neural networks in a straightforward manner [1256, 1806]. We refer readers to libraries such as TensorFlow Compression [87] and CompressAI [108] which provide open-source implementations of learning-based compression methods and serve as a convenient starting point for their own research.

41.1 What is Compression?

Data compression aims to reduce the number of bits needed to represent useful information in the data. This raises such basic questions as "*What is information?*" and "What does it mean to *communicate information?*". We find answers in probability theory, following the approach of Claude Shannon who first studied these questions and founded the field of information theory [1499]. We view the information-producing *source*, i.e., the data to be compressed, as a random object X, which takes values in a predetermined set $\mathcal{X}$ known as the *alphabet*. To compress data, the sender observes some value of $X = x$ and *encodes* it into a short sequence of bits; this is done in such a way that the receiver, after retrieving the same sequence of bits via a perfect binary channel, can then *decode* it into a data reconstruction. For example, in image compression, the data source may be an N-dimensional digital image X that takes value in the space of pixels $\mathcal{X} = \{0, 1, ..., 255\}^N$. The sender observes an image x and uses a compression algorithm (such as JPEG) to encode it into a binary file; the same binary file can then be transmitted to a receiver over a digital channel such as the Internet, and decoded back into a reconstruction of the same image.

Thus we see that a basic problem of compression is that of assigning bit-strings to data observations. If the data can always be perfectly reconstructed under the assignment scheme, we say the compression is *lossless*, and otherwise *lossy*. Since the set of bit-strings is countable, lossless compression is only possible when the data is discrete. As we shall see, many lossy compression methods build on lossless compression as a subroutine; this is done by first transforming the data into a discrete representation, which can then be losslessly compressed/received by the receiver and finally transformed into a reconstruction. This chapter will introduce the fundamentals of data compression and its relation to statistical learning, resulting in learned (or neural) compression methods. Most commonly, the assignment from each data observation to a bit-string is a *deterministic* function, often facilitated by quantization. Chapter 42 will instead discuss more advanced methods which use *stochastic* instead of deterministic encoding.

41.2 Lossless compression

Lossless compression aims to represent discrete data with as few bits as possible, while being able to perfectly recover the data later. Formally, we consider the general problem where the source consists of i.i.d. copies of a discrete random variable X taking values in $\mathcal{X}$. To each possible *message* $(x_1, x_2, ...) \in \bigcup_{n=1}^{\infty} \mathcal{X}^n$ (i.e., a realized source sequence), our goal is to assign a unique string of bits (which we call a *bit-string*) while minimizing the overall number of bits required. Such an assignment of messages to bit-strings that allows us to perfectly recover the former from the latter is known as a lossless *source code*. Since the set of all possible bit-strings $\{0, 1\}^* := \{\emptyset, 0, 1, 00, 01, ...\}$

is countable and we require a unique bit-string for every possible message, lossless compression is only possible for a discrete source. The structure of $\mathcal{X}$ is not important for our conceptual discussions, and we can, without loss of generality, identify $\mathcal{X}$ with the set of natural numbers. Still, a reduction in the number of bits is never guaranteed, and lossless compression relies on the fact that certain outcomes of the source are more probable than others and can be assigned shorter bit-strings. Therefore, we cannot expect to compress a "pure noise" source whose outcomes are all equiprobable. Fortunately, real-world sources (such as the English language) often contain structure that makes a small subset of outcomes much more likely than others (e.g., the string "Hello world!" vs. "bwzjnhxiHcBuq").

To quantify the amount of uncertainty in a source, Shannon [1499] made the following definition: for a discrete random variable X with probability mass function (PMF) P_X, its *entropy* is

$$H[X] := \mathbb{E}\left[\log_2 \frac{1}{P_X(X)}\right] \tag{41.1}$$

$$= \sum_{x\in\mathcal{X}} P_X(x) \log_2 \frac{1}{P_X(x)}. \tag{41.2}$$

For each outcome $x \in \mathcal{X}$, the number $\log_2 \frac{1}{P_X(x)}$ is known as *self-information* and quantifies the amount of surprise of observing $X = x$; entropy is therefore the expected value of surprise. For example, the entropy of a fair coin flip — a Bernoulli(0.5) random variable — is 1 bit, and the entropy of a biased coin flip approaches 0 bits as the probability of one of the two outcomes approaches 1. The definition of entropy can be motivated by natural axioms of "information content" [1499] and turns out to delineate the fundamental performance limit of lossless compression. In the rest of this section, we will give a mathematical description of this performance limit and introduce algorithms that aim to approach this limit, known as *entropy coding* algorithms.

41.2.1 Optimal Symbol Code

We first consider codes that assign a bit-string to one letter of the message at a time, i.e., we want to find a *symbol code* $\gamma : \mathcal{X} \to \{0,1\}^*$, and obtain the bit-string for an entire message by concatenation. We call each assigned bit-string $\gamma(x)$ for the symbol $x \in \mathcal{X}$ a *codeword*; we call the number of bits making up $\gamma(x)$ the *code length*, variously denoted by $l(x)$ or $|\gamma(x)|$. For the assignment to be lossless, clearly γ needs to assign a unique codeword to each x, so that the receiver can recover x unambiguously from $\gamma^{-1}(\cdot)$. To allow the efficient decoding of a message from a sequence of concatenated codewords, we impose the further condition that γ is a *prefix-free code*, i.e., no codeword under γ is a prefix of another.

It turns out that any prefix-free code with code lengths l must satisfy the following *Kraft inequality*,

$$\sum_{x\in\mathcal{X}} 2^{-l(x)} \leq 1, \tag{41.3}$$

and conversely, any code-length function l satisfying the above inequality can always be realized by some prefix-free code [337, Theorem 5.2.1].[1] As a consequence, the optimal average code length among all prefix-free (or uniquely decodable) codes is given by the following optimization problem:

$$\begin{aligned} L^*(X) := \min_{\gamma} & \sum_{x\in\mathcal{X}} P_X(x) l(x), \\ \text{s.t.} & \sum_{x\in\mathcal{X}} 2^{-l(x)} \leq 1, \\ & l(x) \in \mathbb{N} \quad \forall x \in \mathcal{X} \end{aligned}$$

By studying this problem, we arrive at the following basic result which bounds the optimal average code length of prefix-free codes in terms of entropy [337, Theorem 5.4.1]:

$$H[X] \leq L^*(X) \leq H[X] + 1 \qquad \text{bits per source sample.} \tag{41.4}$$

The first inequality states that no prefix-free code (or more generally, no *uniquely decodable* code) can achieve an average code length below the source entropy, and the inequality is tight when $l(x) = \log_2 \frac{1}{P_X(x)}$. The second

[1] The result also holds if we replace "prefix-free code" with "uniquely decodable code" (see [337, Chapter 5.5]). Although the former is a subset of the latter, it turns out we do not lose any coding efficiency by considering prefix-free codes only.

inequality says that there always exists a prefix-free code whose average code length is one bit within the entropy; the existence is guaranteed since the code lengths $l(x) = \lceil \log_2 \frac{1}{P_X(x)} \rceil$ (we use the ceiling operator $\lceil \cdot \rceil$ to ensure integer code lengths) satisfy the Kraft inequality. This result has a significant implication: given any discrete distribution Q, an optimal code for $X \sim Q$ always exists whose code lengths are approximately negative log-probabilities $\log_2 \frac{1}{Q(x)}$; we will give an explicit construction in Section 41.2.3. Thus, the problem of finding an optimal lossless compressor reduces to that of finding an optimal coding distribution, which we turn to next.

41.2.2 Lossless Coding and Statistical Estimation

In practice, the true distribution P_X of the source is almost always unknown. We therefore approximate it by a model P_θ, called an *entropy model* or *coding distribution*, and proceed with entropy coding as if the data came from the distribution P_θ. Using a code designed for P_θ with code length $l(x) = \log_2 \frac{1}{P_\theta(x)}$, the expected code length then satisfies [337, Theorem 5.4.3]

$$H[P_X, P_\theta] \leq \mathbb{E}_{x \sim P_X}[l(x)] \leq H[P_X, P_\theta] + 1, \tag{41.5}$$

where

$$H[P_X, P_\theta] \stackrel{\text{def}}{=} \mathbb{E}_{x \sim P_X}\left[\log_2 \frac{1}{P_\theta(x)}\right], \tag{41.6}$$

is the *cross-entropy* between P_X and P_θ and is related to entropy via

$$H[P_X, P_\theta] = H[X] + \mathrm{D}_{\mathrm{KL}}(P_X \| P_\theta) \geq H[X]. \tag{41.7}$$

Above, the Kullback-Leibler (KL) divergence $\mathrm{D}_{\mathrm{KL}}(P_X \| P_\theta) \stackrel{\text{def}}{=} \mathbb{E}_{x \sim P_X}\left[\log_2 \frac{P_X(x)}{P_\theta(x)}\right]$ is always positive unless $P_X = P_\theta$. Thus, coding $X \sim P_X$ using an incorrect coding distribution P_θ incurs an increase in the average description length given by $\mathrm{D}_{\mathrm{KL}}(P_X \| P_\theta)$.

Consequently, fitting a model P_θ by maximum-likelihood estimation, i.e.,

$$\arg\max_{P_\theta} \mathbb{E}_{x \sim P_X}[\log_2 P_\theta(x)] = \arg\min_{P_\theta} \mathbb{E}_{x \sim P_X}[-\log_2 P_\theta(x)], \tag{41.8}$$

is precisely aligned with minimizing the coding cost of data from P_X under an imperfect model P_θ. Note that in practice we only have access to samples from the data distribution P_X, so that the above expectations are approximated by Monte Carlo averages and often optimized using stochastic gradient descent [160].

41.2.3 Huffman Coding

As an example, we review one of the most widely used entropy coding algorithms due to Huffman [803], which achieves the optimal prefix-free code length in (41.4). In a nutshell, given a probability mass function (PMF) P_X over symbols, the algorithm first derives an optimal prefix-free code from a so-called Huffman tree; any symbol can then be encoded and decoded by traversing the tree. Figure 41.1 shows an example Huffman tree, whose leaf nodes are associated with symbols of the alphabet. A Huffman tree is constructed recursively from the leaves to the root by successively merging the two nodes with the lowest probabilities into a new node. By traversing the tree from the root to a leaf, the sequence of branching directions ("0" for left, "1" for right, in our example) assigns a unique binary codeword to each symbol, with the more frequent symbols receiving shorter codewords. Encoding and decoding are simple lookup operations with complexity linear in the tree depth.

Huffman coding assigns a codeword with length at most $\lceil -\log_2 P_X(x) \rceil$ to each symbol x, and can be shown to be optimal among all prefix-free codes [337]. However, as log-probabilities are generally not integers, Huffman coding (as with all so-called *symbol codes*) incurs an overhead of up to 1 bit per symbol, relative to the information content. For example, a heavily biased coin with entropy close to 0 will still require 1 bit to encode.

One way of reducing this overhead of symbol codes is by spreading the overhead over a group of symbols.[2] Given a message $(x_1, x_2, ...)$, we consider transmitting it in length-n blocks with a symbol code, effectively treating the source as i.i.d. length-n blocks with a new alphabet $\mathcal{X}^n$. The bound on the optimal prefix-free code length (Eq. (41.4)), averaged over each symbol, then becomes

$$\frac{1}{n} H[X_1, ..., X_n] \leq \frac{1}{n} L^*([X_1, ..., X_n]) \leq \frac{1}{n} H[X_1, ..., X_n] + \frac{1}{n}. \tag{41.9}$$

[2] Indeed, Shannon [1499, Theorem 9] already used block coding and the law of large numbers to show that an i.i.d. source can be losslessly compressed with an average code length arbitrarily close to the entropy. However, it was not until decades later with the invention of arithmetic coding that an algorithm made this idea a practical reality.

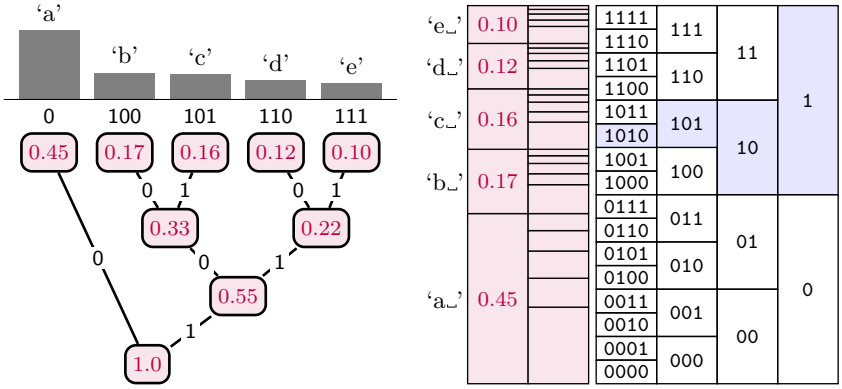


Figure 41.1: *Left:* A Huffman tree over five symbols. The numbers in boxes denote probabilities, while binary strings above the leaf nodes correspond to the codewords assigned by the Huffman algorithm. *Right:* Arithmetic coding of a message of length two. Each message corresponds to an interval whose length is proportional to the message's probability. As an example, any real number inside the interval corresponding to the message 'ca' begins with `0.1010...` when written in binary form so that we can use `1010` to uniquely encode it. The message 'aa' would be encoded as `000`.

Thus, the 1 bit overhead on the RHS of (41.4) is reduced to $\frac{1}{n}$ and can be made arbitrarily low by increasing n. Note that in the usual case where $X_1, X_2, ...$ are i.i.d., the joint entropy factorizes so that $\frac{1}{n}H[X_1, ..., X_n] = H[X]$; otherwise, this block coding scheme can exploit the redundancy between the X_is and do better than coding one symbol at a time, since

$$H[X_1, ..., X_n] \leq \sum_{j=1}^{n} H[X_j],$$

for any n random objects $(X_1, ..., X_n)$.[3]

Despite the theoretically improved coding cost, directly applying a symbol code like Huffman code over long blocks is often computationally infeasible, as the size of the required PMF table for $P_{X_1,...,X_n}$ grows exponentially with the block length. It turns out that *stream codes* [1111, Chapter 6] to be discussed below will allow us to implement the idea behind block coding much more efficiently, while also offering a convenient mechanism for adapting our model to the source (we discussed modelling in Section 41.2.2) *during* compression.

41.2.4 Arithmetic Coding

Arithmetic coding [1400], also known as range coding [1137], is an example of a *stream code* [1111, Chapter 6]. Stream codes differ from symbol codes in that they assign codewords to entire messages and individual symbols do not have unique codewords. Stream codes amortize the coding cost's overhead across the whole sequence of symbols and are therefore able to get closer to the entropy.

The basic idea of arithmetic coding is to associate each symbol with a subinterval of the interval $[0, 1]$ such that the subinterval's length equals the symbol's probability. This procedure can be generalized towards encoding symbols in sequence. For two symbols, the second symbol is no longer considered a subinterval of $[0, 1]$ but a subinterval of the previous symbol's interval such that the second subinterval's length equals the two symbols' joint probability. For example, if symbol "a" has probability $1/5$ and is associated with $[0, 0.2)$, the sequence "aa" would be encoded as $[0, 0.04)$. This procedure can be iterated for sequences of arbitrary length, leading to a sequence of subintervals of decreasing size that contain each other. Any real-valued number contained in the final subinterval allows us to uniquely reconstruct the sequence of symbols and therefore encodes the entire sequence (Figure 41.1). In practice, one picks a representative number which can be represented with the smallest number of bits while uniquely identifying the interval. By construction, the size of the final sub-interval is the product of all previous symbols' conditional probabilities, hence the probability of the sequence. Intuitively, the interval's length (i.e., the sequence's joint probability) determines the number of relevant digits of the number representing the interval, thus the code length equals the log-probability of the sequence as required for entropy codes. If $x^n = (x_1, x_2, ..., x_n)$ is a message of length n, and $\gamma(x^n)$ the codeword assigned by arithmetic coding using a

[3] In information theory, the most general representation of a source is a stochastic process $(X)_{i=1}^{\infty}$, and $\lim_{n\to\infty} \frac{1}{n}H[X_1, ..., X_n]$ defines the *entropy rate* of the process, which generalizes entropy (41.2) to non-i.i.d. sources.

probability model P_θ, then the assigned code length satisfies [1340, Chapter 13.1],

$$-\log_2 P_\theta(x^n) \leq |\gamma(x^n)| < \lceil -\log_2 P_\theta(x^n) \rceil + 1.$$

Thus if the true distribution of messages is P_{X^n}, then the average code length is at most $H[P_{X^n}, P_\theta] + 2$. The excess bits needed for encoding the message are amortized over all n symbols, making arithmetic coding more efficient on long messages than symbol codes. Per symbol, the overhead is only $2/n$ bits, compared to 1 bit per symbol for Huffman coding.

In arithmetic coding, the probability model $P_\theta(x^n)$ is only specified sequentially via the marginal $P_\theta(x_1)$ and conditional distributions $\{P_\theta(x_i|x_{<i}) : i = 2, ..., n\}$, thereby avoiding the exponential complexity of block coding (see discussion in the previous subsection). Moreover, a simple modification allows arithmetic coding to adapt its probability model "on the fly" on the data being compressed. The idea is as follows. Before processing an input message x^n, both the sender and receiver start with the same copy of the marginal model $P_\theta(x_1)$. At each step i, the sender updates its model distribution $P_\theta(x_i|x_{<i})$ based on the encoded sequence $x_{<i}$ so far, and then encode x_i under the updated model; this is repeated for $i = 2, ..., n$, until a final bit-string is produced. The receiver then decodes the message from the bit-string as usual, except the exact same sequence of model updates are also performed based on the decoded sequence $x_{<i}$ at each time step $i = 2, ..., n$; this is to ensure the same model $P_\theta(x_i|x_{<i})$ is used during both encoding and decoding. Adaptive arithmetic coding via sequential density estimation is thus an example of a *universal* source code [337, Chapter 13], in the sense that the method can be applied without prior knowledge of the source distribution while still being asymptotically optimal for a large class of sources (e.g., all finite-order Markov chains).

Notably, arithmetic coding implements a first-in-first-out data structure, that is, a queue. Symbols which are encoded first are also decoded first. As we will see next, this makes it a convenient choice for compression with autoregressive models.

41.2.5 Neural Lossless Compression

As we saw in Section 41.2.3, an entropy code assigns a sequence of bits to each possible value x in the alphabet based on the PMF P_X, so the size of the alphabet (i.e., the set of all possible symbols) needs to be manageable. When working with high-dimensional sources such as images, naively representing the data distribution as a PMF table becomes infeasible, as the number of table entries increases exponentially with the dimensionality. Additionally, we would like our model P_θ to be as close as possible to the true distribution P_X (in KL divergence), so as to achieve the optimal lossless compression cost (see Eq. (41.8)). Fortunately, deep likelihood-based models parametrised by neural networks, such as autoregressive models [115, 975, 1655] and variational autoencoders [892, 1391], provide highly flexible and accurate models of the data distribution without working with the intractable PMF table. By decomposing a high-dimensional distribution into lower-dimensional conditional distributions, such likelihood-based models also provide means for tractable entropy coding using existing entropy codes such as arithmetic coding. Thus neural lossless compression, i.e., lossless compression with a neural probability model, generally follows a two-step recipe:

1. Estimate a probability model P_θ from samples of the true data distribution P_X by maximizing likelihood (Eq. (41.8)) or a surrogate criterion.

2. Compress or decompress any new data observation x with an entropy coding algorithm (e.g., arithmetic coding), using P_θ as the entropy model.

Below we instantiate this recipe with two choices of models: autoregressive models and latent-variable models.

Neural Lossless Compression with Autoregressive Models

An autoregressive model P_θ exploits the fact that we can write any probability distribution as a product of conditional distributions using the chain rule of probabilities [160],

$$P_\theta(x) = \prod_i P_\theta(x_i \mid x_{<i}). \tag{41.10}$$

Here, x_i is the i-th entry of the vector x and $x_{<i}$ corresponds to all previous entries in an arbitrary order. The autoregressive factorization does not make any assumptions about the data distribution yet still allows us to easily incorporate useful assumptions. For example, a Markov assumption can be implemented by conditioning only on a subset of entries in $x_{<i}$ that are closest to i. A stationarity assumption is easily incorporated by using the

same conditional distribution $P_\theta(x_i \mid x_{<i})$ at every location. These two assumptions are often reasonable and can drastically reduce the amount of parameters of a model.

Autoregressive modelling lends itself naturally to lossless compression in combination with arithmetic coding (Section 41.2.4) since both deal with data sequentially. Given a pretrained model P_θ, we can encode (decode) a given data vector x one symbol at a time, where at each step we encode (decode) x_i given the previous $x_{<i}$ using arithmetic coding, feeding it the model's conditional distribution $P_\theta(x_i \mid x_{<i})$. Arithmetic coding is particularly suited for this task as it operates like a queue — symbols encoded earlier will also be decoded earlier, and therefore will be available as input to the model for computing conditional distributions.

Autoregressive models have achieved strong likelihoods for various data modalities such as images, audio, and text, making them powerful lossless data compressors in principle. However, they come with the restriction that decoding (as with sampling) is an inherently sequential procedure: each symbol can only be decoded after all the symbols have been decoded on which its prediction depends. This can make them impractically slow at compressing high-dimensional signals such as images, and recent research [779, 1385] has sought to address this limitation.

Neural Lossless Compression with Latent Variable Models

A latent-variable model represents the data distribution using the sum rule of probability,

$$P_\theta(x) = \sum_{z \in \mathcal{Z}} P^\theta_{X|Z=z}(x) P^\theta_Z(z),$$

where $\mathcal{Z}$ is a latent space, P^θ_Z is a distribution over $\mathcal{Z}$, $P^\theta_{X|Z}$ is a conditional distribution (i.e., $P^\theta_{X|Z=z}$ is a distribution over $\mathcal{X}$ for each value of z), and we refer to P^θ_Z and $P^\theta_{X|Z}$ as the *prior* and *(conditional) likelihood* models in analogy to Bayesian statistics. Unlike in an autoregressive model, evaluating the model probability here is not so straightforward as it involves an expensive marginalization over the latent space. We first focus on the more basic question of how a given data point x can be entropy-coded efficiently with a code length of $\approx -\log_2 P_\theta(x)$, i.e., its self-information, assuming a pretrained latent variable model. The coding procedure will then suggest a training procedure that allows us to learn an effective neural compressor from data.

As a first attempt, we will implement lossless compression using a so-called *two-part code* [676]. Given x, the sender first decides on a latent representation z^* computed with an encoding function $f : \mathcal{X} \to \mathcal{Z}$; then z^* is encoded under the prior P^θ_Z, and x is encoded under the likelihood model $P^\theta_{X|Z=z^*}$. Upon receiving the encoded bit-streams, the receiver decodes z^* and then x using their copy of the same models. The overall code length is therefore $-\log P^\theta_{X|Z=z^*}(x) - \log P^\theta_Z(z^*)$, and we can optimize this compression scheme on the expected code length,

$$\mathbb{E}_{x \sim P_X, z=f(x)}[-\log P^\theta_{X|Z=z}(x) - \log P^\theta_Z(z)], \tag{41.11}$$

with respect to our choices of encoder f, prior P^θ_Z, and likelihood $P^\theta_{X|Z}$ models. Two comments are in order. First, unlike in standard Bayesian statistics, here we *learn* the prior P^θ_Z so as to optimize the compression performance. The relevant term in the objective is $\mathbb{E}_{x \sim P_X, z=f(x)}[-\log P^\theta_Z(z)]$, which is equal to the cross-entropy $H[P_Z, P^\theta_Z]$ where $P_Z(k) \stackrel{\text{def}}{=} \mathbb{P}(f(X) = k) = \sum_{x:f(x)=k} P_X(x)$ is the marginal distribution of the encoding $Z \stackrel{\text{def}}{=} f(X)$ and equals the pushforward of P_X under f. Thus the optimal prior is P_Z, as we saw in Section 41.2.2. Second, to allow for entropy coding, the latent space $\mathcal{Z}$ must be discrete. If we represent f with a neural network or some other smooth function approximator ψ, then we must *quantize* its output to obtain a discrete latent representation, i.e., $z = [\![\psi(x)]\!]$, where $[\![\cdot]\!]$ denotes quantization, e.g., element-wise rounding to a predefined grid. As quantization has zero gradient almost everywhere, a differentiable approximation is typically used for gradient-based training, which we discuss in more depth in Section 41.3.2. As an example, Mentzer et al. [1167] train a two-part-code model $(\psi, P^\theta_Z, P^\theta_{X|Z})$ end-to-end on Eq. (41.11) for image compression. The model resembles a hierarchical autoencoder, in which ψ is a downsampling convolutional neural network, $z = [\![\psi(x)]\!]$ is a collection of convolutional feature tensors at decreasing spatial resolutions, and $P^\theta_Z(z) = \prod_l P^\theta(z_l|z_{<l})$ and $P^\theta_{X|Z}$ are autoregressive models parametrised by an upsampling convolutional neural network. Notably, each of the conditional distributions that make up P^θ_Z or $P^\theta_{X|Z}$ is fully factorized across pixel locations, resulting in orders of magnitude faster decoding than pixel-space autoregressive models like PixelCNN++ [1452].

However, even when optimized end-to-end, the coding cost of a two-part code is generally suboptimal [561, 776].

To see this, fix a data point x and consider the following upper bound on the self-information,

$$\begin{aligned} -\log P_\theta(x) &\le -\log P_\theta(x) + \mathrm{D}_{\mathrm{KL}}\Big(Q_{Z|X=x}\Big\|P^\theta_{Z|X=x}\Big) \\ &= \mathbb{E}_{z\sim Q_{Z|X=x}}[-\log P^\theta_Z(z) - \log P^\theta_{X|Z=z}(x) + \log Q_{Z|X=x}(z)], \end{aligned} \tag{41.12}$$

where $Q_{Z|X=x}$ is any *variational posterior* distribution, $P^\theta_{Z|X=x}(z) \propto P^\theta_Z(z)P^\theta_{X|Z=z}(x)$ is the Bayesian posterior, and Eq. (41.12) is known as the negative *ELBO* (Evidence Lower BOund) [170]. The two-part code corresponds to the special case where $Q_{Z|X=x} = \delta_{z^*}(x)$ is a degenerate distribution concentrated on the encoding $z^* = f(x)$, and the negative ELBO exactly equals the two-part code length $-\log P^\theta_{X|Z=z^*}(x) - \log P^\theta_Z(z^*)$. Thus, for a given choice of prior and likelihood models, we can in principle improve the coding cost of a two-part code by allowing ourselves the possibility of a *stochastic encoder* $Q_{Z|X}$, ideally the Bayesian posterior. This prompts the question "does there exist a compression scheme that can make use of a stochastic encoder $Q_{Z|X}$ and achieve a coding cost equal to the negative ELBO?" If so, we can then reduce the problem of optimal compression to that of posterior inference (Chapter 12), and the problem of learning such a compressor from data samples to that of maximizing the expected ELBO. Fortunately the answer is yes, and is given by the *bits-back coding* algorithm [560, 751] which we explain in Section 42.1.

41.3 Lossy Compression

In lossy compression, information in the data is irrevocably discarded, allowing us to trade away perfect reconstruction for a lower bit-rate than what is possible with lossless compression. Mathematically, a lossy compression procedure can be described by the following Markov chain

$$X \to Z \to \hat{X}.$$

Here, the source random variable $X \in \mathcal{X}$ is only available to the sender, who decides on a compressed *representation* $Z \in \mathcal{Z}$ from a representation space $\mathcal{Z}$ and transmits it to the receiver. The receiver then produces a *reconstruction* $\hat{X} \in \hat{\mathcal{X}}$ on the basis of Z. In this chapter, we focus on the classical setting where Z is discrete (so $\mathcal{Z}$ is countable and can be identified with the set of natural numbers without loss of generality) and the $X \to Z$ process is deterministic, i.e., $Z = f(X)$ for some function f; we treat the case of a stochastic encoder for continuous compressed representations in Section 42.2.2. For simplicity (and without loss of optimality, as it turns out), we can represent $Z \to \hat{X}$ also by a deterministic function g which we call the decoding transform, so that $\hat{X} = g(Z)$.

As in lossless compression, we are interested in transmitting the source $X \sim P_X$ using the fewest number of bits. Commonly, this is done by entropy-coding Z under an entropy model P^θ_Z, so that the expected bit-rate is the cross-entropy (see Section 41.2):

$$\mathcal{R}(f, P^\theta_Z) = \mathbb{E}_{x\sim P_X, z=f(x)}[-\log P^\theta_Z(z)] = \mathbb{E}_{z\sim P_Z}[-\log P^\theta_Z(z)] = H[P_Z, P^\theta_Z]. \tag{41.13}$$

Exactly as in Eq. (41.11), the optimal choice of P^θ_Z is the marginal encoding distribution $P_Z(k) \stackrel{\text{def}}{=} \mathbb{P}(f(X) = k)$, for which the cross-entropy equals the entropy $H[Z]$. Additionally, we want to ensure that the lossy reconstruction preserves useful information in the original data, according to some criteria for "usefulness". We make the standard choice of requiring a small average *distortion*,

$$\mathcal{D}(f, g) = \mathbb{E}\Big[\rho(X, \hat{X})\Big] = \mathbb{E}[\rho(X, g(f(X)))], \tag{41.14}$$

where $\rho : \mathcal{X} \times \hat{\mathcal{X}} \to [0, \infty)$ is a *distortion metric* which measures the deviation between a source and reconstruction pair, e.g., Hamming distance in the discrete case or squared error in the continuous case. Combining the above, we can evaluate the effectiveness of a lossy compression algorithm based on its *rate-distortion* (or "R-D") performance given by the tuple $(\mathcal{D}, \mathcal{R})$, and our goal in lossy compression is to simultaneously minimize both costs with respect to our choice of the encoding, decoding functions (f, g) and the entropy model P^θ_Z. For a given source P_X and distortion metric ρ, what is the best possible rate-distortion performance one can expect? This is a key question in the theory of lossy compression known as *rate-distortion theory* [124, 1500], and the answer is given by a fundamental quantity $R(D)$ known as the *rate-distortion function*, or R-D function for short. For a given source X, distortion metric ρ, and threshold $D \ge 0$ of acceptable distortion, $R(D)$ equals the smallest possible bit-rate (bits per source sample) required to compress the source with an average distortion not exceeding D, regardless of the computation complexity of the compression procedure. The R-D function is defined by [1500]

$$R(D) \stackrel{\text{def}}{=} \inf_{Q_{\hat{X}|X}:\mathbb{E}_{P_X\times Q_{\hat{X}|X}}[\rho(X,\hat{X})]\le D} I[X; \hat{X}], \tag{41.15}$$

where $Q_{\hat{X}|X}$ is any Markov kernel from $\mathcal{X}$ to $\hat{\mathcal{X}}$ (so that $Q_{\hat{X}|X=x}$ is a probability measure on $\hat{\mathcal{X}}$ for every $x \in \mathcal{X}$), and $I[X;\hat{X}]$ denotes the mutual information under the joint distribution $P_X \times Q_{\hat{X}|X}$, defined as the KL divergence between the joint and the products of its two marginals. We can conceptualize each $Q_{\hat{X}|X}$ as a black-box lossy compressor that maps any input x stochastically to an output $\hat{X} \sim Q_{\hat{X}|X=x}$, and $R(D)$ is obtained by minimizing the mutual information over all $Q_{\hat{X}|X}$ whose expected distortion is within the threshold D. In the same way entropy (Eq. (41.2)) characterizes the fundamental performance limit of lossless compression, the R-D function characterizes the fundamental performance limit of *lossy* compression.

Compared to lossless compression with entropy coding (Section 41.2), achieving the optimal R-D performance at a reasonable complexity has proved more difficult in practice. The remaining section will discuss various lossy compression algorithms with increasing sophistication. Instead of fixing a distortion threshold D and performing constrained optimization as in Eq. (41.15), it is often more convenient to minimize a scalar combination of both costs called the R-D Lagrangian [46, 167, 298][4]

$$\mathcal{L}(f, g, P_Z^\theta; \lambda) = \mathcal{R}(f, P_Z^\theta) + \lambda \mathcal{D}(f, g), \tag{41.16}$$

where $\lambda \geq 0$ is a hyperparameter parametrising the R-D trade-off. Assuming perfect optimization, minimizing the above for a fixed λ results in a lossy compressor (f, g, P_Z^θ) whose R-D performance $(\mathcal{D}, \mathcal{R})$ is Pareto-optimal, and repeating this procedure for different λ values traces out a Pareto frontier commonly known as an *R-D curve*.

41.3.1 From Vector Quantization to Transform Coding

We now illustrate the general setup by deriving a basic lossy compression method, vector quantization (VQ). Let the data and reconstructions be d-dimensional real vectors ($\mathcal{X} = \hat{\mathcal{X}} = \mathbb{R}^d$) and the representation Z be some integer in $\mathcal{Z} = \{1, 2, ..., K\}$ for some $K \in \mathbb{N}^+$. Suppose we transmit Z under a uniform entropy model; this can be done by transmitting $Z - 1$ in binary using at most $\lceil \log_2 K \rceil$ bits; e.g., we use the codewords $\{0_2 = 00, 1_2 = 01, 2_2 = 10\}$ if $K = 3$. The rate loss (Eq. (41.13)) is thus constant, and we seek encoding and decoding maps (f, g) that minimize the distortion (Eq. (41.14)) in terms of squared error. The solution is given by the coupled equations below

$$f(x) = \arg \min_{z \in \{1,2,...,K\}} \|x - g(z)\|^2, \quad \forall x \in \mathcal{X}, \tag{41.17}$$

$$g(z) = \mathbb{E}[X | f(X) = z], \quad z = 1, 2, ..., K. \tag{41.18}$$

We have essentially recovered the fixed-point iterations of the *Lloyd-Max* or *k-means* algorithm [1071]. The decoder g essentially serves as a lookup table whose outputs $\mu_k \stackrel{\text{def}}{=} g(k), k \in \mathcal{Z}$ have the common interpretation of *codebook vectors* or *cluster centroids*, and the resulting VQ algorithm maps any given x to its nearest cluster centroid $\hat{x} = g(f(x)) = \mu_{f(x)}$. Instead of fixing the entropy model, we can also incorporate its choice into the design of VQ, resulting in *entropy-constrained VQ* [298]. By optimizing the resulting R-D Lagrangian (Eq. (41.16)), the optimal compressor satisfies similar conditions to the above where the encoding map additionally takes into account the code length associated with each cluster index, the decoding map remains the same, and the optimal entropy model is given by the marginal distribution of the encoding $Z = f(X)$ (same as in Eq. (41.11)).

As with stream codes in lossless compression, by compressing data in increasingly long messages or blocks [5] VQ can be shown to asymptotically achieve the optimal R-D performance given by the R-D function (Eq. (41.15)) [337]. However, this approach runs into the curse of dimensionality: as the block length (or data dimensionality) increases, an exponentially large number of codebook vectors is required to represent the source distribution well, making it exponentially more expensive to store the codebook vectors and to perform the nearest-neighbour search (Eq. (41.17)) to encode a single data point. As a result, VQ is typically only applied to compression problems where a relatively small K (and therefore low bit-rate) suffices, such as speech coding [1465].

Instead of vector quantization, we may consider *scalar quantization* where each dimension of the data vector is separately quantized and then entropy-coded, ignoring the statistical dependencies between the different dimensions. This is computationally much cheaper, albeit at the cost of much worse R-D performance and poor reconstructions which violate the statistical properties of the data. As a middle ground, we can first transform the data into a suitable representation with ideally independent dimensions, and then apply scalar quantization much more effectively. This is the approach taken by *transform coding* [655], and is the basis of virtually all

[4] Strictly speaking, Eq. (41.16) corresponds to the Lagrangian relaxation of a constrained optimization problem where we minimize the bit-rate $\mathcal{R}$ subject to a hard constraint on the distortion $\mathcal{D}$ (as in the definition of the R-D function, Eq. (41.15)), up to a constant that has no effect on the optimization.

[5] We extend the distortion metric to operate on length n blocks by considering the per-block average distortion: $\rho(x^n, \hat{x}^n) \stackrel{\text{def}}{=} {}^1\!/_n \sum_i \rho(x_i, \hat{x}_i)$.

popular compression algorithms for digital media today, such as images, audio, and video. In transform coding, a data source X is first transformed into a vector of *transform coefficients* $Y = \psi(X)$ via an *analysis transform* ψ; then the coefficients are quantized, typically by coordinate-wise rounding, to obtain a discrete representation $Z = \lfloor Y \rceil$, so the overall encoding map is $f = \lfloor \cdot \rceil \circ \psi$. Traditionally, the analysis transform is chosen to be linear and invertible, so that the decoding map, also called the *synthesis transform*, is simply the inverse $g = \psi^{-1}$. A classic example is the Karhunen-Loève Transform (KLT), equivalently computed by Principal Component Analysis, which maps a given data point into the eigen-basis of the covariance matrix of X, resulting in decorrelated transform coefficients. Another example is the discrete cosine transform (DCT), which is shown to empirically approximate KLT asymptotically [14] and is one of the most widely used transforms in signal processing and compression (e.g., in the JPEG algorithm). The choice of a linear (and often orthogonal) transform simplifies the theoretical analysis and design of transform coding algorithms, with the KLT shown to be optimal for Gaussian sources [656].

Compared to VQ whose only effective constraint is the cardinality of the set of discrete representations, transform coding introduces additional structure into the design of the lossy compressor. As a result, transform coding generally cannot achieve the information-theoretically optimal R-D performance, but the lack of theoretical optimality is more than compensated for by its vastly improved computation efficiency over VQ, as evidenced by its wide-spread application.

41.3.2 Neural Lossy Compression

Having introduced the general problem of lossy compression and two prototypical algorithms, we now consider *neural lossy compression*, i.e., lossy compression using deep learning tools such as neural networks, typically learned end-to-end on the source data samples. Compared to earlier algorithms such as those based on linear transform coding, neural lossy compression can be optimized on custom performance metrics and better adapt to the source distribution, thereby offering more flexibility and improved compression performance [88]. Although neural networks have been used for lossy image compression since before the 1990s [844, 1537], techniques only emerged in the last decade that allowed them to be effectively trained end-to-end on a rate-distortion loss [83, 1596, 1618]. A major stumbling block has to do with *quantization*, by which we mean any function that maps from a continuous domain to a discrete one with countably (and often finitely) many values; an example is the VQ encoding map in Eq. (41.17). As neural networks and other smooth function approximators produce real-valued outputs, quantization is required to obtain discrete representations that can be entropy-coded. However, quantization does not have a useful derivative (zero almost everywhere), and the rate loss (Eq. (41.13)) is defined only with respect to discrete representations, which poses a significant challenge to gradient-based optimization. Different neural lossy compression methods over the years have tackled the challenge of quantization in different ways, including stochastic binarization [1618], straight-through estimator [1032], and continuous relaxations to VQ [12, 1166, 1805]. One approach based on uniform quantization [83, 1596] has proved particularly effective and become well-established. We will focus on this approach, known as *nonlinear transform coding* (NTC) since it generalizes transform coding (Section 41.3.1) to nonlinear transforms. We refer the reader to [1808, Section 3.3] and [88] for a comprehensive review.

In nonlinear transform coding, the lossy compressor has the same structure as in transform coding, where the discrete representation $Z = f(X) = \lfloor \psi(X) \rceil$ is obtained by rounding the output of an analysis transform ψ, and the reconstruction $\hat{X} = g(Z)$ is decoded using a synthesis transform g. However, instead of linear ψ and g, we now consider richer, nonlinear transforms parametrised by deep neural networks, and also adopt a neural density estimator for the entropy model P_Z^θ similarly to in neural lossless compression (Section 41.2.5). Note that the assumption of a fixed quantization grid with bin widths equal to one is without loss of generality, assuming a sufficiently powerful pair of nonlinear transforms [84]. The neural network architectures for the nonlinear transforms vary by data modality and may incorporate domain knowledge in their design choices. For example, in image compression the transforms are commonly convolution neural networks [84, 85, 288, 1804] or vision transformers [1881], with spatial downsampling in the analysis and upsampling in the synthesis transforms. As another example, in low-latency video compression [13, 1084, 1802] the transforms receive the reconstruction of the previous video frame in addition to the current frame as input, and therefore can be seen as recurrent neural networks.

To address the challenge of end-to-end optimization posed by quantization, we replace rounding with additive uniform noise and consider a suitable choice of entropy model [83]. In particular, instead of the quantized representation $Z = \lfloor Y \rceil \in \mathbb{Z}^m$ where $Y = \psi(X) \in \mathbb{R}^m$ is the vector of transform coefficients, consider the continuous "noisy" representation perturbed by uniform noise,

$$\tilde{Z} = Y + U, \quad U \sim \mathcal{U}([-0.5, 0.5]^m). \tag{41.19}$$

Here U is an independent random vector with uniform density on the unit hypercube $[-0.5, 0.5]^m$, which has the same width (one) as the quantization bin. Under certain conditions, independent uniform noise approximates the statistical properties of quantization error $Y - \lfloor Y \rceil$, and additive uniform noise has been commonly used as a model for quantization in signal processing [118, 661].

Replacing Z by $\tilde{Z}$, the distortion loss (Eq. (41.14)) now becomes

$$\tilde{\mathcal{D}}(\psi, g) = \mathbb{E}\Big[\rho(X, g(\tilde{Z}))\Big] \stackrel{\text{def}}{=} \mathbb{E}_{x\sim P_X, y=\psi(x), u\sim\mathcal{U}([-0.5,0.5]^m)}[\rho(x, g(y+u))], \tag{41.20}$$

which is now differentiable with respect to parameters of the encoding (and decoding) transform. However, the rate loss (Eq. (41.13)) is still only defined in terms of Z. It turns out that the additive uniform noise approximation also suggests a useful differentiable proxy for the rate loss. Denote the densities of $Y, \tilde{Z}$ and U by $p_Y, p_{\tilde{Z}}$ and p_U respectively, then the distributions of Z and $\tilde{Z}$ can be written in terms of p_Y as

$$P_Z(z) = \int_{[-0.5,0.5]^m} p_Y(z+y)dy, \quad \forall z \in \mathbb{Z}^m, \tag{41.21}$$

$$p_{\tilde{Z}} = p_Y * p_U, \tag{41.22}$$

where $*$ denotes convolution and the density p_U is simply the indicator function for the unit hypercube $[-0.5, 0.5]^m$. Crucially, as illustrated in Fig. 41.2, the density of $\tilde{Z}$ interpolates the probability masses of Z on all integer grid points, and thus can be seen as a continuous relaxation of the latter. This observation suggests the following strategy for entropy modelling: instead of parametrising and learning P_Z (a high-dimension discrete distribution) directly, we learn a continuous density model $p^\theta_{\tilde{Z}}$ of the same form as $p_{\tilde{Z}}$; we can then easily obtain an entropy model P^θ_Z by evaluating $p^\theta_{\tilde{Z}}$ on integer grid points. Thus, we choose

$$P^\theta_Z(z) = \int_{[-0.5,0.5]^m} p_\theta(z+y)dy, \quad \forall z \in \mathbb{Z}^m, \tag{41.23}$$

$$p^\theta_{\tilde{Z}} = p_\theta * p_U, \tag{41.24}$$

where p_θ is our choice of a neural density model over $\mathbb{R}^m$; we will discuss its choice in more detail below. Replacing Z by $\tilde{Z}$ and P^θ_Z by $p^\theta_{\tilde{Z}}$ in the rate loss (Eq. (41.13)), we obtain the following proxy rate loss which reduces to the cross-entropy between $p_{\tilde{Z}}$ and $p^\theta_{\tilde{Z}}$:

$$\tilde{\mathcal{R}} \stackrel{\text{def}}{=} \mathbb{E}_{X\sim P_X, u\sim p_U, \tilde{z}=\psi(x)+u}[-\log p^\theta_{\tilde{Z}}(\tilde{z})] = \mathbb{E}_{\tilde{z}\sim p_{\tilde{Z}}}[-\log p^\theta_{\tilde{Z}}(\tilde{z})]. \tag{41.25}$$

Combining Eqs. (41.20) and (41.25), the overall proxy R-D Lagrangian loss for nonlinear transform coding is then given by

$$\mathcal{L}_{NTC} \stackrel{\text{def}}{=} \mathbb{E}_{X\sim P_X, u\sim p_U, \tilde{z}=\psi(x)+u}[-\log p^\theta_{\tilde{Z}}(\tilde{z}) + \lambda\rho(x, g(\tilde{z}))], \tag{41.26}$$

which is differentiable with respect to the parameters of all model components (ψ, g, p_θ) and can be optimized end-to-end on data samples by stochastic gradient descent. After training, new data can then be compressed and decompressed following the same "analysis → round → entropy-code → synthesis" procedure as in transform coding. This results in a gap between additive noise during training and uniform quantization at test time. One way to reduce this gap is to directly optimize the choice of the discrete representation for each given data instance [1805], which is rooted in iterative amortized inference [1133] and variational optimization [1553]. Alternatively, at test time, one may replace uniform quantization with *dithered quantization* [11], i.e., quantization with a random offset; this essentially performs lossy compression with a stochastic code and will be discussed in Section 42.2.

We now elaborate on the choice of the entropy model further. Note that evaluating P^θ_Z at test time for entropy coding (and similarly $p^\theta_{\tilde{Z}}$ during training) still requires solving an integral (Eqs. (41.23), (41.24)) and can be computationally expensive. Therefore, in practice p_θ is chosen to be fully factorized so that the m-dimensional integral reduces to a product of univariate integrals. We can further avoid computing the univariate integrals by parametrising and learning a cumulative distribution function (CDF) instead of the density p_θ [85]: if $F_\theta : \mathbb{R} \to [0, 1]$ is differentiable, monotonically increasing, and satisfies $\lim_{\xi\to-\infty} F_\theta(\xi) = 0$ and $\lim_{\xi\to\infty} F_\theta(\xi) = 1$, then its derivative $\frac{\partial F_\theta(\xi)}{\partial\xi}$ is a valid probability density function. Parametrising the distribution of the ith dimension by a CDF model $F^i_\theta(\xi)$, then $P^\theta_{Z_i}$ (and similarly $p^\theta_{\tilde{Z}_i}$) can be conveniently computed by subtraction

$$P^\theta_{\tilde{Z}_i}(\xi) = F^i_\theta(\xi + 0.5) - F^i_\theta(\xi - 0.5). \tag{41.27}$$

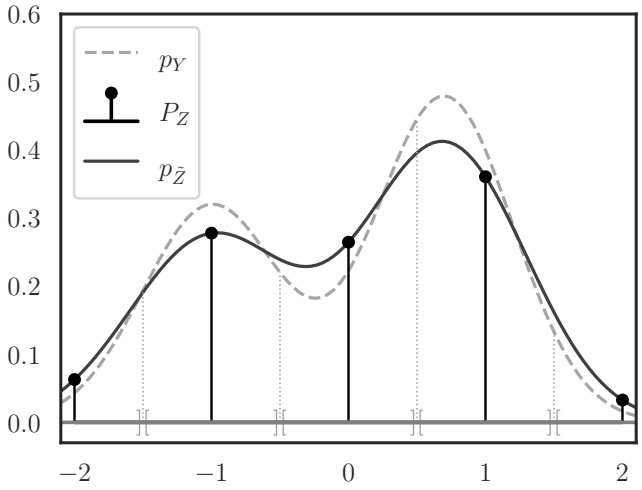


Figure 41.2: Relationship between the distributions of transform coefficients p_Y (dashed curve), quantized coefficients P_Z (vertical "lollipop" stems), and noise-perturbed coefficients $p_{\tilde{Z}}$ (solid curve), in one dimension. P_Z consists of a train of Dirac delta peaks, where the height of each peak is the probability of Y being quantized to the corresponding integer. $p_{\tilde{Z}}$ is obtained by convolving p_Y with uniform noise, and smoothly interpolates P_Z between all integer values.

Some common choices for F_θ are weighted combinations of logistic sigmoid functions [1452] and MLPs with suitable constraints [85]. To increase the expressiveness of the entropy model, state-of-the-art nonlinear transform coding approaches go beyond a fully factorized model and incorporate hierarchical modelling [85, 1183, 1809] and/or autoregressive modelling [1183, 1351].

41.3.3 Connection with latent variable models

There is a natural connection between lossy compression and latent variable modelling. Analogously to in lossless compression Section 41.2.5, we can give a lossy compression interpretation to a family of latent variable models trained by ELBO maximization. In some cases the correspondence is exact and the compression performance implied by the ELBO can be achieved with deterministic encoding, whereas more generally such models optimize an upper bound on the rate-distortion function and can be turned into lossy compressors with the more advanced techniques in Chapter 42.

Gaussian mixture model

As a first example, consider a Gaussian mixture model given by a prior model P_Z^θ and conditional likelihood model with density $p_\theta(x|z)$. Suppose the latent space $\mathcal{Z}$ consists of integers $\{1, 2, ..., K\}$ and P_Z^θ is a categorical distribution parametrised by a probability vector $w \in \Delta^{K-1}$, and the likelihood model is given by a conditional density

$$p_\theta(x|k) = \mathcal{N}(x|\mu_k, \sigma^2 I), \quad k = 1, 2, ..., K, \tag{41.28}$$

where $\{\mu_k \in \mathbb{R}^d : k = 1, 2, ..., K\}$ are the means of the component Gaussians and $\sigma^2 I$ is a shared covariance matrix. We consider σ^2 fixed, so that the overall parameter tensor consists of $\theta = [w; \mu_1, ..., \mu_k]$. The mixture model defines a marginal density over $\mathbb{R}^d$ via

$$p_\theta(x) = \sum_z P_Z^\theta(z) p_\theta(x|z) = \sum_{k=1}^{K} w_k \mathcal{N}(x|\mu_k, \sigma^2 I). \tag{41.29}$$

To estimate the model, it is common to minimize the negative ELBO ("NELBO"):

$$\mathbb{E}_{x\sim P_X}[-\log p_\theta(x)] \le \mathbb{E}_{x\sim P_X, z\sim Q_{Z|X=x}}[-\log p_\theta(x|z)] + \mathbb{E}_{x\sim P_X}[\mathrm{D_{KL}}\big(Q_{Z|X=x}\big\|P_Z^\theta\big)], \tag{41.30}$$

where $Q_{Z|X=x}$ is a variational distribution over $\mathcal{Z}$ for each x. We will see that the NELBO has an interpretation as the rate-distortion cost of lossy compression (Eq. (41.16)), as follows.

Plugging in the Gaussian density for $p_\theta(x|z)$, the first term of the NELBO simplifies to

$$\mathbb{E}_{x\sim P_X, z\sim Q_{Z|X=x}}[-\log p_\theta(x|z)] = \frac{1}{2\sigma^2}\mathbb{E}_{x\sim P_X, z\sim Q_{Z|X=x}}[\|x - \mu_z\|^2] + \text{const.} \tag{41.31}$$

This very much resembles a reconstruction error if we interpret the variational posterior as a stochastic encoder $x \to Q_{Z|X=x}$ and define the decoder g to act as a lookup table $g_\theta(k) = \mu_k, k \in \mathcal{Z}$. Note that we have first seen the negative ELBO in our analysis of lossless compression with latent variable models in Section 41.2.5. A major difference is in the interpretation of the likelihood term: in Eq. (41.12), $P^\theta_{X|Z=z}(x)$ is the probability of discrete data, and $-\log P^\theta_{X|Z=z}(x)$ is the lossless compression cost of the data under a conditional entropy model given z; by contrast, here $p_\theta(x|z)$ is a probability density, and $-\log p_\theta(x|z)$ corresponds to the residual error between x and the lossy reconstruction $g_\theta(z)$.

The expected KL divergence term of the negative ELBO also has an interpretation as the "bit-rate" of lossy compression using the technique in Section 42.2. For now, note that if we restrict our variational posterior distributions to be deterministic, i.e., $Q_{Z|X=x} = \delta_{f(x)}$ for some function $f : \mathcal{X} \to \mathcal{Z}$, then the "rate" term of the NELBO reduces to the same cross-entropy term as in Eq. (41.12), i.e.,

$$\mathbb{E}_{x\sim P_X}[\mathrm{D}_{\mathrm{KL}}\big(\delta_{f(x)}\big\|P^\theta_Z\big)] = \mathbb{E}_{x\sim P_X}[-\log P^\theta_Z(f(x))] = \mathbb{E}_{x\sim P_X, z=f(x)}[-\log P^\theta_Z(z)], \tag{41.32}$$

thus its interpretation as the bit-rate is exact. Additionally, if we ensure a positive w and take the limit $\sigma^2 \to 0$, the NELBO reduces to purely a distortion loss, and the expectation-maximization (EM) algorithm [407][6] for the Gaussian mixture model reduces to the k-means algorithm [160]. In particular, the E-step of the EM algorithm computes the optimal variational posterior via Bayes' rule:

$$Q^*_{Z|X=x}(k) \propto P^\theta_Z(k)p_\theta(x|k) = w_k \exp\left(-\frac{\|x-\mu_k\|^2}{2\sigma^2}\right), \quad k = 1, 2, ..., K. \tag{41.33}$$

In the limit $\sigma \to 0$, a "winner-takes-all" phenomenon occurs where $Q^*_{Z|X=x}$ assigns probability 1 to the index of the closest cluster and reduces to the VQ encoder Eq. (41.17), and the M-step sets each μ_k to its respective cluster mean as in Eq. (41.18).

Variational Autoencoders

We now consider a broader family of latent variable models whose likelihood model is parametrised by an arbitrary decoding map g. When g and the approximate posterior family $Q_{Z|X}$ are parametrised by neural networks and jointly trained on the ELBO, such latent variable models are often known as *variational autoencoders* (VAEs) [892, 1391]. As in the Gaussian mixture example above, we denote the prior and likelihood models by P^θ_Z and $p_\theta(x|z)$, respectively. Now suppose that the conditional likelihood density is *aligned with* a distortion function ρ in the sense that

$$p_\theta(x|z) = C\exp\{-\lambda\rho(x, g_\theta(z))\}, \tag{41.34}$$

where $\lambda > 0$ and C are constants that do not depend on x or z. As a result, the negative log-likelihood density precisely corresponds to the distortion between x and its reconstruction $g_\theta(z)$, weighted by λ,

$$-\log p_\theta(x|z) = \lambda\rho(x, g_\theta(z)) + \text{const.} \tag{41.35}$$

For example, a Gaussian $p_\theta(x|z) = \mathcal{N}(x|g_\theta(z), \frac{1}{2\lambda}I)$ is aligned with squared error distortion $\rho(x, \hat{x}) = \|x - \hat{x}\|^2$, and a Laplacian distribution is aligned with absolute error distortion. Thus, the NELBO (Eq. (41.30)) again has the interpretation of a rate-distortion loss, where the rate term reduces to cross-entropy when the encoder $Q_{Z|X}$ is deterministic. Similarly to lossless compression with bits-back coding (Section 42.1), (amortized) variational inference (i.e., optimizing the NELBO with respect to $Q_{Z|X}$) corresponds to finding the optimal stochastic encoder for a fixed choice of prior and conditional likelihood (equivalently, the decoder g_θ), and training the latent variable model by ELBO maximization corresponds to learning a lossy compressor of the data by minimizing a rate-distortion loss [84, 1805, 1807]. The correspondence is particularly salient in nonlinear transform coding (Section 41.3.2). We can reinterpret the uniform noise perturbation during training (Eq. (41.19)) as sampling from a particular form of stochastic encoder or variational posterior $Q_{\tilde{Z}|X}$, defined by

$$Q_{\tilde{Z}|X=x} = \mathcal{U}(\psi(x) - 0.5, \psi(x) + 0.5) \quad \forall x \in \mathcal{X}, \tag{41.36}$$

i.e., a uniform distribution over a hypercube centred on the vector of transform coefficients $\psi(x)$. The corresponding rate term works out to exactly equal the proxy rate loss in Eq. (41.25),

$$\mathbb{E}_{x\sim P_X}[\mathrm{D}_{\mathrm{KL}}\left(Q_{\tilde{Z}|X=x}\middle\|p^\theta_{\tilde{Z}}\right)] = \mathbb{E}_{x\sim P_X}[\cancelto{0}{-h[Q_{\tilde{Z}|X=x}]}] + \mathbb{E}_{x\sim P_X, \tilde{z}\sim Q_{\tilde{Z}|X=x}}[-\log p^\theta_{\tilde{Z}}(\tilde{z})], \tag{41.37}$$

[6] See also Chapter 40 for an application of the EM algorithm.

where $h[\cdot]$ denotes differential entropy, and we used the fact that the uniform posterior in Eq. (41.36) has zero differential entropy. Thus, the R-D loss of nonlinear transform coding (Eq. (41.26)) exactly corresponds to the NELBO of a VAE specified by prior $p^{\theta}_{\tilde{Z}}$ as in Eq. (41.24), a distortion-aligned likelihood density $p_\theta(x|z)$ as in Eq. (41.34), and a variational posterior family $Q_{\tilde{Z}|X}$ as in Eq. (41.36). We note that unlike the fixed prior in the original VAE [892, 1391] and related work on Bayesian deep learning [174], the prior model in neural compression serves as an entropy model for the latents and is therefore learned jointly with other components of the lossy compressor for the best compression performance.

Empirically, the NELBO proxy of nonlinear transform coding (Eq. (41.26)) closely approximates the actual R-D performance achieved with quantization at test time [84]. This is ensured by design through specific restrictions on the prior and posterior distributions (Eq. (41.24), Eq. (41.36)) in the corresponding VAE. By comparison, in Bayesian deep learning it is common to adopt flexible variational posterior [1390] and prior distributions [1621] to improve the ELBO and the quality of the learned latent variable model. Naturally, we may wonder if an improved latent variable model also results in improved lossy compression performance, as in the case of lossless compression with bits-back coding [1625] (also see Section 42.1). While this is possible to some extent using iterative inference [1805] and posterior-adaptive quantization [1806], a general answer turns out to be non-trivial. Theoretically, *any* latent variable model with a distortion-aligned likelihood (Eq. (41.34)) optimizes a variational upper bound on the R-D function of the data source [1803, 1807], implying that there exists *some* lossy compressor that achieves the R-D performance given by the rate and distortion terms of the NELBO. Empirically, deep hierarchical VAEs have been demonstrated to yield tight upper bounds on the R-D function [1803]. However, it remains an open question how to convert such a latent variable model into a practical lossy compressor in an efficient manner. Our goal would be achieved if there exists an alternative to entropy coding that works with *any* stochastic encoder $Q_{Z|X}$ and prior P^{θ}_{Z} and can transmit samples from $Q_{Z|X=x}$ using a bit-rate equal to the expected KL $\mathbb{E}_{x\sim P_X}[\mathrm{D}_{\mathrm{KL}}\big(Q_{Z|X=x}\big\|P^{\theta}_{Z}\big)]$, averaged over $x \sim P_X$. This is indeed possible, although typically with a logarithmic overhead to the KL divergence, through a technique known as *channel simulation* or *relative entropy coding*. Developing efficient algorithms in this setting is an active research topic that we will explore in Section 42.2.

Chapter 42

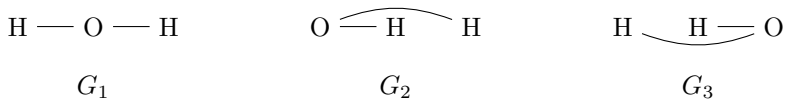
Learned Compression with Stochastic Codes

In the previous chapter, we described how classical source coding theory can be combined with modern machine learning techniques to achieve better data compression. However, we also saw that entropy codes such as Huffman coding (Section 41.2.3) and Arithmetic Coding (Section 41.2.4) operate on discrete data by design. Discreteness meant that it was non-trivial to combine them with neural models. For lossless neural compression using the two-part code presented in Section 41.2.5, it limited our choice of the probabilistic model by fixing the variational posterior over our latent variables to be deterministic (see Equation 41.12 and the discussion below it). For lossy compression, we were forced to quantise the data to ensure discreteness. However, since quantization is not differentiable, we had to introduce continuous approximations to quantisation to train lossy neural compressors in Section 41.3.2.

In this chapter, we describe two alternative approaches – bits-back coding for lossless and relative entropy coding for lossy compression – which lift all restrictions mentioned above. Therefore, these methods unlock a much broader class of probabilistic models for learned data compression. As we shall soon see, the high-level trick in both cases is to *randomise* the data encoding process in a controlled way. However, the benefits of these methods come at a price – implementing them can be more challenging than classical compression methods, and in some cases their runtime can be much worse than the comparable entropy coding algorithm; we will discuss these limitations at the end of their respective sections. Finally, we recommend the tutorial paper by [531] for a more detailed introduction and the excellent monograph of [1024] for a comprehensive treatment of relative entropy coding.

42.1 Lossless Compression with Randomization: Bits-back Coding

In Section 41.2, we saw how we can losslessly compress data $X \sim P_X$ using a probability model P_θ. However, in some more advanced cases, building an efficient probability model might be challenging, or even computationally intractable. Perhaps the simplest example is when we are interested in encoding unordered data or data up to some symmetry. Molecular structures are a prime example, since they often exhibit symmetries between the atomic bonds. For example, the atomic structure of water, H_2O, admits the labelled graph representation $H - O - H$. However, the following are all valid *ordered* representations of the same graph:



G_1 G_2 G_3

$$G_1 = ((\mathrm{H}, \mathrm{O}, \mathrm{H}), \{\{1,2\},\{2,3\}\}), \tag{42.1}$$
$$G_2 = ((\mathrm{O}, \mathrm{H}, \mathrm{H}), \{\{1,2\},\{1,3\}\}), \tag{42.2}$$
$$G_3 = ((\mathrm{H}, \mathrm{H}, \mathrm{O}), \{\{1,3\},\{2,3\}\}) \tag{42.3}$$

The above examples show that we need to take care that our compression algorithm built for labelled graphs does not encode more information than necessary: ideally, we would like to build a compressor that assigns the *same*

code to all equivalent representations of our data. In this section, we describe bits-back coding, which provides an elegant way of achieving this goal using randomisation. Moreover, in 42.1.4, we combine bits-back coding with latent variable models to build efficient lossless data compressors for any type of discrete data.

42.1.1 Invertible Sampling and Asymmetric Numeral Systems

The first fundamental concept bits-back coding relies on is the duality between data compression and random sampling,[1] which we first illustrate at an intuitive level. Consider a stream code, such as arithmetic coding (Section 41.2.4), which has the following components: for any probabilistic model P over the symbols to be encoded, and a pre-existing message/stream m, let $\text{enc}_P(x, m) = m'$ be the function that encodes the symbol x into m, producing the updated message/stream of bits m'. Here, we consider only entropy codes, meaning that the message length increases by roughly the self-information of x: we have $|m'| - |m| \approx -\log_2 P(x)$. Finally, a stream code also has a corresponding decoding function $\text{dec}_P(m') = (x, m)$, which is the inverse of the encoder:

$$\text{dec}_P(\text{enc}_P(x, m)) = (x, m) \tag{42.4}$$

Now, let $X \sim P$ be a discrete random variable and, starting with the empty message ϵ, consider encoding X and setting $M = \text{enc}_P(X, \epsilon)$. A crucial feature of entropy codes is that the bits of M ought to be uniformly distributed. Intuitively, this holds since if we know the true distribution of the source, then entropy codes extract all statistical structure from X and produce incompressible uniform noise. Specifically, if we let F be the CDF of P, we can view arithmetic coding directly as the probability integral transform, since it encodes X as a rational approximation of $F(X)$ [337, Chapter 13.3].

Given the above, consider the reverse scenario: let U be a fair Bernoulli process, i.e. an infinite bitstring consisting of iid fair coin flips. Moreover, assume for the moment that dec_P is a total function, that is, its output is well-defined for any binary string (with some effort, arithmetic coding can be implemented for this to hold). If we "decode" U and set $(X', U') = \text{dec}_P(U)$, what is the distribution of X'? Well, since dec_P is the inverse of enc_P, and we saw above that for $X \sim P$ we have that $\text{enc}_P(X)$ is uniform, we therefore must have that $X' \sim P$. This beautiful identity has the following intuitive interpretation: we can view U as the "seed" of the pseudo-random number generator (PRNG) dec_P, with enc_P as its inverse, which, given X' (and the updated PRNG seed U') returns the random seed U that generated X'. Therefore, we may view stream codes as *invertible samplers*. Here we glossed over several technical details, we refer the interested reader to [906] for a rigorous development of the above ideas.

A key assumption of the above argument was that the decoder of the stream code be a total function over binary strings. While arithmetic coding can be set up so that it fulfils this requirement, this is a non-trivial undertaking in practice. Furthermore, as we will see soon, it has another feature that makes it an unsuitable choice for bits-back coding: it encodes messages in a first-in-first-out (FIFO) structure, that is, if we encode a message $x_1 x_2 x_3 \dots$ into a stream m, the decoder recovers the symbols in the same order. A solution to both of these issues is to use a different stream code, *asymmetric numeral systems* [ANS; 459], instead. We refer the interested reader to Robert Bamler's excellent exposition of ANS [89] – for our exposition of bits-back coding here, it suffices to know that the ANS decoder is trivially seen to be a total function on bitstrings and ANS encodes messages in a last-in-first-out (LIFO) structure, i.e., if we encode the message $x_1 x_2 \dots x_n$, the decoder recovers them in the order $x_n \dots x_2 x_1$.

42.1.2 A very brief introduction to symmetries

Before we can define compression schemes that eliminate the coding inefficencies that arise from symmetries, we need to define what we mean by symmetry. Intuitively, a symmetry of an object is a transformation whose outcome does not change the object, such as rotating a square by 90 degrees. In this section, we develop the bare minimum mathematical machinery to capture the most important aspects of this intuition.

To this end, consider the set S_n, the so-called *symmetric group of degree n*, consisting of all invertible functions on n items. The elements of S_n are called *permutations*, and can be thought of as the possible orderings of n items. The power of permutations is that it turns out that any discrete symmetry can be represented using them.[2] Hence, for the mathematics to follow, we will assume henceforth that all abstract transformations are "implemented" using permutations Using this insight, we make the following abstract definitions of the symmetries of objects, and leave it up to the practitioner to define the precise "implementation" of the symmetry.

[1] Indeed, this duality is so important that it is arguably the central concept underpinning all of algorithmic information theory [1031], but we will not discuss this any further here.

[2] This is known as Cayley's theorem.

Definition 42.1 (Permutable class)**.** *For $n \in \mathbb{N}$, a permutable class of order n is a set $\mathcal{F}$, together with a binary operation $\cdot : S_n \times \mathcal{F} \to \mathcal{F}$, called a group action.*[3]

Intuitively, a permutable class defines the set of *ordered* objects/structures under consideration. Recalling the molecular structure example from earlier in the section, we may pick our permutable class $\mathcal{F}$ (of order three) as the set of labelled, ordered graphs with at most three vertices. Then, for an ordered representation of a graph, we may "implement" the group action as permuting the ordering of the nodes with the edges and labels changed appropriately. Concretely, one way to represent ordered, labelled graphs on a computer is using a labelled adjacency matrix:

$$G_1 \quad \to \quad A = \begin{array}{c} \\ H \\ O \\ H \end{array} \begin{array}{c} \begin{array}{ccc} H & O & H \end{array} \\ \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \end{array} \tag{42.5}$$

where if $A_{ij} = 1$ indicates that node i is connected to node j. Then, we could define the action of a permutation f on a graph as permuting the rows and columns (alongside the labels) of its adjacency matrix: $(f \cdot A)_{i,j} = A_{f^{-1}(i),f^{-1}(j)}$. For example, applying the function f defined as $f(1) = 2, f(2) = 3$ and $f(3) = 1$ to the adjacency matrix of water above, this would result in:

$$f \cdot G_1 = f \cdot \begin{array}{c} \\ H \\ O \\ H \end{array} \begin{array}{c} \begin{array}{ccc} H & O & H \end{array} \\ \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \end{array} = \begin{array}{c} \\ H \\ H \\ O \end{array} \begin{array}{c} \begin{array}{ccc} H & H & O \end{array} \\ \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix} \end{array} = G_3 \tag{42.6}$$

However, these adjacency matrices represent the same graph, as we can verify from the corresponding representations G_1 and G_3 above. But what systematic computational procedure could we use to check automatically whether two elements $x, y \in \mathcal{F}$ are the same? Since we are assuming that our objects are ordered in some way, the idea is to *sort* x and y. Then, denoting the sorted versions as $\bar{x}$ and $\bar{y}$, respectively, we could say that x and y are equivalent if $\bar{x} = \bar{y}$. We formalise this idea below.

Definition 42.2 (Canonical ordering, equivalence)**.** *Given a permutable class $\mathcal{F}$ of order n, a canonical ordering is any function $\texttt{sort} : \mathcal{F} \to \mathcal{F} \times S_n$, such that:*

- *For all $x \in \mathcal{F}$, we get a sorted representation and a sorting permutation $(\bar{x}, s) = \texttt{sort}(x)$ such that $s \cdot \bar{x} = x$. As a shorthand, we will use $\bar{\cdot}$ to denote the sorted element of $\mathcal{F}$ that $\texttt{sort}(\cdot)$ returns.*
- *For $x, y \in \mathcal{F}$, we say that x and y are equivalent if $\bar{x} = \bar{y}$.*

Now, we use canonical orderings to define two important sets related to any $x \in \mathcal{F}$: the *equivalence class* $[x]$ and the *coset* S_n^x.

Definition 42.3 (Equivalence class, coset)**.** *For an element x of a given permutable class $\mathcal{F}$, the equivalence class of x is defined as the set of all elements of $\mathcal{F}$ equivalent to x:*

$$[x] = \{y \in \mathcal{F} \mid \bar{x} = \bar{y}\} \tag{42.7}$$

Then, the coset[4] *S_n^x of x is the set of corresponding permutations $\texttt{sort}$ returns for each element of $[x]$:*

$$S_n^x = \{s \in S_n \mid (s, \bar{y}) = \texttt{sort}(y),\ y \in [x]\} \tag{42.8}$$

Using the water example once more, we have that

$$[G_1] = \{G_1, G_2, G_3\} \tag{42.9}$$

We may view the equivalence classes' bracket notation $[x]$ as a map that takes an ordered structure x and "forgets" its ordering. The final object we require to define bits-back coding is the set (more precisely, the group) of all transformations that leave a particular element of a permutable class invariant. As such, it is the set of all symmetries that an object posesses.

[3] Technically, a *left* group action.

[4] Technically, the *left* coset. While this definition is sufficient for our purposes here, we note that it is related to but does not match the usual group-theoretic use of the term.

Algorithm 33: Shuffle encoder

Input : stream m,
object x,
probability distribution P_θ
Output: updated stream m''

// Note that $s \sim \mathrm{Unif}(S_n^x)$
$(s, m') \leftarrow \mathrm{dec}_{\mathrm{Unif}(S_n^x)}(m)$
$y \leftarrow s \cdot \bar{x}$
return $\mathrm{enc}_{P_\theta}(y, m')$

Algorithm 34: Shuffle decoder

Input : stream m'',
probability distribution P_θ
Output: decoded object y,
remaining stream m
$(y, m') \leftarrow \mathrm{dec}_{P_\theta}(y, m'')$
$(\bar{y}, s) \leftarrow \mathtt{sort}(y)$
$m \leftarrow \mathrm{enc}_{\mathrm{Unif}(S_n^y)}(s, m')$
return y, m

Definition 42.4 (Automorphism group)**.** *For an element x of a permutable class $\mathcal{F}$ of order n, its automorphism group is the set of all permutations that do not change x:*

$$\mathrm{Aut}(x) = \{s \in S_n \mid s \cdot x = x\} \tag{42.10}$$

For example, $\mathrm{Aut}(\mathrm{H}-\mathrm{O}-\mathrm{H}) = \{(x \mapsto x), (x \mapsto 4-x)\}$ is the automorphism group of the water graph.

Let us briefly take stock of what we have so far: we have identified the notion of ordered structures with permutable classes. We have also identified the notion of unordered structures with equivalence classes of isomorphic elements of a permutable class. We defined canonical orderings, using which we can "sort" elements of a permutable class and the automorphism group of an element being the set of all permutations that leave an object unchanged.

42.1.3 Bits-back coding with discrete symmetries: Shuffle Coding

We are now ready to derive the simplest form of bits-back coding, called shuffle coding [950]. The two key ideas of shuffle coding are these: 1) given an ordered structure x (i.e., the element of a permutable class), we encode a *random, but equivalent* ordered representation y; and 2) we can use ANS as an invertible sampler to use a pre-existing stream m for the randomisation. Indeed, this is an almost complete description of shuffle coding, and we will spend the rest of this section unpacking it.

First, let us explain the randomisation more carefully. Assume the encoder receives an element x of a permutable class, and they wish to discard all order structure present in x, which they can achieve using the following strategy. They sort x to obtain $\bar{x}$, and draw a uniformly random permutation from x's coset $s \sim \mathrm{Unif}(S_n^x)$ and compute $y = s \cdot \bar{x}$. Then, y will be a uniformly random element of x's equivalence class $[x]$, and thus the encoder succeeded in discarding the order structure. Nonetheless, y still has some order, albeit random, so encoding it will be no cheaper than encoding x directly. However, note that the decoder has the following "error-correction" ability: after receiving any $y \in [x]$, they can obtain the same $\bar{x}$ the encoder had by computing $\mathtt{sort}(y)$.

We combine this capability with the second idea of using ANS as an invertible sampler to get an efficient coding algorithm: the encoder can use the pre-existing message to randomise $\bar{x}$. Concretely, assume we have some stream of bits m into which we wish to encode x using the ANS encoder using P_θ as the coding distribution. However, before doing so, we use the ANS decoder to set $(s, m') = \mathrm{dec}_{\mathrm{Unif}(S_n^x)}(m)$. Recall from Section 42.1.1, that if m consists of uniformly random bits (and empirically, this is roughly true) then we will have $s \sim \mathrm{Unif}(S_n^x)$, as we require above. Furthermore, since we decoded from the stream, the total message length decreases, that is, $|m'| \leq |m|$. Next, we use our decoded s to compute $y = s \cdot \bar{x}$ as above, and we encode y into the updated stream m' to obtain $m'' = \mathrm{enc}_{P_\theta}(y, m')$. We see that this procedure already produces a shorter overall message length than encoding x into m directly. However, before analysing the gain, we need to demonstrate that this procedure is invertible, otherwise it would not be of much use.

Thus, consider the decoder, who receives m'' from above. Using P_θ, they use the ANS decoder to recover $(y, m') = \mathrm{dec}_{P_\theta}(m'')$. But they can do more: they can recover the encoder's permutation s by sorting y: $(\bar{y}, s) = \mathtt{sort}(y)$. Then, since x and y are equivalent by construction, we also have $S_n^x = S_n^y$ and hence the decoder can re-encode s into m' to obtain the original message $m = \mathrm{enc}_{\mathrm{Unif}(S_n^y)}(s, m')$. This encoding-decoding procedure constitutes bits-back coding, and we summarise the procedure in Algorithms 33 and 34.

Now that we see that the bits-back coding procedure is correct, what is its coding cost? One important aspect we haven't mentioned yet, is what coding distribution P_θ we ought to use. A crucial desideratum is that P_θ be exchangeable, i.e. for $x, y \in \mathcal{F}$ if x and y are equivalent, we should have $P_\theta(x) = P_\theta(y)$. Indeed, it can be shown that for any distribution $\widetilde{P_\theta}$ over the equivalence classes, $\widetilde{P_\theta}$ induces a unique exchangeable distribution P_θ over

$\mathcal{F}$ [950, Lemma 3.1]. Furthermore, it can be shown by applying the so-called orbit-stabiliser theorem from group theory, that $|S_n^x| = n!/|\mathrm{Aut}(x)|$. Hence, $\mathrm{Unif}(S_n^x)$ assigns $|\mathrm{Aut}(x)|/n!$ probability to each $s \in S_n^x$. Putting these together with the fact that ANS is an entropy code, we find that decoding a uniformly random permutation from the coset of x reduces the message length by approximately $\log_2 n! - \log_2|\mathrm{Aut}(x)|$ bits, while encoding y using P_θ increases the message length by approximately $-\log_2 P_\theta(y)$ bits. Thus, in total the cumulative message length increase is

$$|m''| - |m| \approx -\log_2 P_\theta(y) - \log_2 n! + \log_2|\mathrm{Aut}(x)| \text{ bits}, \tag{42.11}$$

which we see is equal to directly encoding $[x]$ using $\widetilde{P_\theta}$.

Shuffle coding is the state-of-the-art method for optimally encoding objects with discrete symmetries. Indeed, Kunze et al. [949] achieve an impressive 46% reduction in the codelength when compressing molecules and a 26% reduction when compressing graphs of certain social networks. However, there are several implementation challenges that one must take into account in practice. In the rest of this section, we discuss the most salient ones and provide some references for further reading.

The biggest outstanding issue of shuffle coding as we presented here is that its efficiency relies on so-called "clean, initial bits." Namely, we need to have a non-empty bitstream m to begin with, otherwise the encoder cannot decode the random permutation at the start of the encoding process. In other words, applying shuffle coding to a single object x will actually cost $-\log_2 P_\theta(y)$ bits, as we need to provide an initial stream $-\log_2 n! + \log_2|\mathrm{Aut}(x)|$ bits so that the shuffle encoder (Algorithm 33) can decode an element of x's coset. Thus, shuffle coding in this form is best applied for compressing entire datasets of unordered objects, because this way the initial overhead in bits amortises over the dataset. Fortunately, it also turns out that shuffle coding can be modified so that it can also encode single unordered objects; we refer the interested reader to Kunze et al. [949] for the details. Furthermore, we need that the bits of m be (roughly) uniformly distributed, otherwise the coding efficiency of bits-back coding can suffer. Fortunately, Townsend et al. [1625] show that the message streams produced by shuffle coding are indeed roughly uniform and thus the coding cost is close to optimal in practice. Note also that, as we saw above, in the optimal case $|\mathrm{Aut}(x)| = 1$, since we make the largest coding gain this way. However, this also means that naively implementing shuffle coding in this case requires $O(\log_2 n!) = O(n \log_2 n)$ initial bits. Fortunately, Kunze et al. [949] show that this can be reduced to $O(\log_2 n)$ bits.

The other big issue with implementing shuffle coding is computing the canonical ordering `sort`. There is currently no known general method to construct a canonical ordering for a given permutable class $\mathcal{F}$. Indeed, it is not even known whether a canonical ordering exists for every permutable class. Thus, at present the implementation of `sort` must rely on the specific properties of the precise underlying class. Unfortunately, even if a canonical ordering is known for a certain class, it might be difficult to compute. For example, in the case of arbitrary unlabelled graphs, there is no known polynomial-time algorithm for computing their canonical ordering or automorphism group. We also omitted implementation details for designing P_θ and drawing samples from $\mathrm{Unif}(S_n^x)$. A detailed discussion on all of these topics can be found in Kunze et al. [949, 950].

42.1.4 Bits-back coding with side-information

The previous section described how to construct coding algorithms for unordered data using shuffle coding, a special case of bits-back coding. This section describes another application of bits-back coding: building lossless compression algorithms using essentially arbitrary latent variable models. Therefore, bits-back coding enables us to build lossless neural data compression algorithms even from probabilistic models with continuous latent variables such as variational autoencoders and normalising flows [754, 1625], thereby removing most limitations we discussed in Section 41.2.5.

To begin, let us restate the main mechanism that made shuffle coding in Section 42.1.3 possible. The key observation is that if the encoder sorts the data and then randomly permutes it, the decoder can sort the data themselves to recover not just the sorted version of the data, but also the random permutation the encoder used. The way shuffle coding exploited this observation was to use ANS (Section 42.1.1) to sample/decode the random permutation from the pre-existing message stream to reduce its length and then proceeded to encode the randomised data into the reduced-length stream.

Abstractly, writing X for the sorted data and Y for the permuted data, we can restate the above by saying that for any $Y \sim P_{Y|X}$, the shuffle coding decoder can always recover X from Y. Then, the question is: can we generalise this procedure to different settings? For example, could we make the same idea work when Y is a real-valued vector? Excitingly, the answer is yes, and this section describes a very general approach to do this.

We start with generalising the scheme to arbitrary discrete spaces and distributions. The key idea is beautifully simple: since in general, the decoder cannot recover X from Y, instead of just encoding the randomised data Y,

the sender encodes X, Y jointly. Thus, we construct the updated, generalised bits-back algorithm as follows. We start with a joint distribution $P_{X,Y} = P_X \times P_{Y|X}$ over the original data X and its randomised variant Y. Then, **the sender**, having pre-existing message stream m and upon receiving X:

1. Decodes $(Y, M') = \mathrm{dec}_{P_{Y|X}}(m)$ using the conditional $P_{Y|X}$. Then, $Y \sim P_{Y|X}$ and the message length is reduced by approximately $-\log_2 P_{Y|X}(Y)$ bits.
2. Encodes X into M' using the conditional $P_{X|Y}$: it sets $M'' = \mathrm{enc}_{P_{X|Y}}(X, M')$, increasing the message length by approximately $-\log_2 P_{X|Y}(X)$ bits.
3. Encodes Y into M'' using the marginal P_Y: it sets $M''' = \mathrm{enc}_{P_Y}(Y, M'')$, increasing the message length by approximately $-\log_2 P_Y(Y)$ bits.

As before, note that this generalised procedure is also invertible, meaning that upon receiving M''', the decoder can always recover the data X and the original stream M. Concretely, they:

1. Decode $(Y, M'') = \mathrm{dec}_{P_Y}(M''')$ using the marginal P_Y.
2. Since Y is now known, decode $(X, M') = \mathrm{dec}_{P_{X|Y}}(M'')$.
3. Since X is now known, re-encode Y into M' using the conditional $P_{Y|X}$ to recover the original stream: $m = \mathrm{enc}_{P_{Y|X}}(Y, M')$.

Having convinced ourselves that the scheme is invertible, let us compute its efficiency. We saw that the encoding process first decreases the message length by approximately $-\log_2 P_{Y|X}(Y)$ bits, then increases it by $-\log_2 P_{X|Y}(X)$ and $-\log_2 P_Y(Y)$ bits, respectively. Thus, the total increase is approximately

$$\log_2 P_{Y|X}(Y) - \log_2 P_{X|Y}(X) - \log_2 P_Y(Y) = -\log_2 \frac{P_{X,Y}(X,Y)}{P_{Y|X}(Y)} = -\log_2 P_X(X) \tag{42.12}$$

which is the same efficiency as using entropy coding directly. This finally completes the picture from Section 41.2.5, showing that it is possible to achieve the theoretically optimal compresion rate by fully eliminating the slack term in Equation 41.12.

So far, we have dealt solely with discrete spaces. However, as we explain next, bits-back coding is also applicable to "sufficiently nice" continuous distributions. Concretely, we will still assume that the noise-free version of the data X is discrete, but let Y be $\mathbb{R}^n$-valued. To make bits-back coding work in this setting, we need to make the additional assumption that the support of Y's distribution can be quantised well. Concretely, let $p(y \mid x)$ and $p(y)$ be the probability densities of $P_{Y|X}$ and P_Y, respectively. Furthermore, for a positive real number δ let $\mathcal{Q}_\delta$ be a quantiser such that $\mathbb{P}(\mathcal{Q}_\delta(Y) = y \mid X = x) \approx p(y \mid x)\delta^n$ and $\mathbb{P}(\mathcal{Q}_\delta(Y) = y) \approx p(y)\delta^n$. Then, using the quantised density functions to decode/encode Y, Equation 42.12 tells us that applying the discrete bits-back procedure from above will increase the message length by

$$|M'''| - |M| \approx -\log_2 \frac{P_{X,Y}(X,Y)}{P_{Y|X}(Y \mid X)} \approx -\log_2 \frac{P_{X|Y}(X \mid Y)p(\mathcal{Q}_\delta(Y))\delta^n}{p(\mathcal{Q}_\delta(Y) \mid X)\delta^n} = -\log_2 P_X(X) \tag{42.13}$$

This means that we still retain efficiency at the entropy coding level. This might make it seem like the level of discretisation is arbitrary, so long as our quantisation assumption is satisfied. However, in practice, we usually want to set the discretisation level as low as possible, since it determines the amount of initial bits necessary to run the compressor. As an example, consider using 16-bit quantisation for compressing 1024×1024 greyscale images. To do this, we need on the order of $2^{20} \times \log_2 16 = 2^{22}$ bits, or roughly 524 kilobytes of initial bits.

Given the bits-back machinery we developed in this section, constructing lossless neural compressors is fairly straightforward using the two-part code construction we gave in Section 41.2.5. Indeed, after bits-back coding was invented in the early 1990s by Hinton and Van Camp [751], it quickly found use for compression with latent variable models, first implemented for a limited model class as early as 1997 by Frey and Hinton [560], who used Arithmetic Coding as the invertible sampler (ANS hadn't yet been invented at this point). However, it was Townsend et al. [1625] who produced the first practical implementation of the bits-back coding technique using ANS for lossless neural compression with variational autoencoders. Since then, their method has been improved upon, scaled up and extended in several interesting directions; we refer the interested reader to the excellent papers of Ho et al. [754], Kingma et al. [896], Townsend et al. [1626].

42.2 Lossy Compression with Randomization: Relative entropy coding

After considering lossless compression in Section 42.1, we now treat lossy compression with stochastic codes in this section. Recall that Section 41.3 introduced quantisation as the fundamental mechanism for discarding information. While quantisation has the advantage that it is simple to implement, and has many decades of dedicated research and industrial practice behind it, recent advances in neural compression have shone light on some of its weaknesses too. First, quantisation is a non-differentiable operation, and hence, as we have already seen in Section 41.3.2, constructing efficient learned lossy compression models involves some non-trivial approximations. More importantly, quantisation error in general is difficult to control practically and analyse theoretically.

Thus, in this section we describe a different lossy compression mechanism: random perturbation. In its simplest form, the idea is to add some independent noise ϵ to our signal X to form the lossy signal $Y = X + \epsilon$. Note that ϵ, and hence Y may remain continuous, and the additive form for Y above immediately suggests to use the reparametrisation trick for gradient-based learning (see Section 13.2.1). Naturally, the question is how to compress Y. The answer is *relative entropy coding* (also called channel simulation, reverse channel coding and channel synthesis in various parts of the information theory literature [531, 1024]). In comparison to quantisation and entropy coding (Section 41.3), relative entropy coding algorithms are generally significantly more complex to implement. However, their benefit goes beyond avoiding the non-differentiability issues quantisation suffers from. Since we get to *pick* the distribution of Y in relative entropy coding (REC), we can develop more advanced features on top of it, such as differential privatisation or leveraging generative modelling to improve the perceptual quality of the compressor's output; we will discuss these in Section 42.3.

42.2.1 The most special case: dithered quantisation

There is undoubtedly a large conceptual hurdle for any newcomer to REC: it is rather counter-intuitive that the stated goal of encoding a random sample from a possibly continuous space is even possible using only finitely many bits. Hence, the goal of this section is to give the reader a first idea of an algorithm.

First, for all our algorithms to work, we need a crucial new ingredient in the framework: shared randomness. From a theoretical perspective, shared randomness comprises some random variable (or collection of variables) W whose realisations are available to both the encoder and the decoder, akin to entangled quantum states.[5] Practically, shared randomness may be achieved in various ways. Perhaps the simplest approach is for the encoder to use a pseudo-random number generator (PRNG) with a seed s that they encode in the preamble of their message (the first thing the receiver decodes), so that the decoder can use the same seed s for their PRNG and thus generate the same samples as the encoder did.

With this additional technical detail in mind, we now describe *dithered quantisation*, which takes some real-valued input signal $X \sim P_X$ and encodes a version of the input corrupted by independent uniform noise: $Y = X + \epsilon$, where $\epsilon \sim \mathrm{Unif}(-1/2, 1/2)$. To achieve this, we make use of the following beautiful, yet little-known distributional identity: for any $c \in \mathbb{R}$ and two independent and identically distributed uniform random variables $U, U' \sim \mathrm{Unif}(-1/2, 1/2)$, it holds that

$$\lfloor c + U \rceil - U \sim c + U' \tag{42.14}$$

We refer to Equation 42.14 as the *dithering identity*, and it holds the key to our efficient coding scheme. Concretely, we set $W \leftarrow U \sim \mathrm{Unif}(-1/2, 1/2)$ to be the shared randomness between the encoder and the decoder. Then, upon receiving the signal $X \sim P_X$, the encoder computes the perturbed and quantised signal $K = \lfloor X + U \rceil$. Since K is discrete, we can use an entropy code to encode it (such as artithmetic coding, see Section 41.2.4); the optimal coding distribution turns out to be also the most natural choice: for $K = k$ we should use[6]

$$P(k \mid U) = \mathbb{P}(\lfloor X + U \rceil = k \mid U) = P_X([k - U - 1/2,\ k - U + 1/2)). \tag{42.15}$$

Note, that conditioning on U here is fine, since we assume that it is shared and hence the receiver knows it. Thus, the receiver first recovers K using their knowledge of U and then computes $Y = K - U$. By Equation 42.14, we will have $Y \sim \mathrm{Unif}(X - 1/2, X + 1/2)$.

The efficiency of dithered quantisation is also quite remarkable: using $P(k \mid U)$ as the coding distribution

[5] Indeed, REC was first conceived as part of investigations of quantum entanglement-assisted communication [117, 1561].

[6] For the second equality, we assumed the definition $\lfloor x \rceil = \lfloor x + 1/2 \rfloor$; for other conventions the second equality changes accordingly.

above, we find that the bitrate is given by the mutual information between X and Y:[7]

$$\mathrm{H}[Y \mid U] = \mathrm{H}[K \mid U] = I[X; K - U] = I[X; Y] \tag{42.16}$$

We emphasise that the Shannon entropy $\mathrm{H}[Y \mid U]$ above is not a mistake: even if Y itself is continuous, its description length *given the common randomness* is finite! Equation 42.16 indicates the significant gain we obtain using the shared randomness: without conditioning, $\mathrm{H}[Y]$ would be infinite since Y is real-valued. In fact, in Section 42.2.2 we show that the coding rate $I[X; Y]$ is optimal, and as such it plays a similar role to the entropy in the classical, deterministic setting.

42.2.2 General Relative Entropy Coding and Its Limitations

In the previous section, we saw that we can leverage shared randomness to encode an additive uniform perturbation. In this section, we zoom out as much as possible and define REC in its greatest generality, and discuss its limitations.

As we already noted, the core idea of REC is to perturb the source X. Thus, if we denote by Y the output of the procedure, we may choose an arbitrary conditional distribution $P_{Y|X}$. Then, the idea is that upon receiving some input X, the sender sends some bits so that the receiver, using also the common randomness W, can simulate a single sample $Y \sim P_{Y|X}$. This point is so important that it is worth dwelling upon a little more. The sender's task **is not** to encode the conditional distribution $P_{Y|X}$ to the receiver. Of course, if they do encode $P_{Y|X}$, then the receiver can simulate a sample $Y \sim P_{Y|X}$, but encoding the whole distribution usually requires far more bits than just a message encoding enough information to draw one sample.

More formally, as was in the case for entropy coding algorithms (Section 41.2), REC algorithms comprise an encoder and a decoder, albeit slightly more involved. Concretely, let $\mathrm{enc}_w : \mathcal{X} \to \{0,1\}^*$ be a parametric function with parameter w, mapping from the source sample space $\mathcal{X}$ to finite-length bitstrings; we refer to enc_w as the encoder. Similarly, let $\mathrm{dec}_w : \{0,1\}^* \to \mathcal{Y}$ be a parametric function with parameter w, mapping from bitstrings to the output sample space $\mathcal{Y}$; we refer to dec_w as the decoder. The fundamental correctness requirement we impose on this pair of functions is as follows. Denoting the shared randomness as a random variable W, we require that for all $x \in \mathcal{X}$

$$\mathrm{dec}_W(\mathrm{enc}_W(x)) \sim P_{Y|X=x} \tag{42.17}$$

Before moving on, we emphasise two important points. First, the randomness in Equation 42.17 comes purely from the shared randomness W and not the source. Second, note that implicit in our requirement is the fact that the shared randomness is independent of the source X (it is difficult to imagine how it could be shared otherwise). Using dithered quantisation from the previous section as an example, the common randomness is $W = U \sim \mathrm{Unif}(-1/2, 1/2)$, and the encoder outputs the entropy code corresponding to $K = \lfloor x + U \rceil$, while the decoder recovers K and computes $K - U$.

Since we are interested in data compression, we are of course interested in the coding rate of the encoder as well, measured by $\mathbb{E}[|\mathrm{enc}_W(X)|]$. We now demonstrate that the coding rate cannot be too small. For brevity, let $M = \mathrm{enc}_W(X)$. Then,

$$\begin{aligned}
\mathbb{E}[|M|] &\geq \mathrm{H}[M \mid W] && \text{(Shannon's noiseless source coding theorem)} \\
&= \mathrm{H}[M \mid W] - \mathrm{H}[M \mid W, X] && (M \text{ is a function of } X \text{ and } W) \\
&= I[X; M \mid W] && \text{(definition of conditional mutual information)} \\
&= I[X; M \mid W] + I[X; W] && (X \text{ and } W \text{ are independent}) \\
&= I[X; (M, W)] && \text{(chain rule for mutual information)} \\
&\geq I[X; Y] && \text{(by Equation 42.17 and the data processing inequality)}
\end{aligned}$$

Note that the above argument is valid for any encoder-decoder pair that satisfy Equation 42.17, and hence the mutual information $I[X; Y]$ provides a fundamental lower bound to our problem. On the other hand, we have already seen that dithered quantisation achieves this lower bound in the case of additive uniform perturbation; this finally motivates the definition of a relative entropy code.

Definition 42.5 (Relative Entropy Code)**.** *For a source $X \sim P_X$ and conditional distribution $P_{Y|X}$, a relative entropy code is a triplet $(W, \mathrm{enc}_w, \mathrm{dec}_w)$ such that:*

1. *W is a random variable independent of X, and is available both at the sender and the receiver's side.*

[7] Recall that $I[X; Y] = \mathrm{D}_{\mathrm{KL}}(P_{X,Y} \| P_X \times P_Y)$.

2. *The triplet satisfies Equation 42.17.*
3. *The coding rate of* enc_w *achieves at least*

$$\mathbb{E}[|\mathrm{enc}_W(X)|] = I[X;Y] + O(\log_2 I[X;Y]) \tag{42.18}$$

Given the discussion so far, the reader may be surprised that we allow for logarithmic rate overhead in Equation 42.18. However, unfortunately this additional logarithmic slack is necessary: it has been shown in the literature, that there exist examples where achieving $I[X;Y]$ within a constant additive term is not possible; see Braverman and Garg [203], Flamich et al. [534], Li and Gamal [1025] for example. Indeed, if we change our information measure from the mutual information $I[X;Y]$ to the *functional information* $F[X \to Y]$ defined recently by Hill et al. [747], we may get rid of the logarithmic slack. However, a proper discussion of the functional information is beyond the scope of the present discussion.

So far, we defined REC in the most general setting and demonstrated a fundamental lower bound to its efficiency. Next, we sketch a general technique for constructing a relative entropy code for any source $X \sim P_X$ and perturbation $P_{Y|X}$. The idea is to recognise that REC is a sampling problem, with the additional requirement that we be able to assign a code to the samples we simulate. Thus, it turns out that we can essentially use rejection sampling (Section 1.3) to construct a relative entropy code. Namely, there are two tricks we need:

1. We can use the marginal P_Y as the proposal distribution for rejection sampling.
2. We can set the proposal samples $Y_1, Y_2, \ldots \sim P_Y$ as the common randomness $W = (Y_1, Y_2, \ldots)$. In practice, the sender would use some PRNG seed s for generating theses sequences, which they would then share with the decoder so that they can also generate the same random sequence.

Let $r_x(y) = \frac{dP_{Y|X=x}}{dP_Y}(y)$ be the density ratio of the posterior and the marginal, let $M_x \geq \|r_x\|_\infty$ be any upper bound on r_x and let $(U_1, U_2, \ldots)$ be an infinite sequence of iid $\mathrm{Unif}(0,1)$ random variables. Then, recall that a rejection sampler finds the first proposed sample such that the acceptance criterion is satisfied; namely, they compute $K = \min\{k \in \mathbb{N} \mid U_k < r_x(Y_k)/M_x\}$, and we have that $Y_K \sim P_{Y|X=x}$. However, since we assumed that the sender and the receiver share all the Y_ks, it is enough for the sender to encode K, since the receiver can simply select $Y_K \sim P_{Y|X=x}$. It can be shown that we can encode K using $\mathbb{E}[\log K] + \log(\mathbb{E}[\log K] + 1) + 1$ bits. Using the fact that $K \sim \mathrm{Geom}(1/M_x)$, by applying Jensen's inequality we get $\mathbb{E}[\log_2 K] \leq \log_2 \mathbb{E}[K] = \log_2 M_x$. Hence, using rejection sampling, we can construct a coding scheme with coding rate

$$\mathbb{E}\big[\big|\mathrm{enc}_W^{RS}(X)\big|\big] \leq \mathbb{E}_X[\log_2 M_X] + \log_2(\mathbb{E}_X[\log_2 M_X] + 1) + 2 \text{ bits} \tag{42.19}$$

In the general case, the above coding rate can be well beyond the desired $I[X;Y] + O(\log_2 I[X;Y])$ cost that we initially wished for. However, Flamich [530] shows that when we require that the rejection sampler always terminate in expectation, that is, $\mathbb{E}[K] < C$ for some $C > 0$, we do indeed achieve the desired order:

$$\mathbb{E}\big[\big|\mathrm{enc}_W^{RS}(X)\big|\big] \leq I[X;Y] + \log_2(I[X;Y] + 1) + \log_2 C + \log_2(\log_2 C + 1) + 2 \text{ bits} \tag{42.20}$$

However, this constant C can still be quite large in practice. Fortunately, more sophisticated algorithms, such as the Poisson functional representation/A* coding [533, 1025] and greedy Poisson rejection sampling [529] eliminate this inefficiency.

We also see another drawback of rejection sampling: its runtime. Namely, we saw that its expected runtime is $\mathbb{E}[K] = \mathbb{E}_X[M_X] \geq \mathbb{E}_X[\|r_X\|_\infty]$, which can be prohibitively large. Thus, the natural question is: could we do better? Unfortunately, in the general case, the answer is no: Goc and Flamich [642] show that the runtime of any general-purpose REC algorithm (that is, an algorithm applicable to any conditional distributon $P_{Y|X}$) is at least $\mathbb{E}_X[\|r_X\|_\infty]$. Upon a little thought, this result becomes less surprising: after all, relative entropy coding is a sampling problem, and it is well-known that sampling is a computationally hard problem in general [1072]. Therefore, fast REC algorithms, such as dithered quantisation (Section 42.2.1) and its variants [727, 909], branch-and-bound A* coding and GPRS [530] and PolarSim [1549] are all only applicable to encoding some restricted domain of perturbations. However, we note that it is still an open problem to find a fast algorithm for the practically most relevant perturbation: when $P_{Y|X}$ is multivariate Gaussian.

42.3 Applications of Relative Entropy Coding

In the previous sections, we have introduced the core concepts, theory, and algorithmic instances of REC. In this section, we reap the rewards of our hard work and look at some concrete applications of REC.

As we have already noted at the beginning of the chapter, the most straightforward benefit of REC over entropy coded quantisation, is that it has essentially no limitation on what probabilistic model we are using for the data. Thus, when it comes to lossy neural compression, we no longer need the approximations from Section 41.3.2 to train our models. As such, REC is a perfect match for improving the standard, autoencoder-based nonlinear transform coding methods. For example, we can combine any variational autoencoder (see Section 13.3.3) into a lossy data compression algorithm. For the interested reader, we recommend the paper by Flamich et al. [532] describing how this may be achieved in practice for variational autoencoders with Gaussian latent variables and the paper by Agustsson and Theis [11] using dithered quantisation (Section 42.2.1) for autoencoders with uniform latent variables. We note, that the approach of Agustsson and Theis [11] is very similar to the autoencoder-based nonlinear transform code introduced in Section 41.3.2. The main idea is that instead of switching to hard quantisation after training, they use dithered quantisation to encode samples from the additive uniform variational posterior. However, REC also enables us to construct compression algorithms using unconventional probabilistic models, such Bayesian neural networks, which we illustrate in the rest of the section, and Gaussian diffusion models, which we leave as an exercise.

42.3.1 Compression with Bayesian implicit neural representations

Many data modalities can be viewed as continuous functions, including images, video, and many others. For instance, an image typically stored as a $W \times H$ array of RGB values can also be regarded as a function $f : [0,1]^2 \to [0,1]^3$ mapping spatial coordinates to (normalised) RGB colours. The discrete array provides samples $\{f(x_i, y_j)\}_{i\in[1:W],j\in[1:H]}$ on a grid; higher resolution corresponds to denser sampling of the same underlying function. Similarly, one can formalise a video clip as a mapping $f : [0,1]^2 \times \mathbb{R}^+ \to [0,1]^3$ (space-time to colour).

As a single data instance can be represented as a continuous function, we can fit a neural network to approximate this function, yielding a neural representation of that data point [1554]. As this representation is implicitly defined through a neural network, it is typically referred to as an *implicit neural representation (INR)*. In practice, this involves treating each data point as a dataset of coordinate-value pairs (c, v), and *overfitting* a small multilayer perceptron (MLP) f to predict v from c.

This approach suggests that we could use INRs for lossy data compression: note that compressing an INR's weights amounts to compressing the single data point it was fit to [465]; the data can be reconstructed by evaluating the INR at each coordinate. However, compressing the network weights directly is non-trivial: without specific treatment, the weights are continuous real values, and storing them at, e.g., 16-bit or 32-bit precision is highly inefficient. A common approach is to first quantise the weights to a lower-precision representation and entropy-code them. Unfortunately, INRs are highly sensitive to quantisation error. This contrasts with VAEs, where a powerful decoder network can learn to capture and compensate for the quantisation artefacts; in INRs, the learned MLP itself is the decoder, so any non-negligible quantisation error directly degrades reconstruction quality. Adding uniform noise during training is also suboptimal: different weights play different roles and have unequal sensitivities, while injecting the same bandwidth of uniform noise everywhere treats all parameters equally and hence will perform poorly.

The discussion above suggests a natural next step: instead of a single set of weights for the INR, we place a probability model over them. Concretely, we treat the INR weights as a random variable Y, specify a prior distribution P_Y, and, given a specific data point $X{=}x$, learn a posterior (by variational approximation) $Q_{Y|X=x}$ that captures which weights are plausible for this particular signal. This "Bayesian INR" provides us with a key advantage that aligns well with compression, as the uncertainty can adapt across layers and parameters, allowing noise to be injected more where the network is insensitive and less where it is fragile.

A key empirical observation for using Bayesian INRs for data compression is that the decoder does not need the full variational distribution: the reconstructions $\hat{x}$ we obtain using different weight samples from $Q_{Y|X=x}$ tend to be very similar. This observation immediately motivates using REC to encode only a single such realisation. Concretely, given a prior distribution P_Y and a variational posterior $Q_{Y|X=x}$ over the weights of an INR for a data point $X = x$, the sender can *encode a single stochastic realisation* $y \sim Q_{Y|X=x}$ with REC, and the receiver can recover the reconstructed data point using the INR with the weight sample y.

Now, we only need to consider how to obtain the variational posterior $Q_{Y|X=x}$ for a given data point, along with the prior distribution P_Y shared across all data points in one modality. We will explain these two parts respectively:

Inferring the posterior distribution $Q_{Y|X=x}$. Let $x = \{(c_i, v_i)\}_{i=1}^n$ be the coordinate–value pairs for a single data point x and let f_y be the INR with weights y. We train $Q_{Y|X=x}$ by balancing reconstruction quality and rate with the standard objective for variational inference, known as the negative β-Evidence Lower Bound, similarly

to other lossy neural compression techniques we already mentioned in Section 41.3.3:

$$\mathcal{L}_{\beta,x} = \underbrace{\sum_{(c,v)\in x} \mathbb{E}_{y\sim Q_{Y|X=x}}\big[\mathrm{MSE}\big(v, f_y(c)\big)\big]}_{\text{distortion (reconstruction error)}} + \underbrace{\beta\, \mathrm{D}_{\mathrm{KL}}\big(Q_{Y|X=x}\big\|P_Y\big)}_{\text{rate (approximate codelength)}} . \tag{42.21}$$

The parameter $\beta > 0$ controls the rate–distortion trade-off: larger β reduces bits at the cost of a larger compression error; smaller β yields better reconstructions but larger file size. We can see that, similar to the VAE case, this single objective both trains the INR and predicts its approximate codelength under REC.

Learning the prior P_Y. Data points within a domain tend to share structure. If we can learn a good prior P_Y from a *training set* that captures these commonalities in INR weights, we should expect better coding performance on the *test set*. But what makes a "goodp̈rior for compression more precisely? A good prior should minimise the *average* negative β-ELBO across training points, when the variational posterior for each training data point is optimised (see Chapter 12). This becomes a *nested optimisation task*. Therefore, a simple and effective recipe is the following coordinate descent-style algorithm:

1. **Fit posteriors for each training data point using the current prior.** For each x_i in the training set, minimise $\mathcal{L}_{\beta,x_i}$ w.r.t. $Q_{Y|X=x_i}$, parametrised as a tractable family (in practice, a diagonal Gaussian), where P_Y is given by the current prior.

2. **Fit the prior using the current posteriors for all training data.** For all x_i in the training set, minimise $\sum_i \mathcal{L}_{\beta,x_i}$ w.r.t. P_Y, where $\{Q_{Y|X=x_i}\}_i$ is given by the posteriors learned in the last step. In fact, this step adopts a closed-form solution if we choose P_Y to also be a diagonal Gaussian.

Putting everything above together, we obtain a lossy codec using REC and INRs. A few practical concerns remain: for example, what to do when $\mathrm{D}_{\mathrm{KL}}\big(Q_{Y|X=x}\big\|P_Y\big)$ is large; how to further reduce redundancy in INR weights; and how to handle very large data points. We refer readers interested in these details and more to the works of Guo et al. [688], Havasi et al. [715], He et al. [719].

Part IX

Applications

Chapter 43

Bayesian LLMs

Modern LLMs show outstanding performance in many scenarios and have taken the world by storm in just a few years. They are used for many tasks like complex reasoning, for example, math problem solving, theorem proving or question answering, as well as more traditional tasks like machine translation, summarization, or simply to improve the writing of an e-mail. And the list goes on and on. However, despite their massive success and widespread adoption, they are not without failure and many open problems persist, for example, calibration, uncertainty estimation and adaptation to new knowledge.

Bayesian methods are attractive for these problems already from a high-level point of view: the core of Bayesian learning, Bayes' theorem, provides simple but powerful principles for model adaptation and prediction with model averaging should be more reliable as biases of single models are averaged out. When looking at Bayesian methods more in-depth one finds even further convincing arguments: Bayesian training can lead to flatter minima and model averaging allows for better treatment of uncertainty by decomposition of its sources.

Still, while many standard LLM techniques can have Bayesian interpretations [883], Bayesian methods are arguably not popular for LLMs yet. Indeed, conventional wisdom is that Bayesian methods are slow, do not work well, and are cumbersome to implement. In this chapter, we intend to show that this does not need to be true.

First, we show that existing LLMs can be improved by "making them more Bayesian" post-hoc in Section 43.1. Then, we show how LLMs can directly be trained using Bayesian methods in Section 43.2, how model averaging for LLMs can be done in a large output space (Section 43.3), Bayesian methods for uncertainty estimation (Section 43.4) and, finally, model merging from a Bayesian point of view (Section 43.5).

43.1 Bayesian fine-tuning of LLMs

Applying Bayesian inference to the weights of an LLM is challenging because of the model's extremely high-dimensional parameter space (which might nowadays contain over one trillion parameters). This scale creates substantial difficulties for both posterior inference and posterior predictive inference:

1. During "training time" (i.e. posterior inference), approximate algorithms can suffer from slow mixing (e.g. MCMC-based approaches) or high gradient variance (e.g. VI-based approaches) due to the dimensionality of the posterior.

2. At "test time" (i.e. posterior predictive inference), Bayesian model averaging can be hard, since each sample from the posterior might require multiple gigabytes of disk storage, and repeatedly loading and unloading samples between the CPU and GPU might incur prohibitive overhead.

In addition to these practical challenges, the motivation for performing Bayesian inference over LLM weights is not immediately clear. Pre-trained LLMs are often well calibrated [1262], largely due to the enormous scale of their pre-training corpora. Consequently, recent works on Bayesian LLMs focus on the *fine-tuning* setting, in which models are trained on a small target dataset. The resulting fine-tuned models demonstrate significant overfitting, poor calibration, and a lack of uncertainty quantification. This is especially critical in safety-sensitive applications or when training data are scarce.

Parameter-efficient fine-tuning (PEFT) makes Bayesian fine-tuning a practical possibility. In particular, one widely adopted PEFT method is low-rank adaptation (LoRA) [788], which freezes the full pre-trained weight matrix $W \in \mathbb{R}^{D_{\text{out}} \times D_{\text{in}}}$. Only the perturbation to the pre-trained weights, $\Delta \boldsymbol{W} = \alpha \boldsymbol{B}\boldsymbol{A}$, is trained, where $\boldsymbol{B} \in \mathbb{R}^{D_{\text{out}} \times r}$ and $\boldsymbol{A} \in \mathbb{R}^{r \times D_{\text{in}}}$ are trainable low-rank matrices with a small rank $r \ll \min(D_{\text{in}}, D_{\text{out}})$ and constant scaling factor

$\alpha \in \mathbb{R}^+$. The forward pass of LoRA is

$$\boldsymbol{h} = (\boldsymbol{W} + \Delta\boldsymbol{W})\boldsymbol{x} = \boldsymbol{W}\boldsymbol{x} + \alpha\boldsymbol{B}\boldsymbol{A}\boldsymbol{x}. \tag{43.1}$$

Typically, $\boldsymbol{A}$ is initialized randomly, while $\boldsymbol{B}$ is initialized as a zero matrix. Thus, at initialization and start of fine-tuning, the forward pass is equivalent to that of the pre-trained model. During fine-tuning, the low-rank adapters are gradually learned. This reduces the number of trainable parameters from $D_{\text{in}}D_{\text{out}}$ to $r(D_{\text{in}} + D_{\text{out}})$ per module, and LoRA is implemented, for example, in the PEFT library [1129].

The advantages of performing Bayesian inference on $\boldsymbol{B}$ and $\boldsymbol{A}$, rather than $\boldsymbol{W}$, during fine-tuning are clear. The dimensions of $\boldsymbol{B}$ and $\boldsymbol{A}$ are much smaller than $\boldsymbol{W}$, making approximate inference engines easier to run. Bayesian model averaging becomes more feasible, since now each sample is a low-rank adapter rather than a full model, allowing many posterior samples to be loaded into GPU memory simultaneously.

Two early works in this direction are LoRA ensemble [1711] and Laplace-LoRA [1791], which apply Deep Ensemble and post-hoc linearized Laplace approximation on LoRA during fine-tuning, respectively.

43.1.1 Evaluation

Before describing the methodology, we first outline the evaluation setting for Bayesian fine-tuning. Note that in the Bayesian deep learning literature, a typical protocol involves training a model from scratch and then evaluating metrics such as likelihood, predictive entropy, or expected calibration error (ECE) on out-of-distribution (OOD) data. The usual promise of Bayesian deep learning approaches is that models will exhibit higher uncertainty on OOD data, whereas the standard point-estimation approach shows degraded accuracy and overconfident predictions.

Similarly, in Bayesian fine-tuning, we adapt the LLM to small multiple-choice QA datasets, such as CommonsenseQA [1584], MMLU [734], or AI2 Reasoning Challenge [312]. The LLM is trained to classify among the answer options given a question prompt.

For evaluation, we test the Bayesian fine-tuned LLM on the held-out evaluation split, where we expect the model to show high accuracy, low ECE, and low negative log-likelihood (NLL). Compared to a point-estimate LoRA, we anticipate fewer confident but incorrect predictions. For OOD evaluation, we choose datasets that differ significantly from the training data, e.g., QA questions in other topics or data subcategories, where we expect the Bayesian approaches to LoRA to show higher uncertainty, allowing us to detect the OOD samples.

43.1.2 LoRA ensemble

To address overconfidence and poor calibration, LoRA ensemble [1711] trains multiple LoRA adapters independently using different initializations. Then, at test time, we average the predictive distributions of the M ensemble members (Section IV). Because storing and loading multiple full copies of LLMs onto the GPU at test time is prohibitively expensive, all ensemble members share the same pre-trained model, and we only fine-tune the LoRA adapters. Thus, the storage and model-loading overhead is nearly the same as when using the original model. A single LLaMA 13B model at 16-bit precision requires 25 GB on disk and takes approximately 6 seconds to load, while a LoRA adapter of rank 8 only takes 30 MB on disk and takes 0.1 seconds to load. This highlights the practical advantage of LoRA ensemble over storing multiple complete LLMs in memory. Importantly, in LoRA ensemble, the large pre-trained model is loaded only once.

The idea is simple: we average the predictive distributions of each individual LoRA:

$$p_{\text{ens}}(\boldsymbol{y} \mid \boldsymbol{x}) = \frac{1}{M}\sum_{m=1}^{M} p(\boldsymbol{y} \mid \boldsymbol{x}; \boldsymbol{W} + \alpha\boldsymbol{B}_m\boldsymbol{A}_m)\,. \tag{43.2}$$

The different adapters $\boldsymbol{A}_m$ and $\boldsymbol{B}_m$ provide the necessary diversity, together with the randomness from the data shuffling (i.e. SGD noise). LoRA ensemble gives considerable improvements in accuracy and calibration, compared to single-component LoRA fine-tuning, last-layer fine-tuning, Monte Carlo dropout, and few-shot prompting (where a few examples of questions and answers are appended in front of the prompt). LoRA ensemble can also be combined with standard regularization approaches, such as weight decay, KL regularization, and early stopping, which keep the output distribution close to that of the pre-trained model. This can generally further improve the predictive accuracy and uncertainty quantification. Furthermore, using larger M improves all metrics, yet the gains become less substantial as M increases (the main experiments use $M = 5$). Lastly, LoRA ensembling is also helpful in OOD settings, leading to lower NLL and ECE. However, it does not necessarily improve OOD detection performance.

43.1.3 Laplace-LoRA

Laplace-LoRA [1791] addresses the calibration of the fine-tuned model by applying a Laplace approximation (Section IV) to the posterior over the LoRA adapters. The key advantage of the method is that the Laplace approximation is used post-hoc, therefore, both pre-training and fine-tuning remain unchanged. Computing Laplace approximation requires computing a $P \times P$ curvature matrix, where P is the number of parameters, which for a full model is infeasible. Laplace-LoRA mitigates this by considering only the LoRA weights. For LLaMA 2 7B, this corresponds to only approximately 6 million parameters (around 0.1% of the total model parameters). This scale remains impractical and can be further addressed by a structured approximation, a block-diagonal Kronecker-factored approximation (KFAC). Each adapter $\Delta \boldsymbol{W} = \boldsymbol{B}\boldsymbol{A}$ is treated as two separate linear layers, with weights $\boldsymbol{A}$ and $\boldsymbol{B}$. However, for $\boldsymbol{A} \in \mathbb{R}^{r \times D_{\text{in}}}$ and $\boldsymbol{B} \in \mathbb{R}^{D_{\text{out}} \times r}$, one Kronecker factor is small ($r \times r$), whereas the other is large, with dimensions ($D_{\text{in}} \times D_{\text{in}}$) for $\boldsymbol{A}$ or ($D_{\text{out}} \times D_{\text{out}}$) for $\boldsymbol{B}$. Laplace-LoRA computes a low-rank approximation of the large Kronecker factor. Additionally, the method allows the prior precision λ to be optimized using marginal likelihood, and does not require a separate held-out validation set which is particularly beneficial for very small datasets. All in all, the resulting posterior approximation is:

$$p(\boldsymbol{A}, \boldsymbol{B} \mid \mathcal{D}) \approx \mathcal{N}\left(\begin{bmatrix} \text{vec}(\boldsymbol{A}_*) \\ \text{vec}(\boldsymbol{B}_*) \end{bmatrix}, \Psi^{-1}_{\boldsymbol{A},\boldsymbol{B}} \right) \tag{43.3}$$

Crucially, to ensure an efficient implementation, the method uses an incremental low-rank SVD to estimate the low-rank components, without ever computing factors of size ($D_{\text{in}} \times D_{\text{in}}$) or ($D_{\text{out}} \times D_{\text{out}}$). Large values of negative log-likelihood, as well as, expected calibration error are reduced significantly, compared with MAP and other approaches such as Monte Carlo dropout, checkpoint ensemble (ensembling most recent checkpoints) or temperature scaling. Laplace-LoRA is compatible with regularized approaches, as it can be applied to any LoRA-based method that generates fine-tuned weights. Lastly, Laplace-LoRA proves to be useful in evaluations under both smaller and larger distribution shifts, with regard to NLL and ECE.

43.2 Bayesian LLM Training with Variational Learning

We discuss training LLMs directly using Bayesian methods. While conventional LLM training predominantly learns point estimates $\boldsymbol{\theta} \in \mathbb{R}^P$ of one specific LLM, we instead aim to obtain a distribution over them which also quantifies uncertainty about the parameter values. While this is also possible by using conventional point estimation followed by Laplace's method, as we have seen in the previous section, we here provide arguments why training a distribution directly can be beneficial. We will show methods that work even at large scales, contrary to the popular belief that Bayesian methods are not scalable to large problems like language modeling. The methods presented here use the framework of variational learning and the natural-gradient-based algorithms which we have introduced formally in Section 15.1. In the following, we will briefly review the formal introduction and then go over to practical aspects of scaling variational learning to LLMs, where we first discuss training, then prediction, and finally adaptation to new data.

43.2.1 Variational Learning for Language Models

Modern LLM training consists of a sequence of multiple stages. The first stage generally uses a language modeling loss,

$$\ell(\mathcal{D}, \boldsymbol{\theta}) = -\sum_{\boldsymbol{y} \in \mathcal{D}} \log p(\boldsymbol{y} \mid \boldsymbol{\theta}) \tag{43.4}$$

$$= -\sum_{\boldsymbol{y} \in \mathcal{D}} \log \left[\prod_{t=1}^{|\boldsymbol{y}|} p(y_t \mid y_1, \ldots, y_{t-1}, \boldsymbol{\theta}) \right], \tag{43.5}$$

that aims to maximize the likelihood of tokens in a large collection of texts $\mathcal{D}$, where $|\mathcal{D}|$ can be on the order of trillions of tokens. The composition of $\mathcal{D}$ is then often changed in later stages of training, for example, to include more math and code data in mid-training, or to include data used for safety tuning in post-training, where the model is fine-tuned for specific capabilities [967, 1139, 1700]. These capabilities can also include input-specific data, denoted by $\boldsymbol{x}$. These inputs often also contain human instructions given to the model [213]. In addition to the data composition, the training criterion often changes, as well. For example, to improve the reasoning capabilities of models, reinforcement learning is often used, where the loss is not always a likelihood anymore but

might be a reward assigned by some heuristic function or a learned model [1273]. Still, in any case the optimizer is usually not switched but only the loss function and gradient estimator.

When training with Bayesian methods we would like to have the same property: the loss function might change but ideally the overall framework and optimization algorithm should stay the same to ease implementation. Variational learning provides such a framework and aims to approximate a posterior $p(\boldsymbol{\theta} \mid \mathcal{D})$ by a (generalized) posterior:

$$q_*(\boldsymbol{\theta}) \propto p_0(\boldsymbol{\theta}) \prod_{i=1}^{N} \exp\{-\ell(\mathcal{D}_i, \boldsymbol{\theta})\}. \tag{43.6}$$

Here p_0 is a prior and $\ell(\mathcal{D}_i, \boldsymbol{\theta})$ can be any loss, though we usually assume it is at least once differentiable. The posterior is learned using the following variational objective:

$$q_*(\boldsymbol{\theta}) = \underset{q(\boldsymbol{\theta}) \in \mathcal{Q}}{\arg\min}\, \mathbb{E}_q \left[\sum_{i=1}^{N} \ell(\mathcal{D}_i, \boldsymbol{\theta}) \right] + \mathbb{D}_{\mathrm{KL}}(q \,\|\, p_0), \tag{43.7}$$

where $\mathcal{Q}$ is a class of distributions, for example, Gaussian distributions to make optimization tractable. For language modeling the objective can then also be written as

$$q_*(\boldsymbol{\theta}) = \underset{q(\boldsymbol{\theta}) \in \mathcal{Q}}{\arg\min} -\mathbb{E}_q \left[\sum_{\boldsymbol{y} \in \mathcal{D}} \log \left[\prod_{t=1}^{|\boldsymbol{y}|} p(y_t \mid y_1, \ldots, y_{t-1}, \boldsymbol{\theta}) \right] \right] + \mathbb{D}_{\mathrm{KL}}(q \,\|\, p_0) \tag{43.8}$$

but we are now not bound to optimizing likelihoods anymore and can easily change the loss function, also during optimization, for the different stages of language model training. Oftentimes, the loss is also normalized by sequence length to improve training stability.

43.2.2 IVON for LLMs

Algorithm 35: Improved Variational Online Newton (IVON). Main differences to Adam in grey. Note that we overload / for notational convenience.

Inputs : learning rates $\{\rho_t\}$; gradient momentum $\beta_1 \in [0, 1)$; Hessian momentum $\beta_2 \in [0, 1)$; weight decay $\delta > 0$; scaling parameter $\lambda > 0$; Hessian initialization $h_0 > 0$, clip radius $\xi > 0$

Outputs: mean parameter $\boldsymbol{m}_{t+1}$ and variance $\boldsymbol{\sigma}^2_{t+1}$

initialize $\boldsymbol{m} \leftarrow$ (NN weight init), $\boldsymbol{h} \leftarrow h_0 \cdot \boldsymbol{I}$, $\boldsymbol{\sigma^2} \leftarrow 1/(\lambda(\boldsymbol{h} + \delta))$, $\boldsymbol{g} \leftarrow 0$, $t \leftarrow 1$, and $\alpha_t \leftarrow (h_0 + \delta)\alpha_t$ for all t

repeat

- $\hat{\boldsymbol{g}} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}_t} \ell_i(\boldsymbol{\theta}_t)$, where $\theta_t \sim q_t(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{m}_t, \mathrm{diag}(\boldsymbol{\sigma}_t^2))$
- $\hat{\boldsymbol{h}} \leftarrow \hat{\boldsymbol{g}} \cdot (\boldsymbol{\theta}_t - \boldsymbol{m}_t)/\boldsymbol{\sigma}_t^2$
- $\boldsymbol{g} \leftarrow \beta_1 \boldsymbol{g} + (1 - \beta_1)\hat{\boldsymbol{g}}$
- $\boldsymbol{h} \leftarrow \beta_2 \boldsymbol{h}_t + (1 - \beta_2)\hat{\boldsymbol{h}} + \frac{1}{2}(1 - \beta_2)^2(\boldsymbol{h}_t - \hat{\boldsymbol{h}})^2/(\boldsymbol{h}_t + \delta)$
- $\bar{\boldsymbol{g}} \leftarrow \boldsymbol{g}/(1 - \beta_1^t)$
- $\boldsymbol{m}_{t+1} \leftarrow \boldsymbol{m}_t - \alpha_t \,\mathrm{clip}((\bar{\boldsymbol{g}} + \delta \boldsymbol{m}_t)/(\boldsymbol{h} + \delta), \xi)$
- $\boldsymbol{\sigma}^2_{t+1} \leftarrow 1/(\lambda(\boldsymbol{h} + \delta))$

until *convergence*

return $\boldsymbol{m}_{t+1}, \boldsymbol{\sigma}^2_{t+1}$

Interestingly, practical algorithms that optimize Equation 43.7 take a very similar form to optimizers that are already known to work well for deep learning, like Adam [894], but now calculate natural gradients to quantify steepest descent in terms of distributions and not point estimates.[1] One of these optimizers is the IVON optimizer shown in Algorithm 35, where the main differences to Adam are highlighted in grey. IVON is formally derived in Section 15.3.2.

The first difference is that IVON learns a Gaussian distribution over model parameters instead of a point estimate. Usually a diagonal covariance is modeled for LLMs but a non-diagonal covariance can also be learned using a recent extension called EVON [1184]. Both of these choices have a similar computational and memory

[1] This is explained in more detail in Section 15.1.

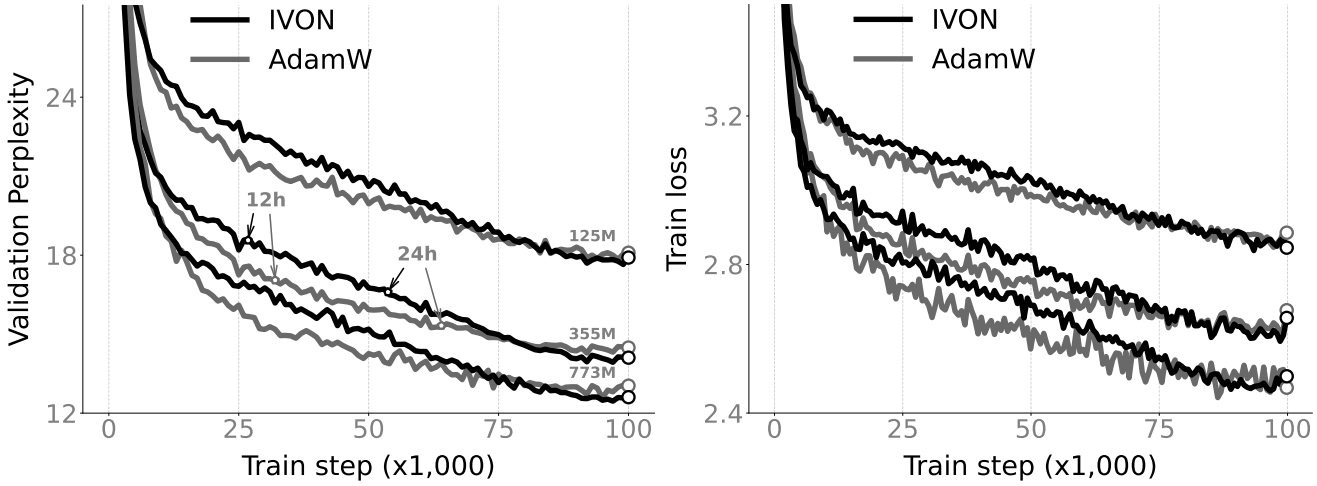


Figure 43.1: Results when pretraining different GPT-2 models using IVON from scratch compared to AdamW. The figure shows validation perplexity (left) and train loss (right). In both cases IVON can improve over AdamW at similar cost. Figure adapted from Shen et al. [1510] with permission.

overhead as AdamW. For example, in the experiments shown in Figure 43.1 IVON has an overhead of ca. 1.2x. The variance in IVON is defined using the inverse of a diagonal Hessian approximation and therefore the update is also preconditioned by the inverse of a diagonal Hessian approximation, instead of a square root second moment estimate in AdamW. In IVON, the diagonal Hessian is estimated by using the reparameterization trick instead of a square of minibatch gradients but the estimator could be switched in principle. Finally, the Hessian estimate is ensured to stay positive by the additional $(1-\beta_2)^2(\cdot)$ term in its update.

One benefit of the reparameterization trick is that it requires sampling parameters at each iteration, which is equivalent to adding learned noise to the current model. As a consequence, IVON constantly explores local neighborhoods of the current mean during optimization which can keep it away from sharp minima and push it to flat minima [620] which have been associated with, among others, improved generalization [544].

With these changes IVON has been successfully used to train language models up to 773M parameters following the GPT-2 architecture in Shen et al. [1510]. The results are summarized in Figure 43.1, where IVON even gets better validation perplexity (evaluated at the mean of the distribution) at the end of training. Consistent with the language modeling loss, perplexity is defined as follows for a corpus $\mathcal{D}$:

$$\mathrm{PPL}(\mathcal{D}, \boldsymbol{\theta}) \coloneqq \exp\left(-\frac{1}{\sum_{\boldsymbol{y}\in\mathcal{D}} |\boldsymbol{y}|} \sum_{\boldsymbol{y}\in\mathcal{D}} \sum_{t=1}^{|\boldsymbol{y}|} \log p(y_t \mid \boldsymbol{y}_{<t}, \boldsymbol{\theta})\right), \tag{43.9}$$

where the shorthand $\boldsymbol{y}_{<t}$ describes $\boldsymbol{y}$ up until the $(t-1)$-th token. The authors also show that sampling models from the posterior and averaging their predictions can improve perplexity. In order to make IVON work for Transformer LLMs, the preconditioned gradient is clipped to a value ξ which can be set, for example, to 10^{-4}. In general, hyperparameters differ quite a bit from AdamW and are important to make the method work. Typically, the gradient momentum $\beta_1 = 0.9$ is similar but β_2 is often chosen to be much larger, for example, 0.99999 or 0.9999 which stabilizes training as the reparameterization-trick-based Hessian estimator can be noisy. The noise can also be reduced by averaging the gradient estimates over multiple Monte-Carlo estimates for each step instead of using one sample. For LLMs it is also important to tune the Hessian initialization h_0 and the effective sample size λ, because both are directly related to the amount of noise injected during optimization. Larger values can stabilize training and therefore it can be good practice to treat it as a hyperparameter rather than using the dataset size directly, especially when fine-tuning. Typically, values like $\lambda = 10^7$ and $h_0 \in [0.01, 1.0]$ can work well but other combinations are also possible and it is up to the practitioner to balance exploration via noise injection and optimization stability. The view as an inverse temperature parameter is justified by the authors of Shen et al. [1510] by rewriting the variational objective as:

$$q_*(\boldsymbol{\theta}) = \underset{q(\boldsymbol{\theta})\in\mathcal{Q}}{\arg\min}\, \mathbb{E}_q\left[\sum_{i=1}^{N} \bar{\ell}(\mathcal{D}_i, \boldsymbol{\theta})\right] - \frac{1}{\lambda}\mathcal{H}(q). \tag{43.10}$$

Here, the rewriting is due to choosing a regularizer $r(\boldsymbol{\theta})$ such that the prior is defined as $p_0(\boldsymbol{\theta}) \propto \exp(-r(\boldsymbol{\theta}))$ [883]. The regularizer is subsumed in the loss which is averaged over all data examples and denoted with $\bar{\ell}$. The rewriting shows that smaller λ prefer a larger entropy in q which increases diversity in models but can decrease data-fit.

43.2.3 LoRA Fine-tuning with IVON

Recently, IVON has also been used to improve LoRA fine-tuning, where the distribution is not learned over the whole model but rather only over the LoRA parameters [326]. Still, many of the benefits of Bayesian Deep Learning are retained and, in particular, the method can achieve improvements in both accuracy and calibration at the same time. The authors also introduce a pruning method based on the posterior which they call uncertainty-guided pruning. There, the model parameters are pruned via their posterior variance

$$\boldsymbol{\sigma}_i^{-2} = \lambda(\boldsymbol{h} + \delta), \tag{43.11}$$

where $\boldsymbol{h}$ is the Hessian estimate from IVON and δ is a standard weight-decay. The diagonal Hessian can be seen as an estimate of (inverse) uncertainty over the parameter: a larger Hessian means less uncertainty as the variance is small and the posterior at this point is concentrated for the given parameter and vice versa. The most uncertain parameters can be pruned to improve efficiency and performance.

The parameter uncertainty is also useful because it allows us to sample models by adding noise to the mean proportional to the variance. Then, we can generate language by taking an expectation over predictions across (samples from) the approximate posterior and also, again, adjust the (inverse) temperature to make predictions more or less diverse. In the following sections we will deal with this in greater detail and outline practical language generation methods based on Bayesian Deep Learning.

43.3 Language Generation with Predictive Posteriors

Language generation is challenging, because the output space is huge and many outputs can be equally valid for a given input. Which of these valid outputs is prefered often depends on user preference. Bayesian methods can deal with such uncertainty via predictive posteriors, where predictions are averaged across models, but applying them requires dealing with nuances, starting from simply defining predictive posteriors to estimating expected risks that can be used to account for preference. Here, we will first review common language generation strategies and then derive generalized Bayesian strategies.

Suppose we are given some input $\boldsymbol{x} \in \mathcal{V}^*$, where the Kleene closure * constructs all possible finite-length outputs from a vocabulary $\mathcal{V}$ that may be based on, for example, byte-pair encoding (BPE) [1490]. We are interested in finding an output $\boldsymbol{y} \in \mathcal{V}^*$ constructed of a sequence of tokens $y_1, y_2, ..., y_{|y|}$. As discussed before, LLMs usually define the probability of $\boldsymbol{y}$ autoregressively by taking the product of per-token probabilities:

$$p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{\theta}) = \prod_{t=1}^{|\boldsymbol{y}|} p(y_t \mid \boldsymbol{y}_{<t}, \boldsymbol{x}, \boldsymbol{\theta}). \tag{43.12}$$

One common strategy is to sample $y_t \sim p(\cdot \mid \boldsymbol{y}_{<t}, \boldsymbol{x}, \boldsymbol{\theta})$ from the model until a stopping criterion, for example, a special end-of-sequence symbol. However, due to the common softmax activation, which produces dense probabilities, and uncertainty over the correct y_t an incorrect token might always be sampled with some probability, for example, an incorrect fact. One might think that, perhaps, we could use the most likely token $y_t^* = \max_{y_t \in \mathcal{V}} p(\cdot \mid \boldsymbol{y}_{<t}, \boldsymbol{x}, \boldsymbol{\theta})$ instead but this greedy decoding assumes a good point estimate on the one hand, and on the other, in general, does not find the most likely *sequence*

$$\boldsymbol{y}^* = \arg\max_{\boldsymbol{y} \in \mathcal{V}^*} p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{\theta}). \tag{43.13}$$

While using a posterior $q(\boldsymbol{\theta})$ should intuitively give us better probability estimates due to averaging, we first again need to decide which probabilities should be considered. We could either define [1125]:

$$p^{\text{tok}}(\boldsymbol{y} \mid \boldsymbol{x}) \coloneqq \prod_{t=1}^{T} \mathbb{E}_q \left[p(y_t \mid \boldsymbol{y}_{<t}, \boldsymbol{x}, \boldsymbol{\theta}) \right] \tag{43.14}$$

or similarly define

$$p^{\text{seq}}(\boldsymbol{y} \mid \boldsymbol{x}) \coloneqq \mathbb{E}_q \left[p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{\theta}) \right] = \mathbb{E}_q \left[\prod_{t=1}^{T} p(y_t \mid \boldsymbol{y}_{<t}, \boldsymbol{x}, \boldsymbol{\theta}) \right]. \tag{43.15}$$

	IWSLT17 En-De			WMT18 Tr-En		
Method	BLEU	COMET	LaBSE	BLEU	COMET	LaBSE
IVON@Mean	19.73	76.60	83.51	15.27	78.44	77.12
Pred. Posterior - Equation 43.22	20.89	77.42	**84.01**	**15.66**	**79.01**	**77.79**
Pred. Posterior - Equation 43.23	**21.08**	**77.63**	83.96	15.46	78.84	77.35

Table 43.1: Results when using the IVON mean and posterior for language generation with uncertainty-aware MBR on two machine translation tasks. The loss used is the negative BERTScore and all methods use 40 output samples in total. Taken from Daheim et al. [354].

However, both will not give the same probabilities for a given $\boldsymbol{y}$ in general. In the end, this is a choice of the practitioner that also depends on implementation and storage requirements.

When attempting to find likely sequences, there is also mismatch between training and prediction: when using general losses during training, we might also want to consider them during generation, and vice versa. For example, in machine translation we would rather want an output with high quality than high probability [558] and the two do not necessarily coincide. As a consequence, estimates of quality have also been used as training criteria [517, 723]. In the following, we present generalized Bayesian language generation methods that can effectively use both posterior and losses.

43.3.1 Uncertainty-Aware Minimum Bayes Risk Decoding with Posteriors

The use of losses in language generation is not a new concept. In fact, Minimum Bayes Risk (MBR) decoding has long been used for sequence generation [947] from a point estimate $\boldsymbol{\theta} \in \mathbb{R}^p$ using the expected risk:

$$\boldsymbol{y}^* = \arg\min_{\boldsymbol{y} \in \mathcal{V}^*} \sum_{\boldsymbol{y}' \in \mathcal{V}^*} p(\boldsymbol{y}' \mid \boldsymbol{x}, \boldsymbol{\theta}) \ell(\boldsymbol{y}, \boldsymbol{y}'), \tag{43.16}$$

where $\ell : \mathcal{V}^* \times \mathcal{V}^* \to \mathbb{R}_+$ is a loss function that compares two strings. Interestingly, this algorithm generalizes many well-known strategies already. For example, we could derive Equation 43.13 by choosing a $0-1$-loss. Another example is choosing $\ell(\boldsymbol{y}, \boldsymbol{y}') = 1\{f(\boldsymbol{y}) \leq f(\boldsymbol{y}')\}$ for some $f : \mathcal{V}^* \to \mathbb{R}_+$, for example, a translation quality estimate. This recovers a ranking according to f whose approximations have recently been popularized under the term quality-aware decoding [516].

By averaging such expected risks across a posterior $q(\boldsymbol{\theta})$ we then arrive at the generalized Bayesian language generation method from Daheim et al. [354]:

$$\begin{aligned}
\boldsymbol{y}^* &= \arg\min_{\boldsymbol{y} \in \mathcal{V}^*} \mathbb{E}_q \left[\sum_{\boldsymbol{y}' \in \mathcal{V}^*} p(\boldsymbol{y}' \mid \boldsymbol{x}, \boldsymbol{\theta}) \ell(\boldsymbol{y}, \boldsymbol{y}') \right] && (43.17) \\
&= \arg\min_{\boldsymbol{y} \in \mathcal{V}^*} \sum_{\boldsymbol{y}' \in \mathcal{V}^*} \mathbb{E}_q \left[p(\boldsymbol{y}' \mid \boldsymbol{x}, \boldsymbol{\theta}) \ell(\boldsymbol{y}, \boldsymbol{y}') \right] && (43.18) \\
&= \arg\min_{\boldsymbol{y} \in \mathcal{V}^*} \sum_{\boldsymbol{y}' \in \mathcal{V}^*} p^{\text{seq}}(\boldsymbol{y}' \mid \boldsymbol{x}) \ell(\boldsymbol{y}, \boldsymbol{y}'), && (43.19)
\end{aligned}$$

where the switch of expectations is valid, because the size of $\mathcal{V}^*$ is countably-infinite, ℓ is non-negative, and sequence-level probabilities over $\boldsymbol{y}$ are used. By defining the loss,

$$\ell(\boldsymbol{y}, \boldsymbol{x}, \boldsymbol{\theta}) = \sum_{\boldsymbol{y}' \in \mathcal{V}^*} p(\boldsymbol{y}' \mid \boldsymbol{x}, \boldsymbol{\theta}) \ell(\boldsymbol{y}, \boldsymbol{y}'), \tag{43.20}$$

we can also make the correspondence to Equation 43.7 more explicit by noting that,

$$\boldsymbol{y}^* = \arg\min_{\boldsymbol{y} \in \mathcal{V}^*} \mathbb{E}_q \left[\ell(\boldsymbol{y}, \boldsymbol{x}, \boldsymbol{\theta}) \right], \tag{43.21}$$

which clearly shows how we can optimize the same objective during training and prediction.[2]

In practice, both expectations over q and $\mathcal{V}^*$ are intractable to compute. Therefore, Daheim et al. [354] use Monte-Carlo estimation which first uses the following two steps:

[2]Note that in Equation 43.7 we optimize in $\mathcal{Q}$ and the regularizer over $q(\boldsymbol{\theta})$ is constant during generation, where $q(\boldsymbol{\theta})$ is fixed.

1. Sample a set $\mathcal{M} = \{\boldsymbol{\theta}_i \mid \boldsymbol{\theta}_i \sim q(\boldsymbol{\theta})\}_{i=1}^{|\mathcal{M}|}$ of models from the posterior q.
2. For each of them, sample a hypothesis set $\mathcal{H}_{\boldsymbol{\theta}_i} = \{\boldsymbol{y}^{(j)} \mid \boldsymbol{y}^{(j)} \sim p(\cdot \mid \boldsymbol{x}, \boldsymbol{\theta}_i)\}_{j=1}^{|\mathcal{H}|}$.

The hypothesis sets are then used to estimate Equation 43.17 by using:

$$\widehat{\boldsymbol{y}}^* = \arg\min_{\boldsymbol{y}' \in \mathcal{H}_\mathcal{M}} \sum_{\boldsymbol{y} \in \mathcal{H}_\mathcal{M}} \ell(\boldsymbol{y}, \boldsymbol{y}') \tag{43.22}$$

or Equation 43.18 with

$$\widehat{\boldsymbol{y}}^* = \arg\min_{\boldsymbol{y}' \in \mathcal{H}_\mathcal{M}} \sum_{\boldsymbol{\theta} \in \mathcal{M}} \sum_{\boldsymbol{y} \in \mathcal{H}_{\boldsymbol{\theta}}} \ell(\boldsymbol{y}, \boldsymbol{y}'). \tag{43.23}$$

Here $\mathcal{H}_\mathcal{M} \coloneqq \uplus_{\boldsymbol{\theta} \in \mathcal{M}} \mathcal{H}_{\boldsymbol{\theta}}$ is a collection of all sampled outputs from all models, where counts are kept to maintain an unbiased estimator. Intuitively, one uses the probability in Equation 43.15 for risk calculation and the other averages per-model risks which is also easier to parallelize. While both may give different estimates in practice, they are still unbiased and will converge to the same risk in the limit and its up to the practitioner to choose. It is also possible to derive a similar decision rule for token-level averaging by replacing the probability in Equation 43.19 with a token-level averaging.

In Daheim et al. [354], the authors find that both methods perform better than using the mean of an approximate posterior learned with IVON. We show an excerpt of their results on two machine translation tasks, namely the IWSLT17 [251] en-de and WMT18 [182] tr-en task in Table 43.1. The model is GEMMA-2B [604] fine-tuned using IVON-LoRA [326]. The loss function is the negative of the BERTScore [1857] between two utterances and each method uses 40 samples in total which means that 40 samples were taken from the mean and 10 for each of the 4 sampled models from the posterior. The results show that, overall, the predictive posterior helps when generating language. This is also shown in Cong et al. [326], who show that scaling the number of models and samples in MBR can be an effective method for test-time compute scaling, for example, for math reasoning with LLMs. In addition, the expected risk is useful for selective prediction which we will discuss next.

43.3.2 Selective Prediction using Expected Risk

When generating language, there can also be cases where it is not possible to generate any useful output. This is covered by selective prediction, as discussed in Chapter 44. There, the idea is to refrain from presenting an output to the user if the output is expected to not be useful. For example, in machine translation we might encounter inputs that are in a language that is not supported by our translation model or in a question answering setting answering might require knowledge that is not present in the model.

The expected risk defined in the previous section can be a useful way of putting this idea into practice, as we might want to refrain from answering when risk is high. Then, it makes sense to define an expected risk across all outputs as follows:

$$\operatorname{risk}(\boldsymbol{x}, q) = \mathbb{E}_{\boldsymbol{y}} \underbrace{\mathbb{E}_q \left[\sum_{\boldsymbol{y}' \in \mathcal{V}^*} p(\boldsymbol{y}' \mid \boldsymbol{x}, \boldsymbol{\theta}) \ell(\boldsymbol{y}, \boldsymbol{y}') \right]}_{\coloneqq \operatorname{risk}(\boldsymbol{y}, \boldsymbol{x}, q)}. \tag{43.24}$$

The risk quantifies the expected risk of *all* possible outputs $\boldsymbol{y} \in \mathcal{V}^*$ across the posterior q which we obtain by comparing it to each other output $\boldsymbol{y}' \in \mathcal{V}^*$.

Again, this is clearly intractable to calculate in practice and, using notation from previous subsections, we can therefore construct an estimator

$$\operatorname{risk}(\boldsymbol{x}, q) \approx \frac{1}{|\mathcal{M}|} \frac{1}{|\mathcal{H}_{\boldsymbol{\theta}_i}|^2} \sum_{\boldsymbol{\theta}_i \in \mathcal{M}} \sum_{\boldsymbol{y} \in \mathcal{H}_{\boldsymbol{\theta}_i}} \sum_{\boldsymbol{y}' \in \mathcal{H}_{\boldsymbol{\theta}_i}} \ell(\boldsymbol{y}, \boldsymbol{y}'). \tag{43.25}$$

The authors of Daheim et al. [354] show that this can be useful for selective prediction, especially when the $\mathcal{H}$ are created with ancestral sampling from a diverse set of models. In IVON, this can also be achieved by decreasing λ during prediction.

43.4 Uncertainty Estimation for Autoregressive LLMs

Bayesian methods are also attractive because they can improve predictive uncertainty estimation and enable distinguishing different sources of uncertainty. Most commonly, a distinction is made between uncertainty stemming

from a lack of knowledge, often called epistemic uncertainty, and uncertainty stemming from randomness in the data itself, often called aleatoric uncertainty [415, 804]. While the former is assumed to be reducible by observing (and training on) more data, the latter is generally thought of as irreducible, because it reflects inherent uncertainty. A simple canonical example is (unbiased) coin tossing, where a model trained on few coin tosses might not equally assign probability to heads and tails, but observing a sufficient amount of data will eventually lead to convergence to a uniform distribution. In LLMs randomness is clearly much more nuanced. For example, it can stem from many outputs being equally valid, speaker perspectives, data collection, and so on. Then, whether certain sources are reducible may even be a choice. We refer the reader to Baan et al. [70] for a further discussion and instead discuss information-theoretic measures for disentangling uncertainties next.

There, it is usually assumed that total uncertainty when predicting any $\boldsymbol{y}$ given $\boldsymbol{x}$ is captured by the entropy of the predictive distribution. That is, the entropy captures both reducible and irreducible uncertainty. In order to decompose it, we will use the mutual information between $\boldsymbol{y}$ and the network weights $\boldsymbol{\theta}$ which tells us how much information about $\boldsymbol{\theta}$ we can gain by observing $\boldsymbol{y}$ and is given by subtracting a measure of aleatoric uncertainty from total uncertainty [804, 1125]:

$$\mathcal{I}[\boldsymbol{y}, \boldsymbol{\theta} \mid \boldsymbol{x}] = \underbrace{\mathcal{H}[p^{\text{seq}}(\boldsymbol{y} \mid \boldsymbol{x})]}_{\text{Tot. Uncertainty}} - \underbrace{\mathbb{E}_q\left[\mathcal{H}[p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{\theta})]\right]}_{\text{Aleat. Uncertainty}}. \tag{43.26}$$

However, as so often, calculating these quantities exactly is not feasible and instead approximations have to be used, such as Monte-Carlo approximations. These can also be tricky for sequence-level model averaging and as a consequence token-level model averaging may be used, too, but as we discussed previously is not guaranteed to be consistent with sequence-level modeling. We refer the reader to Malinin and Gales [1125] for an in-depth discussion of such estimators and, in particular a comparison of token- and sequence-level modeling.

We also note that these measures are clearly related to the expected risk formulations we have seen in the previous section via a suitable loss function. Using these intuitions, Vashurin et al. [1668] therefore combine information-theoretic measures and general loss functions for improved uncertainty estimation and compare this approach to a variety of baselines, especially when estimating uncertainty given only a single output sample.

Monte-Carlo estimators are also challenging because for LLMs it can be unlikely that the same sequence will be sampled multiple times, even though many sequences share the same meaning. Therefore, Kuhn et al. [942] group sequences with the same meaning before calculating entropy which they call *semantic uncertainty* and which has shown state-of-the-art performance in many settings. Let us denote a cluster of sequences with the same meaning as c. Then, we can define the probability of c for given $\boldsymbol{\theta}$ as

$$p(c \mid \boldsymbol{x}, \boldsymbol{\theta}) \coloneqq \sum_{\boldsymbol{y} \in c} p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{\theta}). \tag{43.27}$$

For a set of meanings C, semantic uncertainty is then given by the redefined entropy

$$SE[p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{\theta})] \coloneqq -\sum_{c \in C} p(c \mid \boldsymbol{x}, \boldsymbol{\theta}) \log p(c \mid \boldsymbol{x}, \boldsymbol{\theta}). \tag{43.28}$$

Although extensions to Bayesian modeling with predictive posteriors are not used in Kuhn et al. [942], these can be easily obtained using the definitions of total, aleatoric, and epistemic uncertainty outlined in this section. In fact, under this lens Equation 43.28 may already be seen as an approximation to Bayesian methods by approximating the expectation over an approximate posterior at the mean. This is often practical for LLMs which might only be available as a point estimate.

43.5 Bayesian Model Merging for LLMs

Recently, model merging has gained attention as a way to combine the information from multiple models $\boldsymbol{\theta}_t$ that were trained separately on T different $\mathcal{D}_t$, for example, by a simple weighted-averaging of their parameters using scaling factors $\alpha_t > 0$:

$$\widehat{\boldsymbol{\theta}}_{1:T} = \sum_{t=1}^{T} \alpha_t \boldsymbol{\theta}_t, \tag{43.29}$$

where the α_t usually sum to 1. This can be seen as approximating a model $\boldsymbol{\theta}_{1:T}$ that is trained on all $\mathcal{D}_t$ altogether with a similar weighting [353], for example by minimizing an empirical risk with a weight decay:

$$\boldsymbol{\theta}_{1:T} = \underset{\boldsymbol{\theta} \in \mathbb{R}^p}{\arg\max} \sum_{t=1}^{T} \alpha_t \ell(\mathcal{D}_t, \boldsymbol{\theta}) + \frac{1}{2}\|\boldsymbol{\theta}\|_2^2. \tag{43.30}$$

We largely follow Maldonado et al. [1124] to connect such methods to Bayesian methods. To a Bayesian, the model merging in Equation 43.29 might seem familiar as the averaging of models should intuitively correspond to a multiplication of likelihoods which can be used to construct a posterior. Using similar weights α_t we can write a posterior $p(\boldsymbol{\theta} \mid \mathcal{D}_{1:T})$ by multiplying likelihoods $p(\mathcal{D}_t \mid \boldsymbol{\theta}) \propto p(\boldsymbol{\theta} \mid \mathcal{D}_t)/p_0(\boldsymbol{\theta})$ using prior $p_0(\boldsymbol{\theta})$ as follows:

$$p_\alpha(\boldsymbol{\theta} \mid \mathcal{D}_{1:T}) \propto p_0(\boldsymbol{\theta}) \prod_{t=1}^{T} p(\mathcal{D}_t \mid \boldsymbol{\theta})^{\alpha_t} \propto p_0(\boldsymbol{\theta})^{\gamma} \prod_{t=1}^{T} p(\boldsymbol{\theta} \mid \mathcal{D}_t)^{\alpha_t} \tag{43.31}$$

for $\gamma = 1 - \sum_{t=1}^{T} \alpha_t$. If we define our likelihoods and prior wisely under the generalized Bayesian framework as,

$$p(\mathcal{D}_t \mid \boldsymbol{\theta}) \propto \exp(-\ell(\boldsymbol{\theta}, \mathcal{D}_t)) \qquad p_0(\boldsymbol{\theta}) \propto \exp(-\|\boldsymbol{\theta}\|_2^2)., \tag{43.32}$$

we can indeed already recover the solutions to Equation 43.30 by taking the arg max of the logarithm of Equation 43.31 as follows,

$$\widehat{\boldsymbol{\theta}}_{1:T} = \arg\max_{\boldsymbol{\theta} \in \mathbb{R}^p} \log p_\alpha(\boldsymbol{\theta} \mid \mathcal{D}_{1:T}) = \arg\max_{\boldsymbol{\theta} \in \mathbb{R}^p} \sum_{t=1}^{T} \alpha_t \log p(\boldsymbol{\theta} \mid \mathcal{D}_t) + \gamma \log p_0(\boldsymbol{\theta}), \tag{43.33}$$

where the last equality is just a rewriting of the posterior in log space that makes the similarity to the original multitask objective explicit. This is not very practical yet, though, as we usually can not find the exact $p(\boldsymbol{\theta} \mid \mathcal{D}_t)$ and prior $p_0(\boldsymbol{\theta})$. Therefore, we will again make use of variational methods which restrict these to a specific class of distributions and thereby allow closed form expressions for the mode of the posterior.

43.5.1 Variational Model Merging

In variational methods, we restrict the posterior and prior to lie in a certain class of distributions $\mathcal{Q}$. One convenient class are exponential families, not only because we can derive practical optimizers from them, but also because we can always find the mode of a posterior in closed form to derive a merging equation. The approximate posterior based on, for example, Gaussian $q_t(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\theta}_t, \mathcal{D}_t)$ and Gaussian prior p_0 is then:

$$q_\alpha(\boldsymbol{\theta}, \mathcal{D}_{1:T}) \propto p_0(\boldsymbol{\theta})^{\gamma} \prod_{t=1}^{T} q_t(\boldsymbol{\theta})^{\alpha_t}, \tag{43.34}$$

where q_α is again Gaussian due to conjugacy. The q_t can then be learned using a practical variational optimizer like IVON, where the posterior and prior structure determine the model merging scheme, as we again can derive the merged model $\widehat{\boldsymbol{\theta}}_{1:T}$ by using

$$\widehat{\boldsymbol{\theta}}_{1:T} = \arg\max_{\boldsymbol{\theta} \in \mathbb{R}^p} \log q_\alpha(\boldsymbol{\theta} \mid \mathcal{D}_{1:T}) = \arg\max_{\boldsymbol{\theta} \in \mathbb{R}^p} \sum_{t=1}^{T} \alpha_t \log q_t(\boldsymbol{\theta}) + \gamma \log p_0(\boldsymbol{\theta}). \tag{43.35}$$

We will provide two examples in the following subsections and refer to Maldonado et al. [1124] for more details.

43.5.2 Recovering Averaging using Variational Merging

The posterior and prior in variational merging determine the merging equation. Here, we use simple Gaussian likelihoods with fixed variance and a standard Gaussian prior, i.e.:

$$q_t(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\theta}_t, I) \qquad p_0(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; 0, I). \tag{43.36}$$

Then, merging simply corresponds to weighted averaging (note the change to arg min):

$$\widehat{\boldsymbol{\theta}}_{1:T} = \arg\min_{\boldsymbol{\theta} \in \mathbb{R}^p} -\sum_{t=1}^{T} \alpha_t \log q_t(\boldsymbol{\theta}) - \gamma \log p_0(\boldsymbol{\theta}) = \sum_{t=1}^{T} \alpha_t \boldsymbol{\theta}_t, \tag{43.37}$$

where the regularizer is weighted by a term $\gamma = 1 - \sum_t \alpha_t$ that vanishes due to the α_t summing to 1. In order to arrive at this result, we can use direct calculation, by noting that for the choice of likelihood and prior we can find that

$$-\log q_t(\boldsymbol{\theta}) = \frac{1}{2}\|\boldsymbol{\theta} - \boldsymbol{\theta}_t\|_2^2 + c,$$

where c is a constant under arg max, as we have chosen unit variance. The regularizer corresponds to weight-decay. Setting the derivative of Equation 43.37 to 0 leads to

$$0 = \sum_{t=1}^{T} \alpha_t(\boldsymbol{\theta} - \boldsymbol{\theta}_t) + \gamma\boldsymbol{\theta} = \sum_{t=1}^{T} \alpha_t(\boldsymbol{\theta} - \boldsymbol{\theta}_t) = \sum_{t=1}^{T} \alpha_t\boldsymbol{\theta} - \sum_{t=1}^{T} \alpha_t\boldsymbol{\theta}_t \tag{43.38}$$

$$\Leftrightarrow \boldsymbol{\theta} = \sum_{t=1}^{T} \alpha_t\boldsymbol{\theta}_t, \tag{43.39}$$

where we have used that $\sum_t \alpha_t = 1$ and $\gamma = 1 - \sum_t \alpha_t = 0$.

43.5.3 Uncertainty-Based Gradient Matching & Data Removal

We are often interested in merging fine-tuned LLMs $\boldsymbol{\theta}_t \in \mathbb{R}^p$, where merging generally works well if all $\boldsymbol{\theta}_t$ are initialized using the same LLM $\boldsymbol{\theta}_0 \in \mathbb{R}^p$. Then, it can make sense to account for the initialization explicitly when merging by choosing a regularizer $p_0 = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\theta}_0, \boldsymbol{H}_0^{-1})$, where $\boldsymbol{H}_0$ is the Hessian which captures uncertainty over the parameters $\boldsymbol{\theta}_0$. Likewise, it can be beneficial to account for the uncertainty over the fine-tunings by using posteriors $q_t(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\theta}_t, (\boldsymbol{H}_0 + \boldsymbol{H}_t)^{-1})$. This leads to the following merging equation [353]:

$$\widehat{\boldsymbol{\theta}}_{1:T} = \boldsymbol{\theta}_0 + \sum_{t=1}^{T} \alpha_t(\boldsymbol{H}_0 + \sum_{t=1}^{T} \alpha_t\boldsymbol{H}_t)^{-1} \left(\boldsymbol{H}_0 + \boldsymbol{H}_t\right)\left(\boldsymbol{\theta}_t - \boldsymbol{\theta}_0\right), \tag{43.40}$$

where using $\boldsymbol{H}_0 = I, \boldsymbol{H}_t = 0$ corresponds to Task Arithmetic [811] and using the Fisher to approximate the Hessian with $\sum_t \alpha_t = 1$ corresponds to Fisher-Weighted Averaging [1144] The Hessians here weight each model according to their uncertainty and models with higher uncertainty, which in turn results in larger values of $\boldsymbol{\sigma}$, receive less weight. This is also intuitive when remembering the IVON posterior, where larger $\boldsymbol{h}$ means smaller variance and stronger posterior concentration. The subtraction by $\boldsymbol{\theta}_0$ is similar to Bayes: if the q_t already contain p_0, we would divide out the common p_0 to not overcount it, which is referred to as Bayesian Arithmetic in Khan [882]:

$$q_\alpha(\boldsymbol{\theta}, \mathcal{D}_{1:T}) = p_0(\boldsymbol{\theta}) \prod_{t=1}^{T} \left(\frac{q_t(\boldsymbol{\theta})}{p_0(\boldsymbol{\theta})}\right)^{\alpha_t}. \tag{43.41}$$

While model merging often aims to combine knowledge from different models, it is also possible to remove them by using the same strategy we have used to remove the over-counted prior: we just "divide it out". In parameter-space this corresponds to a subtraction. For example, we might be interested in removing a proportion $\mathcal{D}_t$ from an LLM, denoted by $\boldsymbol{\theta}_0$, trained on a large dataset $\mathcal{D}$ that contains $\mathcal{D}_t$. Then, we can train on $\mathcal{D}_t$ (starting from $\boldsymbol{\theta}_0$) to obtain $\boldsymbol{\theta}_t$ and perform the following merging [353]:

$$\boldsymbol{\theta}_{\setminus t} = \boldsymbol{\theta}_{\text{LLM}} - \alpha_t\boldsymbol{H}_{\setminus t}^{-1}(\boldsymbol{H}_0 + \boldsymbol{H}_t)(\boldsymbol{\theta}_t - \boldsymbol{\theta}_0), \tag{43.42}$$

where $\boldsymbol{H}_{\setminus t}$ is calculated using $\mathcal{D} \setminus \mathcal{D}_t$ and $\boldsymbol{\theta}_0$. The method is also closely related to influence functions and, for linear regression, reduces to the influence function of Cook [327] when using $\alpha_t = 1$. We refer the interested reader to Daheim et al. [353] for further discussion and experimental results, for example, on removing toxic data from LLMs.

43.6 Concluding Remarks

In this chapter, we have reviewed various methods that can be used to apply Bayesian techniques to LLMs. First, we have shown that LLMs can be made Bayesian "post-hoc" via fine-tuning with LoRA, ensembling, and Laplace's method. The advantage of these techniques is that existing LLMs can be reused. There is no requirement for the LLM to be trained with a Bayesian method but empirical improvements are still observed. We have then also shown that LLMs can directly be trained with Bayesian methods to learn a posterior. The information from the posterior can then be used to improve the predictions of LLMs by sampling outputs from many models sampled from the posterior and to adapt LLMs by using the uncertainty of the posterior to weight individual models.

Chapter 44

Confidence-based selective classification

Selective classification offers a principled framework to enhance the reliability of deep learning models, particularly in safety-critical settings where erroneous predictions can have serious consequences. By enabling models to abstain on uncertain inputs, selective classification explicitly controls the trade-off between *coverage* (the fraction of inputs retained rather than rejected) and *selective accuracy* (accuracy on the retained predictions). In this chapter, we systematically evaluate a broad range of uncertainty quantification (UQ) methods for confidence-based selective classification, using image recognition on CIFAR-100 as a concrete and reproducible case study. Our exploration covers novel confidence heuristics derived from softmax probabilities and prediction margins, probabilistic modeling via Bayesian Beta posterior densities, and nonparametric empirical confidence estimation, augmented with class-specific weighting schemes such as the F_1 score and geometric mean. Building on these approaches, we propose `LogitEnsemble-UQ`, a logistic-regression meta-model that integrates multiple confidence signals into a single calibrated rejection score. Evaluated at target selective accuracy levels of 80%, 85%, and 90%, this ensemble consistently achieves improved accuracy–rejection (equivalently, accuracy–coverage) trade-offs, illustrating the benefit of combining complementary uncertainty measures. Figure 44.1 provides a visual overview of the chapter: (i) how confidence estimators vary across ground-truth classes, (ii) how confidence thresholding separates correct from incorrect predictions, and (iii) the resulting selective accuracy–rejection trade-offs for the proposed ensembles.

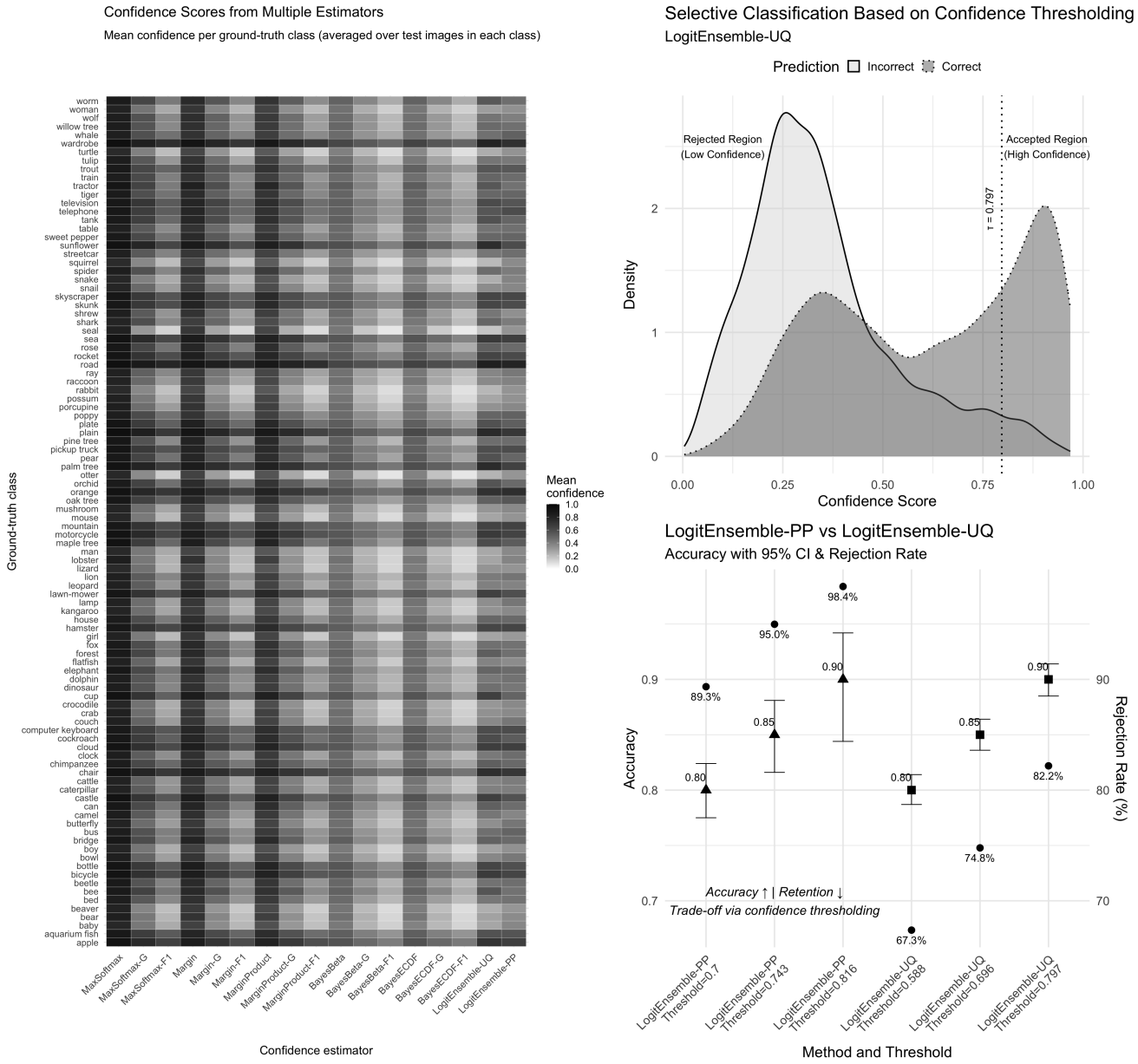


Figure 44.1: **Graphical abstract: confidence-based selective classification. Left:** CIFAR-100 per-class mean confidence heatmap: rows are ground-truth classes and columns are confidence estimators; each tile reports the mean confidence over all test images from that class. **Top-right:** Confidence-score densities for *LogitEnsemble-UQ*, split by correct vs. incorrect predictions. A threshold τ partitions predictions into a rejected low-confidence region and an accepted high-confidence region. **Bottom-right:** Selective classification trade-off for *LogitEnsemble-PP* and *LogitEnsemble-UQ*: selective accuracy (accuracy on accepted predictions; 95% CIs) and rejection rate at several thresholds. *LogitEnsemble-UQ* achieves higher selective accuracy at comparable (or lower) rejection, indicating that engineered uncertainty features improve confidence ranking beyond probabilities alone.

44.1 Background

Uncertainty is an inherent feature of machine learning systems, particularly in high-stakes applications where data may be noisy, incomplete, or shifted relative to the training distribution. Unlike deterministic pipelines, learned predictors must operate under ambiguity: uncertainty can arise from limited data and model misspecification (*epistemic* uncertainty) as well as irreducible noise in the observations (*aleatoric* uncertainty). Quantifying and communicating these uncertainties is central to trustworthy AI because it enables risk-aware decision-making and supports appropriate human oversight when model confidence is unreliable [566, 879].

A principled way to *act* on predictive uncertainty is to equip a classifier with a *reject option*. This idea traces back to Chow's early work on optimal recognition with abstention, which formalized how rejecting uncertain samples can reduce error on the retained set [301, 302]. In contemporary terminology, this setting is typically referred to as *selective classification* (also called *classification with a reject option* or *classification with abstention*). The model is allowed to abstain on low-confidence inputs, thereby trading *coverage* (the proportion of inputs for which a prediction is returned) for improved reliability on the subset of *retained* predictions. Subsequent work developed theoretical and algorithmic foundations for this error–reject trade-off, including formulations based on surrogate risks and guarantees [102, 486, 1828]. Later, more general approaches considered abstention mechanisms implemented by a learned *rejector function* rather than simple thresholding of a scalar confidence score [332]. A recent survey by Hendrickx et al. synthesizes these strands, including integrated (end-to-end) methods that jointly learn predictors and rejectors [732].

A closely related—but conceptually distinct—paradigm is *learning to defer* [1119, 1201, 1224]. Whereas selective classification typically treats abstention as incurring a fixed cost (possibly sample- or class-dependent) independent of what happens after rejection, learning to defer explicitly models a downstream decision-maker (e.g., a human expert or auxiliary system) that will handle deferred inputs. In this setting, deferral is beneficial precisely when the expert is expected to outperform the model on the deferred subset. Leitão et al. discuss practical challenges and limitations of deploying deferral systems, including modeling expert behavior, distribution shift, and operational constraints in human–AI collaboration [37].

These research directions can be organized along three largely orthogonal axes:

1. **Selective classification vs. learning to defer.** Selective classification optimizes an error–reject trade-off under an assumed abstention cost [301, 486], while learning to defer optimizes the performance of a combined system (model plus expert) by exploiting complementary strengths [1119, 1201, 1224].

2. **Rejection/deferral criterion.** Many practical systems implement abstention by thresholding a scalar confidence score [301, 486]. More general approaches learn a rejector/deferral policy that can depend on richer features of the input and the model output [332, 732].

3. **Separate vs. joint (end-to-end) learning.** The base predictor and the rejection/deferral mechanism can be learned separately (two-stage systems), for example by training a classifier and then tuning a post-hoc threshold [486], or learned jointly/end-to-end (one-stage systems), where prediction and abstention are optimized together [102, 332, 732]. In learning-to-defer terminology, these correspond to two-stage and one-stage approaches [1201]. Early selective-model formulations that explicitly incorporated a learnable selection mechanism were already explored by Geifman [592], and integrated rejector approaches have since become a major theme in the literature [732].

In application-driven work, abstention and deferral have been used as practical tools to improve the reliability of machine learning outputs and to reduce human review burden by routing ambiguous cases to more reliable decision pathways. For instance, recent studies report that uncertainty-aware filtering or abstention can improve downstream operating characteristics (e.g., precision/recall-type metrics) in clinical prediction settings [274, 1086]. In sleep staging, abstention based on uncertainty measures has been used to increase agreement with reference labels, including improvements as measured by Cohen's Kappa [867]. In medical imaging, uncertainty-driven selection has similarly been used to concentrate predictions on higher-confidence cases, improving performance metrics such as F_1 on the retained subset [614].

Large-scale deployments further motivate selective prediction as an operational mechanism. For example, selective classification for information extraction from pathology reports has been evaluated at scale with the goal of meeting stringent reliability targets while reducing manual review [1312]. Related efforts in pediatric oncology have explored uncertainty-aware automation for coding/classification from pathology text, reporting strong performance while abstaining on ambiguous cases [1821]. Across such settings, performance hinges on how well the chosen confidence score (or learned rejector/deferral policy) ranks correct predictions above incorrect ones, since threshold selection directly mediates the trade-off between coverage and reliability.

Building on these foundations, this chapter focuses on *selective classification* and systematically evaluates *post-hoc*, confidence-based uncertainty quantification methods for deep neural networks. Using CIFAR-100 as a reproducible testbed, we benchmark confidence scoring approaches—ranging from classic probability-based heuristics rooted in the reject-option literature [301, 302] to Bayesian and nonparametric calibration schemes—by their ability to satisfy user-specified target selective accuracies while maximizing retained coverage. In addition, we examine class-weighted confidence variants and propose a logistic-regression meta-model that combines multiple uncertainty signals into a single calibrated rejection score. The goal is to provide a practical, comparative perspective on

confidence scoring for selective prediction and to inform reliable deployment of uncertainty-aware deep learning systems.

44.2 Mathematical Framework

44.2.1 Problem Setup

We begin by formalizing the supervised classification setting in which selective classification and uncertainty quantification (UQ) will be applied.
Let

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$$

denote a dataset of N labeled samples, where $x_i \in \mathcal{X}$ is an input from the domain $\mathcal{X}$ (e.g., images), and $y_i \in \mathcal{Y} = \{1, \ldots, C\}$ is a categorical label from a set of C mutually exclusive classes. In our CIFAR-100 case study, $C = 100$.
We assume access to a trained base classifier

$$f_\theta : \mathcal{X} \to \Delta^{C-1},$$

parameterized by $\theta \in \Theta$, which maps each input to a probability vector lying on the $(C-1)$-dimensional probability simplex

$$\Delta^{C-1} = \left\{ \mathbf{p} \in \mathbb{R}^C \;\middle|\; p_c \geq 0 \;\; \forall c, \quad \sum_{c=1}^{C} p_c = 1 \right\}.$$

The simplex constraint guarantees that outputs are interpretable as a valid probability distribution.
For each input x_i, the classifier outputs

$$\mathbf{p}_i = f_\theta(x_i) = \left[p_{i,1}, p_{i,2}, \ldots, p_{i,C}\right]^\top,$$

where $p_{i,c}$ denotes the model's estimated probability that x_i belongs to class c. The predicted class label is obtained via the maximum a posteriori (MAP) decision rule:

$$\hat{y}_i = \arg\max_{c \in \mathcal{Y}} p_{i,c}.$$

Intuition: The classifier produces a probability distribution over classes, but these numbers are not necessarily well-calibrated; a 90% prediction may not actually be correct 90% of the time. Selective classification seeks to leverage this distribution to decide when to trust the model.

44.2.2 Confidence Scoring for Selective Classification

In conventional classification, every prediction is returned without qualification. Selective classification changes this: for each prediction, we compute a *confidence score* and compare it to a threshold to decide whether to retain the prediction or not.
Formally, a scoring function is a mapping

$$s : \mathcal{X} \to \mathbb{R}, \quad s_i = s(x_i),$$

where higher s_i indicates greater confidence in $\hat{y}_i$ being correct i.e., matches the true label y_i. The ranking induced by s determines the trade-off between coverage (fraction of samples retained) and risk (error rate on retained samples).
We now outline several confidence scoring strategies, each grounded in either direct probability analysis or a probabilistic generative model.

MaxSoftmax

Perhaps the most widely used confidence score, the *MaxSoftmax* is simply the highest predicted class probability:

$$s_i = \max_c p_{i,c}.$$

This value corresponds directly to the model's own stated belief that its most likely class is correct. If the model produces a well-calibrated probability distribution, s_i can be interpreted as an estimate of the probability of correctness.

Intuition: A high MaxSoftmax value indicates that the model strongly favors one class over all others. For example, $s_i = 0.99$ means the model assigns 99% probability to a single class and only 1% to all others combined. This is a simple, fast-to-compute measure that often works surprisingly well as a first-pass uncertainty estimate. However, it can be overconfident in overparameterized neural networks, especially when probabilities are not perfectly calibrated.

Margin

The *Margin* score focuses on the gap between the top two predicted probabilities:

$$s_i = p_{i,(1)} - p_{i,(2)},$$

where $p_{i,(1)} \geq p_{i,(2)} \geq \cdots \geq p_{i,(C)}$ are the predicted probabilities sorted from largest to smallest.

Intuition: Rather than only considering the top class, the margin measures how decisively it beats the runner-up. A large margin implies a clear "winner" among classes, while a small margin means the decision is contested between at least two plausible labels. This is particularly useful in cases where MaxSoftmax is high but two classes are nearly tied — a sign that the model may still be uncertain despite a confident top probability.

MarginProduct

The *MarginProduct* combines the strengths of both the MaxSoftmax and Margin scores:

$$s_i = (p_{i,(1)} - p_{i,(2)}) \cdot p_{i,(1)}.$$

Here, the margin captures the decisiveness of the prediction, while $p_{i,(1)}$ captures its absolute confidence level.

Intuition: This hybrid score rewards predictions that are both *highly confident* (large $p_{i,(1)}$) and *decisive* (large margin). If either the top probability $p_{i,(1)}$ is low (low confidence) or the margin $p_{i,(1)} - p_{i,(2)}$ is small (runner-up close), the score decreases accordingly. This makes MarginProduct more conservative than either MaxSoftmax or Margin alone, often leading to better separation between correct and incorrect predictions in practice.

BayesBeta

We take a generative perspective, modeling the top predicted probability $p_{i,(1)}$ under two separate Beta distributions:

$$p_{i,(1)} \sim \begin{cases} \text{Beta}(\alpha_{\text{corr}}, \beta_{\text{corr}}), & \hat{y}_i = y_i, \\ \text{Beta}(\alpha_{\text{incorr}}, \beta_{\text{incorr}}), & \hat{y}_i \neq y_i. \end{cases}$$

Let f_{corr} and f_{incorr} denote the corresponding probability density functions, and let t_{prior} denote the prior probability that a prediction is correct. By Bayes' theorem, the posterior probability of correctness given $p_{i,(1)}$ is:

$$s_i = \frac{t_{\text{prior}} \cdot f_{\text{corr}}(p_{i,(1)})}{t_{\text{prior}} \cdot f_{\text{corr}}(p_{i,(1)}) + (1 - t_{\text{prior}}) \cdot f_{\text{incorr}}(p_{i,(1)})}.$$

Intuition: The BayesBeta method explicitly asks: *"Given that my model assigned a highest class probability of $p_{i,(1)}$, how likely is it that this prediction is actually correct?"* Instead of relying on $p_{i,(1)}$ in isolation as a raw confidence proxy, we model how this quantity behaves *differently* for correct versus incorrect predictions. In many models, correct predictions tend to have $p_{i,(1)}$ values concentrated near 1, while incorrect predictions show a broader spread toward lower values. The Beta distribution is well-suited here because: (1) It naturally models probabilities in the $[0, 1]$ interval. (2) Its shape is highly flexible — it can represent sharp peaks, flat distributions, or skewed shapes. By fitting one Beta distribution to the correct cases and another to the incorrect cases, we effectively learn two distinct "signatures" of top-probability behavior. Bayes' theorem then lets us combine these likelihoods with a prior belief about correctness (t_{prior}) to produce a calibrated, probabilistic estimate of correctness. The result is a confidence score that is *interpretable*, *probabilistically grounded*, and capable of capturing subtle distributional differences that simple thresholds on $p_{i,(1)}$ might miss.

BayesECDF

The BayesECDF method follows the same Bayesian reasoning as BayesBeta, but removes the parametric assumption on the shape of the distributions. Instead of fitting Beta distributions, we directly estimate the empirical cumulative distribution functions (ECDFs) of the top predicted probability $p_{i,(1)}$ for correct and incorrect predictions:

$$F_{\text{corr}}(p) = \frac{1}{N_{\text{corr}}} \sum_{i=1}^{N} \mathbb{1}[\hat{y}_i = y_i \ \wedge \ p_{i,(1)} \leq p],$$

$$F_{\text{incorr}}(p) = \frac{1}{N_{\text{incorr}}} \sum_{i=1}^{N} \mathbb{1}[\hat{y}_i \neq y_i \ \wedge \ p_{i,(1)} \leq p],$$

where N_{corr} and N_{incorr} are the total counts of correct and incorrect predictions, respectively.
From these ECDFs, we approximate the probability density functions $f_{\text{corr}}(p)$ and $f_{\text{incorr}}(p)$ by taking finite differences:

$$f_{\text{corr}}(p) \approx \frac{d}{dp} F_{\text{corr}}(p), \quad f_{\text{incorr}}(p) \approx \frac{d}{dp} F_{\text{incorr}}(p).$$

These density estimates are then plugged into Bayes' theorem exactly as in BayesBeta to yield the posterior probability of correctness:

$$s_i = \frac{t_{\text{prior}} \cdot f_{\text{corr}}(p_{i,(1)})}{t_{\text{prior}} \cdot f_{\text{corr}}(p_{i,(1)}) + (1 - t_{\text{prior}}) \cdot f_{\text{incorr}}(p_{i,(1)})}.$$

Intuition: While BayesBeta assumes that $p_{i,(1)}$ follows a Beta distribution for both correct and incorrect predictions, BayesECDF makes no such assumption. This is valuable when the empirical shapes of these distributions are irregular — for example, multimodal patterns or sharp spikes that the Beta family cannot easily capture. The ECDF-based approach lets the data "speak for itself." By counting how often the top predicted probability falls below a given value p, separately for correct and incorrect cases, we obtain a complete nonparametric description of its distribution. From there, the same Bayesian logic applies: we compare the likelihood of observing $p_{i,(1)}$ under each case (correct vs. incorrect) and combine it with the prior probability of correctness. This flexibility comes with a trade-off: while BayesECDF can more faithfully represent unusual empirical patterns, it may require more data to produce smooth, stable estimates of the densities. Nevertheless, in large-scale classification problems, its ability to model complex score distributions without constraining them to a fixed parametric form can lead to more accurate and better-calibrated confidence estimates.

Class-Weighted Variants

The confidence scores described so far treat all classes as equally difficult. However, in many real-world classification tasks — especially those with class imbalance or heterogeneous complexity — some classes are consistently easier for the model, while others are systematically more challenging. To account for these differences, we introduce *class-weighted variants* of any base confidence score.
Formally, given a base confidence score s_i and a per-class weight $w_{\hat{y}_i}$ corresponding to the predicted class $\hat{y}_i$, we define the reweighted score:

$$s_i^{(w)} = s_i \cdot w_{\hat{y}_i}.$$

This weighting scheme increases the effective confidence for classes that the model typically predicts accurately, and decreases it for classes where the model is less reliable. In practice, class-weighted variants are ranking scores and thresholds become method-specific.
Two weighting metrics are considered:

- **Geometric Mean (G-mean):** This metric balances the true negative rate and true positive rate for class k, providing a balanced measure of performance across both correctly rejecting negatives and correctly identifying positives:

$$G_k = \sqrt{\frac{\text{TN}_k}{\text{TN}_k + \text{FP}_k} \cdot \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}}.$$

- **F1-score:** The harmonic mean of precision and recall for class k, capturing the trade-off between avoiding false positives and avoiding false negatives:

$$F1_k = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k},$$

where

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \quad \text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}.$$

Here, k indexes the class, and TP_k, FP_k, TN_k, FN_k are counts from the per-class confusion matrix.

Intuition: Consider a classifier trained on a dataset where some classes are rare or visually ambiguous. If the model predicts such a difficult class with high probability, a raw confidence score might still overestimate the likelihood of correctness. By scaling scores according to the model's estimated per-class performance on a held-out validation (calibration) set, we make the confidence estimates more *class-aware*.
For example, if class A has a high F1-score, we trust the model's predictions for A more and boost its scores; if class B has low G-mean, we discount its scores accordingly. This adjustment is especially valuable in imbalanced datasets, as it mitigates the risk of overconfidence on poorly represented classes while allowing the model to be bolder where it has consistently strong performance.

LogitEnsemble-UQ

Rather than relying on a single uncertainty score, we can combine multiple complementary scores into a unified confidence estimate. To achieve this, we construct a feature vector $\mathbf{z}_i$ for each sample x_i containing all base confidence scores and their class-weighted variants:

$$\mathbf{z}_i = \left[\text{MaxSoftmax}_i, \ldots, \text{BayesBeta}_i, \text{BayesECDF}_i, \text{MaxSoftmax}_i^{(G)}, \ldots, \text{BayesECDF}_i^{(F1)}\right]^\top.$$

A logistic regression model is then trained on these vectors to produce a single scalar confidence score:

$$s_i = \sigma(\mathbf{w}^\top \mathbf{z}_i + b),$$

where $\sigma(t) = \frac{1}{1+e^{-t}}$ is the logistic sigmoid function, and $(\mathbf{w}, b)$ are learned parameters.
Intuition: Each base score (e.g., MaxSoftmax, Margin, BayesBeta, etc.) captures a different aspect of model uncertainty. For example, MaxSoftmax focuses purely on the top predicted probability, while Margin considers the gap between top predictions, and Bayes-based methods incorporate distributional calibration. Individually, each score is imperfect—some are better for certain classes or accuracy ranges, others for different operating points. By feeding all these scores into a logistic regression meta-model, the system learns an optimal way to *weight* and *combine* them. Logistic regression is well-suited here because: (1) It outputs a value between 0 and 1, naturally interpretable as a probability of correctness. (2) It can learn linear combinations that emphasize informative features and downweight noisy ones. (3) It is simple and transparent, making the learned coefficients interpretable—allowing us to see which base scores contribute most. The result is an *ensemble confidence score* that generally outperforms any individual score in ranking predictions from most to least reliable. In practice, this approach often yields lower rejection rates for the same target accuracy, as the meta-model learns to exploit synergies between different uncertainty measures.

LogitEnsemble-PP

As a simpler baseline, we can bypass engineered uncertainty features altogether and feed the raw class probability vector $\mathbf{p}_i$ directly into a logistic regression model:

$$s_i = \sigma(\mathbf{w}^\top \mathbf{p}_i + b),$$

where $\sigma(\cdot)$ is the logistic sigmoid function and $(\mathbf{w}, b)$ are learned parameters. L_2-regularized logistic regression can be applied to reduce overfitting to spurious patterns in the class probability vector $\mathbf{p}_i$, such as calibration biases and dataset-specific probability correlations.

Intuition: The vector $\mathbf{p}_i$ contains the model's full posterior distribution over classes. While this raw representation implicitly encodes uncertainty (e.g., a sharp peak for confident predictions, a flatter distribution for uncertain ones), it does not explicitly combine or transform this information in the way the engineered features do. The logistic regression here learns to map these probability patterns directly to an estimated probability of correctness. This baseline is valuable for two reasons: (1) It isolates the benefit of using *only* the base model's probability outputs, without additional derived features. (2) It provides a clean benchmark to evaluate whether the engineered uncertainty features (and their combinations, as in LogitEnsemble-UQ) meaningfully improve selective classification performance.

Because $\mathbf{p}_i$ is high-dimensional (one feature per class), this method may capture nuanced patterns in the output distribution that simple scalar scores cannot. However, without targeted uncertainty descriptors (e.g., MaxSoftmax, Margin, BayesBeta, etc.), it may fail to fully leverage the structure of model uncertainty—making it a possibly informative but less robust alternative to LogitEnsemble-UQ.

44.2.3 Selective Classification by Thresholding

Once a confidence score s_i has been assigned to each prediction, selective classification proceeds by retaining only those predictions deemed sufficiently reliable. This is achieved via a *thresholding* operation: we specify a confidence threshold $\tau \in \mathbb{R}$ and reject predictions whose scores fall below it.
Formally, let $s(x) \in \mathbb{R}$ denote a confidence score assigned by the model to an input feature vector x. Selective classification is obtained by *thresholding* this score: given a threshold τ, we retain predictions whose confidence is at least τ, and reject the rest.
Formally, the binary *selection function* is

$$g_\tau(x) = \begin{cases} 1, & s(x) \geq \tau, \\ 0, & \text{otherwise}, \end{cases}$$

where $g_\tau(x) = 1$ indicates that the prediction for x is *retained* and $g_\tau(x) = 0$ indicates that it is *rejected*.

Intuition: By adjusting τ, we control the *coverage–accuracy trade-off*. A low threshold retains most predictions (high coverage) but risks keeping low-confidence cases, potentially lowering accuracy. Conversely, a high threshold retains only the most confident predictions (low coverage), often boosting accuracy at the cost of discarding more samples.

44.2.4 Coverage and Rejection Rate

Coverage quantifies the proportion of predictions that are *retained* under a given confidence threshold τ:

$$\text{coverage}(\tau) = \frac{1}{N} \sum_{i=1}^{N} g_\tau(x_i),$$

where $g_\tau(x_i) \in \{0, 1\}$ is the selection function.
The *rejection rate* is simply the fraction of predictions that are discarded:

$$\text{rejection rate}(\tau) = 1 - \text{coverage}(\tau).$$

Intuition: Coverage and rejection rate are two sides of the same coin. A high coverage means that most predictions are kept, which maximizes throughput but may include low-confidence cases. A high rejection rate means that only the most confident predictions survive — improving reliability but potentially reducing the usable output size.

44.2.5 Selective Accuracy and Risk

Selective accuracy measures the fraction of correct predictions *among retained samples*:

$$\text{selective accuracy}(\tau) = \frac{\sum_{i=1}^{N} \mathbb{1}[\hat{y}_i = y_i] \cdot g_\tau(x_i)}{\sum_{i=1}^{N} g_\tau(x_i)}.$$

The corresponding *selective risk* is

$$\text{selective risk}(\tau) = 1 - \text{selective accuracy}(\tau).$$

In this work, selective accuracy is reported with 95% binomial proportion confidence intervals using the Clopper–Pearson method [314].

Intuition: Selective accuracy answers the question: "Given that I only keep high-confidence predictions, how often am I right?" Selective risk flips the perspective: "Among the predictions I keep, how often am I wrong?" These two metrics capture the reliability of the retained set, independent of how many samples are discarded.

44.2.6 Target Accuracy Thresholding

For a desired accuracy target $\alpha \in (0, 1)$, we choose the smallest threshold τ_α satisfying:

$$\tau_\alpha = \min\{\tau : \text{selective accuracy}(\tau) \geq \alpha\}.$$

This approach enforces a pre-specified accuracy guarantee, even if it requires discarding a large portion of the data. In practice, we sweep thresholds over unique score values (or quantiles) on the validation set and pick the smallest τ meeting the constraint.

Intuition: Target accuracy thresholding flips the usual problem: instead of asking "How accurate am I at a given coverage?", it asks "How much data can I keep while meeting my accuracy goal?" The resulting τ_α adapts the system's selectiveness to the desired reliability standard.

44.2.7 Risk–Coverage Curves

By varying τ from low to high, we trace a *risk–coverage curve*, plotting coverage on the x-axis and selective risk on the y-axis:

$$\text{coverage} = 1 - \text{rejection rate}.$$

An ideal method shows a steep drop in risk as coverage decreases, indicating that it quickly eliminates uncertain predictions while retaining as many correct ones as possible.

Intuition: Risk–coverage curves provide a global view of how well a confidence score separates good predictions from bad ones. A shallow curve means the method struggles to find high-confidence subsets without also discarding many correct predictions. A steep curve means that even a small reduction in coverage yields a large drop in risk — a hallmark of an effective uncertainty estimator.

44.3 Case Example on CIFAR-100

To concretely illustrate the selective classification framework introduced in Section 44.2.2, we evaluate a modern image classifier on the CIFAR-100 benchmark. The CIFAR-100 dataset consists of $N = 50{,}000$ labeled images, each belonging to one of $C = 100$ classes. We adopt a split of 40,000 images for training the ResNet and 10,000 images for post-hoc calibration and evaluation. All selective-classification components that require labeled data and the operating thresholds τ_α are estimated on this 10,000-image split, and results are reported on the same split. Consequently, the reported numbers should be interpreted as calibration-set performance rather than performance on an independent test set.

The base model f_θ is a ResNet [721, 1789], trained to output a vector of class probabilities for each input x_i:

$$\mathbf{p}_i = f_\theta(x_i) = [p_{i,1}, p_{i,2}, \ldots, p_{i,C}]^\top,$$

where $p_{i,c}$ represents the model's estimated probability that x_i belongs to class c, with $\sum_{c=1}^{C} p_{i,c} = 1$.

Predicted labels are assigned by maximum a posteriori (MAP) decision:

$$\hat{y}_i = \arg\max_{c \in \mathcal{Y}} p_{i,c}.$$

A natural baseline confidence score for selective classification is simply the highest predicted probability:

$$s_i^{\text{base}} = \max_c p_{i,c} = p_{i,(1)},$$

where $p_{i,(1)}$ denotes the largest component of $\mathbf{p}_i$.

Following the general framework, we compute a range of confidence scores $s_i = s(x_i)$ — including MaxSoftmax, Margin, MarginProduct, BayesBeta, BayesECDF, and their class-weighted variants — to quantify uncertainty and rank predictions by reliability. For each method, we identify the smallest threshold τ_α that achieves a target *selective accuracy* $\alpha \in \{0.80, 0.85, 0.90\}$:

$$\tau_\alpha = \min\left\{\tau \in \mathbb{R} : \text{selective accuracy}(\tau) \geq \alpha\right\}.$$

Here, *selective accuracy* is the accuracy computed over only the retained predictions, i.e., those with $s_i \geq \tau_\alpha$.

Quantitative Performance at Target Accuracies

Tables 44.1–44.3 report the results for all methods at each target accuracy. Each row shows: (1) the achieved selective accuracy with a 95% confidence interval, (2) the rejection rate (percentage of samples discarded), (3) the total set size used to compute the metrics (*support*), and (4) the learned or selected threshold τ_α.

The baseline model (no rejection) achieves $\approx 47.6\%$ accuracy on CIFAR-100, underscoring the dataset's difficulty. To meet higher accuracy targets, a substantial proportion of low-confidence predictions must be rejected. At the most stringent target (90%), most scoring methods reject roughly 82%–83% of samples, retaining only the most certain predictions.

A key observation is that while many methods meet the same target accuracy by construction (via threshold selection), their *rejection rates* vary considerably, reflecting differences in how well each score ranks correct predictions above incorrect ones. Across the *unweighted* scalar heuristics, MaxSoftmax, Margin, and MarginProduct exhibit very similar behavior on CIFAR-100, suggesting that for this model the top probability $p_{(1)}$ already captures most of the separability information available in the softmax vector, and that incorporating the runner-up probability via the margin provides only marginal additional benefit at these operating points. The Bayesian calibration methods (BayesBeta and BayesECDF) also perform comparably to these simple heuristics, indicating that modeling $p_{(1)}$ via a parametric Beta family or a nonparametric ECDF does not substantially change the induced ranking on this test bed, even though these approaches are appealing because they output a more interpretable, "probability-of-correctness" style confidence score.

Across all three target accuracies, the class-weighted variants (G-mean and F_1) are generally *less effective* than their unweighted counterparts, typically requiring higher rejection to meet the same accuracy constraint. One plausible explanation is that CIFAR-100 is approximately class-balanced and relatively small, so class-level weights derived from per-class performance do not primarily correct imbalance; instead, they act as a multiplicative rescaling of the confidence score, $s_i^{(w)} = s_i \cdot w_{\hat{y}_i}$, which can distort the global ranking that selective classification depends on. In particular, downweighting predictions from empirically "hard" classes can cause many otherwise-correct predictions to fall below the threshold τ_α, increasing rejection without a compensating improvement in separating correct from incorrect cases. In addition, with $C = 100$ classes and limited samples per class, per-class metrics such as F_1 and G-mean can be noisy, and this estimation noise can propagate directly into the weighted confidence score.

The logistic regression ensemble with engineered uncertainty features (`LogitEnsemble-UQ`) consistently yields the *lowest* rejection rate at all targets, indicating a more favorable accuracy–coverage trade-off and a stronger ability to combine complementary cues (absolute confidence, decisiveness, and calibrated posteriors) into a single ranking signal. In contrast, the probability-based logistic regression baseline (`LogitEnsemble-PP`) is far less efficient, often rejecting $\geq 95\%$ of the data to satisfy the same target accuracy. A plausible reason is that using the full C-dimensional probability vector as features can encourage the meta-model to fit idiosyncratic, class-specific probability patterns that do not translate into a clean global "correct vs. incorrect" separation, whereas the engineered features in `LogitEnsemble-UQ` act as lower-dimensional, uncertainty-focused summaries that are more directly aligned with the selective classification objective.

We emphasize that CIFAR-100 is used here primarily as a controlled and reproducible *test bed* for comparing uncertainty scores under a common protocol; it is not meant to be the setting where all variants necessarily shine. In larger and more heterogeneous real-world datasets—especially those with pronounced class imbalance, class-dependent difficulty, label noise, or differing misclassification costs—class-aware weighting and calibration can become more valuable. In such regimes, class-weighted confidence may better reflect the fact that "a score of 0.9" can correspond to different empirical reliability depending on the predicted class, potentially enabling higher coverage at a fixed reliability target. Therefore, the weaker performance of the weighted variants on CIFAR-100 should be interpreted as evidence that simple global class-weighting is not universally beneficial, rather than as a general negative result about class-aware selective classification.

Trade-off Between Coverage and Accuracy

Figure 44.2 presents the achieved accuracy (with 95% confidence intervals) as a function of the rejection rate for all uncertainty scoring methods, stratified by the three target accuracy levels (80%, 85%, and 90%). Each panel corresponds to one target accuracy, and within each panel, points represent different methods, grouped by scoring family (linetype) and type (shape). Horizontal position reflects the fraction of predictions rejected, while vertical position reflects the actual accuracy on the retained set. Across all targets, higher rejection rates generally correspond to higher accuracies — confirming the expected trade-off where discarding low-confidence predictions improves reliability. However, the *efficiency* of this trade-off differs across methods. Methods near the

left edge (low rejection) achieve the most favorable balance, meeting the target accuracy with minimal discarded data. Here, the `LogitEnsemble-UQ` stands out: in all three panels, it reaches the target with substantially fewer rejections compared to both simple probability-based scores (e.g., MaxSoftmax) and more complex Bayesian or class-weighted variants. In contrast, the `LogitEnsemble-PP` baseline often lies far to the right, indicating it rejects the majority of predictions to meet the same target — a sign of weaker discrimination between correct and incorrect cases.

Risk–Coverage Perspective

Figure 44.3 compares the `LogitEnsemble-UQ` and `LogitEnsemble-PP` methods through their *risk–coverage* curves. Here, *coverage* is the fraction of predictions retained after thresholding by confidence, while *selective risk* is the error rate computed only on those retained predictions. The primary y-axis (left) displays selective risk, and the secondary y-axis (right) shows the equivalent selective accuracy for reference. An ideal curve hugs the lower right corner — maintaining low risk (high accuracy) while preserving high coverage. Across all thresholds, `LogitEnsemble-UQ` exhibits a steeper decline in risk as coverage decreases, meaning it is more effective at discarding low-confidence errors while keeping correct predictions. By contrast, `LogitEnsemble-PP` follows a more gradual slope, requiring a greater loss of coverage to achieve comparable risk levels. This difference reflects the value of incorporating engineered uncertainty features into the meta-model: `LogitEnsemble-UQ` can separate correct from incorrect predictions more sharply, enabling tighter control over the accuracy–coverage trade-off.

Density Separation of Correct vs. Incorrect Predictions

Figure 44.4 examines how confidence scores are distributed for correct and incorrect predictions under `LogitEnsemble-UQ` and `LogitEnsemble-PP`, at both baseline and target accuracy thresholds (80%, 85%, and 90%). Each panel shows a kernel density estimate of confidence scores, with correct predictions shaded separately from incorrect ones. A desirable pattern is a clear separation — correct predictions clustered toward high confidence, and incorrect predictions concentrated at low confidence. For all accuracy targets, `LogitEnsemble-UQ` exhibits a sharper separation: the correct-prediction density peaks closer to $s_i = 1$, while the incorrect-prediction density is shifted markedly toward lower scores. This separation remains consistent at higher target accuracies, reflecting the model's ability to identify and reject uncertain predictions. In contrast, `LogitEnsemble-PP` shows greater overlap between the two densities, which limits its ability to achieve low error rates without substantial rejection. These density plots visually confirm the calibration advantage of `LogitEnsemble-UQ` observed in the previous risk–coverage analysis.

Practical Implication

These results demonstrate that selective classification is not merely a theoretical construct — it is a practical and versatile mechanism for managing the trade-off between coverage and reliability in real-world systems. In many operational contexts, the ability to abstain from making a prediction can be as important as making one. By selectively rejecting uncertain cases, a model can achieve substantially higher accuracy on the subset of predictions it retains, thereby offering a built-in safeguard against costly errors. The value of this capability depends on the application domain. In safety-critical environments such as medical diagnosis, autonomous driving, or industrial quality control, a single incorrect prediction may have severe consequences, making a high-confidence subset preferable even at the cost of reduced coverage. Conversely, in high-throughput settings where human review is possible but expensive — for example, document triage in legal discovery or content moderation at scale — lowering the rejection rate while still meeting strict accuracy thresholds can translate directly into lower operational costs and faster turnaround times. Our results show that by incorporating richer and more diverse uncertainty features, such as those used in `LogitEnsemble-UQ`, it is possible to meet stringent target accuracies while discarding fewer samples compared to simpler confidence scores. This efficiency means that the model's abstention mechanism is more selective, rejecting only those cases most likely to be incorrect, rather than broadly discarding borderline predictions. Such precision in abstention decisions not only improves downstream efficiency but also enhances user trust by making the system's predictions more predictable and reliable.

44.4 Conclusion

This chapter has presented a comprehensive treatment of selective classification through the lens of uncertainty-aware rejection. By equipping models with the ability to withhold low-confidence predictions, we move toward safer, more controllable AI systems—particularly crucial when mistakes carry high costs. In safety-critical domains such

Table 44.1: Selective classification performance at 90% target accuracy.

Method	Accuracy [95% CI]	Rejection Rate (%)	Support	Threshold
Baseline	0.476 [0.466, 0.485]	0.00	10,000	—
MaxSoftmax	0.901 [0.886, 0.915]	83.06	10,000	0.999850
Margin	0.901 [0.885, 0.914]	83.20	10,000	0.999740
MarginProduct	0.901 [0.885, 0.914]	83.10	10,000	0.999580
BayesBeta	0.900 [0.885, 0.914]	82.97	10,000	0.710200
BayesECDF	0.900 [0.885, 0.914]	83.07	10,000	0.755619
MaxSoftmax-G	0.900 [0.870, 0.925]	95.08	10,000	0.853230
Margin-G	0.901 [0.875, 0.923]	93.55	10,000	0.836330
MarginProduct-G	0.901 [0.875, 0.923]	93.44	10,000	0.820560
BayesBeta-G	0.900 [0.885, 0.914]	83.45	10,000	0.508990
BayesECDF-G	0.900 [0.883, 0.916]	86.24	10,000	0.572145
MaxSoftmax-F1	0.900 [0.862, 0.931]	96.89	10,000	0.749990
Margin-F1	0.900 [0.868, 0.927]	95.78	10,000	0.729300
MarginProduct-F1	0.900 [0.870, 0.925]	95.10	10,000	0.709650
BayesBeta-F1	0.900 [0.884, 0.916]	86.04	10,000	0.406490
BayesECDF-F1	0.900 [0.882, 0.917]	88.25	10,000	0.460190
LogitEnsemble-UQ	0.900 [0.885, 0.914]	82.19	10,000	0.796690
LogitEnsemble-PP	0.900 [0.844, 0.942]	98.39	10,000	0.816200

Table 44.2: Selective classification performance at 85% target accuracy.

Method	Accuracy [95% CI]	Rejection Rate (%)	Support	Threshold
Baseline	0.476 [0.466, 0.485]	0.00	10,000	—
MaxSoftmax	0.850 [0.835, 0.865]	77.15	10,000	0.999150
Margin	0.850 [0.835, 0.865]	77.56	10,000	0.998630
MarginProduct	0.850 [0.835, 0.865]	77.36	10,000	0.997760
BayesBeta	0.851 [0.835, 0.865]	77.12	10,000	0.684890
BayesECDF	0.851 [0.835, 0.865]	77.35	10,000	0.751891
MaxSoftmax-G	0.850 [0.829, 0.869]	87.00	10,000	0.775310
Margin-G	0.851 [0.833, 0.867]	83.14	10,000	0.754990
MarginProduct-G	0.850 [0.833, 0.866]	82.00	10,000	0.737350
BayesBeta-G	0.850 [0.836, 0.864]	75.73	10,000	0.462390
BayesECDF-G	0.850 [0.835, 0.865]	77.24	10,000	0.516190
MaxSoftmax-F1	0.850 [0.828, 0.870]	88.45	10,000	0.647530
Margin-F1	0.850 [0.830, 0.869]	86.57	10,000	0.621790
MarginProduct-F1	0.850 [0.832, 0.868]	84.30	10,000	0.597070
BayesBeta-F1	0.850 [0.834, 0.866]	79.34	10,000	0.349690
BayesECDF-F1	0.850 [0.835, 0.865]	77.79	10,000	0.360490
LogitEnsemble-UQ	0.850 [0.836, 0.864]	74.78	10,000	0.696290
LogitEnsemble-PP	0.850 [0.816, 0.881]	94.98	10,000	0.742600

as medical diagnosis, autonomous driving, or fraud detection, selective classification improves system reliability by ensuring that only high-confidence outputs are automated, while uncertain cases can be deferred to human experts or auxiliary systems. This approach mirrors rational decision-making under uncertainty and provides a principled framework to balance accuracy, coverage, and trustworthiness.

We introduced a unified mathematical framework for confidence scoring and selection, spanning classical and probabilistic scoring functions, and examined their performance under various rejection-aware strategies, including class-specific weighting schemes. Our empirical analysis revealed how different confidence estimators influence the trade-off between accuracy and coverage, and demonstrated that ensemble-based scoring can deliver better calibration and robustness. Crucially, confidence-based thresholding allows models to meet user-specified accuracy targets while minimizing the proportion of filtered samples—an asset in risk-sensitive deployments where precision is non-negotiable.

Yet, selective classification comes with inherent trade-offs. Reduced coverage may be unacceptable in high-throughput applications, and the approach depends critically on the quality of uncertainty estimates. Poor calibration can cause the system to filter easy cases or misclassify difficult ones with unwarranted confidence. Moreover, real-world deployments must contend with operational costs of filtering—delays, human intervention, and potential inequities if filtering rates differ across subpopulations.

Looking ahead, a promising research direction is to move from post-hoc filtering to end-to-end training of abstention-aware models, where classification and selection are jointly optimized. In this context, the confidence scoring functions developed in this chapter have broader utility beyond selective classification; they can be leveraged in active learning paradigms to identify the most informative samples for annotation, thereby improving data efficiency and accelerating model improvement. This synergy highlights the wider impact of principled uncertainty quantification in building adaptive, data-efficient, and trustworthy AI systems.

Selective classification is not merely a performance enhancement; it is a foundational framework for building adaptive, accountable, and risk-aware AI. As the field advances, integrating uncertainty estimation and decision-

Table 44.3: Selective classification performance at 80% target accuracy.

Method	Accuracy [95% CI]	Rejection Rate (%)	Support	Threshold
Baseline	0.476 [0.466, 0.485]	0.00	10,000	—
MaxSoftmax	0.800 [0.785, 0.814]	70.07	10,000	0.996569
Margin	0.800 [0.785, 0.814]	70.59	10,000	0.994400
MarginProduct	0.800 [0.785, 0.814]	70.10	10,000	0.990470
BayesBeta	0.800 [0.785, 0.814]	70.07	10,000	0.635290
BayesECDF	0.800 [0.785, 0.814]	70.13	10,000	0.741190
MaxSoftmax-G	0.800 [0.784, 0.816]	75.42	10,000	0.721790
Margin-G	0.800 [0.784, 0.816]	74.12	10,000	0.702200
MarginProduct-G	0.800 [0.785, 0.815]	72.03	10,000	0.682580
BayesBeta-G	0.800 [0.786, 0.814]	67.85	10,000	0.413990
BayesECDF-G	0.800 [0.786, 0.814]	68.40	10,000	0.441990
MaxSoftmax-F1	0.800 [0.782, 0.818]	80.12	10,000	0.577079
Margin-F1	0.800 [0.783, 0.817]	78.30	10,000	0.541160
MarginProduct-F1	0.800 [0.783, 0.817]	77.22	10,000	0.519430
BayesBeta-F1	0.801 [0.786, 0.815]	70.21	10,000	0.300890
BayesECDF-F1	0.800 [0.785, 0.814]	69.45	10,000	0.304390
LogitEnsemble-UQ	0.801 [0.787, 0.814]	67.35	10,000	0.587890
LogitEnsemble-PP	0.800 [0.775, 0.824]	89.34	10,000	0.700090

theoretic reasoning will be essential. Ultimately, in high-stakes AI, the wisdom to know when not to predict is as important as knowing what to predict.

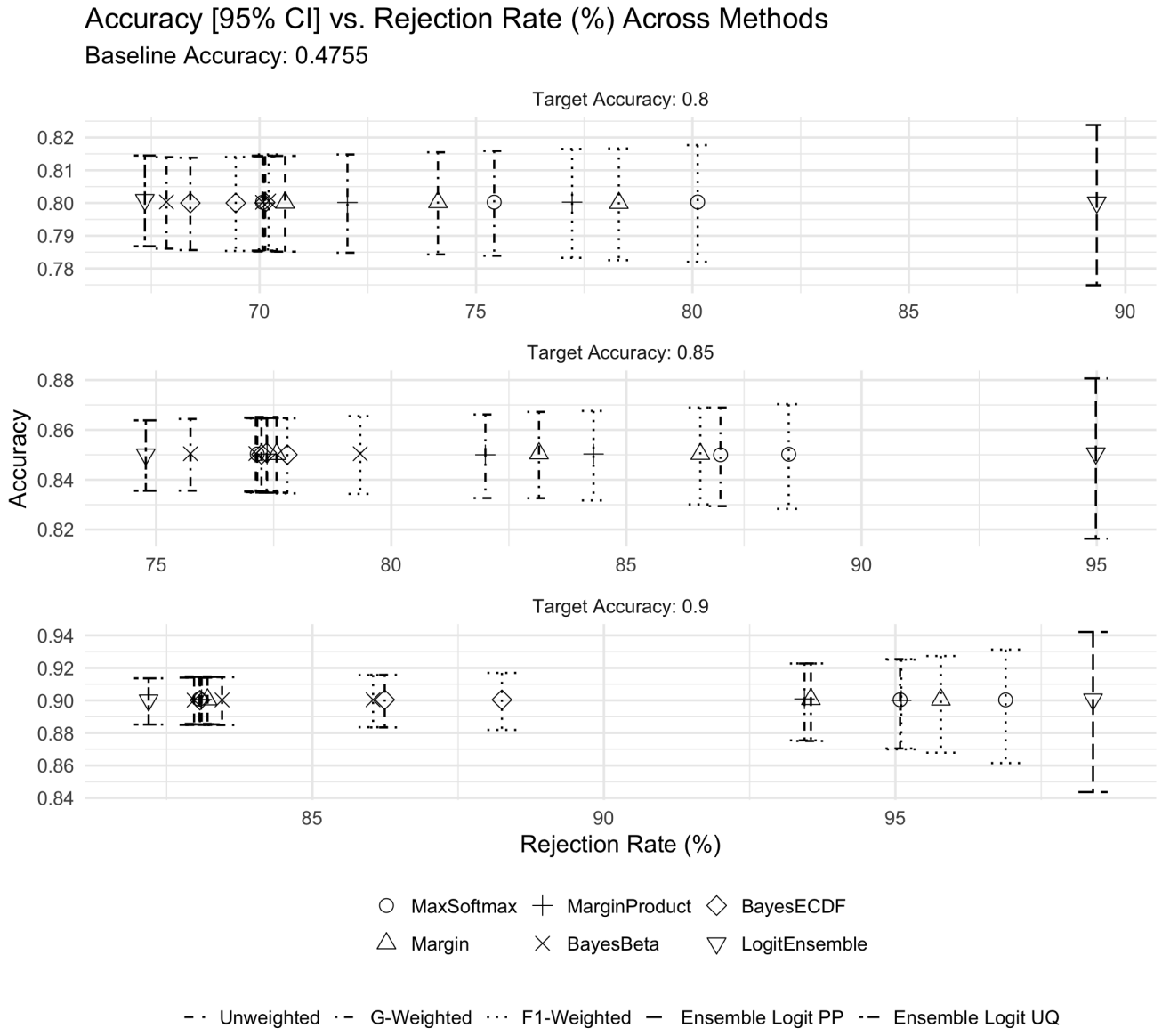


Figure 44.2: Selective accuracy–rejection trade-off on CIFAR-100 for all confidence scoring functions. For each method and target accuracy level, a threshold τ_α is chosen on the confidence score s, and predictions with $s < \tau_\alpha$ are rejected; the x-axis reports the resulting rejection rate $1 - \widehat{\text{coverage}}$ and the y-axis reports selective accuracy (accuracy computed on retained predictions only). Error bars denote 95% confidence intervals for the selective accuracy estimate on the retained set. Across targets, `LogitEnsemble-UQ` achieves the lowest rejection at comparable selective accuracy, indicating a more effective ranking of correct over incorrect predictions.

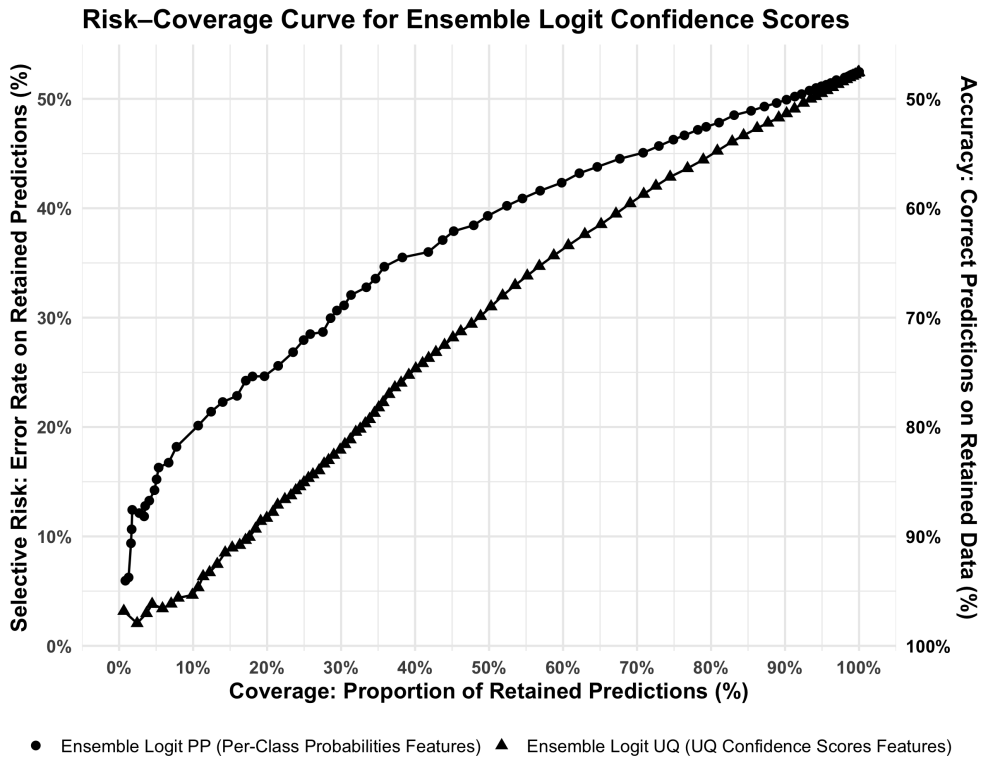


Figure 44.3: Risk–coverage curve for selective classification on CIFAR-100 using ensemble logistic-regression confidence scores. The x-axis shows *coverage* (percentage of predictions retained after applying a confidence threshold τ); the left y-axis shows *selective risk* (error rate computed on the retained predictions only), and the right y-axis shows the corresponding *selective accuracy* (1−risk). Each point corresponds to a different threshold τ: moving to the left (lower coverage) uses a higher τ, rejects more low-confidence samples, and therefore typically decreases selective risk. Curves are shown for *Ensemble Logit PP* (features: per-class probabilities) and *Ensemble Logit UQ* (features: engineered uncertainty scores); the *Ensemble Logit UQ* curve lies below *Ensemble Logit PP*, indicating lower risk at the same coverage.

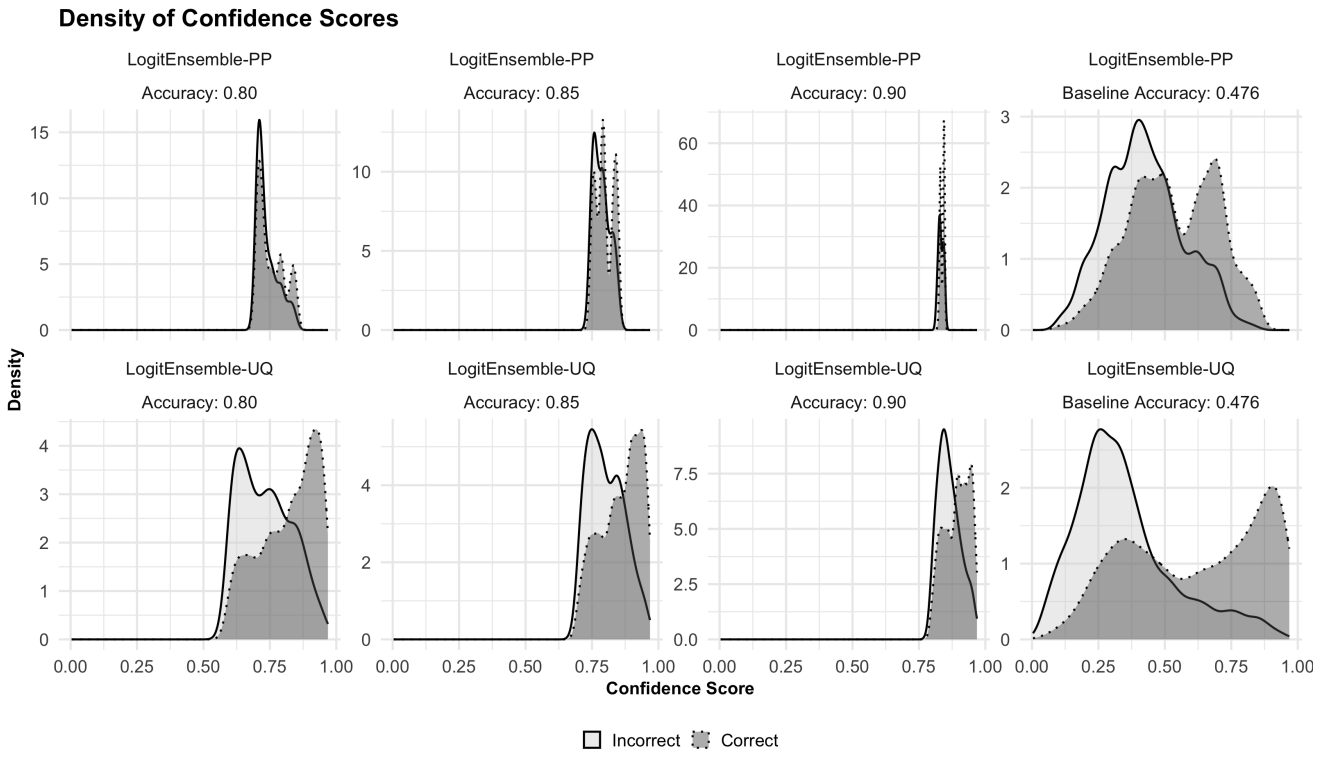


Figure 44.4: Confidence score density distributions for logistic regression ensembles at different operating points. For each target accuracy, we first select a threshold τ_α and *retain only* samples with confidence $s_i \geq \tau_\alpha$; the plotted curves therefore show the *conditional* densities of the retained set, $p(s \mid s \geq \tau_\alpha, \text{correct})$ and $p(s \mid s \geq \tau_\alpha, \text{incorrect})$. As the target accuracy increases, τ_α increases and the retained set becomes smaller, which truncates the left tail of the score distribution and changes the apparent densities across panels. Top panels correspond to *LogitEnsemble-PP*, which uses the raw predicted probability vector $\mathbf{p}_i$ as features. Bottom panels correspond to *LogitEnsemble-UQ*, which uses engineered uncertainty features s_i (including class-conditional weighting). The stronger separation between the retained correct and incorrect densities for *LogitEnsemble-UQ* indicates improved ranking of correct over incorrect predictions, yielding lower rejection for the same target accuracy.

Chapter 45

Biomedical data imputation

45.1 Introduction

Missing data persists as a major barrier in large-scale analyses of multivariate data, due to issues like incomplete collection, data availability, and low coverage. In biomedicine, missing data often arises in clinical trials, health surveys, or longitudinal studies, where patients may drop out or fail to answer certain questions due to various factors, such as side effects or worsening health conditions. Understanding the types of missingness—such as missing completely at random (MCAR), missing at random (MAR), and missing not at random (MNAR)—is crucial for selecting appropriate analytical methods [1057]. Early approaches for dealing with missing data tend to reduce the generalisability of results or skew the trends present in the data [1519]. These include list-wise deletion, where only complete observations are considered, or imputations methods, where the complete data is used to predict plausible values for missing data points. Some of these imputation strategies include substitution by the mean of observed values, stochastic regression techniques, and hot-deck imputation. Single imputation approaches implicitly assume that the imputation is perfect and thereby fail to account for the uncertainty introduced by the prediction, while multiple imputation creates multiple copies of the data with different plausible values for the missing components.

More recently, machine learning approaches have been employed to deal with missingness, offering flexible solutions that can capture complex patterns in the data. For example, genomics datasets tend to be extremely high-dimensional, and deep learning approaches offer an attractive framework to capture high-dimensional relationships during imputation. In particular, deep generative models, such as variational autoencoders, offer an effective approach to learn joint distributions in high dimensions. This enables multiple imputation to account for uncertainty in missing data and subsequently greater robustness in downstream analysis.

45.2 Missing data

This section will explore the types of missing data, present biomedical examples, and briefly discuss traditional and machine learning methods used to address missing data.

45.2.1 Missing data types

Missing data can arise from a range of causes [369]:

- participants may withdraw from a study, or they could be removed if a treatment proves ineffective;
- people might choose not to respond to certain survey questions, such as ones related to gender or race;
- sometimes missingness is part of the study design. For example, limited resources may prevent follow-up with all subjects, leading to data gaps introduced through a known process, such as randomly selecting individuals to follow up.

In some situations, missing data can also occur for different reasons at the same time, and understanding why data is missing is critical for determining the best analysis approach. As stated in the work of Rubin [1432], the underlying cause of missingness plays a crucial role in determining whether a particular strategy is appropriate.

Missing data are therefore typically classified into three categories [1057, 1432], based on the underlying mechanism that leads to missing data.

Let $X = (x_{ij})$ denote a data set of size $N \times D$, where N represents the number of observations and D the number of variables. The i-th row of X, $x_i = (x_{i1}, \ldots, x_{iD})$, contains the values for each variable corresponding to unit i, with x_{ij} representing the value of variable j for the i-th observation. In the case of missing data, it is useful to decompose the row x_i into two parts: x_i^{obs} and x_i^{mis}, which represent the observed and the missing values of x_i, respectively. To capture this missingness pattern, it is also convenient to introduce a *missingness indicator matrix* M, with elements

$$m_{ij} = \begin{cases} 1 & \text{if } x_{ij} \text{ is missing} \\ 0 & \text{if } x_{ij} \text{ is observed} \end{cases} . \tag{45.1}$$

Missing completely at random (MCAR). This is the simplest form of missingness and it occurs when the causes of the missingness are unrelated to the data. In this framework, for each observation, the probability of being missing does not depend on the values of any other variables in the data:

$$\mathbb{P}\left(m_i \mid x_i^{\text{obs}}, x_i^{\text{mis}}\right) = \mathbb{P}\left(m_i\right), \tag{45.2}$$

where m_i represents the vector of zeros and ones characterising the missing pattern. Examples of this missing scenario include situations such as the accidental drop or break of a blood sample tube from a participant, preventing the measurement of blood parameters, or the unintentional loss of a participant's questionnaire [578].

Missing at random (MAR). This assumes that the missingness mechanism can be explained by some of the observed features in the data set. Hence, for each observation x_i, the probability of missingness is conditionally independent of the unobserved data, given the observed data:

$$\mathbb{P}\left(m_i \mid x_i^{\text{obs}}, x_i^{\text{mis}}\right) = \mathbb{P}\left(m_i \mid x_i^{\text{obs}}\right). \tag{45.3}$$

Here, x_i^{obs} may also refer to a subset of the observed features. A classical example of this missing mechanism in clinical trials is when patients drop out based on (observable) characteristics, such as age or side effects of the treatment, but not on their (unobserved) final health status.

Missing not at random (MNAR). This occurs when the probability of missingness depends on the unobserved value itself (i.e. the missing feature), not just on the observed values:

$$\mathbb{P}\left(m_i \mid x_i^{\text{obs}}, x_i^{\text{mis}}\right) \neq \mathbb{P}\left(m_i \mid x_i^{\text{obs}}\right). \tag{45.4}$$

In the MNAR case, the missing variable cannot be predicted solely from the available data. If missing data are MNAR, valuable information is lost and there is no universally applicable method for properly addressing this type of missing data. For example, when a sensor fails to capture information beyond a certain threshold, its data are missing due to MNAR factors, and this is referred to as censoring [578].

Examples in biomedicine

Missing data is persistent in the field of biomedicine, spanning both clinical and experimental data settings. In clinical trials, for example, MNAR and MAR settings are frequent and can arise due to participant dropout before study completion or missed visits. These might originate from increased disease severity or even socio-demographic factors that might influence whether a participant will complete the study. Since missingness can be tied directly to important measurable outputs in these cases, imputation strategies that retain the integrity of the data and limit introduced biases are crucial for robust statistical analyses.

With the rapid expansion of high-throughput sequencing over the past few decades, a wealth of high-dimensional 'omics datasets has become available, offering novel insights into disease origins, progression, and opportunities for precision therapy [92]. However, these datasets are also prone to various forms of missingness. Depending on the modality – whether measuring gene expression (transcriptomics), protein abundance (proteomics), DNA sequence variation (genomics), genome accessibility (epigenomics), or metabolites (metabolomics) – each provides a distinct window into the underlying biological mechanisms, but each is also susceptible to missing data [797]. In transcriptomic data, for example, MCAR values may arise from random instrument failures, MAR values can occur when slight experimental differences are introduced across samples (e.g., use of different library preparation

kits), and MNAR values may result when low-abundance genes in a sample fall below the detection threshold of the sequencing platform.

Transcriptomic datasets can be measured in bulk, where gene expression is quantified across a group of cells (bulk RNA-sequencing; bulkRNA-seq), or at the single-cell level, where each cell is sequenced separately (single-cell RNA-sequencing; scRNA-seq). scRNA-seq is much more sparse and suffers from higher rates of missingness than bulkRNA-seq, where missingness rates are mostly due to real biological zeros, features of the experimental protocol, sequencing machine or the genes themselves [1352, 1800]. Missingness in bulkRNA-seq is relatively low compared to scRNA-seq, ranging from $10 - 40\%$ [390, 1678], whereas scRNA-seq can be as high as 90% [429].

45.2.2 Missing data methods

Methods for handling missing data proposed in the literature can be broadly classified into several categories [578, 1057]:

Deletion. When certain variables contain missing values, a straightforward approach is to eliminate incomplete cases and only use observations with fully recorded data. This method, also referred to as *complete-case analysis* [1057], is simple to apply and may work well when missingness is minimal, but it can introduce significant bias, especially when the learning task involves inference on a subpopulation of the data.

Classic imputation methods. In these procedures, the missing values are estimated and replaced, allowing for the completed dataset to be analysed using standard techniques. The most common imputations methods include: *mean/median/mode imputation*, where the missing values are substituted with the mean (or median/mode) of the available observed data for each variable, and *hot-deck imputation* where missing data values are filled in by drawing from similar, observed data points that are considered comparable to the unit with missing data.

Imputation based on statistical machine learning methods. In this approach, missing data for an observation is predicted using a machine learning (ML) model, leveraging the relationships between observed variables. Some widely used ML methods for imputation include: *regression*, *k-nearest neighbours*, *random forests*, and more; they can be seen as extensions or algorithmic implementations of classical statistical imputation strategies. Although they may offer flexibility and scalability in high-dimensional settings, they still belong to the class of *single imputation* techniques. As such, they share a key limitation: imputed values are treated as *exact*, and therefore uncertainty in the imputation process is overlooked.

To address the uncertainty in imputation, *deep generative models* such as *Variational Autoencoders (VAEs)*, offer a more robust approach by modelling the underlying data distribution. VAEs can learn the conditional distribution of the missing data given the observed data and latent variables, enabling multiple imputations to be generated. This method accounts for uncertainty, as it provides a distribution over possible imputations rather than a single deterministic value.

45.3 Variational autoencoder

45.3.1 Deep latent variable models

Deep latent variable models (DLVMs), a specific class of generative models, aim to model the joint distribution of the observed variables, allowing for generation of new data points similar to the training data [895]. Generally, latent variable models assume that each data point— represented by x — is generated from a model depending on unobserved latent variables, denoted by $z \in \mathbb{R}^K$. These latent variables are typically assumed to lie in a much lower-dimensional space ($K \ll D$) and are used to explain the underlying structure of the observed data. DLVMs are widely used in many areas of ML, especially when dealing with complex or high-dimensional data.

In DLVMs, the joint density of the observed and latent variables is factorised as

$$p_\theta(x, z) = \underbrace{p(z)}_{\text{prior}} \underbrace{p_\theta(x \mid z)}_{\text{model}}. \tag{45.5}$$

The conditional density $p_\theta(x \mid z)$ employs deep learning (e.g. neural networks, with weights and biases contained in θ) to flexibly map the latent variables to parameters of the conditional density, while the prior $p(z)$ is typically assumed to be a standard normal distribution $\mathcal{N}(0, I)$, which dictates how the latent variables behave independently of the data.

There are often two aims in DLVMs: data generation and encoding. Generating a new data point x can be accomplished by sampling from

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz. \tag{45.6}$$

Although this can be done by first sampling the latent variables from their prior, and then passing the sampled z through the deep learning model and generating x from its conditional distribution $p_\theta(x \mid z)$, this requires knowledge of the parameters θ. Ideally, the parameters θ would be learnt by optimising the intractable marginal likelihood

$$p_\theta(X) = \prod_{i=1}^{N} \int p_\theta(x_i \mid z)\, p(z)\, dz. \tag{45.7}$$

Encoding refers to learning the low-dimensional representation of the data, which amounts to computing the posterior of the latent variables given the observed data, defined as

$$p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)} = \frac{p_\theta(x \mid z)\, p(z)}{\int p_\theta(x \mid z)\, p(z)\, dz}, \tag{45.8}$$

which again is intractable and requires knowledge of θ.

To overcome this intractability, VAEs employ *variational inference* to both approximate the posterior distribution and estimate θ. Instead of computing $p_\theta(z \mid x)$ directly, VAEs introduce a variational distribution $q_\phi(z \mid x)$, parameterised by a separate set of parameters ϕ, which serves as a tractable approximation to the true posterior, and find θ by introducing and optimising a lower bound to the marginal likelihood (45.7).

Diffusion models are a complementary family of latent variable generative models that can also be used for imputation; see Chapter 50 for a detailed discussion of this alternative approach and a comparison with VAEs.

45.3.2 Variational autoencoders

A variational autoencoder (VAE) is a particular DLVM [895], that employs neural networks to flexibly map the latent variables to the observed data and variational inference to address intractability. It is made up of two components:

- The *inference model*, also referred to as the *encoder* (or the *recognition* model) is the variational posterior of the form $q_\phi(z \mid x)$. It takes an observed data point, $x \in \mathbb{R}^D$, and encodes it in the lower-dimensional latent space, $z \in \mathbb{R}^K$, via the approximate posterior distribution of the latent variables $q_\phi(\cdot \mid x)$.
- The *generative model*, also called the *decoder*, of the form $p_\theta(x \mid z)$, takes latent variables z and reconstructs the data point, $\hat{x}$, via the generative model $p_\theta(\cdot \mid z)$.

While the prior on the latent variables $p(z)$ is often assumed to be a standard normal distribution $\mathcal{N}(0, I)$, the variational distribution $q_\phi(z \mid x)$ is a design decision that must be considered. Additionally, the choice of likelihood function $p_\theta(x \mid z)$ depends on the nature and statistical properties of the data, such as whether the observations are continuous or discrete. Below are some examples.

Example: Exponential family. Exponential family distributions are a class of probability distributions that can be used to model different types of data (e.g. real-valued, count, binary, and categorical). An *exponential family VAE* [424] chooses a generative model belonging to the exponential family, parametrised by a deep neural network:

$$p_\theta(x \mid z) = \text{ExpFam}\left(x; \eta_\theta(z)\right) \tag{45.9}$$

$$= \nu(x) \exp\left(\eta_\theta(z)^\top x - A(\eta_\theta(z))\right), \tag{45.10}$$

where:

- $\nu(x)$ is the base measure, which depends on the specific distribution in the exponential family;
- $\eta_\theta(z)$ is the natural parameter, which depends on the latent variable z, and specifically, is obtained as the output of the neural network with parameters θ and input z;
- $A(\eta_\theta(z))$ is the log-normaliser of the exponential family.

This class includes several commonly used likelihoods, such as Gaussian, Poisson, and Bernoulli. Therefore, *exponential family VAEs* can be applied to a wide range of data types.

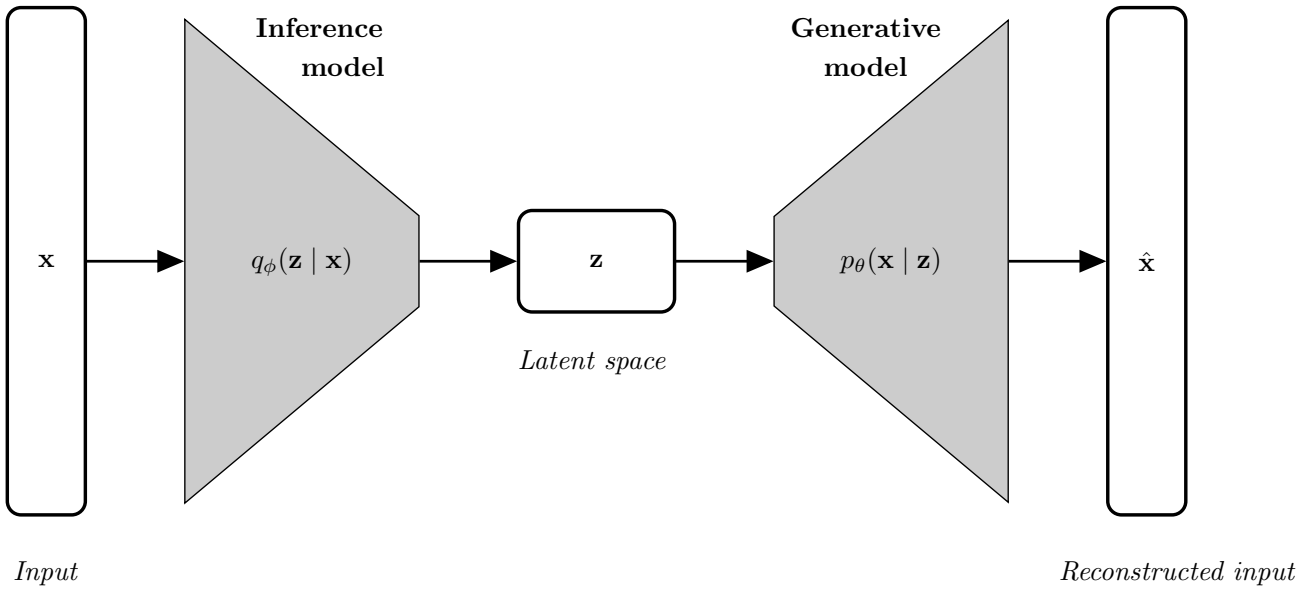


Figure 45.1: Structure of a variational autoencoder.

Example: Factorised Gaussian. In the case of continuous data, a standard choice for both the generative (decoder) and inference model (encoder) is a simple factorised Gaussian:

$$p_\theta(x \mid z) = \mathcal{N}\left(x; \mu_\theta(z), \mathrm{diag}(\sigma^2_\theta(z))\right), \qquad q_\phi(z \mid x) = \mathcal{N}\left(z; \mu_\phi(x), \mathrm{diag}(\sigma^2_\phi(x))\right), \tag{45.11}$$

where $\mu_\theta(z) \in \mathbb{R}^D$, $\sigma^2_\theta(z) \in \mathbb{R}^D_+$, $\mu_\phi(x) \in \mathbb{R}^K$, $\sigma^2_\phi(x) \in \mathbb{R}^K_+$, and neural networks are employed to map the latent variables (or observed data) to the means and variances of the generative (or inference) model:

$$(\mu_\theta(z), \log[\sigma_\theta(z)]) = \mathrm{DecoderNeuralNet}_\theta(z), \tag{45.12}$$

$$(\mu_\phi(x), \log[\sigma_\phi(x)]) = \mathrm{EncoderNeuralNet}_\phi(x). \tag{45.13}$$

Biomedical application: In a biomedical setting, this choice is natural for gene expression data that has been z-scored, such that the expression of each gene across all observations has mean zero and unit standard deviation [895, 1352].

Example: Multivariate Gaussian. The factorised Gaussian (45.11) can be extended to a Gaussian with full covariance:

$$p_\theta(x \mid z) = \mathcal{N}\left(x; \mu_\theta(z), \Sigma_\theta(z)\right), \qquad q_\phi(z \mid x) = \mathcal{N}\left(z; \mu_\phi(x), \Sigma_\phi(x)\right). \tag{45.14}$$

In this case, the encoder and decoder networks are modified to output both the mean vectors and a parametrisation of the covariance matrices. A common choice is to use the Cholesky decomposition: $\Sigma_\theta(z) = L_\theta(z)L_\theta(z)^\top$ and $\Sigma_\phi(x) = L_\phi(x)L_\phi(x)^\top$, where $L_\theta(z)$ and $L_\phi(x)$ are lower triangular matrices, with non-zero elements given by the output of the networks. This approach guarantees positive semi-definiteness of the covariance matrices and facilitates stable training. For more details on the advantages of reparametrisation in VAEs, see [892].
Biomedical application: Multivariate Gaussians, combined with structured forms for the covariance (or precision) matrices to reduce dimensionality, might be applied to multi-omic data integration, whereby different 'omics modalities are modelled under a common framework [116].

Example: Negative binomial. Consider the case of discrete count data x, where $x \in \mathbb{N}^D$ may represent word counts, with D being the size of the vocabulary. A natural choice for the generative model (decoder) is the negative binomial [1865] defined by the shape/dispersion parameters $r \in \mathbb{R}^D_+$ and success probabilities $\rho \in (0,1)^D$:

$$p_\theta(x \mid z) = \mathrm{NB}\left(x; r_\theta(z), \rho_\theta(z)\right). \tag{45.15}$$

These parameters are derived from the latent variable z through nonlinear transformation defined by the decoder neural network:

$$(\log(r_\theta(z)), \mathrm{sigmoid}^{-1}(\rho_\theta(z))) = \mathrm{DecoderNeuralNet}_\theta(z). \tag{45.16}$$

This is referred to as the *negative binomial VAE* (NBVAE). In this model, the variational posterior (encoder) can be either a factorised or full Gaussian.
Biomedical application: scRNA-seq count data tends to be overdispersed and sparse, so is generally modelled using (zero-inflated) Poisson or negative binomial distributions. The NBVAE could therefore be a natural choice for this 'omics setting [673, 1074].

Example: Multinomial. An alternative method for modelling discrete data is to use a multinomial likelihood:

$$p_\theta(x \mid z) = \text{Multi}(x; \rho_\theta(z)), \tag{45.17}$$

where $\rho_\theta(z)$ lies in the simplex of dimension $D-1$ and represents the probabilities of different categories (e.g., different words in a vocabulary). These probabilities are derived from the latent variable z via a neural network (decoder):

$$\text{softmax}^{-1}(\rho_\theta(z)) = \text{DecoderNeuralNet}_\theta(z)\ . \tag{45.18}$$

Like NBVAEs, multinomial VAEs (MultiVAEs) [1170, 1865] use a factorised or full Gaussian variational posterior.

Remark 45.1. *It is worth noting that standard VAEs are generally not identifiable without additional restrictions on the model. This is not a major concern in cases where the primary goal is imputation rather than a scientifically meaningful representation of the latent space. When identifiability of the latent variables is of interest —for instance, when they are intended to carry a biological interpretation— identifiable yet flexible alternatives may be considered instead, such as structured factor models or constrained deep exponential-family models (see, e.g., [101, 1378]).*

Evidence lower bound (ELBO)

From a mathematical perspective, the goal is to find the model parameters θ (decoder) in order to maximise the marginal likelihood of the observed data, or equivalently $\log p_\theta(X)$, and derive the encoding of the data through the posterior $p_\theta(z \mid x)$. However, as discussed earlier, directly computing this likelihood and posterior is intractable. To address this, VAEs use a variational approximation of the posterior, $q_\phi(z \mid x)$, that leads to the introduction of the *evidence lower bound* (ELBO), which can be optimised as a proxy for maximising the marginal likelihood to find θ.

Starting from the log-marginal likelihood:

$$\log p_\theta(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x)] = \mathbb{E}_{q_\phi(z|x)}\left[\log\left(\frac{p_\theta(x,z)}{p_\theta(z \mid x)}\right)\right] \tag{45.19}$$

$$= \mathbb{E}_{q_\phi(z|x)}\left[\log\left(\frac{p_\theta(x,z)}{q_\phi(z \mid x)}\frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\right)\right] \tag{45.20}$$

$$= \underbrace{\mathbb{E}_{q_\phi(z|x)}\left[\log\left(\frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right)\right]}_{\text{ELBO}_{\phi,\theta}(x)} + \underbrace{\mathbb{E}_{q_\phi(z|x)}\left[\log\left(\frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\right)\right]}_{\text{D}_{\text{KL}}(q_\phi(z|x)\|p_\theta(z|x))}, \tag{45.21}$$

we get the decomposition:

$$\log p_\theta(x) = \text{ELBO}_{\phi,\theta}(x) + \text{D}_{\text{KL}}(q_\phi(z \mid x)\|p_\theta(z \mid x)). \tag{45.22}$$

Since the KL divergence term is always non-negative, Equation (45.22) shows that maximising the ELBO is equivalent to maximising a lower bound on $\log p_\theta(x)$. By employing a flexible neural network to approximate the posterior distribution, we aim to minimise the KL divergence (obtaining a good approximation of the latent space), subsequently leading to a tight lower bound (better generative capabilities).

Optimising the ELBO

VAE aims to solve the optimisation problem:

$$\max_{\theta,\phi} \text{ELBO}_{\phi,\theta}(X) = \max_{\theta,\phi} \sum_{i=1}^{N} \text{ELBO}_{\phi,\theta}(x_i), \tag{45.23}$$

as the data points are assumed independent. In practice, it is useful to rewrite the objective function of our optimisation problem (45.23) in a form that highlights its two main components: the reconstruction loss and the

KL divergence. This decomposition allows us to better understand the objective we are minimising and how each part contributes to the overall training process.

For a data point x, the ELBO can be rewritten as:

$$\text{ELBO}_{\phi,\theta}(x) = \mathbb{E}_{q_\phi(z|x)}\left[\log\left(\frac{p_\theta(x \mid z)p(z)}{q_\phi(z \mid x)}\right)\right] \tag{45.24}$$

$$= \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] + \mathbb{E}_{q_\phi(z|x)}\left[\log\left(\frac{p(z)}{q_\phi(z \mid x)}\right)\right] \tag{45.25}$$

$$= \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{D}_{\mathrm{KL}}(q_\phi(z \mid x)\|p(z)). \tag{45.26}$$

Hence, the optimisation problem in Equation 45.23 can be reformulated as the minimisation problem:

$$\min_{\theta,\phi} \underbrace{\sum_{i=1}^{N} \mathbb{E}_{q_\phi(z_i|x)}[-\log p_\theta(x_i \mid z_i)]}_{\text{Reconstruction Loss}} + \underbrace{\sum_{i=1}^{N} \mathrm{D}_{\mathrm{KL}}(q_\phi(z_i \mid x_i)\|p(z_i))}_{\text{KL Divergence}}, \tag{45.27}$$

balancing the trade-off between accurate data reconstruction and regularising the latent space according to the prior.

During training, the ELBO is maximised (or equivalently (45.27) is minimised) by stochastic gradient descent, which uses backpropagation through the hidden layers of the neural network to compute gradients and random sub-sampling of the data at each training step. To obtain an unbiased estimator of the required gradients, the reparametrisation trick [892, Section 2.4] is used, which applies a change of variables. For a fuller treatment of variational inference and the ELBO, see Chapter 12.

45.3.3 Beta-VAEs

In Bayesian statistics, the power likelihood is a generalisation of the classic likelihood that introduces an adjustable parameter to control the weight of the likelihood in the inference process. It offers a more flexible approach to learning from data and can be especially useful when dealing with noisy data, outliers or potential model misspecification [164, 772].

The power likelihood modifies the standard likelihood $p(x \mid \theta)$ by raising it to a power w,

$$p_w(x \mid \theta) = p(x \mid \theta)^w, \tag{45.28}$$

with $w > 0$. In Bayesian inference, this likelihood is combined with the prior to form the posterior distribution:

$$p_w(\theta \mid x) \propto p(x \mid \theta)^w p(\theta)\ . \tag{45.29}$$

Adjusting w provides a mechanism to control the balance between the prior and the likelihood: for $w < 1$, the influence of the data on the posterior is down-weighted, making the model more sensitive to the prior, while for $w > 1$ the likelihood is amplified, making the model more sensitive to the data.

For this reason, when the data contains outliers, the power likelihood with $w < 1$ reduces their influence and can lead to more robust inference, while using the standard likelihood can lead to over-fitting where the model tries to fit anomalous or unreliable data points. In the same way, when the model is misspecified —meaning the model may not perfectly match the true underlying data generating process— the full likelihood can drive the model to fit aspects of the data that are incorrectly captured by the misspecified model. Exploiting the power likelihood with a small w can mitigate this effect of misspecification giving more importance to the prior distribution.

In deep generative models, such as VAEs, given the complex and high-dimensional nature of the data, there is likely to be a mismatch between the model and the true data-generating distribution, which motivates the adoption of power likelihood in the learning process.

Power likelihood in ELBO

Motivated by this reasoning, we can apply the power likelihood in the ELBO in Equation 45.27. This leads to the following modified objective:

$$\sum_{i=1}^{N} \mathbb{E}_{q_\phi(z_i|x_i)}[-\log p_\theta(x_i \mid z_i)^w] + \mathrm{D}_{\mathrm{KL}}(q_\phi(z_i \mid x_i)\|p(z_i)), \tag{45.30}$$

where minimising (45.30) is equivalent to minimising the KL divergence between the variational posterior $q_\phi(Z \mid X)$ and the posterior under the power likelihood:

$$p_{\theta,w}(Z \mid X) \propto \prod_{i=1}^{N} p_\theta(x_i \mid z_i)^w p(z_i). \tag{45.31}$$

This naturally leads to the well-known β-VAEs [746], an extension on the classic VAE that includes a hyperparameter, referred to as β, that enforces regularisation on the latent loss.

In the ML community, β-VAEs are primarily known for improving disentanglement of the latent variables [271]. However, from a statistical perspective, their formulation can be understood as applying the power likelihood with $w = 1/\beta$, providing an alternative role of β in the model objective function. Indeed, it can be easily seen that the optimisation function used in training β-VAEs:

$$\sum_{i=1}^{N} \mathbb{E}_{q_\phi(z_i|x_i)}[\log p_\theta(x_i \mid z_i)] - \beta \, \mathrm{D}_{\mathrm{KL}}(q_\phi(z_i \mid x_i) \| p(z_i)), \tag{45.32}$$

is equivalent to:

$$\sum_{i=1}^{N} \mathbb{E}_{q_\phi(z_i|x_i)}\Big[\log p_\theta(x_i \mid z_i)^{1/\beta}\Big] - \mathrm{D}_{\mathrm{KL}}(q_\phi(z_i \mid x_i) \| p(z_i)), \tag{45.33}$$

which corresponds exactly to (45.30) with $w = 1/\beta$. In this framework, motivated by the use of power-likelihood, β controls the trade-off between the prior and the likelihood: for $\beta > 1$, the influence of the data on the posterior is down-weighted making the model more sensitive to the prior, while for $\beta < 1$ the likelihood is amplified, making the model more sensitive to the data. The classic VAE is retrieved for $\beta = 1$.

It is worth noticing that, after raising $p_\theta(x \mid z)$ to a power, the result is typically a generalized or tempered likelihood unless it is renormalized. In particular, in the factorized Gaussian case, this results in a covariance-inflation (or covariance deflation) interpretation briefly discussed in Section 45.4.1.

45.4 Variational autoencoders and missing data

The generative nature of DLVMs makes them particularly effective for data imputation tasks. They have become increasingly popular for handling missing data due to their ability to model complex, nonlinear relationships and intricate dependencies within multivariate datasets. Much of the progress in imputation using these models can be attributed to foundational generative models, such as VAEs.

One of the first applications of VAEs in missing data imputation appears in Rezende et al. [1391]. By introducing the reparametrisation trick, which enables efficient gradient-based optimisation for VAEs, they were able to achieve scalable training of deep generative models, setting the stage for their application in imputation. Building on this, Mattei and Frellsen [1146] proposed a modification that allows for exact conditional sampling in missing data scenarios. Following these advancements, Mattei and Frellsen [1147] introduced an extension that minimises a tighter lower bound based on importance weighted autoencoders [220]. To handle incomplete, heterogeneous data, Nazábal et al. [1225] developed an approach that incorporates both numerical and nominal data types for more complex imputation tasks. More recent methods, such as Ipsen et al. [820], continue to refine these models, especially in challenging settings such as MNAR. These developments highlight the continued progress in deep generative models for missing data imputation, with each new approach improving the model's ability to handle a wider variety of data types and missing data patterns.

Most approaches typically involve an initial imputation using a simple method (e.g., mean imputation) to create an initial complete dataset. Then, a VAE trained on this initial complete dataset provides parameters (ϕ, θ). Assuming that each data point x_i can be partitioned into observed features x_i^{obs} and missing features x_i^{mis}, the goal is to use the trained VAE to estimate x_i^{mis} conditional on the observed components.

Single imputation

Single imputation provides a single estimate of the missing data. Under a squared error loss, the optimal imputation corresponds to the mean under the generative model, that is the conditional expectation

$$\hat{x}_i^{\mathrm{mis}} = \mathbb{E}\big[x_i^{\mathrm{mis}} \mid x_i^{\mathrm{obs}}\big] = \int x_i^{\mathrm{mis}} p_\theta(x_i^{\mathrm{mis}} \mid x_i^{\mathrm{obs}}) \, dx_i^{\mathrm{mis}}. \tag{45.34}$$

where the conditional distribution is implicitly defined via the VAE's generative model. Since this distribution is generally intractable, it is typically approximated using an iterative scheme. In particular, noting that

$$\hat{x}_i^{\text{mis}} = \mathbb{E}\big[x_i^{\text{mis}} \mid x_i^{\text{obs}}\big] = \mathbb{E}\left[\mathbb{E}\big[x_i^{\text{mis}} \mid z_i, x_i^{\text{obs}}\big] \mid x_i^{\text{obs}}\right], \tag{45.35}$$

Qiu et al. [1352] iterate between an **encoding step**, which computes the expectation of z_i (mean of the encoder) given $\hat{x}_i^{\text{mis}}$ and x_i^{obs}:

$$\hat{z}_i = \mathbb{E}_{q_\phi(z_i|\hat{x}_i)}[z_i] = \int z_i q_\phi(z_i \mid \hat{x}_i^{\text{mis}}, x_i^{\text{obs}})\, dz_i, \tag{45.36}$$

and a **decoding step**, which updates the imputed values via the decoder by computing the expectation of x_i^{mis} (mean of the decoder) given $\hat{z}_i$:

$$\hat{x}_i^{\text{mis}} = \mathbb{E}_{p_\theta(x_i^{\text{mis}}|x_i^{\text{obs}},\hat{z}_i)}\big[x_i^{\text{mis}}\big] = \int x_i^{\text{mis}} p_\theta(x_i^{\text{mis}} \mid x_i^{\text{obs}}, \hat{z}_i)\, dx_i^{\text{mis}}. \tag{45.37}$$

These steps are repeated until convergence of the imputed values. Note that when the likelihood used in the decoder factorises across input dimensions (e.g., in a factorised Gaussian model), the conditional distribution $p_\theta(x_i^{\text{mis}} \mid x_i^{\text{obs}}, z_i)$ simplifies to $p_\theta(x_i^{\text{mis}} \mid z_i)$, further reducing the computational burden.

This procedure yields a single imputed value for each missing entry, hence the term single imputation. While effective and relatively straightforward, it does not account for uncertainty in the imputed values —a limitation addressed by multiple imputation strategies.

Multiple imputation

Multiple imputation [1057, 1215, 1519] explicitly incorporates the uncertainty of the missing data mechanism. The fundamental idea behind multiple imputation is to generate several completed datasets, each of which represents a different draw from the conditional distribution of the missing data given the observed data. These datasets are then analysed independently and final estimates are obtained by combining inference across all plausible datasets, for example by computing an overall mean and variance estimate for a given statistic. This approach allows the resulting inferences to reflect not only the uncertainty due to sampling variability, but also the additional uncertainty introduced by the imputation of missing values.

This methodology assumes that the data are MAR, where the probability of a value being missing depends only on the observed variables. Under this assumption, the goal is to obtain and sample from the predictive distribution of the missing data given the observed data, $p_\theta(x^{\text{mis}} \mid x^{\text{obs}})$, which takes the form:

$$p_\theta(x^{\text{mis}} \mid x^{\text{obs}}) = \int p_\theta(x^{\text{mis}}, z \mid x^{\text{obs}})\, dz = \int p_\theta(x^{\text{mis}} \mid x^{\text{obs}}, z) p_\theta(z \mid x^{\text{obs}})\, dz. \tag{45.38}$$

This expression highlights two sources of uncertainty: the variability in the latent parameters z, and the conditional variability in the missing values given z and the observed data.

While intractability of the posterior of the latent variables renders direct sampling infeasible, various approaches have been developed for approximate or asymptotically exact sampling, such as pseudo-Gibbs [1391] or Metropolis-within-Gibbs [1146]. Here, we focus on sample importance resampling [1147], and refer to Roskams-Hieter et al. [1422] for a description and comparison of these different sampling approaches. In sampling importance resampling (SIR), the variational posterior (encoder) is used as the importance distribution. Specifically, for every data point i with missing features, samples of the latent variables $z_i^{(s)}$, for $s = 1, \ldots, S$, are drawn from

$$z_i^{(s)} \sim q_\phi(z_i \mid x_i^{\text{obs}}, x_i^{\text{mis}\,(0)}), \tag{45.39}$$

where $x_i^{\text{mis}\,(0)}$ denotes an initial imputation (e.g. mean). The importance weights are proportional to

$$\omega_i^{(s)} \propto \frac{p_\theta(x_i^{\text{obs}} \mid z_i^{(s)}) p(z_i^{(s)})}{q_\phi(z_i^{(s)} \mid x_i^{\text{obs}}, x_i^{\text{mis}\,(0)})}. \tag{45.40}$$

Multiple imputations are obtained by first sampling the latent variables $z_i^{(k)}$, for $k = 1, \ldots, K$, with replacement from the importance samples $z_i^{(s)}$, for $s = 1, \ldots, S$, with weights $\omega_i^{(s)}$. Next, for each $z_i^{(k)}$, the missing data is imputed by sampling from:

$$x_i^{\text{mis}\,(k)} \sim p_\theta(x_i^{\text{mis}} \mid z_i^{(k)}, x_i^{\text{obs}}). \tag{45.41}$$

In contrast to Gibbs, an advantage of SIR is parallelisability. To mitigate degeneracy of the weights, a heavy tailed Student-t proposal is recommended, with mean and variance given by the encoder.

It is worth noticing that the Multiple SIR-based imputations mainly capture predictive uncertainty conditional on the fitted VAE parameters. They do not capture full parameter uncertainty unless one adds posterior sampling, bootstrap, or a related approximation.

45.4.1 Multiple imputation with beta-VAEs

Although expressive and powerful, deep models have been shown to be overconfident even when predictions are incorrect [1582] and also underestimate the variability of out-of-sample test data [1235]. In the context of multiple imputation, this leads to underestimation of the uncertainty in missing data [1422]. To combat this, β-VAEs have been employed, which make use of the power likelihood to avoid overfitting and achieve well-calibrated uncertainty of the missing data.

For multiple imputation with a β-VAE, the predictive distribution (in Equation 45.38) is constructed from the power likelihood, that is, the likelihood of the generative model is raised to the power $1/\beta$ (see Equation (45.33)):

$$p_{\theta,\beta}(x^{\text{mis}} \mid x^{\text{obs}}) \propto \int p_\theta(x^{\text{mis}} \mid x^{\text{obs}}, z)^{1/\beta} p_{\theta,\beta}(z \mid x^{\text{obs}})\, dz. \tag{45.42}$$

It can be noticed that in the case of the factored Gaussian generative model, the power likelihood $p_\theta(x^{\text{mis}} \mid x^{\text{obs}}, z)^{1/\beta}$ is simply proportional to a Gaussian with variance rescaled by a factor of β. On the other hand, $p_{\theta,\beta}(z \mid x^{\text{obs}})$ represents the intractable true posterior of the latent variables under the power likelihood given the observed data only, and to sample from it, the SIR algorithm can be adjusted accordingly.

45.4.2 Tuning beta

The β parameter in a β-VAE plays a crucial role in controlling the strength of the regularisation imposed by the KL divergence term, thereby influencing the trade-off between accurate imputation and uncertainty quantification in the missing values. Choosing an appropriate value for β is therefore essential. In principle, many approaches could be used to tune β, depending on the modelling goals and constraints. Below we discuss two strategies: one based on evaluating predictive performance, and another grounded in Bayesian model selection. In both cases, a subset of the observed data is masked, in order to provide ground truth values of the missing data. The β-VAE is trained on this subset of data, for a range of different choices of β; the strategies below are used to select β among these choices, and a β-VAE is subsequently retrained on the original data with the selected β.

Tuning via predictive performance

A common approach is to evaluate model performance using metrics such as the mean absolute error (MAE) to assess imputation accuracy and empirical coverage (EC) to quantify uncertainty in the missing values. The **mean absolute error** measures the average magnitude of the errors between the imputed values and the truth (for the subset of masked data). Formally, it is defined as:

$$\text{MAE} = \frac{1}{N^{\text{mis}}} \sum_{i=1}^{N^{\text{mis}}} \|\hat{x}_i^{\text{mis}} - x_i^{\text{mis}}\|_1 \ , \tag{45.43}$$

where $\|\cdot\|_1$ represents the L_1 distance and N^{mis} is the total number of masked values. Note that in multiple imputation, imputed values $\hat{x}_i^{\text{mis}}$ are obtained as an average across the K imputed datasets:

$$\hat{x}_i^{\text{mis}} = \frac{1}{K} \sum_{k=1}^{K} x_i^{\text{mis}\,(k)} \ . \tag{45.44}$$

Empirical coverage evaluates the proportion of true values that fall within a $100(1-\alpha)\%$ confidence interval (CI). For instance, given a significance level α, the pointwise CIs, denoted by $[l_i, u_i]$, containing the lower and upper bounds for the missing values of the ith point, are computed from quantiles of the K imputed values. Then, the EC is computed as:

$$\text{EC}(\alpha) = \frac{1}{N^{\text{mis}}} \sum_{i=1}^{N^{\text{mis}}} \sum_{j=1}^{D} m_{ij} \mathbb{1}(x_{ij} \in [l_{ij}, u_{ij}]) \ . \tag{45.45}$$

The goal is to select the β that best balances these two objectives: minimising MAE while maximising EC. For this reason, instead of looking separately at the MAE and EC, one may also want to consider a trade-off measure such as:

$$\text{MAE} + \gamma \max(0, 1 - \alpha - \text{EC}(\alpha)) \ , \tag{45.46}$$

that needs to be minimised (where $\gamma > 0$). The metric introduced in Equation (45.46) can be interpreted as a penalized version of the MAE where the hyperparameter γ controls how strongly we penalize models with too small empirical coverage. This provides a convenient way to summarize the accuracy-coverage trade-off in a single criterion, although it comes at the cost of having to set γ.

Tuning via marginal likelihood

A more principled, fully Bayesian alternative is to select β by maximising the marginal likelihood of the missing data given the observed data, that is

$$p_{\theta,\beta}(x^{\text{mis}} \mid x^{\text{obs}}) = \prod_{i=1}^{N^{\text{mis}}} p_{\theta,\beta}(x_i^{\text{mis}} \mid x_i^{\text{obs}}) \ . \tag{45.47}$$

For each data point, an unbiased approximation of the quantity $p_{\theta,\beta}(x_i^{\text{mis}} \mid x_i^{\text{obs}})$ is obtained from the importance sampler as:

$$\widehat{p_{\theta,\beta}}(x_i^{\text{mis}} \mid x_i^{\text{obs}}) = \sum_{s=1}^{S} \omega_i^{(s)} p_{\theta,\beta}(x_i^{\text{mis}} \mid x_i^{\text{obs}}, z_i^{(s)}) \ . \tag{45.48}$$

45.5 Biomedical application

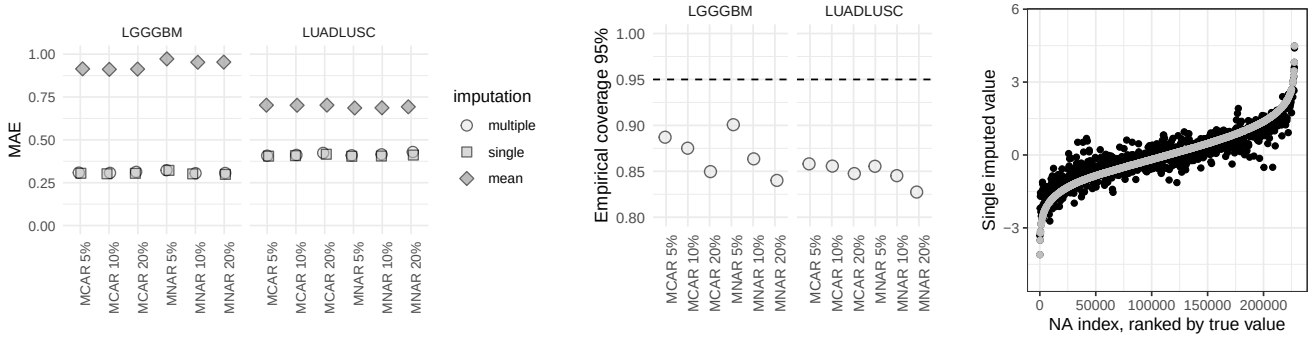
This section illustrates how VAEs and their generalisations (β-VAEs) can be applied to missing data problems in biomedical settings. We focus on evaluating both the quality of the imputations and their impact on downstream analysis tasks. The experiments are conducted on real genomic data, where missingness is introduced under various controlled scenarios. We begin by describing the dataset and the missingness mechanisms. Next, the core results are presented, highlighting the differences between standard VAEs and β-VAEs in terms of uncertainty quantification and the impact on downstream tasks.

45.5.1 Dataset and missingness scenarios

We focus on high-dimensional transcriptomic cancer datasets from the Cancer Genome Atlas (TCGA) that quantify gene expression across 17,175 genes and hundreds of patient samples. Each observation represents a patient who has been diagnosed with either glioblastoma (GBM; $N = 159$), lower-grade glioblastome (LGG; $N = 508$), lung adenocarcinoma (LUAD; $N = 478$) or lung squamous carcinoma (LUSC; $N = 475$). Missing data are introduced under different scenarios, including MCAR and MNAR, and imputation performance is evaluated by assessing the accuracy of imputation (MAE), coverage (EC), and false positive rates (FPR) of downstream tasks. The MCAR setting was generated by randomly masking 5%, 10% and 20% of the values in 20% of the observations across both pairs of cancer types, paired by tissue type (LGG/GBM and LUAD/LUSC). The MNAR setting was designed to mimic a real MNAR setting that occurs in transcriptomic datasets, whereby genes that have high GC content are more likely to experience dropout during sequencing [1352]. In this case, half of the genes with the top 5%, 10% and 20% highest GC content were randomly masked in 20% of the observations in the original transcriptomic datasets to mimic this scenario. This procedure generated 6 missing scenarios across two types (MCAR and MNAR) and three levels (5%, 10% and 20%) for each paired dataset, LGG/GBM and LUAD/LUSC.

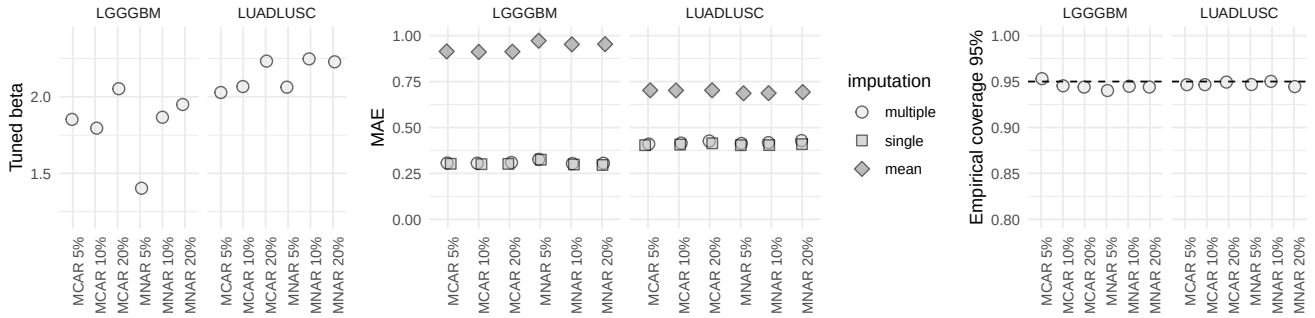
45.5.2 Experiment design

After simulating missingness as described above, the datasets are scaled and the VAEs are subsequently trained using zero imputation at missing value indices. The same model and hyper-parameters described in [1422] are adopted, specifically 250 training epochs, a learning rate of 10^{-5}, two hidden layers of dimension 600, 2000, and a latent space dimension of 200. The grid of β values investigated ranges from 0.4 to 3, and β is tuned using the metric defined in Equation (45.46) with $\gamma = 1$. Once the models are trained, they are used to impute $K = 100$ datasets. Code to reproduce all the experiments is available at `https://github.com/roskamsh/BDL-experiments`.



(a) Comparison of MAE (b) Comparison of EC (c) Single imputed values: LGG/GBM, MCAR 10%

Figure 45.2: Standard VAE ($\beta = 1$) underestimates the uncertainty in multiple imputation.



(a) Tuned values of β (b) Comparison of MAE (c) Comparison of EC

Figure 45.3: Multiple imputation with β-VAEs. (a) Selected β values tend to increase with the proportion of missingness, (b) MAE results are comparable, yet (c) EC improves significantly over standard VAEs.

45.5.3 Experimental results

This section illustrates how VAEs and β-VAEs can be used for uncertainty-aware imputation of missing data. Three key observations emerge from the experiments.

Overconfident imputations in standard VAEs

Standard VAEs tend to produce overconfident imputations when used for multiple imputation. The variability derived by sampling from the posterior over latent space is often insufficient. This results in EC–the proportion of masked values falling within the nominal 95% CIs–well below nominal confidence levels, especially for more extreme or outlying values (see Figure 45.2c, which depicts true values (gray) against true ranked indices). This behaviour is illustrated in Figure 45.2, where MAE is comparable between single and multiple imputations (Figure 45.2a), yet EC is too low (Figure 45.2b), indicating overconfidence, especially as the level of missingness increases.

Improved uncertainty quantification via beta-VAEs

The introduction of β-VAEs improves the quantification of uncertainty in the imputations. Introducing a tunable β parameter in the ELBO formulation enables control over the trade-off between imputation accuracy and uncertainty. By increasing the weight on the KL divergence term in the ELBO, these models encourage posterior distributions that are closer to the prior, resulting in more dispersed latent representations. Figure 45.3 shows that the imputation accuracy (measured by MAE) is similar to standard VAEs (Figure 45.3b), yet β-VAEs achieve coverage levels close to the target 95% (Figure 45.3c), correcting the underestimation observed in standard VAEs. It is worth noting that the optimal β values (Figure 45.3a) tend to increase as the proportion of missing data grows, suggesting that stronger regularisation and greater prior influence is needed under higher uncertainty.

Effects on downstream tasks

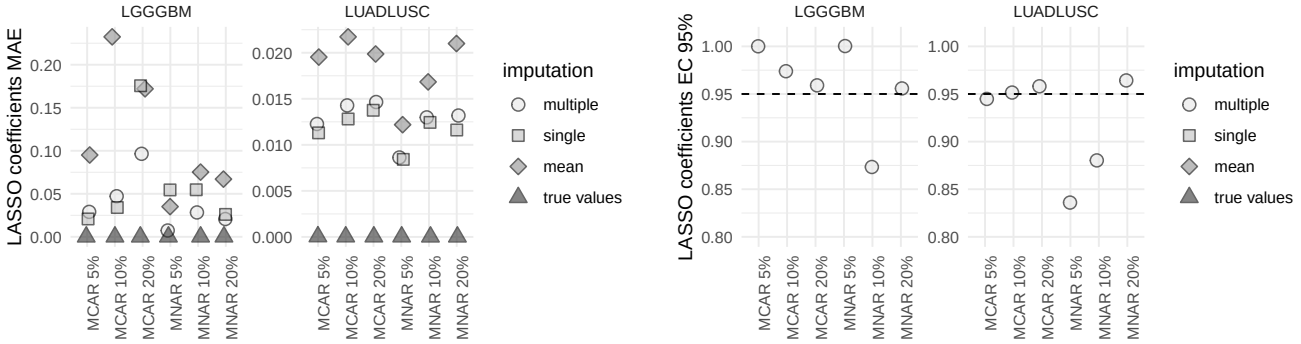
The quality of imputation —both in terms of accuracy and uncertainty— has a substantial impact on downstream tasks. To illustrate this, a logistic regression model with LASSO penalty was used to identify gene sets discriminating between cancer types. The model was fit in four data settings: the fully observed ground truth data with no missingness, the single dataset with mean imputation, the single dataset with standard VAE imputation, and each of the 100 datasets generated via multiple imputations using the tuned β-VAEs.

Model coefficients were compared in terms of MAE against the ground truth estimates. As shown in Figure 45.4, naive strategies, such as mean imputation, result in poor performance with high MAE and hence unreliable coefficient estimates. The flexibility afforded by VAEs helps to combat this, and multiple imputation with β-VAEs achieves slightly better performances compared to standard VAEs, in certain cases such as in MCAR contexts with high missingness (20%). Additionally, multiple imputation allows construction of CIs that reflect variability in the coefficients due to the missing data; the 95% EC of the coefficients lies close to the desired level of 0.95.

A key goal in biomedical inference is the identification of stable and interpretable features, such as genes associated with cancer subtypes. In this context, the sparse coefficient estimates obtained through the LASSO penalty identify discriminating genes. For multiple imputations, a gene is selected if its inclusion probability– defined as the proportion of imputations for which its LASSO coefficient is non-zero–exceeds a given threshold. False positive rates (FPR) and true positive rates (TPR) are computed according to the discriminating gene sets identified in the "true" complete dataset (Figure 45.5). Naive mean imputation yields inflated FPRs, low TPRs, and unreliable selection results, consistent with its poor performance in coefficient estimation (Figure 45.4). Again, VAEs allow for improved gene selection, with multiple imputation leading to lower false discovery of genes in various cases (notably MNAR settings for LGG/GBM in Figure 45.5a). Hence, multiple imputation with β-VAEs not only provides well-calibrated uncertainty in imputations but also results in more acceptable FPRs in downstream gene discovery.

45.6 Discussion

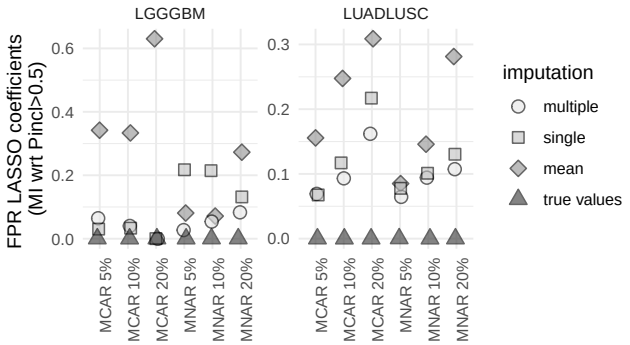
Deep latent variable models offer an attractive framework to capture complex patterns in high-dimensional data. In biomedicine, where large numbers of variables are typically recorded and missingness is commonplace, DLVMs are useful to deal with missing data through imputation. Compared to simple strategies, such as mean imputation, they offer better imputation accuracy, leading to substantial improvements in downstream analysis, including coefficient estimation and variable selection. Moreover, their generative nature allows multiple imputations to account for uncertainty in missing values. This helps to improve robustness in downstream tasks, such as lower false discovery of discriminating genes.



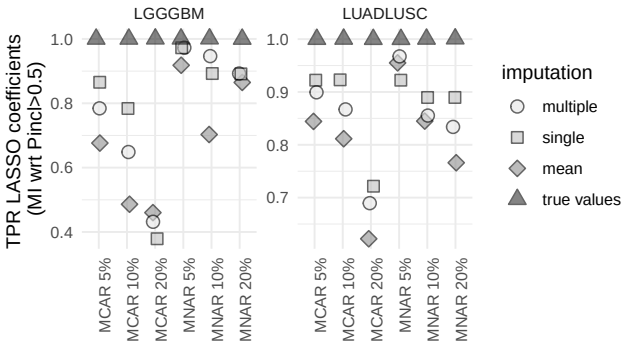
(a) MAE of LASSO coefficients

(b) EC of LASSO coefficients

Figure 45.4: LASSO coefficients compared between ground truth and imputed datasets. Multiple imputation tends to improve accuracy and provide good empirical coverage.



(a) False Positive Rate



(b) True Positive Rate

Figure 45.5: FPR and TPR in gene selection. Uncertainty-aware imputation reduces FPRs, particularly under MNAR. At the same time, TPR is similar, with some improvements or reductions in different settings.

Chapter 46

Deep Bayesian spatio-temporal modelling of wildfires

Wildfires present an ever-present threat to human life, property, health, and the ecosystem. Wildfires are increasing in size and intensity due to climate change, increase in fuel loads, changing land management practices, and an expanding wildland-urban interface. Indeed, in recent years we have seen examples of unusually destructive wildfires across the world, such as those in Australia (2023-24), Canada (2023), Europe (2023), South America (2024), and Southern California (2025). There is a great need to model various aspects of wildfire to learn about and mitigate risk, health impacts, and spread after ignition. This can be challenging because wildfires are complex, representing interacting spatio-temporal processes across many scales and drivers. Such complex processes require models that can accommodate complicated dependence structure and that adequately quantify uncertainty across data, process, and parameters. The deep Bayesian paradigm is ideally suited for these tasks.

46.1 Introduction

Wildfires represent a complex chaotic system driven by extreme weather conditions, variable fuel loads, complex terrain, and physical and biological processes involving feedback across multiple scales in space and time. This, combined with human behaviour (management, mitigation, and accidental or deliberate ignition) and limited data availability, make this one of our most challenging environmental modelling problems.

Since the mid-1990s, statisticians have approached the modelling of complex spatio-temporal dynamical systems via a multi-level Bayesian hierarchical model (BHM) paradigm. In 1996, Berliner [127] described a general hierarchical model for such processes that focused on modelling the process and parameters given observations (i.e., the posterior distribution) as proportional to the product of conditional distributions (data given process and parameters, process given parameters, and prior distributions on parameters). Complex processes require each of these component distributions to be decomposed into sub-models, making the BHM very much a "deep" model (e.g., multiple data sets conditioned on a common latent process, process parameters conditioned on covariates, Gaussian processes, or Markov random fields, etc.; see [1753] and the overview in [345]). One of the strengths of this paradigm is the ability to incorporate prior mechanistic understanding of the process (e.g., motivating the space-time dynamic process via a discretised partial differential equation, incorporating deterministic model output as data, or informing prior distributions from deterministic model output; see [1008, 1750, 1754]).

In recent years, there has been more interest in the development of hybrid statistical/deep neural models that can incorporate the expressiveness of deep neural models, yet can also include dependent random effects, measurement uncertainty, prior information, and missing data in a more formal context than in traditional deep neural models (see the discussion in [1751, 1752]). Examples include the use of convolutional neural networks to learn the transition operator in an integro-difference equation spatio-temporal dynamic model [1830], the incorporation of a deep echo state network within a mechanistic-informed process model [1818], and the incorporation of a novel statistical model for spatio-temporal extremes within a variational autoencoder [1850, 1851]. We present these latter two examples in this chapter. These hybrid models (embedding a deep neural model in a statistical model, or embedding a statistical model in a deep learning framework) leverage the flexibility and generalisability of the deep neural framework and the formal uncertainty quantification components of Bayesian statistical modelling. These approaches tend to work well when the overall framework is a BHM, while incorporating the spirit of deep neural uncertainty quantification that is the foundation of Bayesian deep learning as exemplified in this book.

Bayesian models for probabilistic estimation of fire risk started becoming prevalent in the literature in the early 2000s [207, 1762] and Bayesian networks started being used for evaluating fire risk in the early 2010s [431]. Around the same time, Bayesian methods combined with extreme value theory were used for fire size prediction [1163]. Extreme value theory has remained important for modelling fire risk [853, 910, 983, 1326] and served as a data challenge for the Extreme Value Analysis conference in 2021 [1263]. More recently, there has been a growing interest in modelling wildfire spread with Bayesian methods, some examples of which also considered deep neural models [201, 669, 670, 1568, 1819].

In the next two sections we focus on two examples, one that considers a specialized variational autoencoder to model dependent extremes in spatio-temporal data for a fire weather index, and the other that considers a data-driven level set/deep echo state network model to forecast the spread of a fire front.

46.2 Application: Analysis of Extremes in Fire Weather Index with an Extremes-Aware Variational Autoencoder

Environmental extremes such as wildfires typically appear in clusters in space and time. Traditionally, it has been difficult to model such dependent extremes in high dimensions. Gaussian processes, which are quite useful for modelling the bulk of many environmental spatial and spatio-temporal processes (either directly or as a latent process) are not appropriate given that the dependence in the tails between any two locations, no matter how close, are asymptotically independent (the extremal dependence is guaranteed to go to zero as the quantile increases). More classic distributions for extremes (e.g., generalized extreme value distributions) have been quite useful for looking at return times of extreme events for one location in space, but are more difficult to model joint dependence. Over the last two decades, innovative models for dependent extremes have been constructed, but are often not practical for flexible types of dependence (e.g., max-stable processes only allow asymptotic dependence no matter how far apart). In addition, these approaches have proven to be difficult to scale to very large numbers of spatial locations; see [805] for an overview of recent developments in spatial extremes.

A flexible spatio-temporal extremes model for environmental processes should allow: (a) close locations to have stronger dependence than locations far apart; (b) dependence strength to decay as "extremeness" intensifies; (c) dependence structure to be heterogeneous (non-stationary) across large domains (e.g., continental North America); and (d) both long-range asymptotic independence in the tails and short-range asymptotic independence or dependence to be present. Based on these criteria, [1851] extended the so-called "max-ID" extremes model of [188, 1386] to give a model that exhibits flexible tail characteristics across the domain and does not impose a dependence class *a priori*. As described below in Section 46.2.1, this model is represented as a Bayesian hierarchical model, but its calculation by traditional Bayesian computation approaches (Markov chain Monte Carlo or Hamiltonian Monte Carlo) is prohibitive on large spatial domains. Thus, [1851] embed their novel extremes model within a variational autoencoder, as described below in Section 46.2.2. This model has been used for uncertainty quantification over regions of extreme sea surface temperature in the Red Sea that can lead to coral bleaching [1851] as well as emulating the turbulent plume associated with a computational intensive fluid dynamics model of wildfire [1850].

46.2.1 Flexible Max-ID Extremes Model

As in [1851], we factor our spatio-temporal data of interest as

$$X_t(\xi) = \epsilon_t(\xi) H_t(\xi),\ \xi \in \mathcal{S}, \tag{46.1}$$

where $t = 1, \ldots, T$ indexes time, $\mathcal{S} \subset \mathbb{R}^2$ is the spatial domain of interest and $\epsilon_t(\xi)$ is a noise process with independent Fréchet$(0, \tau, 1/\alpha_0)$ marginal distributions; that is, $\Pr\{\epsilon_t(\xi) \leq x\} = \exp\{-(x/\tau)^{-1/\alpha_0}\}$, where $x > 0$, $\tau > 0$ and $\alpha_0 > 0$. Then, $H_t(\xi)$ is constructed using a low-rank representation:

$$H_t(\xi) = \left\{ \sum_{k=1}^{K} \omega_k(\xi)^{1/\alpha} Z_{kt} \right\}^{\alpha_0}, \tag{46.2}$$

where $\alpha \in (0, 1)$, $\{\omega_k(\xi) : k = 1, \ldots, K\}$ are fixed compactly-supported radial basis functions centred at K prespecified knots such that $\sum_{k=1}^{K} \omega_k(\xi) = 1$ for any $\xi \in \mathcal{S}$, and $\mathbf{Z}_t = \{Z_{kt} : k = 1, \ldots, K\}$ are independently distributed as exponentially-tilted positive-stable (expPS) random variables, whose densities are of the form

$$h(x; \alpha, \gamma_k) = \frac{f_\alpha(x) \exp(-\gamma_k x)}{\exp(-\gamma_k^\alpha)},\ x > 0,\ k = 1, \ldots, K. \tag{46.3}$$

Here, f_α is the density function of a positive-stable variable defined through its Laplace transform $\int_{\mathbb{R}^+} \exp(-rx) f_\alpha(x)\mathrm{d}x = \exp(-r^\alpha), r \geq 0$, $\alpha \in (0,1)$ determines the rate at which the power-law tail of f_α tapers off, and the tilting parameters $\gamma_k \geq 0$ determine the extent of tilting, with larger values of γ_k leading to lighter-tailed Z_{kt}.

Given a finite set of observation locations $\{\xi_1, \ldots, \xi_m\}$, we define the discretised spatio-temporal process as $\mathbf{X}_t = \{X_t(\xi_i) : i = 1, \ldots, m\}$. To implement the Bayesian hierarchical model, we aim to estimate both the model parameters $\boldsymbol{\phi} = (\alpha, \tau, \alpha_0, \gamma_1, \ldots, \gamma_K)^\top$ and the latent process given by $\{\mathbf{Z}_t : t = 1, \ldots, T\}$. Full Bayesian inference (e.g., via Markov chain Monte Carlo) becomes computationally prohibitive due to the high dimensionality of the parameter space and the latent process.

46.2.2 Extremes-Aware Variational Autoencoder (xVAE)

A variational autoencoder (VAE) is a type of generative model that learns to represent data in a lower-dimensional latent space through an encoder (inference network) and generate new data similar to what it has seen during training through a decoder (generative network) [892]. Utilizing the hierarchical construct of our model described in (46.1)–(46.2), we use the encoder to learn to map input data $\mathbf{X}_t$ to a distribution over latent variables $\mathbf{Z}_t$ in encoded space:

$$\begin{aligned} q_{\boldsymbol{\theta}_e}(\mathbf{z}_t \mid \mathbf{x}_t) &= \mathcal{N}(\boldsymbol{\mu}_t, \mathrm{diag}(\boldsymbol{\sigma}_t^2)), \\ (\boldsymbol{\mu}_t^\top, \log \boldsymbol{\sigma}_t^\top)^\top &= \mathrm{EncoderNeuralNet}_{\boldsymbol{\theta}_e}(\mathbf{x}_t), \end{aligned} \tag{46.4}$$

where $\boldsymbol{\theta}_e$ are the bias and weight parameters of the encoder neural network. The decoder then samples from $q_{\boldsymbol{\theta}_e}$ and learns to map sampled latent variables $\mathbf{z}_t$ back to reconstructed data using the distribution $p_{\boldsymbol{\phi}}(\mathbf{x}_t \mid \mathbf{z}_t)$ defined by (46.1)–(46.2). Critically, the model parameters $\boldsymbol{\phi} = (\alpha, \tau, \alpha_0, \gamma_1, \ldots, \gamma_K)^\top$ are estimated by the decoder network:

$$\hat{\boldsymbol{\phi}} = \mathrm{DecoderNeuralNet}_{\boldsymbol{\theta}_d}(\mathbf{z}_1, \ldots, \mathbf{z}_T), \tag{46.5}$$

where $\boldsymbol{\theta}_d$ are the bias and weight parameters of the decoder neural network, and positivity of the parameter estimates is enforced through appropriate output-layer transformation. Thus, the decoder likelihood can equivalently be written as

$$p_{\boldsymbol{\theta}_d}(\mathbf{x}_t \mid \mathbf{z}_t) \equiv p_{\hat{\boldsymbol{\phi}}}(\mathbf{x}_t \mid \mathbf{z}_t),$$

emphasizing that the dependence on $\boldsymbol{\theta}_d$ enters through the decoder-generated distributional parameters $\hat{\boldsymbol{\phi}}$.

Both the encoder and decoder are multilayer perceptron (MLP) neural networks whose weights and biases are updated iteratively to maximize the differentiable Evidence Lower Bound (ELBO) which is given by

$$\mathcal{L}(\boldsymbol{\theta}_e, \boldsymbol{\theta}_d) = \sum_{t=1}^{T} \mathbb{E}_{q_{\boldsymbol{\theta}_e}(\mathbf{z}_t \mid \mathbf{x}_t)} \left[\log p_{\boldsymbol{\theta}_d}(\mathbf{x}_t \mid \mathbf{z}_t)\right] - \sum_{t=1}^{T} D_{\mathrm{KL}} \left[q_{\boldsymbol{\theta}_e}(\mathbf{z}_t \mid \mathbf{x}_t) \,\|\, p_{\boldsymbol{\theta}_d}(\mathbf{z}_t)\right], \tag{46.6}$$

where $p_{\boldsymbol{\theta}_d}(\mathbf{z}_t)$ is the prior on $\mathbf{z}_t$:

$$p_{\boldsymbol{\theta}_d}(\mathbf{z}_t) \equiv p_{\hat{\boldsymbol{\phi}}}(\mathbf{z}_t) = \prod_{k=1}^{K} h(z_{kt}; \hat{\alpha}, \hat{\gamma}_k),$$

where the model parameters $\hat{\boldsymbol{\phi}}$ are learned through the decoder network described in (46.5), which does not introduce additional latent variables; rather, it provides an amortized neural estimator of the dependence parameters. This corresponds to an empirical-Bayes estimation strategy for the max-ID model.

Equivalently, the ELBO can be written as

$$\mathcal{L}(\boldsymbol{\theta}_e, \boldsymbol{\theta}_d) = \sum_{t=1}^{T} \mathbb{E}_{q_{\boldsymbol{\theta}_e}(\mathbf{z}_t \mid \mathbf{x}_t)} \left[\log p_{\boldsymbol{\theta}_d}(\mathbf{x}_t, \mathbf{z}_t) - \log q_{\boldsymbol{\theta}_e}(\mathbf{z}_t \mid \mathbf{x}_t)\right],$$

which is a variational lower bound on the marginal log-likelihood $\sum_{t=1}^{T} \log p_{\boldsymbol{\theta}_d}(\mathbf{x}_t)$. The first term in (46.6) is the expected reconstruction log-likelihood and encourages the latent representation to retain information needed to reconstruct the data. The second term regularizes the approximate posterior so that it remains close to the prior distribution in the latent space.

Since the latent variables have distributions that enable a generative process, we can obtain many copies of the original fields $\{\mathbf{X}_t : t = 1, \ldots, T\}$ that have similar properties, especially in the joint tail. Moreover, the decoder $p_{\hat{\boldsymbol{\phi}}}(\cdot \mid \mathbf{z}_t)$, given a sampled latent variable vector $\mathbf{z}_t$ from the encoder, can utilize the basis function values

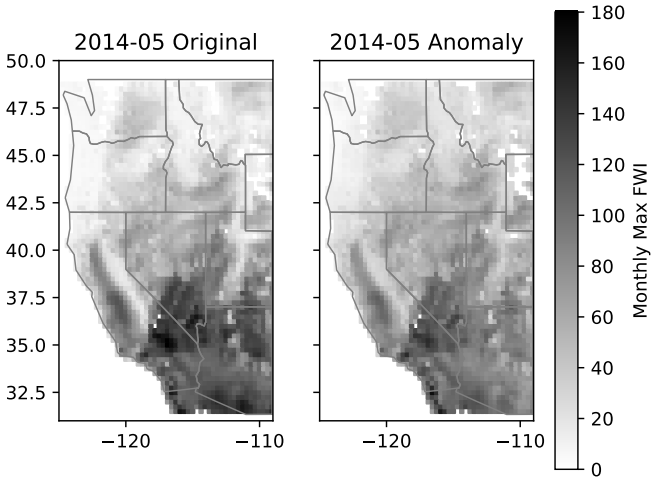


Figure 46.1: May 2014 monthly maxima fire weather index (FWI) values in the western US and its corresponding de-seasonalised anomalies

at the unobserved locations and therefore it naturally produces predictions at unobserved locations using the construction of the max-ID process. That is, $p_{\hat{\boldsymbol{\phi}}}(\cdot \mid \mathbf{z}_t)$ is a predictive distribution at observed and unobserved locations, for each draw of $\mathbf{z}_t$. This ability to perform spatial predictions is crucial in producing spatially-complete data products and giving sensible risk assessments with proper uncertainty.

Lastly, we note that we use a heteroskedastic Gaussian variational family $q_{\boldsymbol{\theta}_e}(\mathbf{z}_t \mid \mathbf{x}_t)$ for computational tractability, although the latent prior $p_{\boldsymbol{\theta}_d}(\mathbf{z}_t)$ is specified through the expPS distribution. This Gaussian distribution should be interpreted only as an approximation to the posterior $p_{\boldsymbol{\theta}_d}(\mathbf{z}_t \mid \mathbf{x}_t)$, rather than as a Gaussian replacement for the expPS prior. The heavy-tailed behavior of the model is retained through the expPS prior and the max-ID decoder. As explained in Appendix C.1 of [1851], this choice is also practically motivated: heteroskedastic Gaussian variational approximations are widely used in variational Bayes, and posterior distributions are often much more concentrated than their corresponding priors (due to the Bernstein–von Mises Theorem). Moreover, our compactly supported basis functions induce a localized dependence structure, which gives heuristic support for a Gaussian posterior approximation under spatial mixing-type arguments. In our experiments, heavier-tailed variational families were less stable and did not improve emulation performance, presumably because the expPS prior already provides the required tail flexibility, while an overly diffuse variational distribution can make the latent encoding less precise.

46.2.3 Application to the Fire Weather Index

The Fire Weather Index (FWI) is commonly used to estimate fire danger based on meteorological conditions, helping to anticipate and manage wildfire risks. A global FWI data product with a spatial resolution of $0.3125° \times 0.25°$ is available from NASA's NCCS Data Portal at https://portal.nccs.nasa.gov/datashare/GlobalFWI/. Various remote sensing and *in situ* sources are integrated with fire weather variables such as temperature, wind speed, and humidity to produce the FWI as in [1690]. Note, conditions at some locations may not satisfy the assumptions required to compute the FWI and so some data are missing at each time. We consider monthly maxima at each location, which limits the amount of missing data (e.g., see Figure 46.1 for May 2014 monthly maxima in the 2,887 grid cells over the interior western U.S. region of interest here). For the analysis presented, we consider FWI monthly maxima for the period May 2014 through December 2024 (128 months), where any location with more than 50% NA records was discarded from subsequent analysis.

Data preprocessing

The extremes model used here assumes temporal independence and stationarity, and so seasonality was removed independently at each grid cell by fitting a 12-month cyclic cubic-spline generalized-additive model at each location and subtracting that from the monthly maxima at each FWI grid location to obtain FWI anomalies (see the right image in Figure 46.1 for an example of such anomalies).

In addition, for each anomaly series at a grid location ξ_i, we denote $\{\text{FWI}^{\text{a}}_{it} : t = 1, \ldots, T\}$. If at least three

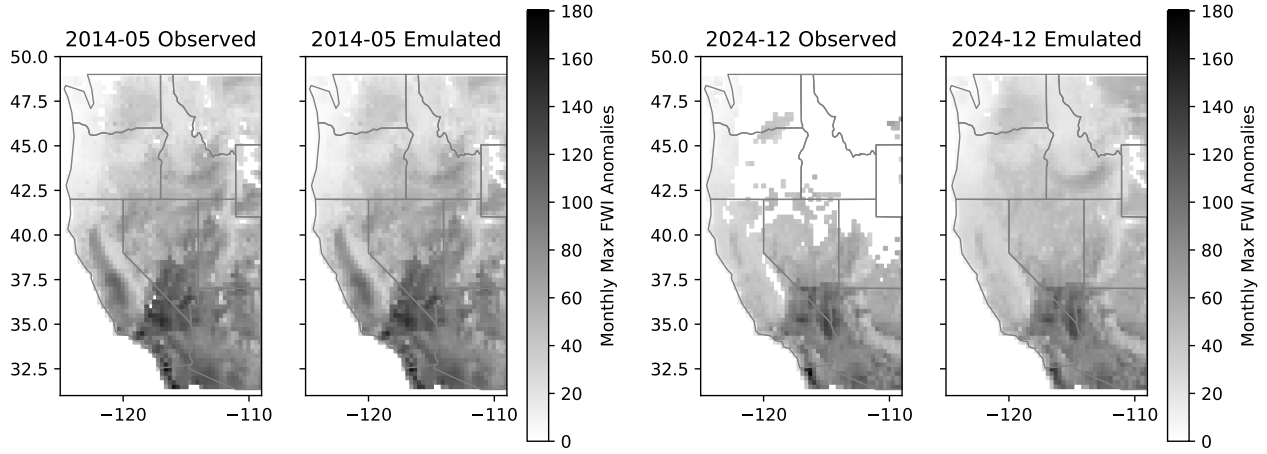


Figure 46.2: Monthly maxima FWI anomalies and corresponding emulated sample in May 2014 (left plots) and December 2024 (right plots).

non-missing values were available, a generalized extreme value (GEV) distribution was fit with the μ_i (location), τ_i (scale), and ζ_i (shape) parameters estimated by maximum likelihood and each anomaly series was monotonically transformed via

$$X_{it} = |\zeta_i(\beta_i - \mathrm{FWI}^{\mathrm{a}}_{it})/\tau_i|^{1/\zeta_i}\ , t = 1, \ldots, T,\ i = 1, \ldots, m, \tag{46.7}$$

so that the X-data at each location is Pareto-tailed. Here, $\beta_i = \mu_i - \tau_i/\zeta_i$ is the upper bound of the marginal GEV distribution if $\zeta_i < 0$ and it is the lower bound if $\zeta_i > 0$. Therefore, the transformation (46.7) always defines a monotone transform. Once we obtain an emulated copy of $\mathbf{X}_t$ (i.e., a generated sample), we make an inverse transformation to get the emulation of $\{\mathrm{FWI}^{\mathrm{a}}_{it} : t = 1, \ldots, T\}$.

FWI data emulation with xVAE

As described in Section 46.2.2, the xVAE uses a standard encoder–decoder architecture adapted to the max-ID spatial extremes model. The encoder takes the spatial field $\mathbf{x}_t = (x_t(\xi_1), \ldots, x_t(\xi_{n_s}))^\top$ as input and maps it to the parameters of a diagonal Gaussian variational distribution for the latent vector $\mathbf{z}_t$. Specifically, the encoder is implemented as an MLP with nonlinear hidden layers,

$$\mathbf{h}_{1,t} = \mathrm{ReLU}(\mathbf{W}_1\mathbf{x}_t + \mathbf{b}_1), \qquad \mathbf{h}_{2,t} = \mathrm{ReLU}(\mathbf{W}_2\mathbf{h}_{1,t} + \mathbf{b}_2),$$

followed by two output layers that return the posterior mean and log-variance,

$$\log \boldsymbol{\sigma}_t^2 = \mathbf{W}_3\mathbf{h}_{2,t} + \mathbf{b}_3, \qquad \boldsymbol{\mu}_t = \mathrm{ReLU}(\mathbf{W}_4\mathbf{h}_{2,t} + \mathbf{b}_4).$$

A latent draw is then obtained through the reparameterization

$$\mathbf{z}_t = \boldsymbol{\mu}_t + \boldsymbol{\sigma}_t \odot \boldsymbol{\eta}_t, \qquad \boldsymbol{\eta}_t \sim N(\mathbf{0}, \mathbf{I}_K),$$

which enables stochastic-gradient optimization of the ELBO. The decoder maps the sampled latent vector $\mathbf{z}_t$ to the distributional parameters of the max-ID model, most importantly the dependence parameters $(\alpha, \boldsymbol{\gamma}^\top)^\top$. Thus, the encoder learns a low-dimensional latent representation of the spatial field, while the decoder translates this representation into interpretable extremal-dependence parameters used to reconstruct or emulate spatial extreme fields.

Figure 46.2 compares the observed and emulated monthly maximum FWI anomaly fields for May 2014 and December 2024. Grey scales are matched across panels for direct comparison. As introduced in Section 46.2.2, a key advantage of our approach lies in its ability to perform accurate spatial prediction, especially in the presence of missing data. Visual inspection shows strong agreement between the emulated and observed fields, with the model capturing both large-scale patterns and fine-scale variations. Note we deliberately left out locations in the December 2024 observations in Figure 46.2 that had over 50% NA records and were not included in the analysis.

To evaluate the quality of the emulations beyond visual assessment, we quantify tail dependence using explicit statistical measures, as discussed in the next section. These results indicate that the emulator captures both

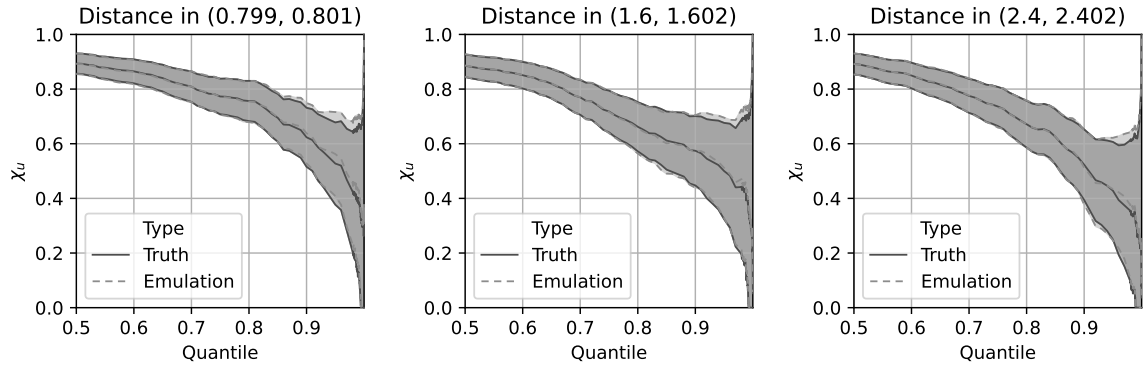


Figure 46.3: Comparison of the empirically-estimated $\chi_h(u)$ at $h = 0.8, 1.601, 2.401$ for 2015 FWI.

bulk variability and extremal dependence effectively. In addition, the framework developed in [1851] allows the dependence parameters $(\gamma_1, \ldots, \gamma_K)^\top$ to vary over time. Temporal changes in these parameters, estimated via the decoder in Equation (46.5), can reveal trends in extremal dependence structure.

46.2.4 Tail–Dependence Validation with Extremal Dependence Measures

In the spatial extremes literature, the extremal dependence between two random variables X_1 and X_2 is commonly quantified using the χ-*measure*, defined as the conditional exceedance probability:

$$\chi_{12}(u) = \Pr\{F_2(X_2) > u \mid F_1(X_1) > u\} = \frac{\Pr\{F_2(X_2) > u, F_1(X_1) > u\}}{\Pr\{F_1(X_1) > u\}}, \tag{46.8}$$

where $u \in (0, 1)$, and F_1, F_2 denote the marginal distribution functions of X_1 and X_2, respectively. This measure captures the likelihood of an extreme event at one location occurring given an extreme event at another. If $\chi_{12} := \lim_{u\to 1} \chi_{12}(u) = 0$, the pair is said to be *asymptotically independent*; if $\chi_{12} > 0$, they are *asymptotically dependent*. This distinction is critical in assessing joint risks of spatially dependent extremes such as floods or heatwaves.

To evaluate whether the xVAE realistically reproduces spatial upper-tail dependence, we assume that $\{X_t(\xi)\}$ exhibits a stationary and isotropic dependence structure at each time point t, so that $\chi_{12}(u) \equiv \chi_h(u)$, where $h = \|\xi_1 - \xi_2\|$ is the distance between locations. For a given lag h, we identify all location pairs whose distances fall within a small tolerance (e.g., $\epsilon = 0.001$), and compute empirical estimates of $\chi_h(u)$ over a grid of high quantiles u, pooling across all 12 monthly replicates. This surely violates our model assumption (our max-ID model is non-stationary over space), but the resulting diagnostic is still informative because it provides a low-dimensional, domain-averaged summary of how upper-tail dependence decays with distance and changes across quantile levels. Since the same summary is computed for both the observed fields and the xVAE emulations, agreement between the two indicates that the emulator preserves the main distance-dependent features of extremal dependence, even if the underlying dependence structure is spatially heterogeneous.

Figure 46.3 presents the estimated $\chi_h(u)$ curves for both the observed data and xVAE emulations across three spatial lags $h = 0.8,\ 1.601,\ 2.401$ for 2015 (with tolerance $\epsilon = 0.01$); other years show similar results. Grey-shaded bands represent 95% confidence intervals for the observation-based estimates. Across all cases, the xVAE reproduces the extremal dependence structure with remarkable fidelity, underscoring its capability to preserve tail dependence across space.

Despite its importance, tail dependence validation is rarely addressed in the emulation literature. While many proper scoring rules exist for assessing predictive performance [639], these typically average over all quantiles and may miss deficiencies in the tail. As highlighted in [1851], specialized rule-based metrics can more effectively assess both bulk and tail dependence. Accurate characterisation of tail behaviour is essential for credible risk assessment and for capturing the spatial footprint of extreme events.

46.2.5 Posterior Inference: Areal Exceedance Probabilities

As briefly commented in Section 46.2.2, one advantage of xVAE is that it can perform spatial predictions and generate a huge number of replicates from the approximate posterior distribution rapidly. Therefore, we can use

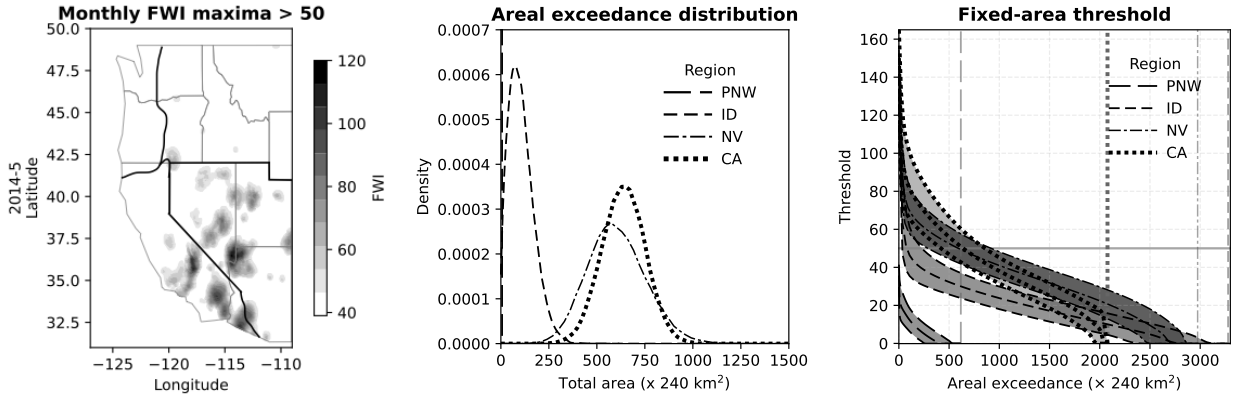


Figure 46.4: The left panels show realisations of monthly FWI maxima emulated with fitted parameters from the xVAE for May 2014. The emulations are censored with a threshold of 50. From 30,000 such emulations, we estimate the distribution of total area exceeding 50 within each region. The western U.S. is divided into four regions according to the ecological regions level I division [1777]: PNW, Idaho region (ID), CA, Nevada region (NV). On the right, we estimate the threshold it takes to have a fixed area of exceedance. The 95% approximate credible intervals are also shown. The vertical dashed lines are total areas of each subregion, and the horizontal slices at 50 yield results that align with the middle panel.

these simulated copies to calculate an important metric for dependence extremes—areal exceedance probabilities—which represents the spatial extent of a region simultaneously at extreme wildfire risk. Using the trained xVAE, we generate 30,000 independent monthly maximum FWI for each time point and compute the total area exceeding 50, which corresponds to extreme fire danger according to operational standards in the Canadian Forest Fire Weather Index system [1565, 1690]. The left panel of Figure 46.4 shows one of the monthly maximum FWI realisations emulated from the xVAE for May 2014 thresholded at FWI > 50. The middle panel shows the probability density functions of total area where FWI exceeds 50 (in units of 240 km^2) within each of four ecological subregions: Pacific Northwest (PNW), Idaho/Montana (ID), California (CA), and Nevada/Arizona/New Mexico (NV), based on the 30,000 sampled fields.

The results reveal substantial regional heterogeneity in the extent of FWI exceedances, with California showing the most persistent large-area exceedances, while the PNW and ID exhibit much lower exceedance volumes. For PNW in particular, the fire risk is consistently low and there is zero areal exceedance of the threshold 50. While the ID subregion has the largest land mass, its areal density concentrates around 70 ($\times$240 km^2), much lower than the CA and NV subregions.

To further analyze joint exceedances at varying extreme levels, the rightmost panels of Figure 46.4 invert the typical fixed-threshold paradigm by asking: *What FWI level corresponds to a given exceedance area*? That is, for each fixed spatial extent, we calculate the minimal threshold needed to reach that area of joint exceedances from each emulated sample, and we then group all 30,000 estimated thresholds together to derive 95% approximate empirical credible intervals. We repeat this process for all spatial extents of exceedances in between 5 to 3,300 ($\times$240) km^2. Note that this can be computed rapidly thanks to the semi-amortized nature of our xVAE. This shows that within each subregion, the spatial extent of exceedances decreases as the threshold increases and extreme events becomes more localized as they get more extreme. Furthermore, as the spatial extent approaches zero, the threshold estimates represent the highest possible FWI for a specific month in a subregion, a valuable metric for wildfire planning.

These results underscore the ability of the xVAE to provide spatially-explicit, probabilistic risk assessments for wildfire danger by capturing both marginal and joint tail behaviour. By leveraging the high-resolution spatial modelling of the xVAE and its ability to generate massive posterior simulations, we obtain stable, data-informed estimates of both the likelihood and spatial scope of extreme fire weather events, which enable risk planners to adaptively set local thresholds based on desired levels of spatial risk coverage. The smooth shape and narrowing confidence bands of these curves highlight the model's strength in providing flexible and robust uncertainty quantification. Together, these plots offer a rigorous and scalable framework for tailoring fire weather warnings and mitigation strategies across heterogeneous geographic regions and varying operational thresholds.

46.3 Application: Level-Set Dynamics with Bayesian Deep Echo State Network (ESN) for Modelling Fire Front Propagation

In this application, we illustrate a model for fire front propagation that is motivated by a mechanistic evolution equation for which a data-driven spatio-temporal spread rate term is critical. Importantly, we model this spread rate in terms of a deep echo state network (ESN). The advantage of the ESN is that only the output parameters need to be learned, which is necessary here because of our very small sample size (12 fire boundaries). Importantly, the model is implemented in a Bayesian framework to properly account for uncertainty, with a novel use of the so-called "cut-Bayes" approach [1331] for dealing with nuisance parameters. Such methods have not been applied to reservoir computing models to the best of our knowledge. We illustrate the model with the infamous Camp Fire, a 2018 devastating mega-fire in California. Before presenting the model and results, we give background on the level set approach and the ESN.

46.3.1 Level Set Background

The level set approach consists of a partial differential equation that describes the evolution of an implicit function in space defined as:

$$\frac{\partial \psi(\xi,t)}{\partial t} + v(\xi,t) \cdot \nabla \psi(\xi,t) = 0, \tag{46.9}$$

where $\psi(\xi,t)$ denotes an implicit function at spatial location $\xi \in \mathbb{R}^d$ and time $t \in \mathbb{R}$, and $\nabla\psi = (\partial\psi/\partial\xi_1, \ldots, \partial\psi/\partial\xi_d)^\top$ is the gradient of ψ over space. The evolution of the process $\psi(\xi,t)$ in (46.9) is deterministic and primarily driven by the spatio-temporal varying velocity field $v(\xi,t) \in \mathbb{R}^d$. Modelling this velocity requires care in order to accurately capture the evolution of the underlying implicit function. In general, there is no closed form solution to Equation (46.9) and numerical methods must be used for its solution [1556].

It is typically a realistic assumption in wildfire modelling that the fire evolves in the direction normal to the boundary [20, 1126, 1818, 1819], in which case Equation (46.9) simplifies to:

$$\frac{\partial \psi(\xi,t)}{\partial t} + v_n(\xi,t) \cdot \|\nabla \psi(\xi,t)\| = 0, \tag{46.10}$$

where $v_n(\xi,t) \in \mathbb{R}$ denotes the scalar velocity (speed) in the *normal* direction and $\|\cdot\|$ denotes the Euclidean (L_2) norm. The implicit function of choice for $\psi(\xi,t)$ is typically a signed distance function [20, 1126, 1818, 1819]. That is, for $\xi \in \mathbb{R}^2$,

$$\psi(\xi) = \begin{cases} -d(\xi) & \text{if } \xi \in \Omega^-, \\ 0 & \text{if } \xi \in \partial\Omega, \\ d(\xi) & \text{if } \xi \in \Omega^+, \end{cases} \tag{46.11}$$

where a location ξ on the boundary $\partial\Omega$ has a signed distance of zero. The signed distance for points inside ($\partial\Omega^-$) and outside ($\partial\Omega^+$) the boundary is defined as the minimum distance from ξ to the boundary, with negative and positive signs, respectively. In wildfire modelling, $d(\xi)$ is defined by the minimum Euclidean distance

$$d(\xi) = \min(\|\xi - \xi_I\|), \tag{46.12}$$

where $\xi_I \in \partial\Omega$ is the set of points on the boundary. Importantly, the fire front is represented "implicitly" by the zero contour of the signed distance function, rather than by an explicit set of points on the fire front. This signed distance representation is simple to interpret and is very flexible in describing potentially complex boundaries (e.g., merges and splits). Essentially any boundary shape can be represented by the signed distance function and the details of the fire front evolution can be preserved in the signed distance function representation if one uses a relatively dense spatial grid.

Because the signed distance function has the property that $\|\nabla\psi(\xi,t)\| = 1$, we can simplify Equation (46.10) so that

$$\frac{\partial \psi(\xi,t)}{\partial t} + v_n(\xi,t) = 0, \tag{46.13}$$

which can be solved by a simple forward Euler numerical approximation:

$$\psi(\xi,t) = \psi(\xi, t - \triangle t) - v_n(\xi, t - \triangle t)\triangle t, \tag{46.14}$$

where $\triangle t$ is the time discretisation interval. Equation (46.14) describes the process at time t in terms of its state at $t - \triangle t$ (Markov property) with the advection governed by the speed $v_n(\xi, t - \triangle t)$. The model for $v_n(\xi, t)$ is critical and should be flexible so as to capture realistic fire evolution behaviour. To achieve this, [1819] employed a latent Gaussian spatio-temporal dynamic process within a Bayesian hierarchical modelling framework, while [1818] leveraged the flexibility of a calibrated ESN to model speed. The ESN approach worked better in real-world wildfire examples, and so we consider it here, extending it to a deep ESN and incorporating this in a fully Bayesian framework.

46.3.2 ESN Background

Deep learning neural network models have improved prediction and classification tasks when data have complex relationships. As with many random effects-based statistical models, accounting for dependency in data is one of the important strengths of neural network models. In the case of sequence data (e.g., language, time series), recurrent neural networks (RNNs) provide a powerful modelling framework to capture complex serial dependence. Despite their flexibility, training such models via "backpropagation through time" [1743] proved to be challenging due to the vanishing/exploding gradient problem [449]. Practitioners have typically turned to gated long short-term memory (LSTM; [760]) models or attention mechanism [1669] as modern alternatives. An earlier alternative, the ESN [1094], has regained popularity in recent years due to its effectiveness in settings where data volume is insufficient for training LSTMs or attention-based deep models.

Put simply, the ESN is a special case of an RNN where the weights connecting the input to the hidden states and weights between hidden states are not learned, but are randomly initialized and remain fixed [1094]. Such models with random weights are considered a type of reservoir computing, with the randomly connected layers serving as the "reservoir." An ESN with a sufficiently large number of hidden states has been shown to satisfy the universal approximation property, which means it can approximate any continuous function [711]. The most basic ESN for time $t = 1, ..., T$ can be written as:

$$\begin{aligned}
\text{output}: \quad & y_t = W^{\text{out}} h_t, \\
\text{hidden state}: \quad & h_t = (1 - \alpha_{\text{leak}}) h_{t-1} + \alpha_\ell \tilde{h}_t, \\
& \tilde{h}_t = g_h\left(\frac{\nu}{|\lambda_w|} W^{\text{in}} h_{t-1} + U x_t\right), \\
\text{parameters}: \quad & W^{\text{in}} = [w^{\text{in}}_{i,\ell}]_{i,\ell} : \gamma^w_{i,\ell} \text{Unif}(-a_w, a_w) + (1 - \gamma^w_{i,\ell})\delta_0, \\
& U = [u_{i,j}]_{i,j} : \gamma^u_{i,j} \text{Unif}(-a_u, a_u) + (1 - \gamma^u_{i,j})\delta_0, \\
& \gamma^w_{i,\ell} \sim \text{Bernoulli}(\pi_w), \\
& \gamma^u_{i,j} \sim \text{Bernoulli}(\pi_u),
\end{aligned} \tag{46.15}$$

where $y_t \in \mathbb{R}^N$, $h_t \in \mathbb{R}^J$, and $x_t \in \mathbb{R}^{n_u}$ denote the output, hidden states (reservoir), and inputs, respectively. In the reservoir framework, the sparse matrices $W^{\text{in}} \in \mathbb{R}^{J \times J}$ and $U \in \mathbb{R}^{J \times n_u}$ are selected randomly. For example, as shown in Equation (46.15), elements of these matrices can be drawn from a uniform distribution with range parameters a_w and a_u. In addition, the parameters π_w and π_u determine the sparsity of W^{in} and U, where δ_0 denotes the Dirac delta function at zero.

The ESN must satisfy the so-called "echo state property" [1839] to effectively model nonlinear processes. Intuitively, this property requires that h_t become insensitive to initial conditions over time, so that the network's behavior is driven primarily by recent inputs rather than by its initial state. In practice, this condition is satisfied when the spectral radius of W^{in} is less than 1 [1094], which can be ensured by scaling W^{in} with a factor ν (with $0 < \nu < 1$), such that the resulting spectral radius is $\nu/\lambda_w < 1$, where λ_w is the largest eigenvalue of W^{in}. The element-wise activation function $g_h(\cdot)$ (e.g., tanh or sigmoid) introduces nonlinearity into the model, as in other neural network models. The recursive nature of the model allows the information in h_{t-1} to be propagated to h_t, and a leaking rate α_{leak} (where $0 \leq \alpha_{\text{leak}} \leq 1$) determines how much of the memory from h_{t-1} is retained in h_t.

An appealing aspect of ESNs relative to RNNs is their low computational cost and training stability, as only the readout matrix $W^{\text{out}} \in \mathbb{R}^{N \times J}$ needs to be learned, whereas RNNs require training all parameters, including weights and biases. By adding an ℓ_2 regularisation penalty to the mean squared error in the loss function, W^{out} can be estimated explicitly using the ridge regression formulation, without using gradient descent optimisation.

That is,

$$\begin{aligned}\hat{W}^{\text{out}} &= \underset{W^{\text{out}}}{\arg\min} \left[\frac{1}{N} \sum_{t=1}^{T} \left\{ (y_t - W^{\text{out}} h_t)^\top (y_t - W^{\text{out}} h_t) \right\} + \rho \sum_{j=1}^{J} \|w_{\cdot j}^{\text{out}}\|^2 \right] \\ &= Y H^\top (H H^\top + \rho I)^{-1},\end{aligned} \tag{46.16}$$

where $Y = [y_1, \cdots, y_T] \in \mathbb{R}^{N \times T}$ and $H = [h_1, \cdots, h_T] \in \mathbb{R}^{J \times T}$ are matrices that concatenate y_t and h_t as columns for $t = 1, ..., T$, respectively. Also, $w_{\cdot j}^{\text{out}}$ denotes the j^{th} column of W^{out}, $\|\cdot\|$ is the ℓ_2 norm, and the regularisation parameter $\rho \in (0, \infty)$ controls the regularisation shrinkage to 0.

Hyperparameter selection

The predictive performance of an ESN depends on its hyperparameters, as is typical for machine learning methods. [1094] recommend assigning small values to π_w and π_u and using a large reservoir size J to promote sparsity in W^{in} and U. The additional model hyperparameters, such as α_{leak}, ν, and ρ, can be optimized using validation or cross-validation techniques with a grid search [1158], genetic algorithms [1159], a backtracking search algorithm [1716], Bayesian optimisation [75, 1101, 1394], or a recycle validation approach [1358]. Alternatively, instead of optimizing hyperparameters, [1818] and [670] sampled them from their prior distributions. A review of ESN hyperparameter optimisation is given in [1150].

Uncertainty quantification

Uncertainty quantification for deep neural models has been an increasingly important research topic in recent years. For ESNs, a simple approach is the "ensemble method", where uncertainty is evaluated using ensembles of forecasts generated from the reservoir given different reservoir weights [1158, 1511, 1810]. That is, each forecast is associated with W^{in} and U, which are independently drawn from their prior distributions given in Equation (46.15). The mean or quantiles of the ensembles can then be obtained from the collection of forecasts. Such methods can also include more sophisticated calibration (e.g., [184]). Alternatively, Bayesian methods can be used, e.g., [262, 1159].

ESN extensions

The ESN presented in Equation (46.15) is very basic. More advanced ESNs have been implemented in the literature for specific applications. For example, [1158] included a quadratic term h_t^2 and the corresponding readout matrix in the output layer to account for quadratic interactions in many physical and biological processes. Deep ESNs have also been used to accommodate multiple scales in the dynamics [185, 890, 1159]. A deep ESN consists of L hidden layers, where the output of the $(\ell - 1)^{\text{th}}$ hidden layer is fed into the ℓ^{th} hidden layer as input. This allows the model to capture more complex temporal dynamic patterns (we utilize a deep ESN in our model presented in Section 46.3.3). Furthermore, as physics-informed neural network models have provided a new paradigm for incorporating mechanistic relationships into neural networks [1371], the same ideas can be applied to ESNs. For example, the physics-informed ESN in [433] accomplishes this by regularizing the readout matrix to ensure that the predictions remain consistent with a specified mechanistic relationship. A systematic review of ESN design is provided in [1573]. Finally, ESNs can be used for other types of data support such as areal-level spatial data [1715] and count observations [1709]. Our approach given below provides a different approach for combining mechanistic information, deep ESNs, and a Bayesian inferential framework.

46.3.3 Bayesian Hybrid Level Set/Deep ESN Model

We now combine the discretised level set evolution equation for the signed-distance function representation of a fire front with a deep ESN to model the rate of spread normal to the fire front. The model is fully Bayesian and the random weights are sampled (although not learned) using a "cut-Bayes" procedure [1331]. To our knowledge, this is the first use of this approach for reservoir computing.

As indicated in Section 46.3.1, modelling the rate of spread $v_n(\xi, t)$ across space and time is crucial, as it determines the propagation of the fire front. To this end, we employ a deep ESN to infer $v_n(\xi, t)$ in Equation

(46.13). The full deep modelling structure is given as:

$$
\begin{aligned}
\text{Response}: \quad & \psi_t = \psi_{t-\triangle t} - v_{n,t-\triangle t}\triangle t + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma^2 I), \\
\text{Rate of spread}: \quad & v_{n,t-\triangle t} = \Psi W^{\text{out}} h^{(L)}_{t-\triangle t}, \\
\ell^{th} \text{ Hidden state}: \quad & h^{(\ell)}_{t-\triangle t} = (1-\alpha^{(\ell)}_{\text{leak}}) h^{(\ell)}_{t-2\triangle t} + \alpha^{(\ell)}_{\text{leak}} \tilde{h}^{(\ell)}_{t-\triangle t}, \\
& \tilde{h}^{(\ell)}_{t-\triangle t} = g_h\left(\frac{\nu^{(\ell)}}{|\lambda^{(\ell)}_w|} W^{in,(\ell)} h^{(\ell)}_{t-2\triangle t} + U^{(\ell)} h^{(\ell-1)}_{t-\triangle t}\right), \\
1^{st} \text{ Hidden state}: \quad & h^{(1)}_{t-\triangle t} = (1-\alpha^{(1)}_{\text{leak}}) h^{(1)}_{t-2\triangle t} + \alpha^{(1)}_{\text{leak}} \tilde{h}^{(1)}_{t-\triangle t}, \\
& \tilde{h}^{(1)}_{t-\triangle t} = g_h\left(\frac{\nu^{(1)}}{|\lambda^{(1)}_w|} W^{in,(1)} h^{(1)}_{t-2\triangle t} + U^{(1)} x_{t-\triangle t}\right), \\
\text{Parameters}: \quad & W^{\text{in},(\ell)} = [w^{\text{in},(\ell)}_{i,j}], \quad w^{\text{in},(\ell)}_{i,j} \sim \gamma^{w,(\ell)}_{i,j} \,\text{Unif}(-a^{(\ell)}_w, a^{(\ell)}_w) + (1-\gamma^{w,(\ell)}_{i,j})\delta_0, \\
& U^{(\ell)} = [u^{(\ell)}_{i,j}], \quad u^{(\ell)}_{i,j} \sim \gamma^{u,(\ell)}_{i,j} \,\text{Unif}(-a^{(\ell)}_u, a^{(\ell)}_u) + (1-\gamma^{u,(\ell)}_{i,j})\delta_0, \\
& \gamma^{w,(\ell)}_{i,j} \sim \text{Bernoulli}(\pi^{(\ell)}_w), \\
& \gamma^{u,(\ell)}_{i,j} \sim \text{Bernoulli}(\pi^{(\ell)}_u),
\end{aligned}
\tag{46.17}
$$

where $\psi_t = (\psi(\xi_1, t), \ldots, \psi(\xi_N, t))^\top$ represents the observed signed distance function for $\xi_1, ..., \xi_N$ at time t and the rate of spread in the normal direction to the boundary at ξ and $t - \triangle t$ is given by $v_{n,t-\triangle t} = (v_n(\xi_1, t - \triangle t), \ldots, v_n(\xi_N, t - \triangle t))^\top$. Additionally, $\epsilon_t = (\epsilon(\xi_1, t), ..., \epsilon(\xi_N, t))^\top$ denotes an error term, and recall $\triangle t$ is the time step size in the forward Euler discretisation. The rate of spread $v_{n,t-\triangle t}$ is parameterized by projecting onto orthogonal global basis functions (e.g., empirical orthogonal functions, EOFs), which is given as the product of Ψ, the readout matrix W^{out}, and the hidden state $h_{t-\triangle t}$, where $h_{t-\triangle t}$ is generated based on the structure of a deep ESN. Other parameters in the hidden state are defined in Section 46.3.2.

As with the ESN regularized by a ridge penalty, a closed-form solution can be derived as shown in (46.16). Uncertainty can be quantified using one of several *ad hoc* approaches as described in Section 46.3.2. Although such approaches have been quite effective, they lack the theory-based approach for uncertainty quantification that a fully Bayesian model provides.

Bayesian implementation

For the hierarchical model (46.17), a Bayesian implementation seems straightforward on first glance. Namely, the posterior distribution for W^{out}, σ^2, and the other parameters $\theta_\ell = (\alpha^{(\ell)}_{\text{leak}}, \nu^{(\ell)}, a^{(\ell)}_u, a^{(\ell)}_w, \pi^{(\ell)}_w, \pi^{(\ell)}_u)^\top$ for $\ell = 1, ..., L$ can be written as $\pi(W^{\text{out}}, \sigma^2, \theta_{1:L} | \psi_{1:T})$, where $\psi_{1:T} = (\psi_1, ..., \psi_T)$ and $\theta_{1:L} = (\theta_1, ..., \theta_L)$. However, updating parameters using standard MCMC methods (e.g., Metropolis-Hastings) or Hamiltonian Monte Carlo suffer from identifiability issues and critically, we are not interested in updating the ESN input or reservoir weights and associated hyperparameters because their role is only to increase the ESN's expressiveness - they are nuisance parameters. One alternative is to simply sample these weights at each iteration of a sampler, accounting for their variability. But this does not produce a valid posterior distribution without further modification, which is known as the "cut-Bayes" approach [1331]. Intuitively, this approach "cuts" the dependence between parameters and data, so that data is not involved in updating the associated parameters. In our case, the "cut-Bayesian posterior" is given by:

$$
\pi^{cut}(W^{\text{out}}, \sigma^2, \theta_{1:L} | \psi_{1:T}) \propto \pi(W^{\text{out}}, \sigma^2 | \psi_{1:T}) \; \pi^{cut}(\theta_{1:L}),
\tag{46.18}
$$

where $\pi^{cut}(\theta_{1:L})$ denotes the cut distribution for $\theta_{1:L}$, which is simply the prior distribution in our implementation.

Using the multiple imputation strategy discussed in [1331], the posterior distribution can be approximated by repeatedly sampling $\theta_{1:L}$ from $\pi^{\text{cut}}(\theta_{1:L})$ and subsequently updating σ^2 and W^{out} for each sampled value in a separate MCMC (Gibbs sampler). The posterior samples are then given by concatenating all samples from each

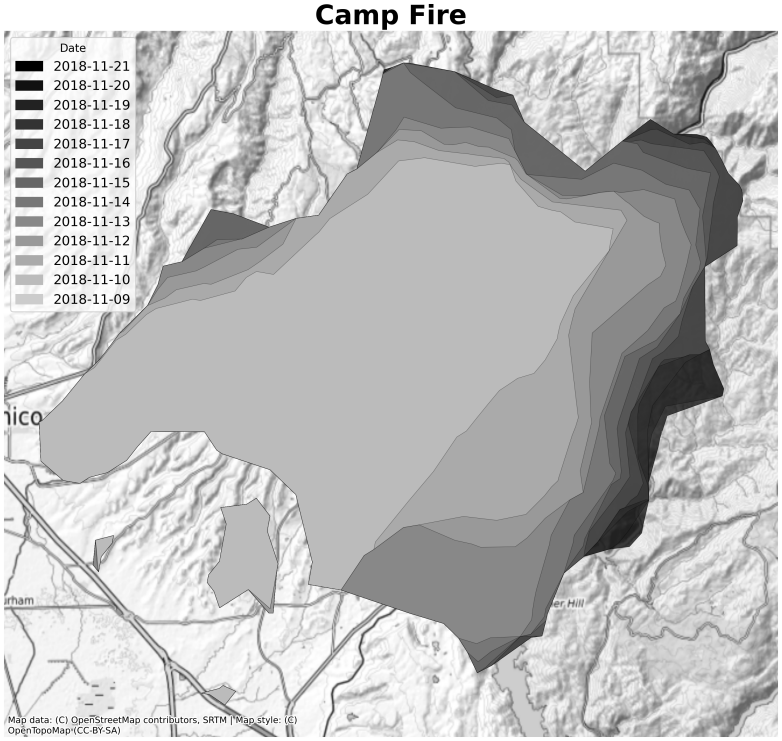


Figure 46.5: The evolution of the California (USA) Camp Fire from 9-20 November 2018.

of these MCMC runs. In this case, we assign prior distributions as:

$$\begin{aligned} W^{\text{out}} &\sim \mathcal{MN}(M_0, U_0, I), \\ \sigma^2 &\sim \text{Inv.Gamma}(a, b), \\ a_u^{(\ell)} &\sim \text{DU}\{0.1, 0.5, 1\}, \\ a_w^{(\ell)} &\sim \text{DU}\{0.1, 0.5, 1\}, \\ \alpha_{\text{leak}}^{(\ell)} &\sim \text{DU}\{0.01, 0.06, ..., 0.94, 1\}, \\ \nu^{(\ell)} &\sim \text{DU}\{0.1, 0.18, ..., 0.81, 0.9\}, \end{aligned} \tag{46.19}$$

where $\mathcal{MN}$ and DU denotes the matrix normal distribution and discrete uniform distribution, respectively. The multiple imputation approach is quite naive, and can be inefficient (given multiple MCMC samplers must be implemented), in which case other cut approaches can be used [1331]. However, in our case, since we are simply implementing a Bayesian regression conditioned on the cut parameters, it is quite efficient to sample from the associated Gibbs sampler for the size of problems of interest with wildfire front propagation.

46.3.4 Application to the 2018 California Camp Fire

In this section, we demonstrate the forecasting performance of our method using the 2018 Camp Fire in California as a case study. The 2018 Camp Fire was one of the most devastating fires in California history, resulting in 85 fatalities and the destruction of nearly 19,000 structures [212]. It was ignited on November 8 due to extremely dry fuels and strong winds and was fully contained on November 25, after burning approximately 153,336 acres. Figure 46.5 shows the evolution of the Camp Fire from November 9 to November 20, 2018. It illustrates the fire's rapid expansion from the initial ignition point, along with distinct "spot fires" appearing to the southwest of the main fire.

For the analysis, we obtain the signed distance field from the boundary data provided by [706]. Specifically, we compute the signed distance function on a 30×30 grid for longitude and latitude ranges $[-121.7844, -121.3351] \times [39.5915, 39.9068]$ for each day from November 9 to November 20, 2018 (i.e., $T = 12$). To assess one-step-ahead forecasting performance, we leave out the data corresponding to November 20, 2018 ($T = 12$).

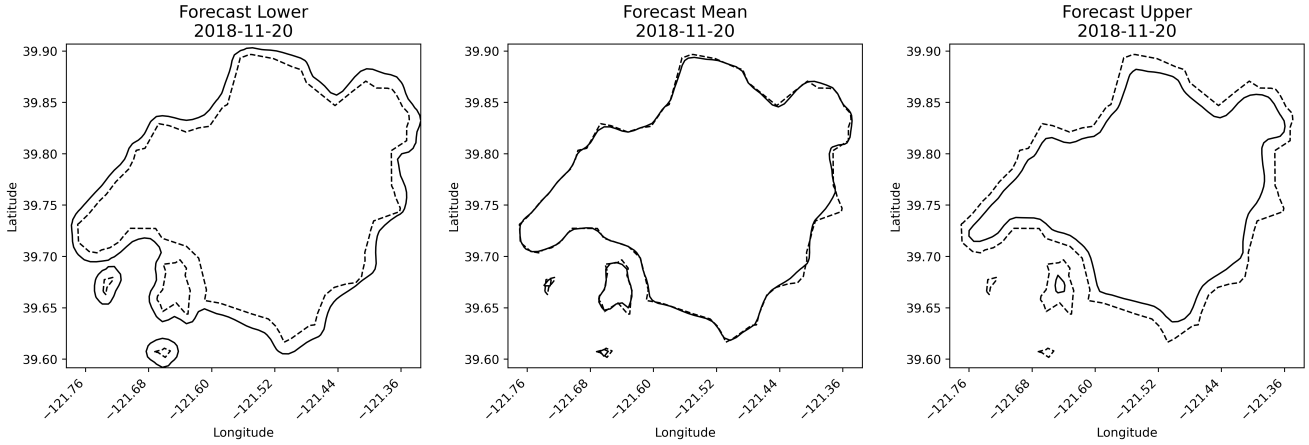


Figure 46.6: The posterior mean and 95% credible interval contours for the one-step-ahead out-of-sample forecast for 20 November 2018. Dashed and solid lines represent the true boundary and the forecast, respectively.

In the implementation, we use two hidden layers in the deep ESN, each with $J = 40$ hidden units. The prior distributions specified in (46.19) are used with $M_0 = \tilde{0}$, $U_0 = 10I$, and $a = b = 0.1$ where $\tilde{0}$ denotes a matrix of zeros. $\pi_w^{(\ell)}$ and $\pi_u^{(\ell)}$ are fixed at 0.3 and 0.5, respectively. We use the EOF basis functions [345] for Ψ calculated from the in-sample values of ψ_t, truncated to the first 5 modes. For the input, we use lagged values of the data, $x_{t-\triangle t} = (\psi_{t-\triangle t}^\top, \psi_{t-2\triangle t}^\top)^\top$, following [1158]. We define $h_1^{(1)}$ and $h_1^{(2)}$ to be $U^{(1)}x_{1+\triangle t}$ and $U^{(2)}h_1^{(1)}$, respectively. To approximate (46.18), we set the number of imputations to 100. For each imputed set of parameters, we draw 5,000 posterior samples of W^{out} and σ^2 using Gibbs sampling, discarding the first 3,000 as burn-in.

Figure 46.6 shows the posterior mean and 95% credible intervals for the one-step-ahead forecasts. The posterior mean closely follows the true boundary, and the 95% credible intervals capture the “spot fires" in the lower left portion of the domain. Forecasting performance is evaluated in terms of Threat Score (TS) [1749] and Interval Score (IS) [639]. The TS score ranges from 0 to 1, with 1 indicating a perfect forecast. Since our primary interest is the fire front, we consider an event to have occurred if $\psi(\xi, t) \leq 0$. We compute the point-wise IS only at locations within the true fire front and then average them over space, noting that a smaller interval score is better. According to both summary measures, our model demonstrates better forecasting performance compared to the (non-deep neural) dynamic linear model Bayesian implementation of [1819] applied to these data. Specifically, the model presented here gives TS = 0.9702 and IS = 0.0219, compared to TS = 0.9691 and IS = 0.0782 for the model in [1819]. Note that the key difference between the two models lies in how the rate of spread is modelled. While [1819] model the rate of spread using spatially varying covariates and a first-order vector autoregression structure to capture linear relationships, the hybrid deep ESN model accounts for nonlinearity through activation functions in the hidden states. These results indicate that modelling nonlinear relationships plays an important role in accurately forecasting fire behaviour.

46.4 Conclusion

Wildfires are increasing in frequency and intensity worldwide, given dramatic climate and land use changes. There are several critical modelling tasks associated with mitigating the risks of wildfires. Here, we consider the problem of performing inference on wildfire risks associated with the fire weather index, implementing a novel flexible model for spatially dependent extremes in a variational Bayesian context, using a novel extreme-aware variational autoencoder (xVAE). We also demonstrate a model that can forecast fire front evolution using a combination of level set dynamics and a deep ESN to model the speed in the normal direction to the fire front. This model is implemented in a fully Bayesian context, with the nuisance ESN reservoir parameters accounted for via a cut Bayes approach. In both cases, we demonstrate realistic uncertainty quantification and efficient implementation through hybrid statistical/deep neural spatio-temporal models.

Several limitations and future research directions remain. First, the xVAE described in this chapter is primarily designed as an emulator of spatial extreme fields learned from the available training data. Although the decoder can generate fast high-dimensional replicates after training, the current formulation does not explicitly condition the latent distribution or decoder parameters on external covariates, such as seasonality, large-scale climate indices,

radiative forcing, or other physical state variables. This limits its direct use for scenario-specific simulation or extrapolation under changing environmental conditions. A promising extension is a conditional xVAE, in which the encoder, latent prior, and decoder are all allowed to depend on a covariate vector $\mathbf{c}_t$.

Second, the algorithm presented for the Bayesian hybrid level set/deep ESN model can become computationally expensive for large T. This is not suitable for operational wildfire management, highlighting the need for scalable algorithms. This need has also been emphasized by [1820], who proposed an online learning algorithm for the hybrid level set/ESN model for real-time forecasting. However, their approach was developed within a non-Bayesian framework, and the uncertainty quantification was somewhat *ad hoc*. Therefore, extending the proposed Bayesian hybrid level set/deep ESN model to an online learning framework, for example based on Sequential Monte Carlo, remains an important direction for future research.

Chapter 47

Software for BDL

This chapter outlines the software foundations that made Bayesian deep learning (BDL) scalable. Probabilistic programming, automatic differentiation, and accelerators now enable efficient posterior inference in neural networks. Frameworks like TensorFlow Probability, Pyro, NumPyro, and PyMC unify model specification, inference, and diagnostics, integrating classical Bayesian methods with deep-learning workflows. These tools implement scalable algorithms—variational inference, MCMC, the Laplace approximation, and ensemble methods—on modern hardware, making Bayesian reasoning a practical, reproducible part of machine learning systems.

47.1 Introduction

Bayesian deep learning (BDL) unites the flexibility of neural networks with the reasoning of Bayesian inference. By placing probability distributions over parameters, latent representations, or outputs, BDL provides not only predictions but also a measure of confidence in those predictions. This uncertainty-aware perspective is essential in domains such as autonomous systems, medicine, and scientific discovery, where understanding what a model does not know can be as valuable as its point estimates.

Early Bayesian neural networks [1108] demonstrated the appeal of marginalising over weights rather than optimising them, but computational limits made such methods impractical for decades. The situation changed with modern automatic differentiation, GPU acceleration, and probabilistic programming. Frameworks now combine expressive architectures with scalable posterior inference, allowing stochastic variational inference (SVI), Hamiltonian Monte Carlo (HMC), or Laplace approximations to run efficiently on contemporary hardware. These advances made Bayesian inference not just a theoretical concept but a practical and widely applicable approach for deep learning.

Framework	Backend	Inference Methods	GPU / TPU Support	Modeling Style
`TensorFlow Probability`	TensorFlow / Keras	VI, HMC, NUTS, SMC	(GPU, TPU)	Hybrid: declarative + functional
`Pyro`	PyTorch	SVI, HMC, NUTS	(GPU)	Imperative (stochastic functions)
`NumPyro`	JAX / XLA	SVI, HMC, NUTS, SMC	(GPU, TPU, multi-chain)	Functional, JIT-compiled
`PyMC`	PyTensor / JAX	VI (ADVI), HMC, NUTS	(GPU via JAX)	Declarative (model objects)
`Stan`	C++ / CmdStanPy	HMC, NUTS, VB	Partial (OpenCL)	Declarative (static model blocks)
`Turing.jl`	Julia / Flux	HMC, NUTS, SMC, VI	(GPU)	Hybrid (DSL + functional)

Table 47.1: **Probabilistic frameworks for BDL.** Overview of major ecosystems, summarizing their computational backends, inference methods, accelerator support, and modeling paradigms.

Software plays a central role in this transformation. Probabilistic programming languages (PPLs) such as TensorFlow Probability, Pyro, NumPyro, PyMC, and Stan provide unified abstractions for model specification, inference, and validation. Their design philosophy differs from that of classical machine-learning libraries: rather than defining deterministic computation graphs, they define generative processes with explicit randomness and automatic inference backends. In parallel, specialised toolkits for approximate Bayesian methods—Bayesian layers, variational posteriors, and Laplace approximations—extend existing deep-learning pipelines with uncertainty estimation at little additional cost. Recent frameworks further standardise ancillary processes such as estimating calibration metrics and evaluating out-of-distribution detection, as well as post-hoc calibration and conformal prediction, enabling the evaluation, benchmarking and deployment of probabilistic models at scale.

The landscape of Bayesian deep learning software is now broad and interconnected. General-purpose PPLs offer the flexibility to express complex hierarchical models, while dedicated libraries streamline practical uncertainty estimation for specific neural architectures and applications. This chapter surveys the most widely used frameworks that operationalise Bayesian deep learning. It emphasises software that is actively maintained, integrates with major deep-learning stacks, and supports scalable inference on modern accelerators. Our discussion covers probabilistic programming systems, approximate Bayesian methods, inference engines and diagnostics, and emerging application frameworks for uncertainty-aware optimisation and deployment.

47.2 Probabilistic programming ecosystems used for BDL

Before automatic differentiation (AD) [105] became a standard component of machine-learning frameworks, most probabilistic inference relied on symbolic gradients or manual derivations. This limited the complexity and scalability of probabilistic models. The integration of reverse-mode AD—first widely popularised through TensorFlow [1] and PyTorch [1297]—enabled gradient-based inference methods such as stochastic variational inference and Hamiltonian Monte Carlo to operate efficiently on large computational graphs. AD also made it feasible to define custom likelihoods, transformations, and reparameterisations without analytic derivatives, dramatically lowering the barrier between probabilistic and deep-learning ecosystems. Gradients are now the common foundation that both deep learning and probabilistic inference rely on. The probabilistic programming ecosystems that we present below, build upon the aforementioned automatic differentiation frameworks.

TensorFlow Probability. TensorFlow Probability (TFP) extends TensorFlow with native support for probabilistic modelling and inference. Its core *Distributions* and *Bijectors* libraries define probability densities, samplers, and composable transformations essential for normalising flows and reparameterisations [427]. TFP implements black-box variational inference, Hamiltonian Monte Carlo (HMC), and No-U-Turn Sampling (NUTS), and provides high-level `JointDistribution` abstractions for constructing complex models [1327]. For Bayesian neural networks, `tfp.layers` includes modules such as `DenseVariational` and `DenseFlipout`, which combine variational posteriors over weights with low-variance gradient estimators [1629, 1738]. Tight integration with Keras allows end-to-end training on GPUs and TPUs using standard optimisers, while tracking the evidence lower bound throughout optimisation.

```
import torch, pyro, pyro.distributions as dist
from pyro.infer import SVI, Trace_ELBO
from pyro.infer.autoguide import AutoDiagonalNormal
from pyro.optim import Adam

# 1) Toy regression data (X: [N, D], y: [N])
X = torch.randn(200, 5)
w_true, b_true = torch.randn(5), 0.5
y = X @ w_true + b_true + 0.1 * torch.randn(200)

# 2) Reuse standard DL components: nn.Linear becomes part of the Bayesian model
net = torch.nn.Linear(5, 1)
pyro.clear_param_store()

def model(X, y=None):
    # 3) Register parameters from torch.nn with Pyro's param store
    #    -> Pyro can learn variational posteriors over these weights
    pyro.module("net", net)

    # 4) Prior on observation noise (log-normal ensures positivity)
    sigma = pyro.sample("sigma", dist.LogNormal(0., 0.1))

```

```
    # 5) Likelihood: predictions from net; to_event(1) treats batch as one event
    mean = net(X).squeeze(-1)
    pyro.sample("obs", dist.Normal(mean, sigma).to_event(1), obs=y)

# 6) Automatic mean-field guide over all latent sites in `model`
guide = AutoDiagonalNormal(model)

# 7) SVI setup: Adam optimizer + ELBO objective
svi = SVI(model, guide, Adam({"lr": 1e-2}), loss=Trace_ELBO())

# 8) Optimization loop: each step updates variational params to minimize ELBO
for _ in range(500):
    svi.step(X, y)
```

Listing 47.1: **Minimal Pyro example**: Bayesian linear regression trained with stochastic variational inference (SVI). The model reuses a `torch.nn` module inside a probabilistic model, illustrating Pyro's seamless integration with deep-learning components. Key steps are annotated inline.

Pyro and NumPyro. Pyro [158] is a probabilistic programming language built on PyTorch. It combines flexible model design with scalable stochastic variational inference (SVI) and efficient MCMC methods such as NUTS and HMC. Through close integration with `torch.nn`, Pyro reuses existing architectures and training tools for Bayesian inference, making it broadly applicable and easy to use (see Listing 47.1 for a minimal example illustrating Pyro's ease of use). NumPyro reimplements Pyro on JAX and XLA, providing the same abstractions with just-in-time (JIT) compilation and automatic vectorisation, making it ideal for large models and parallel sampling on accelerators [1323]. In both frameworks, automatic differentiation and batched linear algebra form the basis of scalable Bayesian learning.

BlackJAX. BlackJAX is a JAX-native [200] library for Bayesian inference built around a functional API that composes naturally with `jit`, `vmap`, and `pmap`. It implements HMC and NUTS as core kernels and includes additional Langevin, stochastic-gradient, sequential Monte Carlo, and variational methods within a unified interface. It also provides Microcanonical Langevin Monte Carlo (MCLMC) [1415], an automatically tuned sampler with fast burn-in that can outperform NUTS by one to two orders of magnitude in high dimensions and has been applied directly to BNN posteriors; it is currently available almost exclusively in BlackJAX. Operating directly on a user-defined unnormalised log density, BlackJAX integrates seamlessly with both probabilistic programming frameworks and hand-written Bayesian deep learning models in the JAX ecosystem. Its vectorised multi-chain execution on accelerators efficiently handles the high-dimensional posteriors common in BDL. See [224] for design details and algorithms.

PyMC. PyMC provides a Pythonic modelling interface for Bayesian models with implementations of state-of-the-art samplers such as NUTS and methods for variational inference such as ADVI [1138]. Although historically associated with hierarchical models, PyMC can also represent Bayesian neural networks by defining random variables for weights and biases, and it interoperates with the ArviZ [946] analysis stack for diagnostics and visualisation. PyMC excels for classical hierarchical models and small pedagogical BNNs, while frameworks such as `posteriors` [461] are more natural at practical deep-learning scale. For networks of moderate size, NUTS with mass-matrix adaptation often produces high-quality posteriors, while ADVI yields fast approximate solutions suitable for initialisation and model exploration.

Bean Machine and Stan. Bean Machine (now archived) provides a declarative modelling interface in the PyTorch ecosystem with automatic differentiation, variational, and Monte Carlo inference backends [1593]. Its design emphasises composability with deep-learning codebases. Stan offers a mature implementation of NUTS and extensive MCMC diagnostics [240]. Although Stan is less frequently used for high-dimensional BNNs due to memory and compile-time constraints, as well as limited support for custom likelihoods on accelerators, it remains a reference standard for smaller networks and methodological validation.

Turing.jl. Turing.jl is a general-purpose probabilistic programming language in Julia that provides flexible model specification and multiple inference backends. It supports gradient-based samplers and particle methods, and couples with `AdvancedHMC.jl` for modular implementations of HMC and NUTS with modern adaptation schemes [528, 1788]. By leveraging Julia's automatic differentiation and interoperability with deep-learning libraries

Pyro (imperative / dynamic)

```
import pyro, pyro.distributions as dist

def model(X, y=None):
    w = pyro.sample("w", dist.Normal(0., 1.).expand([X.size(1)]).to_event(1))
    b = pyro.sample("b", dist.Normal(0., 1.))
    sigma = pyro.sample("sigma", dist.HalfNormal(1.))
    mean = X @ w + b
    pyro.sample("y", dist.Normal(mean, sigma).to_event(1), obs=y)
```

TensorFlow Probability (declarative / static)

```
import tensorflow_probability as tfp
tfd = tfp.distributions

def model(X):
    return tfd.JointDistributionNamed({
        "w": tfd.Normal(0., 1.),
        "b": tfd.Normal(0., 1.),
        "sigma": tfd.HalfNormal(1.),
        "y": lambda w,b,sigma: tfd.Normal(X @ w + b, sigma)
    })
```

Figure 47.1: **Bayesian linear regression in two paradigms.** Pyro defines random variables imperatively within Python code; TFP declares them statically in a joint distribution structure.

such as Flux [816] and Lux [1284], Turing.jl suits Bayesian deep learning models by combining neural components with hierarchical structure. Users can mix HMC/NUTS for key subnetworks with likelihood-free or SMC kernels for non-differentiable parts within a single program, while still benefiting from GPU acceleration.

Probabilistic programming systems differ not only in syntax but also in modelling philosophy (see Table 47.1). Frameworks such as Pyro and NumPyro use an *imperative* or *functional* style, where stochastic variables are declared inline within Python or JAX code. This promotes flexibility and close integration with deep-learning workflows, supporting dynamic architectures and custom control flow. In contrast, systems like Stan and PyMC follow a more *declarative* approach: models are defined as static probabilistic graphs that can be analysed and optimised before inference. Declarative modelling typically yields more efficient inference and a clearer separation between model and algorithm, while imperative designs enable tighter coupling with neural components. TensorFlow Probability bridges the two by offering both graph-based and functional APIs, combining the scalability of static graphs with the flexibility of dynamic modelling. This division is not strict, however: functional JAX-based stacks such as BlackJAX occupy a middle ground, where explicit memory management and composability (`jit`, `vmap`, `pmap`) coexist with the freedom to mix hand-written log-densities and PPL-generated ones—an approach increasingly reflected in libraries like `posteriors` [461] that build on the functional PyTorch API.

47.3 Toolkits for approximate Bayesian deep learning

Bayesian layers and variational training. A pragmatic route to BNNs is to replace deterministic layers by Bayesian counterparts with mean-field or structured posteriors and train via variational objectives. TFP's Bayesian layers implement these designs with reparameterisation and flipout estimators [1629, 1738]. Similar patterns are available in Pyro through auto-guides and custom guides that amortise posterior families over network weights [158]. These approaches prioritise speed and easy deployment, and they integrate well with existing model-selection and training pipelines.

Laplace approximations for neural networks. The Laplace approximation remains compelling for turning pretrained networks into approximate Bayesian predictors by fitting a local Gaussian around a mode. The `laplace-torch` library implements modern Laplace variants, including diagonal, Kronecker-factored, and approximations with scalable marginal-likelihood tuning [380]. This path yields calibrated uncertainty with minimal code changes and can be combined with ensembling or low-rank corrections.

Uncertainty estimation beyond probabilistic programming. A large portion of practical Bayesian deep learning research relies on approximate uncertainty estimation methods implemented outside traditional probabilistic programming frameworks. Techniques such as Monte Carlo dropout [566], deep ensembles [962], stochastic weight averaging [824] and its Bayesian variant SWAG [1116], as well as structured ensemble approaches like MaskEnsembles [469], BatchEnsemble [1739], and Packed-Ensembles [979], among many others, achieve competitive uncertainty quality and out-of-distribution robustness while remaining fully compatible with standard deep-learning pipelines. Each of these methods is accompanied by official implementations, and unified toolkits such as `uncertainty-baselines` [1220] and `torch-uncertainty` [959] further streamline evaluation and benchmarking across architectures and datasets. For tabular data, `bde` [51] implements Bayesian deep ensembles via MILE behind a familiar scikit-learn `fit`/`predict` API with a JAX backend.

Simulation-based inference. Simulation-based inference (SBI) methods [400] make it possible to learn inference algorithms requiring access only to model simulations, and therefore can also be applied to stochastic simulators with intractable likelihoods (Chapter 3). The `sbi` package implements a range of different inference approaches including neural posterior, likelihood, and ratio estimation, with diagnostics and samplers [178]; `BayesFlow` provides a framework focused on amortised Bayesian workflows [941]; and `swyft` targets simulation efficiency via truncated marginal ratio estimation [1176]. All provide methods which can amortise an upfront simulation and training cost into near-instant inference for new observations.

ArviZ. Diagnostic tools are crucial for BNNs, where non-identifiability and symmetries make convergence assessment challenging. ArviZ [946] is a Python library designed for the exploratory analysis of Bayesian models. It provides a unified, backend-agnostic interface for diagnosing, summarising, and visualising results from probabilistic programming frameworks such as PyMC, Stan, NumPyro, and Pyro. ArviZ includes tools for posterior predictive checks, convergence diagnostics, and rich statistical visualisations that help evaluate the quality and reliability of Bayesian inference. By standardising analysis workflows across different backends, ArviZ streamlines the interpretation and comparison of Bayesian model outputs. A caveat, however, is that many standard diagnostics are largely uninformative for BNN posteriors, whose weight-space symmetries and non-identifiability make them ill-suited to convergence assessment in the usual sense (see Chapter 33).

47.4 Applications and downstream toolchains

Bayesian optimisation. Beyond uncertainty estimation within neural networks, Bayesian principles also underpin optimisation and experimental design toolchains. `BoTorch` [78] provides a modular library for Bayesian optimisation built on PyTorch, supporting Gaussian process surrogates, neural network surrogates, and probabilistic acquisition functions. The example in Figure 47.2 illustrates a typical optimisation loop: the code listing (top) shows how a Gaussian process model and the Expected Improvement (EI) acquisition are used to propose new evaluation points, while the sequence of plots (bottom) visualises how the surrogate posterior progressively concentrates around the global optimum over successive iterations. `BoTorch` integrates seamlessly with `Ax` [1259], a high-level experimentation platform for automated hyperparameter tuning and adaptive data collection.

Active learning and dataset curation. Uncertainty estimates from Bayesian neural networks naturally enable active data acquisition strategies [1492]. By quantifying epistemic uncertainty, models can prioritise unlabelled or high-variance samples that most reduce posterior entropy. Libraries such as `modAL` [370] and `small-text` [1474] integrate BDL-style uncertainty measures—predictive entropy, mutual information, and ensemble prediction variance—into selection heuristics. When combined with scalable surrogate models or simulators, as in `BoTorch`, these methods support closed-loop data collection for vision, NLP, and scientific applications.

Out-of-distribution detection. Detecting when a model encounters inputs that differ from its training distribution is one of the most practical uses of uncertainty estimation. Bayesian deep learning methods approach out-of-distribution (OOD) detection by measuring predictive uncertainty—using posterior variance, entropy of the predictive distribution, or disagreement among ensemble members—as a signal of epistemic uncertainty. However, not all OOD detection pipelines require full Bayesian inference. The `PyOD` library [1866] provides a broad suite of classical and neural anomaly detection algorithms, including isolation forests, autoencoders, and deep density models, which can be combined with Bayesian uncertainty scores or calibrated posteriors.

```python
import torch
from botorch.models import SingleTaskGP
from botorch.fit import fit_gpytorch_mll
from botorch.acquisition import ExpectedImprovement
from botorch.optim import optimize_acqf
from gpytorch.mlls import ExactMarginalLogLikelihood

# Objective function on [0,1]^2
def f(X):
    return (torch.sin(6*X[:,0]) + torch.cos(6*X[:,1])).unsqueeze(-1)

# Initial design and GP fit
train_x = torch.rand(10, 2)
train_y = f(train_x)
gp = SingleTaskGP(train_x, train_y)
mll = ExactMarginalLogLikelihood(gp.likelihood, gp)
fit_gpytorch_mll(mll)

# Expected Improvement and candidate selection
EI = ExpectedImprovement(gp, best_f=train_y.max())
bounds = torch.stack([torch.zeros(2), torch.ones(2)])
candidate, acq_val = optimize_acqf(EI, bounds, q=1, num_restarts=5, raw_samples=128)
```

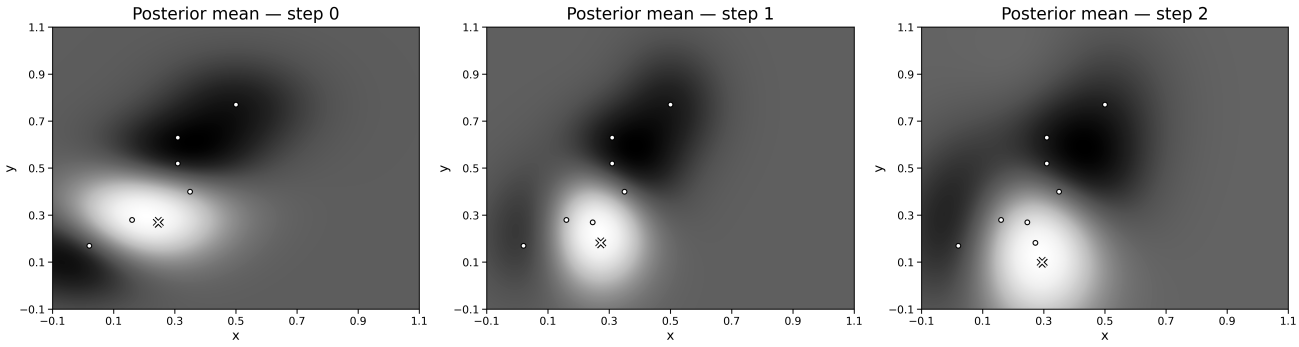


Figure 47.2: **Bayesian optimization of a 2D toy objective with a GP surrogate. (Top):** core BoTorch implementation showing model fitting and Expected Improvement optimization. **(Bottom):** posterior mean evolution across iterations — dots denote evaluated points, and the × marks the current best. The optimizer progressively refines its belief about the function's optimum.

Bayesian neural architecture search and hyperparameter optimisation. Bayesian principles also govern the exploration of neural architectures and hyperparameters under uncertainty. Frameworks such as `Ax` [1259], `Optuna` [17], and `Ray Tune` [1039] employ probabilistic surrogate models—typically Gaussian processes or tree-structured Parzen estimators—to navigate large design spaces efficiently. Recent work integrates variational and Monte Carlo inference directly into architecture hyperparameters, treating network configurations as random variables with learned posteriors. These methods combine the flexibility of BDL with automated model selection, yielding uncertainty-based search procedures that adaptively balance exploration and exploitation across computational budgets.

Large language models and uncertainty estimation. The rise of large pretrained models has renewed interest in scalable uncertainty estimation. Bayesian fine-tuning techniques—such as Bayesian linear probes, low-rank variational adapters, and stochastic weight averaging—extend BDL concepts to foundation models while remaining computationally tractable. A rapidly growing line of work casts LoRA adapters as Bayesian, including Laplace-LoRA [1791] and SWAG-LoRA [1260]. In this context, the open-source framework `LM_Polygraph` [500] offers a unified toolkit for uncertainty estimation in text-generation tasks, supporting a variety of methods (e.g., token entropy, mutual information, sequence probability) and providing a benchmark for modelling and deployment (see Listing 47.2).

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
from lm_polygraph.utils.model import WhiteboxModel
```

```
from lm_polygraph.utils.manager import estimate_uncertainty

model_path = "bigscience/bloomz-560m"
tokenizer  = AutoTokenizer.from_pretrained(model_path)
base_model = AutoModelForCausalLM.from_pretrained(model_path, device_map="cuda:0")
model      = WhiteboxModel(base_model, tokenizer, model_path=model_path)

ue_method = "MeanPointwiseMutualInformation"
input_text = "What is the capital of France?"
ue = estimate_uncertainty(model, ue_method, input_text=input_text)
print(f"Generation: {ue.generation_text}")
print(f"Uncertainty score: {ue.uncertainty:.4f}")
```

Listing 47.2: `LM_Polygraph` **for estimating uncertainty in large language models.** This snippet loads a pretrained model (`bloomz-560m`), wraps it in the `WhiteboxModel` interface, and computes an uncertainty score using the `MeanPointwiseMutualInformation` method for a given input prompt.

47.5 Final remarks

Choosing a stack in practice. The choice depends on architectural preferences and target hardware. TFP and its Bayesian layers offer the most direct Keras integration, supporting end-to-end differentiable variational training and mature MCMC implementations. Pyro provides flexible model construction and scalable SVI around PyTorch modules, while NumPyro enables high-throughput NUTS and SVI on JAX with minimal modelling friction. For BDL specifically, the lower-level BlackJAX is often the better fit than the higher-level NumPyro, as it operates on arbitrary log-density functions and exposes cutting-edge kernels such as MCLMC that NumPyro does not wrap. PyMC offers a concise modelling interface with excellent diagnostics and is well suited for medium-scale BNNs and pedagogical settings. Laplace-based tools such as `laplace-torch` turn pretrained models into Bayesian predictors with minimal refactoring, and libraries like `Fortuna` [418] help standardise uncertainty evaluation and deployment. Since `Fortuna` is no longer actively maintained, we point readers to actively developed alternatives such as `torch-uncertainty` [959], `Lightning UQ Box` [1010], and `posteriors` [461] for this role.

Open challenges. Despite significant progress, Bayesian deep learning (BDL) still faces notable practical and conceptual obstacles:

- **Computational cost.** Inference algorithms such as Hamiltonian Monte Carlo (HMC) and full-rank variational inference (VI) remain computationally demanding for large neural architectures. Even with GPU batching and JIT compilation, runtime and memory consumption can be prohibitive for modern networks with tens of millions of parameters. Research on low-rank, structured, and subnetwork-based approximations continues to seek scalable compromises.
- **Reparameterisation and expressivity.** Reparameterisation gradients make VI efficient but restrict the space of admissible posterior families. Discrete latent variables and non-Gaussian priors often require specialised relaxations or reparameterisation tricks, which complicate implementations.
- **Evaluation metrics and calibration.** While predictive uncertainty is widely discussed, its quantitative evaluation remains inconsistent. Metrics such as negative log-likelihood, expected calibration error (ECE), Brier score, and coverage measure different aspects of uncertainty. Toolkits such as `Fortuna` and `Uncertainty Baselines` provide partial unification, yet widely accepted standardised benchmarks and evaluation protocols remain lacking.
- **Benchmark reproducibility.** Differences in initialisation, priors, datasets, and training schedules can drastically affect posterior quality and calibration. The community lacks a set of canonical, reproducible benchmarks for uncertainty estimation analogous to ImageNet or GLUE for accuracy-based evaluation. Shared evaluation suites and standardised reference implementations are essential for fair comparison and cumulative progress.
- **Storing and serving posterior samples.** Sampling-based BDL lacks widely adopted tooling for efficiently saving, compressing, sharing, and serving large collections of posterior weight samples. With millions of parameters and hundreds of samples, storage, I/O, and serving quickly become non-trivial, which partly explains why post-hoc and single-forward-pass methods still dominate in production.

Outlook. Ecosystems continue to converge on a small set of differentiable probability primitives, high-order transformations, and compiled samplers. For BDL, this unification means that modellers can prototype in a high-level PPL, borrow optimised inference kernels, and ship models with principled uncertainty estimates and reproducible diagnostics. The software landscape already supports most inference methods required in practice, and continued work on scalable samplers, structured variational families, and modular uncertainty evaluation will further lower the barrier to Bayesian methods in modern neural networks.

Future directions. Bayesian principles are increasingly converging with modern large-scale learning paradigms. The next generation of BDL software will likely be shaped by the following trends:

- **Foundation-model uncertainty.** Extending Bayesian fine-tuning to large language and vision models through variational low-rank adapters, Bayesian LoRA, and stochastic weight averaging across massive parameter spaces.
- **Scalable probabilistic inference.** Exploiting distributed JAX and PyTorch backends, multi-chain vectorisation, and compiler-level fusion to enable large-scale posterior sampling and amortised inference on accelerators.
- **Differentiable simulators.** Combining probabilistic inference with differentiable simulators to support Bayesian calibration of scientific models in physics, chemistry, and biology.
- **Uncertainty-aware deployment.** Extending probabilistic programming to production settings through standardised APIs for calibration, interpretability, and runtime uncertainty monitoring.

Together, these developments point toward a synthesis of classical Bayesian reasoning and modern foundation-model infrastructure, where uncertainty estimation becomes a native property of scalable machine-learning systems rather than a post-hoc addition.

Part X

Topical developments

Chapter 48

Subspace inference

Bayesian inference in modern neural networks is challenged by the high dimensionality of the parameter space Θ, which may contain millions of weights. General-purpose posterior inference in this space is often computationally demanding, while symmetries and redundancies can make posterior exploration difficult (see, e.g., Chapter 33). Subspace inference reduces this burden by restricting posterior uncertainty to a selected low-dimensional subset or parameterisation of Θ. Depending on the construction, this reduced space may be a linear subspace, a nonlinear manifold, or a more general low-dimensional region of the original parameter space.

48.1 Method overview

Various methods have been proposed for constructing such reduced parameterisations. In this chapter, we consider two approaches: *subnetwork inference* and inference in parameterisations constructed from connected modes of the loss surface.

Subnetwork inference uses an axis-aligned coordinate subspace by fixing some parameters and treating the remaining parameters as stochastic. Mode-connectivity approaches instead construct linear or nonlinear low-dimensional parameterisations that follow connected low-loss regions of the parameter space.

Once a reduced parameterisation has been selected, posterior inference can be performed over its coordinates using either sampling or an approximate inference method. A prominent example is last-layer Laplace inference, which treats the last-layer parameters as stochastic while fixing the feature extractor.

Related approaches retain the full parameterisation but design structured Markov chain Monte Carlo (MCMC) updates that reduce harmful posterior dependencies and improve mixing [21].

48.2 Subnetwork inference

A natural way to construct a reduced parameter space is to use the network's topological structure as a guide. It has been well known for decades that neural networks can be substantially pruned, meaning that many of their weights can be set to zero, while incurring little or no loss in predictive performance [286]. This raises the question of whether uncertainty quantification in Bayesian neural networks (BNNs) follows a similar principle. In particular, can a Bayesian treatment be restricted to only a subset of the parameters while preserving the expressive benefits of a BNN?

In this section, we first summarise theoretical results showing that, under suitable assumptions, a Bayesian treatment can be restricted to a small subset of the parameters without sacrificing the expressivity of the resulting predictive distribution. We then describe a practical algorithm for selecting a subnetwork on which to perform Bayesian inference.

Let S and D denote disjoint index sets corresponding to the *stochastic* and *deterministic* parameters, respectively. A posterior distribution is placed over θ_S, whereas the parameters θ_D are fixed at values $c_D = (c_j)_{j \in D}$. Although we refer to the latter parameters as deterministic, their values may have been obtained using a learning algorithm or specified manually. The relevant distinction is that they are represented by point masses rather than non-degenerate posterior distributions. The resulting posterior can be written as

$$p(\theta_S, \theta_D \mid \mathcal{D}) = p(\theta_S \mid \mathcal{D}) \prod_{j \in D} \delta_{c_j}(\theta_j), \tag{48.1}$$

where δ_{c_j} denotes a point mass at c_j.

The corresponding posterior predictive distribution is

$$\begin{aligned} p(y_* \mid x_*, \mathcal{D}) &= \int_{\theta_S, \theta_D} p(y_* \mid x_*, \theta_S, \theta_D) p(\theta_S \mid \mathcal{D}) \prod_{j \in D} \delta_{c_j}(\theta_j) \, \mathrm{d}\theta_S \, \mathrm{d}\theta_D \\ &= \int_{\theta_S} p(y_* \mid x_*, \theta_S, c_D) p(\theta_S \mid \mathcal{D}) \, \mathrm{d}\theta_S. \end{aligned} \tag{48.2}$$

The two central questions are therefore whether concentrating all posterior randomness in θ_S restricts the class of posterior predictive distributions, and how the stochastic subset S should be chosen in practice.

48.2.1 Expressivity of subnetwork-derived predictive distributions

One might expect subnetwork inference to trade predictive expressivity for computational tractability. However, Sharma et al. [1501] show that, under the assumptions of their result, a neural network with only a small fixed number of stochastic variables can remain a universal approximator of conditional distributions. Restricting stochasticity therefore need not reduce predictive expressivity in principle. The result is existential, however, and does not prescribe how the stochastic parameters should be selected in practice.

The argument combines two established results. The first is the *noise outsourcing lemma* [64, 1874], which states that a conditional distribution can be represented through a measurable function that maps the input and an independent noise variable to the output. Consequently, samples from the conditional distribution can be obtained by sampling the noise variable and pushing it through this function. The corresponding function is sometimes called a *generator function*; its representation is generally not unique.

The second ingredient is a universal approximation result for feedforward neural networks [1014]. Combining these results yields the universal conditional-distribution approximation result of Sharma et al. [1501]. In particular, sufficiently expressive networks can approximate arbitrary conditional distributions while using only a small, fixed number of stochastic variables, provided that these variables enter the architecture before at least one nonlinear hidden layer.

A crucial feature of this construction is therefore that the stochastic variables are transformed by a nonlinear part of the network. Consequently, last-layer-only constructions do not possess the same universal conditional-distribution approximation property under this argument.

48.2.2 Subnetwork-based Laplace approximation

The theoretical result of Sharma et al. [1501] does not prescribe how to construct a semi-stochastic BNN in practice. It establishes that a small number of stochastic variables can be sufficient, but does not specify which network parameters should be treated stochastically.

Daxberger et al. [381] propose a practical heuristic for selecting these parameters. After fitting the full network by maximum a posteriori (MAP) estimation, they use a diagonal Laplace approximation to estimate the marginal posterior variances of the parameters. The parameters with the largest estimated variances are selected for the stochastic subnetwork. A Laplace approximation is then constructed over this restricted collection of parameters using the corresponding curvature information, while all remaining parameters are fixed at their MAP estimates.

The diagonal approximation is computationally attractive because its inverse can be obtained elementwise. It therefore provides an inexpensive ranking of the approximate marginal parameter variances without first constructing a full posterior approximation.

Other selection strategies are possible. For example, one may choose the parameters whose MAP estimates have the largest magnitudes, similarly to magnitude-based pruning. More sophisticated methods can instead select parameters or curvature entries using targeted subsampling of the Hessian matrix [503].

48.3 Subspaces of connected modes

In contrast to subnetwork inference, which fixes most coordinates in Θ and treats only a selected subset as stochastic around a point estimate, the approaches considered in this section exploit *mode connectivity*. Mode connectivity refers to the empirical observation that high-performing solutions can often be joined by continuous low-loss curves in parameter space [825].

The first step is to construct a low-dimensional connected parameterisation that follows such a curve. Optionally, the curve may be extended to a small ambient "tunnel" that permits posterior exploration in directions

normal to the path. Such constructions can capture richer predictive variation than an axis-aligned subnetwork of comparable dimension, although finding and representing the connected region is generally more expensive.

Bayesian inference is subsequently performed over the coordinates of the constructed parameterisation, while all remaining degrees of freedom are kept fixed.

48.3.1 Constructing a subspace

Let $T = [0, 1]$. A *loss path* is a continuous map $b : T \to \Theta$, evaluated through

$$\rho(t) = \ell\big(b(t)\big),$$

such that the loss remains comparatively small along the path. The map b may be represented using any sufficiently flexible family of curves. A well-performing choice is a regular Bézier curve with control points $\Lambda = \{\theta_0, \dots, \theta_K\}$ and Bernstein weights $\omega_k(t)$:

$$b_\Lambda(t) = \sum_{k=0}^{K} \omega_k(t)\theta_k, \qquad \omega_k(t) = \binom{K}{k}(1-t)^{K-k}t^k. \tag{48.3}$$

A practical training objective is the path integral

$$\mathcal{L}(\Lambda) = \int_0^1 \ell\big(b_\Lambda(t)\big)\,\mathrm{d}t = \mathbb{E}_{t\sim\mathrm{Unif}(0,1)}\left[\ell\big(b_\Lambda(t)\big)\right]. \tag{48.4}$$

This objective weights all values of the curve parameter t equally. It does not, in general, weight points uniformly with respect to arc length, since the speed $\|b'_\Lambda(t)\|$ may vary along the curve. A reparameterisation-invariant alternative is the arc-length-weighted objective

$$\widetilde{\mathcal{L}}(\Lambda) = \frac{\int_0^1 \ell\big(b_\Lambda(t)\big)\|b'_\Lambda(t)\|\,\mathrm{d}t}{\int_0^1 \|b'_\Lambda(t)\|\,\mathrm{d}t}. \tag{48.5}$$

In practice, both objectives have been observed to produce similar results [436].

Optimising Equation 48.4 is straightforward. Let $\mathcal{I} \subseteq \{0, \dots, K\}$ denote the indices of the trainable control points. For each $k \in \mathcal{I}$, the gradient is

$$\begin{aligned}\nabla_{\theta_k}\mathcal{L} &= \mathbb{E}_{t\sim\mathrm{Unif}(0,1)}\left[\frac{\partial b_\Lambda(t)}{\partial\theta_k}^{\top}\nabla_\vartheta \ell(\vartheta)\Big|_{\vartheta=b_\Lambda(t)}\right] \\ &= \mathbb{E}_{t\sim\mathrm{Unif}(0,1)}\left[\omega_k(t)\nabla_\vartheta\ell(\vartheta)|_{\vartheta=b_\Lambda(t)}\right].\end{aligned} \tag{48.6}$$

Drawing a single value $t^* \sim \mathrm{Unif}(0, 1)$ gives the unbiased single-sample estimator

$$\widehat{\nabla_{\theta_k}\mathcal{L}} = \omega_k(t^*)\nabla_\vartheta\ell(\vartheta)|_{\vartheta=b_\Lambda(t^*)}. \tag{48.7}$$

The resulting gradient is the usual network gradient evaluated at $b_\Lambda(t^*)$, multiplied by the corresponding Bernstein weight. When the endpoints are fixed at two previously obtained modes, only the intermediate control points are optimised, so that $\mathcal{I} = \{1, \dots, K-1\}$. In mini-batch optimisation, drawing one value of t^* per iteration is often sufficient to approximate the expectation in Equation 48.4.

From path to tunnel To permit structured deviations from the curve, suppose that $b'_\Lambda(t) \neq 0$ and define the unit tangent

$$\tau(t) = \frac{b'_\Lambda(t)}{\|b'_\Lambda(t)\|}.$$

At each t, complete this tangent to an orthonormal frame

$$\{\tau(t), \nu_1(t), \dots, \nu_r(t)\},$$

where $\nu_1(t), \dots, \nu_r(t)$ are normal directions. A numerically stable choice is a rotation-minimising frame, which transports an initial normal frame along the curve while introducing as little additional rotation as possible. This avoids abrupt curvature-induced changes in the normal directions.

The resulting *tunnel* is parameterised by

$$\Phi = \left\{ g(t, \xi) = b_\Lambda(t) + \sum_{j=1}^{r} \xi_j \nu_j(t) : t \in [0, 1],\ \xi \in \Xi \subset \mathbb{R}^r \right\}. \tag{48.8}$$

This construction defines a connected $(1 + r)$-dimensional parameterised region within Θ. We use the term *subspace* here in the broader sense common in the subspace-inference literature; mathematically, Φ is generally a nonlinear manifold or parameterised subset rather than a linear subspace. The set Ξ must be chosen sufficiently small, or optimised explicitly, to ensure that the tunnel remains within an acceptably low-loss neighbourhood of the original path.

48.3.2 Subspace posterior inference

Assume that a reduced parameterisation Φ has been constructed. The remaining degrees of freedom are fixed at a reference value $\hat{\theta}$, either explicitly or implicitly through the map g, and Bayesian inference is performed over the coordinates (t, ξ).

For the normal coordinates, one possible prior is

$$p(\xi) = \mathcal{N}(0, \sigma^2 I),$$

possibly truncated to Ξ, with σ chosen relative to the tunnel radius. For the path coordinate, two natural but distinct choices are a uniform prior over the parameter,

$$t \sim \mathrm{Unif}(0, 1),$$

and a prior that is uniform with respect to arc length. Let

$$s(t) = \int_0^t \|b'_\Lambda(u)\| \, \mathrm{d}u, \qquad S = s(1).$$

A prior that is uniform in $s \in [0, S]$ induces the density

$$p(t) = \frac{\|b'_\Lambda(t)\|}{S}. \tag{48.9}$$

The two choices coincide only when the curve has constant speed.

Given a prior $p(t, \xi)$ on the coordinates, the posterior target is

$$p(t, \xi \mid \mathcal{D}) \propto p\big(\mathcal{D} \mid g(t, \xi)\big) p(t, \xi). \tag{48.10}$$

Here, $p(t, \xi)$ is interpreted as a density directly over the selected coordinates. If a density with respect to the volume measure induced on Φ is intended instead, the corresponding Jacobian determinant of g must be included.

The posterior can be explored using low-dimensional MCMC methods, such as the No-U-Turn Sampler (NUTS). Elliptical slice sampling is another option when the prior over the subspace coordinates has the required Gaussian structure. Reducing the dimension of the parameter space can make such classical sampling methods substantially easier to apply.

The reduced representation can also be combined with other structured models, facilitating the integration of deep neural networks into broader structured learning systems [435], including the use in large language models [437].

48.4 Related methods

Many other methods construct low-dimensional parameterisations or perform inference in reduced parameter spaces. Common examples exploit the geometry of optimiser trajectories.

Stochastic Weight Averaging (SWA) [SWA; 824] averages parameter snapshots from the late stages of stochastic gradient descent (SGD) to obtain a single, typically flatter solution that can improve generalisation and calibration. Stochastic Weight Averaging-Gaussian (SWAG) [1116] extends this idea by fitting a low-rank-plus-diagonal Gaussian distribution to the trajectory, thereby enabling parameter sampling and approximate Bayesian model averaging.

Another possibility is to apply principal component analysis (PCA) to a gradient-descent (GD) or SGD trajectory. PCA identifies dominant directions of variation in the optimiser iterates, which span a data-adaptive linear subspace. Bayesian inference can then be performed only over the coordinates associated with these directions.

Subspace inference can substantially reduce the computational cost of Bayesian neural network inference, but its performance depends strongly on how the reduced parameterisation is constructed. Axis-aligned subnetworks are comparatively simple, whereas mode-connected parameterisations can represent richer posterior variation at the cost of more involved optimisation and prior specification. Empirical evaluations should therefore assess not only predictive performance, but also sensitivity to the selected subspace, its dimension, and the prior placed over its coordinates.

Chapter 49

Implicitly Bayesian prediction rules

To reap the advantages of a Bayesian approach, most of this book prescribes a structured Bayesian approach: defining a parametric model, setting priors (Part VI), and approximating posteriors and posterior predictive distributions (Parts I, II, and III). But is that the only way? In this chapter, we will show how to characterise what makes a method Bayesian at a *behavioural* level, and illustrate that algorithms without an explicit prior-then-posterior construction can, in effect, be equally Bayesian.

Before answering this question, it is worth asking why we want to be Bayesian in the first place.

First and foremost, the Bayesian framework is often said to handle uncertainty in a principled manner. In machine learning, we are often concerned with making decisions based on predictions for future observations. With a Bayesian approach, that prediction takes the form of a full predictive probability distribution over the range of possible outcomes. Moreover, that distribution captures uncertainty in what the best possible distribution in the chosen model class might be: the Bayesian approach integrates over all possible predictions within the model class, weighting them according to their congruence with the data observed so far. Incorporating this uncertainty into the predictive distribution means that we can make decisions which minimise expected risk with respect to all the unknowns represented within the model. Bayesian decision theory guarantees that these decisions will be optimal under the (rather strict) assumption that our modelling choices are correct [123].

In practice, our prior and likelihood are rarely literal descriptions of either our beliefs or the data-generating process. However, even if the assumptions are wrong, Bayesian predictions retain a certain coherence property: whatever the stated beliefs about the state of the world were at the start, they will be updated rationally in the light of new data. Dutch book arguments give this property an operational interpretation, showing that incoherent updates expose an agent who acts on them to systematic exploitation [970, 1320]. This gives Bayesian predictions a measure of credibility, even when the modelling assumptions are wrong. For instance, when following the Bayesian approach exactly, one does not have to worry (as much) about whether some evidence is being given more weight than another, or whether some evidence is ignored outright. In the i.i.d. setting, for example, one might be concerned with whether a prediction depends arbitrarily on the order the data was presented in. Such order-invariance is not automatic in many approaches to prediction, such as deep learning: the order in which data are presented is known to significantly affect the final predictions [53, 1157]. Following the Bayesian approach prevents this. Even putting computational and approximate-inference considerations aside, a Bayesian still has plenty to worry about with regard to modelling choices; nonetheless, at the very least, some fundamental sanity checks are taken care of.

The conventional way to obtain these desirable predictive properties is through an explicit Bayesian construction: specify a prior, explicitly form a (typically approximate) posterior, then integrate it out to obtain predictions. In deep learning, however, this construction remains challenging. Algorithms that explicitly approximate posteriors in such Bayesian neural networks (BNNs) tend to encounter scalability challenges, which are the subject of much of this book. For instance, Markov chain Monte Carlo (MCMC) schemes (Part I) require, in effect, multiple models to be trained for an accurate approximation. Variational inference schemes, on the other hand, often struggle to fit complex posteriors over the parameters without similarly complex and difficult-to-train models to approximate that posterior.[1]

These difficulties motivate asking whether the same predictive properties can be obtained without an explicit prior–posterior construction. The key observation is that coherence is a property of the predictions themselves, rather than of the mechanism through which they were computed. We therefore take a look at a different perspective on how to incorporate Bayesian principles into deep learning. In this chapter, we characterise the minimal

[1] Function-space variational inference (covered in Chapter 17), however, offers a promising path towards resolving these issues.

conditions under which a prediction rule behaves as if it arose from an explicit Bayesian construction. We argue that this can be a more fruitful way to incorporate Bayesian principles into deep learning, as it only dictates how predictions should behave, rather than constraining how the prediction algorithm is structured. This greater flexibility suggests exploring novel approaches to making an algorithm more Bayesian, e.g. through better hyperparameter choices, explicit penalties, or pretraining techniques [583], without explicitly specifying a prior or approximating a posterior.

We will start this chapter by introducing the i.i.d. prediction setting and recapitulating what constitutes an explicitly Bayesian approach. Then, we will show what characterises the Bayesian approach at the level of predictions. We will give a construction for turning any finite-horizon prediction rule in this setting into one that behaves like a Bayesian rule, and show two ways in which the Bayesian counterpart is always strictly superior to the non-Bayesian one. Lastly, we will discuss how one can go about measuring how close any arbitrary predictor is to behaving like a Bayesian algorithm, and present this as a useful way for evaluating the fidelity of approximate inference for predictive approaches where explicit prior and posteriors are not trivially available (e.g. neural processes [1200]).

49.1 Explicitly vs. Implicitly Bayesian Methods

Whereas statistics often deals broadly with inferences about various unobserved quantities, the prediction of future observations and decision-making based on those predictions are arguably at the core of machine learning. We now make this sequential prediction setting precise. Given a sequence of n observations $X_1, \dots, X_n$, we are interested in predicting the value of X_{n+1}, for any possible n. We'll describe below how one would go about defining a prediction rule in the Bayesian framework in an i.i.d. context — namely, when the observations X_i are assumed to be independent and follow the same (data-generating) law $X_i \sim P_X$ — and then look at various properties that such prediction rules might have. We will restrict the exposition to categorical random variables; namely, the variables $X_1, X_2, \dots$ will take on values in some finite set $\mathcal{X}$. This greatly simplifies the mathematical probabilistic machinery that has to be deployed, but all the concepts readily generalise to more complex spaces (e.g. Euclidean with the usual σ-algebras) [1192].

To discuss and compare the properties of various rules for predicting future observations, it is helpful to precisely define what a *prediction rule* is. If the observations take values in some space $\mathcal{X}$, then a prediction rule is a sequence of functions $(s_0, s_1, s_2, \dots)$ where each s_n maps a finite sequence $(x_1, \dots, x_n) \in \mathcal{X}^n$ of previous observations to a prediction for the next observation. The prediction is probabilistic; namely, each $x_1, \dots, x_n$ maps to a *probability distribution* $s_n(\cdot|x_1, \dots, x_n)$ on $\mathcal{X}$. $s_n(\cdot|x_1, \dots, x_n)$ carries the interpretation of the prediction for the next observation X_{n+1} given the outcomes $(x_1, \dots, x_n)$ for the previous observations $X_1, \dots, X_n$. The following definition encapsulates the requirements:

Definition 49.1. ***Prediction rule (discrete)*** *A sequence $(s_0, s_1, \dots)$ of functions $s_n : \mathcal{X}^{n+1} \to [0, 1]$ is said to be a (discrete) prediction rule if, for every $n \in \mathbb{N}$ and $(x_1, \dots, x_n) \in \mathcal{X}^n$, the mapping $x \mapsto s_n(x|x_1, \dots, x_n)$ is a probability mass function on $\mathcal{X}$. Namely, we require that $s_n(x|x_1, \dots, x_n) \in [0, 1]$ for all $x \in \mathcal{X}$, and $\sum_{x \in \mathcal{X}} s_n(x|x_1, \dots, x_n) = 1$.*

A prediction rule is intended to describe how a practitioner makes predictions about the future observations given the past. In the context of deep learning, a prediction rule might encompass the whole procedure for training a neural network on a dataset of past observations, and then using the trained neural network to make predictions about future observations. For example, in the language modelling case where the examples x_i are sequences of discrete tokens, $s_n(x_{n+1}|x_1, \dots, x_n)$ might be defined as the probability assigned to the sequence x_{n+1} given by an autoregressive model trained on a dataset of examples $(x_1, \dots, x_n)$ following Stochastic Gradient Descent (SGD) with a maximum likelihood objective.[2]

The prediction rule $(s_0, s_1, \dots)$ gives us a way to assign probabilities (predictions) to the sequence of next n observations by defining a joint distribution $s_{1:n}$ like so:

$$s_{1:n}(x_1, \dots, x_n) \stackrel{\text{def}}{=} \prod_{i=1}^{n} s_{i-1}(x_i|x_1, \dots, x_{i-1}). \tag{49.1}$$

In fact, the prediction rule implies a (unique) probability measure on the entire sequence of possible future

[2] The sources of randomness for parameter initialisation and mini-batching would have to be fixed to yield a deterministic prediction rule.

observations $(x_1, x_2, x_3, \dots)$.[3] In this sense, it can be interpreted as an *a priori* prediction for what might happen, even before we observe any data.

Vice versa, we can obtain a prediction rule from any probability measure s over infinite sequences $\mathcal{X}^\infty$ in a natural way by marginalising out to get the conditional of interest. Concretely, if we define a sequence of random variables $(\bar{X}_1, \bar{X}_2, \dots)$ (on some probability space $(\Omega, \mathcal{F}, \mathbb{P})$) to follow the joint law s, we can specify a prediction rule with a sequence of prediction functions $s_n(x_{n+1}|x_1, \dots, x_n) \stackrel{\text{def}}{=} \mathbb{P}[\bar{X}_{n+1} = x_{n+1}|\bar{X}_1 = x_1, \dots, \bar{X}_n = x_n]$. This prediction rule will in turn imply the same joint distribution s on the infinite sequence space. This equivalence between prediction rules and joints over infinite sequences of observations allows us to define properties of prediction rules in terms of the joint they imply over the entire sequence space.

49.1.1 The Bayesian Inferential Approach

In this section, we will go through how we could construct a 'traditionally' Bayesian prediction rule. A prototypical prescription for inference following the Bayesian framework involves: **1)** specifying a joint distribution over all the variables of interest, and then **2)** conditioning on the observed values to obtain a posterior distribution over the unobserved variables of interest. In the context of a sequential prediction problem, one might specify a joint distribution over a sequence of observations in $\mathcal{X}^\infty$, treating the observations as random variables $(X_1, X_2, \dots)$. Then, one would condition on the observed values of $X_1, \dots, X_n$ to obtain a posterior distribution for e.g. X_{n+1}.

In the context of i.i.d. data — where the true observations $(X_1, X_2, \dots)$ are assumed to be independent and identically distributed following the same law $X_i \sim P_X$ — a Bayesian would usually treat the data generating distribution P_X as an unknown, and place a prior on it. To make predictions about the next observation X_{n+1} given observed outcomes $(x_1, \dots, x_n)$ for the previous observations $X_1, \dots, X_n$, one could describe the inferential (explicitly) Bayesian approach as:[4]

I. Specify a prior over the generating distribution;
II. Get a posterior over the generating distribution given the observed data $x_1, \dots, x_n$;
III. Compute the prediction $s_n(\cdot|x_1, \dots, x_n)$ by computing the posterior predictive distribution for X_{n+1}.

For instance, suppose the distributions of interest are described by an unknown parameter Θ, treated as a random variable taking values in $\mathbb{R}^d$, with likelihood $p_{X|\Theta}(x|\theta)$ and prior density $p_\Theta(\theta)$. [5] The procedure above might then look like:

I. Specify a *likelihood* $p_{X|\Theta}(x|\theta)$ and a *prior* density $p_\Theta(\theta)$ over the parameters;
II. Compute the posterior density $p_{\Theta|X_1,\dots,X_n}(\theta|x_1, \dots, x_n) \propto \prod_{i=1}^n p_{X|\Theta}(x_i|\theta) p_\Theta(\theta)$;
III. Compute the prediction $s_n(\cdot|x_1, \dots, x_n) \stackrel{\text{def}}{=} \int p_{X|\Theta}(\cdot|\theta) p_{\Theta|X_1,\dots,X_n}(\theta|x_1, \dots, x_n)\, d\theta$.

In the next section, we will see this is not the only way to obtain prediction rules that are Bayesian.

49.1.2 Implicitly Bayesian Prediction Rules

The explicit computation of the posterior over the parameters can, in many cases, be side-stepped.

For example, consider a Bayesian model for a sequence of paired observations $((X_1, Y_1), (X_2, Y_2), \dots)$, where X_n, Y_n are random variables on $\mathbb{R}^d$ and $\mathbb{R}$ respectively (briefly deviating from the categorical setting). We could specify a Bayesian linear model on (X_n, Y_n), $Y_n = \boldsymbol{\theta}^\intercal X_n + \varepsilon_n$, with a uniform density on the covariates X_n over $[0,1]^d$, a Gaussian prior $\mathcal{N}(\boldsymbol{\theta}; 0, I_d)$ on the weights $\boldsymbol{\theta} \in \mathbb{R}^d$, and Gaussian observation noise $\varepsilon_n \sim \mathcal{N}(0, \sigma^2)$. For $x_{n+1} \in [0,1]^d$, the posterior predictive density for (X_{n+1}, Y_{n+1}) taking the value (x_{n+1}, y_{n+1}), conditioned on the observed values $((X_1, Y_1), \dots, (X_n, Y_n)) = ((x_1, y_1), \dots, (x_n, y_n))$, can then be written as:

$$
\begin{aligned}
s_n((x_{n+1}, y_{n+1})|(x_1, y_1), \dots, (x_n, y_n)) &= \mathcal{N}\left(y_{n+1}; \bar{\mu}, \bar{\sigma}^2\right) \\
\bar{\mu} &= x_{n+1}^\intercal \Phi^\intercal \left(\Phi\Phi^\intercal + \sigma^2 I_n\right)^{-1} y_{1:n}, \\
\bar{\sigma}^2 &= x_{n+1}^\intercal x_{n+1} + \sigma^2 - x_{n+1}^\intercal \Phi^\intercal \left(\Phi\Phi^\intercal + \sigma^2 I_n\right)^{-1} \Phi x_{n+1},
\end{aligned}
$$

[3] Note that this will be a distribution over infinite sequences. In the discrete case, the existence follows from Carathéodory's extension theorem [157].

[4] The "inferential approach" terminology was taken from Berti et al. [138], who distinguish between an explicit prior–posterior construction (the *inferential approach*), and the direct specification of predictive distributions (the *predictive approach*).

[5] We write $p_X(x)$ for the density or probability mass function of a random variable X evaluated at a realisation x, and $p_{X|Y}(x|y)$ for its conditional counterpart given $Y = y$.

with $\Phi = \begin{bmatrix} x_1 & \dots & x_n \end{bmatrix}^\intercal \in \mathbb{R}^{n \times d}$ and $y_{1:n} = \begin{bmatrix} y_1 & \dots & y_n \end{bmatrix}^\intercal$. The uniform density of X_{n+1} equals one on $[0,1]^d$, so its factor disappears from the predictive density shown above.

Following this prediction rule, the practitioner never explicitly *manifests* a posterior over the weights. Nonetheless, they are, of course, acting (making predictions) as *if* they were computing and then marginalising out the posterior. The predictions are the same. In this sense, the prediction rule could be said to be *behaviourally* or *implicitly* Bayesian. The predictive perspective has a long history in statistics [377, 593, 1412]. More recent work has developed direct specifications of prediction rules [133, 135–137, 538, 539], often referred to as the Bayesian *predictive approach* [138].

If prediction rules can behave as if they were following a Bayesian approach, a natural question to ask is: under what conditions is a prediction rule equivalent to an explicitly Bayesian approach for *some* prior and likelihood? In other words, given a prediction rule, can one say whether there exists a prior and likelihood such that the predictions from the prediction rule match those of following the Bayesian framework with that likelihood/prior pair under the i.i.d. assumption? If this is the case, we'll say that the prediction rule is *implicitly Bayesian*. The prediction rule and the corresponding likelihood/prior construction can then be said to be *Bayes-ically* the same.

Implications for Bayesian Deep Learning The fact that prediction rules can be Bayesian without explicitly being constructed as such opens some new avenues for Bayesian deep learning research. It suggests that we could construct predictors in a way that's convenient, with complete disregard for Bayesian principles, and then measure or optimise for how close to being *implicitly* Bayesian they are. For this, we would need some notion of a measure of implicit Bayesianness, which we discuss in later sections. Alternatively, perhaps we could take a non-Bayesian prediction rule, and turn it into a Bayesian one post hoc? For either of those, it is useful to know what actually makes a prediction rule Bayesian. In what follows, we'll look at the properties that the prediction rules defined following the Bayesian framework possess, and the properties that characterise them.

49.2 Characterising Implicitly Bayesian Prediction Rules

In the case of Bayesian models assuming i.i.d. data, de Finetti's theorem gives a simple condition for a prediction rule to be implicitly Bayesian. As mentioned before, a prediction rule implies a unique joint distribution over a sequence of observations $(X_1, X_2, \dots)$. By a version of de Finetti's theorem, under some mild assumptions, a prediction rule is implicitly Bayesian if and only if the joint distribution it implies over $(X_1, X_2, \dots)$ is *exchangeable* [745]:

Definition 49.2 (Exchangeable Sequence of Random Variables)**.** *A finite sequence of n random variables $(X_1, \dots, X_n)$ is said to be exchangeable if for any permutation $\pi : \{1, \dots, n\} \to \{1, \dots, n\}$ the joint distribution of $(X_1, \dots, X_n)$ is the same as the joint distribution of $(X_{\pi(1)}, \dots, X_{\pi(n)})$:*

$$\mathbb{P}[X_1 = x_1, \dots, X_n = x_n] = \mathbb{P}[X_{\pi(1)} = x_1, \dots, X_{\pi(n)} = x_n] \quad \forall (x_1, \dots, x_n) \in \mathcal{X}^n$$

An infinite sequence of random variables $(X_1, X_2, \dots)$ is said to be exchangeable if, for any n, the finite sequence $(X_1, \dots, X_n)$ is exchangeable.

In the binary case, de Finetti's theorem can be stated as:

Theorem 49.1 (de Finetti's theorem)**.** *Consider an infinite sequence of random variables $(X_1, X_2, \dots)$ on $\mathcal{X} = \{0, 1\}$. Then, the following are equivalent:*

1. *There exists a probability measure π on $[0,1]$ (with the usual σ-algebra) such that:*

$$\mathbb{P}[X_1 = x_1, \dots, X_n = x_n] = \int \prod_{i=1}^{n} \theta^{x_i} (1-\theta)^{1-x_i} d\pi(\theta) \quad \forall x_1, \dots, x_n \in \{0,1\} \forall n \in \mathbb{N}$$

2. *The sequence $(X_1, X_2, \dots)$ is exchangeable.*

Of course, condition *1.* above implies that the predictions $s_n(x | x_1, \dots, x_n) = \mathbb{P}[X_{n+1} = x | X_1 = x_1, \dots, X_n = x_n]$ are made by computing and marginalising a posterior:

$$\begin{aligned} \mathbb{P}[X_{n+1} = x | X_1 = x_1, \dots, X_n = x_n] &\overset{(\text{in } x)}{\propto} \mathbb{P}[X_1 = x_1, \dots, X_n = x_n, X_{n+1} = x] \\ &\propto \int \underbrace{\theta^x (1-\theta)^{1-x}}_{\text{likelihood}} \underbrace{\prod_{i=1}^{n} \theta^{x_i} (1-\theta)^{1-x_i} \, d\pi(\theta)}_{\propto \text{posterior}} \end{aligned}$$

Hence, exchangeability is the defining characteristic of implicitly Bayesian prediction rules on i.i.d. data. This suggests one direct way of checking whether a prediction rule is implicitly Bayesian: check whether the joint distribution on $(X_1, X_2, \dots)$ implied by the prediction rule is exchangeable.

49.2.1 Conditionally Identically Distributed

Another desirable property that we might expect of a prediction rule is that the future observations are identically distributed given the past. For example, under the i.i.d. assumption, if we were to observe $(x_1, \dots, x_n)$, the prediction for the next observation X_{n+1} *surely* shouldn't be different from the prediction for the observation after that. After all, we know they are identically distributed, we just don't know what the distribution is; it'd be irrational to make different predictions for X_{n+1}, X_{n+2} given the same data. This property can be formalised as follows:

Definition 49.3 (Conditionally Identically Distributed)**.** *We say that a sequence of random variables $(X_1, X_2, \dots)$ is conditionally identically distributed (c.i.d.) if for any n and any $(x_1, \dots, x_n)$ with a positive probability:*

$$\mathbb{P}\left[X_{n+1} = x | X_1 = x_1, \dots, X_n = x_n\right] = \mathbb{P}\left[X_{n+k} = x | X_1 = x_1, \dots, X_n = x_n\right] \quad \forall k > 0$$

holds almost surely.

A prediction rule is then *c.i.d.* if the joint distribution it implies over $(X_1, X_2, \dots)$ is *c.i.d.*. *c.i.d.* sequences were introduced in [861] and studied and applied in a range of works [134, 135, 540].

It should be clear that each exchangeable sequence is *c.i.d.*, as exchangeability implies $X_1, \dots, X_n, X_{n+1} \overset{d}{=} X_1, \dots, X_n, X_{n+k}$ for any $k > 0$. Hence, the Bayesian framework does yield *c.i.d.* prediction rules, and any implicitly Bayesian prediction rule will be *c.i.d.*. Not all *c.i.d.* sequences are exchangeable, however. *c.i.d.* is a weaker condition than exchangeability.

49.2.2 Stationarity

One property that's missing from just *c.i.d.* sequences that would make them *implicitly Bayesian* is *stationarity*:

Definition 49.4 (Stationary sequence)**.** *A sequence of random variables $(X_1, X_2, \dots)$ is said to be stationary if for any $n, k > 0$:*

$$X_1, \dots, X_n \overset{d}{=} X_{1+k}, \dots, X_{n+k}, \tag{49.2}$$

where $\overset{d}{=}$ denotes equality in distribution.

In the context of prediction rules, stationarity implies sampling k datapoints $x'_1, \dots, x'_k$ from the prediction rule itself and prepending them to the observed data $\mathcal{D} = (x_1, \dots, x_n)$ would yield a prediction rule $s_{n+k}(\cdot | x'_1, \dots, x'_k, \mathcal{D})$ that's in expectation the same as $s_n(\cdot | x_1, \dots, x_n)$.

By the result of Kallenberg [861, Proposition 2.1], exchangeability exactly amounts to stationarity and the *c.i.d.* condition; stationarity and *c.i.d.* properties are another way of characterising implicitly Bayesian prediction rules.

49.2.3 Spreadability

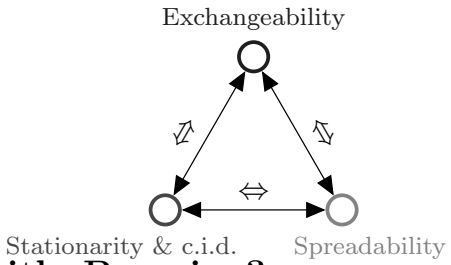
Another condition that turns out to also characterise implicitly Bayesian models is *spreadability*:

Definition 49.5 (Spreadable sequence)**.** *A sequence of random variables $(X_1, X_2, \dots)$ is said to be spreadable if for any $n > 0$ and any sequence of indices $k_n > k_{n-1} > \dots > k_1 \geq 1$:*

$$X_1, \dots, X_n \overset{d}{=} X_{k_1}, \dots, X_{k_n} \tag{49.3}$$

Spreadability says that the distribution of the first n observations is the same as the distribution of any n observations from the sequence. In the context of a prediction rule, it implies that, if we were to construct a new dataset by possibly sampling $k_i \geq 0$ datapoints from the prediction rule conditioned on the previous datapoints in-between "observed" datapoints x_i, x_{i+1}, the final prediction given that dataset would in expectation be the same as the prediction given the original dataset. Kallenberg [861] has shown that spreadability is equivalent to exchangeability. In other words, spreadability is yet another property that characterises implicitly Bayesian prediction rules.

In summary, an implicitly Bayesian prediction rule can be equivalently characterised by *exchangeability*, *spreadability*, or *stationarity* together with the *c.i.d.* condition. Some of these properties might be easier to establish or enforce than others, and each suggests its own way of constructing an implicitly Bayesian prediction rule from a non-Bayesian one. In the next section, we will illustrate how this can be done for the *exchangeability* criterion, and what benefits it can bring.



49.3 Why and how to make predictors implicitly Bayesian?

The equivalence between implicitly Bayesian predictors and "ordinary" Bayesian predictors might be appealing to some readers, but why would a practitioner that has a prediction rule they enjoy care that it's not implicitly Bayesian? If it 'works well' on performance benchmarks, what does it matter? In this section, we'll show that algorithms that are not implicitly Bayesian (ones that lack exchangeability at a fixed horizon) leave some performance on the table. In fact, any non-implicitly Bayesian prediction rule, once restricted to prediction over a finite horizon, has a finitely exchangeable counterpart whose predictive performance is strictly better. Below, we will illustrate a specific approach for taking an arbitrary non-Bayesian prediction rule and turning it into an implicitly Bayesian one.[6] This can be of independent interest as a basis for a practical algorithm, but it will also serve us as a tool for showing non-Bayesian predictors always leave something on the table compared to Bayesian ones.

49.3.1 Turning a prediction rule Bayesian

Consider an arbitrary prediction rule $(s_0, \dots, s_{n-1})$ restricted to a finite horizon $n \in \mathbb{N}$. By Equation (49.1), this implies a joint $s_{1:n}$ on the finite sequence space $\mathcal{X}^n$. We can then define a projection operator Exch_n that takes a joint on $\mathcal{X}^n$, and turns it into a finitely exchangeable joint distribution $\hat{s}_{1:n}$ by averaging over the set Π_n of all possible permutations of $\{1, \dots, n\}$:

$$\hat{s}_{1:n} = \mathrm{Exch}_n(s_{1:n}) \qquad \Leftrightarrow \qquad \hat{s}_{1:n}(x_1, \dots, x_n) = \frac{1}{n!} \sum_{\pi \in \Pi_n} s_{1:n}(x_{\pi(1)}, \dots, x_{\pi(n)}). \tag{49.4}$$

This is a projection, as it will leave an already exchangeable joint unaltered. Similarly, the resulting joint is itself permutation-invariant in its arguments, and hence finitely exchangeable. Finally, from the new joint, we can recover a new finite-horizon prediction rule $(\hat{s}_0, \dots, \hat{s}_{n-1})$ by marginalising in the obvious way:

$$\hat{s}_0(x_1) = \sum_{x_2, \dots, x_n} \hat{s}_{1:n}(x_1, \dots, x_n) \qquad \hat{s}_1(x_2|x_1) = \frac{1}{\hat{s}_0(x_1)} \sum_{x_3, \dots, x_n} \hat{s}_{1:n}(x_1, \dots, x_n) \qquad \dots$$

49.3.2 Exchangeable-ified prediction rules are always better

Below, we show that, when both the original finite-horizon predictive distribution and the exchangeable data-generating distribution have full support, the rule obtained by applying the projection Exch_n is strictly better unless the original finite-horizon rule is already exchangeable (in which case $\mathrm{Exch}_n(s_{1:n}) = s_{1:n}$).

Proposition 49.1. *Consider full-support probability mass functions $s_{1:n}$ and p on a finite sequence space $\mathcal{X}^n$, where p is exchangeable, and take $\hat{s}_{1:n} = \mathrm{Exch}_n(s_{1:n})$. Then:*

$$\mathrm{D}_{\mathrm{KL}}(s_{1:n} \| p) = \mathrm{D}_{\mathrm{KL}}(\hat{s}_{1:n} \| p) + \mathrm{D}_{\mathrm{KL}}(s_{1:n} \| \hat{s}_{1:n}), \tag{49.5}$$

and, in particular, $\mathrm{D}_{\mathrm{KL}}(s_{1:n} \| p) \geq \mathrm{D}_{\mathrm{KL}}(\hat{s}_{1:n} \| p)$ with equality if and only if $s_{1:n}$ is exchangeable.

[6] This will be done, in contrast to the previous sections, for predictions on finite sequences. To our knowledge, it's an open problem as to whether similar constructions can be formulated for infinite sequences, and what the necessary preconditions might be.

Proof. We can expand $\mathrm{D}_{\mathrm{KL}}(s_{1:n}\|p)$ in the following way:

$$\begin{aligned}
\mathrm{D}_{\mathrm{KL}}(s_{1:n}\|p) &= \sum_{x_{1:n}\in\mathcal{X}^n} s_{1:n}(x_1,...,x_n)\log\frac{s_{1:n}(x_1,...,x_n)}{p(x_1,...,x_n)} \\
&= \sum_{x_{1:n}\in\mathcal{X}^n} s_{1:n}(x_1,...,x_n)\left(\log\frac{s_{1:n}(x_1,...,x_n)}{\hat{s}_{1:n}(x_1,...,x_n)} + \log\frac{\hat{s}_{1:n}(x_1,...,x_n)}{p(x_1,...,x_n)}\right) \\
&= \mathrm{D}_{\mathrm{KL}}(s_{1:n}\|\hat{s}_{1:n}) + \sum_{x_{1:n}\in\mathcal{X}^n} s_{1:n}(x_1,...,x_n)\log\frac{\hat{s}_{1:n}(x_1,...,x_n)}{p(x_1,...,x_n)}
\end{aligned}$$

Introduce an average over all possible permutations Π_n

$$= \mathrm{D}_{\mathrm{KL}}(s_{1:n}\|\hat{s}_{1:n}) + \frac{1}{n!}\sum_{\pi\in\Pi_n}\sum_{x_{1:n}\in\mathcal{X}^n} s_{1:n}(x_1,...,x_n)\log\frac{\hat{s}_{1:n}(x_1,...,x_n)}{p(x_1,...,x_n)}$$

Applying a change of variables $(x_1,\ldots,x_n)\mapsto(x_{\pi(1)},\ldots,x_{\pi(n)})$ to the inner summation:

$$= \mathrm{D}_{\mathrm{KL}}(s_{1:n}\|\hat{s}_{1:n}) + \frac{1}{n!}\sum_{\pi\in\Pi_n}\sum_{x_{1:n}\in\mathcal{X}^n} s_{1:n}(x_{\pi(1)},...,x_{\pi(n)})\log\frac{\hat{s}_{1:n}(x_{\pi(1)},...,x_{\pi(n)})}{p(x_{\pi(1)},...,x_{\pi(n)})}$$

By exchangeability of $\hat{s}_{1:n}$ and p we can 'unpermute' the elements:

$$\begin{aligned}
&= \mathrm{D}_{\mathrm{KL}}(s_{1:n}\|\hat{s}_{1:n}) + \frac{1}{n!}\sum_{\pi\in\Pi_n}\sum_{x_{1:n}\in\mathcal{X}^n} s_{1:n}(x_{\pi(1)},...,x_{\pi(n)})\log\frac{\hat{s}_{1:n}(x_1,...,x_n)}{p(x_1,...,x_n)} \\
&= \mathrm{D}_{\mathrm{KL}}(s_{1:n}\|\hat{s}_{1:n}) + \sum_{x_{1:n}\in\mathcal{X}^n}\left(\frac{1}{n!}\sum_{\pi\in\Pi_n} s_{1:n}(x_{\pi(1)},...,x_{\pi(n)})\right)\log\frac{\hat{s}_{1:n}(x_1,...,x_n)}{p(x_1,...,x_n)} \\
&= \mathrm{D}_{\mathrm{KL}}(s_{1:n}\|\hat{s}_{1:n}) + \sum_{x_{1:n}\in\mathcal{X}^n} \hat{s}_{1:n}(x_1,...,x_n)\log\frac{\hat{s}_{1:n}(x_1,...,x_n)}{p(x_1,...,x_n)} \\
&= \mathrm{D}_{\mathrm{KL}}(s_{1:n}\|\hat{s}_{1:n}) + \mathrm{D}_{\mathrm{KL}}(\hat{s}_{1:n}\|p)
\end{aligned}$$

Clearly, by known properties of the KL-divergence, the gap $\mathrm{D}_{\mathrm{KL}}(s_{1:n}\|\hat{s}_{1:n})$ will be 0 if and only if the two distributions are the same. The distributions will only be the same if $s_{1:n}$ is already exchangeable. This completes the proof. □

An equivalent claim can be made for the reverse KL-divergence:

Proposition 49.2. *Consider full-support probability mass functions $s_{1:n}$ and p on a finite sequence space $\mathcal{X}^n$, where p is exchangeable, and take $\hat{s}_{1:n} = \mathrm{Exch}_n(s_{1:n})$. Then:*

$$\mathrm{D}_{\mathrm{KL}}(p\|s_{1:n}) \geq \mathrm{D}_{\mathrm{KL}}(p\|\hat{s}_{1:n}), \tag{49.6}$$

with equality if and only if $s_{1:n}$ is exchangeable.

Proof. The proof rests on applying Jensen's inequality to the term:

$$\log\left(\frac{1}{n!}\sum_{\pi\in\Pi_n} s_{1:n}\left(x_{\pi(1)},...,x_{\pi(n)}\right)\right) \geq \frac{1}{n!}\sum_{\pi\in\Pi_n}\log\left(s_{1:n}\left(x_{\pi(1)},...,x_{\pi(n)}\right)\right)$$

Concretely, by expanding $\mathrm{D}_{\mathrm{KL}}(p\|\hat{s}_{1:n})$ we get:

$$\begin{aligned}
\mathrm{D}_{\mathrm{KL}}(p\|\hat{s}_{1:n}) &= \sum_{x_{1:n}\in\mathcal{X}^n} p(x_1,...,x_n)\log\frac{p(x_1,...,x_n)}{\hat{s}_{1:n}(x_1,...,x_n)} \\
&= \sum_{x_{1:n}\in\mathcal{X}^n} p(x_1,...,x_n)\left(\log p(x_1,...,x_n) - \log\left(\frac{1}{n!}\sum_{\pi\in\Pi_n} s_{1:n}(x_{\pi(1)},...,x_{\pi(n)})\right)\right)
\end{aligned}$$

By Jensen's inequality (since log is concave, $\log(\mathbb{E}[X]) \geq \mathbb{E}[\log(X)]$):

$$\begin{aligned}
&\leq \sum_{x_{1:n}\in\mathcal{X}^n} p(x_1,...,x_n)\left(\log p(x_1,...,x_n) - \frac{1}{n!}\sum_{\pi\in\Pi_n}\log s_{1:n}(x_{\pi(1)},...,x_{\pi(n)})\right)\\
&= \frac{1}{n!}\sum_{\pi\in\Pi_n}\sum_{x_{1:n}\in\mathcal{X}^n} p(x_1,...,x_n)\left(\log p(x_1,...,x_n) - \log s_{1:n}(x_{\pi(1)},...,x_{\pi(n)})\right)\\
&= \frac{1}{n!}\sum_{\pi\in\Pi_n}\sum_{x_{1:n}\in\mathcal{X}^n} p(x_1,...,x_n)\log\frac{p(x_1,...,x_n)}{s_{1:n}(x_{\pi(1)},...,x_{\pi(n)})}
\end{aligned}$$

Change of variables $(x_1,\ldots,x_n)\mapsto(x_{\pi^{-1}(1)},\ldots,x_{\pi^{-1}(n)})$ in the inner summation:

$$= \frac{1}{n!}\sum_{\pi\in\Pi_n}\sum_{x_{1:n}\in\mathcal{X}^n} p(x_{\pi^{-1}(1)},...,x_{\pi^{-1}(n)})\log\frac{p(x_{\pi^{-1}(1)},...,x_{\pi^{-1}(n)})}{s_{1:n}(x_1,...,x_n)}$$

By exchangeability of p, $p(x_{\pi^{-1}(1)},...,x_{\pi^{-1}(n)}) = p(x_1,...,x_n)$:

$$\begin{aligned}
&= \frac{1}{n!}\sum_{\pi\in\Pi_n}\sum_{x_{1:n}\in\mathcal{X}^n} p(x_1,...,x_n)\log\frac{p(x_1,...,x_n)}{s_{1:n}(x_1,...,x_n)}\\
&= \frac{1}{n!}\sum_{\pi\in\Pi_n} \mathrm{D}_{\mathrm{KL}}(p\|s_{1:n})\\
&= \mathrm{D}_{\mathrm{KL}}(p\|s_{1:n})
\end{aligned}$$

For strictly concave functions (such as log), Jensen's inequality becomes an equality only if the averaged terms are all the same. For the inequality above, this implies we get an equality if and only if:

$$s_{1:n}\left(x_{\pi(1)},...,x_{\pi(n)}\right) = s_{1:n}\left(x_1,...,x_n\right),$$

for all $\pi\in\Pi_n$, which completes the proof of the proposition. □

Related arguments in the broader setting of statistical decision theory go back to Wald [1693].

On the basis of these results, one can make a compelling case for being Bayesian without alluding to uncertainty quantification properties. Even if you only care about predictive performance, you would still be universally better off with *some* Bayesian algorithm, at least according to some metrics!

49.3.3 Non-implicitly Bayesian agents are vulnerable to adversarial bets

As a final argument in favour of Bayesian prediction rules, we show that by following non-Bayesian prediction rules, one is vulnerable to accepting seemingly favourable bets that will in expectation always be disadvantageous. An adversary can come up with such bets by, once again, utilising the construction in Equation (49.4), and exploiting any incoherence in the predictive distribution.[7]

Consider a betting game in which two players — an agent and an adversary — make bets on the next sequence of n observations, each observation taking value in some space $\mathcal{X}$. At the end, a transfer of funds happens between the agent and the adversary depending on the observed outcomes.

Formally, a *bet* is a function $r:\mathcal{X}^n\to\mathbb{R}$. The value of r can be thought of as representing the transfer of funds from the adversary to the agent for each possible outcome; i.e. if the observed outcome is $x_1,\ldots,x_n$ and $r(x_1,\ldots,x_n)$ is positive, the agent receives $r(x_1,\ldots,x_n)$ from the adversary. If, on the other hand, the observed outcome is $x_1,\ldots,x_n$ and $r(x_1,\ldots,x_n)$ is negative, the agent gives $-r(x_1,\ldots,x_n)$ to the adversary.

We will consider players that make predictions on the sequence of observations, and choose which bets to accept based on those beliefs. Concretely, *agent beliefs* and *adversary beliefs* are probability mass functions on $\mathcal{X}^n$.

Definition 49.6 (Bets favourable to agent)**.** *Given agent beliefs $q:\mathcal{X}^n\to[0,1]$, a function $r:\mathcal{X}^n\to\mathbb{R}$ is said to be a **bet favourable to the agent** if:*

$$\sum_{x_{1:n}\in\mathcal{X}^n} r(x_{1:n})q(x_{1:n}) > 0. \tag{49.7}$$

[7] A similar result could be shown to follow directly from Proposition 49.2, which would be akin to many known information-theoretic betting results.

In other words, a bet is favourable to the agent if the 'expected return' from the point of view of the agent is strictly positive.

Definition 49.7 (Bets favourable to adversary)**.** *Given adversary beliefs $\bar{q} : \mathcal{X}^n \to [0,1]$, a function $r : \mathcal{X}^n \to \mathbb{R}$ is said to be a* ***bet favourable to the adversary*** *if:*

$$\sum_{x_{1:n} \in \mathcal{X}^n} (-r(x_{1:n}))\bar{q}(x_{1:n}) > 0. \tag{49.8}$$

Definition 49.8 (Admissible bets)**.** *Given agent and adversary beliefs, a bet r is said to be an* ***admissible bet*** *if it is simultaneously a bet favourable to the agent and a bet favourable to the adversary.*

Definition 49.9 (Minimal bets)**.** *Given agent beliefs $q : \mathcal{X}^n \to [0,1]$, and adversary beliefs $\bar{q} : \mathcal{X}^n \to [0,1]$, a function $r : \mathcal{X}^n \to \mathbb{R}$ is said to be a* ***minimal bet*** *if it's a linear combination of agent and adversary beliefs, namely:*

$$r(x_{1:n}) = aq(x_{1:n}) + b\bar{q}(x_{1:n}) \qquad \text{for some } a, b \in \mathbb{R}. \tag{49.9}$$

The rationale for this is that adding components to the bet that are orthogonal to the beliefs of the agent and the adversary will not change the expected return of the bet for either of them. In other words, for any bet r that is not minimal, there exists a 'smaller' minimal bet r' such that the expected returns of r and r' are the same for both the agent and the adversary. Restricting ourselves to minimal bets hence simply ensures that the agents do not arbitrarily make bets that are not justified on the grounds of their beliefs.

Given a bet function $r : \mathcal{X}^n \to \mathbb{R}$, and a *true distribution* $p : \mathcal{X}^n \to [0,1]$, the game proceeds by sampling observations $X_1, \ldots, X_n$ from p — i.e. $(X_1, \ldots, X_n) \sim p$ — and at the end of the game a transfer of funds occurs. The *reward to the agent* is defined as $r(X_1, \ldots, X_n)$, and the *expected return to the agent* is: $\mathbb{E}[r((X_1, \ldots, X_n))] = \sum_{x_{1:n} \in \mathcal{X}^n} p(x_{1:n}) r(x_{1:n})$.

Definition 49.10 (ℓ_2 norm)**.** *For functions $f : \mathcal{X}^n \to \mathbb{R}$, the ℓ_2 norm of f, written $\|f\|_2$, is defined as $\|f\|_2 = \sqrt{\sum_{x_{1:n} \in \mathcal{X}^n} f(x_{1:n})^2}$.*

We will consider the case of an adversary that constructs an "exchangeable-ified" version of the agent's beliefs: $\bar{q} = \mathrm{Exch}_n(q)$ as per Equation (49.4). One can interpret this as a scenario in which the adversary doesn't know the true probabilities of the events, but knows the beliefs of the agent, and so they are able to exploit a possible lack of exchangeability in the agent's beliefs.

The theorem we will state below roughly says "if a sensible bet is made between the agent and the adversary, then *no matter what the true distribution is* the expected return for the agent is strictly negative". The minimality of the bet is required for this to hold: if the bets are allowed to not be minimal, then we could find some true distribution that rewards one of the bettors despite them having no 'edge' or advantage in terms of their beliefs.

This brings us to the following result, which we will now prove:

Theorem 49.2. *Given an agent with beliefs $q : \mathcal{X}^n \to [0,1]$, and an adversary with "exchangeable-ified" beliefs $\bar{q} = \mathrm{Exch}_n(q)$ as defined in Equation (49.4), for any exchangeable true distribution $p : \mathcal{X}^n \to [0,1]$ whose support overlaps that of q, all minimal and admissible bets r have strictly negative expected return for the agent under p:*

$$\sum_{x_{1:n} \in \mathcal{X}^n} p(x_{1:n}) r(x_{1:n}) < 0 \tag{49.10}$$

Furthermore, either:

- *q is exchangeable and $\bar{q} = q$, and there are* no *admissible bets.*
- *q is not exchangeable and there exist admissible (and minimal) bets.*

In particular, Theorem 49.2 holds when p is an i.i.d. distribution.

Proof. **A note on notation** It is helpful to abbreviate the notation in parts of the proof by thinking of the agent and adversary beliefs, as well as bets, as vectors. If we enumerate all the elements of $\mathcal{X}^n$, we can represent functions from $\mathcal{X}^n$ to $\mathbb{R}$ as vectors in $\mathbb{R}^{|\mathcal{X}^n|}$. For example, if $\mathcal{X} = \{0,1\}$, $n = 2$, and the chosen ordering of elements of $\mathcal{X}^n$ is $((0,0),(0,1),(1,0),(1,1))$, then the function r can be represented as a vector in $\mathbb{R}^4$: $r = \begin{bmatrix} r(0,0) & r(0,1) & r(1,0) & r(1,1) \end{bmatrix}^\intercal$. In this proof, we will switch freely between the functions and vectors for the same function, as it should always be apparent from the context what the counterpart is.

For any function $f : \mathcal{X}^n \to \mathbb{R}$ that's permutation-invariant in its arguments we have that:

$$\begin{aligned}\mathbf{f}\cdot\bar{\mathbf{q}} &= \sum_{x_{1:n}\in\mathcal{X}^n} f(x_{1:n})\bar{q}(x_{1:n}) = \sum_{x_{1:n}\in\mathcal{X}^n} f(x_{1:n})\left(\frac{1}{|\Pi_n|}\sum_{\pi\in\Pi_n} q(\pi(x_{1:n}))\right)\\ &= \frac{1}{|\Pi_n|}\sum_{\pi\in\Pi_n}\sum_{x_{1:n}\in\mathcal{X}^n} f(x_{1:n})q(\pi(x_{1:n})) = \frac{1}{|\Pi_n|}\sum_{\pi\in\Pi_n}\sum_{x_{1:n}\in\mathcal{X}^n} f(\pi(x_{1:n}))q(\pi(x_{1:n}))\\ &= \frac{1}{|\Pi_n|}\sum_{\pi\in\Pi_n}\sum_{x_{1:n}\in\mathcal{X}^n} f(x_{1:n})q(x_{1:n}) = \sum_{x_{1:n}\in\mathcal{X}^n} f(x_{1:n})q(x_{1:n}) = \mathbf{f}\cdot\mathbf{q},\end{aligned}$$

where the second to last equality holds because permuting the elements of both vectors in a dot product $\mathbf{f}\cdot\mathbf{q}$ doesn't change the result. Hence:

$$\boldsymbol{p}\cdot\bar{\mathbf{q}} = \boldsymbol{p}\cdot\mathbf{q} \qquad\qquad \bar{\mathbf{q}}\cdot\bar{\mathbf{q}} = \mathbf{q}\cdot\bar{\mathbf{q}} \tag{49.11}$$

I. All admissible and minimal bets have negative expected return. Suppose r is minimal and admissible. Then, by minimality, r can be represented as:

$$r = a\mathbf{q} + b\bar{\mathbf{q}}, \tag{49.12}$$

for some constants $a, b \in \mathbb{R}$. By admissibility of r, we can further get the following inequality for a, b:

$$\begin{aligned}&\mathbf{q}\cdot r > 0 && \Rightarrow a(\mathbf{q}\cdot\mathbf{q}) + b(\mathbf{q}\cdot\bar{\mathbf{q}}) && > 0 && (49.13)\\ &\bar{\mathbf{q}}\cdot r < 0 && \Rightarrow a(\bar{\mathbf{q}}\cdot\mathbf{q}) + b(\bar{\mathbf{q}}\cdot\bar{\mathbf{q}}) = a(\bar{\mathbf{q}}\cdot\mathbf{q}) + b(\bar{\mathbf{q}}\cdot\mathbf{q}) && < 0 &&\\ & && \Rightarrow a + b && < 0. && (49.14)\end{aligned}$$

as $\mathbf{q}\cdot\bar{\mathbf{q}} > 0$. By (49.12), we can rewrite the expected return (for the agent) as:

$$\boldsymbol{p}\cdot r = a\boldsymbol{p}\cdot\mathbf{q} + b\boldsymbol{p}\cdot\bar{\mathbf{q}} = a\boldsymbol{p}\cdot\mathbf{q} + b\boldsymbol{p}\cdot\mathbf{q} = (a+b)(\boldsymbol{p}\cdot\mathbf{q}),$$

where the second to last equality follows by (49.11).

Now, $\boldsymbol{p}\cdot\mathbf{q} > 0$ as long as $\boldsymbol{p}$ and $\mathbf{q}$ have overlapping support, and by (49.14), $a + b < 0$, hence:

$$\boldsymbol{p}\cdot r < 0,$$

as required.

II. Minimal and admissible bets exist if and only if $\mathbf{q} \neq \bar{\mathbf{q}}$. Suppose that $\mathbf{q} = \bar{\mathbf{q}}$. An admissible bet must satisfy:

$$\begin{aligned}0 &< r\cdot\mathbf{q}\\ 0 &> r\cdot\bar{\mathbf{q}} = r\cdot\mathbf{q},\end{aligned}$$

which is a contradiction, and so $\mathbf{q} = \bar{\mathbf{q}}$ implies that no admissible bet exists.

Now, suppose that $\mathbf{q} \neq \bar{\mathbf{q}}$. Note that:

$$\begin{aligned}\|\bar{\mathbf{q}}\|_2^2 &= \sum_{x_{1:n}\in\mathcal{X}^n}\left(\frac{1}{|\Pi_n|}\sum_{\pi\in\Pi_n} q(\pi(x_{1:n}))\right)^2 && \text{(apply Jensen's inequality)}\\ &\leq \sum_{x_{1:n}\in\mathcal{X}^n}\frac{1}{|\Pi_n|}\sum_{\pi\in\Pi_n} q(\pi(x_{1:n}))^2 = \sum_{x_{1:n}\in\mathcal{X}^n} q(x_{1:n})^2 = \|\mathbf{q}\|_2^2,\end{aligned}$$

and so, $\mathbf{q}\cdot\mathbf{q} > \mathbf{q}\cdot\bar{\mathbf{q}}$ (strict inequality, as $\mathbf{q} \neq \bar{\mathbf{q}}$). Hence, picking $a = \left(\frac{\mathbf{q}\cdot\bar{\mathbf{q}}}{\mathbf{q}\cdot\mathbf{q}}\right)^{\frac{1}{2}}$ and $b = -1$ yields an admissible and minimal bet $r = a\mathbf{q} + b\bar{\mathbf{q}}$, because:

$$a(\mathbf{q}\cdot\mathbf{q}) + b(\mathbf{q}\cdot\bar{\mathbf{q}}) = (\mathbf{q}\cdot\bar{\mathbf{q}})^{\frac{1}{2}}\overbrace{(\mathbf{q}\cdot\mathbf{q})^{\frac{1}{2}}}^{>(\mathbf{q}\cdot\bar{\mathbf{q}})^{\frac{1}{2}}} - (\mathbf{q}\cdot\bar{\mathbf{q}}) > 0$$

$$a + b = \underbrace{\left(\frac{\mathbf{q}\cdot\bar{\mathbf{q}}}{\mathbf{q}\cdot\mathbf{q}}\right)^{\frac{1}{2}}}_{<1} - 1 < 0.$$

Hence, whenever $\mathbf{q} \neq \bar{\mathbf{q}}$ we can find a minimal and admissible bet as required. □

In the context of prediction rules, one can see the above result as a failing of non-implicitly Bayesian ones; such prediction rules would induce non-exchangeable distributions over finite sequences of observations, and hence an agent that derives their beliefs from such a prediction rule would be vulnerable to the adversarial construction in Theorem 49.2.

49.4 Measuring Implicit Bayesianness

In the previous sections, we discussed what characterises a strictly implicitly-Bayesian prediction rule, how to turn a finite-horizon non-Bayesian rule into a Bayesian one, and what one leaves on the table by not doing so. Many believe that some ways of using deep learning, such as in-context learning, are already approximately Bayesian [1786]; see also Chapter 43. How could one verify whether this is the case? And how could one quantify how close it is to being true? We briefly digress in this section, to summarise how the findings from previous sections could be used towards the goal of *measuring* implicit Bayesianness.

The KL-gap The results in Propositions 49.1 and 49.2 directly suggest a way to quantify deviation from exchangeability (and hence implicit Bayesianness) at a fixed horizon by measuring how much performance is left on the table by not using the finitely exchangeable projection. For instance, Proposition 49.1 says that, for any full-support i.i.d. true data-generating distribution, we could 'gain' $\mathrm{D}_{\mathrm{KL}}(s_{1:n}\|\hat{s}_{1:n})$ in performance — as measured by reverse-KL proximity to p — by turning our finite-horizon prediction rule into an exchangeable one. Hence, this performance gap $\mathrm{D}_{\mathrm{KL}}(s_{1:n}\|\hat{s}_{1:n})$ could be used as a measure of deviation from exchangeability at horizon n. The gap is agnostic to the choice of exchangeable, full-support p. A prediction rule is only strictly exchangeable if the gap is 0 for all n. Nonetheless, a practitioner could pick some large n, and the metric is otherwise readily operationalisable.

The exchangeability gap In previous work, Mlodozeniec et al. [1192] have proposed directly measuring the average variation in the joint distribution implied by a prediction rule as the order of the elements in the sequence is varied. Of course, for an exchangeable prediction rule, the variation should be 0. Concretely, Mlodozeniec et al. [1192] suggested measuring the *variance* of the log-joint as they randomly sample permutations of the data uniformly at random. Denoting with Π a random variable that takes values in the set of all permutations of $\{1,\dots,n\}$ with equal probability, the measure was:

$$\mathbb{V}\left[\log \prod_{i=0}^{n-1} s_i(x_{\Pi(i+1)}|x_{\Pi(1)},\dots,x_{\Pi(i)})\right] = \mathbb{V}\left[\sum_{i=0}^{n-1} \log s_i(x_{\Pi(i+1)}|x_{\Pi(1)},\dots,x_{\Pi(i)})\right], \tag{49.15}$$

for a given sequence of datapoints $(x_1,\dots,x_n)$. The metric would then be averaged over different choices of $x_1,\dots,x_n \in \mathcal{X}^n$.

Arguably, the KL-gap metric has a stronger motivation, as it can be directly connected to desirable properties that being implicitly Bayesian brings. Nonetheless, Mlodozeniec et al. [1192] operationalised and computed the exchangeability metric for several practical algorithms, illustrating some counter-intuitive results, which we recapitulate and summarise in Figure 49.1.

Figure 49.1 shows empirically evaluated measures of implicit Bayesianness from [1192] for several Bayesian and non-Bayesian methods. They include a 'strong', but expensive, BNN baseline using Hamiltonian Monte Carlo inference (*"BNN MLP (HMC)"* in the plot), as well as a much more scalable but cruder Laplace approximation (*"BNN MLP (Laplace approx.)"* in the plot). These are compared with *exact* Gaussian Processes, as well as Gaussian processes where kernel hyperparameters are updated by maximising marginal likelihood. The latter is arguably much closer to how Gaussian Processes are used in practice. The plot also includes a regular MLP trained with SGD — with the same architecture as that used for the BNN methods — for reference.

Interestingly, the plot shows that the Laplace approximation for BNNs was *more* implicitly Bayesian than exact Gaussian process inference when the hyperparameters were fitted by maximising the marginal likelihood, following common practice. This shows that how strong of a Bayesian motivation an algorithm might *seem* to have might not necessarily correlate with how close to behaving like a Bayesian method it actually is.

49.5 Conclusion

In conclusion, this chapter summarises a different perspective on integrating Bayesian principles into deep learning, shifting the focus from explicit model specification to incorporating Bayesian principles at a *behavioural* level. We have shown what characterises Bayesian predictions, argued why Bayesian predictions are desirable in the first place, and have illustrated avenues for making an existing predictive algorithm Bayesian without engaging in explicit prior and likelihood construction.

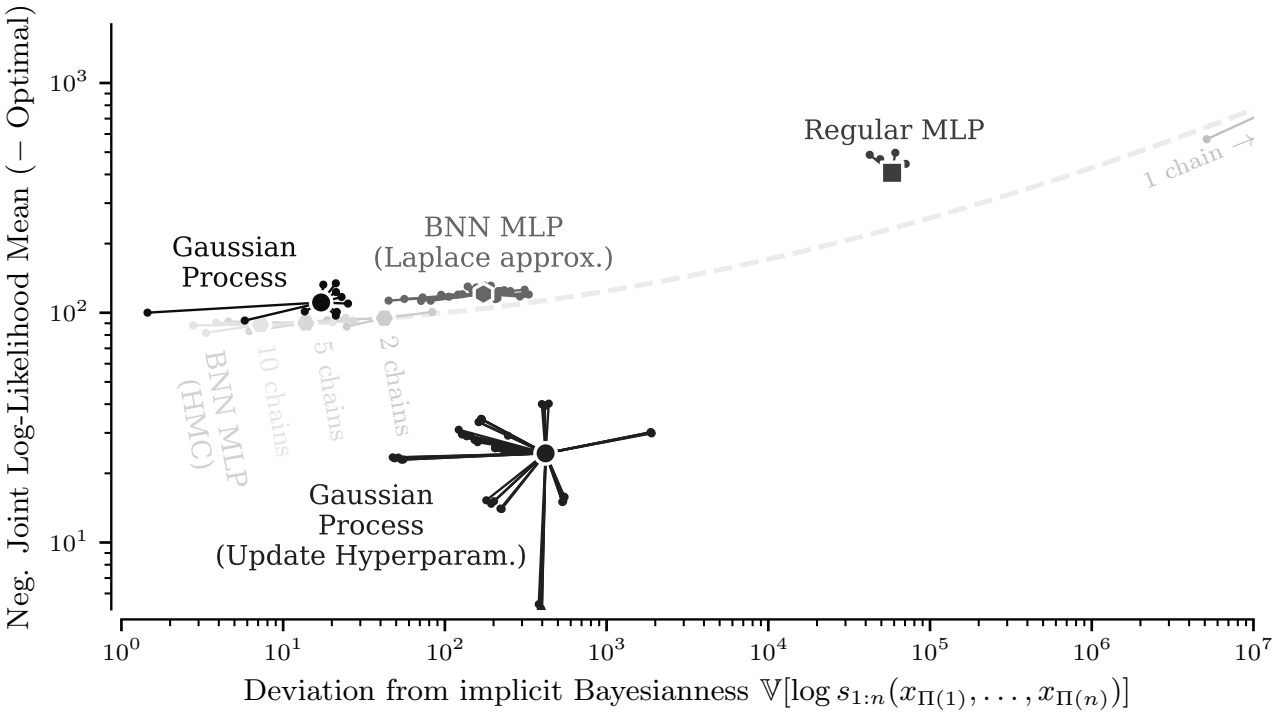


Figure 49.1: Empirically measuring implicit Bayesianness (x-axis) together with log-likelihood performance (y-axis) for several Bayesian Neural Network, Gaussian Process, and regular deep learning training configurations. Small dots indicate the metrics computed for an individual sequence of points, with a big dot at their centre indicating the average over different sequences of datapoints drawn from the regression task in [1192].

Chapter 50

Diffusion models

50.1 Introduction

Generative modelling, a cornerstone of modern machine learning, seeks to build models that can capture and sample from complex data distributions $p(\mathbf{x})$, such as those over natural images, audio waveforms, or molecular structures. In recent years, a new family of models, known as denoising diffusion models, has emerged, demonstrating an ability to generate high-fidelity, diverse samples that surpass the quality of other established model classes.

50.1.1 Defining Diffusion Models

A diffusion model is a latent variable model that generates samples from the data distribution $p(\mathbf{x})$ by iteratively removing noise from a sample generated from a tractable prior distribution. To train such a denoiser, one needs to obtain noisy samples with varying degrees of noise. From the perspective of non-equilibrium thermodynamics [1532], the noisy samples can be viewed as samples from an evolving system that moves from a low-entropy state towards a high-entropy state, for example, the diffusion of a gas in a container starting from some initial state. Concretely, a diffusion model consists of two discrete-time processes: a fixed forward noising process and a learned denoising process. The noising process $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$ is Markovian and fixed, that is, it has no learnable parameters. It iteratively adds noise to a sample $\mathbf{x}_0 \sim q_{\text{data}}(\mathbf{x})$ over a sequence of timesteps $t = 1, \ldots, T$. The noising process is defined such that its terminal distribution, $q(\mathbf{x}_T)$, converges to a simple, tractable prior distribution, such as an isotropic Gaussian $\mathcal{N}(\mathbf{0}, \mathbf{I})$. The forward and reverse processes are illustrated in Figure 50.1 for a single data point $\mathbf{x}_0$.

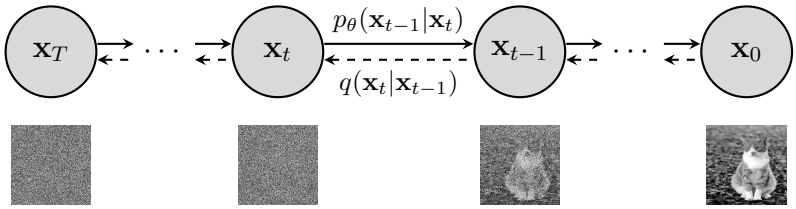


Figure 50.1: The forward process q gradually adds noise to the data, while the reverse process p_θ learns to denoise. Images show the progressive noising from $\mathbf{x}_0$ (clean) to $\mathbf{x}_T$ (pure Gaussian noise).

The denoising process $p_\theta(\mathbf{x}_{0:T})$ is a Markovian, parameterised process that iteratively removes noise from a sample $\mathbf{x}_T \sim q(\mathbf{x}_T)$ over a sequence of timesteps $t = T, \ldots, 1$. It is parameterised by a neural network θ. Starting from a sample $\mathbf{x}_T \sim q(\mathbf{x}_T)$, the denoising process learns to perform a sequence of small *denoising* steps $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$, such that the terminal distribution $p_\theta(\mathbf{x}_0)$ resembles the data distribution $q_{\text{data}}(\mathbf{x})$. To this end, the denoiser is trained to minimise the KL divergence between the denoised distribution and the data distribution, that is, $\min_\theta \mathrm{D}_{\mathrm{KL}}(p_\theta(\mathbf{x}_0) \| q_{\text{data}}(\mathbf{x}))$.

50.1.2 Context within Probabilistic Generative Models

The rise of diffusion models can be understood by comparing them to the other families of generative models, each of which embodies a different set of trade-offs.

- **Variational Autoencoders (VAEs):** VAEs, like diffusion models, are latent variable models trained to optimise the Evidence Lower Bound (ELBO), a lower bound on the data log-likelihood $\log p(\mathbf{x})$. They consist of an encoder $q_\phi(\mathbf{z}|\mathbf{x})$ that maps data to a latent space and a decoder $p_\theta(\mathbf{x}|\mathbf{z})$ that reconstructs the data. VAEs provide a principled latent-variable framework with tractable variational training, but samples from standard VAE decoders are often reported to be overly smooth or of lower perceptual fidelity, especially when trained with pixelwise reconstruction objectives [892, 976].

- **Generative Adversarial Networks (GANs):** GANs reframe generative modelling as a two-player game between a generator $G_\theta(\mathbf{z})$ and a discriminator $D_\phi(\mathbf{x})$. The generator learns to transform prior noise $\mathbf{z}$ into samples, while the discriminator learns to distinguish the real data samples from the generated samples. At equilibrium, the generator produces samples indistinguishable from the true data. GANs produce sharper, higher-fidelity samples compared to VAEs. However, the adversarial objective is notoriously difficult to optimise [264], and these GANs suffer from mode collapse, where the generator learns to produce only a few high-quality samples. Furthermore, they lack a direct mechanism for estimating the likelihood of a data point.

- **Normalising Flows:** Discrete-time Normalising Flows model the data distribution directly by constructing an invertible, differentiable transformation $f_\theta : \mathcal{X} \to \mathcal{Z}$ that maps data $\mathbf{x}$ to a simple latent distribution $p_Z(\mathbf{z})$ (e.g., a Gaussian). The data likelihood can then be computed exactly using the change of variables formula: $\log p_X(\mathbf{x}) = \log p_Z(f(\mathbf{x})) + \log|\det(\mathbf{J}_f(\mathbf{x}))|$, where $\mathbf{J}_f$ is the Jacobian of f. This enables flows to be trained directly via maximum likelihood. While discrete-time normalising flows offer tractable likelihoods and exact latent inference, the need for an explicitly invertible f_θ with a tractable Jacobian determinant imposes significant architectural constraints. Continuous-time normalising flows relax this requirement by defining the map as the solution of an ODE, so that the network only models the vector field, and the log-density is then obtained from the instantaneous change of variables formula [272]. This continuous-time view is closely related to the probability flow ODE derived later in this chapter (Section 50.5.1), and to flow matching, which trains a neural vector field to transport noise to data without simulating an ODE during training [1054].

Diffusion models present a compelling alternative to the trilemma of sample quality, training stability, and likelihood estimation. They are trained with a stable objective similar to VAEs, which can be simplified to a simple regression loss. They produce state-of-the-art sample quality that rivals or exceeds that of GANs. Their primary drawback has been their slow sampling speed: generating a single sample requires T sequential evaluations of the neural network (where T is on the order of 1,000), making them orders of magnitude slower than single-shot models like VAEs or GANs. However, as we will discuss in Section 50.6.1, this limitation has been largely mitigated by recent algorithmic advancements.

50.1.3 Chapter Objectives and Roadmap

This chapter will provide a comprehensive, step-by-step derivation and analysis of denoising diffusion models.

1. We will begin by formalising the mathematical formulation of the fixed **forward perturbation process** (Section 50.2) and the parameterised **reverse generative process** in discrete time (Section 50.3).

2. We will then derive the **variational objective (ELBO)** for this model class (Section 50.4). This section contains the central derivation of the chapter, showing how the complex ELBO simplifies to a tractable, weighted regression objective. This is the key insight that makes diffusion models practical.

3. With the discrete-time model established, we will connect this formulation to its continuous-time limit (Section 50.5), where we will see that both the forward process and the reverse process can be seen as solutions to specific **Stochastic Differential Equations (SDEs)**. We will see how the coefficients of the SDE for the reverse process can be estimated using **denoising score matching**.

4. Finally, we will review the critical practical advancements that have made diffusion models the state-of-the-art, including **accelerated sampling algorithms (DDIM)**, methods for **conditional generation (guidance)**, and extensions to **discrete data domains** (Section 50.6 and Section 50.7).

50.2 The Forward Process: The Perturbation Kernel

The forward or the noising process represents the *"diffusion"* in diffusion models. This process is not learned; it is a fixed, predefined iterative mechanism for destroying the structure in a data point $\mathbf{x}_0$ and transforming it into pure noise.

50.2.1 A Fixed Markovian Process

The forward process $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$ is defined as a finite horizon discrete-time Markov chain of length T. At each timestep $t \in [1, T]$, a small amount of Gaussian noise is added to the sample from the previous timestep, $\mathbf{x}_{t-1}$, which leads to the following transition kernel:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}). \tag{50.1}$$

Here, $\{\beta_t\}_{t=1}^T$ is a sequence of hyperparameters known as the variance schedule. These values are typically small and pre-determined. For example, a common choice is a linear schedule where β_t increases from $\beta_1 \approx 10^{-4}$ to $\beta_T \approx 0.02$. The schedule is designed such that $\beta_1 < \beta_2 < \cdots < \beta_T$, meaning we add progressively more noise at later timesteps. The term $\sqrt{1-\beta_t}$ is a scaling factor that keeps the variance of the signal from exploding. The entire forward process is a chain of these transitions, starting from the initial data $\mathbf{x}_0$:

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1}).$$

50.2.2 The Marginal Distribution

A critical property of this noising process is that we can sample $\mathbf{x}_t$ at any arbitrary timestep t directly from $\mathbf{x}_0$ in a single step, without having to iterate through all T transitions. This is essential for making training tractable.

Let's derive this closed-form marginal distribution $q(\mathbf{x}_t|\mathbf{x}_0)$. We can express $\mathbf{x}_{t-1}$ in terms of $\mathbf{x}_{t-2}$ and noise, and so on. Let $\alpha_t = 1-\beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$. As done in Chapter 13, we will use the reparameterisation trick [892], where sampling from $\mathcal{N}(\boldsymbol{\mu}, \sigma^2\mathbf{I})$ is equivalent to computing $\boldsymbol{\mu} + \sigma\boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Note, however, that since the forward process q is fixed, no gradients need to flow through this sampling, and the trick here merely yields a closed form for the marginal.

From Equation 50.1, we have:

$$\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}_t, \quad \text{where } \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{50.2}$$

Now, let's recursively expand $\mathbf{x}_{t-1}$:

$$\begin{aligned}\mathbf{x}_t &= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1-\alpha_{t-1}}\boldsymbol{\epsilon}_{t-1}\right) + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}_t \\ &= \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\boldsymbol{\epsilon}_{t-1} + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}_t\end{aligned}$$

where $\boldsymbol{\epsilon}_{t-1}, \boldsymbol{\epsilon}_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\mathbf{0}, \mathbf{I})$. Note that the sum of two independent Gaussian variables $\mathcal{N}(0, \sigma_1^2\mathbf{I})$ and $\mathcal{N}(0, \sigma_2^2\mathbf{I})$ is also a Gaussian $\mathcal{N}(0, (\sigma_1^2 + \sigma_2^2)\mathbf{I})$. The combined noise term satisfies

$$\sqrt{\alpha_t(1-\alpha_{t-1})}\boldsymbol{\epsilon}_{t-1} + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}_t \overset{d}{=} \sqrt{1-\alpha_t\alpha_{t-1}}\,\bar{\boldsymbol{\epsilon}}, \quad \text{where } \bar{\boldsymbol{\epsilon}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

since the variance of the left-hand side is

$$(\alpha_t(1-\alpha_{t-1}))\,\mathbf{I} + (1-\alpha_t)\,\mathbf{I} = (\alpha_t - \alpha_t\alpha_{t-1} + 1 - \alpha_t)\,\mathbf{I} = (1-\alpha_t\alpha_{t-1})\,\mathbf{I}.$$

Here, $\bar{\boldsymbol{\epsilon}}$ represents a fresh standard Gaussian with the same distribution as the normalised linear combination. Thus

$$\mathbf{x}_t = \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\,\bar{\boldsymbol{\epsilon}}.$$

By induction, if we continue this expansion all the way back to $\mathbf{x}_0$, we get:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, \quad \text{where } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{50.3}$$

This is a key equation. It provides the marginal distribution $q(\mathbf{x}_t|\mathbf{x}_0)$ in a simple, closed form:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I}) \tag{50.4}$$

This property is the workhorse of the training algorithm. It allows us to sample a $(\mathbf{x}_0, \mathbf{x}_t)$ pair for any t in a single step, without simulating the full chain.

By construction, the variance schedule $\{\beta_t\}$ is designed so that as $t \to T$, $\bar{\alpha}_T = \prod_{i=1}^{T}(1-\beta_i) \to 0$. In this limit, Equation 50.4 becomes:

$$q(\mathbf{x}_T|\mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

Thus our choice of the forward process successfully transforms any data point $\mathbf{x}_0$ into a sample from the standard Gaussian prior distribution.

50.3 The Reverse Process: The Generative Kernel

Now that we have a process to destroy data, we need to learn a process to create it. This is the reverse process, $p_\theta(\mathbf{x}_{0:T})$, which learns to undo the forward diffusion, one step at a time.

50.3.1 A Parameterised Markovian Process

The reverse process is a Markov chain of length T that starts from the prior $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and generates a sequence $\mathbf{x}_{T-1}, \mathbf{x}_{T-2}, \ldots, \mathbf{x}_0$.

$$p_\theta(\mathbf{x}_{0:T}) := p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) \tag{50.5}$$

The key question is how to learn the parameters of the reverse process $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$. One obvious approach would be to try to match the true reverse transition $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ as closely as possible. However, the true reverse transition $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is intractable, as it would require marginalizing over all possible $\mathbf{x}_0$: $q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \int q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) q(\mathbf{x}_0|\mathbf{x}_t) \mathrm{d}\mathbf{x}_0$.

The key to make the objective tractable, is to use the knowledge of $\mathbf{x}_0$. That is, if we know the original data point $\mathbf{x}_0$ that $\mathbf{x}_t$ was noised from, the reverse transition $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ would be tractable, which we can use to train the reverse process. Next, we analyse the posterior $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ to get a closed-form expression for it.

50.3.2 True Reverse Posterior

Let's analyse the true reverse posterior $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$. We can apply Bayes' theorem and the fact that the noising process is Markovian, i.e., $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1})$, to get:

$$\begin{aligned} q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) &= \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}) q(\mathbf{x}_0|\mathbf{x}_{t-1}) q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t, \mathbf{x}_0)} \\ &= \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}. \end{aligned} \tag{50.6}$$

Plugging in the known Gaussian distributions for each term:

$$\begin{aligned} q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) &\propto \exp\left(-\frac{1}{2}\left(\frac{(\mathbf{x}_t - \sqrt{\alpha_t}\mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0)^2}{1-\bar{\alpha}_{t-1}}\right)\right) \\ &= \exp\left(-\frac{1}{2}\left(\frac{\mathbf{x}_t^2 - 2\sqrt{\alpha_t}\mathbf{x}_t\mathbf{x}_{t-1} + \alpha_t\mathbf{x}_{t-1}^2}{\beta_t} + \frac{\mathbf{x}_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_{t-1}\mathbf{x}_0 + \bar{\alpha}_{t-1}\mathbf{x}_0^2}{1-\bar{\alpha}_{t-1}}\right)\right) \\ &= \exp\left(-\frac{1}{2}\left(\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}\right)\mathbf{x}_{t-1}^2 - \left(\frac{2\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{2\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\mathbf{x}_0\right)\mathbf{x}_{t-1} + C\right)\right) \end{aligned}$$

where C is a constant with respect to $\mathbf{x}_{t-1}$. This is a Gaussian distribution $\mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t, \tilde{\beta}_t\mathbf{I})$. By completing the square for $\mathbf{x}_{t-1}$, we can identify its variance $\tilde{\beta}_t$ and mean $\tilde{\boldsymbol{\mu}}_t$.

The precision (inverse variance) is:

$$\frac{1}{\tilde{\beta}_t} = \frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}} = \frac{\alpha_t(1-\bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1-\bar{\alpha}_{t-1})} = \frac{1 - \bar{\alpha}_{t-1} - \beta_t + \bar{\alpha}_{t-1}\beta_t + \beta_t}{\beta_t(1-\bar{\alpha}_{t-1})} = \frac{1-\bar{\alpha}_t}{\beta_t(1-\bar{\alpha}_{t-1})}$$

Therefore, the posterior variance is:

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t \tag{50.7}$$

This variance is a deterministic function of the schedule hyperparameters and does not depend on the data.

The mean $\tilde{\boldsymbol{\mu}}_t$ is given by:

$$\begin{aligned}\tilde{\boldsymbol{\mu}}_t &= \left(\frac{2\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{2\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\mathbf{x}_0\right)\tilde{\beta}_t/2 \\ &= \left(\frac{\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\mathbf{x}_0\right)\frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t \\ &= \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0\end{aligned} \tag{50.8}$$

So, the true reverse posterior $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ is Gaussian with a known mean and variance:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t\mathbf{I}) \tag{50.9}$$

50.3.3 Parameterising the Generative Transition

The model $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ for the reverse process is also parameterised as a Gaussian:

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)). \tag{50.10}$$

Since the posterior is also Gaussian, this choice will greatly simplify the training objective, as we will see in the next section. In Section 50.5.1, we will use the theory of stochastic differential equations to see that for large T, the reverse process is also Gaussian [33, 1541], providing further justification for this parameterisation.

50.4 Optimisation Objective: From an Evidence Lower Bound to a Simplified Denoising Loss

With the forward and reverse processes defined, we can now derive the objective function used to train the model. The model is trained, like a VAE (Section 13.3.3), by maximising the Evidence Lower Bound (ELBO) on the data log-likelihood $\log p_\theta(\mathbf{x}_0)$ under the denoising distribution, but with a fixed variation distribution, i.e. the noising process. See Section 12.1.3 of Chapter 12 for understanding the role of ELBO in variational inference.

50.4.1 The Variational Lower Bound (ELBO)

We wish to maximise $\mathbb{E}_{\mathbf{x}_0\sim p_{\text{data}}}[\log p_\theta(\mathbf{x}_0)]$. We can write:

$$\begin{aligned}\log p_\theta(\mathbf{x}_0) &= \log \int p_\theta(\mathbf{x}_{0:T})d\mathbf{x}_{1:T} \\ &= \log \int q(\mathbf{x}_{1:T}|\mathbf{x}_0)\frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}d\mathbf{x}_{1:T} \\ &= \log \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\left[\frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right] \\ &\geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right] \quad \text{(Jensen's Inequality)}\end{aligned}$$

This last term is the ELBO, which, with a minus sign, can be used as the training objective L_{VLB}. Note that the Jensen's-inequality argument used here is the same one used to derive the ELBO on the model evidence in

Section 12.1.5. We can decompose this into a more interpretable form:

$$\begin{aligned} L_{\text{VLB}}(\mathbf{x}_0, \theta) &= -\mathbb{E}_q \left[\log \frac{p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{\prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right] \\ &= -\mathbb{E}_q \left[\log p(\mathbf{x}_T) + \sum_{t=1}^{T} \log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) - \sum_{t=1}^{T} \log q(\mathbf{x}_t|\mathbf{x}_{t-1}) \right] \\ &= -\mathbb{E}_q \left[\log p(\mathbf{x}_T) + \sum_{t=2}^{T} \log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) + \log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \right. \\ &\qquad \left. - \log q(\mathbf{x}_1|\mathbf{x}_0) - \sum_{t=2}^{T} \log q(\mathbf{x}_t|\mathbf{x}_{t-1}) \right] \end{aligned}$$

Using the identity $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}$ (from the Markov property and Bayes' rule), we can rewrite $\log q(\mathbf{x}_t|\mathbf{x}_{t-1})$ as $\log q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) + \log q(\mathbf{x}_t|\mathbf{x}_0) - \log q(\mathbf{x}_{t-1}|\mathbf{x}_0)$. Substituting this in:

$$\begin{aligned} L_{\text{VLB}} = -\mathbb{E}_q \Bigg[&\log p(\mathbf{x}_T) - \log q(\mathbf{x}_T|\mathbf{x}_0) + \sum_{t=2}^{T} (\log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) - \log q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)) \\ &- \log q(\mathbf{x}_1|\mathbf{x}_0) + \log p_\theta(\mathbf{x}_0|\mathbf{x}_1) + \sum_{t=2}^{T} (\log q(\mathbf{x}_{t-1}|\mathbf{x}_0) - \log q(\mathbf{x}_t|\mathbf{x}_0)) \Bigg] \end{aligned}$$

The last summation is a telescoping sum $\log q(\mathbf{x}_1|\mathbf{x}_0) - \log q(\mathbf{x}_T|\mathbf{x}_0)$, which cancels terms.

$$\begin{aligned} L_{\text{VLB}} &= -\mathbb{E}_q \left[\log \frac{p(\mathbf{x}_T)}{q(\mathbf{x}_T|\mathbf{x}_0)} + \sum_{t=2}^{T} \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)} + \log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \right] \\ &= \mathbb{E}_q \left[\underbrace{\mathrm{D}_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T))}_{L_T} + \underbrace{\sum_{t=2}^{T} \mathrm{D}_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{1:T-1}} - \underbrace{\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right] \end{aligned} \tag{50.11}$$

This gives us a sum of KL divergences and a reconstruction term:

- $L_T = \mathrm{D}_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T))$: The **prior matching term**. This measures how well the final noised distribution $q(\mathbf{x}_T|\mathbf{x}_0)$ matches the prior $p(\mathbf{x}_T)$. Since q is fixed and p is a fixed $\mathcal{N}(\mathbf{0}, \mathbf{I})$, this term is a constant and can be ignored during optimisation.
- $L_{t-1} = \mathrm{D}_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))$: The **denoising matching terms** (each term L_{t-1}, summed as $L_{1:T-1}$ in Eq. (50.11)). This is a sum from $t = 2$ to T. Each term compares the learned reverse transition p_θ to the true (but $\mathbf{x}_0$-conditional) reverse posterior q. This is the primary learning signal.
- $L_0 = \mathbb{E}_q[-\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)]$: The **reconstruction term**. This term measures how well the model reconstructs the final data $\mathbf{x}_0$ from the last latent state $\mathbf{x}_1$.

Following Ho et al. [755], DDPM fixes the reverse-process variance to the schedule value $\tilde{\beta}_t \mathbf{I}$ rather than learning the general covariance $\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)$ introduced in Equation 50.10. The optimisation objective is to minimise the loss $\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}_0 \sim q_{\text{data}}}[L_{\text{VLB}}(\mathbf{x}_0, \theta)]$.

50.4.2 Simplification via Reparameterisation

Naive Monte Carlo estimation of Equation 50.11 will have high variance. We can use the fact that both

$$\begin{aligned} q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) &= \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}) \\ p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) &= \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \tilde{\beta}_t \mathbf{I}), \end{aligned}$$

are Gaussian and compute the KL divergence in closed form. When the variances of the two Gaussians are identical ($\sigma_1^2 = \sigma_2^2 = \tilde{\beta}_t$), the KL divergence simplifies to a simple L_2 distance between their means, scaled by the variance:

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\tilde{\beta}_t} ||\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)||^2 \right]. \tag{50.12}$$

The training objective is therefore to train a neural network $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ to predict the true posterior mean $\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0)$. As noted in Ho et al. [755] that this objective can be simplified even further by writing $\tilde{\boldsymbol{\mu}}_t$ purely in terms of $\mathbf{x}_t$ and the noise $\boldsymbol{\epsilon}$ using the forward process relationship from Equation 50.3. Substituting

$$\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}) \tag{50.13}$$

in the expression for $\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0)$ from Equation 50.8, we get:

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon} \right). \tag{50.14}$$

This motivates a new parameterisation for our model $\boldsymbol{\mu}_\theta$. Instead of training a network $\boldsymbol{\mu}_\theta$ to predict $\tilde{\boldsymbol{\mu}}_t$ directly, let's train a network $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ to predict the noise $\boldsymbol{\epsilon}$. We can then express our predicted mean $\boldsymbol{\mu}_\theta$ using the same functional form as $\tilde{\boldsymbol{\mu}}_t$:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) \tag{50.15}$$

50.4.3 The Simplified Loss Function

We now substitute the $\boldsymbol{\epsilon}$-prediction parameterisation back into our L_{t-1} loss term:

$$\begin{aligned}
L_{t-1} &= \mathbb{E}_q \left[\frac{1}{2\tilde{\beta}_t} ||\tilde{\boldsymbol{\mu}}_t - \boldsymbol{\mu}_\theta||^2 \right] \\
&= \mathbb{E}_q \left[\frac{1}{2\tilde{\beta}_t} \left\| \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon} \right) - \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) \right\|^2 \right] \\
&= \mathbb{E}_q \left[\frac{1}{2\tilde{\beta}_t} \left\| \frac{1}{\sqrt{\alpha_t}} \left(\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \right) (\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) - \boldsymbol{\epsilon}) \right\|^2 \right] \\
&= \mathbb{E}_q \left[\frac{1}{2\tilde{\beta}_t} \frac{1}{\alpha_t} \frac{\beta_t^2}{1-\bar{\alpha}_t} \|\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) - \boldsymbol{\epsilon}\|^2 \right] \\
&= \mathbb{E}_{x_0, \boldsymbol{\epsilon}, t} \left[\left(\frac{\beta_t^2}{2\tilde{\beta}_t \alpha_t (1-\bar{\alpha}_t)} \right) \|\boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) - \boldsymbol{\epsilon}\|^2 \right]
\end{aligned} \tag{50.16}$$

This is the principal objective function. The task of minimising the ELBO (a sum of KL divergences) has been reduced to a simple, weighted L_2 regression problem. We just need to train a neural network $\boldsymbol{\epsilon}_\theta$ (typically a U-Net, shown in Figure 50.2) to predict the noise $\boldsymbol{\epsilon}$ that was added to $\mathbf{x}_0$ to create $\mathbf{x}_t$.

Ho et al. [755] noted that training was more stable and effective when ignoring the complex weighting term in front of the L_2 loss. They propose a simplified objective:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \left[\|\boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) - \boldsymbol{\epsilon}\|^2 \right] \tag{50.17}$$

This is the final, practical objective function used by most modern diffusion models. For training, the L_0 reconstruction term can also be shown to be equivalent to this form for $t = 1$, so this single loss function L_{simple} is all that is needed.

50.4.4 Training and Sampling Algorithms

The simplified objective in Equation 50.17 leads to a very simple training algorithm.

Algorithm 36: DDPM Training

Data: Data distribution $\mathbf{x}_0 \sim q_{\text{data}}(\mathbf{x}_0)$, timesteps T, model $\boldsymbol{\epsilon}_\theta$
repeat
  Sample $\mathbf{x}_0 \sim q_{data}(\mathbf{x}_0)$
  Sample $t \sim \text{Uniform}(\{1, \dots, T\})$
  Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  Compute $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$
  Take gradient descent step on:

$$\nabla_\theta \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2$$

until *convergence*

The sampling algorithm (generation) works by running the reverse process from $t = T$ down to $t = 1$. At each step, we use our network $\boldsymbol{\epsilon}_\theta$ to predict the noise, and then use that to compute the mean of the distribution for $\mathbf{x}_{t-1}$.

Algorithm 37: DDPM Ancestral Sampling

Result: A sample $\mathbf{x}_0 \sim p_\theta(\mathbf{x}_0)$
Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $t = T, \dots, 1$ **do**
  Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
  Compute the predicted noise $\boldsymbol{\epsilon}_\theta = \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$
  Compute the predicted mean $\boldsymbol{\mu}_\theta$ using Eq. 50.15:

$$\boldsymbol{\mu}_\theta = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta\right)$$

  Compute the sample for the previous timestep:

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_\theta + \sqrt{\tilde{\beta}_t}\mathbf{z}$$

end
return $\mathbf{x}_0$

50.5 Continuous-Time Diffusion: Score-Based Generative Models

The discrete-time formulation discussed so far provides a practical algorithm, but it leaves open some deeper questions. For example, why does a fixed variational distribution q, which is a Gaussian, work at all? What are some other design choices that can improve the performance of the model? A deeper mathematical understanding of diffusion models can be found by analysing its continuous-time limit as $T \to \infty$. This connects the discrete-time formulation to score-based generative modelling [1540, 1541] and ultimately to the rich field of SDEs.

50.5.1 Convergence to a Stochastic Differential Equation (SDE)

As the number of timesteps $T \to \infty$ and the per-step perturbation $\beta_t \to 0$, the discrete forward process (Equation 50.1) converges to the solution of a continuous-time Itô SDE:

$$\mathrm{d}\mathbf{x}_t = \mathbf{f}(\mathbf{x}_t, t)\,\mathrm{d}t + g(t)\,\mathrm{d}\mathbf{w}, \tag{50.18}$$

which describes the evolution of a variable $\mathbf{x}(t)$ (which we will write as $\mathbf{x}_t$) subject to a deterministic drift function $\mathbf{f}(\mathbf{x}_t, t) : \mathbb{R}^d \to \mathbb{R}^d$ and a stochastic diffusion coefficient $g(t) : \mathbb{R} \to \mathbb{R}$. Here, t is now a continuous variable, typically normalised to $t \in [0, 1]$ or $[0, T]$, and $\mathbf{w}$ is the standard Wiener process.

Different choices of discrete processes converge to different SDEs. The framework introduced by Song et al. [1541] unifies two major lines of work:

1. **Variance-Preserving (VP) SDE:** This SDE corresponds to the continuous limit of the DDPM formulation [755]. To derive this rigorously, we start with the discrete Markov chain from Eq. 50.2:

$$\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathbf{z}_{t-1}, \quad t = 1, \dots, N \tag{50.19}$$

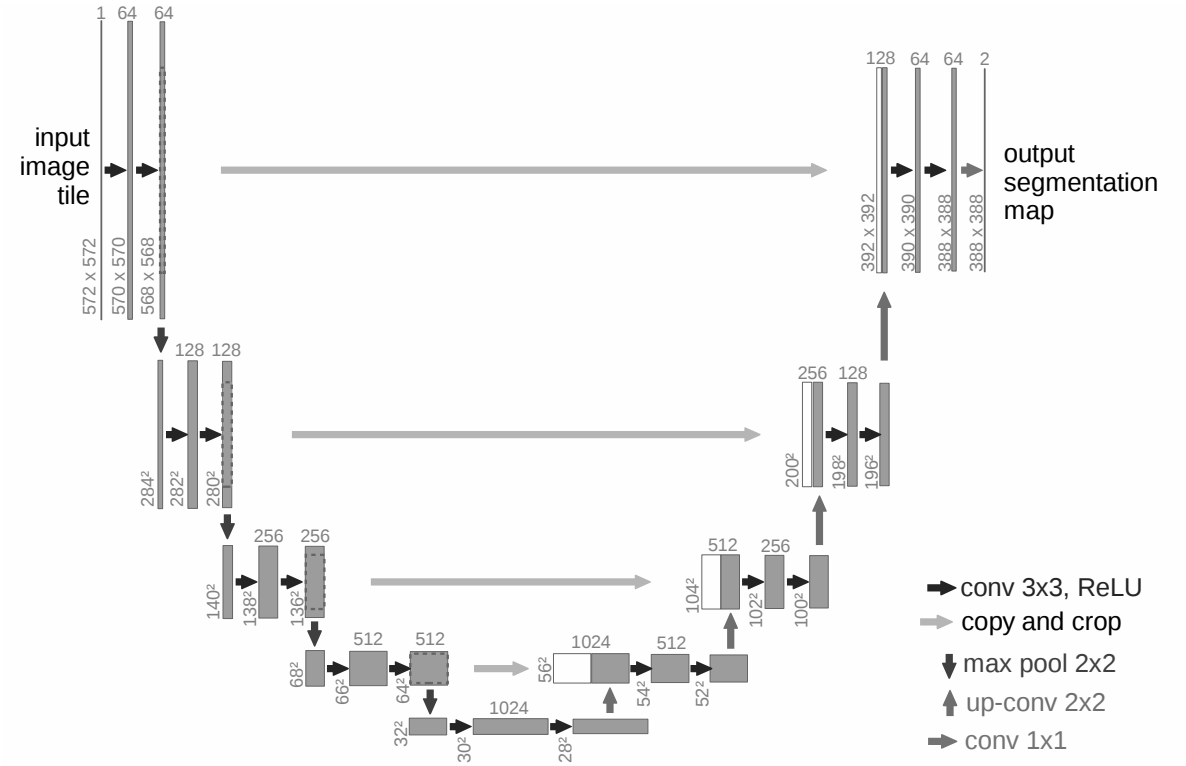


Figure 50.2: A diagram illustrating the U-Net architecture commonly used for the noise-prediction network $\boldsymbol{\epsilon}_\theta$. The skip connections are vital for preserving high-resolution details from $\mathbf{x}_t$ while the network processes information at coarser scales. The figure is adapted from Ronneberger et al. [1419].

where $\mathbf{z}_{t-1} \sim \mathcal{N}(0, \mathbf{I})$. To take the continuum limit as $N \to \infty$, we introduce an auxiliary set of noise scales $\{\bar{\beta}_t = N\beta_t\}_{t=1}^N$ and rewrite the discrete update as [1540]:

$$\mathbf{x}_t = \sqrt{1 - \frac{\bar{\beta}_t}{N}}\mathbf{x}_{t-1} + \sqrt{\frac{\bar{\beta}_t}{N}}\mathbf{z}_{t-1}. \tag{50.20}$$

In the limit of $N \to \infty$, the sequence $\{\bar{\beta}_t\}_{t=1}^N$ becomes a function $\beta(t)$ indexed by $t \in [0, 1]$, with $\beta(t/N) = \bar{\beta}_t$. Similarly, $\mathbf{x}_t$ becomes $x(t)$ and $\mathbf{z}_t$ becomes $z(t)$. With time step $\Delta t = 1/N$, we can write the discrete update for $t \in \{0, 1, \ldots, (N-1)/N\}$ as:

$$\begin{aligned}
\mathbf{x}(t + \Delta t) &= \sqrt{1 - \beta(t + \Delta t)\Delta t}\,\mathbf{x}(t) + \sqrt{\beta(t + \Delta t)\Delta t}\,\mathbf{z}(t) \\
&\approx \mathbf{x}(t) - \frac{1}{2}\beta(t + \Delta t)\Delta t\,\mathbf{x}(t) + \sqrt{\beta(t + \Delta t)\Delta t}\,\mathbf{z}(t) \\
&\approx \mathbf{x}(t) - \frac{1}{2}\beta(t)\Delta t\,\mathbf{x}(t) + \sqrt{\beta(t)\Delta t}\,\mathbf{z}(t),
\end{aligned} \tag{50.21}$$

where the approximations hold when $\Delta t \ll 1$. Using the expansion $\sqrt{1-u} \approx 1 - \frac{1}{2}u$ for small u, and assuming $\beta(t)$ varies slowly over Δt. In the limit $\Delta t \to 0$, the discrete differences $\mathbf{x}(t+\Delta t) - \mathbf{x}(t)$ converge to the differential $\mathrm{d}\mathbf{x}(t)$, and $\mathbf{z}(t)\sqrt{\Delta t}$ converges to the Wiener process increment $\mathrm{d}\mathbf{w}_t$. Thus, Eq. 50.21 yields the VP SDE:

$$\mathrm{d}\mathbf{x}(t) = -\frac{1}{2}\beta(t)\mathbf{x}(t)\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\mathbf{w}_t. \tag{50.22}$$

This is called "variance-preserving" because as the process diffuses forward in time, the variance of $x(t)$ remains bounded and converges to 1 at $t = T$, matching the prior distribution.

2. **Variance-Exploding (VE) SDE:** This SDE corresponds to an earlier line of work on Noise-Conditioned Score Matching [1540], which used annealed Langevin dynamics. This process is defined by $d\mathbf{x}_t = \sqrt{\frac{d[\sigma^2(t)]}{dt}}\,\mathrm{d}\mathbf{w}_t$. Here, the drift is zero, and the process simply adds noise of increasing variance $\sigma^2(t)$, causing the variance of $p_t(\mathbf{x}_t)$ to grow monotonically with t.

This SDE framework is general and can encompass many different noising processes. The key is that they all transform a complex $p_0(\mathbf{x}_0)$ into a simple $p_T(\mathbf{x}_T)$.

50.5.2 The Reverse-Time SDE

The time-reversal of a stochastic process is a process that runs backwards in time. For the stochastic process governed by the SDE of the form in Equation 50.18, a key result from the theory of SDEs by Anderson [33] states that the time-reversal of this process is also governed by an SDE of the same form:

$$\mathrm{d}\mathbf{x}_t = [\mathbf{f}(\mathbf{x}_t, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)]\,\mathrm{d}t + g(t)\,\mathrm{d}\bar{\mathbf{w}}_t \tag{50.23}$$

where $\mathrm{d}\bar{\mathbf{w}}_t$ is a reverse-time Wiener process, and $\mathrm{d}t$ is an infinitesimal time step in the reverse direction. This reverse-time SDE runs from $t = T$ to $t = 0$ and transforms samples from the prior p_T back into samples from the data distribution p_0.

Comparing the forward SDE (Equation 50.18) and the reverse SDE (Equation 50.23), we see they are identical except for the drift term. The reverse drift is the forward drift *minus* a new term that depends on $\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)$. Figure 50.3 shows some trajectories of the forward and reverse SDEs for a simple 1D mixture of Gaussian data distribution.

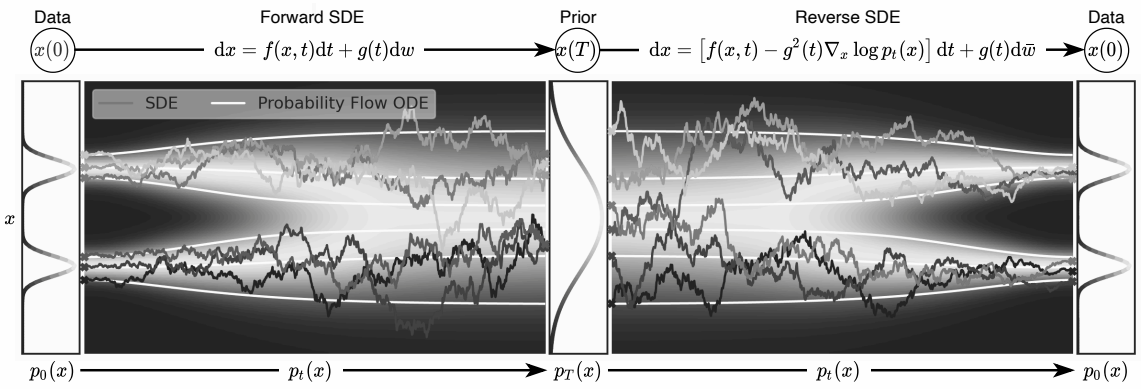


Figure 50.3: The forward and reverse SDEs for a 1D toy data distribution, which is a mixture of two Gaussians. On the left are some simulated trajectories of the forward SDE that converts samples from the data distribution p_0 to samples from the prior p_T. On the right are the trajectories of the reverse SDE with learnt score network that converts samples from the prior p_T back to samples from the data distribution p_0. The figure is adapted from Song et al. [1541].

50.5.3 The Score Function: Unification of Objectives

The only unknown quantity in this reverse SDE is $\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)$. This vector field is known as the score function of the marginals $p_t(\mathbf{x}_t)$. It points in the direction of steepest ascent for the log-density at any point $\mathbf{x}_t$. This immediately reveals the fundamental task of score-based generative modelling: if we can train a neural network $\mathbf{s}_\theta(\mathbf{x}_t, t)$ to estimate the score function $\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)$ for all $t \in [0, T]$, we can plug $\mathbf{s}_\theta$ into the reverse SDE and solve it numerically to generate new samples.

The training objective for this score network $\mathbf{s}_\theta$ is the score-matching objective, which minimises the Fisher divergence between the model and the true data distribution:

$$L(\theta) = \mathbb{E}_t \mathbb{E}_{p_t(\mathbf{x}_t)} \left[\|\mathbf{s}_\theta(\mathbf{x}_t, t) - \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)\|_2^2 \right]$$

This objective is intractable as written, because it requires access to the score of the unknown distribution $p_t(\mathbf{x}_t)$. However, Vincent [1680] shows that this objective is equivalent to minimising:

$$L(\theta) = \mathbb{E}_t \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \mathbb{E}_{p_0(\mathbf{x}_0)} \left[\|\mathbf{s}_\theta(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0)\|_2^2 \right],$$

called the denoising score-matching objective. This new objective is tractable, because $q(\mathbf{x}_t|\mathbf{x}_0)$ is our known, analytic forward perturbation kernel (Eq. 50.4).

Let's compute this tractable score:

$$\begin{aligned}\log q(\mathbf{x}_t|\mathbf{x}_0) &= \log\left(C \cdot \exp\left(-\frac{1}{2(1-\bar{\alpha}_t)}\|\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0\|^2\right)\right) \\ &= \log C - \frac{1}{2(1-\bar{\alpha}_t)}\|\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0\|^2 \\ \nabla_{\mathbf{x}_t}\log q(\mathbf{x}_t|\mathbf{x}_0) &= -\frac{1}{1-\bar{\alpha}_t}(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0)\end{aligned}$$

Using our reparameterisation from Eq. 50.3, $\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0 = \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$:

$$\nabla_{\mathbf{x}_t}\log q(\mathbf{x}_t|\mathbf{x}_0) = -\frac{1}{1-\bar{\alpha}_t}(\sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}) = -\frac{\boldsymbol{\epsilon}}{\sqrt{1-\bar{\alpha}_t}} \tag{50.24}$$

This is a key result. The true score of the perturbed data distribution is just the (scaled) negative noise that was added to obtain $\mathbf{x}_t$ from $\mathbf{x}_0$. If we parameterise our score network $\mathbf{s}_\theta$ in a way that maps to the ϵ-predictor, $\mathbf{s}_\theta(\mathbf{x}_t, t) = -\frac{\boldsymbol{\epsilon}_\theta(\mathbf{x}_t,t)}{\sqrt{1-\bar{\alpha}_t}}$, and plug this into the score-matching loss:

$$\begin{aligned}L(\theta) &\propto \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}}\left[\left\|-\frac{\boldsymbol{\epsilon}_\theta(\mathbf{x}_t,t)}{\sqrt{1-\bar{\alpha}_t}} - \left(-\frac{\boldsymbol{\epsilon}}{\sqrt{1-\bar{\alpha}_t}}\right)\right\|_2^2\right] \\ &= \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}}\left[\frac{1}{1-\bar{\alpha}_t}\|\boldsymbol{\epsilon}_\theta(\mathbf{x}_t,t) - \boldsymbol{\epsilon}\|_2^2\right]\end{aligned}$$

This is precisely the weighted L_{simple} loss from Equation 50.17. Hence, we note the following equivalence: Minimising the DDPM's ELBO is equivalent to training a denoising score-matching model. The ϵ-predictor $\boldsymbol{\epsilon}_\theta$ is just a reparameterised version of the score function $\mathbf{s}_\theta$. Specifically, $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \approx -\sqrt{1-\bar{\alpha}_t}\nabla_{\mathbf{x}}\log p_t(\mathbf{x}_t)$. This unified view is powerful, as it allows us to decouple the training objective from the sampling method.

50.5.4 Sampling via SDE and ODE Solvers

With a trained score model $\mathbf{s}_\theta(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}}\log p_t(\mathbf{x}_t)$, we can now plug it into the reverse SDE (Equation 50.23) and solve it numerically from $t = T$ to $t = 0$. This opens up a wide variety of solvers beyond the original DDPM ancestral sampler in Algorithm 37.

Numerical SDE Solvers

These methods discretise the reverse-time SDE and add the appropriate stochastic noise at each step.

- **Euler-Maruyama**: This is the simplest (and most common) SDE solver. We discretise time into N steps $T = t_N, t_{N-1}, \ldots, t_0 = 0$, with $\Delta t < 0$ being the step size.

  $$\mathbf{x}_{i-1} = \mathbf{x}_i + [\mathbf{f}(\mathbf{x}_i, t_i) - g(t_i)^2\mathbf{s}_\theta(\mathbf{x}_i, t_i)]\Delta t + g(t_i)\sqrt{|\Delta t|}\mathbf{z}_i \tag{50.25}$$

  where $\mathbf{z}_i \sim \mathcal{N}(0, \mathbf{I})$. This is a general sampler that adds noise at each step. The DDPM ancestral sampler in Algorithm 37 can be viewed as a different discretisation of the same reverse-time SDE.

Predictor-Corrector Samplers

Due to discretisation error, a single step of an SDE solver (the predictor) may not land the sample $\mathbf{x}_{i-1}$ in a high-density region of $p_{t_{i-1}}$, and if unchecked these errors accumulate over steps. Since we have the learned score function at different noise levels, we can use it to "correct" the sample x_{i-1} by pushing it towards a higher-density region of $p_{t_{i-1}}$ using MCMC [1541].

- **Langevin Dynamics Corrector:** After the predictor step (Equation 50.25) gives an initial $\hat{\mathbf{x}}_{i-1}$, we can run K steps of MCMC using annealed Langevin dynamics, which targets $p_{t_{i-1}}$:

  $$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \frac{\gamma}{2}\nabla_{\mathbf{x}}\log p_{t_{i-1}}(\mathbf{x}^{(k)}) + \sqrt{\gamma}\mathbf{z}_k$$

  We just plug in our score model $\mathbf{s}_\theta(\mathbf{x}^{(k)}, t_{i-1})$ for the gradient. This predictor-corrector loop (SDE step followed by MCMC steps) is very robust and was the core of the original noise conditioned score network models [1540].

Probability Flow ODE Solvers

Remarkably, there exists a corresponding *deterministic* process, defined by an Ordinary Differential Equation (ODE), whose trajectories, when initialized using the prior distribution, evolve to generate paths with the marginal densities $p_t(\mathbf{x}_t)$ that match those of the paths from the SDE . This is the probability flow ODE:

$$\mathrm{d}\mathbf{x}_t = \left[\mathbf{f}(\mathbf{x}_t, t) - \frac{1}{2}g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)\right] \mathrm{d}t \tag{50.26}$$

This is a deterministic equation. If we solve it backwards in time from $t = T$ to $t = 0$ (using our score model $\mathbf{s}_\theta$ for the gradient $\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)$), we get a deterministic mapping from an $\mathbf{x}_T$ to some $\mathbf{x}_0$. We can solve the ODE using any black-box numerical ODE solver (e.g., Runge-Kutta, Heun's method, etc.). Figure 50.3 shows the trajectories of the probability flow ODE alongside those from the SDE. The use of ODEs instead of SDEs offers several advantages:

1. The sampling path is deterministic, which allows inversions, that is, going from a data sample back to the latent space. ODEs also allow computing exact log-likelihood using the instantaneous change of variables formula [272].
2. We can use high-order, adaptive-step-size ODE solvers to get a more accurate solution with fewer network evaluations.
3. The equivalence provides the theoretical foundation for the DDIM sampler (Section 50.6), which can be shown to be a first-order discretisation of this probability flow ODE.

Although theoretically, the reverse SDE and the probability flow ODE share the same marginal densities p_t in continuous time, the practical choice between them depends on the computational budget and on the desired properties of the sampler. ODE solvers have lower discretisation error and therefore are preferable when few network evaluations (NFEs) are available. However, in the high-NFE regime, ODE solvers plateau, while SDE solvers achieve better sample quality as the injected stochasticity contracts accumulated score-estimation and discretisation errors along the trajectory [1541]. Hybrid samplers that introduce controlled stochasticity can achieve high-quality samples at lower NFEs than pure SDE solvers [873, 1790].

50.6 Improving Sampling Efficiency and Controllability

While the core DDPM/SDE framework is conceptually powerful, its practical utility has been unlocked by several key algorithmic improvements. The most important address two major limitations: the slow, iterative sampling process and the lack of a built-in mechanism for conditional generation.

50.6.1 Accelerated Sampling: Denoising Diffusion Implicit Models (DDIM)

The DDPM ancestral sampler (Algorithm 37) is slow, requiring T (e.g., 1000) sequential network evaluations. This is because it is *stochastic*—at each step t, it samples from $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ by adding noise $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \tilde{\beta}_t \mathbf{I})$. This procedure correctly models the generative process p_θ, which is a reversal of the Markovian forward process q. However, Song et al. [1538] observed that the simplified DDPM loss L_{simple} (Equation 50.17) only depends on the marginal distributions $q(\mathbf{x}_t|\mathbf{x}_0)$, not the full joint distribution $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$. This means the network $\boldsymbol{\epsilon}_\theta$ is trained on $(\mathbf{x}_0, \mathbf{x}_t)$ pairs, regardless of *how* $\mathbf{x}_t$ was reached from $\mathbf{x}_0$.

DDIM [1538] exploits this observation by defining a *new*, non-Markovian forward process with marginals

$$q_\sigma(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \coloneqq \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2}\, \frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0}{\sqrt{1 - \bar{\alpha}_t}},\ \sigma_t^2 \mathbf{I}\right),$$

which has the same marginals $q(\mathbf{x}_t|\mathbf{x}_0)$ as the original DDPM process. This new process allows for a more general, deterministic reverse process. The DDIM update rule is:

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \cdot \left(\frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}}\right) + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) + \sigma_t \boldsymbol{\epsilon}_t \tag{50.27}$$

Note the new parameter σ_t, which controls the stochasticity. If we set $\sigma_t \approx 0$ for all t, the process becomes deterministic. This means that for a given starting noise $\mathbf{x}_T$, the entire sampling trajectory $\mathbf{x}_{T-1}, \ldots, \mathbf{x}_0$ is fixed.

This resulting model is called *implicit* because $\mathbf{x}_0$ is generated deterministically from $\mathbf{x}_T$. This deterministic update is similar to the discretisation of the probability flow ODE (Eq. 50.26).

Another benefit of this formulation is that the generative process is no longer tied to the original T noising steps used to train the model. We can define a new, shorter sampling trajectory using a subsequence of timesteps $\mathcal{T} = \{\tau_1, \tau_2, \ldots, \tau_S\}$ where $S \ll T$. We can then use the same network $\boldsymbol{\epsilon}_\theta$ (trained on all T steps) but only evaluate it at the timesteps in $\mathcal{T}$, jumping from $\mathbf{x}_{\tau_i}$ to $\mathbf{x}_{\tau_{i-1}}$. This enables a 10-50x acceleration in sampling speed (wall clock time) with a surprisingly small drop in sample quality.

50.6.2 Conditional Generation

The models discussed so far are unconditional: they sample from the entire data distribution $p(\mathbf{x})$. However, what if we want to generate data samples of a specific class? For instance, imagine we want an image generator to generate images of dogs. This problem can be framed as conditional generation, wherein we want to sample from $p(\mathbf{x}|y)$ instead of $p(\mathbf{x})$. Here, y is the conditioning variable.

Classifier Guidance

One way to generate samples from $p(\mathbf{x}|y)$ is to use the Bayes' rule on the score function:

$$\nabla_{\mathbf{x}} \log p_t(\mathbf{x}|y) = \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) + \nabla_{\mathbf{x}} \log p_t(y|\mathbf{x}).$$

We already have a model for the unconditional score: $\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) = \mathbf{s}_\theta(\mathbf{x}_t, t)$. The new term $\nabla_{\mathbf{x}} \log p_t(y|\mathbf{x})$ is the gradient of a classifier $p_\phi(y|\mathbf{x}_t)$ trained to predict the class y from the *noised* data $\mathbf{x}_t$. This approach, wherein we use a separate classifier to guide the sampling is referred to as *classifier guidance* [422].

Intuitively, we are steering the score-based sampler by plugging in a modified score $\hat{\mathbf{s}}_\theta$:

$$\hat{\mathbf{s}}_\theta(\mathbf{x}_t, t, y) = \mathbf{s}_\theta(\mathbf{x}_t, t) + w \cdot \nabla_{\mathbf{x}} \log p_\phi(y|\mathbf{x}_t),$$

where w is the guidance scale, a scalar that controls the strength of the conditioning. In the ϵ-prediction formulation, this becomes:

$$\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, y) = \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) - w \cdot \sqrt{1 - \bar{\alpha}_t} \cdot \nabla_{\mathbf{x}} \log p_\phi(y|\mathbf{x}_t) \tag{50.28}$$

This modified $\hat{\boldsymbol{\epsilon}}_\theta$ is then plugged into the DDPM or DDIM sampling algorithm. This technique works quite well, trading off diversity for stronger adherence to the condition y. Moreover, since the classifier is a separate model, we are free to change it without re-training the generator.

50.6.3 Classifier-Free Guidance

Classifier guidance is effective, but it is computationally burdensome. It requires training a separate, noise-conditioned classifier $p_\phi(y|\mathbf{x}_t)$, which must be evaluated at every sampling step, adding to the cost.

Ho and Salimans [753] proposed a simpler and more powerful alternative: classifier-free guidance (CFG). The key idea is to train a *single* diffusion model $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y)$ that is jointly conditioned on the timestep t and the conditioning signal y (which can be a class label, a text embedding, etc.). During training, this conditioning y is randomly replaced with a suitable "null" token (e.g., 10-20% of the time), which implies no conditioning. This forces the same network to learn both the conditional model $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y)$ and the unconditional model $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \emptyset)$.

During sampling, we can then perform an extrapolation. The guided ϵ-prediction is computed as:

$$\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, y) = (1 + w)\, \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y) - w\, \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \emptyset) \tag{50.29}$$

This simple modification is incredibly powerful. The term $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y) - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \emptyset)$ can be viewed as an estimate of the classifier gradient, learned by the network itself. Equivalently, with guidance scale defined as $s \coloneqq w + 1$, Eq. (50.29) can be written as $\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, y) = \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \emptyset) + s\,(\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y) - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \emptyset))$, where $s = 1$ recovers the conditional prediction and $s = 0$ corresponds to unconditional generation. Typical values of the guidance scale s for strong conditional generation are around 7.5. Larger values can reduce sample diversity. CFG is the *de facto* standard for modern large-scale text conditional diffusion models.

50.6.4 Latent Diffusion Models

The improvements discussed so far reduce the number of sampling steps. Latent Diffusion Models (LDMs) [1417] provide a complementary approach that reduces the cost of each step. An LDM is trained in two stages. First, a regularized autoencoder, consisting of an encoder $\mathcal{E}$ and decoder $\mathcal{D}$, maps an image $\mathbf{x}$ to a spatially compressed latent representation

$$\mathbf{z} = \mathcal{E}(\mathbf{x}),$$

from which the image can be approximately reconstructed as

$$\widetilde{\mathbf{x}} = \mathcal{D}(\mathbf{z}).$$

The autoencoder is then frozen, and a diffusion model is trained on the distribution of encoded representations rather than directly on pixels.

At generation time, the learned reverse process produces a clean latent $\mathbf{z}_0$, which is decoded into an image as

$$\widehat{\mathbf{x}} = \mathcal{D}(\mathbf{z}_0).$$

Because the autoencoder absorbs part of the burden of representing local pixel-level variation, the diffusion model can operate on a substantially smaller spatial input. For example, with an $8\times$ spatial downsampling factor, a 512×512 RGB image corresponds to a 64×64 latent grid, reducing the number of spatial locations processed by the diffusion model by a factor of 64.

This two-stage design resembles a latent-variable generative model in which the diffusion model serves as a learned prior over the representations produced by the encoder. Unlike a conventional VAE, however, the autoencoder and latent prior are trained sequentially rather than jointly through a single variational objective. Combined with cross-attention conditioning and classifier-free guidance, this design underpins many large-scale text-to-image systems.

50.7 Diffusion Models for Discrete Data Domains

The entire framework presented so far, from the Gaussian forward process to the L_2 denoising loss, has been designed for continuous data $\mathbf{x} \in \mathbb{R}^n$ (e.g., pixels, audio samples). For discrete, structured data (e.g., text, molecules, graphs), this framework is not directly applicable.

50.7.1 Beyond Gaussian Perturbations

Applying Gaussian noise to discrete data (e.g., one-hot vectors representing text tokens) immediately destroys the discrete structure, pushing the data off its original manifold into a continuous space. While some work [423, 1033] has explored "embedding" discrete data in a continuous space and diffusing there, a more direct approach is to define a diffusion-like process *natively* in the discrete state-space.

50.7.2 Discrete-State Markov Processes

Austin et al. [63], Hoogeboom et al. [780] formalise diffusion like noising process for discrete space using discrete-time Markov chains.

Forward Process

Assume that we are working with sequences of fixed length L, with each entry taking values in a finite vocabulary $\mathcal{V}$. Then the Gaussian kernel is replaced by a discrete-state transition kernel for the full sequence $\mathbf{x}_t, \mathbf{x}_{t-1} \in \mathcal{V}^L$. In general, such a kernel would be a $|\mathcal{V}|^L \times |\mathcal{V}|^L$ matrix with $|\mathcal{V}|^{2L}$ entries, which is extremely large even for modest vocabulary and sequence length. Therefore, we use a factorized transition matrix $Q_t \in [0,1]^{|\mathcal{V}|\times|\mathcal{V}|}$ applied independently at each position of the sequence [63], with $(Q_t)_{ij} = q(x_t^i = j \mid x_{t-1}^i = i)$ for $x_t^i, x_{t-1}^i \in \mathcal{V}$. For the sake of brevity, we will abuse the notation slightly and use $x_t \in \{0,1\}^{|\mathcal{V}|}$ to denote both the one-hot encoding of a single position in the sequence, as well as the token value $x_t \in \mathcal{V}$ at that position, where the exact meaning will be clear from the context. We will use the superscript to denote the position when necessary. For instance, x_t^i will denote the token at position i in sequence $\mathbf{x}_t$. Using this notation, the transition probability of the Markov chain for one position in the sequence is given by:

$$q(x_t|x_{t-1}) = \mathrm{Cat}(x_t;\ x_{t-1}^\top Q_t),$$

where $\mathrm{Cat}(\cdot\,;\theta)$ denotes the categorical distribution with category probabilities θ.

Marginal Distribution: Just as we did for the continuous case in Equation 50.4, we can derive a closed-form expression for the marginal distribution $q(x_t|x_0)$ by marginalizing out all intermediate steps. By the law of total probability and the Markov property:

$$\begin{aligned} q(x_t|x_0) &= \sum_{x_{1:t-1}} q(x_t, x_{t-1}, \ldots, x_1|x_0) \\ &= \sum_{x_{1:t-1}} \prod_{s=1}^{t} q(x_s|x_{s-1}). \end{aligned} \tag{50.30}$$

To evaluate this expression recursively, consider

$$q(x_2 = i|x_0 = j) = \sum_{k=1}^{K} q(x_2 = i|x_1 = k)\, q(x_1 = k|x_0 = j) = \sum_{k=1}^{K} (Q_1)_{jk}\, (Q_2)_{k,i}.$$

This is precisely the matrix multiplication $(Q_1 Q_2)_{ji}$. By induction, we obtain:

$$q(x_t|x_0) = \text{Cat}(x_t; {x_0}^\top \bar{Q}_t), \quad \text{where} \quad \bar{Q}_t = Q_1 Q_2 \cdots Q_t. \tag{50.31}$$

Posterior Distribution $q(x_{t-1}|x_t, x_0)$: Using the Bayes' rule and the Markov property, as we did for the continuous case in Equation 50.6, we get:

$$\begin{aligned} q(x_{t-1} = i|x_t = j, x_0 = k) &= \frac{q(x_t = j|x_{t-1} = i)\, q(x_{t-1} = i|x_0 = k)}{q(x_t = j|x_0 = k)} \\ &= \frac{(Q_t)_{ij}\, (\bar{Q}_{t-1})_{ki}}{(\bar{Q}_t)_{kj}}. \end{aligned}$$

In vectorized form, where $\odot$ denotes element-wise multiplication:

$$q(x_{t-1}|x_t, x_0) = \text{Cat}\left(x_{t-1}; \frac{Q_t^\top\, x_t \odot \bar{Q}_{t-1}^\top\, x_0}{x_t^\top\, \bar{Q}_t^\top\, x_0}\right). \tag{50.32}$$

Next, we will look at one particular choice of the noising process.

Masking Noise

A particularly effective noising process for discrete data is *masking*, where tokens are gradually replaced with a special [MASK] token. Let m denote the index of the mask token in the vocabulary. The transition matrix is defined as:

$$Q_t = (1 - \beta_t)\mathbf{I} + \beta_t \mathbf{1}\mathbf{e}_m^\top,$$

where $\mathbf{e}_m$ is the one-hot vector for the mask token, $\mathbf{I}$ is the identity matrix, and $\mathbf{1}$ is the vector of all ones, and $\beta_t \in [0, 1]$ is the masking rate at timestep t.

Note that

$$\begin{aligned} Q_s Q_{s+1} &= [(1 - \beta_s)\mathbf{I} + \beta_s \mathbf{1}\mathbf{e}_m^\top][(1 - \beta_{s+1})\mathbf{I} + \beta_{s+1}\mathbf{1}\mathbf{e}_m^\top] \\ &= (1 - \beta_s)(1 - \beta_{s+1})\mathbf{I} + [1 - (1 - \beta_s)(1 - \beta_{s+1})]\mathbf{1}\mathbf{e}_m^\top. \end{aligned}$$

Therefore, through induction:

$$\bar{Q}_t = \bar{\alpha}_t \mathbf{I} + (1 - \bar{\alpha}_t)\mathbf{1}\mathbf{e}_m^\top, \tag{50.33}$$

where, as before, $\bar{\alpha}_t = \prod_{s=1}^{t}(1 - \beta_s)$. As with the case with the continuous domain, we can compute the transition probability $q(x_t|x_0)$ as well as sample from it at a $O(1)$ time complexity if we store the values of $\bar{\alpha}_t$ for $t \in \{1, \ldots T\}$.

Reverse Process

The reverse process must now learn to invert the noising Markov chain. It is parameterised as a categorical distribution $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$, which, given a noised sequence $\mathbf{x}_t$, predicts the probability distribution over one step of denoising. Parameterising the full joint distribution over $\mathbf{x}_{t-1}$ will require an expensive procedure like Gibbs sampling. Therefore, we approximate the joint distribution using marginals as follows [780]:

$$\begin{aligned} p(\mathbf{x}_{t-1}|\mathbf{x}_t) &= \sum_{\mathbf{x}_0} p(\mathbf{x}_{t-1}, \mathbf{x}_0|\mathbf{x}_t) \\ &= \prod_j \sum_{x_0^j} p(x_{t-1}^j|x_0^j, \mathbf{x}_t)\ p(x_0^j|\mathbf{x}_t) \\ &\approx \prod_j \sum_{x_0^j} q(x_{t-1}^j|x_0^j, \mathbf{x}_t)\ p(x_0^j|\mathbf{x}_t) \\ &= \prod_j \sum_{x_0^j} q(x_{t-1}^j|x_0^j, x_t^j)\ p(x_0^j|\mathbf{x}_t). \end{aligned}$$

Using this, we can parameterise the reverse process as

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \prod_{i=1}^{L} p_\theta(x_{t-1}^i|\mathbf{x}_t), \quad \text{where,} \quad p_\theta(x_{t-1}^i|\mathbf{x}_t) = \sum_{x_0^i} q(x_{t-1}^i|x_0^i, x_t^i)\ p_\theta(x_0^i|\mathbf{x}_t), \tag{50.34}$$

where we can insert the posterior $q(x_{t-1}^i|x_0^i, x_t^i)$ in closed form from Equation 50.32.

Objective

Since the ELBO in Equation 50.11 was obtained with no additional assumptions on the q and p, it remains fully applicable even for the discrete case. In the case of masked noising, injecting the simple posterior from Equation 50.33 into Equation 50.34, we get:

$$p_\theta(x_{t-1}^i|\mathbf{x}_t) = \begin{cases} 1, & \text{if } x_t^i = x_0^i \text{ and } x_{t-1}^i = x_0^i \\ 0, & \text{if } x_t^i = x_0^i \text{ and } x_{t-1}^i \neq x_0^i \\ \frac{\beta_t \prod_{s=1}^{t-1}(1-\beta_s)}{1-\prod_{s=1}^{t}(1-\beta_s)}\ p(x_0^i = x_{t-1}^i|\mathbf{x}_t), & \text{if } x_t^i = m \text{ and } x_{t-1}^i \neq m \\ \frac{1-\prod_{s=1}^{t-1}(1-\beta_s)}{1-\prod_{s=1}^{t}(1-\beta_s)}, & \text{if } x_t^i = m \text{ and } x_{t-1}^i = m. \end{cases} \tag{50.35}$$

Therefore, we just need to parameterise $p_\theta(x_0^i = x_{t-1}^i|\mathbf{x}_t)$ for the case when $x_t^i = m$. This observation simplifies L_{VLB} from Equation 50.11 into

$$L_{\text{VLB}}(\theta) = -\mathbb{E}_{t,\mathbf{x}_0,\mathbf{x}_t} \frac{\beta_t \bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \sum_{i=1}^{L} \mathbb{1}\big(x_t^i = [\text{MASK}]\big) \log p_\theta(x_0^i|\mathbf{x}_t),$$

where the expectation is over $t \sim \text{Uniform}[0, T]$, $\mathbf{x}_0 \sim q_{\text{data}}(\mathbf{x}_0)$ and $\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_0)$. This objective is just a weighted sum of cross-entropy losses again and is extremely simple to compute.

Extension to Continuous-Time Formulation

As was the case with diffusion in continuous space, the discrete-time formulation in Section 50.7.2 can be extended to continuous time. For the continuous-time process, $\mathbf{Q}_t$ will denote the *rate matrix*, which can be thought of as the infinitesimal version of the transition matrix, but with real valued entries. The off-diagonal entries of $\mathbf{Q}_t$ are non-negative, and the diagonals are the negative sum of the off-diagonal entries: $(\mathbf{Q}_t)_{ij} \geq 0$ for $i \neq j$ and $(\mathbf{Q}_t)_{ii} = -\sum_{j \neq i} (\mathbf{Q}_t)_{ij}$. Using $\mathbf{q}_t$ to denote the vector of marginals, the evolution of the marginals is governed by the Kolmogorov forward equation (KFE):

$$\frac{\mathrm{d}}{\mathrm{d}t} \mathbf{q}_t^\top = \mathbf{q}_t \mathbf{Q}_t, \qquad \mathbf{q}_0 = \mathbf{q}_{\text{data}}. \tag{50.36}$$

The most intuitive way to understand the evolution of the continuous-time Markov chain $\mathbf{x}_t$ is by considering the discretised form of the ODE for small Δt:

$$q(\mathbf{x}_{t+\Delta t}|\mathbf{x}_t) = \mathbb{1}(\mathbf{x}_{t+\Delta t} = \mathbf{x}_t) + \Delta t\, \mathbf{Q}_t(\mathbf{x}_t, \mathbf{x}_{t+\Delta t}) + o(\Delta t). \tag{50.37}$$

Next, we will discuss the factorized masking process governed by the per-position rates $Q_t \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$.

Masking Noise: The rate matrix of the continuous-time analog of token masking noising (Section 50.7.2) can be written using a single function of time $\beta(t) \geq 0$ (which we will denote by β_t):

$$Q_t = -\beta_t \mathbf{I} + \beta_t \mathbf{1}\mathbf{e}_m^\top. \tag{50.38}$$

In order to compute the transition probability $q(x_t|x_0)$ we observe that the rate matrices for different time values commute. Therefore, we get a closed-form solution for the Kolmogorov forward equation, which gives us the transition probability in closed form:

$$q(x_t|x_0) \;=\; \mathrm{Cat}(x_t;\, \bar{\alpha}_t\, x_0 \;+\; \big(1-\bar{\alpha}_t\big)\, \mathbf{e}_m^\top), \quad \text{where} \qquad \bar{\alpha}_t \;=\; \exp\Big(-\int_0^t \beta_s \,\mathrm{d}s\Big), \tag{50.39}$$

and, as before, x_0 is the one-hot vector for the starting state, $\mathbf{e}_m$ the one-hot vector for the mask token. Notice that the transition probability mirrors the discrete-time formula closely, with the product of the discrete time schedule $\prod_k(1-\beta_k)$ replaced by the exponential of the integral of the function β_t.

Reverse Process: The reverse of the Markov noising process is also a Markov process. Moreover, the rate matrix of the reverse process, $\tilde{Q}_t$ can be written in terms of the rate matrix of the forward process:

$$(\tilde{Q}_t)_{ji} \;=\; (Q_t)_{ij}\, \frac{q(x_t = i)}{q(x_t = j)} \qquad \text{for } i \neq j. \tag{50.40}$$

The advantage of using continuous-time is that we can parameterise the per-position marginals of the reverse process without approximation because the probability of two events (e.g. two token changes) occurring at the same time is zero in continuous time.

There are several choices of parameterisations for the reverse process. We can parameterise the per-position rates $\tilde{Q}^i_{\theta,t}(\mathbf{x}_t, y^i)$ directly and set the overall rate to

$$\tilde{\mathbf{Q}}_{\theta,t}(\mathbf{x}_t, \mathbf{y}) = \sum_{i=1}^{L} \mathbb{1}\big(x_t^{\neg i} = y^{\neg i}\big)\, \tilde{Q}^i_{\theta,t}(\mathbf{x}_t, y^i).$$

Lou et al. [1082] uses the known relationship in Equation 50.40 and parameterise the unknown marginal density ratio $\frac{[q(\mathbf{y})]_i}{q(\mathbf{x}_t)}$ using a transformer, where the transformer produces a positive real value for each position i and token y^i in the vocabulary. Notice that the role of this density ratio is similar to that of the *score function* in the continuous diffusion, which acts as the drift that moves the particles of the reverse process towards the data distribution. Another, more standard approach [229] is to parameterise the data probability $p^i_\theta(x^i_0|\mathbf{x}_t)$ directly using the softmax output of a transformer, and set the reverse rate to

$$\tilde{\mathbf{Q}}_{\theta,t}(\mathbf{x}_t, \mathbf{y}) = \sum_{i=1}^{L} Q^i_t(y^i, x^i_t)\, \mathbb{1}\big(x_t^{\neg i} = y^{\neg i}\big) \sum_{x^i_0} \frac{q(y^i|x^i_0)}{q(x^i_t|x^i_0)}\, p^i_\theta(x^i_0|\mathbf{x}_t) \tag{50.41}$$

In the case of masking noise, this parameterisation leads to a simple training objective, which is a weighted sum of cross-entropy losses [1446, 1514]:

$$L_{\mathrm{VLB}}(\theta) = -\mathbb{E}_{t,\mathbf{x}_0,\mathbf{x}_t} \frac{\alpha_t}{1-\bar{\alpha}_t} \sum_{j=1}^{L} \mathbb{1}\Big(x^j_t = [\mathrm{MASK}]\Big) \log p_\theta(x^j_0|\mathbf{x}_t). \tag{50.42}$$

Note that the continuous-time weight $\alpha_t/(1-\bar{\alpha}_t)$ differs from the discrete-time form in Equation 50.11 above.

Sampling: Once we have the learned rate matrix for the reverse process ($\tilde{\mathbf{Q}}_{\theta,t}$), the simplest sampling method is to discretise the continuous-time process and simulate the transitions according to Equation 50.37. For time-inhomogeneous Markov chains, as in our case, this naive approach requires extremely small time steps to ensure accuracy. One can use more sophisticated methods like Gillespie's algorithm [631], which do not require explicit discretisation of time to simulate the process. However, these exact algorithms can be quite slow as they require stepping through each transition individually. In practice, it has been observed that for many applications, one can use fast approximate simulation based on Tau-leaping [630], where the time is discretised, but multiple transitions are performed in a single step. That is, multiple tokens in a sequence change their values simultaneously after a single evaluation of the neural network. The success of simultaneous sampling from masked diffusion models has led to a resurgence of interest in these models for applications such as code generation, where they outperform autoregressive models in terms of speed.

50.8 Chapter Summary

Denoising diffusion models have established themselves as a new pillar of generative modelling, delivering state-of-the-art sample quality with a stable and principled training objective.

- Their core mechanism consists of a fixed forward process q that adds noise and a learned reverse process p_θ that removes it.
- Their training is grounded in the variational inference framework (ELBO), sharing a theoretical lineage with VAEs.
- The central insight of DDPMs is the simplification of the ELBO to a weighted L_2 loss, which trains a network $\boldsymbol{\epsilon}_\theta$ to predict the noise added at a given timestep t.
- This discrete-time formulation is elegantly unified with score-based modelling through the continuous-time SDE framework, where $\boldsymbol{\epsilon}_\theta$ is shown to be a reparameterised score function. This view opens up flexible sampling solvers, including SDE, ODE, and predictor-corrector methods.
- Practical utility is enhanced by algorithms for sampling acceleration (DDIM) and precise conditional generation (classifier-free guidance, CFG).
- The framework is flexible and generalisable from continuous domains (images, audio) to discrete domains (text, graphs) by replacing the Gaussian perturbation kernel with a discrete transition matrix.

50.9 Bibliographical Notes and Further Reading

The concept of diffusion for generative modelling was first proposed by Sohl-Dickstein et al. [1532], framing it in the context of non-equilibrium thermodynamics. The modern resurgence was sparked by the work of Ho et al. [755], who introduced the specific parameterisations (DDPM) and simplifications that made these models practical and state-of-the-art. Independently, Song and Ermon [1540] developed a parallel track of score-based generative models, which was later unified with DDPMs in the SDE framework by Song et al. [1541].

Key practical advancements include the development of Denoising Diffusion Implicit Models (DDIM) for fast sampling by Song et al. [1538], and the introduction of classifier-free guidance (CFG) by Ho and Salimans [753], which is now the standard for conditional image generation. Latent diffusion models were introduced by Vahdat et al. [1650] and later improved upon by Rombach et al. [1417]. Subsequent work refined the training and conditioning recipe for large-scale text-to-image generation [1333], and replaced the latent U-Net denoiser with a transformer operating on latent patches [1309].

A complementary line of work that aims to reduce the number of sampling steps is one-step and few-step generation. The literature in this area has developed along several related directions. A useful starting point is the work of Song et al. [1542] on consistency models, which introduces functions that remain invariant along trajectories of the probability-flow ODE and can therefore map noise to data in a single evaluation, while also supporting multistep sampling when additional computation is available. For approaches based more explicitly on distillation, see distribution matching distillation (DMD) [1817], which trains a one-step generator using differences between the scores of the target and generated distributions. Score identity distillation [1871] and its adversarial extension, SiDA [1872], provide a related perspective based on Fisher-divergence objectives and the use of real-data adversarial supervision. MeanFlow offers a complementary route through flow-based modeling, wherein rather than

numerically integrating an instantaneous velocity field over many steps, it learns the average velocity over a finite time interval, making one-step transport possible [605]. DiffRatio is also relevant in this context, especially for readers interested in density-ratio formulations. It estimates ratios between noised data and model distributions, whose log-gradients recover the score differences needed to train a one-step generator without direct teacher-score supervision [276].

Discrete diffusion models also have a long history. The original formulation of Sohl-Dickstein et al. [1532] included a binomial diffusion process for binary data. A more general DDPM-style treatment of structured categorical state spaces was developed by Austin et al. [63], whose discrete denoising diffusion probabilistic models (D3PMs) support uniform, absorbing-state, and structure-aware transition kernels. Campbell et al. [229] subsequently formulated the forward and reverse processes as continuous-time Markov chains, providing a general continuous-time framework for discrete denoising models. Several later works improved the objectives used to train discrete diffusion models for language. Lou et al. [1082] introduced score-entropy discrete diffusion, which parameterises the reverse process through ratios of probabilities under the noised data distribution. For absorbing-state or masked diffusion, Shi et al. [1514] showed that the continuous-time variational objective reduces to a weighted integral of cross-entropy losses. Independently, Sahoo et al. [1446] derived a Rao-Blackwellised objective expressible as a mixture of masked-language-modeling losses and demonstrated that masked diffusion language models can approach autoregressive perplexity when trained with an appropriate architecture and optimization recipe. For the uniform-state setting, the Diffusion Duality (Duo) relates discrete diffusion to an underlying Gaussian process and uses this connection to adapt techniques from continuous diffusion, including consistency distillation for few-step generation [1447]. Variable-length generation requires models whose state space includes sequences of different lengths. Recent literature has developed insertion-based generative models, wherein insertion of tokens is viewed as the *denoising* operation [1300]. This idea has also been extended to use continuous-time using continuous-time Markov chains in which the forward noising process deletes elements and generative process inserts them [428, 1301]. EditFlows generalize the idea to allow for insertions, deletions, and substitutions within a continuous-time flow-matching framework [716]. Flexible length masked diffusion (FlexMDM) offers a related construction in which sequence length changes through the insertion of mask tokens that are subsequently denoised [887].

Chapter 51

Prior-data Fitted Networks

Chapter 3 introduced Simulation-Based Inference (SBI) as a powerful framework for performing Bayesian inference $p(\theta \mid X)$ when we cannot measure the likelihood $p(X \mid \theta)$, but only sample from it. However, in many scenarios, one may not be interested in estimating the latent parameters θ, but seek to make supervised probabilistic predictions. In a supervised prediction setting we have a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ of pairs of input features and labels, and our task is to predict the label for features x_{test} based on $\mathcal{D}$. We are thus primarily interested in the posterior predictive distribution (PPD) $p(y_{\text{test}} \mid x_{\text{test}}, \mathcal{D})$. The PPD is typically approximated using the posterior over the latent variables $p(\theta \mid \mathcal{D})$ in Bayesian prediction. In this chapter, we instead directly amortise the computation of the PPD without explicitly computing a posterior over latent variables. Müller et al. [1208] proposed this approach, including a parameterisation using neural networks, as *Prior-data Fitted Networks* (PFNs)[1].

A PFN is any neural network that amortises the posterior over the label y_{test}, unlike SBI which amortises the posterior inference of the latent variable θ.

51.1 From Inference to Prediction

To understand the connection between inference of latent variables θ, and the prediction of y_{test}, note that the PPD can be defined using the posterior of θ as

$$p(y_{\text{test}}|x_{\text{test}}, \mathcal{D}) = \int p(y_{\text{test}}|x_{\text{test}}, \theta)p(\theta|\mathcal{D})\,\mathrm{d}\theta, \tag{51.1}$$

where θ are the latent variables, and $p(\theta \mid \mathcal{D})$ is the posterior over θ given the training data.

Using SBI (or any other posterior inference method described in this book), we can formulate a two-step process to approximate the PPD:

1. Inference: Use the inference method to obtain an approximate posterior $q(\theta \mid \mathcal{D}) \approx p(\theta \mid \mathcal{D})$.
2. Prediction: Approximate the PPD integral by drawing samples $\theta_j \sim q(\theta \mid \mathcal{D})$ and averaging their predictions (i.e., a Monte Carlo estimate):

$$p(y \mid x, \mathcal{D}) \approx \frac{1}{M} \sum_{j=1}^{M} p(y \mid x, \theta_j) \tag{51.2}$$

The inference step can be inaccurate, as q has to typically model a complex multivariate distribution, and the prediction step requires sampling many times from the approximate posterior q_ψ.

In this chapter, we make this a single step instead: One can bypass the explicit estimation of the latent posterior $p(\theta \mid \mathcal{D})$ and instead learn an amortised model that directly approximates the posterior predictive distribution $p(y \mid x, \mathcal{D})$. PFNs do this by training a neural network to make the right predictions in one forward pass. This is still amortising, just like SBI, but the amortisation now targets the label and not the latent parameters. Instead of training a model to output the posterior over the latent, the model is trained to output the predictive posterior for a supervised learning problem.

[1] The idea was previously explored in some preliminary experiments by Garnelo et al. [582].

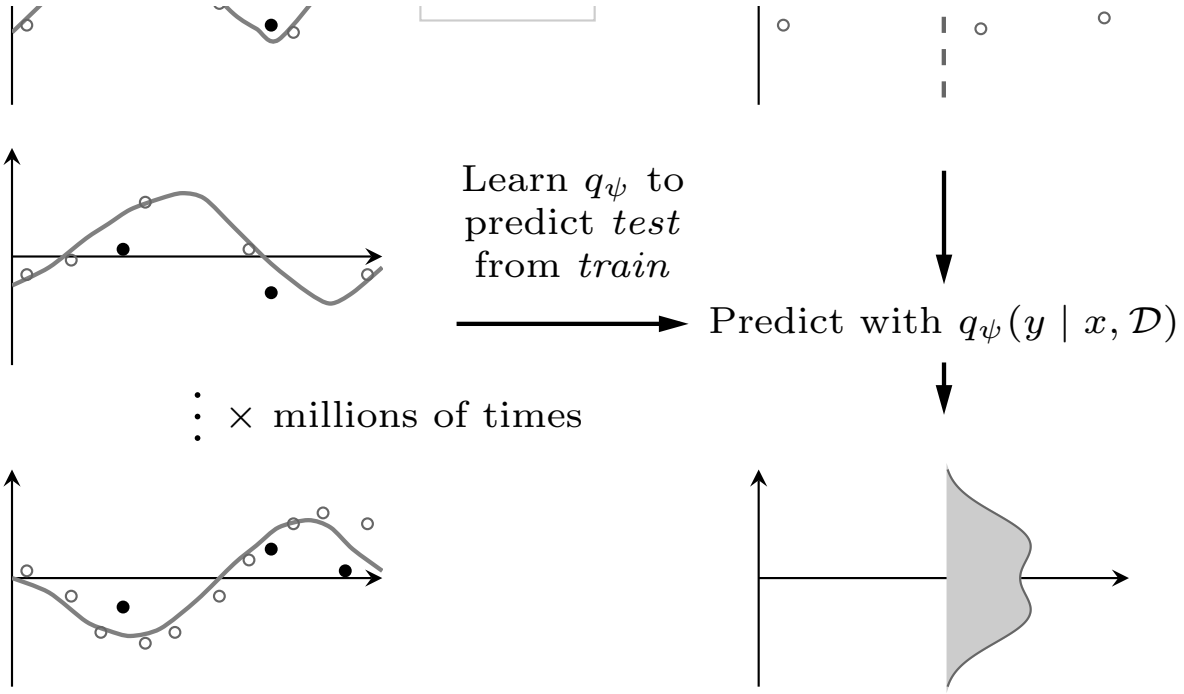


Figure 51.1: An outline of prior-fitting and prediction on a 1D Gaussian Process prior. We first fit the model q_ψ to predict held-out labels on sampled datasets. Then, we use this model to make predictions on new real-world data by feeding it the context. On the bottom right we see the PDF of the model's output, its posterior approximation.

51.2 Prior Fitting

PFNs are trained on entire datasets sampled from the prior. The core idea is simple: one generates a large number of synthetic datasets from the prior and the PFN is fit to predict held-out labels given all the remaining data for all datasets. We illustrate this process in Figure 51.1. Building dataset priors is possible for a very large class of priors, as we describe in the next section.

More formally, a PFN q_ψ with parameters ψ is trained to minimise the cross-entropy loss

$$\mathcal{L}(\psi) = \mathbb{E}_{\mathcal{D}\cup\{(x_{\text{test}},y_{\text{test}})\}\sim p(\mathcal{D})}\left[-\log q_\psi(y_{\text{test}}|x_{\text{test}},\mathcal{D})\right], \tag{51.3}$$

where $p(\mathcal{D})$ is a prior over datasets. This objective alone is enough to ensure that the PFN approximates the PPD.

The objective $\mathcal{L}(\psi)$ is similar to the NPE loss in Equation (3.3), but targets the predictive distribution directly rather than the posterior over latents (see Chapter 3 for NPE, NLE, and NRE). We can show that the learned model converges to the true PPD under appropriate conditions (sufficient model capacity, well-specified prior, convergence to the global minimum) [541, 1208]. To do this, we can rewrite the loss as

$$\begin{aligned}
\mathcal{L}(\psi) &= \mathbb{E}_{\mathcal{D}\cup\{(x_{\text{test}},y_{\text{test}})\}\sim p(\mathcal{D})}\left[-\log q_\psi(y_{\text{test}}|x_{\text{test}},\mathcal{D})\right] \\
&= \mathbb{E}_{\mathcal{D},x_{\text{test}}\sim p(\mathcal{D},x_{\text{test}})}\mathbb{E}_{y_{\text{test}}\sim p(y_{\text{test}}|x_{\text{test}},\mathcal{D})}\left[-\log q_\psi(y_{\text{test}}|x_{\text{test}},\mathcal{D})\right] \\
&= \mathbb{E}_{\mathcal{D},x_{\text{test}}\sim p(\mathcal{D},x_{\text{test}})}\int -p(y_{\text{test}}|x_{\text{test}},\mathcal{D})\log q_\psi(y_{\text{test}}|x_{\text{test}},\mathcal{D})dy_{\text{test}} \\
&= \mathbb{E}_{\mathcal{D},x_{\text{test}}\sim p(\mathcal{D},x_{\text{test}})}\mathrm{H}(p(\cdot|x_{\text{test}},\mathcal{D}),q_\psi(\cdot|x_{\text{test}},\mathcal{D})) \\
&= \mathbb{E}_{\mathcal{D},x_{\text{test}}\sim p(\mathcal{D},x_{\text{test}})}\mathrm{KL}(p(\cdot|x_{\text{test}},\mathcal{D})||q_\psi(\cdot|x_{\text{test}},\mathcal{D})) + C,
\end{aligned} \tag{51.4}$$

where C is a constant that is independent of q_ψ and H denotes cross-entropy. Minimising this loss thus means minimising the KL-divergence to the posterior on tasks sampled from the prior. A distribution that has a KL-divergence of 0.0 with another distribution is the same as the other distribution almost everywhere. Therefore, minimising this loss yields $q_\psi(y_{\text{test}}|x_{\text{test}},\mathcal{D}) \approx p(y_{\text{test}}|x_{\text{test}},\mathcal{D})$ for almost all $(x_{\text{test}},\mathcal{D})$ in the support of the prior.

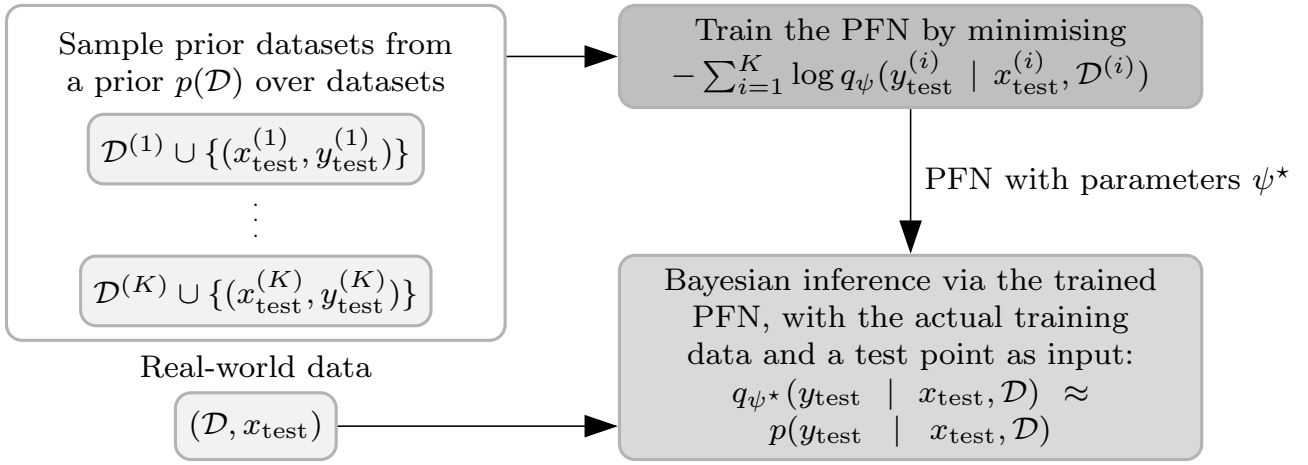


Figure 51.2: This figure outlines the components of the PFN training process more abstractly. We approximate Equation 51.3 by sampling datasets from the prior in small batches and the process yields a model that approximates the PPD.

The objective $\mathcal{L}(\psi)$ is optimised using standard deep learning techniques: sampling batches of datasets and performing stochastic gradient descent on them using Adam [894]. We outline the training process and the resulting model's behaviour in Figure 51.2.

51.3 Dataset Priors

To train PFNs, we need to define priors over datasets $p(\mathcal{D})$. In many scenarios, an applicable Bayesian model $f_\theta(x)$ mapping x to an output, or more generally, a likelihood $p(y|x, \theta)$ with parameters θ sampled from prior $p(\theta)$ already exists, but its inference has generally been carried out with traditional Bayesian methods. In these cases, a prior over datasets can be defined as $p(\mathcal{D}) = \int p(\theta) \prod_{0<i\leq N} p(x_i)p(y_i|x_i, \theta)d\theta$, where $p(x)$ is usually chosen to be easy to sample and have wide coverage, e.g., $x \sim \mathcal{N}(0, 1)$. Note that defining the prior using the conditional distribution $p(y|x, \theta)$ is a common technique, but not the only one, e.g., TabPFNs define $p(\mathcal{D}|\theta)$ directly. The process to sample one dataset based on a latent prior $p(\theta)$ can be outlined as:

1. First, sample a latent parameter vector $\theta \sim p(\theta)$.
2. Second, sample a full dataset $\mathcal{D} = \{(x_i, y_i) \sim p(x, y \mid \theta)\}_{i=1}^{N}$ from the likelihood defined by θ. If θ only defines the likelihood $p(y|x, \theta)$, one typically samples x from some uninformative distribution.

This prior $p(\mathcal{D}) = \int p(\mathcal{D} \mid \theta)p(\theta)\,\mathrm{d}\theta$ defines the assumptions about the kinds of datasets we expect to see. Critically, just as with SBI, the approach only requires sampling from the prior and the likelihood. This allows modellers to define rich and complex priors, such as Bayesian neural networks (BNNs) with uncertainty over their architecture, which we outline later in this section.

Most of the PFN priors used thus far are inspired by priors in traditional Bayesian inference setups, such as Bayesian neural networks or Gaussian processes, thus rely on a latent sampling setup as explained above. PFNs tend to have wider priors compared to traditional methods, though, as they both: i) work with latents that are really hard to model directly, like the architecturally uncertain BNN above, and ii) need to be expensively retrained for every change of prior, thus making it important to choose priors that generalise across tasks. This is, e.g., the case for the TabPFNs [769, 770] or for PFNs4BO [1209], which target whole data modalities (tabular classification, regression and Bayesian optimisation). Nevertheless, there are also domain-specific PFNs, similar to NPEs, e.g., to predict RNA folding times [1469].

As examples of typical priors used with PFNs, we detail two very simple, educational priors and their more practical extensions. The priors we look into are (i) Bayesian linear regression / neural network priors and (ii) Gaussian process (GP) priors.

51.3.1 A Bayesian Linear Regression Prior

Bayesian linear regression provides one of the simplest yet most instructive priors for understanding PFNs.

For educational purposes, we define a Bayesian linear regression prior, whose mean corresponds to ridge regression predictions with regularisation parameter $\alpha \in \mathbb{R}$. The prior uses another parameter σ^2 to control the noise level relative to α. A dataset with N samples and F features can be sampled from this prior using the following steps:

1. Sample weights $w \in \mathbb{R}^F$ as $w_i \sim \mathcal{N}(0, \sigma^2/\alpha)$.
2. Sample a bias $b \sim \mathcal{N}(0, \sigma^2/\alpha^{-1})$.
3. Sample inputs $X \in \mathbb{R}^{N \times F}$ independently from $\mathcal{N}(0,1)$ or $\mathcal{U}(0,1)$.
4. Compute the noiseless predictions $\mu = Xw + b$.
5. Sample noisy targets $y_i \sim \mathcal{N}(\mu_i, \sigma^2)$ for $i = 1, \ldots, N$.
6. Return the dataset $\{(X_i, y_i)\}_{i=1}^N$.

To clarify the connection to the expected PPD, we write out the underlying likelihood and parameter prior. The parameter prior is given by

$$p(w_0, \ldots, w_{F-1}, b) = \left(\prod_{0 \le i < F} f_{\mathcal{N}(0, \sigma^2/\alpha^{-1})}(w_i) \right) \cdot f_{\mathcal{N}(0, \sigma^2/\alpha^{-1})}(b), \tag{51.5}$$

where $f_{\mathcal{N}(\mu, \sigma^2)}$ denotes the density of the normal distribution with mean μ and variance σ^2. The likelihood is defined as

$$p(y|x, w, b) = f_{\mathcal{N}(x^T w + b, \sigma^2)}(y). \tag{51.6}$$

Going from these definitions to the prior sampling procedure boils down to the recipe given in the beginning of the section: sample from the prior $p(w_0, \ldots, w_{F-1}, b)$ first, sample inputs X (in an uninformative fashion), and finally sample the targets y in our dataset from $p(y|X_i, w, b)$ for $0 \le i < N$.

Further, it can be shown that the mean of the Bayesian linear regression predictions actually coincides with the ridge regression predictions, as ridge regression is a MAP estimate on the same prior, and the MAP coincides with the mean posterior as the posterior is a normal distribution; further, as the model is linear, one can factor out the posterior when computing an expectation over latents to get the PPD. Thus, the predictions of a PFN trained on this prior will in the limit have the same mean as the ridge regression prediction. We leave the exact proof as an exercise.

51.3.2 A Bayesian Neural Network Prior with Architectural Uncertainty

In this section, we define a Bayesian neural network (BNN) prior as an extension of the ridge regression prior presented above, which can be seen as a one-layer BNN prior without hidden layers. Notably, PFNs trained on the following prior extend traditional BNNs, which primarily quantify the uncertainty in the network weights, by also quantifying the uncertainty over architectural choices and weight-sparsity patterns. This is difficult to achieve with traditional methods such as VI (Chapter 12) or MCMC (Chapter 1) as it would involve instantiating many neural network architectures at prediction time. For PFNs, on the other hand, it is just a matter of designing a dataset prior. This is a prior that is practically relevant, as slight modifications of it were successfully used for tabular prediction tasks [769] and Bayesian optimisation [1209].

The prior can be described by the following dataset sampling procedure, where the example values are taken from Müller et al. [1209] wherever possible.

1. First, sample the architecture of the neural network modelling the dataset.
   (a) Sample the number of layers $L \sim \mathcal{U}(\{L_{\min}, \ldots, L_{\max}\})$ (e.g., $L_{\min} = 8$, $L_{\max} = 15$).
   (b) Sample the number of units used in all hidden layers $H \sim \mathcal{U}(\{H_{\min}, \ldots, H_{\max}\})$ (e.g., $H_{\min} = 36$, $H_{\max} = 150$).
2. Sample the weights.
   (a) First, the standard deviation of all weights across the network is sampled as $\sigma \sim \mathcal{U}(\sigma_{\min}, \sigma_{\max})$ (e.g., $\sigma_{\min} = 0.1$, $\sigma_{\max} = 0.2$).

(b) Then, all weight matrices θ_l and bias vectors θ'_l in layers $0 \leq l < L$ are sampled independently from $\mathcal{N}(0, \sigma^2)$.

(c) Finally, the layers are sparsified by setting each weight to 0 with sparsity probability p_{sparse} (e.g., $p_{\text{sparse}} = 14.5\%$).

3. Sample inputs $X \in \mathbb{R}^{N \times F}$ independently and uniformly at random in $\mathcal{U}(0, 1)$, where N is the total number of samples in the training set and F is the number of features.

4. Pass the inputs as a batch through the neural network, defined in steps 1 and 2, to retrieve the output vector $y = \theta_{L-1}\text{ReLU}(\theta_{L-2}\text{ReLU}(\dots \text{ReLU}(\theta_0 X + \theta'_0)\dots) + \theta'_{L-2}) + \theta'_{L-1}$, where $y \in \mathbb{R}^N$.

5. Consolidate the dataset as $\{(X_i, y_i)\}_{0 \leq i < N}$.

We can now train the PFN on datasets generated like this by holding out a part of any given dataset as a test set and using the remaining part as the conditioning set. In doing so, the PFN learns to approximate Bayesian inference over the BNN, including its architecture and weight-sparsity pattern, in addition to the values of the weights. Notably, the theoretical justification behind this posterior approximation only holds for $X \in [0, 1]$, as the model is not trained on data outside of this range.

51.3.3 SCM Extension

The BNN from the previous section (Section 51.3.2) can now easily be further extended to a structural causal model (SCM) prior. SCM priors are generalisations of BNNs and widely used in tabular foundation models [769, 770, 1861]. SCMs are directed acyclic graphs of arbitrary shapes, where the value of a node is defined by some deterministic function of its ancestors' values and some noise.

The SCM definition is thus very broad, so even the BNN prior we presented previously could be seen as a narrow SCM prior, where each node of the graph is a neuron in the neural network, it always has the features as roots and the output as the single leaf of this graph. The functions are simply the linear layers, activations and noises.

To build a more proper SCM prior that allows for randomness and more complex dependencies, we can make the following two changes to the prior:

i) First, we add noise to all activations in the neural network. We can write the network now as $h_i = \text{ReLU}(\theta_i h_{i-1} + \theta'_i) + \epsilon_i$, where $h_{-1} = X$ and $y = \theta_{L-1} h_{L-2} + \theta'_{L-1}$ and $\epsilon_i \sim \mathcal{N}(0, \sigma^2_\epsilon)$. ii) Second, and most importantly, we get rid of the streamlined structure. Now when building the dataset, we don't use X as features and y as our target but sample both uniformly out of all activations $h_{i,j}$.

These two simple interventions turn the prior presented above into an SCM prior with complex interdependencies, e.g., one feature might be depending on another feature or the target. One potential problem that we did not encounter so far, though, is that we can't sample from $p(y|x, \theta)$ as y might be represented by an activation that is earlier in the graph (neural network) than some of the features in x. This is not a problem for PFN training, though, as all we need to do is sample from $p(x, y|\theta)$ to build a dataset with shared latents $\theta \sim p(\theta)$. We simply sample our data as before, but read off different values in the computational graph to build the dataset.

51.3.4 A Simple Gaussian Process Prior

We first consider a simple GP prior with fixed hyperparameters (for more background on this prior see Chapter 23), before introducing a more complex GP prior that is applicable to real-world data below. The following simple GP prior with fixed hyperparamaters is supposed to provide an intuition for how kernels can be used as dataset priors.

Datasets with F features and N samples can be sampled from a simple RBF GP prior with length-scale ℓ and a noise variance of σ^2 as:

1. Sample inputs $X \in \mathbb{R}^{N \times F}$ uniformly from $\mathcal{U}(0, 1)^F$.

2. Build the covariance matrix $\Sigma \in \mathbb{R}^{N \times N}$ using an RBF kernel:

$$\Sigma_{i,j} = k(X_i, X_j) = \exp\left(-\frac{r^2_{i,j}}{2\ell^2}\right), \tag{51.7}$$

where $r_{i,j} = \|X_i - X_j\|$ is the Euclidean distance between inputs.

3. Add observation noise to the diagonal: $\tilde{\Sigma} = \Sigma + \sigma^2 \mathbf{I}_N$.
4. Sample targets from the multivariate normal: $y \sim \mathcal{N}(0, \tilde{\Sigma})$.
5. Return the dataset $\{(X_i, y_i)\}_{i=1}^N$.

This simple prior will converge to behave like the GP with the same hyperparameters [1208, 1209]. It is very limited, though, as the hyperparameters that, e.g., control how steep the posterior can be, are fixed.

51.3.5 A Fully Bayesian Gaussian Process Prior

Previously, we outlined a simple GP prior with fixed kernel hyperparameters for educational purposes. We now extend this example to yield the more complex HEBO$^+$ prior presented by Müller et al. [1209]. Training on this prior lets PFNs approximate a Gaussian process with uncertainty over the hyperparameters and feature importances, which yields state-of-the-art Bayesian optimisation results [1209].

The sampling process for this prior is still simple and similar to the one above, but introduces some extra sampling steps:

1. First, a subset of the features is sampled to be used and not ignored by flipping a coin for each feature $i_f \sim \text{Ber}(p_{\text{use}})$ (e.g., $p_{\text{use}} = 0.85$), where p_{use} is the probability that a feature is used. We define $F' = \sum_f i_f$.
2. Next, as many length-scales $\ell \sim p(\ell)$ as relevant features ($i_f = 1$), a noise variance $\sigma^2 \sim p(\sigma^2)$, and an output scale $s \sim p(s)$ are sampled from pre-defined prior distributions.
3. Further, we sample the inputs $X \in \mathbb{R}^{N \times F}$ uniformly from $\mathcal{U}(0, 1)$.
4. Now, we build a covariance matrix using the Matérn-3/2 kernel $\Sigma_{i,j} = k(X'_i, X'_j) = \sigma^2 \left(1 + \frac{\sqrt{3}r}{\ell}\right) \exp\left(-\frac{\sqrt{3}r}{\ell}\right)$, where $r = \|X'_i - X'_j\|$ and $X' \in \mathbb{R}^{N \times F'}$ are the relevant features of X.
5. The targets can now be sampled as $y \sim \mathcal{N}(0, \Sigma + \sigma^2 \mathbf{I}_N)$, where $\mathbf{I}_N$ is the $N \times N$ identity matrix.

This process not only trains a PFN to approximate a classical Gaussian process with uncertainty over its hyperparameters, but additionally trains the model to do Bayesian inference over which subsets of features are inactive and can be ignored, a capability previously reserved to expensive Bayesian ensemble models like the sparse axis-aligned subspaces (SAAS) model of Eriksson and Jankowiak [497].

51.4 PFN Architectures

A key challenge in implementing PFNs is designing neural network architectures that work for the set-based PFN tasks. The architecture should be able to accept variable-length training sets and in most cases also variable numbers of features. Further, it is natural to embed invariances into the architecture present in the prediction task:

(i) **Train invariance**: The predictions should be invariant to the ordering of training samples in $\mathcal{D}$.

(ii) **Test independence**: When predicting multiple test samples at once, their predictions should be modelled independently of one another.

(iii) **Variable train size**: The architecture must handle datasets of varying sizes.

(iv) **Feature invariance**: In most tabular scenarios, the order of the features present in our inputs x should not matter and our models should thus be invariant to feature permutations. There are others, though, where this property is not desirable, e.g., image data.

All requirements, besides **Feature invariance**, are shared with *conditional neural processes* [582]. For their meta-learning context, see Chapter 27.

All PFN architectures to date are based on the Transformer [1669] even though that is not a requirement. In the following, we will discuss two popular types of architectures: the 1-D attention architecture originally proposed together with PFNs and the 2-D attention architecture proposed for TabPFNv2 [770].

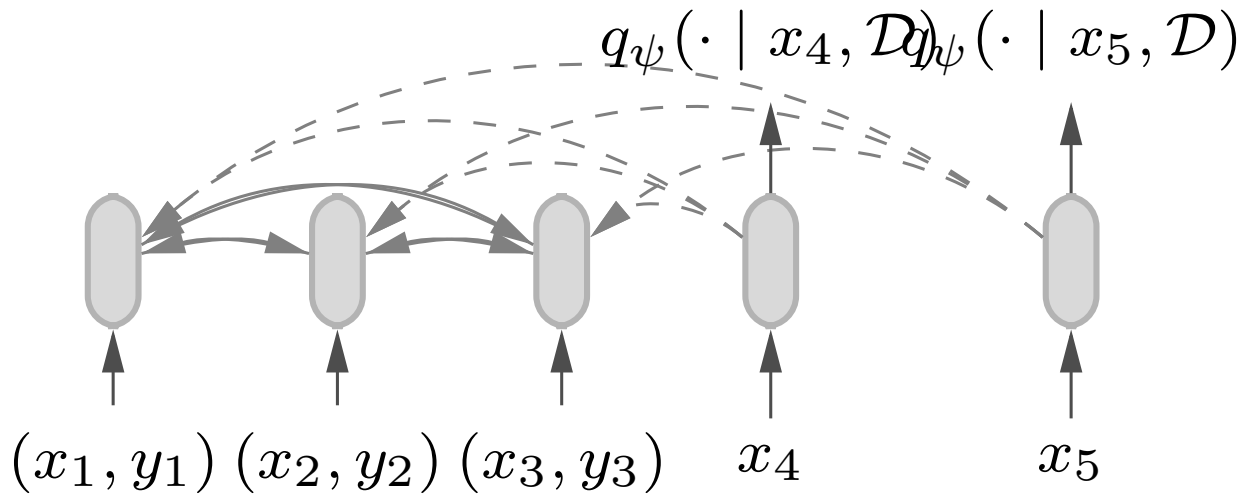


Figure 51.3: Original PFN and TabPFNv1 architecture. Dashed arrows indicate test attention, while solid arrows indicate train attention.

51.4.1 Original PFN Architecture

The original PFN architecture achieved a breakthrough in the accurate modelling of a general set of posteriors, and was most prominently used to power the TabPFNv1 model for tabular data [769]. It leverages a transformer architecture [1669] that was minimally modified to satisfy the invariance requirements (i-iii), akin to the architectural requirements of the conditional neural process, see Figure 51.3 for an illustration. The key modifications to the Transformer encoder are in the attention mechanism: training samples $(x_i, y_i) \in \mathcal{D}$ can attend to each other bidirectionally, but test inputs x_{test} can only attend to the training samples, and not to other test inputs. Further, any positional embeddings are removed. The output of the last layer provides the parameters of the predictive distribution $q_\psi(y_{\text{test}}|x_{\text{test}}, \mathcal{D})$ at each test position x_{test}. Nguyen and Grover [1245]'s Transformer Neural Processes later extended the architecture to additionally approximate the joint PPD of several test points.

We now describe the architecture more formally and with multiple test inputs $\{x_{\text{test}}^{(j)}\}_{j=1}^{n_{\text{te}}}$ that are handled independently of each other by the model. The transformer maintains one state per training sample and per test input. Throughout, we use N for the number of training samples and n_{te} for the number of test inputs $\{x_{\text{test}}^{(j)}\}_{j=1}^{n_{\text{te}}}$, d for the embedding dimension, L for the number of transformer layers, H for the number of attention heads (with per-head width d_h), $e_x(\cdot)$, $e_y(\cdot)$ for affine encoders, $W_{Q,h}^{(\ell)}, W_{K,h}^{(\ell)}, W_{V,h}^{(\ell)}, W_O^{(\ell)}$ for layer parameters and $\text{FFN}^{(\ell)}$ for MLPs, LN for layer normalisation, and for the regression head we use parameters W_{h} and $\mathbf{b}_{\text{h}}$.

The model first encodes the raw dataset using two simple linear (affine) encoders $e_x(\cdot)$ and $e_y(\cdot)$.

$$\mathbf{r}_i^{(0)} = e_x(x_i) + e_y(y_i) \quad (i = 1, \ldots, N), \tag{51.8}$$

$$\mathbf{s}_j^{(0)} = e_x(x_{\text{test}}^{(j)}) \quad (j = 1, \ldots, n_{\text{te}}). \tag{51.9}$$

We do not explicitly mark the test points, as the model can infer that by seeing whether the bias vector of $e_y(\cdot)$ is present in the embedding. Stack our full state as a sequence $H^{(0)} = [\mathbf{r}_1^{(0)}, \ldots, \mathbf{r}_N^{(0)}, \mathbf{s}_1^{(0)}, \ldots, \mathbf{s}_{n_{\text{te}}}^{(0)}] \in \mathbb{R}^{(N+n_{\text{te}}) \times d}$.

The attention mask $A \in \mathbb{R}^{(N+n_{\text{te}}) \times (N+n_{\text{te}})}$ encodes the train invariance by letting all training samples attend to all training samples, and it encodes the independence of the test predictions by letting all test samples attend to all training samples and not to each other. The attention mask has the following form, where 0 means attention is possible, while $-\infty$ forbids attention

$$A_{uv} = \begin{cases} 0, & u \le N,\, v \le N \quad \text{(train}\leftrightarrow\text{train allowed)} \\ 0, & u > N,\, v \le N \quad \text{(test}\rightarrow\text{train allowed)} \\ -\infty, & \text{otherwise} \quad \text{(no train}\rightarrow\text{test, no test}\leftrightarrow\text{test).} \end{cases}$$

For each layer $\ell = 1, \ldots, L$ the architecture performs the standard transformer encoder operations for the given

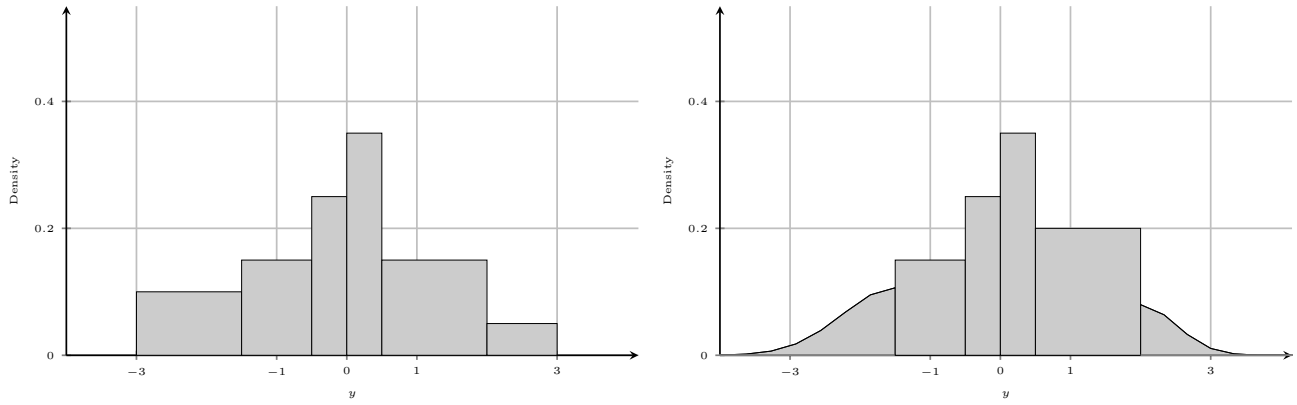


Figure 51.4: Two examples of a simple Riemann distribution with 7 borders (> 5 000 borders are common in practice). On the left we see a distribution without full support, while it is extended to have full support on the right by employing half-normals on the outermost bars.

state, which are

$$Q_h = H^{(\ell-1)}W_{Q,h}^{(\ell)},\ K_h = H^{(\ell-1)}W_{K,h}^{(\ell)},\ V_h = H^{(\ell-1)}W_{V,h}^{(\ell)}, \tag{51.10}$$

$$\mathrm{Attn}_h = \mathrm{softmax}\Big(\frac{Q_h K_h^\top}{\sqrt{d_h}} + A\Big)\, V_h, \tag{51.11}$$

$$\tilde{H}^{(\ell)} = H^{(\ell-1)} + \mathrm{LN}\Big(\Big[\mathrm{Attn}_1 \| \ldots \| \mathrm{Attn}_H\Big] W_O^{(\ell)}\Big), \tag{51.12}$$

$$H^{(\ell)} = \tilde{H}^{(\ell)} + \mathrm{LN}\Big(\mathrm{FFN}^{(\ell)}(\tilde{H}^{(\ell)})\Big). \tag{51.13}$$

Finally, prediction heads are applied to the test states $\mathbf{s}_j^{(L)}$ to predict their respective y. For classification a standard softmax head is used, while curiously for regression a softmax is used as well. We will talk about this next.

A Versatile Output Distribution

PFNs that predict a scalar target y need to model the full distribution of $p(y|x, \mathcal{D})$ in a lot of use cases, necessitating a versatile output distribution. For this, PFNs typically use the so-called *Riemann distribution* [112, 1208], a continuous distribution with a piecewise constant density function, which discretises regression into a classification problem. It can be written as $q_\psi(y \,|\, x_{\text{test}}^{(j)}, \mathcal{D}) = \sum_{b=1}^{B} \pi_{j,b}\, \mathcal{U}([a_{b-1}, a_b))$, where there are $B+1$ boundaries with values a_b and $\boldsymbol{\pi}_j = \mathrm{softmax}(W_{\mathrm{h}}\mathbf{s}_j^{(L)} + \mathbf{b}_{\mathrm{h}})$ is a standard softmax prediction. Its probability density function (PDF) looks like a histogram as we show in Figure 51.4 (left). The Riemann distribution is commonly used with half-normal distributions instead of the last block on each side, as that ensures the distribution's full support on $\mathbb{R}$. This is depicted in Figure 51.4 (right).

The Riemann distribution tends to require a significant amount of training data to attain high accuracy, but PFNs can be trained on virtually infinite data as the data is sampled from a prior. The upside of this distribution is that it allows the PFN to model a wide variety of posterior distributions, e.g., multimodal or asymmetrical distributions.

51.4.2 TabPFNv2 Architecture with 2-D Attention

Hollmann et al. [770] updated the PFN architecture introduced above to include attention across features, in addition to attention across samples. This type of architecture is becoming adopted quickly across tabular foundation models [408, 1861] and other applications like graph foundation models [495] and causal estimation [1414]. This updated architecture is further optimised for tabular tasks, where the inputs x are naturally modelled as a vector with permutation invariance. While previously every sample shared one state in the transformer, now every feature and every target is represented by a separate state, and we have one batched attention layer that attends across samples per feature as before, and one that attends across features per sample. The architecture is outlined in Figure 51.5.

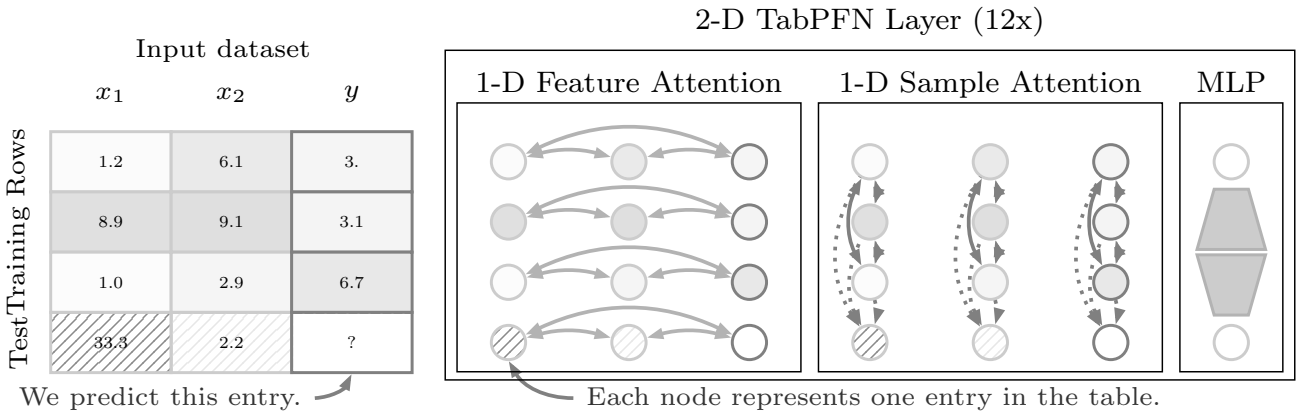


Figure 51.5: The architectural outline of the now commonly used TabPFNv2 architecture with 2-D attention.

51.5 Applications of Prior-data Fitted Networks

PFNs are used across a diverse set of domains – some PFN's capabilities are very general, others narrow. So far, all of these domains share that the models work on featurised data, that is, x is a vector of meaningful entries. Since PFNs provide an approximation of the full PPD, they are also often used in settings where uncertainty estimation is important such as Bayesian optimisation. PFNs were so far only shown to work for smaller datasets of up to 50,000 points in context [1861], though. Let us review a set of applications.

51.5.1 Tabular Classification and Regression

In tabular prediction tasks, we are simply interested in using the PPD to label newly incoming datapoints for arbitrary featurised datasets. The TabPFN models [769, 770] popularised PFNs as foundation models for this application and strong descendants [1353, 1861] further established them in this domain. PFNs are, at the time of writing, seen as state-of-the-art on small to medium-sized tabular datasets (up to about 50,000 samples). They also tend to be much faster than previous methods, as they require only one or a few forward passes, rather than a whole training procedure encapsulated in a hyperparameter-tuning loop. These PFNs have priors that are carefully hand-crafted to represent tabular data well. The priors in this space are inspired by structural causal models, which take the flexible BNN example from Section 51.3.2 to an extreme. They allow for arbitrary sparsity, mixing of activation functions and noise distributions, mixing in of regression trees, as well as not restricting y to be the output and x to be the input of the network. These priors generate diverse synthetic tabular datasets, incorporating various types of feature relationships, noise patterns, and data distributions, that can also be seen in real-world data.

These models are a highlight of PFN development as they show how fast predictions can be with PFNs, while still performing very strongly on a very large and diverse domain. However, this domain might hit a ceiling of scaling PFNs to large in-context data, as posteriors eventually converge to a point estimate in the limit of infinite data, which renders Bayesian models less useful.

51.5.2 Bayesian Optimisation

As PFNs approximate the PPD, their output distribution can be leveraged for Bayesian optimisation (BO) [1209]. The goal of any BO algorithm is to maximise the output of an expensive-to-evaluate black-box function $f : \mathcal{X} \to \mathbb{R}$ with a limited number of function evaluations. The key component of any BO setup is a well-calibrated probabilistic model $p(f(x)|x, \mathcal{D})$, which PFNs provide. Its predictions are used to compute heuristics, so called acquisition functions, that give a trade-off between exploring new inputs x and exploiting regions, where f is known to return high values.

The classical probabilistic model used for BO is the GP. That is why in this domain it is most common to train PFNs on (generalisations of) GP priors. In this context, the advantages of PFNs compared to GPs lie especially in a wider choice of the prior, e.g., Müller et al. [1209] uses priors that involve ranges of kernel hyperparameters, input warping, and ignoring certain dimensions in a fully Bayesian way. Something that is very hard to achieve using MCMC sampling, which is usually employed for this purpose (see Chapter 1).

51.5.3 PFNs for SBI

Vetter et al. [1677] use the pre-trained TabPFNv2 to model the distribution over latent vectors in an SBI setup with a single fixed-size observation. That is, while PFNs can be seen as close kin to SBI, a pre-trained PFN can also be used to model the posterior in an SBI setting. The setup is that we have a prior $p(\theta)$ over latent vector θ and have a fixed size observation vector x, which we can sample via $p(x|\theta)$. The TabPFN is now used to model $p(\theta|x)$. While TabPFN is only trained to model univariate targets, Vetter et al. [1677] use multiple sequential TabPFN predictions to model $p(\theta|x)$ via the chain rule, predicting one entry after the other as $p(\theta_i|\theta_{<i}, x)$. TabPFN-based SBI showcases that while PFNs are trained on a similar objective compared to SBI, they typically target wider prior distributions, which, in this case entails some SBI applications.

51.5.4 Time Series Forecasting

Recent work has shown that PFNs can also be extended to the time domain, leaving the realm of permutation-invariant training and test sets. There are two successful lines of work to achieve this. i) One line of work [443, 1583] creates priors inspired by real-world time series to train PFNs on these with time being treated as a (specially encoded) feature in an architecture like the ones discussed above. ii) Another line of work uses pre-trained PFNs (TabPFN) to do forecasting, despite being trained only on tabular data [777]. This is achieved by converting time series data into a tabular format (e.g., using calendar features, lags).

51.5.5 Graphs

PFNs have very recently started to see some traction as a basis to train foundation models for node classification and regression. Similar to PFNs' value proposition for tabular data, they allow training a single foundation model that can perform tasks for which individual models used to be trained in a single forward-pass. There is work that does feature-engineering and fine-tuning on top of pre-trained tabular PFNs [496], but there is also work on building a prior for graphs specifically [495].

51.5.6 Causal Interventions

In tabular settings it is often of interest to understand the causal impact of changing an attribute on the outcome. In some cases the causal graph underlying a problem might be available, but in many cases it isn't. In these cases one previously needed to perform MCMC sampling over the very large, discrete set of possible causal graphs [80]. PFNs can amortise this effectively [80, 1099, 1414].

51.5.7 Learning Curve Extrapolation

PFNs have been applied to predict the future performance of machine learning models based on their training curves [6]. This is useful for hyperparameter optimisation: by predicting whether a configuration will ultimately achieve good performance, we can terminate unpromising runs early and allocate resources more efficiently. The PFN is trained on synthetic learning curves generated from a prior over training dynamics, enabling it to forecast performance with well-calibrated uncertainty.

51.5.8 Invariant Learning

PFNs, similar to Gaussian processes in many domains, allow adding invariances to models that might be necessary in some downstream tasks. This is explored for tabular prediction by Arbel et al. [44], who make classification PFNs equivariant to the order of the classes, and by Helli et al. [731], who make PFNs invariant to distribution shifts.

51.5.9 Aleatoric and Epistemic Uncertainty Decomposition

Similar to Gaussian processes, PFNs can be trained to predict both the noisy outcomes y and a non-noisy version f to differentiate aleatoric and epistemic uncertainty [125]. In Bergna et al. [125], the authors do this simply by training a PFN with two output heads one on the clean outcomes and one on outcomes with added noise. Similar to GPs, though, this simple approach only yields an approximation to the true decomposition and only is applicable to certain priors (like GP priors).

Chapter 52

Advances in singular learning theory for BDL

Bayesian deep learning is often presented as a clean marriage between Bayesian inference and deep neural networks. In practice, however, the mathematical picture many researchers carry over from standard statistical theory does not survive contact with the singular reality of deep learning. This mismatch leads to persistent misconceptions – not just about mathematical details, but about what quantities truly matter for understanding model complexity and predictive performance.

Singular learning theory (SLT), developed by Sumio Watanabe, is the asymptotic theory of Bayesian inference in singular models, i.e., models whose parameter–likelihood map is non-identifiable and/or Fisher information matrix is degenerate [1723]. Unlike in classical (regular) statistics – where each parameter corresponds to a unique distribution and Fisher information is positive definite – singular models have parameter symmetries, redundancies, and other complex geometric features which invalidate the standard quadratic approximation to the log-likelihood that underpins much of standard asymptotic statistical theory [1726, Ch. 1–2]. Such singular structures are pervasive in modern machine learning — from neural networks to mixture and latent-variable models — making SLT the natural asymptotic framework for their analysis.

SLT replaces the Gaussian asymptotics of regular models with an *algebraic–geometric* asymptotic description of the posterior [1723] controlled by the singularities of the Kullback–Leibler divergence between the true data-generating mechanism and the model, considered as function $K(w)$ of the model parameters. The key invariant of this KL divergence function is the **learning coefficient** λ, which controls the asymptotic behaviour of the volume of parameter space where K is small (see Equation (52.2) below). After a suitable reparametrisation—known as *resolution of singularities*—$K(w)$ locally factors into monomials and λ can be read off the exponents of these monomials [1726, Ch. 5, Def. 17; Ch. 6, Thm. 17] (Equation (52.6)). The learning coefficient can be regarded as a generalised parameter count that reduces to simple parameter counting in regular models. One important feature of the learning coefficient is that it depends on the true data-generating mechanism, and in which ways the model can approximate it.

The learning coefficient governs the first order behaviour of every major asymptotic property of Bayesian learning: the log marginal likelihood contains a $\lambda \log n$ term (Equation (52.10) below, [1726, Ch. 5, Thm. 11]), the Bayes generalisation error decays as λ/n (Equation (52.16), [1726, Ch. 5, Thm. 15]), and model selection criteria such as WAIC or WBIC provide valid extensions of AIC and BIC to singular models [1726, Ch. 8].

SLT is well suited to providing a rigorous language for describing the geometric degeneracies present in deep learning – permutation symmetries, scaling symmetries, and redundant units – that make standard Bayesian approximations fail. It tells us how Bayesian inference behaves when multiple parameter settings yield the same function, offering the only known asymptotic theory (in the large data limit $n \to \infty$) that remains correct in this regime[1]. Besides the two textbooks [1723, 1726], we also recommend the surveys [1722, 1727].

In this chapter, we use SLT to clear up four common misconceptions in Bayesian deep learning:

Misconception 1: The Laplace approximation is an acceptable general-purpose asymptotic theory for Bayesian posteriors, even in singular models.

Misconception 2: The effective dimensionality of Bayesian neural networks is accurately and completely captured by the eigenspectrum of the Hessian.

[1]In the limit where both $n, d \to \infty$ together, Bayesian asymptotic theory has also been studied for some specific models and data distributions, see e.g. [522, 1281]. What is perhaps most distinctive about SLT compared to these results is its generality.

Misconception 3: The generalisation error of the Bayes posterior predictive distribution is similar to that of the maximum a posteriori (MAP) estimator and the maximum likelihood estimator (MLE), asymptotically speaking, in singular models.

Misconception 4: Optimising the evidence lower bound (ELBO) in variational inference necessarily leads to good downstream predictive performance.

Behind all four misconceptions lurks a more basic objection: that singularities cannot matter in practice because real data never places a model exactly at a singularity. Section 52.2.1 addresses this objection directly: for a model that is merely *close* to singular, the singular description of Bayesian inference can be more accurate than the regular one over a very large range of sample sizes.

For each misconception, we will explain why the claim is false, present the mathematically correct statement, and illustrate its implications for Bayesian deep learning. Before doing so, we first give a primer on singular learning theory as it pertains to Bayesian deep learning.

52.1 Setup

Consider the problem of *parametric density estimation*: we are given data $\mathcal{D}_n = \{x_i\}_{i=1}^n$ sampled i.i.d. from a true data-generating distribution $p_0(x)$ on a sample space $X \subset \mathbb{R}^N$, we choose a (finite-dimensional) parametric model $p(x \mid w)$ of distributions on X indexed by a parameter $w \in \mathcal{W} \subset \mathbb{R}^d$, and we want to use this model to learn $p_0(x)$.

Superficially, this is quite different from (supervised) deep learning, where the model is a family of deterministic mappings $x \mapsto f_w(x)$ where w is the collection of neural network weights. However from a statistical and Bayesian perspective, we can augment such deterministic models by replacing $f_w(x)$ by a family of conditional distributions $p(y|x, w)$ with mean $f_w(x)$ and by modelling the input data distribution $p(x)$. This then turns Bayesian deep learning into a parametric density estimation problem in the sense above (on $X \times Y$). The deep linear network model in Section 52.2.2 is an example of this construction.

In the Bayesian formulation, uncertainty about the parameters is expressed through a prior distribution $\varphi(w)$ on w. The matter of eliciting a meaningful prior over neural network weights is nontrivial and forms an active research area; see Chapters 28–31 for a detailed treatment of priors in BDL. In this introductory chapter, we will work with a general prior $\varphi(w)$, which for simplicity of exposition we assume positive everywhere on $\mathcal{W}$.[2]

We will also assume that $p(x|w)$ and $\varphi(w)$ are **real analytic**. In theory, this assumption rules out neural networks that use ReLU activation functions. However, this is not a real obstacle as ReLU networks are still **piecewise analytic**, and are also known empirically to behave very similarly to related analytic activations like SiLU; we refer the reader to Wei et al. [1733] for further details. We will further assume the model is *well-specified* (or *realisable*, in the terminology of Watanabe), i.e., there exists $w_0 \in \mathcal{W}$ such that the true distribution $p_0(x)$ is realised as

$$p_0(x) = p(x \mid w_0).$$

This assumption deserves a comment, since it is in tension with Bayesian deep learning practice: for realistic data distributions and network architectures, no parameter reproduces $p_0(x)$ exactly, so practitioners essentially never operate in the realisable regime. We assume realisability purely for ease of exposition. The SLT results presented in this chapter do not require it: they extend to non-realisable settings in which the variance of the log-likelihood ratio process can be suitably controlled, the *relatively finite variance* condition of Watanabe [1726, Section 3.1, Definition 7]. We return to this point in Section 52.2.3, where realisability makes its only substantive appearance in this chapter: it is used to control the variance of the empirical process ξ_n underlying Watanabe's asymptotics.

Define the *cross-entropy* between $p_0(\cdot)$ and $p(\cdot \mid w)$ (or *population negative log-likelihood*) as

$$L(w) := -\int p_0(x) \log p(x \mid w)\, dx,$$

and the closely related *Kullback–Leibler (KL) divergence* between $p_0(\cdot)$ and $p(\cdot \mid w)$ as

$$K(w) := \mathrm{KL}(p_0(\cdot) \,\|\, p(\cdot \mid w)) = \int p_0(x) \log \frac{p_0(x)}{p(x \mid w)}\, dx,$$

[2] This assumption makes some of the statements cleaner, e.g. the learning coefficient λ becomes independent of the prior. The SLT framework does allow for more general priors; see [1723, Theorem 7.4] for how the learning coefficient changes when using the Jeffreys prior.

and write the set of KL minimisers as

$$W_0 := \{w \in \mathcal{W} : K(w) = 0\}.$$

Because we have assumed realisability, W_0 is non-empty. Note that the difference $L - K$ (the differential entropy of $p_0(x)$) is independent of w.

Regular vs. singular models. A model is *regular* if it is identifiable and the Fisher information matrix at each $w_0 \in W_0$ is positive definite; otherwise it is *singular*. In regular models, identifiability implies that W_0 consists of a single point. In singular models, by contrast, W_0 is a real analytic set which usually fails to be an analytic manifold due to the presence of singularities [1723]. See Section 52.2 below for some simple examples of W_0.

Posterior distribution. Given data $\mathcal{D}_n = \{x_i\}_{i=1}^n$, the *empirical negative log-likelihood* is

$$L_n(w) = -\frac{1}{n}\sum_{i=1}^{n} \log p(x_i \mid w).$$

Most machine learning loss functions (mean-squared error, cross-entropy) can be written as L_n, in which case it might be called a training loss. The posterior distribution can be written in terms of L_n as

$$\pi(w \mid \mathcal{D}_n) \propto \exp\{-nL_n(w)\}\,\varphi(w)dw,$$

with normalising constant

$$Z_n = \int_{\mathcal{W}} e^{-nL_n(w)}\varphi(w)\,dw,$$

which is also known as the marginal likelihood. Related to the empirical negative log-likelihood is the empirical KL divergence

$$K_n(w) := \frac{1}{n}\sum_{i=1}^{n} \log \frac{p_0(x_i)}{p(x_i \mid w)},$$

with which we can alternatively write the posterior as

$$\pi(w \mid \mathcal{D}_n) \propto \exp\{-nK_n(w)\}\,\varphi(w)dw.$$

Let us call the corresponding normalising constant the *normalised marginal likelihood*

$$Z_n^0 = \int_{\mathcal{W}} e^{-nK_n(w)}\varphi(w)\,dw.$$

Note that the difference $K_n - L_n$ as well as the quotient Z_n^0/Z_n are independent of w (and related to the entropy of the data distribution). Moreover, treating the data as a random sample from $p_0(x)$, we have $L = \mathbb{E}_{p_0}[L_n]$ and $K = \mathbb{E}_{p_0}[K_n]$; in machine learning terminology, this means that L is the *population loss function*.

Posterior predictive distribution. Rather than relying on point estimates such as the MLE or MAP, Bayesian deep learning makes predictions by averaging over parameter uncertainty via the *posterior predictive distribution (PPD)*

$$\hat{p}_{\text{Bayes}}(x) := \int_{\mathcal{W}} p(x \mid w)\,\pi(w \mid \mathcal{D}_n)\,dw. \tag{52.1}$$

In regular models, where the posterior contracts to a point mass, the Bayes PPD behaves asymptotically like the MLE and MAP predictors [1726, Ch. 4.5]. In singular models, however, the posterior does not usually collapse to a single point, but rather concentrates around higher-dimensional subsets of W_0, and singular learning theory shows that the Bayes PPD achieves different expected generalisation error asymptotically than either MLE or MAP [1723, Ch. 6.4]; we will come back to this point in Section 52.5.

Now we come to a very important but subtle point. By the realisability assumption, the models $p(x|w_0)$ for all $w_0 \in W_0$ are all equal to the same (true) distribution. In the supervised context, this means that all models in W_0 compute the same function $f_0(x)$. However, they are *not* equal in the eyes of the Bayesian posterior! On reflection, this is quite intuitive because the structure of the model *around* different elements of W_0 can be quite different, and the Bayesian posterior is sensitive to this. This intuition predicts that the posterior distribution concentrates around a specific subset of "maximally singular" parameters $w_0 \in W_0$, those for which the model is minimally sensitive to small changes of parameters. SLT turns this vague picture into solid mathematics.

52.2 Deep learning is singular

Neural networks (deep or shallow) are *inherently* singular, for several overlapping reasons [1733]. First, there are global symmetries of the model, both discrete and continuous (see Chapter 32 for a systematic treatment of symmetries in Bayesian neural networks):

- **Permutation symmetries.** In a fully connected network with hidden units, permuting the units within a layer (and permuting the corresponding rows/columns of weight matrices and biases) leaves the input–output map unchanged. These permutations create large equivalence classes in $\mathcal{W}$ mapping to the same model output, so W_0 is not a singleton and therefore the probabilistic neural network is not identifiable.
- **Scaling symmetries.** For activation functions such as ReLU, scaling the incoming weights to a hidden unit by a constant c and scaling its outgoing weights by $1/c$ leaves the network output unchanged. This introduces continuous curves of equivalent parameters in W_0, again leading to non-identifiability.
- **Linear symmetries.** Whenever a neural network architecture involves a product AB of weight matrices, the transformation $(A, B) \mapsto (AP^{-1}, PB)$ with P an invertible matrix does not change the input–output map. This happens for example with the query and key matrices in the attention mechanism of transformers, and also for the toy model of deep linear networks (which we discuss briefly at the end of Section 52.2.2).
- **Batch and layer-norm.** Applying batch- and/or layer-normalisation makes the input–output map invariant under certain affine transformations of the weights.

Global continuous symmetries of the model have been studied extensively in Bayesian and geometric deep learning. For example, such symmetries force the Hessian of the empirical loss function L_n to be singular (have exactly vanishing eigenvalues) at every critical point, in particular everywhere on W_0. As such, these models fall outside the class of regular models and understanding their Bayesian learning theory requires SLT.

Besides these global (or *generic*) singularities, which are always present independently of the true data-generating mechanism $p_0(x)$, singularities can also arise from the relationship between $p_0(\cdot)$ and the model $p(\cdot|w)$, specifically, from the geometry of W_0 and of the model around parameters $w_0 \in W_0$. For concreteness, we will give examples in the context of neural networks, but such local singularities occur in essentially any parametric model with hierarchical structure, mixtures, latent variables, etc.

- **Redundant units.** Suppose that a network $f_{w_0}(x)$ for some $w_0 \in \mathcal{W}$ has a unit with zero incoming weights. Then the choice of outgoing weights does not change the input–output mapping. Geometrically, this means that W_0 contains a large submanifold around w_0, creating flat directions in the loss landscape. Similarly, suppose that $f_{w_0}(x)$ has two units with identical incoming weights. Then exchanging these units is an additional (discrete) symmetry.
- **Dead and saturated units** For ReLU networks, suppose that a particular ReLU unit never activates on the input data distribution (for example because its bias is very negative), then both incoming and outgoing weights can be changed in a neighbourhood of w_0 (up until the point that they change the preactivation distribution enough to activate the ReLU). Conversely, ReLU units that are always on behave like linear activations, and this leads to additional local linear symmetries of the type discussed above.

Because of such phenomena, the geometry of W_0 and the type of singularities depends on the true data-generating distribution $p_0(x)$. This turns out to have drastic consequences on asymptotic Bayesian inference, which are precisely quantified by SLT.

A simple example. Consider a two-layer, or one-hidden-layer, ReLU network

$$f_w(x) = v^\top \sigma(Ux),$$

with $U \in \mathbb{R}^{m \times p}$, $v \in \mathbb{R}^m$, and $\sigma(z) = \max(z, 0)$ applied elementwise. If $w = (U, v)$ is a solution with $K(U, v) = 0$, then so is (PU, Pv) for any permutation matrix P (unit permutation symmetry), and so is $(DU, D^{-1}v)$ for any invertible diagonal matrix D with positive entries (scaling symmetry). These transformations generate continuous and discrete families of equivalent parameters, and they will engender complex geometry in W_0.

Singularities and model complexity Note that, in the above examples, local singularities emerge because the distribution $p_0(x)$ is *special* or *simple* compared to the distribution $p(x|w)$ for a generic choice of $w \in \mathcal{W}$. Moreover, those examples are not the only phenomena that lead to such local singularities; there can be more subtle interactions between $p_0(x)$ and how it can be represented by the model.

This theoretical discussion is connected to an important empirical aspect of deep learning. Indeed, neural networks trained on "natural" data distributions seem to be able to learn them much more efficiently and with less overfitting than predicted by regular statistical learning theory given their parameter count. From the singular Bayesian perspective, this precisely suggests that neural networks and natural data distributions are special in the sense that trained models (in W_0) have many additional local singularities.

52.2.1 Do singularities really matter?

A common misconception about SLT is that singularities, especially local singularities which depend on the data-generating distribution $p_0(x)$, are irrelevant because they are never exactly realised in practice.

The idea has a grain of truth to it; for a given model, the set of parameters in $\mathcal{W}$ which exhibit additional singularities (besides the generic ones coming from architectural symmetries) form a set of measure 0, and almost every small perturbation of $p_0(x)$ leads to a situation with only generic symmetries. While models with generic symmetries are still singular and one cannot apply regular statistical theory to them, there are sometimes simple transformations of the model which "mod out" those symmetries and reduce to the regular case.

However, we have to remember that both regular statistical learning theory and SLT are *asymptotic* descriptions of learning as $n \to \infty$. When the model $(p(x|w), p_0(x))$ is regular but $p_0(x)$ is very close to $p_0'(x)$ such that $(p(x|w), p_0'(x))$ is very singular, SLT applied to the $(p(x|w), p_0'(x))$ can be a more accurate description of Bayesian inference than the regular theory applied to $(p(x|w), p_0(x))$ for a possibly very large range of n (which can very much include the size of the dataset we actually care about, if that data is insufficient to distinguish between $p_0(x)$ and $p_0'(x)$!). In other words, for an "almost singular" model, you may need a *lot* of data for asymptotic normality of the posterior to kick in! See [1726, Figure 1.5] for an illustration of this in a simple mixture model.

Takeaways:

Because of these built-in symmetries and redundancies, deep networks *always* violate the regularity conditions used in classical asymptotic theory. Moreover, they interact in a complicated way with the precise structure of the training data distribution, and different solutions in W_0 can have different generalisation behaviour on new data. Deep learning therefore fits especially well with the framework of SLT, which is precisely designed to tackle those phenomena.

52.2.2 The learning coefficient

In regular models, a quadratic (Laplace) expansion of the log-likelihood around the true parameter yields Gaussian asymptotics of the posterior distribution as $n \to \infty$ (the celebrated Bernstein–von Mises theorem). In singular models, the posterior is not asymptotically normal and one cannot apply these standard results.

One fundamental insight of SLT is that there is still an asymptotic representation which is controlled by the singularities of $K(w)$. The precise form of this representation is complicated and we will only briefly discuss it in Section 52.2.3. However, its main downstream statistical effects are controlled by a single number, the *learning coefficient* or *real log-canonical threshold* [3] *(RLCT)*, $\lambda = \lambda(K) > 0$, which we now introduce.

Volume of near-optimal parameters. For a small tolerance $\varepsilon > 0$, consider the prior-weighted volume

$$\mathrm{Vol}(\varepsilon) := \int_{K(w)<\varepsilon} \varphi(w)\, dw$$

of parameter values that achieve KL divergence no larger than ε. Then the RLCT is defined as the constant λ such that [1723, Thm. 7.1]

$$\mathrm{Vol}(\varepsilon) \sim C\, \varepsilon^{\lambda} (-\log(\varepsilon))^{m-1} \quad \text{as } \varepsilon \to 0, \tag{52.2}$$

for some constant $C > 0$ and some integer[4] $m \in \mathbb{N}_{\geq 1}$. The asymptotic (52.2) determines λ through a ratio in which both the constant C and the logarithmic correction cancel: for any $a > 1$,

$$\frac{\mathrm{Vol}(a\varepsilon)}{\mathrm{Vol}(\varepsilon)} \sim a^{\lambda} \left(\frac{-\log(\varepsilon) - \log(a)}{-\log(\varepsilon)} \right)^{m-1} \longrightarrow a^{\lambda} \quad \text{as } \varepsilon \to 0,$$

[3] The name "real log-canonical threshold" originates from singularity theory in algebraic and analytic geometry (we will not attempt to explain here what is "log-canonical" about it!). We usually use "real log-canonical threshold" when we consider λ as a geometric invariant of the function K, and "learning coefficient" when we emphasise its statistical significance.

[4] This *(real log-canonical) multiplicity* is a secondary geometric invariant of the singularities which we will largely ignore in this brief survey.

since the ratio of the logarithmic factors tends to 1. Taking logarithms yields

$$\lambda = \lim_{\varepsilon \to 0} \frac{1}{\log(a)} \log\left(\frac{\mathrm{Vol}(a\varepsilon)}{\mathrm{Vol}(\varepsilon)}\right). \tag{52.3}$$

In this definition, K can be any nonnegative real analytic function with $K^{-1}(0) \neq \emptyset$. It is then easy to see that λ is also the supremum of all $s > 0$ such that the function K^{-s} is locally integrable.

Regular and singular models. Recall that for regular models, $K(w)$ is locally a positive definite quadratic form (Morse lemma) so the set $\{K(w) < \varepsilon\}$ is an ellipsoid of radius $\propto \sqrt{\varepsilon}$. Combining Equation (52.2) with the formula for volumes of spheres (or, alternatively, combining the generalised Laplace asymptotic (52.4) of Section 52.2.3 with the standard Laplace approximation), we see that $\lambda = d/2$ for regular models.

Conversely, for any singular model, $K(w)$ can vanish along curves, surfaces, or other manifolds within $\mathcal{W}$. In fact, we then always have $\lambda < d/2$. Geometrically, λ quantifies how rapidly the set of nearly optimal parameters contracts as we tighten the accuracy requirement around the true distribution. Large values of λ correspond to sharp, isolated optima, while small values signal wide or degenerate valleys in parameter space where many different w yield nearly identical predictions. However, despite some apparent similarities, λ is of a different nature than curvature/flatness measures based e.g. on the Hessian eigenspectrum. In particular, λ is invariant under analytic changes of coordinates (bianalytic maps).

Deriving λ in closed-form. There are a few simple functions K where it is easy to derive λ. For example, $\lambda(u^{2k}) = \frac{1}{2k}$ and more generally $\lambda(u_1^{2k_1} u_2^{2k_2} \dots u_d^{2k_d}) = \min\left\{\frac{1}{2k_i}\right\}$. Beyond those simple cases, determining exact values of λ is challenging.

First off, λ depends on the true distribution $p_0(x)$, which we do not have access to. Moreover, even in theoretical settings where access to $p_0(x)$ is assumed, computing λ is in general a difficult algebraic geometry question. This has nevertheless been achieved in a number of models; see [1727, End of Section 3] for a list of references. The most relevant computations for Bayesian deep learning so far are due to Aoyagi for fully connected deep linear networks [39] and for one-hidden-layer ReLU networks [40]. We briefly summarise the linear result below.

Learning coefficient of deep linear networks. Consider a fully connected deep linear network with L hidden layers, input dimension $d_0 = d_{in}$, output dimension $d_{L+1} = d_{out}$ and hidden widths d_i for $0 < i < L+1$. The weight space $\mathcal{W}$ consists of tuples of matrices $A_\bullet = (A_0, \dots, A_L)$ with $A_i \in \mathbb{R}^{d_{i+1} \times d_i}$ for $i = 0, \dots, L$. Note that the total parameter count of the model is $D = \sum_i d_i d_{i+1}$. The function computed by the network is simply the product matrix $A_L \dots A_0$. We turn this into a probabilistic model as

$$p(y, x | A_\bullet) \propto \exp\left(-\frac{1}{2} \left\| y - A_L \dots A_0 x \right\|^2\right) p(x)$$

with the input distribution $p(x)$ also isotropic Gaussian. We also assume the true distribution is realisable: $p_0(y, x) = p(y, x | A_\bullet^*)$ for some $A_\bullet^*$. Then one computes that

$$K(A_\bullet) \propto \int \| (A_L \dots A_0 - A_L^* \dots A_0^*)\, x \|_2^2 \; p(x)\, dx.$$

Let $r := \mathrm{rank}(A_L^* \dots A_0^*) \leq \min d_i$. The model and the function K have a large group of continuous linear symmetries (which becomes larger as r decreases), which can be used to study the geometry of W_0. Aoyagi gives a formula for $\lambda = \lambda(K)$ in terms of r and the widths d_i [39, Theorem 1]. The formula is combinatorially involved and we state two representative special cases. First for a linear network of constant width ($d_i = d$) and sufficiently deep ($L > d - r$):

$$\lambda = \frac{1}{4}(d^2 + d + r^2 - r).$$

Then, for a very wide network ($d_i > d_{in} + d_{out} - r$ for all $0 < i < L + 1$):

$$\lambda = \frac{1}{2} d_{in} d_{out}.$$

To get a feel for the constant-width formula, note that for a full-rank target ($r = d$) it gives $\lambda = d^2/2$, already much smaller than $D/2 = (L+1)d^2/2$. The gap widens for rank-deficient targets: with $d = 10$ and $r = 5$, the condition $L > d - r$ permits $L = 6$, and the formula gives $\lambda = \frac{1}{4}(100 + 10 + 25 - 5) = 32.5$, compared with $D/2 = 350$. Note that in both cases, λ is much smaller than $D/2$ and does not depend on the depth L; this is

true in general as long as L is sufficiently large. In the first case, we also observe that λ is an increasing function of r; more generally, λ is always non-decreasing in r. This shows that deep linear networks are very singular, and become more so as the depth increases or the rank of the target function decreases.

Estimating λ. Given the difficulty of exact computation of λ, we can ask about how to *estimate* it from data. SLT provides tools to this effect, the *WBIC* in particular, see Section 52.3. The learning coefficient estimator based on the WBIC is however quite unstable and in our own research we never had much success in getting good estimates. This is perhaps not too surprising, since sampling from the full posterior distribution is challenging. In Section 52.4.1 we introduce a quantity related to the learning coefficient that we call the **local learning coefficient (LLC)**, as well as the *LLC estimator*. The LLC is both easier to estimate numerically and more sensitive to the local structure of the model, and for those reasons has seen significant uptake since it was first introduced in Lau et al. [978].

52.2.3 Resolution of singularities and mixture of standard forms

From the Bayesian perspective, the main property of λ is that it controls the asymptotics of *exponential integrals* of the KL-divergence K, generalising the standard Laplace approximation. More precisely[5], we have

$$\int_{\mathcal{W}} e^{-nK(w)}\varphi(w)dw \underset{n\to\infty}{\propto} n^{-\lambda}\log(n)^{m-1}. \tag{52.4}$$

Note the (unnormalised) posterior distribution is obtained by replacing K by the K_n in the LHS of (52.4), and that λ does not depend on the (positive) prior φ. One of Watanabe's main technical achievements is to combine the algebro-geometric techniques underlying this deterministic asymptotic result with probabilistic techniques from *empirical process theory* to derive a precise asymptotic description of the posterior itself. We briefly review these techniques below.

Standard form. By applying a change of variables known in algebraic geometry as a *resolution of singularities* [6], one can introduce a finite collection of analytic charts $g_j : U_j \to \mathcal{W}, j = 1, \ldots, J$, such that the parameter regions near $W_0 = \{w : K(w) = 0\}$ are covered by their images, and local coordinate systems $u = (u_1, \ldots, u_d)$ on each U_j such that the Kullback–Leibler divergence takes the *standard (or monomial) form*

$$K(g_j(u)) = u^{2k^{(j)}} = u_1^{2k_1^{(j)}} \cdots u_d^{2k_d^{(j)}}, \tag{52.5}$$

where $k^{(j)} = (k_1^{(j)}, \ldots, k_d^{(j)})$ is a multi-index of nonnegative integers [1723, Thm. 6.5].

Geometric interpretation. The collection of charts $\{(U_j, g_j)\}$ thus provides a local description which "untangles" the singularities of the model near W_0. The coordinates $u = (u_1, \ldots, u_d)$ separate degenerate directions in the original space $\mathcal{W}$ (creating ridges, folds, or intersections within W_0) from the active ones that determine the local asymptotic behaviour of the posterior: coordinates with $k_i^{(j)} > 1$ (resp. $k_i^{(j)} = 0$) represent flatter (resp. completely flat) directions.

Posterior vs generalised Laplace integral. While $K_n \neq K$, we have $\mathbb{E}[K_n] = K$, and Watanabe uses empirical process theory[7] to write

$$K_n(g_j(u)) = K(g_j(u)) - \sqrt{\frac{K(g_j(u))}{n}}\xi_n(u)$$

where $\xi_n(u)$ converges in law to an explicit Gaussian process $\xi(u)$. This more or less reduces the study of the posterior to two contributions: the deterministic generalised Laplace integral in the LHS of Equation (52.4) and the process[8] ξ.

Mixture of standard forms. Substituting the standard form representation in each local chart gives

$$\int_{\mathcal{W}} e^{-nK(w)}\varphi(w)\,dw \asymp \sum_{j=1}^{J}\int_{U_j} e^{-nK(g_j(u))}\,\varphi(g_j(u))\,|g_j'(u)|\,du,$$

[5] Still assuming for simplicity that the model is well-specified ($W_0 \neq \emptyset$), that $\varphi > 0$, and under some minor additional convergence assumptions which we will not spell out here.

[6] We refer to [1723, §3.5] for a down-to-earth introduction to resolution of singularities and Hironaka's theorem. We note in passing that a function K admits many resolutions and that there is no "canonical" choice.

[7] It is precisely for this step that the well-specified assumption comes into play, to bound the *variance* of K_n around K in order to properly apply the CLT. For non well-specified models, everything goes through as long as this variance can be controlled. See [1726, Section 3.1, Definition 7] for a sufficient condition which allows the theory to go through with only minor changes.

[8] The effect of ξ is secondary and we mostly neglect it in this chapter.

so that each term corresponds to the contribution from one local chart [1726, Ch. 6.3]. The analytic map g_j can also be chosen to factorise the prior and Jacobian determinant as

$$\varphi(g_j(u))|g_j'(u)| = b_j(u)\,|u^{h^{(j)}}| = b_j(u)\left|u_1^{h_1^{(j)}}\cdots u_d^{h_d^{(j)}}\right|,$$

where $h^{(j)} = (h_1^{(j)}, \ldots, h_d^{(j)})$ is a multi-index of nonnegative integers and $b_j(u)$ is analytic and strictly positive near the origin [1723, Thm. 6.5]. The integers $h_i^{(j)}$ describe the order of vanishing of the Jacobian along each coordinate axis, so that the change of variables locally scales volume by a monomial factor $|u^{h^{(j)}}|$. By a simple computation for monomials, we now obtain [1723, Thm. 6.7]

$$\int_{U_j} e^{-nK(g_j(u))}\,\varphi(g_j(u))\,|g_j'(u)|\,du = C_j\,n^{-\lambda_j}(\log n)^{m_j-1}\,(1+o(1)),$$

with

$$\lambda_j = \min_{1\le i\le d}\frac{h_i^{(j)}+1}{2k_i^{(j)}}, \qquad m_j = \#\left\{\, i : \frac{h_i^{(j)}+1}{2k_i^{(j)}} = \lambda_j \,\right\},$$

so λ_j captures the dominant rate of decay in a given chart and m_j the number of coordinates that achieve it [1726, Ch. 5, Def. 17].

Consequently, for large n, and after passing to the resolution, the posterior takes the form of a finite mixture of "standard-form" components, each characterised by its own local geometry (λ_j, m_j). Combining with Equation (52.4), we obtain that the **global** learning coefficient λ and multiplicity m introduced in Section 52.2.2 is given by

$$\lambda = \min_{1\le j\le J}\lambda_j, \qquad m = \max_{1\le j\le J}\#\{\, i : \lambda = \frac{h_i^{(j)}+1}{2k_i^{(j)}}\,\}. \tag{52.6}$$

Note that in regular models, $K(w) \approx w_1^2 + \ldots + w_d^2$ is not in standard form when $d > 1$. The resolution of singularities in this case is very explicit (the "blow-up" of $\mathcal{W}$ at the unique point in W_0) and described (in dimension 2, but the general case is similar) in [1723, Example 3.15].

52.3 Misconception 1: Laplace approximation is valid for singular models

We now come to addressing the first misconception, that the Laplace approximation is appropriate for singular models. The discussion from the previous sections has already foreshadowed this, but we will now dig into the details. The Laplace approximation is used in two distinct but related ways in Bayesian deep learning: to approximate the posterior distribution for downstream prediction and to estimate the model evidence for model selection. We will address the invalidity of the Laplace approximation in each use case and point to the SLT corrective, where possible.

52.3.1 Laplace for prediction

As discussed in Section 52.2.3, the Laplace approximation cannot in general describe the posterior in singular models, including neural networks. After a resolution of singularities, the posterior decomposes into a mixture of finitely many local components, each governed by its own pair (λ_j, m_j). This asymptotic structure is fundamentally incompatible with the single Gaussian form assumed by the Laplace method—or even with a finite mixture of Gaussians.

In regular models, by contrast, the Laplace approximation is mathematically justified: for large n, the posterior is approximately Gaussian,

$$\pi(w|\mathcal{D}_n) \approx \mathcal{N}(w_{\mathrm{MAP}}, H(w_{\mathrm{MAP}})^{-1}), \tag{52.7}$$

where w_{MAP} is the maximum a posteriori (MAP) estimate and H is the **Hessian of the negative log posterior** evaluated at w_{MAP}.

This Gaussian approximation is widely used in Bayesian deep learning to approximate the PPD, as it allows efficient sampling. The typical procedure is:

1. Train a neural network using stochastic optimisation and take the resulting weights $\hat{w}$ as w_{MAP};

2. Sample weights from $\mathcal{N}(\hat{w}, H(\hat{w})^{-1})$ to perform Monte Carlo integration for the PPD.

However, in deep networks, $H(\hat{w})$, the Hessian at a trained solution, $\hat{w}$, is typically singular or nearly so, making the Gaussian ill-defined. Practical implementations therefore regularise H—for instance by adding damping or diagonal corrections—yielding numerically stable but ad hoc covariance structures. Moreover even after such corrections computing H is very expensive and is usually replaced by more tractable approximations such as Generalised Gauss–Newton (GGN) or Kronecker-Factored Approximate Curvature (K-FAC); see Chapter 40 for a detailed treatment of scalable Laplace approximations. These modifications produce Laplace-inspired approximations: they retain the computational form of the Laplace method but no longer correspond to the true local curvature of the posterior.

Despite these theoretical shortcomings, Laplace-based methods can perform well empirically, though the evidence is specific to the task. Because they perturb around a trained MAP estimate, they essentially inherit its in-distribution accuracy, and they typically improve calibration on in-distribution data [380]. This success also depends on further engineering choices: sampling naively from a full-network Laplace posterior underfits severely, and practical implementations rely on the linearised predictive [813] (Chapter 8), last-layer application, and prior precision tuned via the marginal likelihood. The quality of uncertainty quantification away from the training distribution is less settled: diagonal approximations fail to represent uncertainty between clusters of training data [542] and Gaussian posterior approximations degrade under dataset shift [1274], although even a last-layer Gaussian provably mitigates the overconfidence of ReLU networks far from the data [928]. None of this implies that the Laplace approximation is valid for neural networks; it reflects the interplay between optimisation dynamics, curvature approximations, and regularisation choices that may yield effective but heuristic posterior samplers. In other words, although SLT clarifies why the Laplace approximation does not hold in singular models, it does not quite tell us why (and when) its variants prove useful in practice. In addition SLT does not directly offer a computable alternative for approximating the PPD. Nonetheless it provides a geometric framework that identifies what the Laplace approximation overlooks—and therefore, what a successful approximation needs to capture. One example, developed in Section 52.5, concerns prediction: a single Gaussian centred at one point of W_0 misses the posterior mass spread across the rest of W_0, and averaging over W_0 is precisely the source of the asymptotic predictive advantage of the exact Bayes PPD. The good in-distribution performance of Laplace variants is therefore performance at the level of point estimates, and is fully consistent with the marginalisation gains discussed there.

52.3.2 Laplace for model selection

The second key use of the Laplace approximation in Bayesian deep learning is to estimate the marginal likelihood (or model evidence), $Z_n = \int_{\mathcal{W}} e^{-nL_n(w)}\varphi(w)\,dw$, or equivalently the log marginal likelihood (LML), $\log Z_n$. The LML can be used to select hyperparameters, such as the prior precision (weight decay), a process known as Type-II maximum likelihood or empirical Bayes. The Laplace approximation provides a tractable expression for the LML in regular models:

$$\log Z_n \approx -nL_n(w_{\mathrm{MAP}}) - \frac{d}{2}\log(n) + \log\varphi(w_{\mathrm{MAP}}) + \frac{d}{2}\log(2\pi) - \frac{1}{2}\log\det\left(\bar{H}(w_{\mathrm{MAP}})\right) \tag{52.8}$$

where $\bar{H}(w_{\mathrm{MAP}}) := H(w_{\mathrm{MAP}})/n$ is the Hessian of the negative log-posterior from Equation (52.7) rescaled by the sample size [1105]; for large n, $\bar{H}$ is approximately the Hessian of the empirical negative log-likelihood L_n.

Asymptotic expansion for singular models. SLT shows that the approximation (52.8) of the LML is incorrect for singular models. Indeed, a direct application of Watanabe's asymptotic expansion of the (unnormalised) posterior discussed in Section 52.2.3 leads to

$$\log Z_n = -nL_n(w_0) - \lambda\log n + (m-1)\log\log n - F_n^R, \tag{52.9}$$

where w_0 is any parameter in W_0 and F_n^R is a random variable which converges in distribution to a constant ([1723, Corollary 6.1]; compare this with the closely related Equation (52.4)). Because w_0 is not known, we can consider plugging in the MAP estimator[9], leading to

$$\log Z_n \approx -nL_n(w_{\mathrm{MAP}}) - \lambda\log n + (m-1)\log\log n + O_p(1). \tag{52.10}$$

This expression generalises the Laplace approximation (52.8), since we have $\lambda = d/2$ and $m = 1$ for regular models. Note that, like in the regular case, the constant order term in Equation (52.10) is the first term in this asymptotic expansion which depends on the prior φ; however, for singular models, that constant order term is

[9] This is valid when for instance the likelihood ratio between w_0 and w_{MAP} is bounded in probability.

more complicated and does not (only) depend on the prior and the Hessian but on the precise geometry of the model (through the Gaussian process ξ mentioned in Section 52.2.3).

Estimating the LML in practice: WBIC. The equations (52.9) and (52.10) are useful to understand the theoretical role of λ in model selection. However to make use of them to approximate the log marginal likelihood, we a priori need to know λ (which itself depends on $p_0(x)$). Using the same proof techniques, Watanabe derived an estimator of the LML which does not require λ, the **Widely Applicable Bayesian Information Criterion (WBIC)** [1725]:

$$\mathrm{WBIC}_n = \mathbb{E}_w^\beta[nL_n(w)],$$

for $\beta = 1/(\log n)$ where $\mathbb{E}_w^\beta$ is the expectation with respect to the *tempered* Bayesian posterior with *inverse temperature* β:

$$p^\beta(w|\mathcal{D}_n) \propto \exp\{-n\beta L_n(w)\}\varphi(w).$$

Watanabe showed $\mathrm{WBIC}_n = -\log Z_n + O(\sqrt{\log n})$ so that

$$\mathrm{WBIC}_n = nL_n(w_0) + \lambda \log n + O(\sqrt{\log n}). \tag{52.11}$$

Estimating the learning coefficient using the WBIC. We end this section by briefly noting that the WBIC estimator can be turned around into an estimator of the learning coefficient itself. Let $\beta_1 = \beta_{01}/\log n$ and $\beta_2 = \beta_{02}/\log n$ where β_{01} and β_{02} are positive constants. Then we have that the following is a consistent estimator of the learning coefficient

$$\hat{\lambda} = \frac{\mathbb{E}_w^{\beta_1}[nL_n(w)] - \mathbb{E}_w^{\beta_2}[nL_n(w)]}{1/\beta_1 - 1/\beta_2}.$$

52.4 Misconception 2: Effective dimensionality is captured by the eigenspectrum of the Hessian

Classical learning theory suggests that models with far more parameters than data points should overfit catastrophically, yet in deep learning practice, larger models often generalise even better. This paradox, which has been discussed at length in Bayesian deep learning (including MacKay [1104], Maddox et al. [1117], Wilson [1768]) reveals a fundamental flaw in relying on simple parameter counting as a measure of model complexity.

To resolve this paradox, one needs alternative complexity measures that are not based on the raw parameter count, but on the model's interaction with the data. One approach is to consider the eigenspectrum of the Hessian of the training loss at a trained model, $H(\hat{w}) := \nabla^2 L_n(\hat{w})$. Empirically, H typically admits a few large positive eigenvalues, a long tail of small positive eigenvalues, some exactly vanishing eigenvalues induced by architectural symmetries, and some small negative eigenvalues. From the perspective of the Laplace approximation of the posterior, one can hope that the number of "large" positive eigenvalues (the *"effective rank"* of H) quantifies posterior contraction—essentially, how many parameter directions are meaningfully constrained by the training data and truly "used" by the model. Consequently, when comparing models that have achieved a similarly low training loss, one could expect that the one with a lower effective Hessian rank generalises better.

There are various ways to operationalise the effective rank of H. A specific implementation is the **effective dimensionality**, denoted N_{eff}. It originated with MacKay [1104], was later modernised by Maddox et al. [1117] (who positioned it as a generalisation metric for deep learning) and further developed by Wilson [1768] (who used it to argue that large models succeed by learning a compressed representation). The effective dimensionality is defined as

$$N_{\text{eff}}(H, z) := \sum_{i=1}^{d} \frac{\mu_i}{\mu_i + z}, \tag{52.12}$$

where μ_i are the eigenvalues of H and $z > 0$ is a regularisation constant. In this formula, eigenvalues $\mu_i \gg z$ contribute approximately 1 to the sum, while small eigenvalues contribute approximately 0, so N_{eff} can indeed be interpreted as an effective rank of H. Although Hessians of neural networks commonly have negative eigenvalues, Maddox et al. [1117] suggest they are largely irrelevant for the calculation, as their magnitudes are typically small compared to the large positive eigenvalues. In line with the discussion above, Maddox et al. [1117] demonstrate that N_{eff} is indeed a robust metric for generalisation, comparing favourably to other norm- and flatness-based measures.

While the spirit of these works is correct in seeking an alternative to parameter counting, the mathematical basis for using the Hessian only makes sense for regular models and therefore not for neural networks. Recent

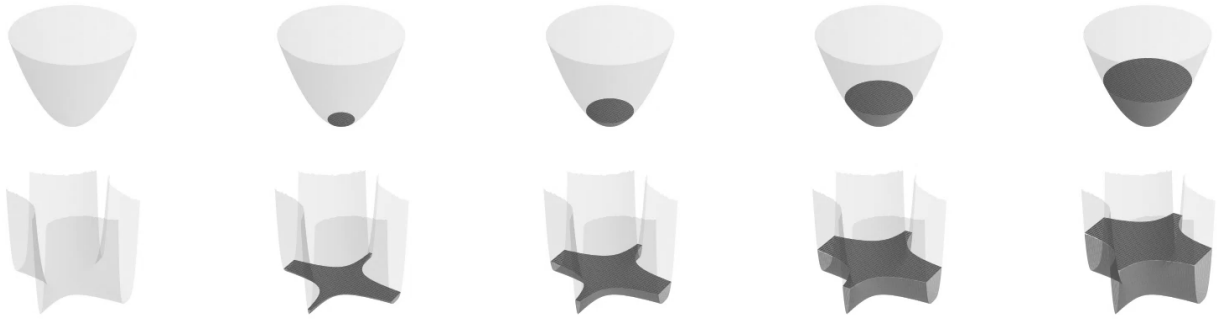


Figure 52.1: The LLC $\lambda(w^*)$ measures volume scaling around a local minimum of the loss. Top: for the regular quadratic loss $K(w) = w_1^2 + w_2^2$, the area (or "2D-volume") of the near-optimal set $\{K(w) < \varepsilon\}$ around $w^* = (0, 0)$ scales as ε, so $\lambda(w^*) = 1$ (equal to $d/2$ with $d = 2$). Bottom: for the singular loss $K(w) = w_1^2 w_2^4$, which vanishes on the entire union of the two coordinate axes, the near-optimal set is a cross-shaped neighbourhood of the axes, and its area scales as $\varepsilon^{1/4}$, so $\lambda(w^*) = 1/4$. A smaller LLC means more volume near the minimum: as $\varepsilon \to 0$, far more parameter values achieve loss below ε than in the regular case. (Adapted from Figure 2 in Lau et al. [978]; images reproduced with permission from Hoogland and van Wingerden [782].)

advances in SLT provide a more rigorous definition of effective dimensionality, given by the local learning coefficient [978], which we will introduce below.

From global to local. The learning coefficient defined above is a *global* property of the *bottom of the loss landscape*: it describes the overall scaling of volume near all global minima of $K(w)$ combined, and the smallest λ among them dominates the asymptotics. However, in complex models such as neural networks, the geometry of $K(w)$ may vary widely across different minima, symmetry classes, or training trajectories. Some regions of parameter space may be flatter or more degenerate than others. Moreover, we cannot usually assume that training reaches a global minimum. To capture this variation, Lau et al. [978] define and develop a *local* analogue of the learning coefficient.

52.4.1 Local learning coefficient

The *local learning coefficient (LLC)* (or local RLCT) $\lambda(w^*)$ is defined in a completely analogous way to the global learning coefficient except we focus on a local minimum w^* of $L(w)$ (or equivalently of $K(w)$). Let $B(w^*)$ be a small closed ball centred on w^* such that for all $w \in B(w^*)$ we have $L(w) \geq L(w^*)$. Instead of the set of almost optimal parameters, we consider the set $\{w \in B(w^*) : L(w) - L(w^*) < \varepsilon\}$. Then the volume definition of the global learning coefficient can be directly extended, as the constant $\lambda(w^*)$ controlling the asymptotics of the prior-weighted volume:

$$\int_{w \in B(w^*): L(w) - L(w^*) < \varepsilon} \varphi(w) dw \sim C\varepsilon^{\lambda(w^*)} (-\log(\varepsilon))^{m(w^*)-1} \quad \text{as } \varepsilon \to 0. \tag{52.13}$$

Figure 52.1 illustrates this volume scaling for a regular (quadratic) loss and for a singular one.

LLC estimator. One can adapt the WBIC estimator of the LML discussed above to estimate the LLC. For details, the reader should consult Lau et al. [978]. First let us define the local-tempered posterior

$$p(w|w^*, \beta, \gamma) \propto \exp\left\{-n\beta L_n(w) - \frac{\gamma}{2}\|w - w^*\|_2^2\right\}, \tag{52.14}$$

where $\beta > 0$ is an inverse temperature and $\gamma > 0$ indicates how close we want to stay to w^*. Let us denote the expectation of a function $f(w)$ with respect to the posterior distribution in (52.14) as

$$\mathbb{E}_{w|w^*,\beta,\gamma}[f(w)] \coloneqq \int f(w) p(w|w^*, \beta, \gamma)\, dw.$$

The **local learning coefficient** estimator is given by

$$\hat{\lambda}(w^*) \coloneqq n\beta^* \left[\mathbb{E}_{w|w^*,\beta^*,\gamma}[L_n(w)] - L_n(w^*)\right]. \tag{52.15}$$

where $\beta^* = 1/(\log n)$. The fact that this estimates $\lambda(w^*)$ follows from Equation (52.11).

Comparing N_{eff} and the LLC. A minimal example separates the two quantities. Consider the loss $K(w) = w_1^2 w_2^4$ from Figure 52.1. At the minimum $w^* = (0, 0)$ every second derivative of K vanishes, so the Hessian of K at w^* is the zero matrix and N_{eff} evaluated on it equals 0 for every $z > 0$. The eigenspectrum reports that no direction is constrained, and it reports the same value for $w_1^4 w_2^6$ or for any other loss whose quadratic part vanishes at the minimum, so it cannot tell these geometries apart. The LLC does distinguish them: the monomial formula of Section 52.2.2 gives $\lambda(w^*) = 1/4$ for the first loss and $\lambda(w^*) = 1/6$ for the second. The reason is that the Hessian captures only the quadratic part of the loss, which is precisely the part that vanishes at a singularity, while the LLC measures the volume scaling of the full function. One caveat in interpreting this comparison: in practice N_{eff} is computed from the Hessian of the empirical loss L_n at a trained network $\hat{w}$, which will generally have small nonzero eigenvalues rather than vanish exactly. The comparison above concerns the population loss exactly at the singularity, the same idealisation used throughout this chapter. The point survives the idealisation: near a singular minimum, the eigenvalues of the Hessian convey how nearly degenerate the quadratic form is, while the LLC quantifies the degeneracy itself.

Applications of the LLC. The LLC estimator has been developed into a practical diagnostic for studying the internal geometry and developmental dynamics of deep learning models. Hoogland et al. [783] apply the LLC to transformer training and show that shifts in local complexity coincide with discrete phases of representational development, linking changes in loss-landscape degeneracy to the formation of new computational subsystems. Building on this framework, Wang et al. [1705] introduce the *refined* LLC to analyse attention heads individually, demonstrating that differentiation and specialisation among heads emerge through measurable reductions in local complexity. Together, these studies use the LLC to reveal how structure and function in deep networks unfold through geometric transitions in the learning process, bridging singular learning theory with observable deep training phenomena.

Takeaways:

Together, the (global) learning coefficient λ and its local refinement $\lambda(w^*)$ describe how model complexity and posterior concentration emerge from the geometry of the parameter–likelihood map. They give SLT its distinctive power: complexity and generalisation are determined not by parameter count or curvature, but by how the *volume* of near-equivalent models collapses under the data.

52.5 Misconception 3: The benefits of marginalisation are not about predictive performance

The motivation for using Bayesian marginalisation in deep learning has traditionally been twofold: for philosophical coherence with Bayesian principles, and more practically, for its ability to produce reliable **uncertainty quantification**. The resulting posterior predictive distribution naturally encodes uncertainty, distinguishing it from point-estimate methods such as MAP or MLE. However, the conversation around raw **predictive performance** (i.e., generalisation error) is more muddled. It is often implicitly assumed that any accuracy gain from the computationally expensive process of marginalisation is modest. In simple, "regular" models, this is largely true, leading to the belief that one pursues marginalisation for the uncertainty estimates, not for the predictions themselves.

This section aims to show that this picture is incomplete. We will demonstrate that for the singular models that dominate modern machine learning, marginalisation may provide significant advantage in predictive performance itself. The basis for this advantage can be understood using SLT, which offers a rigorous foundation for this powerful, yet underappreciated, aspect of marginalisation, or Bayesian averaging. The results in this section concern the exact Bayes PPD of Equation (52.1), in which the intractable posterior is integrated exactly; whether practical approximations to the posterior inherit the advantage is a separate question, to which we return at the end of this section and in Misconception 4 (Section 52.6).

To formalise this, we must first define the three predictive distributions in question, all derived from a training dataset $\mathcal{D}_n = \{x_i\}_{i=1}^n$:

- **Bayesian PPD:** In Bayesian learning, we are uncertain about which set of model parameters is the correct one. Instead of committing to a single "best" set, marginalisation is the process of **averaging the predictions of all possible models**. Each model's prediction is weighted by its posterior probability—how

plausible that set of parameters is, given the training data. This is captured by the Bayesian PPD in (52.1). Here, we are "integrating out" or "marginalising out" the parameters w.

- **MAP Predictive:** This is a point estimate based on the maximum of the posterior.

$$\hat{p}_{\mathrm{MAP}}(x) := p\big(x \mid \hat{w}_{\mathrm{MAP}}\big), \qquad \hat{w}_{\mathrm{MAP}} := \arg\max_{w \in \mathcal{W}} \pi(w \mid \mathcal{D}_n).$$

- **MLE Predictive:** This is a point estimate based on the maximum of the likelihood.

$$\hat{p}_{\mathrm{MLE}}(x) := p\big(x \mid \hat{w}_{\mathrm{MLE}}\big), \qquad \hat{w}_{\mathrm{MLE}} := \arg\max_{w \in \mathcal{W}} \prod_{i=1}^{n} p(x_i \mid w).$$

We shall measure the performance of any such predictive distribution $\hat{p}(x)$ using the following notion of generalisation error:

$$G(\hat{p}) := \mathrm{KL}\big(p_0(x) \,\|\, \hat{p}(x)\big).$$

That is, G is the KL divergence from the true data-generating distribution p_0 to $\hat{p}$. This is what is known, up to an additive constant, as "negative log likelihood" or "test log likelihood" in the Bayesian deep learning literature. We also note that this is not the typical generalisation error used in ML benchmarking tasks such as percentage correct on the test set.

We denote the generalisation errors for our three methods as $G_{\mathrm{Bayes}}(n) := G(\hat{p}_{\mathrm{Bayes}})$, and similarly $G_{\mathrm{MAP}}(n)$ and $G_{\mathrm{MLE}}(n)$, making the dependence on the sample size explicit. SLT provides the asymptotic forms for these errors. For the Bayes PPD, we have [1726, Thm. 15]

$$\mathbb{E}_{\mathcal{D}_n} G_{\mathrm{Bayes}}(n) = \frac{\lambda}{n} + o\left(\frac{1}{n}\right) \tag{52.16}$$

where λ is the global learning coefficient and $\mathbb{E}_{\mathcal{D}_n}$ indicates expectation over the training data drawn from the true p_0. The fact that the same learning coefficient controls both the asymptotics of the LML $\log Z_n$, as in Equation (52.10), and of $G_{\mathrm{Bayes}}(n)$, i.e. both model complexity and generalisation error, is an important feature of SLT[10]. While not our main focus, we briefly note that the generalisation error can be estimated using only training data with the Widely Applicable Information Criterion (WAIC) [1724]. PAC-Bayesian analysis provides a complementary frequentist route to generalisation guarantees for Bayesian and randomised predictors, with bounds that hold at finite sample size; see Chapter 22.

For point estimates, the situation is different. We have [1723, Thm. 6.4]

$$\mathbb{E}_{\mathcal{D}_n} G_{\mathrm{MAP}}(n) = \frac{C_{\mathrm{MAP}}}{2n} + o\left(\frac{1}{n}\right)$$
$$\mathbb{E}_{\mathcal{D}_n} G_{\mathrm{MLE}}(n) = \frac{C_{\mathrm{MLE}}}{2n} + o\left(\frac{1}{n}\right)$$

where the constants $C_{\mathrm{MAP}}, C_{\mathrm{MLE}} > 0$ are distinct but both related to expectations of the maximum value of the square of the Gaussian process $\xi(w)$ on W_0. If the model is regular, then $C_{\mathrm{MAP}} = C_{\mathrm{MLE}} = d/2$, recovering the classical asymptotics. In singular models, the maximum of ξ^2 is taken over the positive-dimensional set W_0, and maxima of Gaussian processes grow with the size of the index set; this mechanism pushes both constants above $d/2$, and hence above λ. We caution that this comparison is an observation discussed by Watanabe [1723] rather than a general theorem: exact values of C_{MAP} and C_{MLE} are known only in special cases.

This means two things. First, we can expect the Bayes PPD to have generalisation error different from that of MAP and MLE. Second, we can expect in general for the Bayes PPD to achieve a smaller leading constant in its $1/n$ generalisation error. This advantage arises because Bayesian averaging explores the entire set of optimal parameters (W_0), smoothing out any overfitting to one specific solution, while MAP/MLE commits to a single point estimate found on the noisy training data.

Example: finite mixture of Gaussians. Consider a mixture of r isotropic unit variance Gaussian components in $\mathbb{R}^p$, fit to data from a mixture with $r_0 < r$ components. This model is singular because there are many ways to configure the redundant components to produce the same optimal solution (e.g., by swapping component labels

[10]This relation is itself less mathematically deep than either of those asymptotics.

or setting weights to zero). In this case, the learning coefficient λ is strictly smaller than $\frac{d}{2} = \frac{1}{2}(pr + r - 1)$. In fact, one can show in this case [1723, Section 7.3, equation (7.3)] that

$$\lambda \leq \frac{1}{2}(pr_0 + r - 1). \tag{52.17}$$

As a result, $G_{\text{Bayes}}(n) \approx \lambda/n$ is smaller than the MAP/MLE generalisation error, which does not fully exploit the symmetry in W_0.

The same principle applies directly to deep learning. The symmetries and redundancies in neural networks (e.g., permutation of neurons) create vast, flat sets W_0 of equivalent parameters. Bayesian prediction effectively averages over these equivalent solutions, reducing generalisation error in proportion to the model's true complexity, λ. Point estimates cannot exploit this geometry and therefore suffer a higher generalisation error.

Despite these theoretical advantages, performing Bayesian marginalisation faithfully in modern deep networks remains an open challenge. Several studies have shown that popular approximations often fall short of true posterior averaging. For instance, variational methods fail to capture posterior correlations [542], and empirical analyses reveal that approximate posteriors such as SWA-Gaussian or Laplace differ substantially from true ones [826]. Other works have critiqued Bayesian interpretations of dropout (Chapter 14) [536] and deep ensembles [1756] and shown that many practical improvements stem from implicit ensembling rather than accurate marginalisation [1116]. Misconception 4 (Section 52.6) makes this gap precise in the setting of variational inference: the quality of a posterior approximation as measured by the ELBO and the quality of its predictive distribution are governed by different asymptotic constants.

52.6 Misconception 4: Good ELBO means good predictive performance

Variational inference (VI; see Chapter 12 for an introduction) remains one of the most widely used and theoretically appealing approaches to approximate Bayesian deep learning. It offers a principled optimisation objective — the evidence lower bound (ELBO) — and, unlike more ad hoc methods such as deep ensembles (Chapter 19), VI is directly connected to Bayesian principles via a Kullback–Leibler projection onto a tractable family of approximate posteriors. This combination of conceptual clarity and scalability has made VI a popular alternative to MCMC.

In Bayesian deep learning, VI is often used to approximate the PPD. It might be natural to expect that improving the ELBO will also improve the resulting predictive distribution: since the ELBO lower-bounds the log marginal likelihood, tightening the bound should bring the approximation closer to the true posterior, and hence to the true Bayesian predictive distribution. In *regular* models, where the posterior is asymptotically Gaussian, this expectation can often be justified. However, in *singular* models — which include all modern deep neural networks — the relationship between ELBO and predictive performance can break down. One variational approximation may achieve a *smaller* ELBO gap than another, yet still produce *worse* predictions. This exposition is based mainly on Wei and Lau [1732] and we largely follow the notation and terminology therein.

Setup. We work in the supervised setting, unlike in previous sections of this chapter. To approximate the intractable posterior distribution, VI requires two ingredients: a variational family and a divergence. Let $\mathcal{Q}$ denote a variational family. We present standard VI, which uses the KL divergence, leading to the optimisation problem

$$q^\star \in \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big(q(w) \,\|\, \pi(w \mid \mathcal{D}_n)\big).$$

Note this is equivalent to *minimising* the (normalised) variational free energy (VFE),

$$\bar{F}_{\text{vb}}(n) \;=\; \mathbb{E}_q\big[nK_n(w)\big] + \mathrm{KL}\big(q(w) \,\|\, \varphi(w)\big),$$

where φ is the prior and $K_n(w)$ is the empirical KL divergence. The **VFE is equal to the ELBO**, up to an additive constant independent of q and a sign. So while we maximise the ELBO, we seek to minimise the VFE. Let us denote the optimum value of the VFE

$$\bar{F}^\star_{\text{vb}}(n) = \min_{q \in \mathcal{Q}} \bar{F}_{\text{vb}}(n).$$

Once we have obtained our variational approximation $q^\star$ to the posterior distribution, we can use it for prediction. Let us call this the variational predictive distribution, which is given by

$$\hat{p}_{\text{vb}}(y \mid x) \;:=\; \int p(y \mid x, w)\, q^\star(w)\, dw.$$

Let us denote the variational generalisation error (VGE),

$$G_{\mathrm{vb}}(n) \;:= \mathbb{E}_x \,\mathrm{KL}\big(p_0(y \mid x) \,\|\, \hat{p}_{\mathrm{vb}}(y \mid x)\big),$$

where p_0 is the true conditional distribution of y given x and the expectation is over the input distribution.

Two different asymptotic coefficients. In singular models, the minimum VFE has the following asymptotic expansion

$$\mathbb{E}_{\mathcal{D}_n} \bar{F}^{\star}_{\mathrm{vb}}(n) \;=\; \lambda_{\mathrm{vfe}} \log n \;+\; o(\log n),$$

while the VGE admits

$$\mathbb{E}_{\mathcal{D}_n} G_{\mathrm{vb}}(n) \;=\; \frac{\lambda_{\mathrm{vge}}}{n} \;+\; o\Big(\frac{1}{n}\Big).$$

The constants λ_{vfe} and λ_{vge} describe *different* aspects of the approximation:

- λ_{vfe} governs the $\log n$ term in the minimum VFE and hence the optimal ELBO.
- λ_{vge} governs the $1/n$ term in the generalisation error of the downstream predictive distribution that uses the variational approximation to the posterior.

There is, unfortunately, no reason for λ_{vfe} and λ_{vge} to coincide in a singular model, unlike for standard Bayesian inference. A variational family can have a *smaller* VFE gap (i.e. λ_{vfe} closer to the optimal learning coefficient λ) but a *larger* predictive error (larger λ_{vge}).

The clearest concrete setting is reduced rank regression, the linear network model of Section 52.2.2 with one hidden layer. For Gaussian mean-field variational families, Nakajima and Watanabe [1221] derived the asymptotics of both the minimum VFE and the VGE in closed form, and the two coefficients genuinely differ: depending on the input and output dimensions, the number of hidden units, and the rank of the true map, either $\lambda_{\mathrm{vfe}} > \lambda_{\mathrm{vge}}$, in which case minimising the VFE offers control over the VGE, or $\lambda_{\mathrm{vfe}} < \lambda_{\mathrm{vge}}$, in which case even a small variational approximation gap is compatible with an induced predictive distribution with high generalisation error. The same inversion arises when comparing two variational families: Wei and Lau [1732] illustrate schematically (their Figure 2(b)) that the family with the smaller VFE gap can induce the predictive distribution with the larger λ_{vge}, and some of their experiments with tanh networks exhibit exactly this scenario, with the ordering of two families by ELBO opposite to their ordering by predictive performance.

Takeaways:

In singular Bayesian deep learning, it is *natural* to expect that a better ELBO will lead to better predictions, but this is not guaranteed. ELBO and predictive accuracy are governed by different asymptotic constants, and can be ranked in opposite directions.

52.7 Summary

Singular learning theory (SLT) provides a coherent asymptotic framework for Bayesian deep learning, clarifying where classical intuitions break down. It explains why local quadratic (Laplace) approximations fail in neural networks, why model complexity cannot be read off from parameter counts or Hessian spectra, why Bayesian marginalisation can improve predictive accuracy rather than merely uncertainty estimates, and why optimising the ELBO does not necessarily yield a good predictive distribution. Across these settings, SLT replaces classical asymptotics based on curvature with a geometry-based description of how the posterior and marginal likelihood behave.

Chapter 53

Active Learning and Experimental Design

Today we have powerful methods for using data once we have it [450, 1212, 1213, 1632]. Yet the way we *gather* data in the first place remains strikingly low-tech, from the design of clinical trials [1075, 1413, 1431] to the construction of model evaluations [249, 1042, 1550]. This is a significant bottleneck. We are limited by the number of human experts able to effectively direct data-gathering, but also by the capacity of even the most capable experts to handle masses of information and complex tradeoffs. We are thus underequipped to gather an essential resource: good data is central to scientific progress [389, 555, 556, 696, 697], is a core requirement for successful machine learning [438, 701, 1037], and more broadly influences the quality of decision-making across society [505, 1161, 1670].

The challenge of addressing the data-gathering bottleneck—and the payoff of enhancing a whole spectrum of data-driven activities, multiplying the value they produce—motivates research on *experimental design*. This formalises the data-gathering problem: the design of an abstractly defined experiment represents aspects of data generation that we can control, such as which questions to ask a user or how to configure lab equipment; the choice of design influences the data produced by the experiment, such as user responses or lab measurements; and our aim is to choose the best design we can, where rigorously defining "best" is a central challenge. More generally we can think of designing a whole sequence of experiments, with our task then being to optimise a design policy that specifies how to design each experiment based on the data we have already gathered.

The study of experimental design has evolved considerably since early formal work by Gergonne [608] and Peirce [1310]. Spurred on by Fisher [525, 526] and Smith [1525], it grew significantly within the field of statistics in the twentieth century [165, 198, 290, 291, 395, 396, 510, 886, 1049, 1050, 1403, 1692].[1] Recent years have seen yet more growth as practical methods have improved [790, 1369, 1443]. Alongside work within statistics, there has been interest and innovation within machine learning, especially since the late 1980s, often under the banner of *active learning* [60, 321, 322, 1106, 1107] but also associated with topics including curiosity-driven learning [1470, 1481], exploration [1202, 1604, 1673] and query-based learning [34, 104, 1652].

This research has yielded concrete progress. In machine learning we have seen promising demonstrations of automated data-gathering for image classification [1237], model evaluation [1464], reward modelling [694], robot control [106] and text classification [1019]. Meanwhile in the sciences we have seen successful investigations in biology [866], chemistry [671], neuroscience [1497], physics [475] and psychology [1651].

Further work is nevertheless needed. While existing proof-of-concept studies are encouraging, there remains a substantial gap between them and widespread deployment of automated data-gathering. Real-world problems are more complex and diverse than the problems that have been studied so far. Solving them will require methods that can be flexibly tailored to different problems and that can be relied upon to perform well.

In this chapter we aim to lay the groundwork for future advances. We begin by covering some foundational ideas from statistical decision theory, particularly how the contrasting Bayesian and frequentist attitudes towards observed data lead to different notions of optimal decision-making. Following this we translate optimality criteria for general decisions into optimality criteria for experimental design, and we discuss some important challenges in bridging from theory to practice, including instantiating concrete realisations of abstract constructs and approximating the solutions to intractable optimisation problems. We then present active learning as a particular form of experimental design focused on accurate predictions (more specifically predictions of outputs for inputs that are often unknown at the time of data-gathering), present some of the most popular methods in the field, and discuss some key methodological considerations in active learning.

[1] See Garnett [584] for an excellent history.

53.1 Decision-theoretic foundations

Before directly addressing data-gathering, we first lay out the theoretical foundations of more general decision-making. The rationale for this is that many of the core principles of effective data-gathering require a deeper understanding of decision theory than just the well-known notions of Bayesian inference and empirical-risk minimisation.

At a high level, statistical decision theory concerns how to evaluate and optimise policies, $\pi : \mathcal{D} \to \mathcal{A}$, that map from data, $d \in \mathcal{D}$, to actions, $a \in \mathcal{A}$ [123, 166, 395, 514, 1050, 1364, 1405, 1463, 1694]. The *consequences* of these actions will depend on some unknown state of the world, $z \in \mathcal{Z}$, and our aim is generally to minimise the loss (or equivalently maximise the utility) of these consequences. Notably this formalism encompasses both concrete real-world actions, such as prescribing a treatment to a patient, and more abstract actions, such as estimating a parameter in a model.

Frameworks to solve such problems can generally be broken into two branches, *Bayesian* and *frequentist*, each of which we will cover. Both use notions of expected loss to evaluate policies, but they differ in their treatment of observed data, d, and the unknown world state, z. While inevitably linked to the respective Bayesian and frequentist definitions of probability (i.e. subjective beliefs under incomplete information and long-run relative frequency of an event), it is important to appreciate that these decision-theoretic formalisms extend far beyond this more philosophical distinction and have profound implications across modern statistics and machine learning.

53.1.1 Bayesian decision theory

The underlying question of Bayesian decision theory is *how should an agent choose actions in the presence of incomplete information?* It considers this problem from the point of view of the agent itself, who must formulate probabilistic beliefs about the world and then make decisions about how to best act in light of these.

Mathematically, a decision-making problem under the Bayesian framework is defined by two terms: a belief state about the world given the observed data expressed in the form of a probability distribution over states, $p(z|d)$, and a loss function (or equivalently negative utility function), $\ell : \mathcal{A} \times \mathcal{Z} \to \mathbb{R}$, that measures the cost of taking action a if the true state of the world is z. Given these, we can evaluate actions using their Bayesian expected loss, defined as follows.

Definition 53.1 (Bayesian expected loss)**.** *The Bayesian expected loss, $L_B(a, d)$, of taking action $a \in \mathcal{A}$ given observed data d, is the expectation over an agent's subjective belief distribution on world states, $z \sim p(z|d)$, of an integrable and measurable loss function, $\ell : \mathcal{A} \times \mathcal{Z} \to \mathbb{R}$, that represents its preferences over action–state pairs:*

$$L_B(a, d) = \mathbb{E}_{p(z|d)}[\ell(a, z)]\,. \tag{53.1}$$

The underlying justification for this Bayesian-expected-loss formulation will be discussed in 53.1.1. For now we just note the importance of the loss function needing to represent the agent's *preferences*: the choice of the loss function is not arbitrary for a given agent. In fact, in a canonical derivation of the Bayesian-expected-loss formulation [1463], both the beliefs and loss function are simultaneously derived from an underlying system of rational preferences.

Here the data, d, is an observed quantity and is treated as fixed in the Bayesian framework. As such, the Bayesian expected loss is effectively just a function of our action and so can be used to define the notion of a Bayes-optimal action (also known as a Bayes-optimal decision, Bayes action, or, in the context of estimation, a Bayes estimator) as follows.

Definition 53.2 (Bayes-optimal action)**.** *An action, $a_B \in \mathcal{A}$, is known as Bayes-optimal for a given belief distribution, $p(z|d)$, and loss function, $\ell : \mathcal{A} \times \mathcal{Z} \to \mathbb{R}$, if it optimises the Bayesian expected loss, that is*[2]

$$a_B = \arg\min_{a \in \mathcal{A}} \mathbb{E}_{p(z|d)}[\ell(a, z)]\,. \tag{53.2}$$

We can further extend this idea of a Bayes-optimal action to that of a Bayes-optimal *policy*, $\pi_B : \mathcal{D} \to \mathcal{A}$, that maps from any given realisation of the data, d, to a corresponding Bayes-optimal action.

Definition 53.3 (Bayes-optimal policy)**.** *A policy, $\pi_B : \mathcal{D} \to \mathcal{A}$, is known as a Bayes-optimal policy if it produces Bayes-optimal actions for any given input data, d:*

$$\pi_B(d) = \arg\min_{a \in \mathcal{A}} \mathbb{E}_{p(z|d)}[\ell(a, z)] \;\; \forall d \in \mathcal{D}. \tag{53.3}$$

[2] In the interest of clarity, we implicitly assume here and throughout that there is a unique minimiser of the Bayesian expected loss for any given d. In practice, the minimiser might not be unique, so formally we should say that any $a_B \in \arg\min_{a \in \mathcal{A}} \mathbb{E}_{p(z|d)}[\ell(a, z)]$ is a Bayes-optimal action.

One of the key upshots of 53.2 is that the data only influences the Bayes-optimal action through the belief over states, $p(z|d)$. We can thus think of a Bayes-optimal policy as mapping from beliefs to actions, rather than requiring it to be a mapping from the data itself. This is arguably why the Bayesian-statistics and Bayesian-machine-learning literatures are so overwhelmingly focused on estimating posteriors [23, 170, 826, 874, 1609, 1742]: our beliefs on z contain everything we need to know to make downstream decisions (unlike in frequentist frameworks, where we need to reason about other data we *could* have seen). It is also crucial for sequential decision-making, where it shows that we need only keep track of our current beliefs to make optimal sequential decisions.

It is important, though, to appreciate that we have not made any direct assumptions on how the belief state, $p(z|d)$, is derived in formulating this decision-theoretic framework. As such, while Bayes' *rule* separately gives a mechanism for how beliefs should be updated to ensure they are coherent, this framework for making *static decisions* still holds up when beliefs have been derived by some other mechanism, as long as they really do represent the beliefs of the agent. Thus, we do not need to have explicitly defined prior and likelihood to follow a Bayesian approach to decision-making, at least for static decisions. Instead, we might, for example, directly define our belief state after seeing data, take it from some foundation model that we input the data to, or even take the belief state as the output from some training procedure on the data. As such, for static settings we should arguably use the notation $p(z; d)$ instead of $p(z|d)$, to emphasise that the data is taken as fixed and we want to construct our most faithful beliefs using it, but do not necessarily need to explicitly construct a joint model on z and d (whereas in the experimental-design setting we will find that such a joint *is* explicitly required).

Justification and interpretation

At a high level, the two core principles that lead to the above formulation of the Bayesian expected loss (and by extension Bayes-optimal actions) are *coherence* [383, 1376] and *rational preference* [1687].

Coherence is the idea that *subjective beliefs* follow the standard axioms of probability [912], namely non-negativity, additivity and normalisation. This statement is equivalent to saying that beliefs do not permit the construction of a Dutch book [383], wherein the agent can make a set of bets according to their beliefs that would guarantee they lose money. Cox's theorem [338] provides a further direct justification of coherence by showing that any *logical* approach for reasoning about beliefs must follow the laws of probability, where this notion of logical reasoning is defined through a set of intuitive axioms. This in turn implies that conditioning on observations yields Bayesian updates in beliefs: Bayes' rule is formally a result about probabilities and not beliefs, but by showing that logical beliefs must behave like probabilities, Cox's theorem allows us to apply Bayes' rule to belief updating.

Rational preference is the idea that the agent's preferences between different "lotteries" themselves adhere to natural logical rules, where each lottery is a given distribution over possible outcomes (e.g. an agent might prefer a lottery that assigns probabilities 0.3 and 0.7 respectively to the events that it rains and does not rain tomorrow, to a lottery that assigns probabilities 0.6 and 0.4). Specifically, von Neumann and Morgenstern [1687] showed that if preferences follow the axioms of completeness, transitivity, continuity and independence, then they can be represented through some utility function, such that the agent prefers lottery A over lottery B if and only if the expected utility of A exceeds that of B. In transferring these ideas to the Bayesian decision formulation, we take the space of outcomes to be the *consequences* that result from a given action–state pair, (a, z), and the loss is then simply the negative of the utility function implied by our preferences between such consequences. Note, though, that in this VNM utility theory the expectation is over an "objective" stochasticity in outcomes, with the key conceptual step needed to go from this to Bayesian decision theory being to show the axioms still imply an expected-utility formulation when the expectation is instead over subjective beliefs.

Together these axioms are already enough to derive the form of the Bayesian expected loss if one is also willing to accept the idea of subjective probabilities existing independently of preferences: coherence ensures that subjective beliefs behave like real probabilities and we can then use VNM utility theory to show that our preferences can in turn be represented as an expected loss. Savage [1463] was able to take things a step further and show how we can directly derive *both* subjective beliefs and a utility function together from a rational agent's preferences under uncertainty, subject to additional, slightly more complex axioms on nontriviality, measurability, richness, consistency and separability. In essence, Savage's work shows that if rational preference is defined in this stronger sense then coherence does not need to be a separate assumption: it is a property that directly follows from his axioms. The idea that we should update our beliefs using Bayes' rule in a sequential setting follows in turn from this need to be coherent to be rational.

The idea of deriving beliefs from preferences may at first seem a little odd. However, it becomes much more natural when we consider the idea of having preferences between different *bets*. For example, if we prefer a bet that returns £10 if it rains tomorrow to a bet that returns £10 if it does not, this implies that we *believe* it is more likely to rain tomorrow than not. Extending this idea to preferences over all possible bets will then yield a

full belief distribution, with rational preferences leading to coherent beliefs to avoid Dutch books. For example, if we are completely scale-insensitive in our preferences (i.e. our preferences only depend on the ratio of returns, not the stake) and prefer a bet returning $£X$ if it rains tomorrow over a bet returning $£10$ if it does not if and only if $X \geq 5$, then this implies we believe the probability of it raining tomorrow is exactly $2/3$.

The key intuition to take away from Savage's results is that any "rational" agent can implicitly be represented as a Bayes-optimal policy that optimises some Bayesian expected loss, where the quotations on "rational" are used to indicate this is meant in the formal sense of the agent's preferences following the Savage axioms. Note that this is not the same thing as saying that we can only derive rational agents from Bayesian assumptions or that an agent following any given Bayesian formulation will behave sensibly for a particular problem. It simply means that if our agent takes actions in a "rational" manner then it must have some implicit Bayesian-expected-loss representation, i.e. there must be some belief–loss pairing under which it is the Bayes-optimal policy.

Criticisms

There is a wide variety of criticisms of the Bayesian decision framework and it is only feasible to provide a very brief summary of them here.[3] We emphasise that many of these criticisms are disputed by people who advocate Bayesian approaches, with the Bayes-vs-frequentist debate representing by far the largest historical disagreement in the statistics community, at times bordering on a full-blown schism.

There are firstly various largely uncontroversial practical criticisms of Bayesian approaches regarding difficulties in effectively deriving belief states, the intractability of exact inference in most problems, and other computational issues. In particular, the need to derive a full belief distribution over z can be a lot more challenging than just deriving a point estimate. The scale of these challenges will naturally vary and they need to be considered on a problem-by-problem basis.

Most of the more fundamental criticisms of Bayesian decision theory stem from objections to the use of subjective beliefs to guide decision-making, or as it is more commonly put, objections to placing subjective priors on the state of the world [485]. These objections can themselves vary from more philosophical objections [1017] about subjective decision-making itself (e.g. that we should instead strive to make decisions with "objective" frequentist guarantees) to more pragmatic objections about our ability to construct probabilistic models that represent actual internal beliefs and account for all possible eventualities [858]. With regard to the latter, it is clear that just because an "optimal" agent must have a Bayesian representation does not mean that using a particular Bayesian *model* will yield effective behaviour in practice; most modern scholars would agree that there are some problems where a Bayesian approach is not practically sensible and others where it can be effective.

The former fundamental objections, though, are arguably more controversial. The Bayesian decision framework is a *normative* one specifying *idealised* decision-making, rather than a descriptive framework with respect to real human behaviour or a prescriptive framework for constructing effective decision-making systems in practice. One can, of course, dispute the practical utility of such a normative framework; but to dispute its normative statement that rational agents are Bayes-optimal requires criticising the Savage axioms themselves, as the statement directly follows from these.

Regardless, it is important to appreciate that using many modern Bayesian machine-learning methods—including virtually all Bayesian neural networks (BNNs)—to make decisions and predictions actually *violates* the assumptions of the Bayesian decision framework. Bayesian decision-making is fundamentally rooted in *beliefs* and when we, say, place a prior over the huge number of parameters of a neural network, it is not reasonable to assume this actually corresponds to our internal beliefs. By extension, any posterior we derive from this prior is also unlikely to be a true representation of our beliefs and the decisions we make from it unlikely to be Bayes-optimal with respect to our true beliefs. This is not to say that taking a Bayesian approach in such cases is foolish, or even that there are necessarily better ways of deriving our beliefs, but simply that we need to be careful in how we interpret the "posteriors" and decisions arising from such models. In particular, we should not blindly assume in such settings that the posterior is the best thing to take an expectation with respect to when it comes to making decisions. Moreover, the inexact nature of approximate inference can further push the behaviour of such models away from the Bayesian ideal; for example, it has recently been shown that common BNN approaches with approximate-inference schemes exhibit problematic behaviours that true Bayesian methods should not have [1329].

53.1.2 Frequentist decision theory

The Bayesian approach to making decisions that we have just laid out was rooted in how a rational agent should make decisions in the presence of incomplete knowledge. The frequentist view, by contrast, tries to remove

[3] See e.g. Berger [123], Efron [485], Gelman [597] and Rainforth [1365] for more comprehensive discussions.

subjectivity from the problem and instead defines the optimality of decision rules in terms of expected or long-run performance over *possible data that might have been generated*. At a high level, one can usually think of Bayesian methods as probabilistically reasoning about the state of the world, $z \in \mathcal{Z}$, while frequentist methods take this to be fixed and probabilistically reason about what data, $d \in \mathcal{D}$, we *could* have observed for a given z.

In other words, the frequentist approach takes an unconditional perspective towards observed data: d is treated as one of many possible realisations, and *decision rules* (i.e. policies), $\pi : \mathcal{D} \to \mathcal{A}$, are evaluated in terms of their expected performance in expectation over possible d, using the *frequentist risk* defined as follows.

Definition 53.4 (Frequentist risk)**.** *The frequentist risk, $R(\pi, z)$, of a decision rule, $\pi : \mathcal{D} \to \mathcal{A}$, given a world state, $z \in \mathcal{Z}$, and a data-generating distribution, $p(d; z)$, is the expectation over data, $d \sim p(d; z)$ (and any stochasticity in the decision rule itself) of a loss function, $\ell : \mathcal{A} \times \mathcal{Z} \to \mathbb{R}$:*

$$R(\pi, z) = \mathbb{E}_{p(d;z)}[\ell(\pi(d), z)] \,. \tag{53.4}$$

Here the concept of a loss function is shared with that in Bayesian decision theory, again with underlying motivations from VNM utility theory. However, $p(d; z)$ is not interpreted as a belief distribution but instead as a true data-generating distribution, with approximations to this then required for practical estimation of the frequentist risk (e.g. assuming a particular model form, typically known as a likelihood). Note our deliberate use of the notation $p(d; z)$, as opposed to $p(d|z)$, to represent the fact that z is not being treated probabilistically so we cannot think about probabilistically conditioning on it.

Unlike the Bayesian setup—where optimal actions were fully specified by the model and loss—the frequentist risk is a function of both π and z. Choosing a policy in the frequentist setup therefore still requires us to either specify some particular value of z or reason over the possible values that z could take. Because of this, there is no singular notion of a "frequentist-optimal policy" in the same way as there is a Bayes-optimal policy and we instead need to explicitly define an *optimality criterion* on top of the definition of the frequentist risk in order to choose a policy. There is significant flexibility in how this optimality criterion is set up in the frequentist framework and we essentially have free rein in what we choose for it, provided it only ever returns *admissible* decision rules as defined below.

Definition 53.5 (Admissible decision rule)**.** *A decision rule, or policy, π, is admissible for a given frequentist-risk formulation if it is not dominated by any other policy across all possible z, that is*

$$\nexists \pi' : (\forall z : R(\pi', z) \leq R(\pi, z)) \wedge (\exists z : R(\pi', z) < R(\pi, z)) \,. \tag{53.5}$$

Below we highlight some common optimality criteria used in practice.

- **Local optimality**: assuming a particular value of z (e.g. a maximum-likelihood estimate) and then choosing a policy that is optimal for this chosen value.
- **Uniform optimality**: under certain model and loss setups (or assumptions) we can sometimes find policies that are optimal for all z (e.g. confidence intervals under the assumption estimates are Gaussian distributed about the true value).
- **Minimax optimality**: choosing a policy that has the best worst-case performance over some allowed set of z, i.e. $\pi_{\text{minimax}} = \arg\min_{\pi:\mathcal{D}\to\mathcal{A}} \max_{z\in\mathcal{Z}_C} R(\pi, z)$ for some $\mathcal{Z}_C \subseteq \mathcal{Z}$.
- **Asymptotic optimality**: choosing π based on providing optimal performance in the limit of large data (noting that the true z can often be recovered from the data in this limit), e.g. zero-regret bandit strategies and estimators which converge at the optimal rate as given by the Cramér–Rao bound.

Another optimality criterion that is guaranteed to return an admissible policy is to choose the policy that optimises the so-called integrated risk,[4]

$$\bar{R}(\pi) = \mathbb{E}_{p(z)}[R(\pi, z)] \tag{53.6}$$

for some chosen distribution, $p(z)$. This, though, starts to blur the lines with the Bayesian approach as $p(z)$ is analogous to a prior in the Bayesian setup and one also arrives at the same mathematical integrated-risk formulation

[4]This is sometimes known as the Bayes risk. We stick with the term integrated risk because there is some inconsistency in the use of the term Bayes risk in the literature. For example, Robert [1405] used the term Bayes risk to refer to $\bar{R}(\pi_B)$, the integrated risk evaluated with the Bayes-optimal policy.

by taking an expectation of the Bayesian expected loss over possible observed data [123]. The interpretation is still arguably different.

Rather than trying to directly optimise some criterion, we can instead try to establish a policy that guarantees some required conditions regardless of the true value of z. For example, we might seek an estimator with provably consistent error bars.

Randomised decision-making

In the Bayesian setting, there is always a deterministic Bayes-optimal policy, $\pi_B : \mathcal{D} \to \mathcal{A}$, though it may not be the only Bayes-optimal policy [1050]. However, in frequentist settings, especially when using minimax optimality, it can be the case that the optimal policy is stochastic if either the action space, $\mathcal{A}$, or loss, ℓ, is not convex [123, Section 1.4]. For example, if $\mathcal{A} = \mathcal{Z} = \{0, 1\}$, $\ell(a, z) = \mathbb{I}(a = z)$ and we have no data, then $\max_{z \in \mathcal{Z}} R(\pi, z) = 1$ for any policy, π, that deterministically chooses a single action; but if our policy selects from $\mathcal{A}$ uniformly at random, then $\max_{z \in \mathcal{Z}} R(\pi, z) = 1/2$. In fact, this random policy turns out to be minimax-optimal.

As such, our earlier notation of decision rules as mappings $\mathcal{D} \mapsto \mathcal{A}$ was thus arguably a slight abuse of notation, and we should more carefully think of π as living in the space of stochastic mappings. For example, we can use a reparameterised decision rule, setting $a = \pi_r(d, \varepsilon)$ with $\varepsilon \sim p(\varepsilon)$, where $\varepsilon \in \mathbb{R}^n$ is some underlying randomness and we now have $\pi_r : \mathcal{D} \times \mathbb{R}^n \to \mathcal{A}$. The overall frequentist risk is then

$$R(\pi_r, z) = \mathbb{E}_{p(d;z)p(\varepsilon)}[\ell(\pi_r(d, \varepsilon), z)] \,. \tag{53.7}$$

We can, though, reformulate such problems back into cases of learning a decision rule that deterministically maps to a *distribution* over actions instead of a single action, by redefining our action space as a distribution over individual actions and/or redefining our loss as itself being an expectation, such that we now instead have $\pi : \mathcal{D} \to \mathcal{P}_{\mathcal{A}}$ and

$$R(\pi, z) = \mathbb{E}_{p(d;z)}\left[\bar{\ell}(\pi(d), z)\right] \tag{53.8}$$

$$\text{where} \quad \bar{\ell}(\pi(d), z) = \mathbb{E}_{a \sim \pi(d)}[\ell(a, z)] = \mathbb{E}_{p(\varepsilon)}[\ell(\pi_r(d, \varepsilon), z)] \,. \tag{53.9}$$

To keep notation simple and maintain consistency with the Bayesian framework, in what follows we will thus assume the setting of a deterministic policy (i.e. decision rule), with the implicit assumption that the problem has been reformulated to the above in cases where it might be necessary for this to be deterministic to achieve optimality in our original action space.

53.2 Theoretically optimal experimental design

Having provided a solid foundation in decision theory, we now switch our focus to the specific problem of data-gathering, leading up to a formal definition of *Bayesian experimental design* [1050] and construction of the analogous frequentist setup.

53.2.1 Data-gathering policies

Whereas in 53.1 we considered choosing actions or policies based on some existing data, here we consider policies for the combined process of gathering data and then using it. In the Bayesian setup we will use d to represent the future data being gathered, with any existing data absorbed into our model and left implicit; and in the frequentist setup we will use d to represent the combined existing and future data.

Following Rossa et al. [1424], we start by breaking up the overall decision-making policy as $\pi := (\pi_d, \pi_a)$, where $\pi_d \in \Pi_d$ is the part of the policy that can influence the data-gathering, which we call the *design policy*, and $\pi_a : \mathcal{D} \to \mathcal{A}$ is the part of the policy associated with downstream actions once the data is collected (often known as the *terminal* decision problem), which we refer to as the *action policy*.[5] Here the allowable set of data-gathering policies, Π_d, is dictated by the possible ways we can influence the data-gathering process. This is analogous to restrictions on the set of possible actions that can be performed in a more conventional post-data-gathering setting, and we can formally think of Π_d as being a space of "actions".

It is important to note that the elements of Π_d themselves generally correspond to adaptive data-gathering policies that sequentially map from previously collected data to sequential decisions known as *designs*. As such, choosing an "action" does not generally itself correspond to choosing a specific design, but instead an autoregressive function that will be applied to make decisions during a sequential data-gathering process.

[5] Formally, π_a can also directly depend on π_d rather than just the data itself. We omit consideration of this in our notation for simplicity.

More concretely, assume there are T points at which we can choose designs and thus influence the data-gathering process.[6] Further define $\xi_1, \xi_2, \ldots, \xi_T$ as the corresponding series of designs with each $\xi_t \in \Xi_t$ and $o_t \in \mathcal{O}_t$ as the experimental outcomes observed between the t-th and $(t+1)$-th decisions, such that overall data is given by $d = (d_1, d_2, \ldots, d_T)$, where each $d_t = (\xi_t, o_t) \in \mathcal{D}_t = \Xi_t \times \mathcal{O}_t$. Note that the data is defined here as containing the design decisions for notational convenience (e.g. in active learning the design could be an input and it is natural to think of the data as input-output pairs). However, each design ξ_t itself provides no new information in isolation given the previously observed $d_{1:t-1} = (d_1, d_2, \ldots, d_{t-1}) \in \mathcal{D}_{1:t-1} = \mathcal{D}_1 \times \mathcal{D}_2 \times \cdots \times \mathcal{D}_{t-1}$.

We can now think of π_d as an autoregressive mapping from previous data to the next design decision, such that $\xi_t = \pi_d(d_{1:t-1})$ and

$$\Pi_d \in \Xi_1 \times \{f \mid f : \mathcal{D}_1 \to \Xi_2\} \times \cdots \times \{f \mid f : \mathcal{D}_{1:t-1} \to \Xi_T\} \tag{53.10}$$

is a space formed by a collection of functional mappings from data *histories* to next design decisions. This setup is completely general in the sense that we utilise all information available at each step so that any optimal policy must conform to this form.

Analogously to in 53.1.2, in principle we can also use a stochastic design policy that instead maps from the previous data to a distribution of possible design decisions, such that $\pi_d(d_{1:t-1}) \in \mathcal{P}_{\Xi_t}$. Though this is sometimes needed in frequentist decision-making, and can also be helpful in practical training even when the optimal policy is deterministic [793], the optimal policy will still always be deterministic for all Bayesian and many frequentist settings. As in 53.1.2, we will thus stick to the notation of using a deterministic design policy, noting that this can still incorporate stochastic policies by viewing the design decisions as our choice of the distributions $p(\xi_t; d_{1:t-1})$, rather than the ξ_t themselves.

53.2.2 Bayesian experimental design

In the data-gathering setting we are uncertain about *both* z and the future data, d. Our Bayesian expected loss should thus be an expectation over both z and d. We also further have the restriction that our marginal beliefs on z should not depend on π_d, as our beliefs about the state of the world should not depend on the *future* policy we plan to make decisions with respect to. This then directly leads to the ActionBED formulation of the Bayesian future loss [1424]

$$L_B(\pi) = \mathbb{E}_{p(z,d;\pi_d)}[\ell(\pi_a(d), z)] = \mathbb{E}_{p(z)p(d|z;\pi_d)}[\ell(\pi_a(d), z)]\,. \tag{53.11}$$

Though formally the loss (and downstream-action policy) here can also depend on π_d and d (e.g. because different experimental choices incur different financial costs or to account for early-stopping rules), we will omit such consideration here in the interest of simplicity, referring the reader to Rainforth et al. [1370] for a more formal treatment of the fully general case. Note that we may, of course, also have access to historic data before d is gathered, but as per standard Bayesian decision rules, this influences our optimal decisions only through the beliefs on z, so we can omit direct consideration of this in our notation without loss of generality, as we simply update our definition of the prior, $p(z)$.

An overall Bayes-optimal policy is now a global minimizer for $L_B(\pi)$, that is $\pi^B = \arg\min_{\pi \in \Pi_d \times \Pi_a} L_B(\pi)$, with the Bayes-optimal design policy then just the part of this overall optimal policy corresponding to our design decision-making. However, existing work on practical BED methods has rarely directly reasoned about an explicit downstream-action policy (with Rossa et al. [1424] being the main notable exception in the modern literature). Instead, we can note that the coherence of Bayesian decision-making means that we can instead achieve a Bayes-optimal policy by making decisions sequentially in a way that is Bayes-optimal given the data seen so far and accounting for future decision-making through the loss we use at the current step. Namely, we can start with the optimal decision-making policy that would be carried out once data is observed, as per 53.3, then step backwards through the system, making each decision based on the knowledge that our future decisions will also be made Bayes-optimally. This then leads to the more classical definition for a Bayes-optimal design policy (see Rossa et al. [1424] for a proof of its equivalence to the minimiser of $L_B(\pi)$ discussed above).

Definition 53.6 (Bayes-optimal experimental-design policy; [254, 376, 584, 792, 799, 1050, 1364, 1424])**.** *A policy, π_d^B, for controlling a data-gathering process is known as a Bayes-optimal experimental-design policy for a given*

[6] Note that T does not necessarily need to itself be fixed, e.g. due to early-stopping rules. In such cases we can think of simply allowing "stop" as a design decision. We omit such consideration from our notation for simplicity, but note that none of our results or derivations rely on the assumption of a fixed T, with only the definition of Π_d itself needing updating in such cases.

belief–loss pairing, $(p(z)p(d|z;\pi_d),\ell)$, within an allowable space of possible policies, Π_d, if

$$\pi_d^B = \arg\min_{\pi_d\in\Pi_d} \mathbb{E}_{p(d;\pi_d)}\left[\min_{a\in\mathcal{A}} \mathbb{E}_{p(z|d)}[\ell(a,z)]\right], \tag{53.12}$$

where $p(z)$ represents our prior belief distribution over world states, $p(d|z;\pi_d)$ represents the conditional belief states over data given states for a particular data-gathering policy π_d, $\ell : \mathcal{A}\times\mathcal{Z}\to\mathbb{R}$ is a loss function that represents preferences of combined realisations of actions and states, $p(d;\pi_d) = \int p(z)p(d|z;\pi_d)dz$ is the implied marginal data distribution, and $p(z|d) = p(z)p(d|z;\pi_d)/p(d;\pi_d)$ is the implied posterior.

We can further now formally define *Bayesian experimental design* (BED) as being the study of data-gathering strategies that target a Bayes-optimal experimental-design policy or some approximation thereof.

A subtlety within this definition was that we deliberately wrote the posterior as $p(z|d)$ and not $p(z|d;\pi_d)$. This is because it turns out that the posterior is always independent of design policy given the observed data (noting that the data itself contains the design decisions) as shown by [1324], with the underlying intuition following from classical results on Bayesian ignorability and the likelihood principle [1432]. As such, the mechanisms for approximating posteriors can, if desired, be tackled independently from the problem of learning the design policy, e.g. by learning amortised-inference artefacts [794] or marginal score terms [1324].

From losses to uncertainties

While 53.6 reasons directly on the optimality of our policy in terms of the loss on terminal actions, Rossa et al. [1424] recently showed, based on earlier work [156, 376, 396, 677], that there is interestingly an alternative interpretation of Bayes-optimal design policies that only reasons about a proper scoring rule on our posterior beliefs, as demonstrated by the following result, which in turn we will reformulate to show that BED can be thought of in terms of minimising expected posterior uncertainty.

Theorem 53.1 ([1424])**.** *All Bayes-optimal experimental-design policies can be represented as minimisers of expected proper scoring rules on the posterior, and all minimisers of any expected scoring rule on the posterior are Bayes-optimal experimental-design policies for one or more belief–loss pairings.*

Namely, if $s : \mathcal{P}_z\times\mathcal{Z}\to\mathbb{R}$ is a proper scoring rule on $\mathcal{P}_z$ then

$$\pi_d^* = \arg\min_{\pi_d\in\Pi_d} \mathbb{E}_{p(z)p(d|z;\pi_d)}[s(p_z(\cdot|d),z)] \tag{53.13}$$

is a Bayes-optimal experimental-design policy for the belief model $p(z)p(d|z;\pi_d)$ and any loss function, $\ell\in\mathcal{L}$, where

$$\mathcal{L} = \left\{\ell : \ell\left(\tilde{\pi}_a^\ell(q_z),z\right) = s(q_z,z)\ \forall q_z\in\mathcal{P}_z, \pi_d\in\Pi_d, d\in\mathcal{D}, z\in\mathcal{Z}\right\} \tag{53.14}$$

and

$$\tilde{\pi}_a^\ell(q_z) = \arg\min_{a\in\mathcal{A}} \mathbb{E}_{z\sim q_z}[\ell(a,z)]\,. \tag{53.15}$$

Furthermore, $\mathcal{L}$ is never an empty set, and any integrable and measurable ℓ produces a corresponding s that is a proper scoring rule (though not necessarily a strictly proper scoring rule) given by

$$s(q_z,z) = \ell\left(\tilde{\pi}_a^\ell(q_z),z\right). \tag{53.16}$$

This is a somewhat profound result: it tells us that to conduct Bayes-optimal decision-making in the data-gathering setting, we do not need to explicitly account for downstream actions and a terminal loss if we are able to instead define an appropriate proper scoring rule on our future posterior distribution that also encodes our downstream preferences. Which of these is easier to faithfully specify will generally vary between problems.

This result becomes even more interesting when we realise that an expectation of such a proper scoring rule can be interpreted as an uncertainty measure, leading to uncertainty-based notions of Bayes-optimal design policies. In particular, given some distribution, $q(z)$, and some scoring rule, s, we can formally think of

$$h_s[q(z)] = \mathbb{E}_{q(z)}[s(q(\cdot),z)] \tag{53.17}$$

as defining a measure of uncertainty in $z \sim q(z)$ [376, 396]. In fact, this is arguably *the* definition of a Bayesian uncertainty measure; the choice of s here is not arbitrary but directly dictated by our downstream preferences via the corresponding loss function [1370].[7] The resulting h_s is also known as a *generalised entropy* [677].

In turn, we can use this to define experimental-design objectives by considering the expected posterior uncertainty using such a measure:

$$\mathrm{EPU}_s(\pi_d) = \mathbb{E}_{p(d;\pi_d)}[h_s[p(z|d)]] = \mathbb{E}_{p(z)p(d|z;\pi_d)}[s(p_z(\cdot|d), z)]\,. \tag{53.18}$$

Using 53.1, *all* BED objectives can be written in this way. As such, we can think of the set of h_s of the form 53.17 as being a *completely general notion of uncertainty* in a Bayesian-decision-theory context, in the sense that uncertainty measures that cannot be written as a particular h_s will not lead to Bayes-optimal decision-making.[8]

In practice, BED problems are often instead represented as *maximising* an expected uncertainty reduction from prior to posterior [156]:

$$\begin{aligned}\mathrm{EUR}_s(\pi_d) &= \mathbb{E}_{p(d;\pi_d)}[h_s[p(z)] - h_s[p(z|d)]] &(53.19)\\ &= \mathbb{E}_{p(z)p(d|z;\pi_d)}[s(p_z(\cdot), z) - s(p_z(\cdot|d), z)]\,. &(53.20)\end{aligned}$$

In the same way that h_s can be thought of as defining a generalised entropy, we can here think of the term $h_s[p(z)] - h_s[p(z|d)]$ as a *generalised information* (or generalised information gain) provided by the data. If the loss does not directly depend on π_d or d, then maximising such expected uncertainty reductions is exactly equivalent to the formal BED setup, but an extra cost term is required when it does.

We have now established that, alongside 53.6, we can formally think of BED approaches as equivalently minimising expected future losses of the form given in 53.11, minimising expected posterior uncertainties of the form given in 53.18, or maximising expected uncertainty reductions of the form given in 53.19 (with appropriate adjustment for costs if our loss depends directly on π_d and/or d). Using the terminology of h_s being a generalised entropy, we can also think of this last construction as maximising a *generalised expected information gain* (generalised EIG; [396]), noting that a gain in information is formally defined through a reduction in entropy.

53.2.3 Frequentist experimental design

Mathematically, it is straightforward to incorporate experimental design into the frequentist-decision-theory setup, as we simply use the fact that the data-generating procedure depends on the experimental-design policy. Namely, the frequentist risk becomes

$$R(\pi, z) = \mathbb{E}_{p(d;z,\pi_d)}[\ell(\pi_a(d), z)]\,, \tag{53.21}$$

where, as with the Bayesian setup, $\pi = (\pi_d, \pi_a)$ with $\pi_d \in \Pi_d$ and $\pi_a : \mathcal{D} \to \mathcal{A}$ (and Π_d as per 53.10). Note here that the expectation in 53.21 is always over *both* past and future data: unlike in the Bayesian setup, we continue to reason about alternative data we might have previously seen, rather than taking it as fixed.

As with other frequentist decision-making, the optimal data-gathering policy will now depend on the optimality criterion used and in particular how we choose z. Unlike in the Bayesian setup, we are further not tied down to a particular mechanism for choosing π_a: frequentist systems do not need to be coherent and so we do not need to assume we will act Bayes-optimally in the future. For example, it is common to assume that we will later carry out maximum-likelihood estimation, motivating the use of strategies based around Fisher information matrices due to their link to the asymptotic maximum-likelihood-estimator distribution and the Cramér-Rao bound [342, 1379].

More broadly, the differences between the Bayesian and frequentist setups are more subtle in the data-gathering setting than in data analysis because we do not necessarily already have access to any data. Properly going into this is beyond the scope of this chapter; we refer to Rainforth et al. [1370] for more extensive discussion on the topic.

[7] More precisely, this can be viewed as the "uncentered" measure of Bayesian uncertainty, as it does not subtract the expected loss that would be incurred under perfect information. However, as this perfect-information loss is independent of any actions we actually take, it can generally be ignored for decision-making. See [1370] for details.

[8] Note that the interpretation here gets a little more complicated when ℓ, and by extension s, can depend on π_d and/or d, as here our design policy and/or data will change the form of the generalised entropy itself. In such cases, it arguably does not make sense to think about the expected future loss as an expected posterior uncertainty, as the "uncertainty measure" is itself being optimised in the process. An example of when this can occur is if there are costs associated with our data gathering, or if we want to control for particular properties of the data itself like privacy or fairness [726]. See Rainforth et al. [1370] for full discussion of this more complex case.

Scoring rule $s(q(\cdot), z)$	**Uncertainty** $h_s[q(z)]$	**Expected uncertainty reduction** $h_s[p(z)] - \mathbb{E}_{p(d;\pi_d)}[h_s[p(z\|d;\pi_d)]]$
Log loss $-\log q(z)$	Shannon entropy $\mathrm{H}[q(z)]$	Expected information gain $\mathbb{E}_{p(z)p(d\|z;\pi_d)}\left[\log \frac{p(d\|z;\pi_d)}{\mathbb{E}_{p(z)}[p(d\|z;\pi_d)]}\right]$
Squared distance to mean $\left\|z - \mathbb{E}_{q(z)}[z]\right\|_2^2$	Trace of covariance $\mathrm{Tr}\big(\mathrm{Cov}_{q(z)}[z]\big)$	Expected total variance reduction $\mathrm{Tr}\big(\mathrm{Cov}_{p(z)}[z] - \mathbb{E}_{p(d;\pi_d)}\big[\mathrm{Cov}_{p(z\|d)}[z]\big]\big)$
Hyvärinen [807] score $2\Delta_z \log q(z) + \left\|\frac{\partial \log q(z)}{\partial z}\right\|_2^2$	Fisher "entropy" $\mathbb{E}_{q(z)}\left[-\left\|\frac{\partial \log q(z)}{\partial z}\right\|_2^2\right]$	Expected trace of Fisher information $\mathbb{E}_{p(z)}\left[\mathrm{Tr}\left(\mathbb{E}_{p(d\|z;\pi_d)}\left[-\frac{\partial^2 \log p(d\|z;\pi_d)}{\partial z \partial z^\top}\right]\right)\right]$

Table 53.1: Common examples of proper scoring rules, their corresponding uncertainty measures, and the resulting BED objectives that maximise an expected uncertainty reduction. See Walker [1697] and Prangle et al. [1344] for proof that the Hyvärinen score (in which Δ_z denotes a Laplace operator) yields the expected trace of the Fisher information.

53.3 Practical experimental design

Building on the decision-theoretic foundations of experimental design from both a Bayesian and a frequentist perspective, we now turn our attention to how these frameworks are in turn used to (adaptively) optimise experiments. Properly covering the large space of existing approaches here is inevitably not feasible, so we instead focus on two classes of approaches that together cover the vast majority of methods used in practice: those based on expected uncertainty reduction, including information-theoretic approaches; and those based on Fisher information matrices. These respectively correspond to the most common approaches within the Bayesian and frequentist frameworks.

53.3.1 Expected-uncertainty-reduction approaches

When working within the Bayesian framework, it has historically been rare for practical methods to explicitly reason about downstream actions after the data has been gathered, with almost all methods instead implicitly relying on 53.1 to express criteria in terms of expected scoring rules on the posterior. In particular, most methods used in practice formulate BED problems in terms of maximising an *expected uncertainty reduction* (EUR) from the prior to the posterior (53.2.2); some prominent examples are presented in 53.1. Here we discuss some important features of EUR approaches in general before paying closer attention to the particular case of maximising expected information gain.

Key considerations and practicalities

Indexing between different BED objectives simply comes down to different choices of our model and scoring rule (or downstream loss in the cases this is done directly). Fundamentally, the success of any BED approach thus relies on three things:

(a) the faithfulness of the choice of uncertainty measure, scoring rule or loss to our actual preferences;

(b) the effectiveness of our model in capturing both the true data-generating process (remembering $p(d;\pi_d)$ represents our beliefs about possible data that might be generated, not the ground-truth process) and our beliefs and uncertainty (where the way we measure this uncertainty is itself implied from the choice of scoring rule using h_s); and

(c) our ability to practically optimise the BED objective and thus find a suitable data-gathering policy, π_d.

It is difficult to talk about how to achieve (a) and (b) in a fully general sense, as they are inherently about the accuracy to which our problem construction truly conveys the real world and our preferences, while these considerations are also often wrapped up with the practical considerations, (c). It is thus beyond the scope of this chapter to really delve into these fully, but we note that Shannon entropy has become almost the canonical measure of uncertainty, as discussed in 53.3.1. We also highlight the clear link to Bayesian deep learning, in that our success is rooted in our ability to capture uncertainty.

Much of the BED literature, especially in recent times, has focused instead on (c). EPUs and EURs are inherently doubly intractable, making them extremely difficult to estimate and optimise [1367, 1369]. This can become particularly challenging when dealing with adaptive sequential experimentation, where approximations are typically made to the pure BED framework to make design strategies practical. As we will see in 53.3.4, various methods have been proposed to try and circumvent, or at least mitigate, these computational issues.

An important point to note here is that methods should *not* be considered Bayesian just because they incorporate (predictive) uncertainty, even if that uncertainty is derived from a Bayesian model. The key feature of BED approaches is reasoning about expected *future* losses or uncertainties, not the immediate predictive uncertainty. For example, gathering data in active learning by choosing inputs with the highest predictive uncertainty has no direct justification from Bayesian decision theory and is implicitly based on the heuristic assumption that if one is more uncertain about the outcome of an experiment then it will likely provide more useful data.

This distinction between Bayesian decision-making and Bayesian models is an important one: the problem of experimental design is fundamentally one of decision-making and the use of Bayesian models stems from the underlying decision theory (and is specific to the sequential decision-making setting) rather than the other way around.

Furthermore, there are no universal measures for how effectively a model captures uncertainty. So when justifying models like BNNs, this must be rooted in the tasks we wish to conduct and the decisions we want to make to be meaningful [1329, 1501]. Indeed, an important result for all BED approaches is that we only need to consider uncertainty in variables that directly influence our downstream loss [1370]. Thus, when defining BED procedures through uncertainty measures or score functions, we should make sure these are always in the space of the minimal set of variables that we expect any hypothetical future losses to directly depend on. For example, if we are interested in prediction then we should define a proper scoring rule on the posterior predictive, not the model parameters, and if we care about estimation of some particular quantities then we should define a scoring rule measuring the error in estimating those precise quantities rather than on some preimage or mapping of them.

Expected information gain

By far the most common form of expected uncertainty reduction in the BED literature is the *expected information gain* (EIG) of the experiment, specifically the expected reduction in Shannon entropy from the prior to the posterior [129, 1049, 1050]. This exactly conforms to the formal BED setup, simply corresponding to the expected uncertainty reduction that results from using log loss on beliefs about some variables of interest, ψ, as our associated proper scoring rule, yielding the objective

$$\mathrm{EIG}_\psi(\pi_d) = \mathbb{E}_{p(d;\pi_d)}[\mathrm{H}[p(\psi)] - \mathrm{H}[p(\psi|d)]] = \mathbb{E}_{p(\psi)p(d|\psi;\pi_d)}\left[\log \frac{p(\psi|d)}{p(\psi)}\right] \tag{53.22}$$

where H denotes Shannon entropy [1499]. Note here that we have deliberately formulated the EIG in terms of a particular set of variables of interest instead (which are implicitly some pushforward of the world state, z) because of the arguments presented in 53.3.1 and because the ability to target the EIG in different quantities often itself provides alternative experimental-design strategies (e.g. the BALD vs EPIG acquisition objectives covered in 53.4.2 respectively target EIG in model parameters and predictions).

The EIG admits a wide variety of different useful equivalent forms for any given choice of design policy, such as the *mutual information* between ψ and the data, along with the following equivalent formulations:

$$\begin{aligned}
\mathrm{EIG}_\psi(\pi_d) &= \mathbb{E}_{p(\psi)p(d|\psi;\pi_d)}\left[\log \frac{p(d|\psi;\pi_d)}{p(d;\pi_d)}\right] && (53.23)\\
&= \mathbb{E}_{p(\psi)}[\mathrm{H}[p(d;\pi_d)] - \mathrm{H}[p(d|\psi;\pi_d)]] && (53.24)\\
&= \mathbb{E}_{p(d;\pi_d)}[\mathrm{KL}[p(\psi|d) \,\|\, p(\psi)]] && (53.25)\\
&= \mathrm{KL}[p(\psi)p(d|\psi;\pi_d) \,\|\, p(\psi)p(d;\pi_d)] && (53.26)
\end{aligned}$$

where KL denotes the KL divergence [945]. It has been justified as an experimental-design objective in a wide variety of different ways using both Bayesian decision theory and information theory (e.g. [129, 396, 790, 1049, 1050, 1369]). Many of these different interpretations are important in different contexts, particularly when it comes to the practical estimation and optimisation of the EIG. For example, because the EIG is equivalent to a mutual information for any given design policy, we can utilise the symmetry of mutual informations to avoid ever having to directly work with posterior distributions, using the ratio of the likelihood to the marginal likelihood as per 53.23 instead. In turn, this is critical in applications like active learning (where ψ can be the parameters

of a large model but the label we acquire could be a single categorical variable) as discussed later, but also other large-scale BED settings as well [300].

While its overwhelming popularity in the experimental-design setting likely stems, in part, from simply being the best-known approach that formally follows the BED framework (to the extent that the two have often historically been seen as synonymous), it does also have many appealing properties that provide some compelling justification for this status. For example, it is unique in being invariant to reparametrisations and even injective mappings of ψ, and is also unique in being a *strictly* proper scoring rule that only depends on the probability (density) of events that actually occur rather than hypothetical alternatives; see Rainforth et al. [1370] for details.

These properties make the EIG a very appealing choice as a default strategy for BED when we are unsure exactly how our data will be used. In particular, while VNM utility theory tells us that, in principle, a loss exists that encapsulates our preferences over downstream consequences whether we know exactly how the data will be used or not (with this loss implicitly encapsulating possible ways the data *could* be used if we do not yet know), in practice it can be difficult or impossible to properly write these down in the form of a concrete function. The EIG is often an attractive proposition in such settings, though care should still be taken about *which* variables we target the EIG in.

53.3.2 Fisher-information and estimator-sharpness approaches

The classical, frequentist approach to experimental design has a long and extensive history dating back to before Bayesian (and most frequentist) decision theory had been developed. Early seminal work in the area includes that of Fisher [527], Kiefer [886], Smith [1525] and Wald [1692].[9]

We will focus on the predominant approach of choosing designs according to minimising invariants of the *Fisher information matrix* (e.g. determinant or trace of inverse), defined as [524, 527]

$$F(\pi_d, z) = \mathbb{E}_{p(d;z,\pi_d)}\left[\left(\frac{\partial}{\partial z}\log p(d;z,\pi_d)\right)\left(\frac{\partial}{\partial z}\log p(d;z,\pi_d)\right)^\top\right] \tag{53.27}$$

$$= \mathbb{E}_{p(d;z,\pi_d)}\left[-\frac{\partial^2}{\partial z \partial z^\top}\log p(d;z,\pi_d)\right]. \tag{53.28}$$

The Fisher information represents the curvature of the log likelihood around a particular value, z (whose value must generally be chosen using some optimality criterion as per standard frequentist decision theory). We can thus think of it as a *local* measurement of the *sharpness* of the likelihood around z: if the likelihood is very tightly peaked, then small variations in z dramatically change the likelihood of the data, implying that the data is (locally) highly informative for learning about z. In the frequentist literature it is typically described as measuring how much information the data provides about the parameters.

This notion of information is different to the Shannon information and generalised informations we have seen in the BED literature, but does share some similar properties, such as the chain rule of expected uncertainty reductions. We can think of FIMs as measuring a set of uncertainty reductions given by the Hessian of the corresponding densities [1370], which have the unusual, and from a frequentist perspective highly beneficial, property of being independent of the prior when we look at the difference in score between prior and posterior. However, we should also note that it is just one *possible* choice of uncertainty, rather than being appropriate for all problems. In fact, it can lead to various pathologies when used in practice, such as the fact it is generally optimised by *non-adaptive* design strategies unless adaptation is forced from the optimality criterion itself [1370], in turn sometimes leading to catastrophic failures when used as an adaptive design or active-learning strategy, such as never reducing uncertainty in certain dimensions of z.

Because $F(\pi_d, z)$ is a matrix instead of a scalar, it is not itself a suitable objective for optimal experimental design, and we must instead optimise some invariant of it. Different choices of the chosen invariant are typically known as the *alphabetic criteria*. Below we highlight some common choices.

- **A-optimality**: minimising the trace of the inverse of the FIM (the A stands for "average"). It can be viewed as minimising the asymptotic sum of the variances of different dimensions of z. Though computationally convenient, it can be quite a brittle objective in practice, as the optimal designs change under reparameterisation or even just rescaling of the dimensions of z (if we scale a dimension of z by a factor of 10, that dimension will have 10 times less influence on the objective). It also fails to account for any reduction in information that comes from co-dependence between different dimensions of z.

[9] Covering the long history in this area is beyond the scope of this chapter. We refer the reader to Atkinson et al. [59] and Pukelsheim [1347] for comprehensive textbook references.

- **c-optimality**: minimising $v^\top F(\pi_d, z)^{-1} v$ for some projection vector v. Sometimes this is extended to the trace of $M^\top F(\pi_d, z)^{-1} M$ for a matrix M (sometimes known as L-optimality) or minimising for a worst case projection vector v (sometimes known as G-optimality). Prangle et al. [1344] further provide a formal link to working with worst-case matrix projections to D-optimality below.

- **D-optimality**: maximising the determinant of the FIM. This can be viewed as maximising the "volume" of the FIM and equivalently can be viewed as maximising the geometric mean of the FIM eigenvalues or minimising the asymptotic generalised variance (i.e. the determinant of the covariance) of the maximum-likelihood estimator for z. In general, the D-optimal designs do not change under smooth one-to-one reparameterisation of z.

- **E-optimality**: maximising the minimum eigenvalue of the FIM. This can be useful as a robust criterion where we need to make sure all components of z are estimated well; it can be thought of as minimising the asymptotic largest principal-axis variance of the maximum-likelihood estimator for z.

Of these criteria, A-optimality and D-optimality are generally the most commonly used, with the former typically favoured for computational reasons and the latter because of its reparameterisation invariance and incorporation of codependency effects.

As with all frequentist methods, we also need to have some kind of optimality criterion for dealing with the fact that z is unknown. Bizarrely, this problem is often somewhat ignored in the frequentist-experimental-design literature, especially in older papers. The primary reason for this is that if one uses a linear Gaussian model of the form $d|z;\xi \sim \mathcal{N}(\xi z, \sigma^2)$ for some fixed variance, σ^2, then the FIM turns out to be independent of z and simply equal to $\xi\xi^\top/\sigma^2$. Thus, for the linear models that have dominated much of classical statistics, this problem conveniently disappears. In the modern world this is, of course, far less often the case and we must indeed come up with a mechanism to choose z. In principle, any admissible optimality criterion can be used, but perhaps the most common is simply to use a point estimate for z, typically the current maximum-likelihood estimate given previous data [290]. This assumption of a particular value for the precise quantity we are gathering data to estimate is arguably somewhat unsatisfying. More complex procedures have been proposed as well, such as minimax approaches [419].

53.3.3 Adaptive design

Optimal experimental design often requires us to be *adaptive*, utilising information gathered from previous data to guide our design decisions and learn on the fly. Namely, following 53.2.1, we can use the data gathered so far, $d_{1:t-1}$, to help inform each sequential design decision, ξ_t, where t indexes the design decisions we must make as part of the experimental design process. This adaptive aspect of optimal experimental design has been implicit in everything we have covered, through the notion of a design policy, π_d, with design decisions explicitly framed as mappings from previously observed data.

Traditional myopic adaptive design

The idea of training an *explicit* design policy is relatively new, with the concept of a design policy first introduced by Huan and Marzouk [789], and later refined into the form presented in 53.2.1 and popularised by Foster et al. [552] and Ivanova et al. [823], with the concept of a downstream action policy in turn introduced by [1424]. Prior to this work, the standard approach to adaptive design had been to follow an *implicit* policy, in which each individual design decision, ξ_t, is chosen by directly optimising some experimental-design objective that incorporates the data gathered so far, rather than following a pre-determined decision rule. In the BED setting, this objective has typically been taken to be the *incremental* EIG in the target variables ψ,

$$\mathrm{EIG}_{\psi|d_{1:t-1}}(\xi_t) = \mathbb{E}_{p(d_t|d_{1:t-1},\xi_t)}[\mathrm{H}[p(\psi|d_{1:t-1})] - \mathrm{H}[p(\psi|d_{1:t})]]\,, \tag{53.29}$$

which is equivalent to a standard EIG on the next experiment step, with a prior given by our current posterior beliefs given the data observed so far, $p(\psi|d_{1:t-1})$. Frequentist and active-learning approaches often follow a similar procedure with non-Bayesian updates to incorporate new data (e.g. using point estimates based on maximum-likelihood estimation in frequentist settings, or re-running a training procedure on all the current data in active-learning settings). In what follows, we will mostly focus on the BED setting, noting that adaptation has historically been a bigger focus for the Bayesian literature than for the frequentist literature.

An important result shown by Foster et al. [552] and then refined by Ivanova et al. [823] is that the expected sum of the incremental EIGs is equal to the total EIG across all experiment iterations:

$$\mathbb{E}_{p(d_{1:T-1};\pi_d)}\left[\sum_{t=1}^{T}\mathrm{EIG}_{\psi|d_{1:t-1}}(\xi_t)\right] = \mathbb{E}_{p(d_{1:T};\pi_d)}[\mathrm{H}[p(\psi)] - \mathrm{H}[p(\psi|d_{1:T})]] = \mathrm{EIG}_{\psi}(\pi_d), \tag{53.30}$$

where π_d can be either an explicit or implicit policy. This result turns out not to be specific to EIGs, but holds for EURs more generally [1370]. The implication of this is that the best overall policy is the one that maximises the average of the individual incremental EURs at each step. There is thus an underlying *dense* reward (in the reinforcement-learning sense) that can help provide signal for individual design decisions and provides some justification for this incremental approach.

However, this does not mean that this incremental approach will generally be optimal: it is making *myopic* (or "greedy") decisions that do not account for the fact that the design decision made at one iteration may influence not just the EUR at the next experiment iteration, but also future EURs as well. For example, imagine a scenario where we have one-dimensional designs in the space $[0, 1]$ and our loss is such that the EPU is minimised when the designs are as evenly spaced as possible. If we have two designs and plan ahead about where to place them, the optimal designs will be at $1/3$ and $2/3$. If we follow a myopic approach, though, the first design will be placed at $1/2$ and the second at either $1/4$ or $3/4$, leading to a sub-optimal final spacing. In the language of reinforcement learning, we are essentially neglecting the fact that the value function typically varies between different design decisions.

Explicit-policy approaches

In addition to the theoretical suboptimality from myopic decision-making, the traditional adaptive design approach has two critical weaknesses: (a) all computation must be performed *during* the experiment and (b) for all Bayesian and some frequentist approaches, we need to perform model updates between experiment iterations. Both of these can be extremely computationally restrictive in practical problems, where design decisions must typically be made in real time to be useful.

To circumvent these issues, many modern BED approaches instead look to learn an explicit policy, which can then be directly evaluated during the experiment to make design decisions quickly. One can view such approaches as amortising the overall experimental-design problem, performing all required computation upfront rather than during the experiment. While still a groundbreaking development, the first policy-based approach, introduced by Huan and Marzouk [789], was not well suited in this regard as it used the posterior beliefs as the "state" within a reinforcement-learning problem construction, meaning that inference (and thus substantial computation) was still required at deployment time.

The key breakthrough for solving these problems was provided by Foster et al. [552], whose deep adaptive design (DAD) approach instead constructed the design policy as a deep neural network that directly maps from the current data itself, $d_{1:t-1}$, to the next design decision, ξ_t, as per the framework presented in 53.2.1. This then allowed the design decisions to be made in a matter of milliseconds through a simple forward pass of their learned *design network*. They further showed that this policy can be trained using end-to-end stochastic gradients through the full experiment rollout, training directly on (bounds or estimates of) the total EIG, $\mathrm{EIG}_{\psi}(\pi_d)$. That is, DAD trains the design network by generating a full dataset using the current policy, estimating the information gain provided by that dataset, then backpropagating through this whole process to update the policy.

Interestingly, DAD showed significant performance improvements over the traditional myopic approach in design quality, not just computational cost at deployment. Part of the reason for this is in allowing non-myopic policies that plan ahead to be learnt. A second, more subtle factor is that it avoids the need to explicitly perform inference between experiment steps; this is critical because inaccuracies in this inference transpire to be one of the main sources of error in previous approaches, and learning direct mappings from previous data to next design decision is sometimes easier than the posterior-inference problem itself. It was also found that the cost of training effective design networks was often comparable to running a *single* rollout of the traditional myopic framework, such that this approach can still be very effective and practical even when fast deployment is not a requirement. Consequently, virtually all current state-of-the-art adaptive BED methods now rely on some form of explicit policy learning.

A wide variety of work has since directly built on DAD's explicit policy-learning approach. For example, Ivanova et al. [823] extended the framework to work on implicit models when the likelihood density is not available by introducing a learned critic network, and Hedman et al. [725] showed that periodically finetuning the policy during the experiment using collected data can provide benefits.

Following Huan and Marzouk [789], other work has formulated the policy-learning problem through the lens of reinforcement learning [168, 1040], while using data collected as the state so far, as in DAD, to ensure computational efficiency at deployment. Huang et al. [792] took this further by allowing for different downstream utilities than EIG; Huang et al. [793] built on this to simultaneously learn amortised predictive distributions that can help support the training of the policy. Huang et al. [794] showed that neither the raw data nor an explicit posterior approximation are typically optimal for representing the current state as the input to the policy network, showing that significant empirical gains can be achieved by using internal representations of amortised-inference foundation models, which can help capture invariances in the posterior without introducing the error that originates from an explicit approximation.

Most recently, Rossa et al. [1424] proposed directly targeting expected future losses instead of expected uncertainty reductions and co-training an explicit downstream action policy alongside the explicit design policy, and showed that this approach can simplify training while providing improvements in both customisability and performance.

53.3.4 Estimation of BED objectives

A property shared by virtually all BED objectives is that they are very difficult to estimate and optimise, because we are inherently reasoning about possible future beliefs given data, rather than just our current beliefs. In particular, unless the posterior has a known analytic form, the general BED formulation as an expected scoring rule of the posterior given in 53.18, and by extension the expected-uncertainty-reduction formulation of 53.19, are *nested expectations* that are *doubly intractable* to estimate except in special cases [905, 1367], with the inner expectation corresponding to the unknown normalising constant of the posterior. That is, we are trying to optimise an expectation of a nonlinear function of an expectation (noting that non-constant proper scoring rules must be nonlinear), which cannot generally be done directly with standard techniques like Monte Carlo and stochastic gradient descent, and instead requires more complex and costly strategies like nested Monte Carlo [1367] or multi-level Monte Carlo [628].

Extensive research has been conducted into how to best carry out this estimation in the specific case of using the EIG as the objective, with such work forming a prominent part of the BED literature in recent years. Examples of approaches that have been proposed include nested Monte Carlo [330, 1219, 1365, 1367, 1444], variational and (amortised) density-approximation methods [93, 550–552, 793, 1026], multi-level Monte Carlo [454, 643], Laplace approximations [103, 247, 1073], score-based approximations [1324], and other approximate sampling-based strategies [38, 228, 818, 819, 821, 1459]. It is notable that the problem becomes even more challenging when working with *implicit* models, wherein we can sample from our likelihood but not evaluate its density [823, 903]. Covering these properly is beyond the scope of the current work; we refer the reader to Huan et al. [790] and Rainforth et al. [1369] for comprehensive recent reviews on the topic.

Research on dealing with the required estimation for other losses has been more limited, but is starting to become a more active area of research. The most common traditional approach in such settings, which have been more commonly considered in the specific active-learning literature than the rest of the experimental-design literature, has been to rely on 53.1 to formulate the BED objective as an EUR where the scoring rule is directly specified rather than the loss itself, then calculate approximate posteriors (or retrain a model in a non-Bayesian way) each time new data is sampled [1429], thus treating it as a nested-inference problem [1366]. More modern work has built on this with amortisation, reweighting, exploiting an ensemble, or similar [792, 1218, 1589]. Various methods have also been proposed for specific other EUR objectives, such as expected variance reduction [320, 758].

Huang et al. [799] recently devised an approach for the setting where the loss is directly specified, noting that deriving the scoring rule from the underlying loss is itself typically intractable. Namely, they showed that whenever the loss takes the form of a weighted Bregman divergence, the optimisation over actions in 53.12 has an analytic solution where the optimal action is some kind of weighted posterior mean, such that the scoring rule associated with the loss can be derived without requiring a nested optimisation to be performed. In turn, this allowed them to derive a simple and practical (albeit expensive) nested Monte Carlo estimator that can be applied to any BED objective whose loss takes this form, which is itself quite general and incorporates many useful losses, including those that give rise to the EIG and expected variance reduction.

Finally, Rossa et al. [1424] recently suggested a new approach that can be applied for any downstream loss function, only requires optimisation of a singly intractable objective without any posterior approximations at all, and which is naturally implicit (i.e. it does not require access to model densities, only forward samples). This approach is notably straightforward: instead of relying on 53.1 to reformulate the problem into uncertainties, they simply introduce an explicit action network, analogous to the design network in deep adaptive design (53.3.3), and then directly minimise the expected future loss in the form of 53.11 using stochastic gradients. Their critical

insight is that, if we go back to this foundational formulation of what it means to be Bayes-optimal and jointly train design and action policies together, this yields a singly-intractable problem formulation that is naturally task-driven and avoids posterior calculations completely (even after the data has been gathered), thereby providing a much simpler and, at least in principle, more computationally friendly framework than dealing with EUR. In other words, the nested nature of BED objectives is actually an artefact of reformulating them into EPUs or EURs, rather than inherent to the original problem formulation. Though the impact this work will have on the field remains to be seen, it suggests an interesting potential paradigm shift in how we think about BED, moving from the traditional idea of maximising EUR, to instead thinking directly about minimising expected future loss on final tasks of interest.

53.4 Active learning

So far we have used statistical decision theory to identify notions of optimality in experimental design, and we have discussed the translation of these into practical methods. Now we turn to active learning, which is often considered synonymous with experimental design but which we suggest can more usefully be interpreted as a particular form of experimental design focused on *accurate predictions* of outputs for inputs that are often unknown at the time of data-gathering. We highlight distinctive aspects of active learning, review a number of popular data-gathering methods, and discuss some key methodological considerations.[10]

53.4.1 Overview

Like other technical topics, active learning can be thought of as three things at once: "a problem, a class of solution methods that work well on the problem, and the field that studies this problem and its solution methods" [1579]. As a field it is home to much of the machine-learning field's study of intelligent data-gathering, while also being effectively separate from the machine-learning subfield of reinforcement learning and the statistical subfield of experimental design. These fields overlap but it is useful to distinguish between them because of differences in their predominant problem definitions and solution methods, which we explore throughout this section.

Problem

Active learning as a problem class inherits the structure from 53.2. We seek a combination of design policy, π_d, and action policy, π_a, that minimises the expected value (under a Bayesian or frequentist formulation) of a loss function, ℓ, with our choice of π_d influencing the sequence of designs, $\xi_{1:T}$, and the new data, $d_{1:T}$, we observe.

Where the active-learning literature is most distinctive is in its explicit focus on accurately predicting the outputs, $y_{1:N} \in \mathcal{Y}^N$, corresponding to $N \geq 1$ target inputs, $x_{1:N} \in \mathcal{X}^N$, where the relationship between inputs and outputs is often very complex. Common prediction problems include image classification [569, 1489], text classification [1019, 1493], speech recognition [452, 1643] and molecular property-prediction [707, 1838]. In *inductive* problems $x_{1:N}$ is not known until after data-gathering, whereas in *transductive* problems it is known up front [1667].

Also notable is an emphasis on high-dimensional design spaces, Ξ_t, and data spaces, $\mathcal{D}_t$, where again we typically face a challenging prediction problem in mapping from designs to data. There are active-learning problems with low-dimensional spaces [60, 321, 322, 1106, 1107] and alternative experimental-design problems with higher-dimensional spaces [551, 903, 1849]. But the highest-dimensional spaces studied so far have overwhelmingly been within the active-learning literature. For example, it is common in active learning for ξ_t to represent a batch of images and d_t to represent those images combined with class labels [52, 54, 898, 900, 1489], such that Ξ_t is a continuous-valued vector space that can easily have millions of dimensions, and $\mathcal{D}_t$ is a discrete space of size exponential in the batch size.

While active learning is flexible in its notion of designs and data, for concreteness we will focus on a variant of that common image-classification case in which each design, $\xi_t \in \mathcal{X}^B$, is a batch of $B \geq 1$ inputs (not necessarily images) for labelling and each unit of data, $d_t \in \mathcal{Y}^B$, is a corresponding batch of input-label pairs (not necessarily with categorical labels), where these inputs and labels exist in the same spaces as the target inputs and outputs. Often there will be some initial data, d_0, which can comprise just labelled data, just unlabelled data, or a mixture. It is typical in the active-learning literature for d_0 to contain a discrete set of unlabelled inputs known as a *pool* [1019, 1494], which we denote as $\mathcal{X}_{\text{pool}} \subseteq \mathcal{X}$. In the case of transductive learning, another form of unlabelled data known ahead of data-gathering is $x_{1:N}$.

Whereas handling high-dimensional design and data spaces is a practical difficulty that is increasingly addressable in other experimental-design settings [1459], the focus on challenging predictions stands to remain a distinctive

[10] See Rainforth et al. [1370] for a deeper decision-theoretic treatment of active learning.

feature of active learning as a problem class. In principle the scope of active learning could be expanded to include alternative goals, to the point of it being synonymous with experimental design. But maintaining a tighter scope—one that reflects the de-facto definition of active learning in the literature—allows us to more efficiently communicate substantive differences in emphasis between communities.

Models

The nature of active learning as a problem helps explain some salient features of active-learning methods in relation to alternative experimental-design methods. One such feature is how we construct models. The emphasis on accuracy in challenging prediction problems typically requires us to forgo conventional statistical-modelling approaches in favour of generic *predictive algorithms*, crossing a historical gap between machine learning and statistics [206] that is arguably only now starting to meaningfully reduce [538, 540]. These predictive algorithms often produce more accurate models while also posing relatively low computational costs, albeit with the notable caveats that they can struggle in the low-data regimes commonly associated with active learning [921] and that they generally do not produce coherent belief updates [501, 1329].

In some cases the models we construct will contain stochastic parameters, $\theta \sim p(\theta; d)$. These are often defined such that predictions are independent conditioned on θ:

$$p(y_{1:N}; x_{1:N}, d) = \mathbb{E}_{p(\theta;d)}\left[\prod_{n=1}^{N} p(y_n|\theta; x_n, d)\right]. \tag{53.31}$$

This setup encompasses most conventional Bayesian models (following standard arguments; [383]), for which $p(\theta; d) = p(\theta|d) \propto p(d, \theta)$ is an exact Bayesian posterior derived from an explicit joint model, $p(d, \theta)$, and $p(y_n|\theta; x_n, d) = p(y_n|\theta; x_n)$ is a fixed sampling distribution. It also includes ensemble models [426] and stochastic neural networks [1501].

Another factor in how we construct models is whether we have a mixture of labelled and unlabelled data. For example, if d_0 contains unlabelled inputs and $d_{1:t}$ contains input-label pairs then $p(\,\cdot\,; d_{0:t})$ could be a *semi-supervised* model that incorporates information from both the labelled and unlabelled data. Early work along these lines used a range of models, including ensemble [1643], Gaussian-mixture [1237], Gaussian-random-field [1879], n-gram [1396] and naive-Bayes [1156, 1217] models. More recent work has tended to focus on deep-learning models that combine an encoder and a prediction head, where the encoder is pretrained on unlabelled data and then either the whole model or just the prediction head is trained on labelled data [55, 155, 698, 1120, 1130, 1194, 1271, 1502, 1587, 1815, 1827].

In addition to a model of outputs given inputs, for some methods we need a model of the unknown target inputs. This could be a discrete distribution over the pool, $\mathcal{X}_{\text{pool}}$ (if it exists); a model trained on $\mathcal{X}_{\text{pool}}$; or an analytic distribution over the input space [154, 1156, 1429].

Data-gathering

Aside from the models and action policies they use, active-learning methods are distinctive in how they typically select designs, ξ_t, even though there are no restrictions on this in principle (i.e. within the problem definition). First, whereas in 53.2 we showed that optimal data-gathering needs to plan ahead based on full simulations of $(\xi_t, d_t)_{t=1}^T$, the vast majority of active-learning methods choose a single design at a time and treat each design decision as if it were the last. This *myopic* approach (matching that discussed in 53.3.3 except that design selection does not necessarily involve optimisation) reduces the difficulty of forecasting future data and the resulting model updates, which is particularly challenging in active learning due to the complex design–data relationships and flexible predictive algorithms involved. Nonmyopic approaches remain rare but are beginning to receive more attention; Huang et al. [794] recently presented a successful demonstration.

Second, even if ξ_t is an element of a continuous design space, Ξ_t, it is common to construct a discrete design space, $\Xi_{\text{pool}} = \mathcal{X}_{\text{pool}}^B$, based on a pool of unlabelled inputs, $\mathcal{X}_{\text{pool}}$, available (or generated) at the start of data-gathering (53.4.1), and then to select ξ_t from Ξ_{pool}. This contrasts with *query synthesis* [1494, 1652], in which ξ_t is generated from scratch. Pool-based design selection can help filter out pathological designs: if ξ_t represents an image for a human to label, we might want to avoid selecting images that are hard to label [972, 1459], and we can do that by using a pool of "natural" images.

Third, the selection of ξ_t from Ξ_{pool} does not always take the form of a straightforward optimisation. Many methods formulate an *acquisition objective* that can be evaluated on any candidate input batch, ξ, and in principle we can evaluate this objective on all $\xi \in \Xi_{\text{pool}}$ before selecting the ξ with the best value (lowest for some methods; highest for others). While this exact-optimisation approach is typical when ξ_t represents $B = 1$ input, it becomes

costly for selecting larger batches of $B > 1$ inputs. Some methods instead approximate the optimisation over Ξ_{pool} by greedily constructing ξ_t, namely choosing the best first input, the best second input given the chosen first input, and so on [52, 898, 1489]. Others use stochastic sampling of ξ_t, for example using a Boltzmann distribution derived from the acquisition objective [506, 835, 900, 1837], by top-B selection based on a two-sample Monte Carlo estimate of the objective [834], or by Thompson sampling [842]. There are also methods that do not formulate an objective at all, such as BADGE [54], a method we cover later.

53.4.2 Data-gathering methods

Here we present some of the most prominent data-gathering methods in the literature. Reflecting the dominance of myopic approaches, we consider selecting a single design, ξ_t, rather than a design policy, π_d. For notational simplicity we also default to not indexing by step, t, and leaving the dependence of models on the current data, $d_{0:t-1}$, implicit.

Information-based methods

The idea of maximising EIG (53.3.1) in active learning, which was pioneered by MacKay [1107] and later became known as *Bayesian active learning*, has been very influential within the field. Multiple forms of EIG have been proposed, each corresponding to an expected reduction in Shannon entropy in a chosen variable.

Parameter EIG MacKay [1107] studied four forms of EIG. The form that went on to receive the most attention is what he called the expected total information gain (ETIG), now widely known as the BALD score following work on "Bayesian active learning by disagreement" [784, 785]. The BALD score for a candidate design, ξ, is the expected information gain in a set of stochastic model parameters, $\theta \sim p(\theta)$, from observing the data, d, produced by ξ:

$$\begin{aligned} \text{BALD}(\xi) &= \text{EIG}_\theta(\xi) && (53.32)\\ &= \text{KL}[p(d, \theta; \xi) \,\|\, p(d; \xi)p(\theta)]\,. && (53.33) \end{aligned}$$

MacKay [1107] proposed an estimator of this objective for regression with a stochastic neural network, Houlsby et al. [784, 785] an estimator for classification with a Gaussian-process model and Gal et al. [569] an estimator for classification with a stochastic neural network; the last of these was also concurrently proposed in the BED literature [1365, 1367]. Later Kirsch et al. [898] proposed BatchBALD, a greedy scheme for approximately optimising $\text{BALD}(\xi)$ over $\Xi_{\text{pool}} = \mathcal{X}_{\text{pool}}^B$ for $B > 1$.

Region-of-uncertainty sampling [60, 321] and query by committee [1496] can be understood as instances of BALD-based acquisition with a particular form of model whose predictions of d are deterministic given ξ and θ [559]. The former randomly samples (not necessarily uniformly) from the set of candidate inputs with nonzero BALD score, while the latter maximises BALD.

Transductive predictive EIGs MacKay [1107] also considered three objectives applicable to transductive active learning, where the target inputs are known up front: what he called the expected marginal information gain (EMIG), expected joint information gain (EJIG) and expected mean marginal information gain (EMMIG). The first of these is the EIG in the output, y, corresponding to a known target input, x, from observing d:

$$\begin{aligned} \text{EMIG}(\xi, x) &= \text{EIG}_y(\xi, x) && (53.34)\\ &= \text{KL}[p(d, y; \xi, x) \,\|\, p(d; \xi)p(y; x)]\,. && (53.35) \end{aligned}$$

EJIG and EMMIG extend this to a set of $N > 1$ known target inputs. EJIG has the same form as EMIG but replaces (x, y) with $(x_{1:N}, y_{1:N})$. Meanwhile EMMIG is a weighted sum of the EMIG values across $x_{1:N}$, using weights $w_{1:N} \in \mathbb{R}_+^N$:

$$\text{EMMIG}(\xi, x_{1:N}, w_{1:N}) = \sum_{n=1}^{N} \bar{w}_n \text{EMIG}(\xi, x_n) \tag{53.36}$$

where $\bar{w}_n = w_n / \sum_{n'=1}^{N} w_{n'}$ is a normalised version of the weight on x_n. MacKay [1107] proposed estimators of these three predictive EIGs for regression with a stochastic neural network; MacKay [1106] added EMIG and EMMIG estimators for classification.

While EJIG was used by for example Guestrin et al. [681], Kirsch et al. [899] and Krause et al. [925], and EMMIG by MacKay [1106] and Wang et al. [1702], they have otherwise been relatively neglected in the active-learning literature. BALD is much more often used to represent Bayesian active learning in empirical investigations.

General predictive EIG Bickford Smith et al. [154] later re-emphasised the idea of maximising EIG in predictions but also importantly extended beyond the transductive setting, allowing for cases where the target inputs are not known ahead of time. Their proposed objective, the expected predictive information gain (EPIG), is the EIG in an unknown target input-output pair, (x, y), from observing d:

$$\begin{aligned}\mathrm{EPIG}(\xi) &= \mathrm{EIG}_{(x,y)}(\xi) && (53.37)\\ &= \mathrm{KL}[p(x)p(d,y;\xi,x) \,\|\, p(d;\xi)p(y;x)] && (53.38)\\ &= \mathbb{E}_{p(x)}[\mathrm{EMIG}(\xi,x)]\,. && (53.39)\end{aligned}$$

In the general case, $p(x)$ is a belief distribution over unknown target inputs, the construction of which is flexible. This belief distribution can also be specialised to recover EMIG and EMMIG: for the former we define it to be a point mass on the single known target input, x; for the latter we define it to be a discrete distribution over $x_{1:N}$, with probability masses $\bar{w}_{1:N}$.

Variance-based methods

Whereas maximising a predictive EIG corresponds to a focus on Shannon entropy in target outputs, another popular class of methods focuses on variance in those outputs. A frequentist-style method proposed by Cohn et al. [323] has drawn the most attention in the active-learning literature, but also notable are earlier Bayesian-style methods.

Bayesian-style methods A pair of methods follows the Bayesian approach of aiming for maximal uncertainty reduction or minimal future uncertainty (53.2.2), except that they use a generically updated belief distribution in place of the posteriors used in Bayesian objectives. This is a key distinction between these methods and information-based methods, in addition to the use of variance rather than Shannon entropy.

Cohn [320] proposed maximising a form of expected variance reduction[11] (EVR):

$$\mathrm{EVR}(\xi) = \mathbb{E}_{p(x)p(d;\xi)}\left[\mathbb{V}_{p(y;x)}[y] - \mathbb{V}_{p(y;x,d)}[y]\right]. \tag{53.40}$$

Noting that $\mathbb{V}_{p(y;x)}[y]$ does not vary with ξ, an equivalent approach is to minimise the form of expected future variance[12] (EFV) proposed by Cohn et al. [322]:

$$\mathrm{EFV}(\xi) = \mathbb{E}_{p(x)p(d;\xi)}\left[\mathbb{V}_{p(y;x,d)}[y]\right]. \tag{53.41}$$

Frequentist-style method The alternative form of EFV proposed by Cohn et al. [323] follows the frequentist approach of averaging over existing data as well as future data. In particular it takes any inputs, $x_{d_{<t}}$, whose labels, $y_{d_{<t}}$, are already known (i.e. within $d_{<t} = d_{0:t-1}$) and averages over alternative labels, $y'_{d_{<t}}$, for those inputs, which we refer to as label resampling (LR):

$$\mathrm{EFV\text{-}LR}(\xi_t, x_{d_{<t}}, z) = \mathbb{E}_{p(x;z)}\left[\mathbb{V}_{p(d_t;\xi_t,z)p(y'_{d_{<t}};x_{d_{<t}},z)}\left[f(x;(x_{d_{<t}}, y'_{d_{<t}}, d_t))\right]\right] \tag{53.42}$$

where $f(x;d)$ denotes a deterministic predictive model trained on data d. This objective is derived using a frequentist-risk formulation along with assumptions on data-gathering and model training, hence its explicit dependence on an index over data-generating distributions, z [1370].

Other uncertainty-based methods

All of the methods we have introduced so far depend on some notion of future uncertainty in either a model's parameters or its predictions. Another set of methods is based on *current uncertainty in the data we are acquiring*

[11] Cohn [320] used a model for which $\mathbb{V}_{p(y;x,d)}[y]$ only depends on the input component of d but not the label component. We present an objective that covers a more general model class.

[12] Both Cohn et al. [322] and Cohn et al. [323] were ambiguous in their definitions of predictive variance, so we present our best-guess interpretation of their mathematical expressions. We also relax their assumption of knowing the true distribution over target inputs, using a model in its place.

as a function of the design, ξ. In entropy sampling [1493] we maximise

$$\text{Entropy}(\xi) = \text{H}[p(d;\xi)]\,, \tag{53.43}$$

while in least-confidence sampling [1493] we minimise

$$\text{Confidence}(\xi) = \max_{d\in\mathcal{D}} p(d;\xi) \tag{53.44}$$

and in margin sampling [1467] we minimise

$$\text{Margin}(\xi) = p(d = d^{(1)};\xi) - p(d = d^{(2)};\xi) \tag{53.45}$$

where $d^{(1)} = \arg\max_{d'\in\mathcal{D}} p(d';\xi)$ and $d^{(2)} = \arg\max_{d'\in\mathcal{D}\setminus\{d^{(1)}\}} p(d';\xi)$.

Earlier use of this general approach includes the work of Lewis and Catlett [1018] and Lewis and Gale [1019], who focused on binary classification (for which these methods are equivalent in their ranking of designs) and on batch acquisition (selecting $B > 1$ inputs at a time). To select a batch, they first used their model to assign binary pseudolabels to the individual candidate inputs, then split the candidate inputs into two sets based on these pseudolabels, and finally selected the $B/2$ highest-uncertainty inputs from each set.

As noted in 53.3.1, these methods are hard to motivate from a decision-theoretic perspective, regardless of whether $p(d;\xi)$ is derived from a Bayesian model or not. Perhaps the closest we can get is to recognise that entropy sampling is equivalent to maximising the EIG in some variable, ψ, if the model's predictions of d given ξ and ψ have equal entropy for all (ξ,ψ) pairs. In this special case we have

$$\text{EIG}_\psi(\xi) = \text{H}[p(d;\xi)] - \mathbb{E}_{p(\psi)}[\text{H}[p(d|\psi;\xi)]] = \text{Entropy}(\xi) + \text{constant} \tag{53.46}$$

where $p(d;\xi) = \mathbb{E}_{p(\psi)}[p(d|\psi;\xi)]$. Setting aside the strong restriction on the model, there is generally no computational advantage to entropy sampling when it comes to Monte Carlo estimation of the objective: each instance of $p(d|\psi;\xi)$ with some sampled ψ can be used for both terms in this writing of the EIG, so the number of instantiations is the same regardless of whether we use the EIG or the entropy [300].

Error-based methods

Pushing back against methods including query by committee (53.4.2) and least-confidence sampling (53.4.2), Roy and McCallum [1429] advocated focusing on *predictive error*. Each of their two proposed objectives (to be minimised) is a form of expected future error (EFE) in classifying inputs from a given pool, $\mathcal{X}_{\text{pool}}$:

$$\text{EFE}(\xi,\mathcal{X}_{\text{pool}}) = \frac{1}{|\mathcal{X}_{\text{pool}}|}\sum_{x\in\mathcal{X}_{\text{pool}}} \mathbb{E}_{p(d;\xi)}[\ell_y(p(y;x,d),p(y;x,d))] \tag{53.47}$$

where $\ell_y : \mathcal{P}(\mathcal{Y})\times\mathcal{P}(\mathcal{Y}) \to \mathbb{R}$ is a loss measuring the mismatch between a predictive distribution and a reference distribution, and $p(y;x,d)$ is a predictive distribution over outputs given x and d (which, as in 53.4.2, is not necessarily produced by a Bayesian update on d). They highlighted two key steps in constructing this objective: the use of an empirical average over $x \in \mathcal{X}_{\text{pool}}$ to approximate an expectation over unknown target inputs; and the use of $p(y;x,d)$ in place of an external reference distribution.

They instantiated this general form of EFE with two different losses. Defining $\ell_y^{\log}(q,p) = \mathbb{E}_{p(y)}[-\log q(y)]$ and substituting this into the general form gives

$$\text{EFE}^{\log}(\xi,\mathcal{X}_{\text{pool}}) = \frac{1}{|\mathcal{X}_{\text{pool}}|}\sum_{x\in\mathcal{X}_{\text{pool}}} \mathbb{E}_{p(d;\xi)}[\text{H}[p(y;x,d)]]\,, \tag{53.48}$$

while using $\ell_y^{0/1}(q,p) = \mathbb{E}_{p(y)}\big[\mathbb{I}(y \neq \arg\max_{y'\in\mathcal{Y}} q(y'))\big]$ gives

$$\text{EFE}^{0/1}(\xi,\mathcal{X}_{\text{pool}}) = \frac{1}{|\mathcal{X}_{\text{pool}}|}\sum_{x\in\mathcal{X}_{\text{pool}}} \mathbb{E}_{p(d;\xi)}\left[1 - \max_{y\in\mathcal{Y}} p(y;x,d)\right]. \tag{53.49}$$

Although they did not explicitly aim to target future uncertainty in their approach, both of these objectives are measures of expected uncertainty in $p(y;x,d)$, with uncertainty measured by Shannon entropy in the log-loss case and by a confidence-based measure (53.4.2) in the zero-one-loss case. As with the Bayesian-style variance-based objectives presented in 53.4.2, the use of generic belief updating in the definition of these EFE objectives sets them apart from EIG objectives in the general case.

In later work Zhu et al. [1879] used a transductive variant of the zero-one EFE objective, replacing $\mathcal{X}_{\text{pool}}$ with a separate set of known target inputs. Mussmann et al. [1218] similarly used separate target inputs in their EFE objectives but also deviated from the original EFE objectives in specifying $p(y;x,d) = p(y|d;x) \propto p(d,y;x)$ to be the result of a Bayesian update derived from an explicit joint model.

Coverage-based methods

Recent years have seen considerable interest in a style of approach distinct from those we have discussed so far, namely maximising some notion of *coverage* within a feature space, $\Phi = \{\phi(x) \mid x \in \mathcal{X}\}$, such as that of a deep neural network trained on $d_{0:t-1}$. These tend to be motivated by an aim to reduce frequentist risk, with theoretical results connecting this to coverage under strong assumptions on the setup, such as using a one-nearest-neighbour model on a deterministic classification problem [73, 1815].

Much of the interest in coverage-based methods is due to work by Sener and Savarese [1489]. Their "greedy k centres" method builds up a batch of B inputs, $\xi = x^+_{1:B} \in \mathcal{X}^B_{\text{pool}}$, through B iterations of selecting the new input, x^+_b, that maximises the minimal distance from inputs that are already labelled or due to be labelled:

$$\text{MinimalDistance}(x^+_b, \mathcal{X}_{\text{lab}}) = \min_{x \in \mathcal{X}_{\text{lab}}} \left\|\phi(x^+_b) - \phi(x)\right\|_2 \tag{53.50}$$

where $\mathcal{X}_{\text{lab}} = \{x \mid x \in (x_{d_{<t}}, x^+_{<b})\}$ with $x_{d_{<t}}$ denoting inputs in $d_{<t} = d_{0:t-1}$ whose labels are already known, and $x^+_{<b} = x^+_{1:b-1}$ denoting inputs already in the current batch.

More recent work has proposed other notions of coverage. MaxHerding [73] considers the similarity between each pool input and its most similar labelled or selected input, and selects the new input, x^+_b, that maximises the average of those similarities, where the similarity is defined by a real-valued function, such as a kernel, $k : \Phi \times \Phi \to \mathbb{R}_{\geq 0}$:

$$\text{KernelCoverage}(x^+_b, \mathcal{X}_{\text{lab}}, \mathcal{X}_{\text{pool}}, k) = \frac{1}{|\mathcal{X}_{\text{pool}}|} \sum_{x \in \mathcal{X}_{\text{pool}}} \max_{x_{\text{lab}} \in \mathcal{X}_{\text{lab}} \cup \{x^+_b\}} k(\phi(x), \phi(x_{\text{lab}})). \tag{53.51}$$

The intuitive goal is to construct a labelled dataset that closely matches the pool in feature space, corresponding to a high average similarity between pool inputs and labelled inputs. ProbCover [1815] is a special case of MaxHerding where the similarity function simply measures whether two inputs are within a specified distance, $\delta \in \mathbb{R}_+$, of each other in feature space, namely $k(\phi_1, \phi_2) = \mathbb{I}(\|\phi_1 - \phi_2\|_2 \leq \delta)$.

Another related method is TypiClust [698], which is motivated by the idea of seeking inputs that are diverse but also representative of the pool. It uses clustering to promote diversity, and a maximisation of a typicality score to promote representativeness. More specifically it splits $\mathcal{X}_{\text{pool}}$ into clusters then performs B iterations of selecting a cluster (minimising the number of labelled inputs; splitting ties by maximising cluster size) and within the cluster, $\mathcal{X}_{\text{clust}}$, selecting the input, x^+_b, that maximises "typicality" within a neighbourhood of size $1 \leq k \leq |\mathcal{X}_{\text{clust}}|$:

$$\text{Typicality}(x^+_b, \mathcal{X}_{\text{clust}}, k) = \frac{k}{\sum_{x \in k\text{-NN}_\phi(x^+_b, \mathcal{X}_{\text{clust}})} \left\|\phi(x^+_b) - \phi(x)\right\|_2} \tag{53.52}$$

where k-$\text{NN}_\phi(x^+_b, \mathcal{X}_{\text{clust}})$ denotes the k nearest neighbours to x^+_b within $\mathcal{X}_{\text{clust}}$, which are found by sorting $x \in \mathcal{X}_{\text{clust}}$ by $\left\|\phi(x^+_b) - \phi(x)\right\|_2$.

Gradient-based methods

Also notable for their recent popularity are methods based on *gradients* of losses with respect to model parameters, which have been put forward with a range of motivations. Settles et al. [1495] proposed seeking inputs that are expected to most change the parameters, ψ, of a model as the result of a gradient-based update on the labelled input. They defined the expected gradient length (EGL) for a candidate input, x^+, as

$$\text{EGL}(x^+) = \mathbb{E}_{p(y^+; x^+, \psi)} \left[\left\| \text{vec}\left(\frac{\partial}{\partial \psi} \ell_y(p_{y^+}(\,\cdot\,; x^+, \psi), y^+) \right) \right\|_2 \right] \tag{53.53}$$

where $\text{vec}(\cdot)$ denotes vectorisation (i.e. flattening) and $\ell_y : \mathcal{P}(\mathcal{Y}) \times \mathcal{Y} \to \mathbb{R}$ is a loss function on probabilistic predictions. They suggested maximising this objective to select a single input, but did not propose a scheme for selecting a batch of $B > 1$ inputs at once.

Ash et al. [54] later paid particular attention to batch acquisition for neural networks, seeking a way to select multiple high-uncertainty inputs (as in 53.4.2) that satisfy some notion of diversity. Similar to Settles et al. [1495], they used gradients of a loss function evaluated with model-generated pseudolabels, with the idea of capturing potential model change. But their desire for diversity led them to use these gradients in a different way, namely forming so-called gradient embeddings of inputs and then passing these to k-means++ [50], an algorithm designed for choosing the initial cluster centres in k-means clustering. More specifically their method, BADGE ("batch

active learning by diverse gradient embeddings"), selects a batch of B inputs by running k-means++ with $k = B$ on a set of embeddings, $\mathcal{E}$, defined as

$$\mathcal{E} = \left\{ \text{vec}\left(-\frac{\partial}{\partial \psi'_L} \log p(y = y^{(1)}; x, \psi) \right) \,\middle|\, x \in \mathcal{X}_{\text{pool}}, x \notin x_{d_{<t}} \right\} \tag{53.54}$$

where $\psi = \psi'_{1:L}$ parameterises an L-layer neural network, $y^{(1)} = \arg\max_{y \in \mathcal{Y}} p(y; x, \psi)$ is the output assigned highest weight by the neural network for input x, and $x_{d_{<t}}$ are the inputs in $d_{<t} = d_{0:t-1}$ whose labels are already known.

Ash et al. [52] later drew a connection between the embeddings used in BADGE and the Fisher information matrix of the neural network at x,

$$F(x, \psi) = \mathbb{E}_{p(y;x,\psi)} \left[\left(\frac{\partial}{\partial \psi} \log p(y; x, \psi) \right) \left(\frac{\partial}{\partial \psi} \log p(y; x, \psi) \right)^\top \right], \tag{53.55}$$

and argued that a more principled approach is to minimise an objective sometimes known as the Fisher-information ratio (FIR; [1493, 1494]):

$$\text{FIR}(\xi, x_{1:N}, \psi) = \text{Tr}\big(F(x_{1:N}, \psi)(F(x_{d_{<t}}, \psi) + F(\xi, \psi))^{-1} \big) \tag{53.56}$$

where $x_{1:N}$ is a known set of target inputs and we note that $F(x'_{1:N}, \psi) = \sum_{n=1}^{N} F(x'_n, \psi)$. The use of known target inputs makes this a transductive objective. A non-transductive form, replacing $F(x_{1:N}, \psi)$ with an expectation of $F(x, \psi)$ with respect to an unknown target input, x, was proposed by Fukumizu and Watanabe [562]. Both forms of the objective have been studied elsewhere in the literature [563, 767, 1468, 1493, 1859], with some work highlighting a connection between FIR and a form of frequentist risk.

53.4.3 Methodological considerations

Finally, having covered a range of existing active-learning methods, we now highlight some important issues relating to their use and to the development of new methods.

Targeting predictions A key factor of variation across the methods in 53.4.2 is how directly they target the predictions of interest. One notable case is BALD, which targets uncertainty in a set of stochastic model parameters (53.4.2), not uncertainty in the model's predictions directly, and can thus be of limited use in improving predictive performance [154, 155, 559, 1107]. In other cases the predictive focus is present in the high-level motivation for the method, but the concrete method is not directly derived from a mathematical formulation of that motivation (e.g. coverage-based methods). This is an important consideration when assessing existing methods and designing new ones. A clear connection to the decision problem of interest is a necessary condition for a method to be principled, although it is also not sufficient for good performance in practice.

Belief updating In 53.3.1 we highlighted that principled uncertainty-based objectives are critically expressed in terms of *future* beliefs after updating on new data; and in 53.4.2 we noted that EIGs assume those beliefs are derived from the joint model underlying the EIG, while variance-based and error-based objectives assume the beliefs are produced by a generic procedure (e.g. re-running a training algorithm). Interestingly, while the Bayesian-update assumption is theoretically principled (and can be applied to cases beyond targeting Shannon entropy), the way beliefs are actually updated in practical active learning warrants attention to the assumption. For example, it is standard practice in information-based active learning to produce final predictions after data-gathering using a generic predictive algorithm [154, 155, 569, 784, 785, 898, 900, 1106, 1107]. There is thus a mismatch between the belief update assumed during data-gathering and the belief update used after data-gathering. At the same time, information-based methods often perform well in empirical investigations, and currently the literature lacks empirical evidence on the influence the mismatch has on performance. This warrants further investigation.

Constructing accurate models Many of the methods we covered in 53.4.2 are based on probabilistic models: models of the target inputs we will make predictions on; models of the future data we will gather; and, in frequentist approaches, models of the data we already have. All of these models are subjective and in the general case will not match reality [156]. The extent of that mismatch is a critical determinant of how well the methods perform

in practice. Using more accurate models of course improves baseline performance, but it can also make data-gathering methods more effective. For example, switching from a fully supervised to a semi-supervised model that incorporates unlabelled data (53.4.1) has been shown to significantly improve the effectiveness of information-based methods [155]. Whether it is through semi-supervised learning or not, any approach that allows us to construct better models has the potential to benefit data-gathering. Designing predictive algorithms that work well in low-data regimes is thus a research priority.

An important factor in designing such predictive algorithms is that elements of an actively acquired data sequence are not independent and identically distributed, leading to statistical bias in models produced by standard empirical-risk minimisation [147, 304, 373, 571, 1107, 1343]. Reweighted risk estimators can be used to prevent this bias in the context of pool-based active learning, but interestingly these do not always lead to better predictions, with the bias actually helping in some cases [506].

53.5 Conclusion

We opened this chapter by highlighting the enormous value that could be unlocked by improving data-gathering methods. To that end, we have presented context and conceptual tools that we hope will accelerate progress. Our approach has been to emphasise the framework of decision theory, which enables us to directly target what we care about, and which fundamentally motivates the use of probabilistic methods. Recognising the primacy of decision-making—and moving beyond a narrow focus on inference, which cannot be decoupled from decision-making in anything but the simplest of problems [956]—is crucial if the field of Bayesian deep learning is to achieve the practical relevance and mainstream impact it aspires to [1292].

In using decision theory to explore notions of optimality in data-gathering, we have demonstrated deep connections between loss functions (or corresponding utility functions), proper scoring rules, and generalised forms of entropy and information gain. We have also drawn attention to a crucial insight: while the popular approach of minimising expected posterior uncertainty is theoretically optimal, so too is an approach that avoids posteriors entirely and that also naturally targets downstream decision-making [1424]. Thus we have seen the value of a foundational perspective to make sense of a wide array of ideas but also to directly influence the design of practical methods.

Challenges remain in developing methods ready for widespread deployment. One of these is dealing with *model misspecification*, in which no configuration of the model we are using can recover the true data-generating process. While this has received some consideration with respect to experimental design [2, 71, 98, 227, 513, 546, 641, 1280, 1572, 1590], it remains drastically understudied given the prevalence and impact of model misspecification in practice.

A second challenge, solving which will rely on addressing model misspecification, is scaling up from simple proof-of-concept demonstrations to complex problems, such as medical decision-making or supporting creative work, where the spaces of designs and data are high-dimensional and especially hard to map between. This requires innovation to combine two threads of work: that which has demonstrated more sophisticated methods [552] and that which has tackled harder problems [300].

One complex problem that warrants special attention is *active testing* [922, 923, 1464], in which the goal is to evaluate the behaviour of a model, conventionally in terms of its predictive performance. While progress has been made in scaling up active testing to large models [132] and providing confidence intervals on performance estimates [56], there is more work to be done to handle more complex notions of model behaviour than the basic measures of predictive performance that have been the focus of most prior work.

Another problem that we have not directly covered in this chapter is *Bayesian optimisation* [584], in which the goal is to produce an action that optimises an unknown objective function. Bayesian optimisation is like active learning in being a particular form of experimental design around which an effectively separate community and literature has grown; in this case its distinctive features include widespread use of Gaussian-process models. There are thus parallel threads of work and inevitably cases where one field's solutions could be useful in another. Efforts to identify such cases and translate ideas between fields could be fruitful.

There is therefore a considerable amount of work to be done. At the same time, we find ourselves increasingly well equipped to make progress. Ever-improving algorithms [450, 1212, 1213, 1632] let us construct more capable models, and ever-cheaper computation [340, 494, 1249] lets us simulate at greater scale. Poorly designed methods will squander the dividends these trends yield, so we cannot stick with what we have and expect our problems to be solved. If instead we pursue the right ideas, a great reward is within reach.

Chapter 54

Causal-aware BDL

54.1 Introduction

Bayesian deep learning (BDL) equips flexible neural models with a coherent probabilistic treatment of parameters, latent variables, and predictions, as discussed in previous chapters. Combined with deep neural networks, BDL has achieved remarkable success in recognising and generating images, learning representations, and translating languages by exploiting statistical correlations in data. However, many such models remain primarily associational: they learn what tends to *co-occur*, rather than what *produces* what. This distinction becomes critical when the environment changes, when a model is used to support decisions, or when we ask hypothetical questions about interventions. Correlations describe what is observed; causality aims to describe what would happen under changes to the system.

Causality therefore focuses on the mechanisms that generate data. It asks not only whether two variables move together, but whether one *causes* the other and how that influence would change under different interventions. For example, does a new drug improve recovery, or are healthier patients simply more likely to receive it? In practice, causal analysis involves two intertwined tasks. The first is *causal discovery*: learning the underlying cause–effect structure among variables from the data or active experimentation. The second is *causal inference*: estimating the effect of an intervention once a causal structure is known, hypothesised, or partially identified. In essence, causal discovery asks "who causes whom", while causal inference asks "by how much" and "what would happen if we intervened".

To make these questions mathematically precise, causal reasoning relies on *structural equation models* (SEMs), sometimes called *structural causal models* (SCMs). An SEM describes how each variable is generated as a function of its direct causes together with an independent source of noise. This formulation encodes the idea that a system is composed of modular mechanisms: interventions modify one mechanism while leaving the others unchanged. SEMs therefore provide a bridge between observed data and hypothetical reasoning, making interventions and counterfactuals mathematically coherent.

Causal graphs provide a complementary language for representing the cause–effect structure. A causal graph consists of nodes, representing variables, and directed edges, representing direct causal relationships. Although cyclic relations can arise in certain dynamical systems, this chapter primarily focuses on *directed acyclic graphs* (DAGs), which are the standard setting for the methods considered here. Graphs also provide a powerful language for inference: they encode conditional independencies, reveal possible confounding pathways, and help determine when causal effects can be identified from observational data.

Seen from this perspective, causality is not an alternative to probabilistic modelling, but a natural extension of it. In this view, Bayesian methods become particularly useful. They offer principled tools for quantifying uncertainty over parameters, causal structures, and variables, while causal modelling provides the language for reasoning about interventions and changes in the underlying system. Combining the two yields a framework for reasoning under uncertainty about causal mechanisms themselves: we can express prior knowledge about possible causal structures, update it with data, and propagate uncertainty into downstream causal estimates.

In this chapter, we illustrate how the tools from Bayesian deep learning can be combined with causal reasoning for both static and time-series data. We first introduce the basic causal language used throughout the chapter, including causal graphs, SEMs, causal estimands, and identifiability. We then describe two representative methods for static data: *DECI* and *BayesDAG*. Next, we move to the temporal domain, introducing *Rhino* for discrete-time data and *SCOTCH* for continuous-time data. Finally, we discuss fixed-point causal models and foundation-model approaches to causal inference.

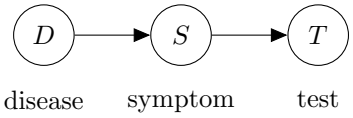


Figure 54.1: A DAG for $D \to S \to T$. The Markov property implies $D \perp T \mid S$.

54.2 Basics for causal machine learning

54.2.1 Basics and notations

Causal modelling seeks to understand and reason about how changes in one part of a system influence another. Causal analysis goes further, asking what would happen to a variable Y if we were to *intervene* on another variable X.

This distinction between observation and intervention lies at the heart of causality [1307, 1318].

Three levels of questions. Following Pearl [1307], it is helpful to distinguish three kinds of questions:

1. *Associational:* What relationships are present in the data, e.g. $P(Y \mid X)$?
2. *Interventional:* What would happen under an external manipulation, e.g. $P(Y \mid \mathrm{do}(X{=}x))$?
3. *Counterfactual:* What would have happened had something been different for a specific instance?

The first can be answered from observational data alone; the latter two require additional causal assumptions, often represented by a graph.

Variables and notation. Here, we focus primarily on the static-data regime; most concepts extend naturally to temporal settings. Let $\boldsymbol{X} = (X_1, \ldots, X_D)$ denote D random variables of interest, jointly distributed according to $p(\boldsymbol{X})$. We write lowercase letters x_i for realisations of X_i. A bold symbol $\boldsymbol{x}$ denotes a vector of all variables, and for a subset $\mathcal{S} \subseteq \{1, \ldots, D\}$ we use $X_{\mathcal{S}} = \{X_i : i \in \mathcal{S}\}$ and $x_{\mathcal{S}}$ analogously.

Graphs as causal structure. A *directed graph* $\boldsymbol{G} = (V, E)$ with node set $V = \{1, \ldots, D\}$ and directed edges $E \subseteq V \times V$ is used to encode causal relations between variables. An edge $i \to j$ means that X_i is a *cause* of X_j. For each node i, the *parents* and *children* sets are

$$\mathbf{Pa}(i) = \{\, j \in V : (j, i) \in E \,\}, \qquad \mathbf{Ch}(i) = \{\, j \in V : (i, j) \in E \,\}.$$

Thus, the parents of node i are precisely the nodes j with edges $j \to i$. We denote the *ancestors* $\mathbf{An}(i)$ as all nodes from which a directed path leads to i, and the *descendants* $\mathbf{De}(i)$ as all nodes reachable from i via directed paths. When the graph has no directed cycles (i.e. no node is its own ancestor), it is called a *directed acyclic graph* (DAG). Throughout this chapter, we assume $\boldsymbol{G}$ is a DAG. An adjacency matrix encodes the causal graph: for a binary square matrix $\boldsymbol{G}$, $\boldsymbol{G}_{ij} = 1$ denotes $i \to j$, whereas $\boldsymbol{G}_{ij} = 0$ denotes the absence of that edge. In some cases, $\boldsymbol{G}$ can also be real-valued, representing the weight of the associated directed edge.

Paths and separation. A *path* in $\boldsymbol{G}$ is a sequence of distinct nodes connected by edges, regardless of their orientation. A triple of nodes (A, B, C) can form a *chain* $A \to B \to C$, a *fork* $A \leftarrow B \to C$, or a *collider* $A \to B \leftarrow C$, where B is the collider. Conditioning on a non-collider blocks a path, whereas conditioning on a collider or one of its descendants can open it. The concept of *d-separation* formalises when a set of variables Z blocks every path between X and Y. We write $X \perp_{\boldsymbol{G}} Y \mid Z$ for this graphical relation; under the causal Markov property, it implies the corresponding conditional independence in the induced distribution.

From graphs to probability. Graphs alone are qualitative; to reason quantitatively, we link them to probability distributions that respect the graph's structure. Roughly, each node depends only on its parents, and the joint distribution factorises accordingly:

$$p(x_1, \ldots, x_D) \;=\; \prod_{i=1}^{D} p\big(x_i \mid x_{\mathbf{Pa}(i)}\big).$$

This factorisation will later arise naturally from the *structural equation models* (SEMs) introduced in the next section and is formalised by the *causal Markov property*. The key takeaway is that a DAG represents how probabilistic dependencies arise from an underlying system of cause–effect mechanisms.

Running example. Figure 54.1 shows the running example $D \to S \to T$. Under faithfulness, D and T are marginally dependent; the Markov property implies that they become conditionally independent given S, because conditioning on S blocks the path between them.

This graphical language provides the foundation for all subsequent sections: structural equations will make the above factorisation explicit, interventions will modify parts of it, and causal discovery will aim to recover $\boldsymbol{G}$ from data.

54.2.2 Structural Equation Model

In probabilistic modelling, we often describe data generation by specifying priors and likelihoods for latent variables. A *structural equation model* (SEM) plays an analogous role for causality, but its primitives are explicit *cause–effect mechanisms*. Each variable is expressed as a deterministic function of its direct causes, together with an exogenous noise term that accounts for unexplained variation. The collection of these mechanisms jointly defines how the system generates observations.

Definition 54.1 (Structural Equation Model)**.** *Let $\boldsymbol{X} = (X_1, \dots, X_D)$ denote a set of random variables. A structural equation model consists of a family of measurable functions $\{f_i\}_{i=1}^{D}$ and jointly independent noise variables $\{U_i\}_{i=1}^{D}$ such that*

$$X_i := f_i\big(X_{\mathbf{Pa}(i)}, U_i\big), \qquad i = 1, \dots, D, \tag{54.1}$$

where $\mathbf{Pa}(i) \subseteq \{1, \dots, D\} \setminus \{i\}$ denotes the parents *(direct causes) of X_i. The directed edges $j \to i$ induced by the parent sets form a graph $\boldsymbol{G}$ on nodes $\{1, \dots, D\}$, called the* causal graph*. When $\boldsymbol{G}$ contains no directed cycles, it is a* directed acyclic graph *(DAG).*

The exogenous variables U_i represent influences external to the system and are often referred to as *noise terms*. Given mutually independent $\{U_i\}$, Equation 54.1 defines a joint distribution over all variables. Restricting the causal graph to be a DAG offers many advantages that facilitate analysis. However, real-world systems often contain feedback: demand $\rightleftarrows$ price, predator $\rightleftarrows$ prey. A *cyclic* SEM allows directed cycles in $\boldsymbol{G}$. This can introduce challenges; for example, the system may have no solution or multiple equilibria without additional assumptions. Therefore, this chapter focuses primarily on the DAG setting. When the causal graph is a DAG, sampling proceeds recursively in topological order: one first samples the root nodes (those without parents) and then successively applies each mechanism f_i using already generated parent values.

This construction induces a joint density of the form

$$p(x_1, \dots, x_D) = \prod_{i=1}^{D} p\big(x_i \mid x_{\mathbf{Pa}(i)}\big), \tag{54.2}$$

where $p(x_i \mid x_{\mathbf{Pa}(i)})$ is the distribution obtained by pushing forward the noise U_i through $f_i(\cdot, \cdot)$. Equation (54.2) expresses that, in an SEM, each variable depends probabilistically only on its parents. This factorisation follows from the *causal Markov property*.

Definition 54.2 (Causal Markov Property)**.** *Given a DAG $\boldsymbol{G}$, each node X_i is independent of its non-descendants conditional on its parents:*

$$X_i \;\perp\; \mathrm{ND}(i) \,\big|\, X_{\mathbf{Pa}(i)}.$$

Here, $\mathrm{ND}(i)$ denotes the non-descendants of node i: all nodes that are not reachable from i via a directed path in $\boldsymbol{G}$. This assumption encodes that parents *cut off* other upstream information about a node.

Two other assumptions required in this chapter are *causal sufficiency* and *causal minimality and faithfulness*.

Definition 54.3 (Causal Sufficiency)**.** *There are no unobserved common causes of any two or more variables in the variable set $\{X_i\}_{i=1}^{D}$. Equivalently, there are no latent confounders in the causal graph $\boldsymbol{G}$.*

This is an important assumption that rules out the possibility of hidden common causes in the system.

Definition 54.4 (Causal Minimality and Faithfulness)**.** *For a distribution $p(\boldsymbol{X})$ induced by an SEM with causal graph $\boldsymbol{G}$, we say the model satisfies causal minimality if $p(\boldsymbol{X})$ is Markov with respect to $\boldsymbol{G}$ but is not Markov with respect to any proper subgraph of $\boldsymbol{G}$. Faithfulness asserts that* all and only *conditional independencies in $p(\boldsymbol{X})$ arise via d-separation in $\boldsymbol{G}$.*

Intuitively, minimality requires that no edge in $\boldsymbol{G}$ is superfluous for representing the distribution, while faithfulness rules out conditional independencies caused only by special parameter cancellations. Together, these assumptions allow us to read statistical independencies from the graph while excluding degenerate coincidences [1318].

54.2.3 Structural identifiability

An essential question in causal discovery is whether the causal structure underlying a set of variables can be recovered from data. When we observe samples from the joint distribution of $\boldsymbol{X} = (X_1, \ldots, X_D)$, there may exist several distinct structural equation models that induce exactly the same distribution. In such cases, it is impossible to determine which causal graph generated the data, even with infinite samples. The concept of *structural identifiability* formalises the conditions under which this ambiguity disappears. Here, the object to be identified is the causal graph from the observational distribution, rather than a particular parameterisation of its mechanisms. This differs from the neural-network parameter non-identifiability caused by symmetries discussed in Chapter 32.

Let $\mathcal{M}$ be a model family of structural equation models as defined in Section 54.2.2, parameterised by functional relations and noise distributions. Each element $(\boldsymbol{G}, \theta) \in \mathcal{M}$, consisting of a causal graph $\boldsymbol{G}$ and structural parameters θ, induces an observational distribution $P_{\boldsymbol{G},\theta}(\boldsymbol{X})$. Fix a data-generating model $(\boldsymbol{G}^\star, \theta^\star) \in \mathcal{M}$. We say that its causal structure $\boldsymbol{G}^\star$ is *structurally identifiable* within $\mathcal{M}$ if, for every $(\boldsymbol{G}, \theta) \in \mathcal{M}$,

$$P_{\boldsymbol{G},\theta}(\boldsymbol{X}) = P_{\boldsymbol{G}^\star,\theta^\star}(\boldsymbol{X}) \quad \Longrightarrow \quad \boldsymbol{G} = \boldsymbol{G}^\star.$$

Thus, within the assumed model family, even infinite observational data identify the graph but need not identify a unique value of θ.

In general, however, the mapping from structural models to observational distributions is not injective [638]. Different graphs can lead to identical statistical dependencies among variables. Under the causal Markov and faithfulness assumptions, the observational distribution $P(\boldsymbol{X})$ determines only a *Markov equivalence class* of DAGs rather than a single one. Two DAGs are Markov equivalent if they encode the same set of conditional independences, meaning that every distribution that factorises with respect to one also factorises with respect to the other. To make an SEM structurally identifiable, one common approach is to restrict its flexibility.

Additive noise model The definition of an SEM in Section 54.2.2 is fully general: the noise U_i can interact with parents in arbitrary, even non-additive, ways. This expressiveness comes with costs:

1. *Non-identifiability:* multiple distinct causal graphs and functions can induce the same observational distribution; without structure, recovering the causal graph from observations alone is impossible in general [1318].

2. *Ambiguous noise–signal separation:* without restrictions on how U_i enters, the observational distribution need not determine a unique split between the mechanism and its noise.

For example, any continuous bivariate distribution can be represented in either direction as $Y = F^{-1}_{Y|X}(U \mid X)$ or $X = F^{-1}_{X|Y}(V \mid Y)$, where the corresponding $U, V \sim \mathrm{Unif}(0,1)$ are independent of the conditioning variable. Unrestricted function–noise pairs therefore do not identify the causal direction. *Additive noise models* (ANMs), on the other hand, impose the simplifying form

$$X_i = f_i\big(X_{\mathbf{Pa}(i)}\big) + U_i, \qquad U_i \perp X_{\mathbf{Pa}(i)}, \qquad U_i \perp \{U_k : k \neq i\}, \tag{54.3}$$

ANMs simplify the SEM in a way that can establish identifiability and facilitate estimation of causal quantities under suitable conditions [786].

54.2.4 Causal inference through SEMs

Once a structural equation model has been specified, it provides more than a description of observed dependencies. Because the model explicitly encodes how each variable is generated from its direct causes, it allows us to reason about the consequences of interventions—what would happen if we were to change a variable by external manipulation rather than observation. This ability to predict the effect of hypothetical actions is what distinguishes causal models from purely statistical ones.

In an SEM defined by

$$X_i := f_i(X_{\mathbf{Pa}(i)}, U_i), \qquad i = 1, \ldots, D,$$

each function f_i represents a stable causal mechanism: it describes how X_i would respond to changes in its parents. Intervening on a variable, for instance setting X_k to a fixed value a, means that the normal mechanism generating X_k is replaced by a constant function. This operation is formalised through the *do-operator* $\mathrm{do}(X_k{=}a)$, and it defines a new system of equations

$$X_k := a, \qquad X_i := f_i(X_{\mathbf{Pa}(i)}, U_i) \text{ for } i \neq k.$$

The corresponding post-intervention distribution is denoted by $P(\boldsymbol{X} \mid \mathrm{do}(X_k{=}a))$. Conceptually, this distribution describes what would be observed if X_k were fixed externally, disconnecting it from its usual causes. In the causal graph, this amounts to deleting all incoming edges into node k while leaving the rest of the graph unchanged. Under this intervention, the factorisation of the joint distribution changes accordingly:

$$p(x_1, \ldots, x_D \mid \mathrm{do}(X_k{=}a)) = \delta_a(x_k) \prod_{i \neq k} p\big(x_i \mid x_{\mathbf{Pa}(i)}\big), \tag{54.4}$$

where $\delta_a(\cdot)$ is a point mass at the intervened value a. This expression illustrates that the effect of an intervention can be computed directly from the SEM by modifying only the local mechanism of the intervened variable, while all other structural relations remain intact. The notion of a *causal effect* arises naturally from this intervention semantics. For two variables X_i and X_j, the causal effect of setting X_i to x_i on the outcome X_j is defined as the expected value of X_j under that intervention:

$$\mathbb{E}[X_j \mid \mathrm{do}(X_i{=}x_i)]\,.$$

This expectation can be compared across different intervention values of X_i to quantify the strength or direction of the causal influence.

A particularly useful summary measure is the *average treatment effect* (ATE), which captures the expected change in the outcome when comparing two intervention levels x_i and x_i' of X_i:

$$\mathrm{ATE}(x_i, x_i') = \mathbb{E}[X_j \mid \mathrm{do}(X_i{=}x_i)] - \mathbb{E}[X_j \mid \mathrm{do}(X_i{=}x_i')]\,. \tag{54.5}$$

This quantity represents the population-level causal impact of changing X_i from x_i' to x_i. In many applications, however, one is interested in how this effect varies across subpopulations characterised by other variables Z. The corresponding conditional quantity,

$$\mathrm{CATE}(x_i, x_i'; z) = \mathbb{E}[X_j \mid \mathrm{do}(X_i{=}x_i), Z{=}z] - \mathbb{E}[X_j \mid \mathrm{do}(X_i{=}x_i'), Z{=}z]\,, \tag{54.6}$$

is called the *conditional average treatment effect* (CATE). It measures the causal effect of X_i on X_j for individuals or groups sharing the same value of Z. When the full SEM is known, causal effects such as these can be computed directly by propagating the modified structural equations as in Equation 54.4.

54.2.5 DAG constraints for causal discovery

The previous sections have described how an SEM encodes causal relations, how it supports inference about interventions, and how the ANM family can be structurally identifiable. In practice, however, the causal graph $\boldsymbol{G}$ itself is rarely known and must be learned from data. Causal discovery, therefore, aims to infer the graph that best explains the observed dependencies among variables. Formally, this is a constrained optimisation problem: among all possible adjacency matrices, only those corresponding to DAGs are admissible. Because the number of possible DAGs grows super-exponentially with the number of variables and acyclicity is a combinatorial constraint, one approach to sidestep this problem is a continuous relaxation or reparameterisation of the DAG constraint.

Continuous relaxation of DAG constraints

Recent developments in differentiable causal discovery reformulate this problem as continuous optimisation with a differentiable DAG constraint, allowing gradient-based methods to scale to higher dimensions.

Let $\boldsymbol{G} \in \mathbb{R}^{D \times D}$ be a weighted adjacency matrix for D variables. To characterise whether $\boldsymbol{G}$ is acyclic, NOTEARS [1868] introduces the smooth and differentiable DAG constraint

$$h(\boldsymbol{G}) \;=\; \mathrm{tr}\big(\exp(\boldsymbol{G} \odot \boldsymbol{G})\big) \;-\; D, \tag{54.7}$$

where $\odot$ denotes elementwise multiplication and exp is the matrix exponential. This constraint computes the total weight associated with closed walks of any length encoded by $\boldsymbol{G}$. Therefore, $h(\boldsymbol{G}) \geq 0$, and $h(\boldsymbol{G}) = 0$ *if and only if* there is no directed cycle in $\boldsymbol{G}$. Hence, a standard way to impose DAGness is to augment the learning objective with $h(\boldsymbol{G})$, translating the constrained optimisation into an unconstrained one.

In practice, this constraint is often used with an augmented Lagrangian to encourage the resulting graph to be acyclic. Suppose $\mathcal{L}_{\mathrm{fit}}(\theta, \boldsymbol{G})$ is the data-fit objective for a chosen SEM class (e.g., the log-likelihood or ELBO introduced later) with model parameters θ. We wish to solve

$$\max_{\theta,\, \boldsymbol{G}} \;\; \mathcal{L}_{\mathrm{fit}}(\theta, \boldsymbol{G}) \quad \text{s.t.} \quad h(\boldsymbol{G}) = 0. \tag{54.8}$$

The *augmented Lagrangian* replaces the hard constraint with the practical objective

$$\mathcal{L}_{\mathrm{AL}}(\theta, \boldsymbol{G};\, \alpha, \rho) \;=\; \mathcal{L}_{\mathrm{fit}}(\theta, \boldsymbol{G}) \;-\; \alpha\, h(\boldsymbol{G}) \;-\; \frac{\rho}{2}\, h(\boldsymbol{G})^2, \tag{54.9}$$

where $\alpha \geq 0$ is the Lagrange multiplier and $\rho > 0$ is the quadratic-penalty coefficient. Optimisation alternates between maximising this objective and updating α, while increasing ρ only when $h(\boldsymbol{G})$ does not decrease sufficiently. This avoids fixing an excessively large penalty in advance while driving $h(\boldsymbol{G})$ towards zero.

Reparameterisation of DAGs through node potentials

An alternative strategy is to *reparameterise* the hypothesis space so that every member already encodes a DAG. One idea is to assign a scalar *potential* $p_i \in \mathbb{R}$ to each node i and orient edges *downhill* in that potential order. Formally, define the matrix

$$\big(\operatorname{grad} \boldsymbol{p}\big)(i,j) \;=\; p_i - p_j, \qquad \boldsymbol{G}(\boldsymbol{p}) \;=\; \mathrm{ReLU}\big(\operatorname{grad} \boldsymbol{p}\big),$$

where ReLU is applied elementwise. Thus, $\boldsymbol{G}(\boldsymbol{p})_{ij} > 0$ only if $p_i > p_j$, and $\boldsymbol{G}(\boldsymbol{p})$ must define a DAG. Let $\boldsymbol{W} \in \mathbb{R}^{D\times D}$ be a *skew-symmetric* score matrix ($\boldsymbol{W} = -\boldsymbol{W}^\top$) that carries edge strengths. The mapping

$$\gamma(\boldsymbol{W}, \boldsymbol{p}) \;=\; \boldsymbol{W} \odot \boldsymbol{G}(\boldsymbol{p}) \tag{54.10}$$

yields a *continuously weighted DAG*: every nonzero entry of $\gamma(\boldsymbol{W}, \boldsymbol{p})$ points from a higher potential to a lower one, so potentials strictly decrease along any directed path; a cycle would force a net decrease around a loop, which is impossible. Hence, $\gamma(\boldsymbol{W}, \boldsymbol{p})$ is acyclic by construction.

For any pair of nodes i, j, at least one of $\boldsymbol{G}(\boldsymbol{p})_{ij}$ and $\boldsymbol{G}(\boldsymbol{p})_{ji}$ is zero. A skew-symmetric $\boldsymbol{W}$ therefore reduces the number of free parameters to $\frac{D(D-1)}{2}$. Yu et al. [1826] proved that, with skew-symmetric $\boldsymbol{W}$, $\gamma(\boldsymbol{W}, \boldsymbol{p})$ characterises all possible DAGs. This node-potential formulation is therefore complete and translates the DAG-learning problem into inference over the continuous-valued node potentials $\boldsymbol{p}$.

54.3 Bayesian Deep Learning for Causality with Static Data

The previous sections have established the theoretical foundations of causality: how structural equation models (SEMs) encode causal mechanisms, how interventions define causal effects, and how directed acyclic graphs (DAGs) constrain possible relations among variables. While these concepts provide the language for causal reasoning, modern data analysis often requires learning both the causal structure and the corresponding functional mechanisms directly from large, high-dimensional observational datasets. This section focuses on static data. Recent advances bridge causal discovery and causal inference by treating both as components of a single generative model represented by an SEM. Traditional causal discovery methods, such as constraint-based or score-based approaches [1318, 1546], typically learn DAGs from conditional-independence tests or scoring functions but do not estimate the functional mechanisms between variables. Recent probabilistic frameworks bring these perspectives together by formulating causal learning as joint inference over the graph and the structural equations. This viewpoint has given rise to differentiable structure-learning methods such as NOTEARS [1868], GraN-DAG [955], and MissDAG [572], as well as Bayesian approaches such as DiBS [1077], BayesDAG [35], and the flow-based *Deep End-to-End Causal Inference* (DECI) framework [590].

In this chapter, we focus on two representative examples of this emerging paradigm: *DECI* and *BayesDAG*. Both operate on static datasets and infer the underlying SEMs with structural identifiability guarantees. DECI provides a unified, end-to-end framework that combines a nonlinear ANM with Bayesian variational inference to jointly learn a graph posterior, structural functions, and exogenous noise distributions. BayesDAG, in contrast, uses stochastic-gradient MCMC (SG-MCMC) and continuous relaxations for sampling-based posterior inference over DAGs.

54.3.1 Deep End-to-End Causal Inference

DECI combines Bayesian variational inference with causal machine learning to represent uncertainty in inferred causal graphs. It applies the variational-inference framework introduced in Chapter 12 to learn a distribution over causal graphs from observational data, infer the associated SEM, and estimate causal quantities. We first describe DECI's Bayesian formulation of an ANM and how it is learned from observational data. We then state the conditions under which DECI recovers the true causal graph with infinite data and explain how the learned model can simulate interventional samples for treatment-effect estimation. The code is available at `https://github.com/microsoft/causica`.

Combining variational inference with ANM

Let $\boldsymbol{X} = (X_1, \dots, X_D)$ denote the variables, let $\boldsymbol{Z} = (Z_1, \dots, Z_D)$ denote mutually independent exogenous noise, and let $\boldsymbol{G} \in \{0,1\}^{D \times D}$ encode a DAG using the convention $\boldsymbol{G}_{ij} = 1$ for $i \to j$. Let θ collect the parameters of the structural mechanisms f_i and the exogenous noise model. DECI assumes the ANM introduced in Equation 54.3:

$$X_i \;=\; f_i\big(X_{\mathbf{Pa}(i;\boldsymbol{G})};\, \theta\big) + Z_i. \tag{54.11}$$

We rewrite it as the invertible map $\boldsymbol{Z} = \boldsymbol{g}_{\boldsymbol{G}}(\boldsymbol{X};\theta) := \boldsymbol{X} - \boldsymbol{f}_{\boldsymbol{G}}(\boldsymbol{X};\theta)$. In a topological ordering of the DAG, each f_i depends only on variables preceding X_i. The Jacobian $\partial \boldsymbol{g}_{\boldsymbol{G}}/\partial \boldsymbol{X}$ is therefore triangular with unit diagonal, so its determinant is one [590]. The likelihood consequently factorises as

$$p_\theta(\boldsymbol{X} \mid \boldsymbol{G}) \;=\; p_{\boldsymbol{Z}}\big(\boldsymbol{g}_{\boldsymbol{G}}(\boldsymbol{X};\theta)\big) \;=\; \prod_{i=1}^{D} p_{Z_i}\big(g_{\boldsymbol{G},i}(\boldsymbol{X};\theta)\big), \tag{54.12}$$

where the first equality follows from the change-of-variables formula and $g_{\boldsymbol{G},i}$ is the ith component of $\boldsymbol{g}_{\boldsymbol{G}}$. The function $\boldsymbol{f}_{\boldsymbol{G}} : \mathbb{R}^D \to \mathbb{R}^D$ must respect the adjacency relations in $\boldsymbol{G}$: if $\boldsymbol{G}_{ji} = 0$, then f_i must satisfy $\partial f_i(\boldsymbol{X})/\partial X_j = 0$. DECI uses the flexible parameterisation

$$f_i(\boldsymbol{X}) \;=\; \zeta_i\Big(\sum_{j=1}^{D} \boldsymbol{G}_{ji}\, \ell_j(X_j)\Big), \qquad \zeta_i(\cdot) = \zeta(u_i, \cdot),\;\; \ell_j(\cdot) = \ell(v_j, \cdot), \tag{54.13}$$

where $\zeta(\cdot)$ and $\ell(\cdot)$ are multilayer perceptrons (MLPs) shared across nodes and (u_i, v_j) are trainable node embeddings. A naive implementation would require training $2D$ separate MLPs; weight sharing ensures that f_i depends only on $\boldsymbol{X}_{\mathbf{Pa}(i;\boldsymbol{G})}$ while keeping the parameter count modest.

For the exogenous noise $\boldsymbol{Z}$, DECI supports either (1) Gaussian marginals $p_{Z_i} = \mathcal{N}(0, \sigma_i^2)$ or (2) flexible flows κ_i with standard-normal base distributions [1390]:

$$p_{Z_i}(Z_i) \;=\; \mathcal{N}\big(\kappa_i^{-1}(Z_i);\, 0, 1\big) \left| \frac{\partial \kappa_i^{-1}(Z_i)}{\partial Z_i} \right|. \tag{54.14}$$

DECI uses rational-quadratic spline flows [470], parameterised independently across dimensions to preserve noise independence. Although the base distribution is Gaussian, it can be replaced by another distribution with a differentiable density.

Let $\boldsymbol{x}^1, \dots, \boldsymbol{x}^N$ denote N observational samples. DECI models these data jointly with the causal graph as

$$p_\theta(\boldsymbol{G}, \boldsymbol{x}^1, \dots, \boldsymbol{x}^N) = p(\boldsymbol{G}) \prod_{n=1}^{N} p_\theta(\boldsymbol{x}^n \mid \boldsymbol{G}), \tag{54.15}$$

where the conditional distribution is defined in Equation 54.12. Fitting θ then gives the posterior $p_\theta(\boldsymbol{G} \mid \boldsymbol{x}^1, \dots, \boldsymbol{x}^N)$, which characterises uncertainty about the causal structure.

The graph prior $p(\boldsymbol{G})$ should favour sparse DAGs, but directly representing a distribution over the discrete DAG space is difficult. DECI instead uses the soft DAG constraint $h(\boldsymbol{G})$ introduced in Equation 54.7 together with the augmented Lagrangian:

$$p(\boldsymbol{G}) \;\propto\; \exp\Big(-\lambda_s \|\boldsymbol{G}\|_F^2 \;-\; \alpha\, h(\boldsymbol{G}) \;-\; \tfrac{\rho}{2}\, h(\boldsymbol{G})^2 \Big), \tag{54.16}$$

where λ_s controls the sparsity of $\boldsymbol{G}$, while α and ρ are the augmented-Lagrangian coefficients.

Direct computation of $p_\theta(\boldsymbol{G} \mid \boldsymbol{x}^1, \dots, \boldsymbol{x}^N)$ is intractable. DECI therefore uses the mean-field, edge-factorised approximation $q_\phi(\boldsymbol{G})$ with the ENCO parameterisation [1056]:

$$q_\phi(\boldsymbol{G}) = \prod_{i \neq j} \mathrm{Ber}\big(\boldsymbol{G}_{ij} \mid \sigma(\gamma_{ij})\sigma(\vartheta_{ij})\big)\,, \tag{54.17}$$

where $\sigma(\cdot)$ is the sigmoid function, γ_{ij} represents edge existence, and $\vartheta_{ij} = -\vartheta_{ji}$ represents orientation. The orientation probabilities for $i \to j$ and $j \to i$ therefore sum to one, so increasing one decreases the other and helps avoid 2-cycles. Separating existence from orientation also simplifies optimisation of edge probabilities. This

Bernoulli construction is the graph-valued analogue of the mean-field factorisations discussed in Chapter 13. The corresponding evidence lower bound (ELBO) is

$$\mathrm{ELBO}(\theta, \phi) \;=\; \mathbb{E}_{q_\phi(\boldsymbol{G})}\Big[\log p(\boldsymbol{G}) + \sum_{n=1}^{N} \log p_\theta(\boldsymbol{x}^n \mid \boldsymbol{G})\Big] \;+\; \mathcal{H}(q_\phi) \le \log p_\theta(\boldsymbol{x}^1, \ldots, \boldsymbol{x}^N), \tag{54.18}$$

where $\mathcal{H}(q_\phi)$ is the entropy of q_ϕ. The SEM parameters θ and variational parameters ϕ are trained by maximising the ELBO. The Gumbel–Softmax trick [831] provides stochastic gradient estimates with respect to ϕ.

Statistical guarantees of DECI

The structural-identifiability discussion in Section 54.2.3 motivates a central question for DECI: does maximising the ELBO in Equation 54.18 recover the ground-truth graph and data-generating distribution with infinite data?

The recovery result in Geffner et al. [590] uses the following assumptions:

1. *Non-trivial mechanisms:* each structural mechanism is non-invertible, three-times differentiable, and non-constant in each of its parent inputs.
2. *Regular noise density:* the noise distribution has a differentiable density and bounded log-likelihood.
3. *Correct specification:* there exists a parameter $\theta^\star$ such that $p_{\theta^\star}(\boldsymbol{X}; \boldsymbol{G}_0)$ equals the data-generating distribution, where $\boldsymbol{G}_0$ is the ground-truth graph. The parameter itself need not be unique; recovery is in the distributional or functional sense.
4. *No latent confounders:* the data-generating process satisfies causal sufficiency as defined in Definition 54.3.

The non-constancy and non-invertibility conditions play distinct roles: non-constancy yields causal minimality, whereas non-invertibility excludes non-identifiable functional relationships. Together with the other assumptions, they imply the causal Markov property and structural identifiability while excluding cases such as linear Gaussian ANMs [590].

Theorem 54.1 (DECI recovers the true data-generating process [590])**.** *Assume that DECI's SEM is a continuous additive noise model whose mechanisms are non-invertible, three-times differentiable, and non-constant in every parent input, and whose noise density is differentiable with bounded log-likelihood. If the model is correctly specified, the data-generating process has no latent confounders, and infinitely many observations are available, then an ELBO maximiser (θ', q'_ϕ) satisfies*

$$q'_\phi(\boldsymbol{G}) = \delta_{\boldsymbol{G}_0}(\boldsymbol{G}) \qquad \text{and} \qquad p_{\theta'}(\boldsymbol{X}; \boldsymbol{G}_0) = p(\boldsymbol{X}; \boldsymbol{G}_0),$$

where $\delta_{\boldsymbol{G}_0}$ is a point mass at the unique ground-truth graph $\boldsymbol{G}_0$.

Proof intuition. With infinite data, correct specification and regularity make maximum-likelihood estimation recover the true data-generating distribution. Non-constancy and non-invertibility make the associated graph structurally identifiable, so the maximum-likelihood graph is uniquely $\boldsymbol{G}_0$. After dividing the ELBO by the sample size, the graph-prior KL term vanishes asymptotically; any graph receiving positive variational mass must therefore attain the same maximum expected log-likelihood. Structural identifiability leaves only $\boldsymbol{G}_0$, forcing q_ϕ to collapse to a point mass at the true graph [590].

Estimating causal quantities

We now explain how a trained DECI model estimates two causal quantities: the *average treatment effect* (ATE) and *conditional average treatment effect* (CATE). Henceforth, T, Y, and C denote disjoint treatment, outcome, and conditioning index sets, respectively; the corresponding variables are X_T, X_Y, and X_C. The ATE compares two intervention values a and b:

$$\mathrm{ATE}(a, b \mid \boldsymbol{G}) \;=\; \mathbb{E}[X_Y \mid \mathrm{do}(X_T{=}a), \boldsymbol{G}] - \mathbb{E}[X_Y \mid \mathrm{do}(X_T{=}b), \boldsymbol{G}]\,, \tag{54.19}$$

where $\mathrm{do}(\cdot)$ removes incoming edges to X_T and replaces its structural mechanism by the chosen intervention value [1307, 1318]. The CATE additionally conditions on $X_C = c$:

$$\mathrm{CATE}(a, b \mid c, \boldsymbol{G}) \;=\; \mathbb{E}[X_Y \mid \mathrm{do}(X_T{=}a),\, X_C{=}c, \boldsymbol{G}] - \mathbb{E}[X_Y \mid \mathrm{do}(X_T{=}b),\, X_C{=}c, \boldsymbol{G}]\,. \tag{54.20}$$

As explained above, DECI returns $q_\phi(\boldsymbol{G})$, an approximation of the posterior over graphs given observational data. Then, interventional distributions, ATE and CATE can be obtained by marginalising over graphs as

$$\mathbb{E}_{q_\phi(\boldsymbol{G})}\big[p_\theta(X_Y \mid \mathrm{do}(X_T{=}a),\, \boldsymbol{G})\,\big], \qquad \mathbb{E}_{q_\phi(\boldsymbol{G})}\big[\mathrm{ATE}(a, b \mid \boldsymbol{G})\big], \qquad \mathbb{E}_{q_\phi(\boldsymbol{G})}\big[\mathrm{CATE}(a, b \mid c, \boldsymbol{G})\big]. \tag{54.21}$$

This is a probabilistic relaxation of traditional causal estimators. When the posterior is concentrated on one graph, i.e. $q_\phi(\boldsymbol{G}) = \delta_{\boldsymbol{G}_i}(\boldsymbol{G})$, the procedure reduces to causal inference under the fixed graph $\boldsymbol{G}_i$.

Estimating ATE. After training, DECI can simulate samples $\boldsymbol{x}$ from $p_\theta(\boldsymbol{X} \mid \boldsymbol{G})$ (recall Equation 54.12). We sample a graph $\boldsymbol{G} \sim q_\phi(\boldsymbol{G})$ and exogenous noise $\boldsymbol{z} \sim p_{\boldsymbol{Z}}$, then apply the ANM in Equation 54.11, using the mechanisms in Equation 54.13, in a topological ordering of $\boldsymbol{G}$.

However, ATE requires samples from the interventional distribution $p\left(x_{\setminus T} \mid \mathrm{do}(X_T{=}b),\, \boldsymbol{G}\right)$. These can be obtained from the definition of the do operator:

$$p\left(x_{\setminus T} \mid \mathrm{do}(X_T{=}b),\, \boldsymbol{G}\right) \;=\; p\left(x_{\setminus T} \mid X_T{=}b,\, \boldsymbol{G}_{\mathrm{do}(X_T)}\right),$$

where $\boldsymbol{G}_{\mathrm{do}(X_T)}$ is the *mutilated* graph obtained by removing incoming edges to X_T. Thus, samples from this distribution are obtained by fixing $X_T = b$ and using $\boldsymbol{G}_{\mathrm{do}(X_T)}$ instead of $\boldsymbol{G}$. Their sample mean gives the Monte Carlo estimate required by Equation 54.19. When the posterior $q_\phi(\boldsymbol{G})$ has not collapsed to one graph, these samples are drawn from a mixture distribution, translating the uncertainty in the discovered graph into uncertainty in ATE estimates.

Estimating CATE. Under the assumption that no variable in X_T is a cause of a variable in X_C, we can estimate CATE by sampling from $p\big(X_{\setminus T} \mid \mathrm{do}(X_T), \boldsymbol{G}\big)$ and then estimating the conditional distribution of X_Y given X_C. The conditional density is not directly available because its normalising marginal is intractable. DECI therefore regresses X_Y on X_C using samples simulated under each intervention. Specifically,

$$p_\theta(X_Y \mid \mathrm{do}(X_T{=}b),\, X_C{=}c,\, \boldsymbol{G}) \;=\; \frac{p_\theta(X_Y, X_C{=}c \mid X_T{=}b,\, \boldsymbol{G}_{\mathrm{do}(X_T)})}{p_\theta(X_C{=}c \mid X_T{=}b,\, \boldsymbol{G}_{\mathrm{do}(X_T)})} \tag{54.22}$$

The denominator is not tractable in the DECI model. However, we can sample from the *joint* interventional distribution $p_\theta(X_Y, X_C \mid X_T{=}b,\, \boldsymbol{G}_{\mathrm{do}(X_T)})$.

We use samples from this joint to train a surrogate regression model $h_{\boldsymbol{G}}$[1] that maps $X_C \mapsto X_Y$ by minimising the squared loss

$$\mathbb{E}_{p_\theta(X_Y, X_C \mid X_T=b,\, \boldsymbol{G}_{\mathrm{do}(X_T)})}\big[\, \|X_Y - h_{\boldsymbol{G}}(X_C)\|_2^2 \,\big], \tag{54.23}$$

making $h_{\boldsymbol{G}}$ approximate the conditional mean of X_Y given X_C under the intervention. We choose $h_{\boldsymbol{G}}$ to be a basis-function linear model with random Fourier features that approximate an RBF kernel. As illustrated in the original DECI exposition, we train two separate surrogate models, one for the intervention $X_T = a$ and one for the reference $X_T = b$. We then estimate CATE as the difference between their outputs evaluated at $X_C = c$. Repeating this for multiple posterior graph samples $\boldsymbol{G} \sim q_\phi$ yields the graph-marginalised estimate

$$\mathbb{E}_{\boldsymbol{G}\sim q_\phi(\boldsymbol{G})}\Big[\, h^{(a)}_{\boldsymbol{G}_{\mathrm{do}(X_T)}}(c) \;-\; h^{(b)}_{\boldsymbol{G}_{\mathrm{do}(X_T)}}(c) \,\Big].$$

Theoretical guarantees. The recovery result in Section 54.3.1 concerns the true graph and data-generating distribution, not a unique parameter vector. Under correct specification, DECI also satisfies causal-effect identifiability and positivity, so its treatment-effect estimates are correct in the infinite-data limit; Geffner et al. [590] provides the full argument.

DECI provides an end-to-end route from observational data to a flexible nonlinear ANM, a variational posterior over DAGs, and (C)ATE estimates that propagate structural uncertainty. Its edge-factorised variational family can nevertheless be limiting for complicated graph posteriors. Moreover, acyclicity is enforced *softly*, so non-DAGs can be sampled before the augmented-Lagrangian optimisation converges. The next section introduces *BayesDAG*, a sampling-based approach using a node-potential parameterisation that produces DAGs throughout training.

54.3.2 BayesDAG: Fully Bayesian treatment through sampling

The DECI framework already brings a Bayesian perspective to causal discovery: it learns a variational posterior over DAGs and propagates structural uncertainty into interventional estimates. Its edge-factorised posterior and soft acyclicity constraint, however, motivate an alternative treatment of the graph space. *BayesDAG* keeps the same downstream causal-inference recipe but replaces the discovery component with an acyclicity-by-construction

[1] The subscript $\boldsymbol{G}$ distinguishes surrogate models fitted to samples from different graphs drawn from q_ϕ.

parameterisation and a stochastic-gradient Markov chain Monte Carlo (SG-MCMC) scheme. Concretely, it represents DAGs through the node potentials introduced in Section 54.2.5 and a binary edge mask, and then connects this representation to permutation learning to obtain useful gradients for the potentials. SG-MCMC, introduced in Chapter 2, is then used to sample the continuous node potentials and SEM parameters, while variational inference handles the discrete mask. Once samples from the joint graph-and-model posterior are available, causal quantities are estimated using the graph-marginalised simulation approach introduced for DECI. An implementation of BayesDAG is available at `https://github.com/microsoft/Project-BayesDAG`.

Bayesian inference in the DAG space

To guarantee that every sampled graph is a DAG, BayesDAG reparameterises the graph space so that acyclicity is *built in* rather than enforced by a smooth penalty as in DECI. Recall the weighted NoCurl map in Equation 54.10, which uses ReLU(grad $\boldsymbol{p}$) to orient continuous edge weights. For posterior inference over *binary* DAGs, this form has two drawbacks: (i) the potential-based orientation permits one edge for every unordered pair of nodes, so the continuous entries of $\boldsymbol{W}$ must be driven exactly to zero to disable edges, and (ii) the same continuous weights encode both edge existence and strength, entangling structure with parameter learning. BayesDAG separates these roles by replacing the weighted ReLU construction with the binary map

$$\tau:\ \{0,1\}^{D\times D}\times\mathbb{R}^D \longrightarrow \{0,1\}^{D\times D}, \qquad \tau(\boldsymbol{W},\boldsymbol{p}) \ =\ \boldsymbol{W}\odot \mathrm{Step}(\mathrm{grad}\,\boldsymbol{p}), \tag{54.24}$$

where Step$(\cdot)$ is applied elementwise. Here, $\boldsymbol{W}$ is a *binary* edge mask that encodes existence and controls the graph sparsity, while $\boldsymbol{p}$ orients those edges. By construction, $\tau(\boldsymbol{W},\boldsymbol{p})$ is a binary adjacency matrix with no directed cycles.

Equivalence of Bayesian inference. Let $\mathcal{G}_D$ be the set of binary DAG adjacencies on D nodes and let $p(\boldsymbol{G})$ be any prior on $\mathcal{G}_D$. The core idea of BayesDAG is to perform inference in the product space $(\boldsymbol{W},\boldsymbol{p})\in\{0,1\}^{D\times D}\times\mathbb{R}^D$ and push the resulting posterior forward through τ. The following result [35] formalises the equivalence, provided that the prior over $(\boldsymbol{W},\boldsymbol{p})$ induces $p(\boldsymbol{G})$ and ties among the potentials are excluded.

Theorem 54.2 (Equivalence of inference in $(\boldsymbol{W},\boldsymbol{p})$ and binary DAG space [35])**.** *Assume that the entries of $\boldsymbol{p}$ are pairwise distinct. Let $p_\tau(\boldsymbol{W},\boldsymbol{p})$ be a joint prior mass-density with respect to counting measure on binary $\boldsymbol{W}$ and Lebesgue measure on continuous $\boldsymbol{p}$. If this prior induces the desired graph prior,*

$$p(\boldsymbol{G}) = \sum_{\boldsymbol{W}\in\{0,1\}^{D\times D}} \int_{\mathbb{R}^D} p_\tau(\boldsymbol{W},\boldsymbol{p})\,\mathbf{1}\{\boldsymbol{G}=\tau(\boldsymbol{W},\boldsymbol{p})\}\ d\boldsymbol{p},$$

then its posterior induces the corresponding graph posterior:

$$p(\boldsymbol{G}\mid\boldsymbol{D}) = \sum_{\boldsymbol{W}\in\{0,1\}^{D\times D}} \int_{\mathbb{R}^D} p_\tau(\boldsymbol{W},\boldsymbol{p}\mid\boldsymbol{D})\,\mathbf{1}\{\boldsymbol{G}=\tau(\boldsymbol{W},\boldsymbol{p})\}\ d\boldsymbol{p}. \tag{54.25}$$

Proof intuition. The map τ is complete: every pair $(\boldsymbol{W},\boldsymbol{p})$ produces a DAG, and every DAG can be recovered by choosing potentials consistent with one of its topological orderings and setting $\boldsymbol{W}$ to its edge mask. The prior condition above therefore says that $p_\tau(\boldsymbol{W},\boldsymbol{p})$ pushes forward to $p(\boldsymbol{G})$. For a fixed graph, the likelihood depends on $(\boldsymbol{W},\boldsymbol{p})$ only through $\tau(\boldsymbol{W},\boldsymbol{p})=\boldsymbol{G}$; applying Bayes' rule and summing and integrating over all pairs in this preimage consequently yields $p(\boldsymbol{D}\mid\boldsymbol{G})p(\boldsymbol{G})$ divided by the same evidence as direct inference in $\mathcal{G}_D$. Hence, the posterior pushforward in Equation 54.25 is exactly $p(\boldsymbol{G}\mid\boldsymbol{D})$.

Model and posterior in the augmented space. Let $\boldsymbol{\Theta}$ denote the SEM parameters, such as the parameters of the nonlinear functions introduced for DECI. BayesDAG reuses the standard ANM likelihood in Equation 54.12, but conditions it on $\boldsymbol{G}=\tau(\boldsymbol{W},\boldsymbol{p})$:

$$p(\boldsymbol{D},\boldsymbol{W},\boldsymbol{p},\boldsymbol{\Theta}) \ =\ p\big(\boldsymbol{D}\mid\boldsymbol{G}=\tau(\boldsymbol{W},\boldsymbol{p}),\boldsymbol{\Theta}\big)\ p_\tau(\boldsymbol{W},\boldsymbol{p})\ p(\boldsymbol{\Theta}).$$

A convenient choice places a Gaussian prior on $\boldsymbol{p}$ and factorised Bernoulli priors on the entries of $\boldsymbol{W}$, thereby controlling expected sparsity. The resulting posterior target is

$$p_\tau(\boldsymbol{W},\boldsymbol{p},\boldsymbol{\Theta}\mid\boldsymbol{D}) \ \propto\ p\big(\boldsymbol{D}\mid\tau(\boldsymbol{W},\boldsymbol{p}),\boldsymbol{\Theta}\big)\ p_\tau(\boldsymbol{W},\boldsymbol{p})\ p(\boldsymbol{\Theta}). \tag{54.26}$$

The continuous variables $\boldsymbol{p}$ and $\boldsymbol{\Theta}$ can be sampled with SG-MCMC without a Metropolis–Hastings acceptance step. However, the piecewise-constant Step$(\cdot)$ in τ provides no useful derivative with respect to $\boldsymbol{p}$, so it cannot be used directly in a gradient-based sampler.

Equivalent formulation: enabling gradients for node potentials

Intuition. The node potentials $\boldsymbol{p}$ define a topological order via pairwise differences: $\text{Step}((\text{grad}\,\boldsymbol{p})(i,j))$ keeps only downhill orientations. Thus, ranking nodes by their potentials yields a permutation that orients all admissible edges from higher to lower potential. This observation lets us replace the non-differentiable $\text{Step}(\cdot)$ with a permutation-learning view.

Alternative formulation. Let $\boldsymbol{L} \in \{0,1\}^{D\times D}$ be the strictly lower-triangular matrix of ones and $\boldsymbol{o} = [1,\dots,D]^\top$. Define

$$\boldsymbol{G} \;=\; \boldsymbol{W} \odot \big[\text{perm}(\boldsymbol{p})\,\boldsymbol{L}\,\text{perm}(\boldsymbol{p})^\top\big], \qquad \text{where} \quad \text{perm}(\boldsymbol{p}) \;=\; \underset{\boldsymbol{\Pi}\in\boldsymbol{\Sigma}_D}{\arg\max}\, \boldsymbol{p}^\top(\boldsymbol{\Pi}\,\boldsymbol{o}). \tag{54.27}$$

Here, $\boldsymbol{\Sigma}_D$ is the set of $D \times D$ permutation matrices. The permutation $\text{perm}(\boldsymbol{p})$ encodes the ranking induced by the potentials, $\text{perm}(\boldsymbol{p})\boldsymbol{L}\text{perm}(\boldsymbol{p})^\top$ orients all possible edges according to that ranking, and the mask $\boldsymbol{W}$ removes absent edges. The following result shows that this construction coincides with Equation 54.24.

Theorem 54.3 (Equivalence to original formulation [35])**.** *Assume that the conditions of Theorem 54.2 hold. Then, for a given* $(\boldsymbol{W},\boldsymbol{p})$*,*

$$\boldsymbol{G} = \boldsymbol{W} \odot \text{Step}(\text{grad}\,\boldsymbol{p}) = \boldsymbol{W} \odot \left[\text{perm}(\boldsymbol{p})\boldsymbol{L}\text{perm}(\boldsymbol{p})^\top\right],$$

where $\boldsymbol{G}$ *is a DAG and* $\text{perm}(\boldsymbol{p})$ *is defined in Equation 54.27.*

This theorem translates the original formulation into finding a corresponding permutation $\text{perm}(\boldsymbol{p})$. Although this does not itself resolve the lack of an informative gradient, it allows the permutation to be approximated using tools from the differentiable-permutation literature [171, 1162]. BayesDAG uses the standard Sinkhorn relaxation [1162], which replaces a discrete assignment with a differentiable doubly-stochastic approximation; other differentiable permutation estimators could be used instead.

Sinkhorn operator. The argmax over permutations in Equation 54.27 is non-differentiable. Let $\text{Sinkhorn}(\boldsymbol{M})$ denote the operator that iteratively normalises the rows and columns of a matrix $\boldsymbol{M}$ to obtain a doubly-stochastic matrix. This entropy-regularised relaxation of the assignment problem is a standard differentiable surrogate widely used in optimal transport and learning-to-rank [351], and is not specific to the causal setting. The generic assignment problem

$$\boldsymbol{\Pi}^\star \;=\; \underset{\boldsymbol{\Pi}\in\boldsymbol{\Sigma}_D}{\arg\max}\, \langle \boldsymbol{\Pi},\, \boldsymbol{M}\rangle, \tag{54.28}$$

admits an entropy-regularised relaxation whose solution is expressed by the Sinkhorn operator [1162]:

$$\text{Sinkhorn}(\boldsymbol{M}/t) \;=\; \underset{\boldsymbol{\Pi}\in\mathcal{B}_D}{\arg\max} \Big\{ \langle \boldsymbol{\Pi},\, \boldsymbol{M}\rangle + t\,h(\boldsymbol{\Pi}) \Big\}, \qquad \lim_{t\to 0} \text{Sinkhorn}(\boldsymbol{M}/t) \;\in\; \boldsymbol{\Sigma}_D,$$

where $\mathcal{B}_D$ is the Birkhoff polytope of $D \times D$ doubly-stochastic matrices and $h(\cdot)$ is the entropy function. Applying this relaxation to Equation 54.27 with the rank-1 score matrix $\boldsymbol{M} = \boldsymbol{p}\boldsymbol{o}^\top$ gives

$$\underset{\boldsymbol{\Pi}\in\boldsymbol{\Sigma}_D}{\arg\max}\, \boldsymbol{p}^\top(\boldsymbol{\Pi}\boldsymbol{o}) \;=\; \underset{\boldsymbol{\Pi}\in\boldsymbol{\Sigma}_D}{\arg\max} \langle \boldsymbol{\Pi},\, \boldsymbol{p}\boldsymbol{o}^\top\rangle \;=\; \lim_{t\to 0} \text{Sinkhorn}\left(\frac{\boldsymbol{p}\boldsymbol{o}^\top}{t}\right). \tag{54.29}$$

In practice, a small positive temperature t yields differentiable doubly-stochastic scores. The Hungarian algorithm [1210] projects these scores to a binary permutation for the forward computation, while a straight-through estimator backpropagates through the Sinkhorn relaxation. This provides useful approximate gradients for the node potentials $\boldsymbol{p}$.

Bayesian causal discovery via sampling

The preceding relaxation provides approximate gradients of the reparameterised graph posterior with respect to $\boldsymbol{p}$. BayesDAG uses these gradients in a combined scheme: SG-MCMC samples the continuous variables $(\boldsymbol{p},\boldsymbol{\Theta})$, while a lightweight variational update handles the discrete mask $\boldsymbol{W}$.

For its nonlinear SEM, BayesDAG uses the same ANM likelihood as DECI in Section 54.3.1:

$$p(\boldsymbol{x} \mid \boldsymbol{G},\boldsymbol{\Theta}) \;=\; p_{\boldsymbol{\epsilon}}\big(\boldsymbol{x} - \boldsymbol{f}(\boldsymbol{x};\boldsymbol{\Theta})\big) \;=\; \prod_{i=1}^{D} p_{\epsilon_i}\big(x_i - f_i(\boldsymbol{x}_{\mathbf{Pa}(i,\boldsymbol{G})})\big). \tag{54.30}$$

Prior design. The graph prior $p(\boldsymbol{G})$ is induced through $p_\tau(\boldsymbol{W}, \boldsymbol{p})$. Let $p(\boldsymbol{W}) = \prod_{i \neq j} \mathrm{Ber}(W_{ij} \mid \pi_{ij})$ be a factorised Bernoulli baseline prior. One convenient joint prior is

$$p(\boldsymbol{W}, \boldsymbol{p}, \boldsymbol{\Theta}) \propto \mathcal{N}(\boldsymbol{\Theta}; \mathbf{0}, \boldsymbol{I})\, \mathcal{N}(\boldsymbol{p}; \mathbf{0}, \alpha \boldsymbol{I})\, p(\boldsymbol{W}) \exp\big(-\lambda_s \|\tau(\boldsymbol{W}, \boldsymbol{p})\|_F^2\big), \tag{54.31}$$

where α controls the prior scale of $\boldsymbol{p}$, the probabilities π_{ij} control baseline edge inclusion, and λ_s further encourages sparsity in $\boldsymbol{G}$.

The main remaining challenge is the binary nature of $\boldsymbol{W} \in \{0, 1\}^{D \times D}$. BayesDAG addresses it by combining SG-MCMC for $\boldsymbol{p}, \boldsymbol{\Theta}$ with variational inference for $\boldsymbol{W}$. Figure 54.2 summarises the dependencies used in this hybrid inference scheme.

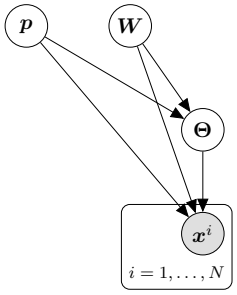


Figure 54.2: Graphical model of the BayesDAG inference formulation. Here, $\boldsymbol{p}$ and $\boldsymbol{W}$ denote the node potentials and binary edge mask, $\boldsymbol{\Theta}$ denotes the SEM parameters, and $\boldsymbol{x}^i$ denotes observed sample i.

The Gibbs-style procedure alternates between (1) sampling $\boldsymbol{p}, \boldsymbol{\Theta} \sim p(\boldsymbol{p}, \boldsymbol{\Theta} \mid \boldsymbol{D}, \boldsymbol{W})$ with SG-MCMC and (2) updating the variational posterior $q_\phi(\boldsymbol{W} \mid \boldsymbol{p})$ to approximate $p(\boldsymbol{W} \mid \boldsymbol{p}, \boldsymbol{\Theta}, \boldsymbol{D})$.

The following proposition specifies the gradients required by SG-MCMC, where $U(\boldsymbol{p}, \boldsymbol{W}, \boldsymbol{\Theta}) = -\log p(\boldsymbol{D}, \boldsymbol{W}, \boldsymbol{\Theta}, \boldsymbol{p})$.

Proposition 54.1 ([35])**.** *For the model above, the required gradients are*

$$\nabla_{\boldsymbol{p}} U = -\nabla_{\boldsymbol{p}} \log p(\boldsymbol{p}) - \nabla_{\boldsymbol{p}} \log p(\boldsymbol{D} | \boldsymbol{\Theta}, \tau(\boldsymbol{W}, \boldsymbol{p})) \tag{54.32}$$

and

$$\nabla_{\boldsymbol{\Theta}} U = -\nabla_{\boldsymbol{\Theta}} \log p(\boldsymbol{\Theta}) - \nabla_{\boldsymbol{\Theta}} \log p(\boldsymbol{D} | \boldsymbol{\Theta}, \tau(\boldsymbol{W}, \boldsymbol{p})). \tag{54.33}$$

For $\boldsymbol{W}$, the variational posterior $q_\phi(\boldsymbol{W} \mid \boldsymbol{p})$ approximates $p(\boldsymbol{W} \mid \boldsymbol{p}, \boldsymbol{\Theta}, \boldsymbol{D})$. Specifically, a neural network $\mu_\phi(\boldsymbol{p})$ maps the node potentials to edge logits, which are converted to independent Bernoulli probabilities:

$$q_\phi(\boldsymbol{W} \mid \boldsymbol{p}) = \prod_{i \neq j} \mathrm{Ber}(W_{ij} \mid \sigma(\mu_\phi(\boldsymbol{p})_{ij}))\,. \tag{54.34}$$

Similar to DECI, we can derive the corresponding *evidence lower bound* (ELBO) to train the approximate posterior.

$$\mathrm{ELBO}(\phi) = \mathbb{E}_{q_\phi(\boldsymbol{W}|\boldsymbol{p})} \left[\log p(\boldsymbol{D}, \boldsymbol{p}, \boldsymbol{\Theta} \mid \boldsymbol{W})\right] - \mathrm{D}_{\mathrm{KL}} \left[q_\phi(\boldsymbol{W} \mid \boldsymbol{p}) \| p(\boldsymbol{W})\right]. \tag{54.35}$$

The following algorithm describes the procedure of BayesDAG:

BayesDAG extends DECI's Bayesian treatment of static causal discovery, but many real-world systems are temporal, a setting that neither method handles. The next section adapts these causal-discovery ideas to temporal models while retaining structural-identifiability guarantees.

54.4 Bayesian Deep Learning for Causality with Temporal Data

The causal mechanisms underlying many real-world systems evolve over time. In domains such as climate science, finance, neuroscience, and healthcare, observations arrive as multivariate time series, and causal influence may act both within and across time steps. Learning these relations from observational data introduces three challenges beyond the static setting of Section 54.3: mechanisms may be nonlinear, effects may be instantaneous at the sampling resolution, and the noise distribution may depend on the system's history.

Algorithm 38: BayesDAG: SG-MCMC + VI Inference

Input : dataset $\boldsymbol{D}$; priors $p_\tau(\boldsymbol{W}, \boldsymbol{p})$, $p(\boldsymbol{\Theta})$; SG-MCMC sampler $\mathcal{S}$; sampler hyperparameters Ψ; neural network $\mu_\phi(\cdot)$; training iterations T
Output: samples $\{\boldsymbol{\Theta}, \boldsymbol{p}\}$ and variational posterior q_ϕ
initialize $\boldsymbol{\Theta}^{(0)}, \boldsymbol{p}^{(0)}, \phi$
for $t = 1$ **to** T **do**
  sample $\boldsymbol{W}^{(t-1)} \sim q_\phi(\boldsymbol{W} \mid \boldsymbol{p}^{(t-1)})$
  evaluate $\nabla_{\boldsymbol{p},\boldsymbol{\Theta}} U$ (Equations 54.32 and 54.33) with $\boldsymbol{\Theta}^{(t-1)}, \boldsymbol{p}^{(t-1)}, \boldsymbol{W}^{(t-1)}$
  $\boldsymbol{\Theta}^{(t)}, \boldsymbol{p}^{(t)} \leftarrow \mathcal{S}(\nabla_{\boldsymbol{p},\boldsymbol{\Theta}} U; \Psi)$
  if *storing condition met* **then**
    append $\{\boldsymbol{p}^{(t)}, \boldsymbol{\Theta}^{(t)}\}$ to samples
  maximise the ELBO (Equation 54.35) with respect to ϕ using $\boldsymbol{p}^{(t)}, \boldsymbol{\Theta}^{(t)}$

Classical *Granger causality* [658] identifies lagged predictive dependencies by asking whether one series' past improves prediction of another. Classical Granger models focus on lagged effects and commonly rely on linearity and stationarity, so they do not directly represent instantaneous structure or history-dependent noise. DYNOTEARS [1285] adds a differentiably learned instantaneous DAG to a vector-autoregressive model, but retains linear mechanisms and additive, history-independent errors. *Rhino* [647] addresses all three challenges with a nonlinear temporal SEM and a variational posterior over graph structures.

Continuous-time methods address a complementary difficulty: irregular observations may not admit a natural fixed sampling interval. After developing discrete-time Rhino, we therefore briefly introduce *SCOTCH* [1701], which combines a latent stochastic differential equation (SDE) with variational inference over causal graphs and latent trajectories.

54.4.1 Rhino: Deep Causal Temporal Relationship Learning with History-Dependent Noise

In multivariate time series, coarse sampling can aggregate events between observations and make their effects appear instantaneous. At the same time, the magnitude and shape of the noise may change with the past. For example, an announcement made after a stock market closes may have a nonlinear, apparently instantaneous effect at reopening and may also change subsequent volatility. Rhino extends DECI to this setting by combining flexible neural mechanisms, a temporal causal graph, and a conditional noise model whose distribution depends on lagged observations. It is trained with a DECI-style variational objective and retains structural-identifiability guarantees in this history-dependent regime.

Rhino model formulation

We first extend the static SEM notation to the temporal regime. Let $\{\boldsymbol{X}_t\}_{t=0}^{T}$ be a discrete-time, D-variate process with $\boldsymbol{X}_t = (X_t^1, \ldots, X_t^D)^\top$, where T is the final sequence index. Let K be the maximum lag and let $\boldsymbol{G}_{0:K} = \{\boldsymbol{G}_\tau\}_{\tau=0}^{K}$ denote the stationary temporal-graph template. For $\tau \geq 1$, $(\boldsymbol{G}_\tau)_{ji} = 1$ encodes the lagged edge $X_{t-\tau}^j \to X_t^i$, while $\boldsymbol{G}_0$ collects instantaneous edges among $\{X_t^i\}_{i=1}^{D}$ at a fixed time. Denote the corresponding parent sets by

$$\mathbf{Pa}^i_{\boldsymbol{G}}(<t) \;=\; \{X^j_{t-\tau} : (\boldsymbol{G}_\tau)_{ji} = 1,\ \tau \in \{1, \ldots, K\}\}, \qquad \mathbf{Pa}^i_{\boldsymbol{G}}(t) \;=\; \{X_t^j : (\boldsymbol{G}_0)_{ji} = 1\},$$

a temporal SEM specifies, for each i and t,

$$X_t^i \;=\; f_{i,t}\big(\mathbf{Pa}^i_{\boldsymbol{G}}(<t),\, \mathbf{Pa}^i_{\boldsymbol{G}}(t),\, \epsilon_t^i\big), \tag{54.36}$$

with mutually independent exogenous noises ϵ_t^i across nodes and times. The mechanism $f_{i,t}$ respects the template: its derivative with respect to $X_{t-\tau}^j$ is zero whenever $(\boldsymbol{G}_\tau)_{ji} = 0$. Under causal stationarity, the mechanisms and $\boldsymbol{G}_{0:K}$ do not change with t. The template $\boldsymbol{G}_{0:K}$ should not be confused with the *time-unrolled graph*: instantiating it over $t = 0, \ldots, T$ produces a graph on the individual nodes $\{X_t^i\}_{i,t}$. Lagged edges in this unrolled graph always point forward in time, and the full graph is a DAG provided that the instantaneous template $\boldsymbol{G}_0$ is itself a DAG. A linear special case is the structural vector autoregressive (VAR) model

$$X_t^i = \beta^i + \sum_{\tau=0}^{K} \sum_{j=1}^{D} B_{\tau,ji} X_{t-\tau}^j + \epsilon_t^i, \tag{54.37}$$

where $\boldsymbol{B}_\tau$ is the weighted adjacency matrix at lag τ. Its structural identifiability follows under assumptions such as non-Gaussian errors or equal error variances [808, 1285], as in the static setting.

Rhino extends the DECI model in Section 54.3.1 to this temporal setting. Its structural equation model retains an additive decomposition while allowing nonlinear dependencies, instantaneous effects, and history-dependent noise:

$$X_t^i \;=\; f_i\big(\mathbf{Pa}_{\boldsymbol{G}}^i(<t),\, \mathbf{Pa}_{\boldsymbol{G}}^i(t)\big) \;+\; g_i\big(\mathbf{Pa}_{\boldsymbol{G}}^i(<t),\, \epsilon_t^i\big), \tag{54.38}$$

where f_i is a differentiable nonlinear causal mechanism and g_i is a differentiable, history-dependent transform that is invertible in its second argument, ϵ_t^i. Compared with the additive noise models used in DECI and other temporal SEMs [1285, 1317], this formulation allows the conditional noise distribution itself to vary with the lagged parents.

The masks enforce the template: f_i may depend only on its lagged and instantaneous parents, whereas g_i may depend only on lagged parents. In practice, f_i uses a masked, weight-sharing neural architecture similar to Equation 54.13:

$$f_i\big(\mathbf{Pa}_{\boldsymbol{G}}^i(<t), \mathbf{Pa}_{\boldsymbol{G}}^i(t)\big) = \zeta_i\left(\sum_{\tau=0}^{K}\sum_{j=1}^{D} G_{\tau,ji}\, \ell_{\tau j}(X_{t-\tau}^j)\right), \tag{54.39}$$

where ζ_i and $\ell_{\tau j}$ are neural networks. For efficiency, weights are shared across nodes and lags through $\zeta_i(\cdot) = \zeta(\cdot, \boldsymbol{u}_{0,i})$ and $\ell_{\tau j}(\cdot) = \ell(\cdot, \boldsymbol{u}_{\tau,j})$, with trainable embeddings $\boldsymbol{u}_{\tau,i}$ for node i at lag τ. This generalises DECI's weight-sharing parameterisation to temporal graphs.

The design of g_i must balance flexibility with tractability of the likelihood. Rhino uses a *conditional spline flow* [470, 1304, 1635], with base Gaussian noise ϵ_t^i for all t and i. The spline parameters are predicted by a hypernetwork whose inputs are the lagged parents $\mathbf{Pa}_{\boldsymbol{G}}^i(<t)$, again in the masked, weight-sharing style of (54.39) but with $\tau \in [1, K]$ only (instantaneous parents excluded). Because g_i is invertible in its second argument, the noise likelihood conditional on lagged parents is tractable:

$$p_{g_i}\big(z_t^i \mid \mathbf{Pa}_{\boldsymbol{G}}^i(<t)\big) = p_\epsilon\big(g_i^{-1}\big(\mathbf{Pa}_{\boldsymbol{G}}^i(<t), z_t^i\big)\big) \left|\frac{\partial g_i^{-1}\big(\mathbf{Pa}_{\boldsymbol{G}}^i(<t), z_t^i\big)}{\partial z_t^i}\right|, \tag{54.40}$$

where $z_t^i = g_i(\mathbf{Pa}_{\boldsymbol{G}}^i(<t), \epsilon_t^i)$ and p_ϵ is the base Gaussian density. Thus, the conditional distribution of X_t^i can depend flexibly on the system's history while retaining a tractable likelihood.

Variational inference for Rhino

Rhino adopts the same Bayesian perspective as DECI: rather than outputting a single graph, it approximates a posterior over temporal-graph templates using the variational framework reviewed in Chapter 12. For N observed multivariate time series $\{\boldsymbol{X}_{0:T}^{(1)}, \ldots, \boldsymbol{X}_{0:T}^{(N)}\}$ and model parameters θ, the joint distribution is

$$p\big(\boldsymbol{X}_{0:T}^{(1)}, \ldots, \boldsymbol{X}_{0:T}^{(N)}, \boldsymbol{G}_{0:K}\big) \;=\; p(\boldsymbol{G}_{0:K}) \prod_{n=1}^{N} p_\theta\big(\boldsymbol{X}_{0:T}^{(n)} \mid \boldsymbol{G}_{0:K}\big). \tag{54.41}$$

Once fitted, $p(\boldsymbol{G}_{0:K} \mid \boldsymbol{X}_{0:T}^{(1)}, \ldots, \boldsymbol{X}_{0:T}^{(N)})$ quantifies uncertainty over lagged and instantaneous causal relationships.

Graph prior. The design of $p(\boldsymbol{G}_{0:K})$ balances structural constraints with flexibility. Following the smooth DAG formulation used in DECI, we combine three ingredients: (i) a DAG constraint, (ii) a sparsity prior, and (iii) optional domain-specific information:

$$p(\boldsymbol{G}_{0:K}) \;\propto\; \exp\Big(-\lambda_s \|\boldsymbol{G}_{0:K}\|_F^2 - \rho\, h^2(\boldsymbol{G}_0) - \alpha\, h(\boldsymbol{G}_0) - \lambda_p \|\boldsymbol{G}_{0:K} - \boldsymbol{G}_{0:K}^p\|_F^2\Big), \tag{54.42}$$

where $h(\boldsymbol{G}_0) = \operatorname{tr}(e^{\boldsymbol{G}_0 \odot \boldsymbol{G}_0}) - D$ is the smooth DAG penalty and $\boldsymbol{G}_{0:K}^p$ is an optional prior template encoding domain knowledge. The lagged matrices $\boldsymbol{G}_{1:K}$ automatically respect temporal direction in the unrolled graph, so the acyclicity penalty applies only to $\boldsymbol{G}_0$. Here, λ_s and λ_p control sparsity and prior strength, while $\alpha \geq 0$ and $\rho > 0$ are the augmented-Lagrangian coefficients updated as in DECI.

Variational objective. As in DECI, the exact posterior over $\boldsymbol{G}_{0:K}$ is intractable. Rhino therefore uses an edge-factorised Bernoulli approximation $q_\phi(\boldsymbol{G}_{0:K})$ and maximises the corresponding evidence lower bound (ELBO):

$$\log p_\theta\Big(\boldsymbol{X}_{0:T}^{(1)}, \dots, \boldsymbol{X}_{0:T}^{(N)}\Big) \;\geq\; \underbrace{\mathbb{E}_{q_\phi(\boldsymbol{G}_{0:K})}\Big[\sum_{n=1}^{N} \log p_\theta(\boldsymbol{X}_{0:T}^{(n)} \mid \boldsymbol{G}_{0:K}) + \log p(\boldsymbol{G}_{0:K})\Big] + H\big(q_\phi(\boldsymbol{G}_{0:K})\big)}_{\mathrm{ELBO}(\theta,\phi)}, \tag{54.43}$$

where $H(\cdot)$ denotes entropy.

From the causal Markov assumption and autoregressive structure of Rhino, the sequence likelihood factorises as

$$\log p_\theta(\boldsymbol{X}_{0:T}^{(n)} \mid \boldsymbol{G}_{0:K}) = \sum_{t=0}^{T}\sum_{i=1}^{D} \log p_\theta\big(X_t^{i,(n)} \mid \mathbf{Pa}_{\boldsymbol{G}}^i(<t), \mathbf{Pa}_{\boldsymbol{G}}^i(t)\big), \tag{54.44}$$

and from Rhino's SEM (Equation 54.38) we obtain

$$\log p_\theta\big(X_t^{i,(n)} \mid \mathbf{Pa}_{\boldsymbol{G}}^i(<t), \mathbf{Pa}_{\boldsymbol{G}}^i(t)\big) = \log p_{g_i}\big(z_t^{i,(n)} \mid \mathbf{Pa}_{\boldsymbol{G}}^i(<t)\big), \tag{54.45}$$

where $z_t^{i,(n)} = X_t^{i,(n)} - f_i(\mathbf{Pa}_{\boldsymbol{G}}^i(<t), \mathbf{Pa}_{\boldsymbol{G}}^i(t))$ and p_{g_i} is the history-conditioned flow density defined in Equation 54.40. The parameters θ and ϕ are learned by maximising this ELBO with a Gumbel–Softmax gradient estimator and the same augmented-Lagrangian updates as DECI, driving the instantaneous subgraph $\boldsymbol{G}_0$ towards a DAG. Once trained, Rhino provides a variational posterior over the stationary template $\boldsymbol{G}_{0:K}$. Each template sample specifies both lagged and instantaneous relations and induces a time-unrolled graph over the observed sequence of length $T+1$. Because Rhino also learns the functional mechanisms, it supports temporal causal inference using the simulation strategy introduced in Section 54.3.1.

Structural identifiability of Rhino

Like DECI's recovery result in Section 54.3.1, Rhino provides two guarantees: (i) structural identifiability of the temporal SEM in (54.38)–(54.40), and (ii) validity of the variational objective used for discovery. Under the stated assumptions, these results recover the true temporal-graph template and data-generating distribution in the infinite-data limit. The full proofs are given by Gong et al. [647].

The central challenge is to ensure that two distinct temporal templates cannot induce the same observational distribution.

Theorem 54.4 (Identifiability of Rhino SEMs [647])**.** *Assume that the stationary SEM in* (54.38) *satisfies the causal Markov property, causal minimality, causal sufficiency, acyclicity of* $\boldsymbol{G}_0$*, and induces a proper joint density. Suppose further that: (1) all functions and induced densities are third-order differentiable; (2)* f_i *is nonlinear and, for every instantaneous parent* $X_t^j \in \mathbf{Pa}_{\boldsymbol{G}}^i(t)$*, its dependence on* X_t^j *with the other inputs fixed is non-invertible; and (3) the second derivative of* $\log p_{g_i}(g_i(\epsilon_t^i) \mid \mathbf{Pa}_{\boldsymbol{G}}^i(<t))$ *with respect to* ϵ_t^i *is zero only at isolated points. Then the SEM in* (54.38) *is structurally identifiable for both bivariate and multivariate time series.*

Proof logic. The proof proceeds in three steps. First, an identifiable source model fixes the initial segment, while the causal Markov and minimality assumptions distinguish templates that disagree only on lagged parents. Conditioning on the observed history then reduces the remaining directional ambiguity to the instantaneous parent–child relations. Second, Rhino is embedded in a history-dependent post-nonlinear class; after applying its invertible outer transformations, the conditional bivariate problem reduces to an additive-noise argument of the type used by Peters et al. [1317]. The differentiability and density conditions exclude the exceptional cases in which both instantaneous directions generate the same conditional distribution. Finally, the pairwise result is lifted to the multivariate setting using identifiable functional model class arguments [1316], with the bivariate condition required for each instantaneous parent.

The standard history-independent ANM is recovered by setting $g_i(\mathbf{Pa}_{\boldsymbol{G}}^i(<t), \epsilon_t^i) = \epsilon_t^i$. Rhino therefore extends the static ANM result: the restriction above identifies each instantaneous direction, while the invertible transform g_i permits the noise distribution to vary with lagged history.

Validity of variational objective

Rhino's variational objective mirrors DECI's ELBO. Under correct specification and the assumptions of Theorem 54.4, maximising it recovers the true temporal template in the infinite-data limit.

Theorem 54.5 (Validity of the variational objective [647])**.** *Under the conditions of Theorem 54.4 and assuming no model misspecification, any maximiser* $(\theta', q'_\phi(\boldsymbol{G}_{0:K}))$ *of the ELBO in* (54.43) *with infinite data satisfies*

$$q'_\phi(\boldsymbol{G}_{0:K}) = \delta(\boldsymbol{G}_{0:K} = \boldsymbol{G}^\star_{0:K}) \quad \textit{and} \quad p_{\theta'}(\boldsymbol{X}_{0:T} \mid \boldsymbol{G}^\star_{0:K}) = p(\boldsymbol{X}_{0:T} \mid \boldsymbol{G}^\star_{0:K}),$$

where $\boldsymbol{G}^\star_{0:K}$ *is the ground-truth temporal template and the right-hand side is the true data-generating distribution.*

Rhino thus extends DECI to discrete-time systems with lagged and instantaneous effects and history-dependent noise. Its fixed temporal grid, however, is unsuitable when observations arrive irregularly or the underlying dynamics are intrinsically continuous.

54.4.2 Continuous-time extension of Rhino: SCOTCH

Physical, financial, and epidemiological systems are often observed at irregular intervals, for which imposing a fixed discrete-time grid can obscure the underlying dynamics or induce spurious dependencies. Neural ordinary differential equation methods such as NGM [113, 272] model a deterministic mean process and extract its graph from a neural network, but NGM assumes a single observed series with a multivariate Gaussian law. This restricts it to comparatively simple stochastic dynamics.

SCOTCH [1701] instead models intrinsic process noise with a latent SDE and separates it from observation noise:

$$\begin{aligned} d\boldsymbol{Z}_t &= \boldsymbol{f}_\theta(\boldsymbol{Z}_t, \boldsymbol{G})\, dt + \boldsymbol{g}_\theta(\boldsymbol{Z}_t, \boldsymbol{G})\, d\boldsymbol{W}_t, \\ \boldsymbol{X}_{t_i} &= \boldsymbol{Z}_{t_i} + \boldsymbol{\epsilon}_{t_i}, \end{aligned}$$

where $\boldsymbol{Z}_t$ is the latent state, $\boldsymbol{W}_t$ is Brownian motion, and $\boldsymbol{\epsilon}_{t_i}$ is independent measurement noise. The graph-masked drift $\boldsymbol{f}_\theta$ captures deterministic dependencies, while the diagonal diffusion $\boldsymbol{g}_\theta$ captures state-dependent stochasticity. With no instantaneous effects, an Euler discretisation of this model has the same residual form as Rhino, making SCOTCH its continuous-time analogue.

SCOTCH uses variational inference to approximate both the graph posterior $q_\phi(\boldsymbol{G})$ and, for each observed series, a posterior SDE over latent trajectories. Its ELBO combines the expected observation likelihood, a path-space term comparing the posterior and prior dynamics, and the graph divergence $\mathrm{D}_{\mathrm{KL}}[q_\phi(\boldsymbol{G}) \| p(\boldsymbol{G})]$. This extends the graph-level variational machinery of Chapter 12 to continuous latent paths.

Together, the four methods in this chapter illustrate complementary uses of Bayesian deep learning for causality: DECI and BayesDAG address static discovery, Rhino extends discovery to discrete-time systems with history-dependent noise, and SCOTCH handles continuous-time latent dynamics and irregular observations.

54.5 Further advances in causal modelling

The preceding sections focused on Bayesian causal discovery and inference for static, discrete-time, and continuous-time data. Causal reasoning is also increasingly relevant in the era of foundation models: it underpins the ability to anticipate the consequences of actions, which lies at the core of *world modelling*. We conclude with two complementary directions that use transformer architectures to scale causal modelling. The first recasts SCM generation as a fixed-point problem on causally ordered variables [1466]; the second amortises treatment-effect estimation across datasets [1848]. We then close the section with pointers to other active research relating to causal foundation models. These examples illustrate recent advances rather than providing a comprehensive survey.

Fixed-point causal generative modelling. DECI and BayesDAG represent the causal graph explicitly, through a variational distribution over adjacency matrices or a node-potential parameterisation. Fixed-point causal models take a different route. Given a topological ordering represented by a permutation P, they define a joint mechanism $\boldsymbol{H}$ on the causally ordered variables and their exogenous noise. Under suitable architectural conditions, solving the resulting fixed-point problem generates the same observations as the corresponding SCM. The graph therefore need not be explicitly constructed or regularised while learning the mechanisms, although a valid causal ordering is still required.

The fixed-point approach, called FiP, separates learning into two stages: (i) an amortised model predicts a valid topological ordering from observational data, and (ii) a transformer-based model learns $\boldsymbol{H}$ conditional on that ordering. Its attention architecture enforces a lower-triangular dependency pattern, so each variable can depend only on variables preceding it in the proposed order. This removes the need for an acyclicity penalty in the

mechanism-learning stage. The fixed-point formalism is equivalent to a classical SCM, and Scetbon et al. [1466] establish conditions for unique recovery of the SCM when the topological ordering is given.

Causal inference with attention. CInA (*Causal Inference with Attention*) instead addresses treatment-effect estimation. Its starting point is optimal covariate balancing: non-negative weights are assigned to observational units so that the weighted covariate distributions of the treated and control groups are similar. Under the usual causal-identification assumptions, differences between weighted outcomes can then estimate the average treatment effect (ATE). Such weights can be obtained through a convex support-vector-machine (SVM) formulation [1591].

Zhang et al. [1848] establish a primal–dual connection between this optimal-balancing objective and self-attention, and use it to design CInA. The interpretation is specific to their transformer parameterisation, training objective, and causal assumptions: it is not an inherent property of arbitrary attention weights. Quantities produced by CInA's final layer are transformed and normalised into balancing weights. Training across multiple unlabelled observational datasets amortises this computation, allowing the trained model to estimate ATEs on unseen datasets without per-dataset retraining.

FiP and CInA thus use transformers for different causal tasks. FiP learns a causal generative model after inferring an ordering, whereas CInA learns to produce balancing weights for treatment-effect estimation. Both illustrate how computation can be amortised across datasets, but neither removes the structural or identification assumptions required for causal conclusions.

Other foundation models with causal considerations. Beyond these two methods, the interplay between causality and foundation models is being explored from several aspects, and we highlight two active groups of directions.

The first treats causal reasoning as one of the important general reasoning capabilities [393, 1793] of foundation models. The question is whether large foundation models can perform classical causal tasks, such as causal discovery and causal inference, from their pretrained knowledge or in context. A growing body of evaluations probes this ability [846, 885], ranging from causal-discovery benchmarks in different domains [998, 1641] to broader critiques arguing that such models often recite memorised causal facts rather than reason causally [1836, 1841]. These efforts sit alongside related evaluations of tabular-data and graph reasoning [1642], and are increasingly viewed as part of assessing a foundation model's reasoning capabilities more generally.

The second angle concerns *world models*, which aim to go beyond pattern recognition by capturing how actions lead to consequences. The specific learning methods vary widely, and the tasks and evaluations can only approximate the true underlying dynamics of the world. Causal structure is especially important in two settings: agentic foundation models and scientific foundation models. In *physical intelligence*, where models are used to control embodied agents, correctly modelling the effect of actions becomes essential [690, 1734]; a similar need for causal, verifiable state transitions arises for long-horizon digital agents [633, 1778]. In the sciences, for example drug discovery and biological design, the value of a world model lies in predicting the outcome of interventions such as candidate treatments [349, 1420, 1598], where causal validity directly determines its usefulness of treatment design.

Chapter 55

Credal BDL and Imprecise Probability in BDL

By now, the reader should be convinced of the predictive uncertainty quantification capabilities of Bayesian Neural Networks (BNNs). While that is true, classical Bayesian Deep Learning (BDL) is not able to distinguish properly between different *sources* of predictive uncertainty. In this chapter, we present Credal Bayesian Deep Learning (CBDL) with the aim to overcome such a shortcoming [236].

Intuitively, CBDL allows us to train an (uncountably) infinite ensemble of BNNs (explored in Part IV of this book), using only finitely many elements. This is possible thanks to prior and likelihood Finitely Generated Credal Sets (FGCSs), a concept from the Imprecise Probability literature. Intuitively, convex combinations of a finite collection of prior-likelihood pairs are able to represent infinitely many such pairs. After training, CBDL outputs a set of posteriors on the parameters of the neural network. At inference time, such posterior set is used to derive a set of (posterior) predictive distributions that is in turn utilised to distinguish between (predictive) Aleatoric and Epistemic Uncertainties, and to quantify them. The predictive set also produces either (i) a collection of outputs enjoying desirable probabilistic guarantees, or (ii) the single output that is deemed the best in the "worst case scenario", that is, the one having the highest predictive lower probability – another Imprecise-Probabilistic concept. CBDL is more robust than single BNNs to prior and likelihood misspecification, and to distribution shift. Furthermore, it is better at quantifying and disentangling different types of (predictive) uncertainties than single BNNs and ensemble of BNNs.

55.1 Introduction

As we have seen in previous chapters, the Bayesian paradigm provides a rigorous framework to analyse and train uncertainty-aware neural networks, and more generally to support the development of learning algorithms [854]. In addition, it overcomes some of the drawbacks of other deep learning models, namely that they are prone to overfitting, which adversely affects their generalisation capabilities, and that they tend to be overconfident about their predictions when they provide a confidence interval. BNNs, though, are trained using a single prior, which may still suffer from miscalibration and robustness issues [1013].

In this chapter, we introduce Credal Bayesian Deep Learning (CBDL) [236], a procedure that draws on concepts from the Imprecise Probability (IP) literature [62, 235, 237, 1636, 1699], and that can be used for both classification and regression tasks. It captures the ambiguity that the designer faces when selecting which prior to choose for the parameters of a neural network, and which likelihood distribution to choose for the training data at hand.

CBDL can be thought of as a neural network (NN) trained using prior and likelihood Finitely Generated Credal Sets (FGCSs), $\mathcal{P}_{\text{prior}}$ and $\mathcal{P}_{\text{lik}}$, respectively.[1] They are convex sets of probability measures having finitely many extreme elements (the elements that cannot be written as a convex combination of one another), $\text{ex}\mathcal{P}_{\text{prior}}$ and $\text{ex}\mathcal{P}_{\text{lik}}$, respectively (see also Remark 55.2). A very simple example of a prior FGCS $\mathcal{P}_{\text{prior}}$ is the collection of all the convex combinations of two one-dimensional Normal distributions $\mathcal{N}(\mu_1, \sigma_1^2)$ and $\mathcal{N}(\mu_2, \sigma_2^2)$, i.e.,

$$\mathcal{P}_{\text{prior}} = \{P : p = \beta\mathcal{N}(\mu_1, \sigma_1^2) + (1-\beta)\mathcal{N}(\mu_2, \sigma_2^2), \text{ for some } \beta \in [0,1]\}, \tag{55.1}$$

where we denoted by p the probability density function (pdf) of $P \in \mathcal{P}_{\text{prior}}$. Consequently, in this toy example we

[1] Intuitively, the larger these credal sets, the higher prior and likelihood ambiguity the user faces.

have that $\text{ex}\mathcal{P}_{\text{prior}} = \{\mathcal{N}(\mu_1, \sigma_1^2), \mathcal{N}(\mu_2, \sigma_2^2)\}$.[2] FGCSs are further examined in Section 55.2.2.

Given the use of finitely generated credal sets, CBDL can also be seen as a non-condensed (uncountably) infinite ensemble of BNNs – each BNN corresponding to a pair (P, L) of prior P from the prior FGCS $\mathcal{P}_{\text{prior}}$ and likelihood L from the likelihood FGCS $\mathcal{P}_{\text{lik}}$. Such infinite ensemble, though, is carried out using only finitely many elements, that is, only pairs $(P^{\text{ex}}, L^{\text{ex}})$ of components of the sets of extreme elements $\text{ex}\mathcal{P}_{\text{prior}}$ and $\text{ex}\mathcal{P}_{\text{lik}}$. This is because every element P of $\mathcal{P}_{\text{prior}}$ can be obtained from a convex combinations of the elements of $\text{ex}\mathcal{P}_{\text{prior}}$, and similarly for the likelihood FGCS. After training, CBDL produces a posterior FGCS $\mathcal{P}_{\text{post}}$ on the parameters of the neural network. At inference time, $\mathcal{P}_{\text{post}}$ is used to derive an FGCS $\mathcal{P}_{\text{pred}}$ of posterior predictive distributions; that is, given a new input, a set of plausible distributions over the space of outputs.[3] In turn, $\mathcal{P}_{\text{pred}}$ generates a set of outputs – or a single output, depending on the user's needs – that enjoys desirable probabilistic guarantees, even when the elements of credal set $\mathcal{P}_{\text{pred}}$ are approximated, e.g. using variational inference. CBDL also gives a way of quantifying and disentangling different types of uncertainties within $\mathcal{P}_{\text{pred}}$.

A credal set approach is used to overcome some of the drawbacks of single BNNs. In particular, CBDL allows to counter the criticism to the practice in (standard) Bayesian statistics of (i) using a single, arbitrary prior to represent the initial state of ignorance of the agent, (ii) using non-informative priors to model ignorance, and (iii) using a single, arbitrary likelihood to represent the agent's knowledge about the sampling model.[4] As a consequence, credal sets make the analysis more robust to prior and likelihood misspecification. In addition, they make it possible to quantify and distinguish between epistemic and aleatoric uncertainties (EU and AU, respectively). This is desirable in light of several areas of recent Machine Learning (ML) research, such as Bayesian deep learning [415, 879], adversarial example detection [1526], and data augmentation in Bayesian classification [870].

AU refers to the uncertainty that is inherent to the data generating process; as such, it is irreducible. Think, for example, of a coin toss. No matter how many times the coin is tossed, the stochastic variability of the experiment cannot be eliminated. EU, instead, refers to the lack of knowledge about the data generating process; as such, it is reducible. It can be lessened on the basis of additional data. For example, after only a few tosses, we are unable to gauge whether a coin is biased or not, but if we repeat the experiment long enough, this type of uncertainty vanishes. We note in passing that predictive EU cannot be adequately captured using a single BNN [512, 804]. One reason for this is that selecting a unique prior and a unique likelihood implicitly assumes perfect knowledge around the true prior and the true data generating process. Another, more subtle one, is explored in Section 55.2.1. Methods that disentangle between the two types of (predictive) uncertainties that are based on a single distribution are ad-hoc, but are not theoretically well-justified. EU can be typically reduced by retraining the model using an augmented training set [1043] (e.g. via semantic preserving transformations [876], Puzzle Mix [888], etc.) On the other hand, since AU is irreducible, there is an increasing need for ML techniques that are able to detect and flag its excess, so that the user can "proceed with caution".

Remark 55.1. *EU should not be confused with the concept of epistemic probability [384, 385, 1699]. In the subjective probability literature, epistemic probability can be captured by a single distribution. Its best definition can be found in Walley [1699, Sections 1.3.2 and 2.11.2]. There, the author specifies how epistemic probabilities model logical or psychological degrees of partial belief of the agent. We remark, though, how de Finetti and Walley work with finitely additive probabilities, while in this chapter we use countably additive probabilities.*

The motivation for working with credal sets is threefold: (i) it allows to be robust against prior and likelihood misspecification; (ii) unlike when using a single probability distribution, it permits to represent ignorance in the sense of lack of knowledge; (iii) it allows to quantify and disentangle between EU and AU. A more in-depth discussion can be found in Caprio et al. [236, Appendix A]. In addition, we point out that despite a hierarchical Bayesian model (HBM) approach seems to be a viable alternative to one based on credal sets, these latter are better justified philosophically, and do not suffer from the same theoretical shortcomings of HBM procedures [130, 804, 840, 1699]. A more detailed explanation can be found in Caprio et al. [236, Appendix B]. Let us mention that, in Section 55.4, CBDL is only related with Bayesian techniques because a comparison with non-Bayesian ones is either difficult to justify, or unfair. Indeed, the initial knowledge component, captured by the prior distribution (or the set of priors, in our case), has no direct comparisons with other approaches. Take conformal prediction [94, 625, 1498, 1689] for instance. It is a model-free method, which means that no prior knowledge around the experiment at hand is required. The only choice the user makes is which non-conformity score to

[2] To be more precise, the elements of $\text{ex}\mathcal{P}_{\text{prior}}$ are the two *probability measures* whose pdf's are the two Normals $\mathcal{N}(\mu_1, \sigma_1^2)$ and $\mathcal{N}(\mu_2, \sigma_2^2)$.

[3] As we shall see in Section 55.3.1, since computing the posteriors and the predictive distributions is oftentimes an intractable problem, the elements of $\mathcal{P}_{\text{post}}$ and of $\mathcal{P}_{\text{pred}}$ are approximated using Variational Inference (VI, studied in detail in Part III of this book). As a consequence, we denote the VI-approximated posterior and predictive credal sets as $\check{\mathcal{P}}_{\text{post}}$ and $\hat{\mathcal{P}}_{\text{pred}}$, respectively.

[4] Criticisms (i) and (iii) are also pointed out in Manchingal and Cuzzolin [1127, Section 2.2].

use. How would this approach be compared with the choice of different priors, which conveys the idea that the user knows that a true distribution over the space of neural network parameters exists, but it is not fully known? In addition, despite the ubiquitous claim that conformal prediction (CP) is an uncertainty *quantification* tool, it is actually not. CP is an uncertainty *representation* tool. Indeed, CP *represents* uncertainty via the conformal prediction region (on the other hand, in this chapter, uncertainty is represented via credal sets). It does not quantify it: there is no real value attached to any kind of predictive uncertainty (aleatoric or epistemic). Some claim that the size of the conformal prediction region quantifies the uncertainty, but even in that case, it is unable to distinguish between AU and EU. Indeed, the size is a positive function of both: it increases as both increase, and hence it cannot be used to distinguish between the two.

We summarise the content of this chapter next: (1) We present CBDL, and develop the theoretical tools and the algorithm required to use it in practice. (2) We show that a CBDL approach is more robust than single BNNs to prior and likelihood misspecification, and to distribution shifts. We also explain how, during inference, the credal set of posteriors $\mathcal{P}_{\text{post}}$ on the network parameters obtained during training is used to derive a credal set of predictive distributions $\mathcal{P}_{\text{pred}}$, and in turn a set of outcomes – or a single outcome – that enjoys probabilistic guarantees.[5] (3) We show how CBDL is better at quantifying and disentangling predictive AU and EU than single BNNs and ensemble of BNNs.[6]

Before moving on, let us add two comments. First, the proofs for the propositions in this chapter can be found in Caprio et al. [236, Appendix M], and the downstream tasks capabilities of CBDL are demonstrated in Caprio et al. [236, Section 4.2], where CBDL is applied to two safety-critical systems (motion prediction for autonomous driving and human insulin and blood glucose dynamics for artificial pancreas control). Second, while CBDL pays a computational price coming from the use of credal sets, it is able to quantify both (predictive) EU and AU, and to do so in a principled manner, unlike single and ensembles of BNNs. In addition, it requires less stringent assumptions on the nature of the prior and likelihood ambiguity faced by the agent than other imprecise-probabilities-based techniques, as explained in Caprio et al. [236, Appendices B and L]. The existing work related to the ideas introduced in the present chapter can be found in Caprio et al. [236, Section 5].

Structure of the Chapter. Section 55.2 presents the needed preliminary concepts, followed by Section 55.3 that introduces and discusses the CBDL algorithm, together with its theoretical properties. The experimental results are presented in Section 55.4. Section 55.5 concludes this chapter.

Notation Summary. In this chapter, we make extensive use of different notation and acronyms. We summarise them in Table 55.1 to help the reader navigate through them.

55.2 Background and Preliminaries

In this Section, we present the background notions that are needed to understand our main results. In Section 55.2.1 we argue why posterior predictive distributions are not able to gauge predictive Epistemic Uncertainty. Section 55.2.2 discusses (finitely generated) credal sets, upper and lower probabilities, and imprecise highest density regions. Section 55.2.3 introduces the concepts of upper and lower entropy, which are used to quantify and disentangle AU and EU. The reader familiar with these concepts can skip to Section 55.3.

55.2.1 Posterior Predictive Distributions and EU

Suppose that we collect a new (test) input x_* of interest, and that we want to predict its correct output y_*. In this case, the posterior distribution $p(\theta \mid \mathcal{D})$ over the network parameters comes to the rescue, as it allows us to compute the so-called *posterior predictive distribution*,

$$p(y_* \mid x_*, \mathcal{D}) = \int_\Theta p(y_* \mid x_*, \theta) p(\theta \mid \mathcal{D}) \mathrm{d}\theta = \mathbb{E}_{\theta \sim p(\theta \mid \mathcal{D})}[p(y_* \mid x_*, \theta)], \tag{55.2}$$

where $p(y_* \mid x_*, \theta)$ is the model distribution (i.e. the likelihood that we choose). Posterior predictive $p(y_* \mid x_*, \mathcal{D})$ tells us "how likely" output y_* is to be the "correct one" for input x_*, given the knowledge encapsulated in the training data $\mathcal{D}$, which enters the computation via the posterior probability $p(\theta \mid \mathcal{D})$. Oftentimes, scholars claim

[5] Once again, as we shall see in Section 55.3.1, the elements of the posterior and the predictive credal sets are approximated using Variational Inference (VI).

[6] We note in passing how recently Mucsányi et al. [1204] show that (finite) deep ensembles are the current state-of-the-art methods for quantifying and disentangling different types of uncertainties on ImageNet. The experimental results that we present show that CBDL improves on (finite) ensembles of BNNs, both in uncertainty quantification and disentanglement, and in downstream tasks performances. Together with the findings in Mucsányi et al. [1204], this is an additional argument in favour of the effectiveness of CBDL.

Notation and Acronyms	Meaning
FGCS	Finitely Generated Credal Set
CBDL	Credal Bayesian Deep Learning
Upper case Latin letters, such as P and L	Probability measures, e.g. prior or likelihood
Lower case Latin letters, such as p and l	The pdf/pmf associated to probability measures
Π	A finite collection of probability measures
$\text{Conv}(\cdot)$	The convex hull operator
Π'	The convex hull of Π, i.e. $\Pi' = \text{Conv}(\Pi)$
$\mathcal{P}_{\text{prior}}$	A prior FGCS
$\mathcal{P}_{\text{lik}}$	A likelihood FGCS
$\mathcal{P}_{\text{post}}$	A posterior FGCS
VI	Variational Inference
$\mathring{\mathcal{P}}_{\text{post}}$	Posterior FGCS with VI-approximated extrema
$\mathcal{P}_{\text{pred}}$	A predictive FGCS
$\mathring{\mathcal{P}}_{\text{pred}}$	Posterior predictive FGCS with VI-approximated extrema
$\text{ex}\mathcal{P}$	The extreme elements of an FGCS $\mathcal{P}$
BNN	Bayesian Neural Network
$\mathcal{D}$	Training dataset
EBNN	Ensemble of Bayesian Neural Networks
$x \mapsto f(x)$	Function representing the neural network architecture
$\underline{P}$, $\overline{P}$	Lower and upper probabilities, respectively
HDR	Highest Density Region
IHDR	Imprecise Highest Density Region
$IR_\alpha(\mathcal{P})$	IHDR of level α associated with FGCS $\mathcal{P}$
$H(P)$	Entropy of probability measure P
$\underline{H}(P)$, $\overline{H}(P)$	Upper and lower entropy associated with a credal set
$\text{EU}(\mathcal{P})$	Epistemic Uncertainty encoded in an FGCS $\mathcal{P}$
$\text{AU}(\mathcal{P})$	Aleatoric Uncertainty encoded in an FGCS $\mathcal{P}$
$\text{TU}(\mathcal{P})$	Total Uncertainty encoded in an FGCS $\mathcal{P}$
$\#A$	Cardinality of a generic set A
post, pred	The act of computing posterior and posterior predictive, resp.
BMA	Bayesian Model Averaging

Table 55.1: Notation and acronyms that we use throughout the chapter, together with their meaning.

that, in a Bayesian setting, the distribution on the parameters θ captures the epistemic uncertainty (EU) faced by the agent. This is a somehow agreeable premise, akin to a second-order distribution reasoning. However, if we accept this assertion, we see that EU at the predictive level is no longer quantifiable, since it is washed away by taking the expectation $\mathbb{E}_{\theta\sim p(\theta|\mathcal{D})}[\cdot]$. As we shall see later, CBDL allows us to overcome this conceptual shortcoming.

55.2.2 Imprecise Probabilities

As CBDL is rooted in the theory of imprecise probabilities (IPs), in this Section we give a gentle introduction to the IP concepts we will use throughout the chapter.

CBDL is based on the *Bayesian sensitivity analysis* (BSA) approach to IPs, that in turn is grounded in the *dogma of ideal precision* (DIP) [122], [1699, Section 5.9]. The DIP posits that in any problem there is an *ideal probability model* which is precise, but which may not be precisely known. We call this condition *ambiguity* [488, 626].

Facing ambiguity can be represented mathematically by a set $\mathcal{P}_{\text{prior}}$ of priors and a set $\mathcal{P}_{\text{lik}}$ of likelihoods that seem "plausible" or "fit" to express the agent's beliefs on the parameters of interest and their knowledge of the data generating process (DGP). Generally speaking, the farther apart the "boundary elements" of the sets (i.e., their infimum and supremum), the higher the agent's ambiguity. Of course, if $\mathcal{P}_{\text{prior}}$ and $\mathcal{P}_{\text{lik}}$ are singletons we go back to the usual Bayesian paradigm.

A procedure based on sets $\mathcal{P}_{\text{prior}}$ and $\mathcal{P}_{\text{lik}}$ yields results that are more robust to prior and likelihood misspecification than a regular Bayesian method. In the presence of prior ignorance and indecisiveness about the sampling model, it is better to give answers in the form of intervals or sets, rather than arbitrarily select a prior and a likelihood, and then update. Sets $\mathcal{P}_{\text{prior}}$ and $\mathcal{P}_{\text{lik}}$ allow to represent *indecision*, thus leading to less informative but more robust and valid conclusions.

Remark 55.2. *Throughout the chapter, we denote by $\Pi = \{P_1, \dots, P_k\}$, $k \in \mathbb{N}$, a finite set of probabilities on a generic space Ξ, such that for all $j \in \{1, \dots, k\}$, P_j cannot be written as a convex combination of the other $k-1$ components of Π. We denote by Π' its convex hull $\Pi' \equiv Conv(\Pi)$, i.e., the set of probabilities Q on Ξ that can be written as $Q(A) = \sum_{j=1}^{k} \beta_j P_j(A)$, for all $A \subset \Xi$, where the β_j's are elements of $[0, 1]$ that sum up to 1. In the literature, it is referred to as a Finitely Generated Credal Set [FGCS 341, 1017]. Notice then that the extreme elements of Π' correspond to the elements of Π, in formulas $ex\Pi' = \Pi$. Simple graphical representations of finitely generated credal sets are given in Figures 55.1 and 55.2.*

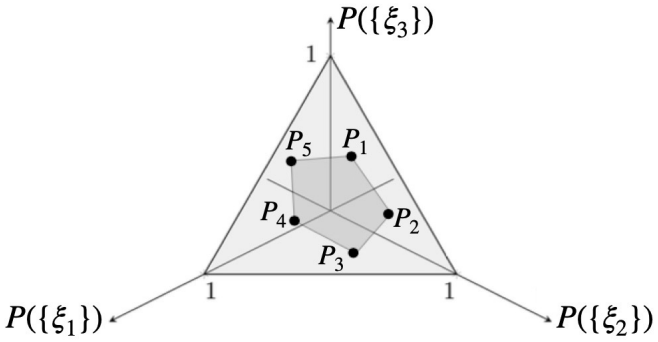


Figure 55.1: Suppose we are in a 3-class classification setting, so $\Xi = \{\xi_1, \xi_2, \xi_3\}$. Then, any probability measure P on Ξ can be seen as a probability vector. For example, suppose $P(\{\xi_1\}) = 0.6$, $P(\{\xi_2\}) = 0.3$, and $P(\{\xi_3\}) = 0.1$. We have that $P \equiv (0.6, 0.3, 0.1)^\top$. Since its elements are positive and sum up to 1, probability vector P belongs to the unit simplex, the triangle in the figure. Then, we can specify $\Pi = \{P_1, \dots, P_5\}$, and obtain as a consequence that $\Pi' = \text{Conv}(\Pi)$ is the pentagon. It is a convex polygon with finitely many extreme elements, and it is the geometric representation of a finitely generated credal set.

Let us now introduce the concepts of *lower* and *upper probabilities*. The lower probability $\underline{P}$ associated with Π is given by $\underline{P}(A) = \inf_{P\in\Pi} P(A)$, for all $A \subset \Xi$. The upper probability $\overline{P}$ associated with Π is defined as the conjugate to $\underline{P}$, that is, $\overline{P}(A) := 1 - \underline{P}(A^c) = \sup_{P\in\Pi} P(A)$,[7] for all $A \subset \Xi$. These definitions hold even if Π is not finite. Then, we have the following important result.

[7] Here, $A^c := \Xi \setminus A$.

Proposition 55.1. *$\overline{P}$ is the upper probability for Π if and only if it is also the upper probability for Π'. That is, $\overline{P}(A) = \sup_{P\in\Pi} P(A) = \sup_{P'\in\Pi'} P'(A)$, for all $A \subset \Xi$. The same holds for the lower probability.*

A version of Proposition 55.1 was proven in Dantzig [371], while a variant for finitely additive probability measures can be found in Walley [1699, Section 3.6]. A simple graphical representation of upper and lower probabilities for a set A is given in Figure 55.2.

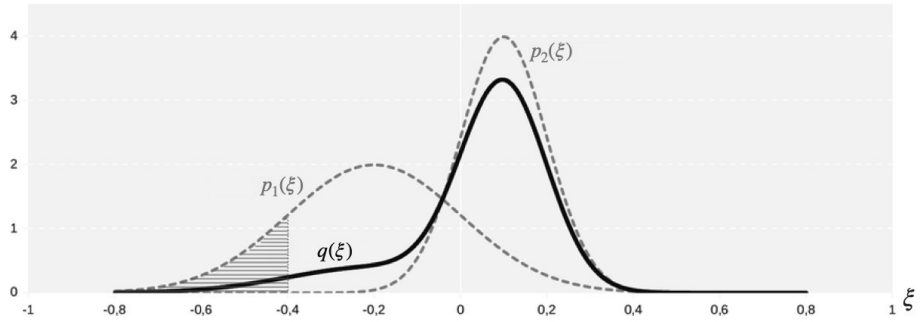


Figure 55.2: In this figure, a replica of Flint and Maré [535, Figure 1], $\Pi = \{P_1, P_2\}$, where P_1 and P_2 are two Normal distributions whose probability density functions (pdf's) p_1 and p_2 are given by the dashed curves. Their convex hull is $\Pi' = \text{Conv}(\Pi) = \{Q : Q = \beta P_1 + (1-\beta)P_2$, for all $\beta \in [0,1]\}$. The pdf q of an element Q of Π' is depicted by a solid black curve. In addition, let $A = [-0.8, -0.4]$. Then, $\underline{P}(A) = \int_{-0.8}^{-0.4} p_2(\xi)\text{d}\xi \approx 0$, while $\overline{P}(A)$ is given by the shaded area under p_1, that is, $\overline{P}(A) = \int_{-0.8}^{-0.4} p_1(\xi)\text{d}\xi$.

We now use lower probability $\underline{P}$ to define the *α-level Imprecise Highest Density Region* (IHDR), for some $\alpha \in [0,1]$.

Definition 55.1. *[329, Section 2] Let α be any value in $[0,1]$. Then, set $IR_\alpha(\Pi') \subset \Xi$ is called a $(1-\alpha)$-Imprecise Highest Density Region (IHDR) if*

1. *$\underline{P}[\{\xi \in IR_\alpha(\Pi')\}] \geq 1-\alpha$;*
2. *$\int_{IR_\alpha(\Pi')} d\xi$ is minimal. If Ξ is at most countable, we replace $\int_{IR_\alpha(\Pi')} d\xi$ with $\#IR_\alpha(\Pi')$, where $\#$ denotes the cardinality operator.*

Definition 55.1 holds also if $\Pi = \text{ex}\Pi'$ is not finite. Notice that condition 2 is needed so that $IR_\alpha(\Pi')$ is the subset of Ξ having the lowest possible cardinality, which still satisfies condition 1. By the definition of lower probability, Definition 55.1 implies that $P'[\{\xi \in IR_\alpha(\Pi')\}] \geq 1-\alpha$, for all $P' \in \Pi'$. Here lies the appeal of the IHDR concept. Let us give a simple example, borrowed from Caprio et al. [239]. Suppose $\Xi = \{\xi_1, \ldots, \xi_5\}$, $\Pi = \{P_1, P_2, P_3\}$, and $\alpha = 0.1$. The numerical values for $P_s(\{\xi_j\})$ are given in Table 55.2, for all $s \in \{1,2,3\}$ and all $j \in \{1, \ldots, 5\}$. Then, from Proposition 55.1 and Definition 55.1, we have that $IR_\alpha(\Pi') = \{\xi_1, \xi_2, \xi_3\}$.

	ξ_1	ξ_2	ξ_3	ξ_4	ξ_5
P_1	**0.7**	**0.25**	0.03	0.01	0.01
P_2	**0.6**	**0.2**	**0.1**	0.05	0.05
P_3	**0.5**	**0.3**	**0.15**	0.025	0.025

Table 55.2: Numerical values for our example. It is easy to see that the smallest subset of Ξ that is assigned a probability of at least **0.9** by all the elements of Π is $\{\xi_1, \xi_2, \xi_3\}$.

An operative way of building the IHDR is to consider the union of the (precise) Highest Density Regions (HDRs) of the elements of $\Pi = \text{ex}\Pi'$.[8] Let us define them formally.

Definition 55.2. *[329, Section 1] Pick any $P_j \in \Pi$, $j \in \{1, \ldots, k\}$. Let α be any value in $[0,1]$. Then, set $R_\alpha(P_j) \subset \Xi$ is called a $(1-\alpha)$-Highest Density Region (HDR) for P_j if*

$$P_j[\{\xi \in R_\alpha(P_j)\}] \geq 1-\alpha \quad \text{and} \quad \int_{R_\alpha(P_j)} d\xi \text{ is minimal.} \tag{55.3}$$

[8] Here, by "operative" we mean that this procedure to compute the IHDR is easy to carry out in practice.

If Ξ is at most countable, we replace $\int_{R_\alpha(P_j)} d\xi$ with $\#R_\alpha(P_j)$. Equivalently [806],

$$R_\alpha(P_j) = \{\xi \in \Xi : p_j(\xi) \geq p_j^\alpha\} \subset \Xi, \tag{55.4}$$

where p_j is the pdf or the probability mass function (pmf) of P_j, and p_j^α is a constant value. In particular, it is the largest constant such that $P_j[\{\xi \in R_\alpha(P_j)\}] \geq 1 - \alpha$.

In dimension 1, $R_\alpha(P_j)$ can be interpreted as the smallest collection of elements of Ξ (interval or union of intervals) to which distribution P_j assigns probability of at least $1 - \alpha$. As we can see, HDRs are a Bayesian counterpart of confidence intervals.[9] We give a simple visual example in Figure 55.3.

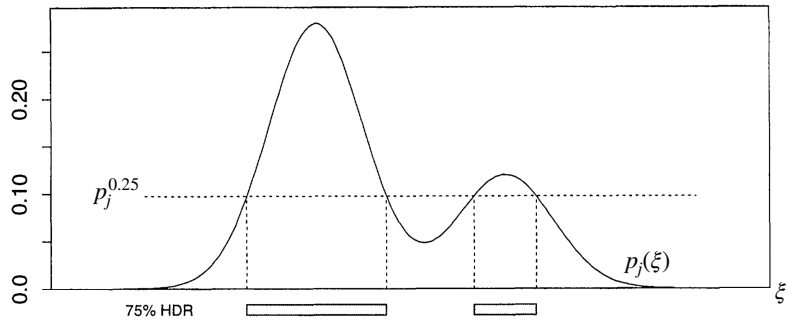


Figure 55.3: The 0.25-HDR from a Normal Mixture density. This picture is a replica of Hyndman [806, Figure 1]. The geometric representation of "75% probability according to P_j" is the area between the ξ axis and the part of the density p_j where $p_j(\xi) \geq p_j^{0.25}$ is equal to $1 - 0.25 = 0.75$.

As we mentioned earlier, an operative way of obtaining IHDR $IR_\alpha(\Pi')$ is by putting $IR_\alpha(\Pi') = \cup_{j=1}^k R_\alpha(P_j)$. Thanks to Proposition 55.1, by taking the union of the HDRs, we ensure that all the probability measures in the credal set $\Pi' = \text{Conv}(\Pi)$ assign probability of at least $1 - \alpha$ to the event $\{\xi \in IR_\alpha(\Pi')\}$. In turn, this implies that $\underline{P}[\{\xi \in IR_\alpha(\Pi')\}] = \min_{j\in\{1,\dots,k\}} P_j[\{\xi \in IR_\alpha(\Pi')\}] \geq 1 - \alpha$. We also have that the difference between the upper and lower probabilities of $\{\xi \in IR_\alpha(\Pi')\}$ is bounded by α. To see this, notice that $\overline{P}[\{\xi \in IR_\alpha(\Pi')\}] \leq 1$, so $\overline{P}[\{\xi \in IR_\alpha(\Pi')\}] - \underline{P}[\{\xi \in IR_\alpha(\Pi')\}] \leq \alpha$.

55.2.3 Quantifying and Disentangling Aleatoric and Epistemic Uncertainties

Recall that, given a probability measure P on a generic space Ξ, the (Shannon) entropy of P is defined as $H(P) := \mathbb{E}[-\log p] = -\int_\Xi \log[p(\xi)]P(\mathrm{d}\xi)$ if Ξ is uncountable, where p denotes the pdf of P. If Ξ is at most countable, we have that $H(P) = -\sum_{\xi\in\Xi} P(\{\xi\}) \log[P(\{\xi\})]$. As pointed out by Dubois and Hüllermeier [457], Hüllermeier and Waegeman [804], the entropy primarily captures the shape of the distribution, namely its "peakedness" or non-uniformity, and hence informs about the predictability of the outcome of a random experiment: the higher its value, the lower the predictability. Then, we can define the imprecise versions of the Shannon entropy as proposed by Abellán et al. [4], Hüllermeier and Waegeman [804], $\overline{H}(P) := \sup_{P\in\Pi'} H(P)$ and $\underline{H}(P) := \inf_{P\in\Pi'} H(P)$, called the *upper* and *lower Shannon entropy*, respectively. Notice that these definitions hold for all sets of probabilities, not just for (finitely generated) credal sets. The upper entropy is a measure of total uncertainty since it represents the minimum level of predictability associated with the elements of Π'. In Abellán et al. [4], Hüllermeier and Waegeman [804], the authors posit that it can be decomposed as a sum of aleatoric and epistemic uncertainties, and that the latter can be specified as the difference between upper and lower entropy, thus obtaining

$$\underbrace{\overline{H}(P')}_{\text{TU}(\Pi')} = \underbrace{\underline{H}(P')}_{\text{AU}(\Pi')} + \underbrace{\left[\overline{H}(P') - \underline{H}(P')\right]}_{\text{EU}(\Pi')}, \tag{55.5}$$

where $\text{TU}(\Pi')$ denotes the total uncertainty associated with set Π', $\text{AU}(\Pi')$ is the AU associated with Π', and $\text{EU}(\Pi')$ represents the EU associated with Π'. As we can see, if Π' is a singleton, then $\text{TU}(\Pi') = \text{AU}(\Pi')$, and $\text{EU}(\Pi') = 0$. This captures the idea that, in general, a single distribution can only gauge aleatoric uncertainty.[10] We have the following proposition.

[9] Hence standard choices for the value of α are 0.1, 0.05, 0.01.

[10] It can gauge EU, though, if it is a second-order distribution, or if it is the result of an ensemble of probabilities.

Algorithm 39: Credal Bayesian Deep Learning (CBDL) – Training and Inference

During Training

Step 1 Specify K priors $\text{ex}\mathcal{P}_{\text{prior}} = \{P_k^{\text{ex}}\}_{k=1}^K$

Step 2 Specify S likelihoods $\text{ex}\mathcal{P}_{\text{lik}} = \{L_s^{\text{ex}}\}_{s=1}^S$

Step 3 Compute $P_{k,s}(\cdot \mid \mathcal{D}) = \mathsf{post}[P_k^{\text{ex}}, L_s^{\text{ex}}]$, for all k and s {% VI-Appr. by $\breve{P}_{k,s}$ %}

During Inference

Inputs : New input $x_* \in \mathcal{X}$

Params : Confidence parameter $\alpha \in [0, 1]$

Outputs: Predictive Aleatoric and Epistemic Uncertainties, α-level IHDR

Step 4 Compute $P_{k,s}^{\text{pred}} = \mathsf{pred}[\breve{P}_{k,s}, L_s^{\text{ex}}]$, for all k and s {% VI-Appr. by $\hat{P}_{k,s}^{\text{pred}}$ %}

Step 5 Compute and return $\text{AU}(\hat{\mathcal{P}}_{\text{pred}})$ and the bounds for $\text{EU}(\hat{\mathcal{P}}_{\text{pred}})$

Step 6 Compute and return the $(1-\alpha)$-IHDR $IR_\alpha(\hat{\mathcal{P}}_{\text{pred}})$

Proposition 55.2. *Let Π, Π' be sets of probability measures as the ones considered in Remark 55.2. Then, $\sup_{P\in\Pi} H(P) = \overline{H}(P) \leq \overline{H}(P') = \sup_{P'\in\Pi'} H(P')$ and $\inf_{P\in\Pi} H(P) = \underline{H}(P) = \underline{H}(P') = \inf_{P'\in\Pi'} H(P')$.*

Proposition 55.2 tells us that the upper entropy of the extreme elements in $\Pi = \text{ex}\Pi'$ is a lower bound for the upper entropy of the whole credal set Π', and that the lower entropy of the extreme elements in Π is equivalent to the lower entropy of the whole credal set Π'. These facts imply that $\text{AU}(\Pi') = \text{AU}(\Pi)$, and that $\text{EU}(\Pi') \geq \text{EU}(\Pi)$. In addition, as a consequence of Smieja and Tabor [1523, Theorem III.1], we have that $\text{EU}(\Pi') \leq \text{EU}(\Pi) + \log(\#\Pi)$. In turn, we have that $\text{EU}(\Pi') \in [\text{EU}(\Pi), \text{EU}(\Pi) + \log(\#\Pi)]$.

We briefly mention that other uncertainty measures based on credal sets are also available (see Bronevich and Rozenberg [209], Caprio et al. [238], Hofman et al. [766], Hüllermeier and Waegeman [804] for a few examples) and they can be used in place of upper and lower entropy[11] to quantify EU and AU within the credal set $\mathcal{P}$, as long as the measure chosen for the total uncertainty is bounded.

Let us also add a small remark. Although EU can be reduced with an increasing amount of data, it is very seldom the case that it goes to zero when a finite amount of data is collected. When that happens, it means that the initial uncertainty was very low in the first place. Typically, EU goes to zero only asymptotically, as a finite amount of data is almost never enough to overcome initial uncertainty [1699, 1773].

55.3 CBDL and Its Properties

This is the main portion of the chapter. In Section 55.3.1, we present and discuss the CBDL algorithm. Its theoretical properties are derived in Section 55.3.2.

55.3.1 CBDL algorithm

Recall that $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ denotes the training set, and let $\mathcal{X}$ and $\mathcal{Y}$ denote the input and output spaces, respectively. We then denote by P a generic prior on the parameters $\theta \in \Theta$ of a BNN having pdf p, and by $L \equiv L_{x,\theta}$ a generic likelihood on the space $\mathcal{Y}$ of outputs having pdf $l \equiv l_{x,\theta}$. The act of computing the posterior from prior P and likelihood L using a BNN is designated by $\mathsf{post}[P, L]$, and the act of deriving the posterior predictive from posterior $P(\cdot \mid \mathcal{D})$ and likelihood L is designated by $\mathsf{pred}[P(\cdot \mid \mathcal{D}), L]$. The CBDL procedure is presented in Algorithm 39, and it is discussed in the following paragraphs.

During training, in **Step 1** the user specifies K priors on the parameters of the neural network, that constitute the extrema $\text{ex}\mathcal{P}_{\text{prior}}$ of the prior FGCS $\mathcal{P}_{\text{prior}}$. Similarly, in **Step 2** they elicit S likelihoods capturing the possible architectures of the neural network, that correspond to the extrema $\text{ex}\mathcal{P}_{\text{lik}}$ of the likelihood FGCS $\mathcal{P}_{\text{lik}}$. Let us give an example. Outlined in Jospin et al. [854, Sections IV-B and IV-C1], for classification, the standard process for BNNs involves

- A Normal prior with zero mean and diagonal covariance $\sigma^2 I$ on the coefficients of the network, that is, $p(\theta) = \mathcal{N}(0, \sigma^2 I)$. In the context of CBDL, we could specify, e.g. $\text{ex}\mathcal{P}_{\text{prior}} = \{P : p(\theta) = \mathcal{N}(\mu, \sigma^2 I),\ \mu \in \{\mu_-, \mathbf{0}, \mu_+\},\ \sigma^2 \in \{3, 7\}\}$. That is, the extreme elements of the prior credal set are six independent Normals having different levels of "fatness" of the tails, and centred at a vector μ_+ having positive entries, a vector

[11] Which, although easy to compute, can sometimes suffer from shortcomings coming from lacking the monotonicity property. That is, there are credal sets that, although nested into each other, have the same upper and/or lower entropy [804].

μ_- having negative entries, and a vector $\mathbf{0}$ having entries equal to 0. They capture the ideas of positive bias, negative bias, and no bias of the coefficients, respectively. This is done to hedge against possible prior misspecification.

- A Categorical likelihood, $p(y \mid x, \theta) = \text{Cat}(f_\theta(x))$, whose parameter is given by the output of a functional model f_θ (e.g. a neural network having parameters θ). In the context of CBDL, we could specify the set of extreme elements of the likelihood credal set as $\text{ex}\mathcal{P}_{\text{lik}} = \{L : l_{x,\theta}(y) = \text{Cat}(f_{s,\theta}(x)), s \in \{1, \dots, S\}\}$. Specifying set $\text{ex}\mathcal{P}_{\text{lik}}$, then, corresponds to eliciting a finite number S of possible (parametrised) architectures for the neural network $f_{s,\theta}$, $s \in \{1, \dots, S\}$, and obtain, as a consequence, S categorical distributions $\{\text{Cat}(f_{s,\theta}(x))\}_{s=1}^S$. This captures the ambiguity around the true data generating process faced by the agent, and allows them to hedge against likelihood misspecification.

More in general, we can use the priors and likelihoods that better fit the type of analysis we are performing.[12] For example, for the choice of the priors we refer to Fortuin et al. [548] and to Part VI of this book, where the problem of selecting the right type of prior for BNNs, and their properties, are studied in depth.

Step 3 performs an element-wise application of Bayes' rule for all the elements of $\text{ex}\mathcal{P}_{\text{prior}}$ and $\text{ex}\mathcal{P}_{\text{lik}}$. Each posterior is approximated using the Variational Inference (VI) method, studied in detail in Part III of this book. That is, we project every posterior $P_{k,s}(\cdot \mid \mathcal{D})$ onto a set $\mathbb{S}$ of "well-behaved" distributions (e.g. Normals) using the KL divergence. In formulas, $\breve{P}_{k,s} = \arg\min_{Q \in \mathbb{S}} KL[Q \| P_{k,s}(\cdot \mid \mathcal{D})]$. By "well-behaved", we mean that they have to satisfy the conditions in Zhang and Gao [1844, Sections 2 and 3].[13] This ensures that as the sample size goes to infinity, the approximated posteriors converge to the true data generating process. We also point out how despite using a VI approximation for the exact posteriors, as we shall see in the next Section, the credal set of approximated posteriors is closer to the "oracle" posterior $P^o(\cdot \mid \mathcal{D})$ than any of its elements taken individually. As a consequence, working with credal sets leads to VI posterior approximations that are better than the ones resulting from a single BNN, or an ensemble of BNNs, where several BNNs are combined into one.

Remark 55.3. *Although highly unlikely in practice, it is theoretically possible that the VI approximation of the (finite) set $\{P_{k,s}(\cdot \mid \mathcal{D})\}_{k,s}$ of posteriors is a singleton, see Figure 55.4. While the conditions in Zhang and Gao [1844] guarantee that asymptotically the approximated posteriors coincide with the true data generating process, this typically does not happen with finite-dimensional datasets. As a consequence, obtaining a singleton when projecting $\{P_{k,s}(\cdot \mid \mathcal{D})\}_{k,s}$ onto $\mathbb{S}$ may result in an underestimation of the uncertainties faced by the user. In that case, we either consider a different set – whose elements still satisfy the conditions in Zhang and Gao [1844, Sections 2 and 3] – on which to project $\{P_{k,s}(\cdot \mid \mathcal{D})\}_{k,s}$ according to the KL divergence, or we use a different "projection operator", that is, a divergence different from the KL. For example, Rényi and χ^2 divergences, or Hellinger and total variation metrics are suggested by Zhang and Gao [1844]. Alternatively, we can consider a different approximation strategy altogether, for instance the Laplace approximation (see e.g. Ritter et al. [1402] and Part II of this book).*

After **Step 3**, we obtain a finite set $\{\breve{P}_{k,s}\}_{k,s}$ of VI approximation of the posteriors on the network parameters, whose cardinality is $K \times S$. Its convex hull constitutes $\breve{\mathcal{P}}_{\text{post}}$, that is, the VI-approximated posterior FGCS. We assume that $\text{ex}\breve{\mathcal{P}}_{\text{post}} = \{\breve{P}_{k,s}\}_{k,s}$. This is an assumption because it may well be that – due to the approximation procedure – some of the elements of $\{\breve{P}_{k,s}\}_{k,s}$ are not independent of one another. An interesting open research question is that of designing a procedure that finds the elements of $\{\breve{P}_{k,s}\}_{k,s}$ which cannot be written as a convex combination of one another.

Being a combinatorial task, **Step 3** is a computational bottleneck of Algorithm 39. We have to calculate $K \times S$ VI approximations to as many posteriors, but this allows us to forego any additional assumptions on the nature of the lower and upper probabilities that are oftentimes required by other imprecise-probabilities-based techniques.[14] Clearly, CBDL is simplified if either $\mathcal{P}_{\text{prior}}$ or $\mathcal{P}_{\text{lik}}$ are singletons. Notice that in the case that $\mathcal{P}_{\text{prior}}$ and $\mathcal{P}_{\text{lik}}$ are both not singletons, for all $A \subset \Theta$ the interval $[\underline{\breve{P}}(A), \overline{\breve{P}}(A)]$ is wider than the case when one or the other is a singleton. In the limiting case where both are singletons, we retrieve the usual Bayesian updating, so the interval shrinks down to a point.

Before moving to inference time, let us remark a difference between Bayesian Model Averaging (BMA) and CBDL. In BMA, the user specifies a distribution on the models. Translated in the notation we use in this chapter, this means having a discrete distribution Q over the $K \times S$ prior-likelihood combinations, that is, over the elements

[12] Choosing 2 to 5 priors and likelihoods is usually enough to safely hedge against prior and likelihood misspecification.

[13] We assume that the conditions on the priors and the likelihoods given in Zhang and Gao [1844] are satisfied.

[14] If we are willing to make such assumptions, Caprio et al. [236, Theorem 10] shows how to compute the upper posterior using only upper prior and upper likelihood.

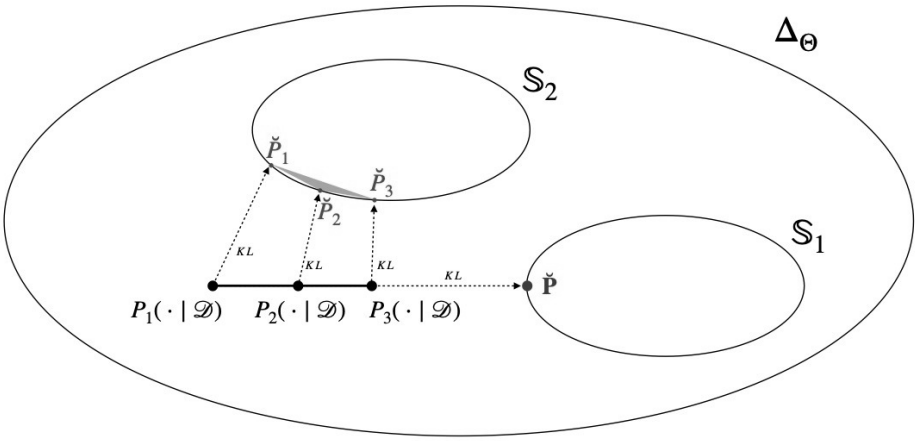


Figure 55.4: Let Δ_Θ denote the space of probability measures on Θ. Suppose that in the analysis at hand we specified three priors and only one likelihood, so $S = 1$ and we can drop the s index. Let $\{P_k(\cdot \mid \mathcal{D})\}_{k=1}^3$ be the collection of exact posteriors, so that the solid black segment represents the exact posterior FGCS. Then, if we project the elements of $\{P_k(\cdot \mid \mathcal{D})\}_{k=1}^3$ onto $\mathbb{S}_1$ via the KL divergence, we obtain the same distribution $\breve{\mathbf{P}}$. This is detrimental to the analysis because such an approximation underestimates the (posterior) epistemic (and possibly also aleatoric) uncertainty faced by the agent. Then, the user could specify a different set $\mathbb{S}_2$ of "well-behaved" distributions onto which project the elements of $\{P_k(\cdot \mid \mathcal{D})\}_{k=1}^3$. In the figure, we see that they are projected onto $\mathbb{S}_2$ via the KL divergence to obtain $\breve{P}_1$, $\breve{P}_2$, and $\breve{P}_3$. The convex hull of these latter, captured by the shaded triangle, represents the variational approximation of the exact posterior FGCS.

of $\text{ex}\breve{\mathcal{P}}_{\text{post}}$. Such Q is then used to select an element $\breve{P}^\star$ from the (VI-approximated) posterior FGCS $\breve{\mathcal{P}}_{\text{post}}$ as

$$\breve{\mathcal{P}}_{\text{post}} \ni \breve{P}^\star = \sum_{k,s} Q(\{\breve{P}_{k,s}\})\breve{P}_{k,s}, \tag{55.6}$$

where $Q(\{\breve{P}_{k,s}\}) \in [0,1]$ for all k and all s, and $\sum_{k,s} Q(\{\breve{P}_{k,s}\}) = 1$ [234]. For example, if Q is the discrete uniform distribution on $\{1, \dots, K \times S\}$, then $\breve{P}^\star$ is the so-called *centre of gravity* of the credal set $\breve{\mathcal{P}}_{\text{post}}$ [1185, Section 3.2]. Instead, CBDL does not select a unique distribution from $\breve{\mathcal{P}}_{\text{post}}$. Rather, distributions are kept separate so to be able to derive a predictive FGCS in **Step 4** of Algorithm 39, that is in turn used to quantify and disentangle predictive uncertainties, and to compute the predictive IHDR, that is, a collection of outputs having a high probability of being the correct ones for a new input x_*.

During inference, a new input x_* is provided. In **Step 4**, every element of $\text{ex}\breve{\mathcal{P}}_{\text{post}}$ is used to derive a predictive distribution $P_{k,s}^{\text{pred}} = \mathsf{pred}[\breve{P}_{k,s}, L_s^{\text{ex}}]$ on the output space $\mathcal{Y}$. In particular, for every k and every s, the pdf $p_{k,s}^{\text{pred}}$ of $P_{k,s}^{\text{pred}}$ is obtained as

$$\begin{aligned} p_{k,s}^{\text{pred}}(y_* \mid x_*, x_1, y_1, \dots, x_N, y_N) &= \int_\Theta l_s^{\text{ex}}(y_* \mid x_*, \theta) \cdot p_{k,s}(\theta \mid x_1, y_1, \dots, x_N, y_N) \mathrm{d}\theta \\ &\approx \int_\Theta l_s^{\text{ex}}(y_* \mid x_*, \theta) \cdot \breve{p}_{k,s}(\theta) \mathrm{d}\theta, \end{aligned} \tag{55.7}$$

where l_s^{ex} is the pdf of likelihood L_s^{ex}, $p_{k,s}$ is the pdf of the true posterior $P_{k,s}(\cdot \mid \mathcal{D})$, $\breve{p}_{k,s}$ is the pdf of the VI-approximated posterior $\breve{P}_{k,s}$, and y_* is the output associated to the new input x_* (see Caprio et al. [236, Appendix F] for more details). Each predictive distribution $P_{k,s}^{\text{pred}}$ is approximated to $\hat{P}_{k,s}^{\text{pred}}$ (e.g. using Normals) via Variational Inference. The convex hull of the collection $\{\hat{P}_{k,s}^{\text{pred}}\}_{k,s}$ having cardinality $K \times S$ constitutes the VI-approximated predictive FGCS $\hat{\mathcal{P}}_{\text{pred}}$. Similarly to what we did for $\text{ex}\breve{\mathcal{P}}_{\text{post}}$, we assume that $\text{ex}\hat{\mathcal{P}}_{\text{pred}} = \{\hat{P}_{k,s}^{\text{pred}}\}_{k,s}$.

In **Step 5**, building on the results in Section 55.2.3, and in particular on Proposition 55.2, we compute and return $\text{AU}(\hat{\mathcal{P}}_{\text{pred}})$ and the bounds for $\text{EU}(\hat{\mathcal{P}}_{\text{pred}})$. In particular, we have

$$\text{AU}(\mathcal{P}_{\text{pred}}) \approx \text{AU}(\hat{\mathcal{P}}_{\text{pred}}) = \underline{H}(\hat{P}^{\text{pred}}) \tag{55.8}$$

and

$$\text{EU}(\mathcal{P}_{\text{pred}}) \approx \text{EU}(\hat{\mathcal{P}}_{\text{pred}}) \in [\overline{H}(\hat{P}^{\text{pred}}) - \underline{H}(\hat{P}^{\text{pred}}), \overline{H}(\hat{P}^{\text{pred}}) - \underline{H}(\hat{P}^{\text{pred}}) + \log(K \times S)], \tag{55.9}$$

where (i) $\underline{H}(\hat{P}^{\text{pred}}) = \min_{k,s} H(\hat{P}^{\text{pred}}_{k,s})$; (ii) $\overline{H}(\hat{P}^{\text{pred}}) = \max_{k,s} H(\hat{P}^{\text{pred}}_{k,s})$; and (iii) $\mathcal{P}_{\text{pred}}$ is the "true" predictive FGCS, that is, the one we would have obtained had we been able to compute the exact posteriors in **Step 3** and the exact predictive distributions in **Step 4**. Let us point out a salient feature of CBDL. The AU and the (bounds for) the EU associated with the VI-approximated predictive FGCS $\hat{\mathcal{P}}_{\text{pred}}$ embed uncertainty comparable to an uncountably infinite ensemble of BNNs, i.e., an ensemble of BNNs of cardinality $\aleph_1$, despite the simple and intuitive mathematics over the finite set $\text{ex}\hat{\mathcal{P}}_{\text{pred}}$. They are not merely pessimistic results on the uncertainties associated with a finite ensemble of BNNs.

We observe that we compute the AU and EU associated with the (VI-approximated) predictive credal set $\hat{\mathcal{P}}_{\text{pred}}$, rather than those related to the (VI-approximated) posterior credal set $\hat{\mathcal{P}}_{\text{post}}$. We do so because we are ultimately interested in reporting the uncertainty around the predicted outputs given a new input x_* in the problem at hand, more than the uncertainty on the parameters of the NN.

Before commenting on the next step, let us pause here and add a remark. While a "bad choice" of priors and likelihoods may lead to large values of upper and lower entropy – $\overline{H}(\hat{P}^{\text{pred}})$ and $\underline{H}(\hat{P}^{\text{pred}})$, respectively – this is not a risk confined to CBDL. Poor modelling choices are an unavoidable risk in model-based techniques. This gave rise to the famous adage by George Box "essentially, all models are wrong but some are useful" [199].[15] We maintain that CBDL is indeed useful, since it overcomes some of the shortcomings of traditional Bayesian techniques – as explained in Caprio et al. [236, Appendices A and B]. As for "regular" Bayesian methods, though, for CBDL too the designer will need to make "plausible" choices for priors and likelihoods. A further positive aspect of working with credal sets is that they are able to "self-regulate". That is, in the case of prior-likelihood conflict – which happens e.g. if the prior set is ill-specified [1389] – the posterior credal set, and in turn the predictive credal set too, will be wider. This will be reflected in the uncertainty measures which will register an excess of posterior, and in turn predictive, epistemic uncertainties.

Finally, in **Step 6**, we compute and return the α-level Imprecise Highest Density Region $IR_\alpha(\hat{\mathcal{P}}_{\text{pred}})$ for the VI-approximated predictive FGCS $\hat{\mathcal{P}}_{\text{pred}}$, which approximates the IHDR for the "true" predictive FGCS $\mathcal{P}_{\text{pred}}$. It is the smallest subset of $\mathcal{Y}$ such that $\hat{P}^{\text{pred}}[\{y_* \in IR_\alpha(\hat{\mathcal{P}}_{\text{pred}})\}] \geq 1-\alpha$, for all $\hat{P}^{\text{pred}} \in \hat{\mathcal{P}}_{\text{pred}}$, and for some $\alpha \in [0,1]$. It can be interpreted as the smallest collection of outputs y_* that have a high probability of being the correct ones for the new input x_*, according to all the distributions in $\hat{\mathcal{P}}_{\text{pred}}$. Or equivalently, we can say that the correct output for the new input x_* belongs to IHDR $IR_\alpha(\hat{\mathcal{P}}_{\text{pred}})$ with lower probability of at least $1-\alpha$, a probabilistic guarantee for the set of outputs generated by CBDL. Notice also that the size of $IR_\alpha(\hat{\mathcal{P}}_{\text{pred}})$ is an increasing function of both (predictive) AU and EU. As a consequence, it is related, but it is not equal, to the (predictive) AU the agent faces. If we want to avoid to perform the procedure only to discover that $IR_\alpha(\hat{\mathcal{P}}_{\text{pred}})$ is "too large", then we can add an "AU check" after **Step 5**. This, together with computing $IR_\alpha(\hat{\mathcal{P}}_{\text{pred}})$ in a classification setting, is explored in Caprio et al. [236, Appendices G and H].

As we have seen in Section 55.2.2, we find $IR_\alpha(\hat{\mathcal{P}}_{\text{pred}})$ by taking the union of the $K \times S$ HDRs $R_\alpha(\hat{P}^{\text{pred}}_{k,s})$ of the extreme elements $\text{ex}\hat{\mathcal{P}}_{\text{pred}}$ of $\hat{\mathcal{P}}_{\text{pred}}$. The HDR of a well-known distribution (for instance, a Normal) can be routinely obtained in R, e.g. using package HDInterval [856].

We conclude this Section with two remarks. First, we point out how CBDL does not depend on the method used to approximate the posterior and the predictive distributions: Variational Inference can be substituted by other approaches. CBDL can also be easily adapted to other TU, AU, and EU measures, as long as the measure chosen for the total uncertainty is bounded. Second, **Step 6** can be effortlessly modified so that CBDL produces the collection of outputs having the highest lower density. This is in line with the Naive Credal Classifier theory [341, 1829]. In this case, we forego control on the accuracy level $1-\alpha$ of the output region produced by CBDL. It is implemented as follows. In the modified version of **Step 6**, we compute

$$\arg\max_{y_* \in \mathcal{Y}} \min_{k,s} \hat{p}^{\text{pred}}_{k,s}(y_*), \tag{55.10}$$

where $\hat{p}^{\text{pred}}_{k,s}$ is the pdf of the VI-approximated predictive distribution $\hat{P}^{\text{pred}}_{k,s}$. If such arg max is not a singleton, and the user is set on CBDL outputting a unique value y_* for the new input x_*, they can select one element uniformly at random from the arg max.

55.3.2 Theoretical Properties of CBDL

Working with credal sets makes CBDL more robust to distribution misspecification and shifts than single BNNs. To see this, we present the following general result, and then we apply it to our case.

[15] Curiously, a similar motivation was brought forward by Pseudo-Dionysius the Areopagite in favour of the use of sacred images in the Christian tradition [1172]. While they do not capture the essence of God, these defective approximations help the believer's thought to elevate.

Let Π' be an FGCS as in Remark 55.2, and consider a probability measure Ψ such that $\Psi \notin \Pi'$.

Proposition 55.3. *Call d any metric and div any divergence on the space of probability measures of interest. Let $d(\Pi', \Psi) := \inf_{P' \in \Pi'} d(P', \Psi)$ and $div(\Pi' \| \Psi) := \inf_{P' \in \Pi'} div(P' \| \Psi)$. Then, for all $P' \in \Pi'$, $d(\Pi', \Psi) \leq d(P', \Psi)$ and $div(\Pi', \Psi) \leq div(P' \| \Psi)$.*

Proposition 55.3 holds if Π' is any set of probabilities, not just an FGCS.[16] In Caprio et al. [236, Appendix I], the authors show that the above result still holds if the elements of Π' and Ψ are defined on Euclidean spaces having different dimensions [225, 233].

Let us now apply Proposition 55.3 to CBDL. Suppose that, when designing a single BNN, an agent chooses likelihood $\mathbf{L}$, while when implementing CBDL, they specify in **Step 2** a finite set of likelihoods $\text{ex}\mathcal{P}_{\text{lik}} = \{L_s^{\text{ex}}\}_{s=1}^S$, $S \geq 2$, and then let the induced credal set $\mathcal{P}_{\text{lik}} = \text{Conv}(\text{ex}\mathcal{P}_{\text{lik}})$ represent their uncertainty around the sampling model. Assume that $\mathbf{L} \in \text{ex}\mathcal{P}_{\text{lik}}$. This means that when designing the single BNN, the agent chooses arbitrarily which of the elements of $\text{ex}\mathcal{P}_{\text{lik}}$ to use. Suppose also that the "oracle" data generating process L^o is different from $\mathbf{L}$, $L^o \neq \mathbf{L}$, so that we are actually in the presence of likelihood misspecification. Then, we have two cases. (1) If the true sampling model L^o belongs to $\mathcal{P}_{\text{lik}}$, then the distance – measured via a metric or a divergence – between $\mathcal{P}_{\text{lik}}$ and L^o is 0, while that between $\mathbf{L}$ and L^o is positive. (2) If $L^o \notin \mathcal{P}_{\text{lik}}$, then the distance between $\mathcal{P}_{\text{lik}}$ and L^o is no larger than the distance between $\mathbf{L}$ and L^o, no matter (i) which metric or distance we use (Proposition 55.3), (ii) whether or not L^o and the elements of $\mathcal{P}_{\text{lik}}$ are defined on the same Euclidean space [236, Lemma 17]. A visual representation is given in Figure 55.5. A similar argument holds if part of the training dataset $\mathcal{D}$ is generated by one distribution L_1^o, and the remaining part is generated by L_2^o, a situation that we can categorise as distribution shift. Even if – in the case of a single BNN – the user were able to elicit exactly $\mathbf{L} = L_1^o$, the likelihood would still be misspecified for the data that come from L_2^o. This shortcoming is overcome by a credal set approach, in which either both oracle distributions L_1^o and L_2^o belong to the credal set $\mathcal{P}_{\text{lik}}$, or the distance between $\mathcal{P}_{\text{lik}}$ and either of them is no larger than that between the single chosen one $\mathbf{L}$, and L_1^o and L_2^o.

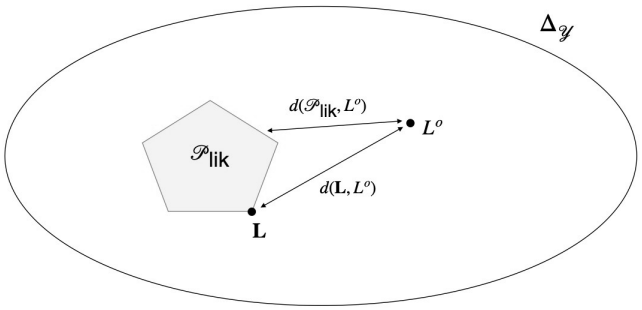


Figure 55.5: CBDL is more robust to distribution shifts than single BNNs. Here $\mathcal{P}_{\text{lik}}$ is the convex hull of five plausible likelihoods, and d denotes a generic metric on the space $\Delta_{\mathcal{Y}}$ of probabilities on $\mathcal{Y}$. We see how $d(\mathcal{P}_{\text{lik}}, L^o) < d(\mathbf{L}, L^o)$; if we replace metric d by a generic divergence div, the inequality would still hold.

Let us add a remark here. Assume that the "oracle" prior P^o is in the prior credal set $\mathcal{P}_{\text{prior}}$ and that the "oracle" likelihood L^o is in the likelihood credal set $\mathcal{P}_{\text{lik}}$. Then, it is immediate to see that the "oracle" posterior $P^o(\cdot \mid \mathcal{D})$ belongs to the posterior credal set $\mathcal{P}_{\text{post}}$. Naturally, this does not imply that the posterior credal set collapses to $P^o(\cdot \mid \mathcal{D})$. In general, it is unlikely that a finite amount of data is able to completely annihilate all the epistemic uncertainty faced by the agent. What may happen is that if the training set is large enough, $\mathcal{P}_{\text{post}}$ may be inscribed in a ball of small radius around $P^o(\cdot \mid \mathcal{D})$. This does not mean that we suffer from under-confidence due to larger-than-necessary epistemic uncertainty. Rather, the relative epistemic uncertainty (measured by the difference between prior and posterior uncertainty, divided by the prior uncertainty) drops significantly. In addition, working with sets of prior and likelihoods allows us to hedge against prior and likelihood misspecification, a consequence of Proposition 55.3.

55.4 Experiments

From the previous Sections, it is clear that CBDL improves on the uncertainty quantification capabilities of single BNNs.[17] CBDL allows for a better quantification of predictive AU because of its robustness to misspecification

[16] More in general, Proposition 55.3 holds for any type of function f, not just metrics or divergences, because Proposition 55.3 essentially only applies the definition of the infimum.

[17] This includes empirical Bayes methodologies [927].

stemming from Proposition 55.3. In addition, as we pointed out before (in Section 55.2.1), predictive EU cannot be obtained in a theoretically principled manner from a single BNN [512, 804]. Here, we show that these theoretical arguments are also backed by empirical evidence.

55.4.1 (Predictive) Uncertainty Quantification

Distribution shifts can introduce uncertainties in a system, which in turn can render the predictions meaningless. This can be due to naturally occurring corruptions, as introduced in Hendrycks and Dietterich [733] for image datasets. The authors introduced 18 different noise types, which can be varied across 5 different severity levels, ranging from low (severity = 1), to medium (severity = 2, 3), and high (severity = 4, 5). The intuition is that in the current context, increasing the noise severity should generally result in higher uncertainty. We evaluate the CBDL method on 4 standard image datasets, CIFAR-10 [934], SVHN [1232], Fashion-MNIST [1785], and MNIST [993]. We use a slightly different set of perturbations than those introduced in Mu and Gilmer [1203] for gray-scale images like MNIST and Fashion-MNIST. Additionally, we perform cross domain testing for each dataset, where we expect the predictive uncertainties to be higher. We implement and train a Resnet-20 Bayesian Neural Network model inside the library Bayesian-torch [927]. For each dataset, we train 4 different networks initialised with different seeds on the prior and with the same architecture. This corresponds to eliciting a prior FGCS $\mathcal{P}_{\text{prior}}$ such that $\text{ex}\mathcal{P}_{\text{prior}} = \{P_1^{\text{ex}}, \dots, P_4^{\text{ex}}\}$, so that K in **Step 1** of Algorithm 39 is equal to 4, and a likelihood FGCS that is a singleton, $\mathcal{P}_{\text{lik}} = \text{ex}\mathcal{P}_{\text{lik}} = \{L\}$, so that S in **Step 2** of Algorithm 39 is equal to 1. We use a learning-rate of 0.001, batch-size of 128, and train the networks using Mean-Field Variational Inference for 200 epochs. The inference is carried out by performing multiple forward passes through parameters drawn from the posterior distribution. We used 20 Monte Carlo (MC) samples in the experiments (MC methods were introduced in Part I of this book).

Baselines. In Krishnan and Tickoo [926], for a single BNN output, the predictive distribution is obtained through multiple stochastic forward passes on the network while sampling from the weight posteriors using Monte Carlo estimators. In their work, the authors define the overall entropy and the entropy of the predictive distribution as the *predictive entropy* [926, Equation (C.2)]. This quantity captures a combination of predictive aleatoric and epistemic uncertainties. Additionally, they define the *mutual information* between weight posterior and predictive distribution as the predictive epistemic uncertainty. Finally, the predictive aleatoric uncertainty is defined as the *expected entropy*. The predictive epistemic and aleatoric uncertainties sum up to the predictive entropy. In the experiments, we use these definitions to compute the respective quantities.

In Cobb et al. [316], the authors consider different BNNs, but instead of keeping them separate and use them to build a predictive credal set, they average them out. Similar to theirs, we elicit the following procedure, that we call ensemble of BNNs (EBNN).

Consider $R \in \mathbb{N}_{\geq 2}$ different BNNs, and compute the posterior distribution on the parameters. They induce R predictive distributions on the output space $\mathcal{Y}$, each having mean μ_r and variance σ_r^2, $r \in \{1, \dots, R\}$. We call *EBNN distribution* P_{ens} a Normal having mean $\mu_{\text{ens}} = 1/R \sum_{r=1}^{R} \mu_r$ and covariance matrix $\sigma_{\text{ens}}^2 I$, where $\sigma_{\text{ens}}^2 = 1/R \sum_{r=1}^{R} \sigma_r^2 + 1/(R-1) \sum_{r=1}^{R} (\mu_r - \mu_{\text{ens}})^2$.

Following Cobb et al. [316], for EBNN we posit that $1/R \sum_{r=1}^{R} \sigma_r^2$ captures the (predictive) aleatoric uncertainty associated with P_{ens}, and $1/(R-1) \sum_{r=1}^{R} (\mu_r - \mu_{\text{ens}})^2$ captures the (predictive) epistemic uncertainty associated with P_{ens}.[18] For CBDL, we look at the value of $\text{AU}(\hat{\mathcal{P}}_{\text{pred}}) = \underline{H}(\hat{P}^{\text{pred}})$ from (55.8), and at the lower bound $\overline{H}(\hat{P}^{\text{pred}}) - \underline{H}(\hat{P}^{\text{pred}})$ for $\text{EU}(\hat{\mathcal{P}}_{\text{pred}})$ from (55.9). It is enough to focus on the lower bound because the upper bound is given by $\overline{H}(\hat{P}^{\text{pred}}) - \underline{H}(\hat{P}^{\text{pred}}) + \log(4)$, and $\log(4) \approx 0.6$ is a fixed value. Hence, the trend of both lower and upper bounds for $\text{EU}(\hat{\mathcal{P}}_{\text{pred}})$ as the severity of corruption changes is the same.

Summary of the Results. Two scenarios are evaluated: an In-Distribution (ID) scenario with corruption by noises of increasing severity, and an Out-Of-Distribution (OOD) assessment. The following main points summarise our findings:

1. (ID Evaluation) For both CBDL and the method in Krishnan and Tickoo [926], for increasing severities of noise corruption, predictive epistemic and aleatoric uncertainties increase. In comparison, the baseline EBNN method demonstrates a counterintuitive result, where the predictive aleatoric uncertainty decreases as the severity of corruption increases. The table of values can be found in Caprio et al. [236, Appendix N].

2. (ID Evaluation) To assess the utility of CBDL in downstream tasks, we analyse the accuracy versus rejection rate for all methods. Table 55.3 summarises the results. Across MNIST-C and Fashion-MNIST-C,

[18] Notice that in the experiments we retain the assumption that the EBNN distribution P_{ens} on $\mathcal{Y}$ has mean μ_{ens} and covariance matrix $\sigma_{\text{ens}}^2 I$, but we do not require that it is a Normal. That is because in the four image datasets that we consider, the output space $\mathcal{Y}$ is finite.

CBDL exhibits a higher average accuracy across the range of rejection rates. For the remaining datasets, it demonstrates results comparable to the baseline (see Section 55.4.1 for evaluation details)

3. (OOD Evaluation) To evaluate OOD detection, models trained on a particular dataset are tested on the other three datasets with the goal of analysing the behavior of the predictive uncertainties. Tables 55.4 and 55.5 summarise the results. Across all the evaluated datasets, CBDL had the most consistent behavior of exhibiting a large increase in predictive AU, relative to the increase in predictive EU when tested on an OOD dataset. In comparison, EBNN shows a decrease in predictive AU, and the single BNN shows inconsistent behavior depending on the dataset.

	CIFAR-10C			MNIST-C			Fashion MNIST-C			SVH	
Severity	BNN	Ensemble of BNN	CBDL	BNN	Ensemble of BNN	CBDL	BNN	Ensemble of BNN	CBDL	BNN	Ensemble o
1	0.754	0.983	0.982	0.645	0.919	0.929	0.525	0.730	0.748	0.698	0.913
2	0.761	0.947	0.946	0.680	0.879	0.896	0.537	0.655	0.641	0.703	0.843
3	0.749	0.906	0.907	0.617	0.804	0.841	0.442	0.498	0.529	0.669	0.764
4	0.733	0.838	0.836	0.639	0.713	0.737	0.332	0.421	0.489	0.608	0.586
5	0.661	0.734	0.734	0.404	0.495	0.539	0.300	0.351	0.391	0.455	0.378
Mean	0.732	**0.882**	0.881	0.597	0.762	**0.788**	0.433	0.531	**0.560**	0.627	**0.697**

Table 55.3: In-Distribution evaluation on noise corruptions of increasing severity. Average area under accuracy vs rejection rate curve. Higher numbers indicate overall better accuracy over a range of rejection rates.

		CBDL		Ensemble of BNN		BNN	
		Epistemic	Aleatoric	Epistemic	Aleatoric	Epistemic	Aleatoric
CIFAR	In Dist-Clean	0.102	0.031	0.008	0.076	0.151	0.179
	MNIST	0.184 (1.801)	0.142 (4.626)	0.021 (2.723)	0.044 (0.572)	0.445 (2.95)	0.772 (4.312)
	Fashion MNIST	0.183 (1.791)	0.147 (4.808)	0.022 (2.822)	0.042 (0.557)	0.431 (2.861)	0.811 (4.53)
	SVHN	0.183 (1.789)	0.141 (4.606)	0.019 (2.421)	0.046 (0.608)	0.394 (2.612)	0.647 (3.616)
		Epistemic	Aleatoric	Epistemic	Aleatoric	Epistemic	Aleatoric
MNIST	In Dist-Clean	0.188	0.057	0.012	0.063	0.198	0.071
	CIFAR	0.185 (0.986)	0.167 (2.94)	0.023 (1.948)	0.042 (0.67)	0.821 (4.155)	0.372 (5.124)
	Fashion MNIST	0.162 (0.866)	0.187 (3.287)	0.019 (1.618)	0.035 (0.564)	0.919 (4.654)	0.461 (6.496)
	SVHN	0.190 (1.015)	0.168 (2.963)	0.022 (1.847)	0.043 (0.69)	0.923 (4.674)	0.297 (4.183)
		Epistemic	Aleatoric	Epistemic	Aleatoric	Epistemic	Aleatoric
Fashion MNIST	In Dist-Clean	0.121	0.029	0.007	0.075	0.197	0.351
	CIFAR	0.205 (1.705)	0.104 (3.561)	0.022 (3.178)	0.050 (0.667)	0.408 (2.073)	0.221 (0.629)
	MNIST	0.165 (1.367)	0.182 (6.192)	0.021 (3.069)	0.033 (0.443)	0.909 (4.618)	0.328 (0.932)
	SVHN	0.218 (1.81)	0.109 (3.742)	0.026 (3.827)	0.046 (0.618)	0.598 (3.036)	0.305 (0.867)
		Epistemic	Aleatoric	Epistemic	Aleatoric	Epistemic	Aleatoric
SVHN	In Dist-Clean	0.061	0.01	0.004	0.083	0.097	0.107
	CIFAR	0.181 (2.992)	0.121 (11.778)	0.026 (6.029)	0.040 (0.479)	0.655 (6.765)	0.716(6.69)
	MNIST	0.198 (3.272)	0.063 (6.154)	0.024 (5.495)	0.056 (0.675)	0.329 (3.393)	0.336 (3.139)
	Fashion MNIST	0.199 (3.288)	0.113 (10.984)	0.026 (6.002)	0.041 (0.491)	0.103 (6.375)	0.125 (6.178)

Table 55.4: Out-Of-Distribution evaluation. Across all datasets, when tested on datasets outside of the training dataset, CBDL demonstrated the most consistent behavior in terms of predictive uncertainty quantification. The quantity in the parenthesis is the magnitude of increase relative to the quantity when tested on the clean dataset.

In-distribution Evaluation

Overall, CBDL demonstrates the most consistent behavior in terms of increasing levels of predictive epistemic and aleatoric uncertainties as the severity of noise corruption increases. The full table of results can be found in Caprio et al. [236, Appendix N]. In the experiments, for CIFAR-10 the single BNN also exhibits similarly consistent behavior. In MNIST and Fashion-MNIST, the BNN seems to be more consistent than CBDL, whereas for SVHN it seems to show a more inconsistent behavior. These results allow us to conclude that CBDL is comparable if not better than BNN in terms of In-Distribution behavior.

Accuracy vs Rejection Rate. As the quantities pertaining to different approaches are not comparable, we perform an additional evaluation comparing the accuracy vs the rejection rate of a given method. In this task, for a given uncertainty threshold, if the uncertainty quantity exceeds such a threshold, then the prediction is rejected. Of those not rejected, the accuracy is computed. A range of uncertainty thresholds is tested for each method and plotted. Figure 55.6 depicts an example of the plot. There, for lower severities (e.g. 1-2) of Gaussian blur

		CBDL		Ensemble of BNN		BNN	
		Epi. AUROC	Alea. AUROC	Epi. AUROC	Alea. AUROC	Epi. AUROC	Alea. AUROC
CIFAR	MNIST	**0.905**	0.826	0.805	0.104	0.860	**0.929**
	Fashion MNIST	**0.920**	0.836	0.816	0.093	0.808	**0.937**
	SVHN	**0.851**	0.823	0.789	0.117	0.826	**0.909**

		CBDL		Ensemble of BNN		BNN	
		Epi. AUROC	Alea. AUROC	Epi. AUROC	Alea. AUROC	Epi. AUROC	Alea. AUROC
MNIST	CIFAR	**0.930**	**0.912**	0.827	0.040	0.835	0.882
	Fashion MNIST	**0.956**	0.935	0.804	0.030	0.929	**0.960**
	SVHN	**0.971**	**0.926**	0.808	0.033	0.874	0.898

		CBDL		Ensemble of BNN		BNN	
		Epi. AUROC	Alea. AUROC	Epi. AUROC	Alea. AUROC	Epi. AUROC	Alea. AUROC
Fashion MNIST	CIFAR	**0.933**	0.724	0.862	0.121	0.703	**0.770**
	MNIST	0.925	0.881	0.770	0.085	**0.962**	**0.933**
	SVHN	**0.972**	0.830	0.865	0.066	0.855	**0.904**

		CBDL		Ensemble of BNN		BNN	
		Epi. AUROC	Alea. AUROC	Epi. AUROC	Alea. AUROC	Epi. AUROC	Alea. AUROC
SVHN	CIFAR	**0.979**	0.941	0.890	0.014	0.964	**0.952**
	MNIST	**0.781**	0.679	0.777	0.205	0.728	**0.709**
	Fashion MNIST	**0.967**	0.881	0.899	0.030	0.921	**0.917**

Table 55.5: Out-Of-Distribution evaluation. AUROC for OOD detection is computed using both aleatoric and epistemic predictive uncertainty measures of the different approaches. Each model is trained on the training partition of the dataset and tested on the respective testing set of the OOD dataset. When using predictive AU, a single BNN performs relatively better than CBDL for OOD detection, though CBDL is still comparable and outperforms the single BNN when trained on Fashion-MNIST. When using predictive EU, CBDL outperforms the other approaches on most datasets.

noise, CBDL exhibits better performance and converges to 100% accuracy at a lower rejection rate. Intuitively, for the same range of rejection rates, a higher curve signifies better performance, as less samples are rejected while achieving a higher accuracy. However, for higher severities (above 3), the single BNN seems to show better performance. Even so, the overall rejection rate is high to achieve such accuracy. The entirety of the results can be found in Caprio et al. [236, Appendix P]. To quantify the differences, the areas under the curve are computed and averaged for a given noise severity level. Finally, this quantity is averaged across all severity levels. The results are shown in Table 55.3. As we can see, when EBNN outperforms CBDL in specific instances, the difference is minor, and when the opposite happens, the difference is significant. Over all datasets, CBDL outperforms a single BNN.

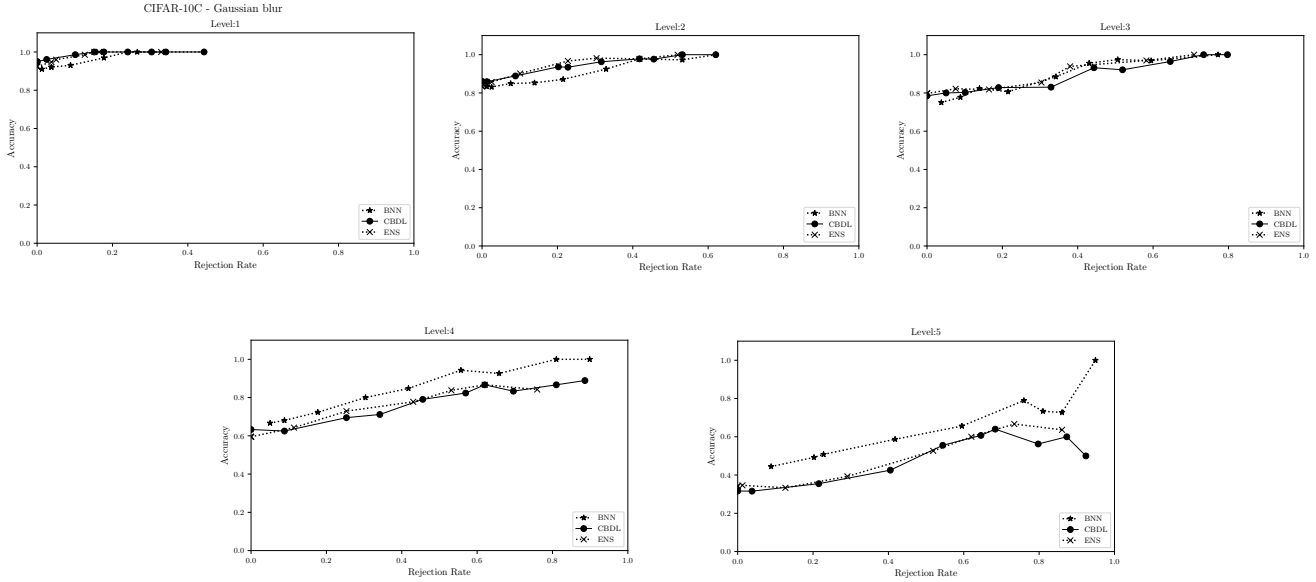


Figure 55.6: Accuracy vs Rejection Rate - CIFAR-10C Gaussian blur. For lower severities (e.g. 1-2) of Gaussian blur noise, CBDL exhibits better performance and converges to 100% accuracy at a lower rejection rate. For higher severities (above 3), the single BNN seems to show better performance. Even so, the overall rejection rate is high to achieve such accuracy. The ensemble method (EBNN) exhibits comparable performance in this specific instance.

Out-of-distribution Evaluation

In this final evaluation, we analyse the behavior of the three approaches (CBDL, EBNN, and BNN) when tested on an OOD dataset. For a given dataset, for example CIFAR, a model is trained on the dataset, and evaluated on the remaining datasets, i.e. MNIST, SVHN, and Fashion-MNIST. The relative increase in the average predictive EU and AU is computed compared to when an approach is tested on the In-Distribution testing set. The In-Distribution uncertainty quantities for each dataset are shown in Table 55.4. The relative magnitude of the increase in each quantity is shown in the parenthesis for each respective OOD dataset.

The baseline EBNN exhibits a counterintuitive behavior, viz. a decrease in predictive AU. Although this behavior is consistent across the different dataset combinations, this would be an undesirable quality for OOD detection scenarios.

The single BNN exhibits inconsistent behavior in terms of the relative increase or decrease in magnitude. For CIFAR-10, the predictive AU increases by a magnitude larger than the predictive EU, while for SVHN the increase is smaller. Moreover, for Fashion-MNIST the predictive AU decreases, and for MNIST the behavior is inconsistent.

Only CBDL demonstrates a stable, consistent increase in predictive AU, which is greater in magnitude relative to the increase in predictive EU. These results demonstrate a clear improvement in performance by CBDL.

We also give results pertaining AUROCs for the OOD detection performance, presented in Table 55.5. Each group of results shows which dataset the models are trained on, and which they are tested on. We report AUROC when using both the predictive EU and AU of each approach. In general, the baseline EBNN method performs the worst in all cases. When using predictive AU, a single BNN performs relatively better than CBDL for OOD detection, though CBDL is still comparable and outperforms it when trained on Fashion-MNIST. When using predictive EU, CBDL clearly outperforms the other approaches on most datasets. This is in line with other works where predictive EU is considered important for OOD detection [879]. We conjecture that this may be due to the fact that a single BNN is not able to gauge predictive EU properly. Hence, the single BNN flags an instance as OOD when it comes from the "tail" of the distribution; this is well-captured by (predictive) aleatoric uncertainty. On the contrary, CBDL is able to gauge predictive EU properly, and hence it is able to capture when OOD happens by looking at the "disagreement" between the elements of the predictive credal set, captured by the difference between upper and lower entropy.

55.5 Conclusion and Open Problems

We presented CBDL, a procedure that can be seen as a non-condensed, uncountably infinite ensemble of BNNs, carried out using only finitely many elements. It allows to distinguish between predictive AU and EU, and to quantify them. We showed how it can be used to specify a set of outputs – the IHDR – that enjoys probabilistic guarantees.

A region that improves on $IR_\alpha(\hat{\mathcal{P}}_{\text{pred}})$, meaning that it would be tighter, is $IR'_\alpha(\hat{\mathcal{P}}_{\text{pred}}) = \{y \in \mathcal{Y} : \underline{\hat{p}}^{\text{pred}}(y) \geq \underline{\hat{p}}^{\alpha}\}$, where $\underline{\hat{p}}^{\text{pred}} := \min_{k,s} \hat{p}^{\text{pred}}_{k,s}$, and $\underline{\hat{p}}^{\alpha}$ is the largest constant such that $\underline{\hat{P}}^{\text{pred}}[y \in IR'_\alpha(\hat{\mathcal{P}}_{\text{pred}})] \geq 1 - \alpha$. The problem with IR'_α is that, while the highest density regions $R_\alpha(\hat{P}^{\text{pred}}_{k,s})$ associated with the predictive distributions in $\text{ex}\hat{\mathcal{P}}_{\text{pred}}$ can be computed using off-the-shelf tools, calculating $\underline{\hat{p}}^{\text{pred}}$ and $\underline{\hat{p}}^{\alpha}$ would have been much more computationally expensive. In addition, it would have required to come up with a new technique to find $\underline{\hat{p}}^{\text{pred}}$ and $\underline{\hat{p}}^{\alpha}$. In turn, this remains an open problem. Another interesting endeavour is to derive a credal-evidential approach, that could allow the user to bypass the bottleneck in Algorithm 39 by merging imprecise probabilistic and evidential methods [29, 260, 413, 414, 1491].

In the future, CBDL should also be related to Bayesian Model Selection (BMS) [623], e.g. via a finite number of regularised horseshoe priors [623, Section 3.2] as extreme elements of the prior credal set, to induce shrinkage while hedging against prior misspecification. For more open problems, see Caprio et al. [236, Section 6].

Chapter 56

Bayesian Reinforcement Learning

Reinforcement learning asks a central question: how should an agent act while it is still learning about its environment so that rewards are maximised over time? The challenge appears across domains. A robot must learn locomotion without falling. A trading system must try new strategies without risking ruin. A clinical assistant must improve recommendations while caring for current patients. The tension between learning and performance is the theme of this chapter.

At the heart of this challenge lie two competing objectives. **Exploration** refers to selecting actions that gather information about unknown aspects of the environment. **Exploitation** refers to selecting actions that are expected to yield high rewards based on current knowledge. Balancing these two objectives is the central problem in learning under uncertainty. This exploration-exploitation trade-off is the core challenge that distinguishes learning problems from pure optimisation. Standard Reinforcement Learning (RL) typically defines performance with respect to the true environment, but during learning the agent does not know the transition dynamics or reward function that make this objective well defined. As a result, exploration is often added externally through heuristics such as random actions, count-based bonuses, or optimism. These mechanisms can be useful, but they do not directly represent what the agent does and does not know. Bayesian Reinforcement Learning (BRL) makes this uncertainty explicit by maintaining beliefs over plausible environments and using those beliefs to guide decisions. Uncertainty is therefore not only a source of error to be eliminated, but a resource for deciding which actions are worth taking. Consider a hospital delivery robot that moves among patient rooms, closets, and restricted areas. Heuristic exploration can be disruptive and uninformative. A Bayesian view directs exploration to places where learning will most improve future performance while avoiding actions that add little information. In short, experiments are chosen for what they will change downstream.

Not all uncertainty merits exploration. Some can be reduced by collecting data and therefore warrants directed exploration, other forms are inherent and cannot be reduced by collecting more data.

56.0.1 Why Traditional Exploration Often Fails

A common exploration strategy, ϵ-greedy, selects random actions with fixed probability. It treats all unknowns as equally promising and ignores where information matters. Optimism in the face of uncertainty adds bonuses for rarely visited states, which improves coverage but still does not measure the value of what is learned or the cost of risky probes.

Such methods can be costly in risky domains. A hospital robot driven by ϵ-greedy may enter operating theatres at the wrong time, disrupting procedures without learning anything useful. A medical recommender might randomise treatments in cases where learning yields little benefit. A Bayesian approach prioritizes experiments that can change future choices and avoids those that are unlikely to influence policy.

56.0.2 Historical Development: From Bandits to Full Environments

The Bayesian view of exploration under uncertainty traces to Thompson [1603], who balanced helping current patients with learning for future patients. This led to the multi-armed bandit model and Thompson sampling, which maintains beliefs over payoffs and samples actions according to those beliefs. Briefly, in the multi-armed bandit problem an agent repeatedly selects among K actions (arms), each yielding stochastic rewards from an unknown distribution, seeking to maximise cumulative reward while learning which arms are best.

RL extends bandits by introducing a state variable s. Actions a modify the current state, yield a reward r, which couples exploration with future action combinations. Duff [460] formalized this in the Bayes-adaptive MDP,

where beliefs about the unknown environment are part of the state. The environment represents the external system with which the agent interacts, generating state transitions and rewards in response to actions. The formulation is complete but computationally demanding, which motivates the approximations developed in this chapter. A useful bridge is posterior sampling for RL, which samples a Markov decision process (MDP) from the current belief, solves it, then executes the resulting policy for a period before resampling, thereby turning belief updates into an implicit exploration schedule. The policy defines the agent's behaviour by assigning a probability distribution over actions for each state.

56.0.3 The Bayesian Framework: Beliefs and Updates

BRL represents knowledge about the environment with probability distributions. Let the unknown environment be a random variable $\mathcal{M}$ with prior $p(\mathcal{M})$. As experience accrues we update beliefs via Bayes rule,

$$p(\mathcal{M} \mid \text{experience}) = \frac{p(\text{experience} \mid \mathcal{M})\, p(\mathcal{M})}{p(\text{experience})}. \tag{56.1}$$

This turns experience into updated beliefs and makes uncertainty explicit and operable.

Experience can be written as a trajectory $\tau_t = \{s_0, a_0, r_0, s_1, \ldots, s_t\}$ with likelihood

$$p(\tau_t \mid \mathcal{M}) = \rho_{\mathcal{M}}(s_0) \prod_{i=0}^{t-1} P_{\mathcal{M}}(s_{i+1} \mid s_i, a_i)\, p_{\mathcal{M}}(r_i \mid s_i, a_i), \tag{56.2}$$

which reflects the Markov property.

56.0.4 The Mathematical Structure of Sequential Decision Making

We model RL problems as MDPs with state space $\mathcal{S}$, action space $\mathcal{A}$, transitions $P(s' \mid s, a)$, rewards $r(s, a)$, discount $\gamma \in [0, 1)$, and initial distribution ρ_0. The policy $\pi(a \mid s)$ maximises expected discounted return $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$.

Value functions describe optimal behaviour. The action value function satisfies

$$Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a)\, V^\pi(s') \quad \text{with} \quad V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a). \tag{56.3}$$

When the MDP is known, dynamic programming solves these equations. In BRL expectations are taken under beliefs rather than a single model, which couples planning with learning and makes value a function of both physical state and knowledge state.

56.0.5 Illustrative Example: Hazardous Navigation

A hospital robot in a 10×10 grid begins with uncertain room types: patient rooms, supply closets, restricted areas, or open corridors. Each room type has different transition dynamics and reward structures that the robot must learn through experience. As it navigates and observes outcomes, it updates its beliefs about room categories using Bayesian inference. Observations that confirm or contradict expectations shift probability mass between hypotheses; after repeated visits, the posterior concentrates around the true room type, while unvisited locations retain high uncertainty.

The core challenge is deciding how to act while simultaneously learning the environment. At each decision point, the robot evaluates actions along two dimensions. First, it computes expected immediate return under current beliefs by averaging over possible environment configurations weighted by their posterior probability. Second, it estimates how much each action will reduce uncertainty about the environment, and how this knowledge reduction translates to better decisions in future episodes. From a doorway the robot can test an unknown room or take a safe corridor. The unknown room may have low immediate reward but high expected information value, which can shorten many future routes by revealing a fast passage. When time is critical it exploits reliable paths; when time allows it explores regions where resolving uncertainty is most useful. Because beliefs retain variance, BRL can reason about risk and reduce the chance of rare but costly outcomes through targeted information gathering.

Suppose the robot reaches a junction with two options: a corridor whose properties are well understood from many previous traversals, or an unexplored room whose type remains uncertain. Taking the known corridor provides predictable progress toward the goal. Entering the unknown room provides information about its type, which affects optimal routing decisions across all future episodes. If the room turns out to contain a shortcut, the robot can exploit this discovery repeatedly. If it is a restricted area, the robot learns to avoid it. The value

of exploration depends on how many future decisions will benefit from resolving this uncertainty. This trade-off between immediate performance and information gathering depends on temporal context. When the robot must deliver urgent medication, exploiting known reliable paths maximises the probability of timely delivery. During routine supply transport with relaxed time constraints, exploring uncertain regions provides information that improves long-term efficiency. The optimal balance between exploration and exploitation emerges from explicitly modelling how observations today affect decision quality tomorrow.

The maintained probability distributions also enable reasoning about risk. Rather than committing to a single best guess about room types, the robot tracks uncertainty through posterior variance. High variance in a region's posterior distribution indicates that outcomes in that region are unpredictable under current knowledge. The robot can identify actions where this uncertainty creates risk of rare but costly outcomes, such as accidentally entering an operating theatre during a procedure. By reasoning about the full distribution rather than only expected values, the robot distinguishes between low expected reward due to genuine low value versus low expected reward due to high uncertainty about potentially high value. The subsequent sections formalize these concepts, beginning with exact Bayesian methods for problems where posterior distributions can be computed analytically, then progressing to approximate methods that scale to complex environments.

56.0.6 Information as a Resource

Actions generate reward now and information for later. One measure of expected information value from action a in state s is the expected Kullback-Leibler divergence between posterior and prior beliefs,

$$\mathcal{I}_t(s,a) = \mathbb{E}_{s',r}[\mathrm{KL}(p(\mathcal{M} \mid \tau_t \cup \{s,a,r,s'\}) \,\|\, p(\mathcal{M} \mid \tau_t))]\,. \tag{56.4}$$

This quantity predicts how much a decision can change future behaviour and is the target that practical approximations try to mimic efficiently. Random or count-based exploration does not estimate this quantity, which explains wasted samples in regions that do not influence policy.

56.1 Exact Bayesian methods: the tabular foundation

Exact Bayesian inference provides a theoretical gold standard for RL under uncertainty. In tabular domains with conjugate priors, we can maintain exact posterior distributions and compute optimal exploration strategies without approximation. These methods establish fundamental principles that all subsequent approximations attempt to preserve while scaling to more complex domains.

This section develops three foundational approaches. Posterior sampling (PSRL) demonstrates how to sample from belief distributions to guide exploration. Bayes-Adaptive Monte Carlo Planner (BAMCP) shows how to plan optimally when exact dynamic programming becomes intractable. Bayesian Exploration Bonuses (BEB) illustrate how to approximate information value through explicit reward augmentation. Together, these methods establish the theoretical framework that guides all modern Bayesian RL algorithms.

56.1.1 Posterior Sampling for Reinforcement Learning (PSRL)

Posterior sampling provides the most direct implementation of Bayesian exploration principles. Rather than attempting to solve the computationally intractable Bayes-adaptive MDP exactly, PSRL samples a single environment from the posterior distribution and acts optimally within that sampled environment. This simple procedure captures the essence of uncertainty-driven exploration while remaining computationally tractable.

Mathematical Foundation

Consider a tabular MDP where states $\mathcal{S} = \{1, 2, \ldots, S\}$ and actions $\mathcal{A} = \{1, 2, \ldots, A\}$ are discrete and finite. The unknown transition probabilities $P(s'|s,a)$ and reward function $r(s,a)$ must be learned from experience.

We place conjugate priors over unknown parameters to enable exact Bayesian inference. For transition probabilities, we use Dirichlet priors:

$$P(\cdot|s,a) \sim \mathrm{Dirichlet}(\alpha_{s,a,1}, \alpha_{s,a,2}, \ldots, \alpha_{s,a,S}) \tag{56.5}$$

The hyperparameters $\alpha_{s,a,s'}$ represent prior counts for transitions from state s to state s' via action a. For rewards, we assume Gaussian distributions with known variance but unknown means:

$$r(s,a) \sim \mathcal{N}(\mu_{s,a}, \sigma_r^2) \tag{56.6}$$

with normal priors over the means:

$$\mu_{s,a} \sim \mathcal{N}(\mu_0, \sigma_0^2) \tag{56.7}$$

After observing trajectory data $\tau_t = \{s_0, a_0, r_0, s_1, \ldots, s_t\}$, we update beliefs using conjugacy properties. Let $N_{s,a,s'}(t)$ denote the count of transitions from state s to state s' via action a up to time t. The posterior over transition probabilities becomes:

$$P(\cdot|s,a)|\tau_t \sim \text{Dirichlet}(\alpha_{s,a,1} + N_{s,a,1}(t), \ldots, \alpha_{s,a,S} + N_{s,a,S}(t)) \tag{56.8}$$

For reward parameters, let $\bar{r}_{s,a}(t)$ denote the sample mean reward and $n_{s,a}(t)$ the visit count for state-action pair (s,a). The posterior over reward means becomes:

$$\mu_{s,a}|\tau_t \sim \mathcal{N}\left(\frac{\sigma_r^2\mu_0 + n_{s,a}(t)\sigma_0^2\bar{r}_{s,a}(t)}{\sigma_r^2 + n_{s,a}(t)\sigma_0^2}, \frac{\sigma_r^2\sigma_0^2}{\sigma_r^2 + n_{s,a}(t)\sigma_0^2}\right) \tag{56.9}$$

These posterior distributions capture our uncertainty about environment parameters. Initially, when visit counts are low, posteriors remain close to priors. As data accumulates, posteriors concentrate around observed frequencies and sample means.

Theoretical Guarantees

Osband et al. [1268] prove that PSRL achieves Bayesian regret $\tilde{O}(HS\sqrt{AT})$ where H is episode length, S is number of states, A is number of actions, and T is total time steps. This bound is order optimal, meaning no algorithm can achieve fundamentally better scaling without additional assumptions.

The $\sqrt{T}$ dependence indicates sublinear regret growth, so average per-step regret approaches zero as $T \to \infty$. The $S\sqrt{A}$ dependence reflects the exploration cost: larger state-action spaces require more exploration to identify optimal actions. The episodic commitment structure proves essential for these guarantees [1267].

Bayesian regret measures the expected cumulative performance gap between the algorithm and the optimal policy, with the expectation taken over the prior distribution over environments. In the episodic setting, let H denote the episode length, let E denote the number of episodes, and let $T = EH$ be the total number of interaction steps. If $s_{e,0}$ is the initial state of episode e, the Bayesian regret can be written as:

$$\text{BayesRegret}(T) = \mathbb{E}_{M\sim p(M)}\left[\sum_{e=1}^{E}\left(V_H^{*,M}(s_{e,0}) - \mathbb{E}_{\pi_e}\left[\sum_{h=0}^{H-1} r_{e,h}\,\middle|\, M\right]\right)\right], \tag{56.10}$$

where M represents the true environment sampled from prior $p(M)$, $V_H^{*,M}(s_{e,0})$ is the optimal value function in environment M starting from initial state $s_{e,0}$, and the inner expectation is over trajectories generated by the algorithm's policy π.

The PSRL Algorithm

Algorithm 40 presents the complete PSRL procedure, implementing the theoretical framework developed by Strens [1569] and analyzed by Osband et al. [1268]. The algorithm operates episodically, committing to a single sampled environment for an entire episode before resampling—a design choice that proves necessary for achieving optimal regret guarantees [1267]. The episodic structure addresses a fundamental challenge: how to maintain temporal consistency in exploration strategies.

The algorithm decomposes into four distinct phases that repeat each episode. Phase 1 implements Bayesian learning by updating posterior distributions using conjugacy relationships. Phase 2 implements Thompson's principle: sample one plausible environment and act optimally within it. Phase 3 solves the standard MDP using value iteration. Value iteration is a dynamic programming algorithm that computes the optimal value function and policy for a known MDP by repeatedly applying the Bellman optimality update $V_{k+1}(s) = \max_a [r(s,a) + \gamma \sum_{s'} P(s'|s,a)V_k(s')]$ until convergence. In the context of PSRL, once an environment is sampled from the posterior distribution, value iteration solves this sampled MDP to obtain the optimal policy for that specific environment hypothesis. Phase 4 executes the policy and collects experience through interaction with the real environment. PSRL's exploration behaviour emerges from the interaction between posterior sampling and optimal decision-making according to the sampled environment hypothesis. When visit counts are low, posterior distributions remain diffuse with high variance. Sampling from these distributions produces diverse environment hypotheses, leading to exploratory policies. As evidence accumulates, posteriors concentrate around true parameter values, and sampled environments become increasingly similar to the true environment, leading to exploitation of accumulated knowledge.

Algorithm 40: Posterior Sampling for Reinforcement Learning (PSRL)

Data: Prior parameters α, μ_0, σ_0^2; episode length H; number of episodes E
Initialize:
Set $N_{s,a,s'} = 0$ for all (s, a, s') // transition counts
Set $R_{s,a} = 0$, $n_{s,a} = 0$ for all (s, a) // reward sums, visit counts
for *episode* $e = 1, 2, \ldots, E$ **do**
  STEP 1: Update posterior beliefs from data
  for *each state-action pair* (s, a) **do**
    $\alpha^{\text{post}}_{s,a,s'} = \alpha_{s,a,s'} + N_{s,a,s'}$ for all s' // posterior Dirichlet parameters
    if $n_{s,a} > 0$ **then**
      $\bar{r}_{s,a} = R_{s,a}/n_{s,a}$ // sample mean reward
      Update $\mu^{\text{post}}_{s,a}$, $\sigma^{\text{post}}_{s,a}$ via Eq. (56.9) // posterior mean and variance
    else
      $\mu^{\text{post}}_{s,a} = \mu_0$, $\sigma^{\text{post}}_{s,a} = \sigma_0$ // unvisited state-action pairs use priors
  STEP 2: Sample one plausible environment
  for *each state-action pair* (s, a) **do**
    $\tilde{P}(\cdot|s, a) \sim \text{Dirichlet}(\alpha^{\text{post}}_{s,a,1}, \ldots, \alpha^{\text{post}}_{s,a,S})$ // sample transitions
    $\tilde{r}(s, a) \sim \mathcal{N}(\mu^{\text{post}}_{s,a}, (\sigma^{\text{post}}_{s,a})^2)$ // sample rewards
  STEP 3: Solve MDP with sampled parameters
  $\pi^{(e)} = \text{ValueIteration}(\tilde{P}, \tilde{r}, \gamma)$ // compute optimal policy
  STEP 4: Execute policy and collect experience
  $s = s_0$ // start at initial state
  for *step* $h = 1$ *to* H **do**
    $a = \pi^{(e)}(s)$ // select action from policy
    Execute action a, observe s' and r // interact with real environment
    $N_{s,a,s'} = N_{s,a,s'} + 1$ // update transition count
    $R_{s,a} = R_{s,a} + r$ // update reward sum
    $n_{s,a} = n_{s,a} + 1$ // update visit count
    $s = s'$ // transition to next state

56.1.2 Bayes-Adaptive Monte Carlo Planner

While PSRL provides optimal exploration guarantees, it requires exact value iteration that becomes intractable in large state spaces. Bayes-Adaptive Monte Carlo Planner [682] extends the posterior sampling principle to domains where exact planning is infeasible by combining root sampling with Monte Carlo Tree Search (MCTS).

Motivation and Computational Challenge

The exact Bayes-adaptive approach requires planning in the augmented state space (s, b) where b represents the current belief state over environments. This augmented space grows exponentially with the original state space size, making exact planning intractable even for moderate tabular domains.

BAMCP resolves this intractability through root sampling: instead of planning over the continuous belief space, the algorithm samples multiple environments from the current posterior distribution and performs standard MCTS planning within each sampled environment. This approach leverages the law of total expectation:

$$Q^*(s, a, b) = \mathbb{E}_{\mathcal{M}\sim b}[Q^*_{\mathcal{M}}(s, a)] \approx \frac{1}{K}\sum_{k=1}^{K} Q^*_{\mathcal{M}^{(k)}}(s, a), \tag{56.11}$$

where $Q^*(s, a, b)$ is the Bayes-optimal action-value function that represents the expected optimal Q-value under the posterior belief b over environments, $Q^*_{\mathcal{M}^{(k)}}(s, a)$ is the optimal Q-value for taking action a in state s under the specific sampled environment $\mathcal{M}^{(k)}$, $\mathcal{M}^{(k)}$ are independent samples from posterior belief b, and K is the number of sampled environments.

Convergence Properties

BAMCP inherits convergence guarantees from both MCTS and posterior sampling [683]. As simulation budget K increases, action value estimates converge to their true expectations under the posterior belief.

Theorem 56.1 (BAMCP Convergence)**.** *Let $Q_b^*(s,a) = \mathbb{E}_{\mathcal{M}\sim b}[Q_{\mathcal{M}}^*(s,a)]$ denote the true expected action value under posterior belief b. Then with probability $1-\delta$:*

$$|Q_K(a) - Q_b^*(s,a)| \leq \sqrt{\frac{V_{\max}\log(1/\delta)}{2n_a(K)}}, \tag{56.12}$$

where $Q_K(a)$ is BAMCP's estimate after K simulations, $n_a(K)$ is the number of times action a was selected, and $V_{\max} = R_{\max}/(1-\gamma)$ is an upper bound on the discounted return when rewards are bounded by $R_{\max}$.

The convergence rate depends on how frequently each action is selected during simulations. Actions that appear optimal in many sampled environments receive more simulations through adaptive allocation of the Upper Confidence Bound (UCB) algorithm [61], leading to more accurate value estimates. The UCB algorithm balances exploration and exploitation by selecting actions that maximise $\bar{x}_a + \sqrt{\frac{2\ln n}{n_a}}$, where $\bar{x}_a$ is the empirical mean reward, n is the total number of trials, and n_a is the number of times action a has been selected. This exploration bonus ensures that under-explored actions receive attention while favoring actions with high observed rewards.

Relationship to PSRL

BAMCP shares the root sampling principle with PSRL but differs in planning methodology and temporal commitment. PSRL commits to one sampled environment per episode and computes globally optimal policies using exact methods. BAMCP samples multiple environments per decision and uses MCTS to approximate optimal actions through simulation-based planning.

The choice between PSRL and BAMCP depends on problem characteristics. Use PSRL when exact planning is feasible, episodic structure is natural, and temporal consistency is important. Use BAMCP when state spaces are large, online planning is required, or computational budgets are limited. Both methods implement the same underlying Bayesian principle of sampling plausible environments and acting optimally within those environments.

The BAMCP Algorithm

Algorithm 41 presents the complete BAMCP procedure. The algorithm performs online planning at each decision point by sampling multiple environments and running MCTS simulations within each sampled environment. The algorithm decomposes into three phases that repeat for each simulation. Phase 1 samples one complete environment from the posterior belief using conjugate sampling procedures. Phase 2 executes one complete MCTS simulation in the sampled environment, treating it as the true environment. Phase 3 updates action value estimates and visit counts using incremental averaging.

Algorithm 41: Bayes-Adaptive Monte Carlo Planner

Data: Current state s, posterior belief b, simulation budget K
Data: Exploration constant c, rollout policy π_{rollout}, discount factor γ, horizon H
Initialize:
Set $Q(a) = 0$ and $N(a) = 0$ for all $a \in \mathcal{A}$ `// action values`
for *simulation* $k = 1, 2, \ldots, K$ **do**
  STEP 1: Sample environment from posterior
  $\tilde{\mathcal{M}}^{(k)} \leftarrow \text{SampleEnvironment}(b)$ `// e.g., PSRL-style c`
  STEP 2: MCTS simulation in sampled environment
  $(a^{(k)}, R^{(k)}) \leftarrow \text{MCTS_SIM}\big(s, \tilde{\mathcal{M}}^{(k)}, c, \pi_{\text{rollout}}, \gamma, H\big)$
  STEP 3: Update action statistics
  $N(a^{(k)}) \leftarrow N(a^{(k)}) + 1$
  $Q(a^{(k)}) \leftarrow Q(a^{(k)}) + \dfrac{R^{(k)} - Q(a^{(k)})}{N(a^{(k)})}$ `// increm`
Return $\arg\max_a Q(a)$ `// action with high`

56.1.3 Bayesian Exploration Bonuses (BEB)

Although PSRL and BAMCP provide principled approaches to Bayesian exploration, they require episodic commitment or online planning that may be computationally expensive. Bayesian Exploration Bonuses [913] offer an alternative by augmenting immediate rewards with terms that approximate information value, enabling integration with standard value-based RL algorithms.

Information-Theoretic Foundation

BEB approximates information value using a computationally tractable heuristic. The theoretical foundation builds on expected information gain. Let $\mathcal{M}$ represent a specific environment instance and $\tau_t = \{s_0, a_0, r_0, s_1, \ldots, s_t\}$ denote observed trajectory. Taking action a in state s provides information gain measured by expected change in posterior beliefs:

$$\mathcal{I}_t(s,a) = \mathbb{E}_{s',r\sim p(\cdot|s,a,\tau_t)}\left[\mathrm{KL}\left(p(\mathcal{M}|\tau_t \cup \{s,a,r,s'\}) \| p(\mathcal{M}|\tau_t)\right)\right] \tag{56.13}$$

For tabular MDPs with conjugate priors, information gain can be computed analytically. However, this computation remains expensive. BEB approximates information gain using a simple heuristic based on visit counts.

The Exploration Bonus Heuristic

BEB approximates expected information gain using the exploration bonus:

$$b_t(s,a) = \frac{\beta}{1 + n_{s,a}(t)}, \tag{56.14}$$

where $n_{s,a}(t)$ denotes the accumulated count for state-action pair (s,a), including prior pseudo-counts when using the Dirichlet-belief notation of [913], and $\beta > 0$ controls the strength of exploration.

The $1/(1+n)$ decay reflects the BEB analysis: under Dirichlet beliefs, the effect of one additional observation on the posterior transition model decreases on the order of $1/n$. This makes BEB greedier than PAC-MDP bonuses such as MBIE-EB, which use $1/\sqrt{n}$-type bonuses and therefore encourage more persistent exploration.

This implements the principle of "optimism in the face of uncertainty" [61]. State-action pairs with low visit counts receive large bonuses, making them appear artificially attractive and encouraging exploration. As visit counts increase, bonuses decrease, naturally shifting behaviour toward exploitation.

Integration with Temporal Difference (TD) Learning

BEB integrates with standard temporal difference learning algorithms by treating exploration bonuses as additional reward signals. The agent selects actions to maximise augmented Q-values:

$$a_t = \arg\max_a \left[Q_t(s_t,a) + b_t(s_t,a)\right], \tag{56.15}$$

where $Q_t(s,a)$ represents current value estimates learned via Q-learning, SARSA, or other methods.

The exploration bonus affects action selection but not the Q-learning update itself. This separation ensures Q-values learn about actual environmental rewards while bonuses guide exploration toward uncertain regions.

Theoretical Guarantees

BEB provides theoretical guarantees by showing that, after a polynomial number of exploratory steps, its policy is near-optimal relative to the optimal Bayesian policy for the current belief state [913].

Theorem 56.2 (BEB Regret Bound)**.** *Let A_t denote the policy followed by BEB at time t, with $\beta = 2H^2$. Suppose the belief for a state-action pair is no longer updated once its total count exceeds $4H^3/\epsilon$. Then, with probability at least $1-\delta$,*

$$V_H^{A_t}(b_t, s_t) \geq V_H^*(b_t, s_t) - \epsilon \tag{56.16}$$

for all but

$$O\left(\frac{|\mathcal{S}||\mathcal{A}|H^6}{\epsilon^2}\log\frac{|\mathcal{S}||\mathcal{A}|}{\delta}\right) \tag{56.17}$$

time steps.

This guarantee is not a standard regret bound against the optimal policy of a fixed true MDP. Instead, it states that BEB is near-optimal relative to the optimal Bayesian policy for the current belief state, except for a polynomial number of time steps. This distinction matters: BEB can use a greedier bonus than PAC-MDP algorithms because it aims to approximate Bayesian exploration rather than guarantee near-optimality for every possible underlying MDP.

56.1.4 Synthesis and Practical Guidance

These three methods represent different points in the trade-off between statistical rigor and computational efficiency.

PSRL provides the most principled approach when episodes have natural boundaries, planning in sampled environments is tractable, strong theoretical guarantees are important, and domains have moderate state-action spaces ($< 10^4$ pairs).

BAMCP works best when exact planning is intractable but simulation is feasible, online planning budgets are available at decision time, domains require sophisticated forward search, and belief state representations are complex but sampling is straightforward.

BEB offers practical advantages when real-time decision making is required, integration with existing value-based methods is desired, computational resources are severely constrained, and domain structure makes visit count approximations reasonable.

The key insight from exact methods is that optimal exploration requires commitment to hypotheses for extended periods [1267]. Random or myopic exploration fails because it cannot sustain consistent action sequences needed to test complex hypotheses about environment structure. Whether implemented through posterior sampling (PSRL), planning within sampled environments (BAMCP), or information-driven bonuses (BEB), this commitment principle guides all subsequent approximation methods.

56.2 Gaussian Process Methods: Nonparametric Bayesian RL for Continuous Spaces

Gaussian process (GP) methods extend BRL to continuous state and action spaces through nonparametric function approximation. Unlike linear methods that assume fixed basis functions, Gaussian processes define probability distributions over function spaces, enabling flexible representations that adapt to data complexity while maintaining principled uncertainty quantification.

This section examines GP-SARSA for value function learning, PILCO for model-based policy search, and briefly discusses GP-based Model Predictive Control (MPC) for safe planning under uncertainty. These methods establish the theoretical framework for nonparametric function approximation while providing practical algorithms that achieve data efficiency through uncertainty-guided exploration.

56.2.1 From parametric to Nonparametric Function Approximation

Moving beyond tabular methods, Gaussian process approaches increase flexibility by modelling value functions or dynamics directly in function space, at the cost of greater computational complexity.

Limitations of Parametric Approaches

Linear methods assume value functions can be written as $V(s) = \phi(s)^\top \theta$ where $\phi(s) \in \mathbb{R}^d$ represents feature vectors. This reduces parameters from $|\mathcal{S}|$ to d, enabling tractable learning when $d \ll |\mathcal{S}|$. However, if the chosen features cannot represent the true value function, no amount of data can overcome this representational bias.

Gaussian processes address this by placing priors over function spaces rather than parameter spaces, adapting their representation based on observed data. This flexibility comes at a computational cost: exact inference scales as $O(n^3)$ where n is the number of observations, compared to $O(d^2)$ for linear methods.

Gaussian Processes as Distributions over Functions

As introduced in Chapter 23, a Gaussian process is a Bayesian model that represents uncertainty over functions. It defines a distribution where any set of function values is jointly Gaussian, allowing us to predict both expected outputs and their uncertainty for new inputs.

56.2.2 GP-SARSA: Value Function Learning with Gaussian Processes

GP-SARSA extends temporal difference learning to continuous spaces using Gaussian processes to represent action-value functions. The method learns $Q(s, a)$ as a Gaussian process over the joint state-action space.

Given a transition $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$, the TD target is computed as:

$$y_t = r_t + \gamma \tilde{Q}(s_{t+1}, a_{t+1}), \tag{56.18}$$

where $\tilde{Q}(s_{t+1}, a_{t+1})$ is sampled from the GP posterior $\mathcal{N}(\mu_Q(s_{t+1}, a_{t+1}), \sigma_Q^2(s_{t+1}, a_{t+1}))$ for Thompson sampling exploration, or alternatively set to the posterior mean $\mu_Q(s_{t+1}, a_{t+1})$ for deterministic bootstrapping. The stochastic sampling variant enables natural exploration through posterior uncertainty, while the mean variant provides lower-variance targets at the cost of reduced exploration. This target forms a training pair (x_t, y_t) where $x_t = (s_t, a_t)$ represents the concatenated state-action input.

The GP posterior after observing n transitions provides:

$$\mu_Q(s, a) = \mathbf{k}^\top (K + \sigma_n^2 I)^{-1} \mathbf{y} \tag{56.19}$$

$$\sigma_Q^2(s, a) = k((s, a), (s, a)) - \mathbf{k}^\top (K + \sigma_n^2 I)^{-1} \mathbf{k}, \tag{56.20}$$

where $\mathbf{k}$ contains covariances between (s, a) and observed state-action pairs, K represents the gram matrix of observed pairs, and $\mathbf{y}$ contains TD targets.

The uncertainty $\sigma_Q^2(s, a)$ enables exploration through Thompson sampling or upper confidence bounds. Actions are selected by:

$$a_t = \arg\max_a \left[\mu_Q(s_t, a) + \beta \sigma_Q(s_t, a)\right], \tag{56.21}$$

where $\beta > 0$ controls exploration intensity, balancing exploitation of high-value actions against exploration of uncertain regions.

Exploration mechanisms. GP-SARSA supports multiple exploration strategies leveraging posterior uncertainty. Thompson sampling draws $\tilde{Q}(s, a)$ from the GP posterior and acts greedily with respect to this sample, naturally trading off exploration and exploitation through posterior sampling. The UCB exploration, shown in the equation above, adds $\beta \sigma_Q(s, a)$ to the mean, with β controlling optimism about uncertain regions. The UCB strategy can be interpreted as optimism in the face of uncertainty: actions with high uncertainty receive exploration bonuses proportional to their standard deviation. These strategies share a common principle: uncertainty guides exploration toward regions where the agent can most improve its knowledge, enabling more efficient learning than ϵ-greedy approaches that explore uniformly regardless of epistemic uncertainty.

Algorithm 42: GP-SARSA with UCB Exploration

Data: Initial state s_0, discount factor γ, exploration parameter β, observation noise σ_n^2
Data: Covariance function $k(\cdot, \cdot)$
Initialize dataset $\mathcal{D} = \emptyset$
Select initial action a_0 using exploration strategy
for *episode* $= 1, 2, \ldots$ **do**
  Initialize state s_t, select action a_t
  while *not terminal* **do**
    Execute action a_t, observe r_t and s_{t+1}
    Select $a_{t+1} = \arg\max_a \left[\mu_Q(s_{t+1}, a) + \beta \sigma_Q(s_{t+1}, a)\right]$
    Compute TD target $y_t = r_t + \gamma \mu_Q(s_{t+1}, a_{t+1})$
    Add transition to dataset $\mathcal{D} \leftarrow \mathcal{D} \cup \{((s_t, a_t), y_t)\}$
    Update GP posterior:
    Compute K with entries $K_{ij} = k((s_i, a_i), (s_j, a_j))$ for all pairs in $\mathcal{D}$
    For any (s, a), compute $\mu_Q(s, a) = \mathbf{k}^\top (K + \sigma_n^2 I)^{-1} \mathbf{y}$
    For any (s, a), compute $\sigma_Q^2(s, a) = k((s, a), (s, a)) - \mathbf{k}^\top (K + \sigma_n^2 I)^{-1} \mathbf{k}$
    $s_t \leftarrow s_{t+1}$, $a_t \leftarrow a_{t+1}$

56.2.3 PILCO: Probabilistic Inference for Learning Control

PILCO achieves sample-efficient learning through model-based policy search with Gaussian process dynamics models. The method learns probabilistic models of system dynamics and analytically propagates uncertainty

through time to optimize policies.

Probabilistic Dynamics Models

PILCO models state transitions as:

$$s_{t+1} = f(s_t, a_t) + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \Sigma_\epsilon), \tag{56.22}$$

where $f(\cdot, \cdot) \sim \mathcal{GP}(0, k_f)$ follows a Gaussian process and Σ_ϵ represents process noise covariance. When the state $s_t \in \mathbb{R}^D$ is a vector with D dimensions, PILCO learns D independent Gaussian process models $\{f_i\}_{i=1}^D$, one for each dimension of the next state: $s_{t+1,i} = f_i(s_t, a_t) + \epsilon_{t,i}$ where $f_i \sim \mathcal{GP}(0, k_{f_i})$ and $\epsilon_{t,i} \sim \mathcal{N}(0, \sigma^2_{\epsilon,i})$.

Policy optimisation through Moment Matching

The policy optimisation objective maximises expected cumulative reward:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} r(s_t, a_t) \right], \tag{56.23}$$

where the expectation is taken over trajectories generated by policy π interacting with the learned dynamics model.

Policy parameterization and gradient computation. PILCO typically uses Gaussian policies $\pi(a|s) = \mathcal{N}(a; \mu_\pi(s), \Sigma_\pi)$ where the mean $\mu_\pi(s)$ is a parameterized function of state, often implemented as a radial basis function network or linear controller. Given a Gaussian belief over the current state $p(s_t) = \mathcal{N}(s_t; \mu_t, \Sigma_t)$, the action distribution is computed by marginalizing over state uncertainty. The resulting state distribution at $t+1$ after applying the GP dynamics is approximated through moment matching: the exact (non-Gaussian) distribution resulting from pushing the Gaussian state-action distribution through the nonlinear GP mean function is projected onto a Gaussian with matching first and second moments. Policy gradients $\nabla_\theta J(\pi_\theta)$ are computed by backpropagating through this chain of moment computations, enabling gradient-based optimisation without costly Monte Carlo rollouts. This analytical gradient computation is PILCO's key advantage, though it requires careful implementation of moment matching derivatives and is limited to scenarios where the Gaussian approximation remains accurate.

PILCO's key contribution is analytical computation of this expectation through moment matching. When the policy is Gaussian and the GP dynamics model uses specific kernel choices (such as squared exponential), the distribution over states at each time step can be approximated as Gaussian. By propagating only the mean and covariance of these state distributions through the GP dynamics model and reward function, PILCO computes expected returns and their gradients in closed form, avoiding expensive trajectory sampling. This analytical approach enables gradient-based policy optimisation with dramatically fewer environment interactions than model-free methods, because PILCO uses the learned GP model to simulate trajectories and compute policy gradients entirely in silico, only requiring actual environment interactions for collecting data to train the dynamics model.

56.2.4 Connection to Uncertainty Types and Exploration

As mentioned previously, in Bayesian methods, total uncertainty decomposes into two fundamental types: epistemic uncertainty and aleatoric uncertainty, the latter being irreducible uncertainty inherent to the environment's stochastic nature. This decomposition is key for principled exploration because only epistemic uncertainty justifies collecting additional data—aleatoric uncertainty persists regardless of how much data is collected and instead requires robust planning methods. Gaussian process methods preserve this fundamental uncertainty decomposition while extending to continuous spaces. The GP posterior variance $\sigma^2(x)$ captures epistemic uncertainty about function values that decreases as relevant data accumulates. This uncertainty exhibits distance-dependent decay near observations, growth in unexplored regions, and data-dependent reduction as informative observations accumulate.

The observation noise variance σ_n^2 represents aleatoric uncertainty reflecting environmental stochasticity and measurement errors that persist regardless of model accuracy. Distinguishing epistemic and aleatoric uncertainty enables appropriate exploration: epistemic uncertainty justifies additional data collection, while aleatoric uncertainty requires robust planning methods.

56.3 Neural Network Approximations: Scaling Bayesian RL to High-Dimensional Problems

The Gaussian process methods in the previous section provide principled uncertainty quantification but face computational challenges in high-dimensional spaces. Computing kernel matrices scales cubically with dataset size, and the inverse operations required for prediction become prohibitive for problems with thousands of observations. Neural network approximations address these limitations by replacing exact inference with scalable optimisation-based methods. These approaches maintain the core Bayesian principle—quantifying uncertainty to guide exploration—while sacrificing theoretical guarantees for practical applicability to complex domains like robotic control and game playing.

This section examines two complementary approaches to neural function approximation in Bayesian RL. Variational inference methods provide principled posterior approximations through optimisation, explicitly representing distributions over network parameters and computing gradients through the evidence lower bound. Ensemble methods approximate posterior uncertainty through multiple independently trained networks, sacrificing theoretical rigor for computational simplicity and robust empirical performance. Both approaches preserve the fundamental exploration mechanisms from earlier sections—Thompson sampling and upper confidence bounds—while enabling application to problems beyond the reach of exact methods.

56.3.1 From Exact Inference to Neural Approximation

To understand how neural networks fit into the Bayesian framework, consider the value function approximation problem. In earlier sections we represented $Q(s, a)$ exactly through tables or nonparametrically through Gaussian processes. Neural networks instead parameterize the value function as $Q_\theta(s, a)$ where $\theta \in \mathbb{R}^p$ denotes the weights and biases across all network layers. For a network with p parameters, this vector might contain millions of elements organized into weight matrices and bias vectors connecting successive layers of neurons. The Bayesian perspective treats these parameters as random variables with a prior distribution $p(\theta)$ rather than point estimates to be optimized. Given experience $\mathcal{D} = \{(s_i, a_i, y_i)\}_{i=1}^n$ where y_i represents target values (such as TD targets in temporal difference learning), we seek the posterior distribution:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})} \tag{56.24}$$

Here $p(\mathcal{D}|\theta)$ is the likelihood of observing the target values given parameters θ, $p(\theta)$ is the prior encoding our initial beliefs about reasonable parameter values, and $p(\mathcal{D})$ is the marginal likelihood that normalizes the distribution. For neural networks, the marginal likelihood $p(\mathcal{D}) = \int p(\mathcal{D}|\theta)p(\theta)d\theta$ is intractable. The integral requires summing over all possible parameter configurations, but neural networks produce complex nonlinear transformations through multiple layers that create multi-modal posterior geometries. Even simple networks exhibit weight symmetries—permuting hidden units produces identical functions—that create multiple posterior modes. These computational barriers motivate approximate inference methods.

56.3.2 Variational Inference for Bayesian Neural Networks

Variational inference for Bayesian Neural Networks (BNN) has been covered in detail in Chapter 12, including the evidence lower bound (ELBO), mean-field approximations, and the reparameterization trick for gradient computation [174]. In the context of reinforcement learning, variational inference is applied to Q-networks or policy networks by maintaining variational distributions $q_\phi(\theta)$ over network parameters θ [1397]. During training, the ELBO balances fitting observed temporal difference targets (the expected log-likelihood term) against staying close to the prior (the KL regularization term), resulting in posterior distributions that quantify epistemic uncertainty about value functions or policies. This uncertainty can then be leveraged for exploration through Thompson sampling (sampling one parameter configuration per episode) or upper confidence bounds (adding uncertainty bonuses to mean predictions), enabling principled exploration in high-dimensional state spaces where exact Bayesian inference would be intractable.

56.3.3 Variational Bayesian Deep Q-Networks

Variational inference integrates naturally with Deep Q-Networks (DQN), combining neural function approximation with principled uncertainty quantification for exploration. The algorithm alternates between collecting experience through interaction with the environment and updating the variational posterior over network parameters.

Temporal Difference Learning with Distributional Parameters

Standard DQN uses temporal difference learning to update Q-value estimates. The update rule computes TD targets:

$$y_t = r_t + \gamma \max_{a'} Q_{\theta_{\text{target}}}(s_{t+1}, a'), \tag{56.25}$$

where r_t is the observed reward, γ is the discount factor, and θ_{target} represents target network parameters that stabilize learning. The network parameters θ are then updated to minimize the squared TD error $\mathbb{E}[(Q_\theta(s_t, a_t) - y_t)^2]$. Variational Bayesian DQN maintains distributions over parameters instead of point estimates. The expected log-likelihood term in the ELBO becomes:

$$\mathbb{E}_{q_\phi(\theta)}\left[\sum_{(s,a,y)\in\mathcal{D}} \log p(y|s,a,\theta)\right] \tag{56.26}$$

Assuming Gaussian observation noise with variance σ_n^2, this is proportional to negative squared error:

$$-\frac{1}{2\sigma_n^2}\mathbb{E}_{\theta\sim q_\phi} \sum_{(s,a,y)\in\mathcal{D}} (Q_\theta(s,a) - y)^2 \tag{56.27}$$

The algorithm samples parameter configurations from q_ϕ, computes TD errors for each sample, and uses these errors to update both means and variances through the ELBO gradient.

Target Networks and Posterior Updates

DQN uses target networks to stabilize temporal difference learning. Similarly, Variational Bayesian DQN maintains target variational parameters ϕ_{target} that update slowly:

$$\phi_{\text{target}} \leftarrow \tau\phi + (1-\tau)\phi_{\text{target}}, \tag{56.28}$$

where $\tau \in (0, 1)$ is a small constant (typically 0.001) that controls update speed. TD targets are computed using samples from the target distribution $q_{\phi_{\text{target}}}(\theta)$ rather than the current distribution $q_\phi(\theta)$, reducing correlation between predictions and targets. Experience replay buffers store transitions (s_t, a_t, r_t, s_{t+1}) and sampling mini-batches from this buffer provides the dataset $\mathcal{D}$ for variational updates. The KL term $\text{KL}(q_\phi(\theta)\|p(\theta))$ acts as regularization, pulling the posterior toward the prior and preventing excessive variance growth. The uncertainty estimates from variational Bayesian DQN face systematic issues due to the bootstrapping nature of temporal difference learning. Because TD targets $y_t = r_t + \gamma \max_{a'} Q_{\theta_{\text{target}}}(s_{t+1}, a')$ are computed using the current Q-function estimate rather than unbiased Monte Carlo returns, the likelihood term in the ELBO becomes:

$$\mathbb{E}_{q_\phi(\theta)}\left[\sum_{(s,a,y)\in D} \log p(y|s,a,\theta)\right] \approx -\frac{1}{2\sigma_n^2}\mathbb{E}_{\theta\sim q_\phi} \sum_{(s,a)\in D} (Q_\theta(s,a) - \tilde{y})^2 \tag{56.29}$$

where $\tilde{y}$ itself depends on uncertain Q-value estimates. This recursive dependency means the posterior captures not only epistemic uncertainty about the true Q-function, but also propagates errors and biases from bootstrapped targets, leading to poorly calibrated uncertainty estimates that may be overconfident in some regions and under-confident in others. Additionally, the relative weighting between the expected log-likelihood term and the KL regularization term $\text{KL}(q_\phi(\theta)\|p(\theta))$ remains an open practical question. While the ELBO objective theoretically specifies equal weighting, practitioners often scale the KL term by $\beta \in (0, 1]$ or by $1/N$ where N is the dataset size, treating it as a form of weight decay.

Exploration through Posterior Sampling

Thompson sampling for exploration becomes straightforward with distributional parameters. At the start of each episode, sample one parameter configuration $\theta^* \sim q_\phi(\theta)$ and act greedily with respect to Q_{θ^*} throughout that episode:

$$a_t = \arg\max_a Q_{\theta^*}(s_t, a) \tag{56.30}$$

Different parameter samples produce different action preferences, and sampling from regions with high posterior variance naturally encourages exploration of uncertain regions. This exploration mechanism requires no explicit

bonuses or ϵ-greedy randomization; uncertainty guides exploration implicitly through the distribution over Q-functions. Alternatively, upper confidence bound exploration computes:

$$a_t = \arg\max_a \left[\mathbb{E}_{\theta \sim q_\phi}[Q_\theta(s_t, a)] + \beta \sqrt{\mathrm{Var}_{\theta \sim q_\phi}[Q_\theta(s_t, a)]} \right], \tag{56.31}$$

where β controls optimism about uncertain actions. Computing the mean and variance requires sampling multiple parameter configurations, but this overhead is typically small compared to environment interaction costs.

Bootstrap Ensembles and Diversity

Bootstrap ensembles train M networks $\{Q_{\theta_1}, \ldots, Q_{\theta_M}\}$ on different subsets of experience. Each network m optimizes:

$$\theta_m = \arg\max_\theta \sum_{(s,a,y) \in \mathcal{D}_m} \log p(y|s, a, \theta), \tag{56.32}$$

where $\mathcal{D}_m$ is a bootstrap sample of the experience replay buffer, created by sampling transitions with replacement. Different bootstrap samples lead to different trained networks, and the diversity of network predictions approximates posterior uncertainty. In RL, bootstrapped DQN maintains separate Q-networks and assigns each transition to a random subset of networks. Each network receives a different view of the data, leading to different learned Q-functions. For exploration, the algorithm samples one network at the beginning of each episode and acts greedily with respect to that network's Q-values throughout the episode. This Thompson sampling approximation balances exploration and exploitation: uncertain state-action pairs exhibit high variance across ensemble members, and sampling different networks naturally explores these uncertain regions. The ensemble variance provides an uncertainty measure:

$$\mathrm{Var}[Q(s, a)] = \frac{1}{M} \sum_{m=1}^{M} \left(Q_{\theta_m}(s, a) - \bar{Q}(s, a) \right)^2 \tag{56.33}$$

where $\bar{Q}(s, a) = \frac{1}{M} \sum_{m=1}^{M} Q_{\theta_m}(s, a)$ is the ensemble mean. This variance decreases as networks agree, indicating well-explored regions, and increases where networks disagree, indicating uncertainty that warrants exploration.

56.3.4 Bayesian Successor Features for Exploration

Successor Uncertainties [836] combine successor features with Bayesian linear regression to enable principled exploration under uncertainty. The key computational advantage is that instead of maintaining uncertainty over Q-values for all state-action pairs, the method tracks uncertainty only over a d-dimensional reward parameter vector w. When $d \ll |S \times A|$, this decomposition dramatically reduces computational burden while maintaining the ability to quantify uncertainty for exploration. The method performs Bayesian regression on the reward function, assuming a linear-Gaussian model:

$$r(s, a) = \phi(s, a)^\top w + \eta, \tag{56.34}$$

where $\phi(s, a) \in \mathbb{R}^d$ represents state-action features, $w \in \mathbb{R}^d$ is the reward parameter vector under which we place uncertainty, and $\eta \sim \mathcal{N}(0, \sigma^2)$ represents observation noise. Successor features capture the expected discounted occupancy of features when following policy π:

$$\psi^\pi(s, a) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t \phi(s_t, a_t) \mid s_0 = s, a_0 = a \right], \tag{56.35}$$

where $\gamma \in [0, 1)$ is the discount factor. These can be learned via temporal difference methods analogous to Q-learning, or computed exactly in tabular settings. The Bayesian update places a Gaussian prior $w \sim \mathcal{N}(\mu_0, \Sigma_0)$ over reward parameters and updates it through standard Bayesian linear regression as reward observations are collected, yielding posterior $p(w|\mathcal{D}) = \mathcal{N}(\mu_n, \Sigma_n)$ where the posterior covariance Σ_n decreases with more observations. Exploration proceeds via Thompson sampling: at each step, the agent samples $w^{(i)} \sim \mathcal{N}(\mu_n, \Sigma_n)$ and selects $a_t = \arg\max_a \psi^\pi(s_t, a)^\top w^{(i)}$. This naturally trades off exploitation and exploration, as uncertainty in Σ_n encourages visiting state-action pairs where the reward structure is poorly understood.

56.4 Bayesian RL Methods Comparison

Table 56.1 summarizes the main Bayesian RL method families discussed in this chapter. Exact tabular methods correspond to the posterior-sampling, Bayes-adaptive planning, and Bayesian exploration approaches introduced in the early sections, including PSRL, BAMCP, and BEB. Gaussian process methods refer to GP-SARSA, PILCO, and related model-based control approaches, while variational BNNs and neural ensembles summarize the scalable neural approximations discussed above. The Linear-Gaussian row captures methods such as Bayesian successor features, where uncertainty is maintained over a low-dimensional linear reward or value model. MCMC sampling is included as a reference point rather than as a method developed in detail here: it represents a sampling-based route to Bayesian posterior approximation, but its computational cost and mixing difficulties often make it less practical than variational or ensemble approximations in large-scale reinforcement learning. The table organizes methods by problem class, computational complexity, uncertainty quality, implementation prerequisities, and primary failure modes. Each method family is analyzed in terms of scaling properties, implementation constraints, and application domains.

Table 56.1: Bayesian RL Methods Comparison.

Method Family	Applicable Problem Class	Complexity and Scaling	Uncertainty Quality	Implement. Prerequisites	Primary Failure Modes	Example Applications
Exact Tabular	$\|\mathcal{S}\|\|\mathcal{A}\| < 10^5$ discrete state-action pairs	$O(\|\mathcal{S}\|\|\mathcal{A}\|)$ per episode	Optimal epistemic uncertainty if model class correct	Compatible priors, sufficient exploration	Model misspecification, curse of dimensionality	Multi-armed bandits, grid-world exploration, clinical trial design
Linear-Gaussian	$V(s) = \phi(s)^\top \theta$ with known features ϕ	$O(d^2)$ per update (rank-1)	Well-calibrated uncertainty, biased mean if nonlinear	Feature engineering, stationarity assumptions	Poor feature design, nonlinear value functions	Portfolio optimisation, Linear-Quadratic Regulator (LQR) control, linear bandits
Gaussian Process	Smooth functions, $n < 5000$ observations	$O(n^3)$ training cost	Well-calibrated epistemic uncertainty for smooth functions	Kernel selection, hyperparameter tuning	Poor kernel choice, scalability limits	Robot control, hyperparameter optimisation, continuous bandits
Neural Ensembles	Large-scale problems, no structural assumptions	M heads × base training cost	Approximates epistemic uncertainty, planning-dependent quality	Ensemble diversity techniques, computational infrastructure	Heads converge identically, overconfidence	Atari games, robotic exploration, large-state RL
Variational BNN	Large networks requiring uncertainty estimates	2× base training cost	Underestimates epistemic uncertainty (mean-field limitation)	Variational inference skills, optimisation debugging	Posterior collapse, poor calibration	Complex robotic control, neural dialogue systems
MCMC Sampling	Small-medium networks, complex posteriors	10-50× base cost	Asymptotically exact epistemic uncertainty if converged	Sampling expertise, convergence assessment	Slow convergence, poor mixing	Safety-critical control, Bayesian neural policies

56.5 Applications and case studies

Robotics. *Data-efficient control.* PILCO achieves cart-pole swing-up in ∼20 trials using GP dynamics with $M \approx 200$ inducing points; policy gradients propagate through predictive variances [397]. Actuator saturation can violate Gaussian moment matching and degrade rollouts [657]. Model-based alternatives plan under uncertainty via ensembles with MPC (PETS) [305] and GP-based probabilistic MPC with chance constraints [863]. These frameworks illustrate how Bayesian models can explicitly balance exploitation and active learning by planning trajectories that reduce uncertainty in regions critical for control performance.

Dialogue systems. *Belief-aware planning under partial observability.* Partially Observable Markov Decision Process (POMDP) managers maintain posteriors over goals and slots; Bayes-adaptive planning lowers failure rates relative to heuristic exploration [1332, 1423]. Monte Carlo planning with belief sampling scales to larger domains while retaining uncertainty-awareness [1515]. These approaches demonstrate how maintaining posterior beliefs over latent user intentions enables adaptive dialogue policies that query or confirm information when uncertainty is high.

Atari. *Exploration via approximate posteriors.* Bootstrapped DQN with randomised priors samples a head per episode to induce temporally extended exploration in sparse-reward tasks (e.g., *Montezuma's Revenge*) [1269, 1270]. Without priors, ensemble heads can collapse to similar Q-functions; excessive intrinsic rewards can reduce task returns [221]. The approach highlights how posterior sampling approximations in deep networks can provide

temporally coherent exploration signals in environments with sparse or delayed feedback.

Spatial navigation. *Structured inductive biases.* Ring-attractor modules maintain distributions over headings; embedding them in RL agents yields uncertainty-aware localisation and planning under partial observability [1230]. This line of work connects Bayesian reasoning with biologically inspired neural mechanisms, showing how structured representations can encode uncertainty in spatial variables to support robust navigation.

Safe control and constraint handling. *Safety during learning.* Bayesian model-based controllers plan over posterior dynamics with risk constraints. GP-based safe exploration uses confidence sets to restrict actions to high-probability safe regions [1644]. Surveyed mechanisms include Bayesian constraint models, risk-sensitive objectives, and shielded exploration [576]. Together, these methods formalize safety as a probabilistic constraint satisfaction problem, allowing exploration only when predicted violation probabilities remain below specified thresholds.

Recommenders and advertising (contextual bandits). *Personalisation with uncertainty estimates.* Posterior-sampling bandit methods adapt decisions using uncertainty over reward models; scalable deep approximations (variational BNNs, ensembles) perform well on large contextual problems [1397]. In this setting, uncertainty quantification drives exploration toward under-sampled users or contexts, improving long-term reward while controlling the cost of suboptimal recommendations.

56.6 Future directions

Two main open problems in scalable Bayesian RL are posterior approximation quality and safety guarantee provision. Current posterior approximations exhibit systematic biases that limit their effectiveness: variational methods underestimate uncertainty, leading to overconfident policies [543, 1274], while ensemble methods experience head collapse, where ensemble members converge to similar functions, and lack theoretical regret bounds [1270]. Structured variational families that explicitly capture temporal correlations across time steps [57] and ensemble architectures with diversity regularization to prevent collapse [310, 1739] represent approaches to address these limitations. The second problem concerns using uncertainty estimates to provide safety guarantees for real-world deployment: GP-based methods provide rigorous confidence bounds through posterior variance but require $O(n^3)$ computation that limits scalability [1644], while Deep Learning (DL) methods scale efficiently but produce poorly calibrated uncertainty estimates [1274]. Conformal prediction provides distribution-free safety bounds without distributional assumptions [1602], and Bayesian constraint modelling enables explicit incorporation of safety requirements [576], but integrating these techniques with deep RL at scale remains an active research direction.